

YTBLND

1.0

Generated by Doxygen 1.14.0

1 Namespace Index	1
1.1 Namespace List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	9
4.1 File List	9
5 Namespace Documentation	13
5.1 ModelJson Namespace Reference	13
5.1.1 Detailed Description	13
5.1.2 Function Documentation	13
5.1.2.1 escape()	13
5.1.2.2 quote()	14
5.1.2.3 toIso8601()	14
5.2 UIColors Namespace Reference	14
5.2.1 Detailed Description	15
5.2.2 Function Documentation	15
5.2.2.1 GetCurrentTheme()	15
5.2.2.2 SetTheme()	15
5.2.3 Variable Documentation	15
5.2.3.1 Accent	15
5.2.3.2 AccentHover	16
5.2.3.3 Background	16
5.2.3.4 Current	16
5.2.3.5 Danger	16
5.2.3.6 DarkMode	16
5.2.3.7 LightMode	17
5.2.3.8 NeonMode	17
5.2.3.9 Separator	17
5.2.3.10 Surface	17
5.2.3.11 SurfaceRaised	17
5.2.3.12 TextMuted	18
5.2.3.13 TextPrimary	18
5.2.3.14 TextSecondary	18
6 Class Documentation	19
6.1 ActiveBlendsPanel Class Reference	19
6.1.1 Detailed Description	21
6.1.2 Constructor & Destructor Documentation	21

6.1.2.1 ActiveBlendsPanel()	21
6.1.3 Member Function Documentation	21
6.1.3.1 OnLeaveBlend()	21
6.1.3.2 OnSelectBlend()	21
6.1.3.3 RebuildList()	21
6.1.3.4 Refresh()	21
6.1.4 Member Data Documentation	22
6.1.4.1 m_controller	22
6.1.4.2 m_emptyLabel	22
6.1.4.3 m_listInner	22
6.1.4.4 m_listScroll	22
6.1.4.5 m_listSizer	22
6.1.4.6 m_nav	22
6.2 ApplicationController Class Reference	23
6.2.1 Detailed Description	25
6.2.2 Constructor & Destructor Documentation	25
6.2.2.1 ApplicationController() [1/2]	25
6.2.2.2 ApplicationController() [2/2]	25
6.2.2.3 ~ApplicationController()	25
6.2.3 Member Function Documentation	25
6.2.3.1 clearProgressReporter()	25
6.2.3.2 enrichActiveBlendFeedIfMissingMetadata()	26
6.2.3.3 enrichIfMissingMetadata()	26
6.2.3.4 enrichMissingMetadataSubsetMultithreaded()	27
6.2.3.5 fetchUserBlends()	27
6.2.3.6 get_current_email()	27
6.2.3.7 get_current_user()	28
6.2.3.8 get_current_username()	28
6.2.3.9 getEventRouter()	28
6.2.3.10 handleCreateBlend()	28
6.2.3.11 handleDeleteAccount()	28
6.2.3.12 handleLeaveBlend()	29
6.2.3.13 handleLogin()	29
6.2.3.14 handleLogout()	29
6.2.3.15 handleOpenChat()	29
6.2.3.16 handlePlayVideo()	30
6.2.3.17 handleRefresh()	30
6.2.3.18 handleRegister()	30
6.2.3.19 handleSelectBlend()	30
6.2.3.20 handleSendMessage()	31
6.2.3.21 handleUploadData()	31
6.2.3.22 loadParticipantsWithWatchLater()	31

6.2.3.23 loadParticipantsWithWatchLaterParallel()	32
6.2.3.24 lookupUser()	32
6.2.3.25 parseBlendID()	32
6.2.3.26 parseChatHistory()	32
6.2.3.27 parseParticipantIDs()	33
6.2.3.28 parseUserFromAuthResponse()	33
6.2.3.29 parseWatchLaterVideos()	33
6.2.3.30 registerBlendOnServer()	34
6.2.3.31 registerEvents()	34
6.2.3.32 reportProgress()	34
6.2.3.33 setProgressReporter()	34
6.2.4 Member Data Documentation	35
6.2.4.1 appState	35
6.2.4.2 currentBlend	35
6.2.4.3 eventRouter	35
6.2.4.4 http	35
6.2.4.5 isConnected	35
6.2.4.6 pendingAccountCreated_	35
6.2.4.7 pendingRegistration_	35
6.2.4.8 progressReporter	36
6.2.4.9 server_url	36
6.3 AppState Class Reference	36
6.3.1 Detailed Description	37
6.3.2 Constructor & Destructor Documentation	38
6.3.2.1 AppState()	38
6.3.3 Member Function Documentation	38
6.3.3.1 addSessionUser()	38
6.3.3.2 clearSession()	38
6.3.3.3 clearSessionUsers()	38
6.3.3.4 createChatRoom()	38
6.3.3.5 getActiveBlend()	39
6.3.3.6 getActiveChatRoom()	39
6.3.3.7 getChatRoom()	39
6.3.3.8 getCurrentUser()	40
6.3.3.9 getInstance()	40
6.3.3.10 getSessionUsers()	40
6.3.3.11 getUsersWithoutData()	40
6.3.3.12 isGeneratingBlend()	40
6.3.3.13 setActiveBlend()	40
6.3.3.14 setCurrentUser()	41
6.3.3.15 setIsBlendGenerating()	41
6.3.3.16 setPendingBlendMessage()	41

6.3.3.17 setPendingUploadError()	41
6.3.3.18 setUsersWithoutData()	42
6.3.3.19 takePendingBlendMessage()	42
6.3.3.20 takePendingUploadError()	42
6.3.4 Member Data Documentation	43
6.3.4.1 activeBlend	43
6.3.4.2 chatRooms	43
6.3.4.3 currentUser	43
6.3.4.4 isBlendGenerating	43
6.3.4.5 pendingBlendMessage	43
6.3.4.6 pendingUploadError	43
6.3.4.7 sessionUsers	44
6.3.4.8 usersWithoutData	44
6.4 Blend Class Reference	44
6.4.1 Detailed Description	45
6.4.2 Constructor & Destructor Documentation	45
6.4.2.1 Blend()	45
6.4.3 Member Function Documentation	45
6.4.3.1 getAlgorithmUsed()	45
6.4.3.2 getBlendID()	45
6.4.3.3 getParticipants()	46
6.4.3.4 getTitle()	46
6.4.3.5 getVideo()	46
6.4.3.6 getVideoList()	46
6.4.3.7 setBlendID()	46
6.4.3.8 setVideoList()	47
6.4.3.9 size()	47
6.4.3.10 toString()	47
6.4.4 Member Data Documentation	47
6.4.4.1 algorithmUsed	47
6.4.4.2 blendID	48
6.4.4.3 participants	48
6.4.4.4 title	48
6.4.4.5 videoList	48
6.5 BlendChatPanel Class Reference	48
6.5.1 Detailed Description	50
6.5.2 Constructor & Destructor Documentation	50
6.5.2.1 BlendChatPanel()	50
6.5.3 Member Function Documentation	50
6.5.3.1 ConnectWebSocket()	50
6.5.3.2 DoSend()	50
6.5.3.3 OnInputEnter()	50

6.5.3.4 OnParticipantsToggle()	51
6.5.3.5 OnPollTimer()	51
6.5.3.6 OnSend()	51
6.5.3.7 Refresh()	51
6.5.3.8 Reload()	51
6.5.4 Member Data Documentation	51
6.5.4.1 m_controller	51
6.5.4.2 m_incoming	52
6.5.4.3 m_incomingMx	52
6.5.4.4 m_inputCtrl	52
6.5.4.5 m_inputPanel	52
6.5.4.6 m_msglInner	52
6.5.4.7 m_msgScroll	52
6.5.4.8 m_msgSizer	52
6.5.4.9 m_nav	52
6.5.4.10 m_noBlendLabel	53
6.5.4.11 m_participantsBtn	53
6.5.4.12 m_participantsDropdown	53
6.5.4.13 m_participantsSizer	53
6.5.4.14 m_pollTimer	53
6.5.4.15 m_ws	53
6.5.4.16 m_wsBlendID	53
6.6 BlendCreationPanel Class Reference	54
6.6.1 Detailed Description	55
6.6.2 Constructor & Destructor Documentation	55
6.6.2.1 BlendCreationPanel()	55
6.6.3 Member Function Documentation	55
6.6.3.1 OnAdd()	55
6.6.3.2 OnCreate()	56
6.6.3.3 OnRemoveUser()	56
6.6.3.4 RebuildUserList()	56
6.6.3.5 Refresh()	56
6.6.3.6 Reload()	56
6.6.3.7 UpdateCountLabel()	56
6.6.4 Member Data Documentation	56
6.6.4.1 m_addedUsers	56
6.6.4.2 m_blendNameCtrl	57
6.6.4.3 m_controller	57
6.6.4.4 m_countLabel	57
6.6.4.5 m_createBtn	57
6.6.4.6 m_emptyLabel	57
6.6.4.7 m_nav	57

6.6.4.8 m_userListInner	57
6.6.4.9 m_userListScroll	57
6.6.4.10 m_userListSizer	58
6.7 BlendFeedPanel Class Reference	58
6.7.1 Detailed Description	60
6.7.2 Constructor & Destructor Documentation	60
6.7.2.1 BlendFeedPanel()	60
6.7.3 Member Function Documentation	60
6.7.3.1 LoadFromBlend()	60
6.7.3.2 NextPage()	60
6.7.3.3 Refresh()	60
6.7.4 Member Data Documentation	61
6.7.4.1 kPageSize	61
6.7.4.2 m_cards	61
6.7.4.3 m_controller	61
6.7.4.4 m_offset	61
6.8 AppController::BlendSummary Struct Reference	61
6.8.1 Detailed Description	62
6.8.2 Member Data Documentation	62
6.8.2.1 blendID	62
6.8.2.2 title	62
6.9 BtnConfig Struct Reference	62
6.9.1 Detailed Description	62
6.9.2 Member Data Documentation	63
6.9.2.1 maxHeight	63
6.9.2.2 maxWidth	63
6.9.2.3 minHeight	63
6.9.2.4 minWidth	63
6.10 Channel Class Reference	63
6.10.1 Detailed Description	64
6.10.2 Constructor & Destructor Documentation	64
6.10.2.1 Channel()	64
6.10.3 Member Function Documentation	64
6.10.3.1 getCategories()	64
6.10.3.2 getChannelID()	65
6.10.3.3 getDisplayName()	65
6.10.3.4 toString()	65
6.10.4 Member Data Documentation	65
6.10.4.1 categories	65
6.10.4.2 channelID	65
6.10.4.3 displayName	65
6.11 ChatRoom Class Reference	66

6.11.1 Detailed Description	66
6.11.2 Constructor & Destructor Documentation	66
6.11.2.1 ChatRoom()	66
6.11.3 Member Function Documentation	67
6.11.3.1 addMessage() [1/2]	67
6.11.3.2 addMessage() [2/2]	67
6.11.3.3 clearMessages()	67
6.11.3.4 getBlendID()	67
6.11.3.5 getMessages()	68
6.11.3.6 getParticipantIDs()	68
6.11.3.7 isParticipant()	68
6.11.3.8 toString()	68
6.11.4 Member Data Documentation	69
6.11.4.1 blendID	69
6.11.4.2 messages	69
6.11.4.3 participantIDs	69
6.12 ChatWebSocket Class Reference	69
6.12.1 Detailed Description	70
6.12.2 Constructor & Destructor Documentation	70
6.12.2.1 ChatWebSocket()	70
6.12.2.2 ~ChatWebSocket()	70
6.12.3 Member Function Documentation	71
6.12.3.1 build_fullUrl()	71
6.12.3.2 connect()	71
6.12.3.3 content_formatter()	71
6.12.3.4 get_fullUrl()	71
6.12.3.5 send_message()	71
6.12.3.6 set_blendID()	72
6.12.3.7 set_userID()	72
6.12.3.8 setOnMessage()	72
6.12.4 Member Data Documentation	73
6.12.4.1 blendID	73
6.12.4.2 fullUrl	73
6.12.4.3 isStarted	73
6.12.4.4 m_onMessage	73
6.12.4.5 url	73
6.12.4.6 userID	73
6.12.4.7 ws	73
6.13 ConfirmationDialog Class Reference	74
6.13.1 Detailed Description	74
6.13.2 Constructor & Destructor Documentation	75
6.13.2.1 ConfirmationDialog()	75

6.13.3 Member Function Documentation	75
6.13.3.1 Confirm()	75
6.14 CsvParser Class Reference	75
6.14.1 Detailed Description	76
6.14.2 Constructor & Destructor Documentation	76
6.14.2.1 CsvParser() [1/2]	76
6.14.2.2 CsvParser() [2/2]	76
6.14.2.3 ~CsvParser()	77
6.14.3 Member Function Documentation	77
6.14.3.1 getParsed()	77
6.14.3.2 parse()	77
6.14.3.3 setData()	77
6.14.4 Member Data Documentation	78
6.14.4.1 parse_id	78
6.14.4.2 raw_data	78
6.15 CsvSource Class Reference	78
6.15.1 Detailed Description	79
6.15.2 Constructor & Destructor Documentation	79
6.15.2.1 CsvSource()	79
6.15.3 Member Function Documentation	80
6.15.3.1 getSourceId()	80
6.15.3.2 read()	80
6.15.3.3 setSource()	80
6.15.4 Member Data Documentation	80
6.15.4.1 source_id	80
6.15.4.2 src	81
6.16 DataExtractor Class Reference	81
6.16.1 Detailed Description	81
6.16.2 Constructor & Destructor Documentation	81
6.16.2.1 DataExtractor()	81
6.16.2.2 ~DataExtractor()	82
6.16.3 Member Function Documentation	82
6.16.3.1 extract()	82
6.16.3.2 setParser()	82
6.16.3.3 setSource()	83
6.16.4 Member Data Documentation	83
6.16.4.1 par_format	83
6.16.4.2 src_format	83
6.17 DataInstructionsPanel Class Reference	84
6.17.1 Detailed Description	85
6.17.2 Constructor & Destructor Documentation	85
6.17.2.1 DataInstructionsPanel()	85

6.17.3 Member Function Documentation	85
6.17.3.1 OnBrowse()	85
6.17.4 Member Data Documentation	85
6.17.4.1 m_controller	85
6.17.4.2 m_nav	86
6.18 DataManager Class Reference	86
6.18.1 Detailed Description	86
6.18.2 Member Function Documentation	87
6.18.2.1 createUser()	87
6.18.2.2 findBlendForUser()	87
6.18.2.3 findUserByID()	87
6.18.2.4 loadBlendParticipants()	87
6.18.2.5 loadWatchLater()	88
6.18.2.6 saveBlend()	88
6.18.2.7 saveWatchLater()	88
6.18.2.8 validatePassword()	89
6.19 DataSaver Class Reference	89
6.19.1 Detailed Description	89
6.19.2 Constructor & Destructor Documentation	89
6.19.2.1 DataSaver()	89
6.19.3 Member Function Documentation	90
6.19.3.1 save()	90
6.19.3.2 setFileBin()	90
6.19.3.3 setFileFormatter()	90
6.19.4 Member Data Documentation	91
6.19.4.1 bins	91
6.19.4.2 formatters	91
6.20 EventRouter Class Reference	91
6.20.1 Detailed Description	91
6.20.2 Constructor & Destructor Documentation	92
6.20.2.1 EventRouter()	92
6.20.3 Member Function Documentation	92
6.20.3.1 deregisterAll()	92
6.20.3.2 dispatch()	92
6.20.3.3 registerListener()	92
6.20.4 Member Data Documentation	93
6.20.4.1 listeners	93
6.21 File_ID Class Reference	93
6.21.1 Detailed Description	93
6.21.2 Member Data Documentation	93
6.21.2.1 CSV	93
6.21.2.2 HTML	93

6.21.2.3 JSON	94
6.22 FileBin Class Reference	94
6.22.1 Detailed Description	94
6.22.2 Constructor & Destructor Documentation	94
6.22.2.1 ~FileBin()	94
6.22.3 Member Function Documentation	94
6.22.3.1 setFileName()	94
6.22.3.2 setOutputFile()	95
6.22.3.3 setRawData()	95
6.22.3.4 write()	95
6.23 FileFormatter Class Reference	95
6.23.1 Detailed Description	96
6.23.2 Constructor & Destructor Documentation	96
6.23.2.1 ~FileFormatter()	96
6.23.3 Member Function Documentation	96
6.23.3.1 format()	96
6.23.3.2 setData()	96
6.24 FileSource Class Reference	97
6.24.1 Detailed Description	97
6.24.2 Member Function Documentation	97
6.24.2.1 getSourceId()	97
6.24.2.2 read()	98
6.24.2.3 setSource()	98
6.25 Friend Class Reference	98
6.25.1 Detailed Description	99
6.25.2 Constructor & Destructor Documentation	99
6.25.2.1 Friend()	99
6.25.3 Member Function Documentation	99
6.25.3.1 getDisplayName()	99
6.25.3.2 getEmail()	100
6.25.3.3 getUserID()	100
6.25.3.4 setDisplayName()	100
6.25.3.5 setEmail()	100
6.25.3.6 toString()	101
6.25.4 Member Data Documentation	101
6.25.4.1 displayName	101
6.25.4.2 email	101
6.25.4.3 userID	101
6.26 HtmlParser Class Reference	102
6.26.1 Detailed Description	103
6.26.2 Constructor & Destructor Documentation	103
6.26.2.1 HtmlParser() [1/2]	103

6.26.2.2 HtmlParser() [2/2]	103
6.26.3 Member Function Documentation	103
6.26.3.1 extract_links()	103
6.26.3.2 get_text()	103
6.26.3.3 getParsed()	104
6.26.3.4 parse()	104
6.26.3.5 setData()	104
6.26.4 Member Data Documentation	104
6.26.4.1 parse_id	104
6.26.4.2 raw_data	105
6.27 HtmlSource Class Reference	105
6.27.1 Detailed Description	106
6.27.2 Constructor & Destructor Documentation	106
6.27.2.1 HtmlSource()	106
6.27.3 Member Function Documentation	106
6.27.3.1 getSourceId()	106
6.27.3.2 read()	106
6.27.3.3 setSource()	107
6.27.4 Member Data Documentation	107
6.27.4.1 source_id	107
6.27.4.2 src_path	107
6.28 HttpClient Class Reference	107
6.28.1 Detailed Description	108
6.28.2 Constructor & Destructor Documentation	108
6.28.2.1 HttpClient()	108
6.28.3 Member Function Documentation	109
6.28.3.1 build_auth_user_lookup_endpoint()	109
6.28.3.2 build_blend_participant_endpoint()	109
6.28.3.3 build_chat_history_endpoint()	109
6.28.3.4 build_chatroom_detail_endpoint()	110
6.28.3.5 build_delete_user_endpoint()	110
6.28.3.6 build_latest_blend_endpoint()	110
6.28.3.7 build_leave_blend_endpoint()	111
6.28.3.8 build_user_blends_endpoint()	111
6.28.3.9 build_watch_later_endpoint()	111
6.28.3.10 del()	112
6.28.3.11 get()	112
6.28.3.12 getLastStatusCode()	113
6.28.3.13 isRequestSuccessful()	113
6.28.3.14 post()	113
6.28.3.15 request()	114
6.28.3.16 wasLastRequestSuccessful()	114

6.28.3.17 write_callback()	115
6.28.4 Member Data Documentation	115
6.28.4.1 ALT_SUCC	115
6.28.4.2 AUTH_USER_PREFIX	115
6.28.4.3 baseUrl	115
6.28.4.4 BLEND	116
6.28.4.5 D	116
6.28.4.6 DELETE_USER_PREFIX	116
6.28.4.7 DUPLICATE	116
6.28.4.8 G	116
6.28.4.9 hdr	116
6.28.4.10 INV_USR_LOG_ERR	116
6.28.4.11 lastStatusCode	116
6.28.4.12 LOGIN	117
6.28.4.13 MAL_REQ_ERR	117
6.28.4.14 MISS_RM_CNTXT	117
6.28.4.15 P	117
6.28.4.16 REG_USER	117
6.28.4.17 SERVER_ERR	117
6.28.4.18 SUCC	117
6.28.4.19 UNAUTH_RES_REQ_ERR	118
6.29 IBlendAlgorithm Class Reference	118
6.29.1 Detailed Description	118
6.29.2 Member Function Documentation	118
6.29.2.1 generateBlend()	118
6.30 ImageBackgroundPanel Class Reference	119
6.30.1 Detailed Description	120
6.30.2 Constructor & Destructor Documentation	120
6.30.2.1 ImageBackgroundPanel()	120
6.30.3 Member Function Documentation	120
6.30.3.1 OnPaint()	120
6.30.4 Member Data Documentation	120
6.30.4.1 m_imageStore	120
6.30.4.2 m_key	121
6.31 IRefreshablePanel Class Reference	121
6.31.1 Detailed Description	121
6.31.2 Member Function Documentation	122
6.31.2.1 Refresh()	122
6.32 LoginPanel Class Reference	122
6.32.1 Detailed Description	124
6.32.2 Constructor & Destructor Documentation	124
6.32.2.1 LoginPanel()	124

6.32.3 Member Function Documentation	124
6.32.3.1 BuildRegisterForm()	124
6.32.3.2 BuildSignInForm()	124
6.32.3.3 OnPaint()	125
6.32.3.4 OnRegister()	125
6.32.3.5 OnSignIn()	125
6.32.3.6 ProceedAfterLogin()	125
6.32.3.7 Refresh()	125
6.32.3.8 Reset()	125
6.32.3.9 ShowTab()	125
6.32.4 Member Data Documentation	126
6.32.4.1 m_bgImage	126
6.32.4.2 m_controller	126
6.32.4.3 m_formBook	126
6.32.4.4 m_nav	126
6.32.4.5 m_regConfirm	126
6.32.4.6 m_regEmail	126
6.32.4.7 m_regError	126
6.32.4.8 m_registerTab	127
6.32.4.9 m_regPassword	127
6.32.4.10 m_regUsername	127
6.32.4.11 m_siError	127
6.32.4.12 m_signinTab	127
6.32.4.13 m_siPassword	127
6.32.4.14 m_siUsername	127
6.33 MainFrame Class Reference	128
6.33.1 Detailed Description	129
6.33.2 Member Enumeration Documentation	129
6.33.2.1 ImageKey	129
6.33.3 Constructor & Destructor Documentation	130
6.33.3.1 MainFrame()	130
6.33.4 Member Function Documentation	130
6.33.4.1 BuildHomePage()	130
6.33.4.2 LoadImage()	130
6.33.4.3 NavigateTo()	130
6.33.4.4 RecolorAll()	131
6.33.4.5 RecolorWidget()	131
6.33.4.6 TriggerFeedRefresh()	131
6.33.5 Member Data Documentation	131
6.33.5.1 activeBlendsPanel	131
6.33.5.2 book	131
6.33.5.3 chatPanel	132

6.33.5.4 controller	132
6.33.5.5 creationPanel	132
6.33.5.6 dataInstrPanel	132
6.33.5.7 feedPanel	132
6.33.5.8 images	132
6.33.5.9 loginPanel	132
6.33.5.10 refreshBtn	132
6.33.5.11 refreshInProgress	133
6.33.5.12 settingsPanel	133
6.33.5.13 userPanel	133
6.34 Message Struct Reference	133
6.34.1 Detailed Description	134
6.34.2 Constructor & Destructor Documentation	134
6.34.2.1 Message()	134
6.34.3 Member Function Documentation	134
6.34.3.1 toString()	134
6.34.4 Member Data Documentation	134
6.34.4.1 text	134
6.34.4.2 timestamp	135
6.34.4.3 userID	135
6.35 OperationProgressDialog Class Reference	135
6.35.1 Detailed Description	136
6.35.2 Constructor & Destructor Documentation	136
6.35.2.1 OperationProgressDialog()	136
6.35.2.2 ~OperationProgressDialog()	137
6.35.3 Member Function Documentation	137
6.35.3.1 ShowModal()	137
6.35.3.2 UpdateProgress()	137
6.35.3.3 UpdateProgressFill()	137
6.35.4 Member Data Documentation	137
6.35.4.1 currentProgress	137
6.35.4.2 messageLabel	138
6.35.4.3 percentLabel	138
6.35.4.4 progressFill	138
6.35.4.5 progressTrack	138
6.36 Palette Struct Reference	138
6.36.1 Detailed Description	139
6.36.2 Member Data Documentation	139
6.36.2.1 Accent	139
6.36.2.2 AccentHover	139
6.36.2.3 Background	139
6.36.2.4 Danger	139

6.36.2.5 Separator	140
6.36.2.6 Surface	140
6.36.2.7 SurfaceRaised	140
6.36.2.8 TextMuted	140
6.36.2.9 TextPrimary	140
6.36.2.10 TextSecondary	140
6.37 Parser Class Reference	141
6.37.1 Detailed Description	141
6.37.2 Member Function Documentation	141
6.37.2.1 getParsed()	141
6.37.2.2 parse()	142
6.37.2.3 setData()	142
6.38 RandomBlendAlgorithm Class Reference	142
6.38.1 Detailed Description	143
6.38.2 Constructor & Destructor Documentation	143
6.38.2.1 RandomBlendAlgorithm()	143
6.38.3 Member Function Documentation	143
6.38.3.1 generateBlend()	143
6.38.4 Member Data Documentation	144
6.38.4.1 videosPerUser	144
6.39 SettingsPanel Class Reference	144
6.39.1 Detailed Description	146
6.39.2 Constructor & Destructor Documentation	146
6.39.2.1 SettingsPanel()	146
6.39.3 Member Function Documentation	146
6.39.3.1 OnLogout()	146
6.39.3.2 OnShow()	146
6.39.3.3 OnThemeChanged()	146
6.39.3.4 RefreshUserInfo()	146
6.39.4 Member Data Documentation	147
6.39.4.1 m_controller	147
6.39.4.2 m_nav	147
6.39.4.3 m_themeChoice	147
6.40 SqliteDataManager Class Reference	147
6.40.1 Detailed Description	148
6.40.2 Constructor & Destructor Documentation	148
6.40.2.1 SqliteDataManager()	148
6.40.2.2 ~SqliteDataManager()	149
6.40.3 Member Function Documentation	149
6.40.3.1 createUser()	149
6.40.3.2 findBlendForUser()	149
6.40.3.3 findUserById()	149

6.40.3.4	initSchema()	150
6.40.3.5	loadBlendParticipants()	150
6.40.3.6	loadWatchLater()	151
6.40.3.7	saveBlend()	151
6.40.3.8	saveWatchLater()	152
6.40.3.9	validatePassword()	152
6.40.4	Member Data Documentation	152
6.40.4.1	db	152
6.41	ThumbnailEvent Class Reference	153
6.41.1	Detailed Description	154
6.41.2	Constructor & Destructor Documentation	154
6.41.2.1	ThumbnailEvent()	154
6.41.3	Member Function Documentation	154
6.41.3.1	Clone()	154
6.41.4	Member Data Documentation	154
6.41.4.1	generation	154
6.41.4.2	image	154
6.41.4.3	success	155
6.42	TopBar Class Reference	155
6.42.1	Detailed Description	156
6.42.2	Constructor & Destructor Documentation	156
6.42.2.1	TopBar()	156
6.42.3	Member Function Documentation	156
6.42.3.1	OnBack()	156
6.42.4	Member Data Documentation	157
6.42.4.1	m_backDest	157
6.42.4.2	m_nav	157
6.43	User Class Reference	157
6.43.1	Detailed Description	158
6.43.2	Constructor & Destructor Documentation	158
6.43.2.1	User()	158
6.43.3	Member Function Documentation	159
6.43.3.1	addFriend()	159
6.43.3.2	getEmail()	159
6.43.3.3	getFriends()	159
6.43.3.4	getPassword()	160
6.43.3.5	getUserID()	160
6.43.3.6	getUsername()	160
6.43.3.7	getYouTubeData() [1/2]	160
6.43.3.8	getYouTubeData() [2/2]	160
6.43.3.9	removeFriend()	160
6.43.3.10	setEmail()	161

6.43.3.11 setPassword()	161
6.43.3.12 setUsername()	161
6.43.3.13 setYouTubeData()	161
6.43.3.14 toString()	162
6.43.4 Member Data Documentation	162
6.43.4.1 email	162
6.43.4.2 friends	162
6.43.4.3 password	162
6.43.4.4 userID	162
6.43.4.5 username	163
6.43.4.6 youTubeData	163
6.44 UserPanel Class Reference	163
6.44.1 Detailed Description	165
6.44.2 Constructor & Destructor Documentation	165
6.44.2.1 UserPanel()	165
6.44.3 Member Function Documentation	165
6.44.3.1 OnConfirmDelete()	165
6.44.3.2 OnDeleteRequest()	165
6.44.3.3 OnLogout()	166
6.44.3.4 OnReuploadCSV()	166
6.44.3.5 OnShow()	166
6.44.3.6 Refresh()	166
6.44.3.7 RefreshUserInfo()	166
6.44.3.8 SetDeleteReauthVisible()	167
6.44.4 Member Data Documentation	167
6.44.4.1 m_confirmDeleteBtn	167
6.44.4.2 m_controller	167
6.44.4.3 m_deleteErrorLabel	167
6.44.4.4 m_deletePasswordField	167
6.44.4.5 m_emailLabel	167
6.44.4.6 m_nav	167
6.44.4.7 m_usernameLabel	168
6.45 Video Class Reference	168
6.45.1 Detailed Description	168
6.45.2 Constructor & Destructor Documentation	169
6.45.2.1 Video()	169
6.45.3 Member Function Documentation	169
6.45.3.1 getChannelID()	169
6.45.3.2 getChannelLogoURL()	169
6.45.3.3 getChannelName()	170
6.45.3.4 getDuration()	170
6.45.3.5 getTags()	170

6.45.3.6	getThumbnailURL()	170
6.45.3.7	getTitle()	170
6.45.3.8	getVideoID()	171
6.45.3.9	toString()	171
6.45.4	Member Data Documentation	171
6.45.4.1	channelID	171
6.45.4.2	channelLogoURL	171
6.45.4.3	channelName	171
6.45.4.4	duration	171
6.45.4.5	tags	172
6.45.4.6	thumbnailURL	172
6.45.4.7	title	172
6.45.4.8	videoID	172
6.46	VideoCard Class Reference	172
6.46.1	Detailed Description	174
6.46.2	Constructor & Destructor Documentation	174
6.46.2.1	VideoCard()	174
6.46.3	Member Function Documentation	175
6.46.3.1	Clear()	175
6.46.3.2	DoGetBestSize()	175
6.46.3.3	DrawWrappedText()	175
6.46.3.4	FetchChannelLogoAsync()	175
6.46.3.5	FetchThumbnailAsync()	176
6.46.3.6	OnChannelLogoLoaded()	176
6.46.3.7	OnClick()	176
6.46.3.8	OnMouseEnter()	176
6.46.3.9	OnMouseLeave()	176
6.46.3.10	OnPaint()	176
6.46.3.11	OnThumbnailLoaded()	176
6.46.3.12	SetVideo()	176
6.46.4	Member Data Documentation	177
6.46.4.1	m_channelID	177
6.46.4.2	m_channelLogo	177
6.46.4.3	m_channelLogoURL	177
6.46.4.4	m_channelName	177
6.46.4.5	m_controller	177
6.46.4.6	m_hovered	177
6.46.4.7	m_logoFailed	178
6.46.4.8	m_logoGen	178
6.46.4.9	m_logoLoading	178
6.46.4.10	m_thumbFailed	178
6.46.4.11	m_thumbGen	178

6.46.4.12 m_thumbLoading	178
6.46.4.13 m_thumbnail	178
6.46.4.14 m_thumbnailURL	179
6.46.4.15 m_title	179
6.46.4.16 m_videoID	179
6.47 VideoEntry Class Reference	179
6.47.1 Detailed Description	180
6.47.2 Constructor & Destructor Documentation	180
6.47.2.1 VideoEntry()	180
6.47.3 Member Function Documentation	180
6.47.3.1 getLastWatched()	180
6.47.3.2 getVideo()	181
6.47.3.3 getWatchCount()	181
6.47.3.4 setLastWatched()	181
6.47.3.5 setWatchCount()	181
6.47.3.6 toString()	182
6.47.4 Member Data Documentation	182
6.47.4.1 lastWatched	182
6.47.4.2 video	182
6.47.4.3 watchCount	182
6.48 YouTubeMetadataFetcher::VideoMeta Struct Reference	182
6.48.1 Detailed Description	183
6.48.2 Member Data Documentation	183
6.48.2.1 channelId	183
6.48.2.2 channelName	183
6.48.2.3 thumbnailUrl	183
6.48.2.4 title	183
6.49 WatchHistoryParser Class Reference	183
6.49.1 Detailed Description	184
6.49.2 Constructor & Destructor Documentation	184
6.49.2.1 WatchHistoryParser()	184
6.49.3 Member Function Documentation	184
6.49.3.1 extractVideoIdFromHref()	184
6.49.3.2 getParserId()	185
6.49.3.3 parse()	185
6.49.3.4 parseMultiThreaded()	185
6.49.3.5 parseRange()	185
6.49.4 Member Data Documentation	186
6.49.4.1 filePath	186
6.49.4.2 parser_id	186
6.50 WatchLaterParser Class Reference	186
6.50.1 Detailed Description	186

6.50.2 Constructor & Destructor Documentation	186
6.50.2.1 WatchLaterParser()	186
6.50.3 Member Function Documentation	187
6.50.3.1 getParserId()	187
6.50.3.2 parse()	187
6.50.3.3 parseMultiThreaded()	187
6.50.4 Member Data Documentation	188
6.50.4.1 filePath	188
6.50.4.2 parser_id	188
6.51 WeightedBlendAlgorithm Class Reference	188
6.51.1 Detailed Description	189
6.51.2 Constructor & Destructor Documentation	189
6.51.2.1 WeightedBlendAlgorithm() [1/2]	189
6.51.2.2 WeightedBlendAlgorithm() [2/2]	189
6.51.3 Member Function Documentation	190
6.51.3.1 generateBlend()	190
6.51.3.2 normalizeWeights()	190
6.51.3.3 selectWeightedVideos()	190
6.51.4 Member Data Documentation	191
6.51.4.1 participantWeights	191
6.51.4.2 videosPerUser	191
6.52 YouTubeData Class Reference	191
6.52.1 Detailed Description	192
6.52.2 Constructor & Destructor Documentation	192
6.52.2.1 YouTubeData()	192
6.52.3 Member Function Documentation	192
6.52.3.1 getLikedVideos()	192
6.52.3.2 getMostWatchedVideos()	192
6.52.3.3 getSubscribedChannels()	193
6.52.3.4 getTopChannels()	193
6.52.3.5 getWatchHistory()	193
6.52.3.6 getWatchLaterVideos()	193
6.52.3.7 setLikedVideos()	193
6.52.3.8 setSubscribedChannels()	194
6.52.3.9 setWatchHistory()	194
6.52.3.10 setWatchLaterVideos()	194
6.52.3.11 toString()	194
6.52.4 Member Data Documentation	195
6.52.4.1 likedVideos	195
6.52.4.2 subscribedChannels	195
6.52.4.3 watchHistory	195
6.52.4.4 watchLaterVideos	195

6.53 YouTubeDataImporter Class Reference	195
6.53.1 Detailed Description	196
6.53.2 Member Function Documentation	196
6.53.2.1 import()	196
6.53.2.2 importMultiThreaded()	196
6.53.2.3 lowerExtension()	196
6.54 YouTubeMetadataFetcher Class Reference	197
6.54.1 Detailed Description	198
6.54.2 Constructor & Destructor Documentation	198
6.54.2.1 YouTubeMetadataFetcher()	198
6.54.3 Member Function Documentation	198
6.54.3.1 enrich()	198
6.54.3.2 fetchChannelLogos()	198
6.54.3.3 fetchVideoBatch()	198
6.54.3.4 httpGet()	199
6.54.3.5 writeCallback()	199
6.54.4 Member Data Documentation	199
6.54.4.1 apiKey	199
6.54.4.2 kBatchSize	199
6.55 YTLNDApp Class Reference	200
6.55.1 Detailed Description	201
6.55.2 Member Function Documentation	201
6.55.2.1 OnExit()	201
6.55.2.2 OnInit()	201
6.55.3 Member Data Documentation	201
6.55.3.1 controller	201
7 File Documentation	203
7.1 IBlendAlgorithm.hpp File Reference	203
7.1.1 Detailed Description	204
7.2 IBlendAlgorithm.hpp	204
7.3 RandomBlendAlgorithm.cpp File Reference	205
7.3.1 Detailed Description	205
7.4 RandomBlendAlgorithm.cpp	206
7.5 RandomBlendAlgorithm.hpp File Reference	206
7.5.1 Detailed Description	207
7.6 RandomBlendAlgorithm.hpp	208
7.7 WeightedBlendAlgorithm.hpp File Reference	208
7.7.1 Detailed Description	209
7.8 WeightedBlendAlgorithm.hpp	209
7.9 CsvParser.cpp File Reference	209
7.9.1 Detailed Description	210

7.10 CsvParser.cpp	210
7.11 CsvParser.hpp File Reference	211
7.11.1 Detailed Description	212
7.12 CsvParser.hpp	212
7.13 CsvSource.cpp File Reference	213
7.13.1 Detailed Description	213
7.14 CsvSource.cpp	213
7.15 CsvSource.hpp File Reference	214
7.15.1 Detailed Description	215
7.16 CsvSource.hpp	215
7.17 DataExtractor.cpp File Reference	215
7.17.1 Detailed Description	216
7.18 DataExtractor.cpp	216
7.19 DataExtractor.hpp File Reference	217
7.19.1 Detailed Description	217
7.20 DataExtractor.hpp	218
7.21 DataSaver.hpp File Reference	218
7.21.1 Detailed Description	219
7.22 DataSaver.hpp	219
7.23 File_ID.hpp File Reference	220
7.23.1 Detailed Description	220
7.24 File_ID.hpp	220
7.25 FileBin.hpp File Reference	221
7.25.1 Detailed Description	221
7.26 FileBin.hpp	222
7.27 FileFormatter.hpp File Reference	222
7.27.1 Detailed Description	223
7.28 FileFormatter.hpp	223
7.29 FileSource.hpp File Reference	224
7.29.1 Detailed Description	225
7.30 FileSource.hpp	225
7.31 HtmlParser.cpp File Reference	225
7.31.1 Detailed Description	226
7.32 HtmlParser.cpp	226
7.33 HtmlSource.cpp File Reference	227
7.33.1 Detailed Description	228
7.34 HtmlSource.cpp	228
7.35 Parser.hpp File Reference	228
7.35.1 Detailed Description	229
7.36 Parser.hpp	230
7.37 WatchHistoryParser.cpp File Reference	230
7.37.1 Detailed Description	230

7.38 WatchHistoryParser.cpp	231
7.39 WatchHistoryParser.hpp File Reference	233
7.39.1 Detailed Description	234
7.40 WatchHistoryParser.hpp	235
7.41 WatchLaterParser.cpp File Reference	235
7.41.1 Detailed Description	235
7.42 WatchLaterParser.cpp	236
7.43 WatchLaterParser.hpp File Reference	238
7.43.1 Detailed Description	239
7.44 WatchLaterParser.hpp	239
7.45 YouTubeDataImporter.cpp File Reference	239
7.45.1 Detailed Description	240
7.46 YouTubeDataImporter.cpp	240
7.47 YouTubeDataImporter.hpp File Reference	241
7.47.1 Detailed Description	242
7.48 YouTubeDataImporter.hpp	242
7.49 AppController.cpp File Reference	243
7.49.1 Detailed Description	243
7.49.2 Typedef Documentation	244
7.49.2.1 Json	244
7.49.3 Variable Documentation	244
7.49.3.1 kYouTubeAPIKey	244
7.50 AppController.cpp	244
7.51 AppController.hpp File Reference	263
7.51.1 Detailed Description	264
7.52 AppController.hpp	264
7.53 AppState.cpp File Reference	266
7.53.1 Detailed Description	266
7.54 AppState.cpp	267
7.55 AppState.hpp File Reference	268
7.55.1 Detailed Description	269
7.56 AppState.hpp	270
7.57 EventRouter.cpp File Reference	270
7.57.1 Detailed Description	271
7.58 EventRouter.cpp	271
7.59 EventRouter.hpp File Reference	272
7.59.1 Detailed Description	272
7.59.2 Typedef Documentation	273
7.59.2.1 EventHandler	273
7.59.2.2 EventPayload	273
7.60 EventRouter.hpp	274
7.61 main.cpp File Reference	274

7.61.1 Detailed Description	275
7.62 main.cpp	275
7.63 Blend.cpp File Reference	276
7.63.1 Detailed Description	276
7.64 Blend.cpp	276
7.65 Blend.hpp File Reference	278
7.65.1 Detailed Description	278
7.66 Blend.hpp	279
7.67 Channel.cpp File Reference	279
7.67.1 Detailed Description	280
7.68 Channel.cpp	280
7.69 Channel.hpp File Reference	281
7.69.1 Detailed Description	282
7.70 Channel.hpp	282
7.71 ChatRoom.cpp File Reference	282
7.71.1 Detailed Description	283
7.72 ChatRoom.cpp	283
7.73 ChatRoom.hpp File Reference	284
7.73.1 Detailed Description	285
7.74 ChatRoom.hpp	285
7.75 Friend.cpp File Reference	286
7.75.1 Detailed Description	286
7.76 Friend.cpp	286
7.77 Friend.hpp File Reference	287
7.77.1 Detailed Description	288
7.78 Friend.hpp	288
7.79 JsonUtils.hpp File Reference	288
7.79.1 Detailed Description	289
7.80 JsonUtils.hpp	290
7.81 Message.cpp File Reference	290
7.81.1 Detailed Description	291
7.82 Message.cpp	291
7.83 Message.hpp File Reference	291
7.83.1 Detailed Description	292
7.84 Message.hpp	293
7.85 User.cpp File Reference	293
7.85.1 Detailed Description	293
7.86 User.cpp	294
7.87 User.hpp File Reference	294
7.87.1 Detailed Description	295
7.88 User.hpp	296
7.89 Video.cpp File Reference	296

7.89.1 Detailed Description	297
7.90 Video.cpp	297
7.91 Video.hpp File Reference	298
7.91.1 Detailed Description	299
7.92 Video.hpp	299
7.93 VideoEntry.cpp File Reference	300
7.93.1 Detailed Description	300
7.94 VideoEntry.cpp	301
7.95 VideoEntry.hpp File Reference	301
7.95.1 Detailed Description	302
7.96 VideoEntry.hpp	302
7.97 YouTubeData.cpp File Reference	303
7.97.1 Detailed Description	303
7.98 YouTubeData.cpp	303
7.99 YouTubeData.hpp File Reference	305
7.99.1 Detailed Description	306
7.100 YouTubeData.hpp	307
7.101 ServerConfig.hpp File Reference	307
7.101.1 Detailed Description	308
7.101.2 Variable Documentation	308
7.101.2.1 kAppBackendBaseUrl	308
7.101.2.2 kAppBackendWsPrefix	308
7.101.2.3 kTestBackendBaseUrl	309
7.101.2.4 kTestBackendWsPrefix	309
7.102 ServerConfig.hpp	309
7.103 ChatWebSocket.cpp File Reference	309
7.103.1 Detailed Description	310
7.103.2 Function Documentation	310
7.103.2.1 urlencode()	310
7.104 ChatWebSocket.cpp	310
7.105 ChatWebSocket.hpp File Reference	311
7.105.1 Detailed Description	312
7.106 ChatWebSocket.hpp	313
7.107 DataManager.hpp File Reference	313
7.107.1 Detailed Description	314
7.108 DataManager.hpp	315
7.109 HttpClient.cpp File Reference	315
7.109.1 Detailed Description	316
7.110 HttpClient.cpp	316
7.111 HttpClient.hpp File Reference	318
7.111.1 Detailed Description	319
7.112 HttpClient.hpp	319

7.113 RequestJsonBuilder.cpp File Reference	320
7.113.1 Detailed Description	321
7.113.2 Function Documentation	321
7.113.2.1 buildBlendJson()	321
7.113.2.2 buildDeleteAccountJson()	322
7.113.2.3 buildLoginJson()	322
7.113.2.4 buildRegisterJson()	323
7.113.2.5 buildWatchLaterJson()	323
7.114 RequestJsonBuilder.cpp	324
7.115 RequestJsonBuilder.hpp File Reference	325
7.115.1 Detailed Description	326
7.115.2 Function Documentation	326
7.115.2.1 buildBlendJson()	326
7.115.2.2 buildDeleteAccountJson()	327
7.115.2.3 buildLoginJson()	327
7.115.2.4 buildRegisterJson()	328
7.115.2.5 buildWatchLaterJson()	328
7.116 RequestJsonBuilder.hpp	329
7.117 SqliteDataManager.cpp File Reference	329
7.117.1 Detailed Description	330
7.118 SqliteDataManager.cpp	330
7.119 SqliteDataManager.hpp File Reference	334
7.119.1 Detailed Description	335
7.120 SqliteDataManager.hpp	336
7.121 YouTubeMetadataFetcher.cpp File Reference	336
7.121.1 Detailed Description	337
7.121.2 Typedef Documentation	337
7.121.2.1 json	337
7.122 YouTubeMetadataFetcher.cpp	337
7.123 YouTubeMetadataFetcher.hpp File Reference	340
7.123.1 Detailed Description	341
7.124 YouTubeMetadataFetcher.hpp	341
7.125 ActiveBlendsPanel.cpp File Reference	342
7.125.1 Detailed Description	343
7.126 ActiveBlendsPanel.cpp	343
7.127 ActiveBlendsPanel.hpp File Reference	345
7.127.1 Detailed Description	346
7.128 ActiveBlendsPanel.hpp	346
7.129 BlendChatPanel.cpp File Reference	347
7.129.1 Detailed Description	347
7.129.2 Function Documentation	347
7.129.2.1 FormatTimestamp()	347

7.130 BlendChatPanel.cpp	348
7.131 BlendChatPanel.hpp File Reference	352
7.131.1 Detailed Description	353
7.131.1.1 LAYOUT	354
7.131.1.2 MESSAGE LIST	354
7.131.1.3 SENDING	354
7.132 BlendChatPanel.hpp	354
7.133 BlendCreationPanel.cpp File Reference	355
7.133.1 Detailed Description	356
7.134 BlendCreationPanel.cpp	356
7.135 BlendCreationPanel.hpp File Reference	361
7.135.1 Detailed Description	361
7.135.1.1 LAYOUT	362
7.135.1.2 USER-ADD FLOW	362
7.135.1.3 BLEND CREATION	362
7.135.1.4 RELOAD	362
7.136 BlendCreationPanel.hpp	362
7.137 BlendFeedPanel.cpp File Reference	363
7.137.1 Detailed Description	364
7.138 BlendFeedPanel.cpp	364
7.139 BlendFeedPanel.hpp File Reference	365
7.139.1 Detailed Description	366
7.139.1.1 PAGING	366
7.139.1.2 LOADING	367
7.140 BlendFeedPanel.hpp	367
7.141 ButtonsConfig.cpp File Reference	367
7.141.1 Detailed Description	368
7.141.2 Function Documentation	368
7.141.2.1 ApplySizeBounds()	368
7.141.2.2 GetConfig()	368
7.141.2.3 GetMaxSize()	369
7.141.2.4 GetMinSize()	369
7.141.2.5 GetSize()	369
7.142 ButtonsConfig.cpp	370
7.143 ButtonsConfig.hpp File Reference	371
7.143.1 Detailed Description	372
7.143.2 Enumeration Type Documentation	372
7.143.2.1 ButtonType	372
7.143.3 Function Documentation	373
7.143.3.1 ApplySizeBounds()	373
7.143.3.2 GetConfig()	373
7.143.3.3 GetMaxSize()	373

7.143.3.4 GetMinSize()	373
7.143.3.5 GetSize()	374
7.144 ButtonsConfig.hpp	374
7.145 ConfirmationDialog.cpp File Reference	375
7.145.1 Detailed Description	375
7.146 ConfirmationDialog.cpp	375
7.147 ConfirmationDialog.hpp File Reference	377
7.147.1 Detailed Description	377
7.148 ConfirmationDialog.hpp	378
7.149 DataInstructionsPanel.cpp File Reference	378
7.149.1 Detailed Description	379
7.150 DataInstructionsPanel.cpp	379
7.151 DataInstructionsPanel.hpp File Reference	381
7.151.1 Detailed Description	382
7.152 DataInstructionsPanel.hpp	382
7.153 IRefreshablePanel.hpp File Reference	383
7.153.1 Detailed Description	383
7.154 IRefreshablePanel.hpp	384
7.155 LoginPanel.cpp File Reference	384
7.155.1 Detailed Description	384
7.155.2 Function Documentation	385
7.155.2.1 MakeError()	385
7.155.2.2 MakeField()	385
7.156 LoginPanel.cpp	385
7.157 LoginPanel.hpp File Reference	389
7.157.1 Detailed Description	390
7.157.1.1 SIGN IN	391
7.157.1.2 REGISTER	391
7.157.1.3 POST-LOGIN ROUTING	391
7.157.1.4 RESET	391
7.158 LoginPanel.hpp	391
7.159 MainFrame.cpp File Reference	392
7.159.1 Detailed Description	393
7.160 MainFrame.cpp	393
7.161 MainFrame.hpp File Reference	398
7.161.1 Detailed Description	399
7.162 MainFrame.hpp	399
7.163 OperationProgressDialog.cpp File Reference	400
7.163.1 Detailed Description	400
7.164 OperationProgressDialog.cpp	401
7.165 OperationProgressDialog.hpp File Reference	402
7.165.1 Detailed Description	403

7.166 OperationProgressDialog.hpp	404
7.167 SettingsPanel.cpp File Reference	404
7.167.1 Detailed Description	405
7.168 SettingsPanel.cpp	405
7.169 SettingsPanel.hpp File Reference	406
7.169.1 Detailed Description	407
7.170 SettingsPanel.hpp	408
7.171 TopBar.cpp File Reference	408
7.171.1 Detailed Description	408
7.172 TopBar.cpp	409
7.173 TopBar.hpp File Reference	409
7.173.1 Detailed Description	410
7.174 TopBar.hpp	411
7.175 UIColors.cpp File Reference	411
7.175.1 Detailed Description	411
7.175.2 Variable Documentation	412
7.175.2.1 sCurrentTheme	412
7.176 UIColors.cpp	412
7.177 UIColors.hpp File Reference	413
7.177.1 Detailed Description	414
7.177.2 Enumeration Type Documentation	415
7.177.2.1 ThemeType	415
7.178 UIColors.hpp	415
7.179 UIPages.hpp File Reference	416
7.179.1 Detailed Description	417
7.179.2 Typedef Documentation	417
7.179.2.1 NavigateFn	417
7.179.3 Enumeration Type Documentation	417
7.179.3.1 Page	417
7.180 UIPages.hpp	418
7.181 UserPanel.cpp File Reference	418
7.181.1 Detailed Description	418
7.182 UserPanel.cpp	419
7.183 UserPanel.hpp File Reference	422
7.183.1 Detailed Description	423
7.183.1.1 LAYOUT	423
7.183.1.2 REFRESH ON SHOW	423
7.183.1.3 LOGOUT	423
7.183.1.4 FUTURE FEATURES	423
7.184 UserPanel.hpp	424
7.185 VideoCard.cpp File Reference	424
7.185.1 Detailed Description	425

7.185.2 Function Documentation	425
7.185.2.1 LineHeight()	425
7.186 VideoCard.cpp	425
7.187 VideoCard.hpp File Reference	430
7.187.1 Detailed Description	431
7.188 VideoCard.hpp	432
Index	433

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

ModelJson	JSON formatting utilities for model serialization	13
UIColors	Shared dark-theme colour constants for the YTBLND UI	14

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AppController	23
AppState	36
Blend	44
AppController::BlendSummary	61
BtnConfig	62
Channel	63
ChatRoom	66
ChatWebSocket	69
DataExtractor	81
DataManager	86
SqliteDataManager	147
DataSaver	89
EventRouter	91
File_ID	93
FileBin	94
FileFormatter	95
FileSource	97
CsvSource	78
HtmlSource	105
Friend	98
HttpClient	107
IBlendAlgorithm	118
RandomBlendAlgorithm	142
WeightedBlendAlgorithm	188
IRefreshablePanel	121
ActiveBlendsPanel	19
BlendChatPanel	48
BlendCreationPanel	54
BlendFeedPanel	58
LoginPanel	122
UserPanel	163
Message	133
Palette	138
Parser	141

CsvParser	75
HtmlParser	102
User	157
Video	168
VideoEntry	179
YouTubeMetadataFetcher::VideoMeta	182
WatchHistoryParser	183
WatchLaterParser	186
wxApp	
YTBLNDApp	200
wxDialog	
ConfirmationDialog	74
OperationProgressDialog	135
wxEvent	
ThumbnailEvent	153
wxFrame	
MainFrame	128
wxPanel	
ActiveBlendsPanel	19
BlendChatPanel	48
BlendCreationPanel	54
BlendFeedPanel	58
DataInstructionsPanel	84
ImageBackgroundPanel	119
LoginPanel	122
SettingsPanel	144
TopBar	155
UserPanel	163
VideoCard	172
YouTubeData	191
YouTubeDataImporter	195
YouTubeMetadataFetcher	197

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ActiveBlendsPanel	
ActiveBlendsPanel class declaration	19
AppController	
Stub for future use	23
AppState	
AppState class declaration	36
Blend	
Blend class declaration	44
BlendChatPanel	
BlendChatPanel class declaration	48
BlendCreationPanel	
BlendCreationPanel class declaration	54
BlendFeedPanel	
Panel that renders paged blend videos in a fixed 3x2 card layout	58
AppController::BlendSummary	
BlendSummary class declaration	61
BtnConfig	
Width/height dimensions for a button preset	62
Channel	
Channel class declaration	63
ChatRoom	
ChatRoom class declaration	66
ChatWebSocket	
ChatWebSocket class declaration	69
ConfirmationDialog	
ConfirmationDialog class declaration	74
CsvParser	
CsvParser class declaration	75
CsvSource	
CsvSource class declaration	78
DataExtractor	
DataExtractor class declaration	81
DataInstructionsPanel	
DataInstructionsPanel class declaration	84
DataManager	
DataManager class declaration	86

DataSaver	
DataSaver class declaration	89
EventRouter	
EventRouter class declaration	91
File_ID	
File_ID class declaration	93
FileBin	
FileBin class declaration	94
FileFormatter	
FileFormatter class declaration	95
FileSource	
FileSource class declaration	97
Friend	
Friend class declaration	98
HtmlParser	
HTML file parser implementation using Gumbo	102
HtmlSource	
HTML file source/reader implementation	105
HttpClient	
HttpClient class declaration	107
IBlendAlgorithm	
IBlendAlgorithm class declaration	118
ImageBackgroundPanel	119
IRefreshablePanel	
Interface for panels that support refresh operations	121
LoginPanel	
LoginPanel class declaration	122
MainFrame	
Root application window containing the full-window page switcher	128
Message	
A single chat message sent within a ChatRoom	133
OperationProgressDialog	
Non-modal progress dialog showing upload/processing progress	135
Palette	
Palette class declaration	138
Parser	
Parser class declaration	141
RandomBlendAlgorithm	
RandomBlendAlgorithm class declaration	142
SettingsPanel	
SettingsPanel class declaration	144
SqliteDataManager	
SqliteDataManager type declaration	147
ThumbnailEvent	
Event fired on the VideoCard when an async image fetch completes	153
TopBar	
Reusable horizontal top bar for interior pages	155
User	
User class declaration	157
UserPanel	
Panel for user account display and account actions	163
Video	
Video class declaration	168
VideoCard	
Custom wxPanel widget that displays a single YouTube video preview	172
VideoEntry	
VideoEntry class declaration	179

YouTubeMetadataFetcher::VideoMeta	
VideoMeta class declaration	182
WatchHistoryParser	
WatchHistoryParser class declaration	183
WatchLaterParser	
WatchLaterParser class declaration	186
WeightedBlendAlgorithm	
WeightedBlendAlgorithm class declaration	188
YouTubeData	
YouTubeData class declaration	191
YouTubeDataImporter	
YouTubeDataImporter class declaration	195
YouTubeMetadataFetcher	
YouTubeMetadataFetcher class declaration	197
YTBLNDApp	200

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

IBlendAlgorithm.hpp	Abstract interface for blend-generation algorithms	203
RandomBlendAlgorithm.cpp	Implementation for RandomBlendAlgorithm	205
RandomBlendAlgorithm.hpp	Random Blend implementation of IBlendAlgorithm	206
WeightedBlendAlgorithm.hpp	Weighted Blend implementation of IBlendAlgorithm	208
CsvParser.cpp	Implementation for CsvParser	209
CsvParser.hpp	CSV file parser implementing the Parser interface	211
CsvSource.cpp	Implementation for CsvSource	213
CsvSource.hpp	CSV file reader implementing the FileSource interface	214
DataExtractor.cpp	DataExtractor class implementation	215
DataExtractor.hpp	Extracts raw file data and parses it into structured format	217
DataSaver.hpp	Coordinates FileFormatter and FileBin to persist structured data to files	218
File_ID.hpp	File format ID constants	220
FileBin.hpp	Abstract base class for writing data to files	221
FileFormatter.hpp	Abstract base class for formatting structured data to file format	222
FileSource.hpp	Abstract base class for reading file source data	224
HtmlParser.cpp	HTML file parser implementation using Gumbo	225
HtmlSource.cpp	HTML file source/reader implementation	227
Parser.hpp	Abstract base class for parsing file data into structured format	228

WatchHistoryParser.cpp	Implementation for WatchHistoryParser	230
WatchHistoryParser.hpp	Specialized parser for YouTube watch-history HTML exports	233
WatchLaterParser.cpp	Implementation for WatchLaterParser	235
WatchLaterParser.hpp	Specialized parser for YouTube Takeout Watch Later CSV files	238
YouTubeDataImporter.cpp	Implementation for YouTubeDataImporter	239
YouTubeDataImporter.hpp	Entry-point importer that routes YouTube exports to the right parser	241
AppController.cpp	Implementation for AppController	243
AppController.hpp	Central application coordinator	263
AppState.cpp	Implementation for AppState	266
AppState.hpp	Session-scoped singleton holding all runtime application state	268
EventRouter.cpp	Implementation for EventRouter	270
EventRouter.hpp	Lightweight event-dispatch system connecting the UI layer to AppController	272
main.cpp	Implementation for main	274
Blend.cpp	Implementation for Blend	276
Blend.hpp	Model representing a generated video feed shared by a group of users	278
Channel.cpp	Implementation for Channel	279
Channel.hpp	Model representing a YouTube channel	281
ChatRoom.cpp	Implementation for ChatRoom	282
ChatRoom.hpp	In-memory chat session tied to a specific Blend	284
Friend.cpp	Implementation for Friend	286
Friend.hpp	Model representing a friend/contact entry on a User 's list	287
JsonUtils.hpp	Utility functions for JSON string formatting and serialization	288
Message.cpp	Implementation for Message	290
Message.hpp	A single chat message sent within a ChatRoom	291
User.cpp	Implementation for User	293
User.hpp	Model representing a registered application user	294
Video.cpp	Implementation for Video	296
Video.hpp	Immutable data model for a single YouTube video	298
VideoEntry.cpp	Implementation for VideoEntry	300

VideoEntry.hpp	
A Video decorated with watch-history metadata	301
YouTubeData.cpp	
Implementation for YouTubeData	303
YouTubeData.hpp	
Container for all YouTube data associated with a User	305
ServerConfig.hpp	
Centralized backend server addresses	307
ChatWebSocket.cpp	
Implementation for ChatWebSocket	309
ChatWebSocket.hpp	
Simple WebSocket wrapper	311
DataManager.hpp	
Interface for all user-persistence operations	313
HttpClient.cpp	
Implementation for HttpClient	315
HttpClient.hpp	
Simple HTTP client wrapper	318
RequestJsonBuilder.cpp	
Implementation for RequestJsonBuilder	320
RequestJsonBuilder.hpp	
JSON request body builders used by ServiceLayer HTTP calls	325
SqliteDataManager.cpp	
Implementation for SqliteDataManager	329
SqliteDataManager.hpp	
SQLite-backed implementation of DataManager	334
YouTubeMetadataFetcher.cpp	
Implementation for YouTubeMetadataFetcher	336
YouTubeMetadataFetcher.hpp	
Enriches a list of Video objects (video-ID only) with full snippet metadata from the YouTube Data API v3	340
ActiveBlendsPanel.cpp	
Implementation for ActiveBlendsPanel	342
ActiveBlendsPanel.hpp	
Panel for viewing and managing active blends	345
BlendChatPanel.cpp	
Implementation for BlendChatPanel	347
BlendChatPanel.hpp	
Blend chat screen	352
BlendCreationPanel.cpp	
Implementation for BlendCreationPanel	355
BlendCreationPanel.hpp	
New-blend setup screen	361
BlendFeedPanel.cpp	
Implementation for BlendFeedPanel	363
BlendFeedPanel.hpp	
Home-page video feed panel that displays paged blend results	365
ButtonsConfig.cpp	
Implements shared button sizing presets used across UILayer	367
ButtonsConfig.hpp	
Centralized button size presets for UI layer widgets	371
ConfirmationDialog.cpp	
Implementation for ConfirmationDialog	375
ConfirmationDialog.hpp	
Dark-themed modal confirmation dialog	377
DataInstructionsPanel.cpp	
Implementation for DataInstructionsPanel	378

DataInstructionsPanel.hpp	
Setup instructions panel for importing Google Takeout CSV data	381
IRefreshablePanel.hpp	
Interface for panels that support refresh operations	383
LoginPanel.cpp	
Implementation for LoginPanel	384
LoginPanel.hpp	
Sign In / Register screen	389
MainFrame.cpp	
Implementation for MainFrame	392
MainFrame.hpp	
Root application window containing the full-window page switcher	398
OperationProgressDialog.cpp	
Implementation of non-modal progress dialog	400
OperationProgressDialog.hpp	
Dark-themed non-modal progress dialog for file upload/processing	402
SettingsPanel.cpp	
Implementation for SettingsPanel	404
SettingsPanel.hpp	
Theme selection and logout panel	406
TopBar.cpp	
Implementation for TopBar	408
TopBar.hpp	
Reusable interior-page header bar	409
UIColors.cpp	
Implementation for UIColors	411
UIColors.hpp	
Shared dark-theme colour palette used throughout the UI	413
UIPages.hpp	
Page enum and NavigateFn callback type for the MainFrame page switcher	416
UserPanel.cpp	
Implementation for UserPanel	418
UserPanel.hpp	
Account info and logout panel	422
VideoCard.cpp	
Implementation for VideoCard	424
VideoCard.hpp	
Custom wxPanel widget that displays a single YouTube video preview	430

Chapter 5

Namespace Documentation

5.1 ModelJson Namespace Reference

JSON formatting utilities for model serialization.

Functions

- `std::string escape` (`const std::string &input`)
- `std::string quote` (`const std::string &input`)
- `std::string toIso8601` (`std::time_t timestamp`)

5.1.1 Detailed Description

JSON formatting utilities for model serialization.

5.1.2 Function Documentation

5.1.2.1 escape()

```
std::string ModelJson::escape (  
    const std::string & input) [inline]
```

Escapes special characters in a string for use in JSON values.

Converts escape sequences such as quotes, backslashes, newlines, tabs, and control characters into their JSON-safe representations. Characters below 0x20 are converted to Unicode escape sequences.

Parameters

<i>input</i>	The input string to escape.
--------------	-----------------------------

Returns

A new string with all special characters properly escaped for JSON.

Definition at line 35 of file [JsonUtils.hpp](#).

5.1.2.2 quote()

```
std::string ModelJson::quote (
    const std::string & input) [inline]
```

Wraps a string in double quotes after escaping special characters.

Combines [escape\(\)](#) and quoting to produce a JSON-safe string value. The returned string includes the surrounding double quotes.

Parameters

<i>input</i>	The input string to quote and escape.
--------------	---------------------------------------

Returns

A JSON-safe quoted string (e.g., "hello world").

Definition at line 69 of file [JsonUtils.hpp](#).

5.1.2.3 toIso8601()

```
std::string ModelJson::toIso8601 (
    std::time_t timestamp) [inline]
```

Converts a UNIX timestamp to an ISO 8601 formatted string.

Formats the given timestamp (seconds since epoch) as an ISO 8601 string in UTC timezone. The format is: YYYY-MM-DDTHH:MM:SSZ

Parameters

<i>timestamp</i>	UNIX timestamp (seconds since epoch).
------------------	---------------------------------------

Returns

ISO 8601
formatted timestamp string in UTC (e.g., "2026-04-03T12:30:45Z").

Definition at line 82 of file [JsonUtils.hpp](#).

5.2 UIColors Namespace Reference

Shared dark-theme colour constants for the YTBLND UI.

Functions

- void [SetTheme](#) (ThemeType theme)
- ThemeType [GetCurrentTheme](#) ()

Variables

- [Palette DarkMode](#)
- [Palette LightMode](#)
- [Palette NeonMode](#)
- [Palette * Current](#) = &UIColors::DarkMode
- wxColour [Background](#) = UIColors::DarkMode.Background
- wxColour [Surface](#) = UIColors::DarkMode.Surface
- wxColour [SurfaceRaised](#) = UIColors::DarkMode.SurfaceRaised
- wxColour [Accent](#) = UIColors::DarkMode.Accent
- wxColour [AccentHover](#) = UIColors::DarkMode.AccentHover
- wxColour [TextPrimary](#) = UIColors::DarkMode.TextPrimary
- wxColour [TextSecondary](#) = UIColors::DarkMode.TextSecondary
- wxColour [TextMuted](#) = UIColors::DarkMode.TextMuted
- wxColour [Separator](#) = UIColors::DarkMode.Separator
- wxColour [Danger](#) = UIColors::DarkMode.Danger

5.2.1 Detailed Description

Shared dark-theme colour constants for the YTBLND UI.

5.2.2 Function Documentation

5.2.2.1 GetCurrentTheme()

```
ThemeType UIColors::GetCurrentTheme ()
```

Returns the current ThemeType index.

Definition at line 91 of file [UIColors.cpp](#).

5.2.2.2 SetTheme()

```
void UIColors::SetTheme (  
    ThemeType theme)
```

Definition at line 95 of file [UIColors.cpp](#).

5.2.3 Variable Documentation

5.2.3.1 Accent

```
wxColour UIColors::Accent = UIColors::DarkMode.Accent [extern]
```

Definition at line 81 of file [UIColors.cpp](#).

5.2.3.2 AccentHover

```
wxColour UIColors::AccentHover = UIColors::DarkMode.AccentHover [extern]
```

Definition at line 82 of file [UIColors.cpp](#).

5.2.3.3 Background

```
wxColour UIColors::Background = UIColors::DarkMode.Background [extern]
```

Definition at line 78 of file [UIColors.cpp](#).

5.2.3.4 Current

```
Palette * UIColors::Current = &UIColors::DarkMode [extern]
```

Definition at line 26 of file [UIColors.cpp](#).

5.2.3.5 Danger

```
wxColour UIColors::Danger = UIColors::DarkMode.Danger [extern]
```

Definition at line 87 of file [UIColors.cpp](#).

5.2.3.6 DarkMode

```
Palette UIColors::DarkMode [extern]
```

Initial value:

```
= {  
    wxColour(13,13,13),  
    wxColour(25,25,25),  
    wxColour(38,38,38),  
  
    wxColour(138,0,255),  
    wxColour(160,40,255),  
  
    wxColour(255,255,255),  
    wxColour(160,160,160),  
    wxColour(100,100,100),  
  
    wxColour(50,50,50),  
    wxColour(211, 47, 47)  
}
```

Definition at line 28 of file [UIColors.cpp](#).

5.2.3.7 LightMode

`Palette` `UIColors::LightMode` [extern]

Initial value:

```
= {  
    wxColour(245, 245, 245),  
    wxColour(255, 255, 255),  
    wxColour(230, 230, 230),  
  
    wxColour(0, 102, 204),  
    wxColour(0, 120, 215),  
  
    wxColour(33, 33, 33),  
    wxColour(117, 117, 117),  
    wxColour(189, 189, 189),  
  
    wxColour(224, 224, 224),  
    wxColour(211, 47, 47)  
}
```

Definition at line 45 of file [UIColors.cpp](#).

5.2.3.8 NeonMode

`Palette` `UIColors::NeonMode` [extern]

Initial value:

```
= {  
    wxColour(0, 0, 0),  
    wxColour(15, 15, 15),  
    wxColour(30, 30, 30),  
  
    wxColour(57, 255, 20),  
    wxColour(102, 255, 0),  
  
    wxColour(255, 255, 255),  
    wxColour(0, 255, 255),  
    wxColour(0, 138, 138),  
  
    wxColour(57, 255, 20),  
    wxColour(255, 0, 153)  
}
```

Definition at line 62 of file [UIColors.cpp](#).

5.2.3.9 Separator

`wxColour` `UIColors::Separator` = `UIColors::DarkMode.Separator` [extern]

Definition at line 86 of file [UIColors.cpp](#).

5.2.3.10 Surface

`wxColour` `UIColors::Surface` = `UIColors::DarkMode.Surface` [extern]

Definition at line 79 of file [UIColors.cpp](#).

5.2.3.11 SurfaceRaised

`wxColour` `UIColors::SurfaceRaised` = `UIColors::DarkMode.SurfaceRaised` [extern]

Definition at line 80 of file [UIColors.cpp](#).

5.2.3.12 TextMuted

```
wxColour UIColors::TextMuted = UIColors::DarkMode.TextMuted [extern]
```

Definition at line 85 of file [UIColors.cpp](#).

5.2.3.13 TextPrimary

```
wxColour UIColors::TextPrimary = UIColors::DarkMode.TextPrimary [extern]
```

Definition at line 83 of file [UIColors.cpp](#).

5.2.3.14 TextSecondary

```
wxColour UIColors::TextSecondary = UIColors::DarkMode.TextSecondary [extern]
```

Definition at line 84 of file [UIColors.cpp](#).

Chapter 6

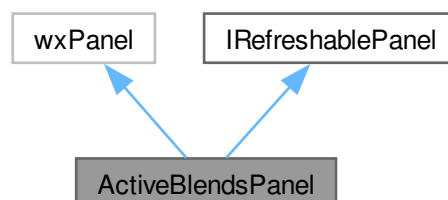
Class Documentation

6.1 ActiveBlendsPanel Class Reference

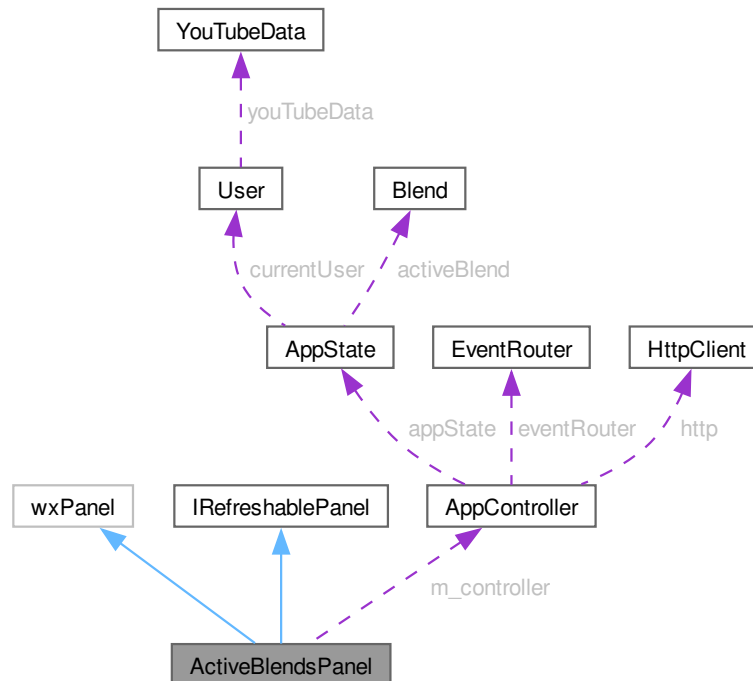
[ActiveBlendsPanel](#) class declaration.

```
#include <ActiveBlendsPanel.hpp>
```

Inheritance diagram for ActiveBlendsPanel:



Collaboration diagram for ActiveBlendsPanel:



Public Member Functions

- [ActiveBlendsPanel](#) (`wxWindow *parent`, [AppController](#) &controller, [NavigateFn](#) nav)
- void [Refresh](#) () override

Private Member Functions

- void [RebuildList](#) ()
- void [OnSelectBlend](#) (const std::string &blendID, const std::string &title)
- void [OnLeaveBlend](#) (const std::string &blendID, const std::string &title)

Private Attributes

- [AppController](#) & `m_controller`
- [NavigateFn](#) `m_nav`
- `wxScrolledWindow *` `m_listScroll`
- `wxPanel *` `m_listInner`
- `wxBoxSizer *` `m_listSizer`
- `wxStaticText *` `m_emptyLabel`

6.1.1 Detailed Description

[ActiveBlendsPanel](#) class declaration.

Definition at line 21 of file [ActiveBlendsPanel.hpp](#).

6.1.2 Constructor & Destructor Documentation

6.1.2.1 ActiveBlendsPanel()

```
ActiveBlendsPanel::ActiveBlendsPanel (  
    wxWindow * parent,  
    AppController & controller,  
    NavigateFn nav)
```

Definition at line 18 of file [ActiveBlendsPanel.cpp](#).

6.1.3 Member Function Documentation

6.1.3.1 OnLeaveBlend()

```
void ActiveBlendsPanel::OnLeaveBlend (  
    const std::string & blendID,  
    const std::string & title) [private]
```

Definition at line 159 of file [ActiveBlendsPanel.cpp](#).

6.1.3.2 OnSelectBlend()

```
void ActiveBlendsPanel::OnSelectBlend (  
    const std::string & blendID,  
    const std::string & title) [private]
```

Definition at line 138 of file [ActiveBlendsPanel.cpp](#).

6.1.3.3 RebuildList()

```
void ActiveBlendsPanel::RebuildList () [private]
```

Definition at line 66 of file [ActiveBlendsPanel.cpp](#).

6.1.3.4 Refresh()

```
void ActiveBlendsPanel::Refresh () [override], [virtual]
```

Refreshes the panel's display to reflect current application state. Called by [MainFrame](#) whenever the panel becomes active.

Implements [IRefreshablePanel](#).

Definition at line 61 of file [ActiveBlendsPanel.cpp](#).

6.1.4 Member Data Documentation

6.1.4.1 m_controller

[AppController](#)& ActiveBlendsPanel::m_controller [private]

Definition at line 27 of file [ActiveBlendsPanel.hpp](#).

6.1.4.2 m_emptyLabel

wxStaticText* ActiveBlendsPanel::m_emptyLabel [private]

Definition at line 33 of file [ActiveBlendsPanel.hpp](#).

6.1.4.3 m_listInner

wxPanel* ActiveBlendsPanel::m_listInner [private]

Definition at line 31 of file [ActiveBlendsPanel.hpp](#).

6.1.4.4 m_listScroll

wxScrolledWindow* ActiveBlendsPanel::m_listScroll [private]

Definition at line 30 of file [ActiveBlendsPanel.hpp](#).

6.1.4.5 m_listSizer

wxBoxSizer* ActiveBlendsPanel::m_listSizer [private]

Definition at line 32 of file [ActiveBlendsPanel.hpp](#).

6.1.4.6 m_nav

[NavigateFn](#) ActiveBlendsPanel::m_nav [private]

Definition at line 28 of file [ActiveBlendsPanel.hpp](#).

The documentation for this class was generated from the following files:

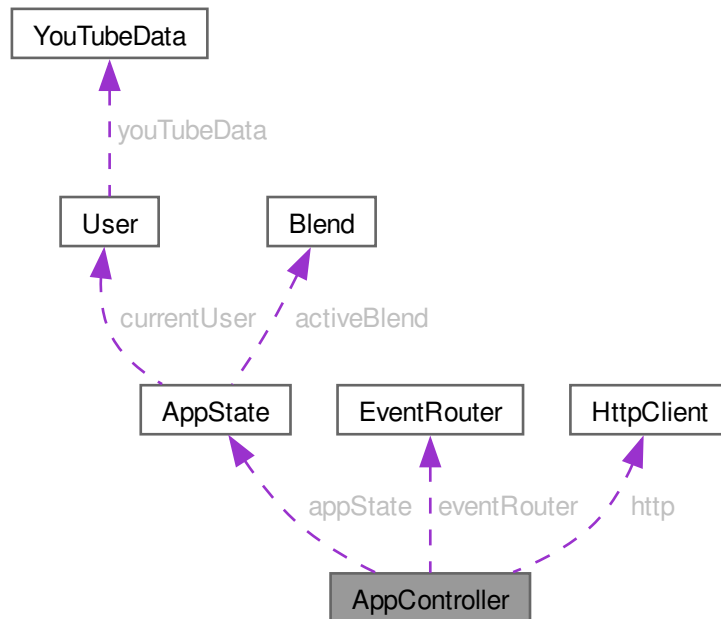
- [ActiveBlendsPanel.hpp](#)
- [ActiveBlendsPanel.cpp](#)

6.2 AppController Class Reference

Stub for future use.

```
#include <AppController.hpp>
```

Collaboration diagram for AppController:



Classes

- struct [BlendSummary](#)
[BlendSummary](#) class declaration.

Public Member Functions

- [AppController](#) ()
- [AppController](#) (const std::string &dbPath)
- [~AppController](#) ()
- [EventRouter](#) & [getEventRouter](#) ()
- void [setProgressReporter](#) (std::function< void(double, const std::string &)> reporter)
- void [clearProgressReporter](#) ()
- void [handleRegister](#) (const [EventPayload](#) &payload)
- void [handleLogin](#) (const [EventPayload](#) &payload)
- void [handleLogout](#) (const [EventPayload](#) &payload)
- void [handleDeleteAccount](#) (const [EventPayload](#) &payload)
- void [handleUploadData](#) (const [EventPayload](#) &payload)

- void [handleCreateBlend](#) (const [EventPayload](#) &payload)
- void [handlePlayVideo](#) (const [EventPayload](#) &payload)
- void [handleRefresh](#) (const [EventPayload](#) &payload)
- void [handleOpenChat](#) (const [EventPayload](#) &payload)
- bool [lookupUser](#) (const std::string &userID)
- void [handleSendMessage](#) (const [EventPayload](#) &payload)
- const [User](#) * [get_current_user](#) ()
- std::string [get_current_username](#) ()
- std::string [get_current_email](#) ()
- std::vector< [BlendSummary](#) > [fetchUserBlends](#) ()
- void [handleLeaveBlend](#) (const [EventPayload](#) &payload)
- void [handleSelectBlend](#) (const [EventPayload](#) &payload)

Static Public Member Functions

- static std::list< [Message](#) > [parseChatHistory](#) (const std::string &response)
- static std::list< [Video](#) > [enrichMissingMetadataSubsetMultithreaded](#) (const std::list< [Video](#) > &videos, std::size_t maxVideosToEnrich, std::size_t workerThreads, const std::function< std::list< [Video](#) >(const std::list< [Video](#) > &)> &enrichChunkFn)

Private Member Functions

- void [registerEvents](#) ()
- void [enrichIfMissingMetadata](#) (const std::string &userID, std::list< [Video](#) > &videos)
- void [enrichActiveBlendFeedIfMissingMetadata](#) ()
- std::list< [User](#) > [loadParticipantsWithWatchLater](#) (const std::vector< std::string > &participantIDs, std::vector< std::string > *missingData, bool enrichParticipantLists=false)
Loads participant users and enriches each with watch-later data.
- std::list< [User](#) > [loadParticipantsWithWatchLaterParallel](#) (const std::vector< std::string > &participantIDs, std::vector< std::string > *missingData, std::size_t workerThreads)
- void [reportProgress](#) (double progress, const std::string &message)
- bool [registerBlendOnServer](#) (const [Blend](#) &blend, const std::string &creatorID)

Static Private Member Functions

- static std::optional< [User](#) > [parseUserFromAuthResponse](#) (const std::string &response, const std::string &fallbackPassword)
Parses an authenticated user object from the login response payload.
- static std::optional< std::string > [parseBlendID](#) (const std::string &response)
Extracts a blend identifier from a blend-related JSON response.
- static std::vector< std::string > [parseParticipantIDs](#) (const std::string &response)
Parses participant user IDs from a blend participant response.
- static std::list< [Video](#) > [parseWatchLaterVideos](#) (const std::string &response)
Parses watch-later videos from a watch-later endpoint response.

Private Attributes

- [AppState](#) & [appState](#)
- [EventRouter](#) [eventRouter](#)
- const std::string [server_url](#)
- [HttpClient](#) [http](#)
- bool [isConnected](#)
- std::function< void(double, const std::string &)> [progressReporter](#)
- std::unique_ptr< [Blend](#) > [currentBlend](#)
- std::optional< [User](#) > [pendingRegistration_](#)
- bool [pendingAccountCreated_](#) = false

6.2.1 Detailed Description

Stub for future use.

Central application coordinator.

[AppController](#) class declaration.

Definition at line 39 of file [AppController.hpp](#).

6.2.2 Constructor & Destructor Documentation

6.2.2.1 AppController() [1/2]

```
AppController::AppController () [explicit]
```

Initialises [AppState](#), [EventRouter](#), and backend services.

Definition at line 327 of file [AppController.cpp](#).

6.2.2.2 AppController() [2/2]

```
AppController::AppController (  
    const std::string & dbPath) [explicit]
```

Compatibility constructor used by tests that inject a DB path. The current HTTP-backed controller ignores this value.

Definition at line 351 of file [AppController.cpp](#).

6.2.2.3 ~AppController()

```
AppController::~~AppController ()
```

Persists any unsaved changes, deregisters all [EventRouter](#) listeners, and releases backend resources.

Definition at line 354 of file [AppController.cpp](#).

6.2.3 Member Function Documentation

6.2.3.1 clearProgressReporter()

```
void AppController::clearProgressReporter ()
```

Clears the active UI progress sink.

Definition at line 407 of file [AppController.cpp](#).

6.2.3.2 enrichActiveBlendFeedIfMissingMetadata()

```
void AppController::enrichActiveBlendFeedIfMissingMetadata () [private]
```

Re-enriches only the currently displayed blend feed videos when any metadata fields are missing. This keeps display quality high without re-processing full watch-history sized lists on the UI path.

Definition at line 1216 of file [AppController.cpp](#).

6.2.3.3 enrichIfMissingMetadata()

```
void AppController::enrichIfMissingMetadata (  
    const std::string & userID,  
    std::list< Video > & videos) [private]
```

Re-enriches `videos` via the YouTube API if any are missing a title. Saves the updated list back to the database when enrichment occurs. No-op if all videos already have titles.

Parameters

<i>userID</i>	Owner of the video list (used for the DB save).
<i>videos</i>	Video list to inspect and potentially update in-place.

Definition at line 1100 of file [AppController.cpp](#).

6.2.3.4 enrichMissingMetadataSubsetMultithreaded()

```
std::list< Video > AppController::enrichMissingMetadataSubsetMultithreaded (
    const std::list< Video > & videos,
    std::size_t maxVideosToEnrich,
    std::size_t workerThreads,
    const std::function< std::list< Video >(const std::list< Video > &)> & enrich←
ChunkFn) [static]
```

Applies bounded metadata enrichment to at most `maxVideosToEnrich` videos that have incomplete metadata, partitioned across worker threads.

This utility is shared by production blend-loading paths and unit tests to keep metadata workflow behavior consistent.

Parameters

<i>videos</i>	list of videos to go through
<i>maxVideosToEnrich</i>	upper bound on the number of videos the algorithm should process
<i>workerThreads</i>	number of threads to be working on video enrichment
<i>enrichChunkFn</i>	function each thread will use to enrich their assigned sub-set of videos

Definition at line 1134 of file [AppController.cpp](#).

6.2.3.5 fetchUserBlends()

```
std::vector< AppController::BlendSummary > AppController::fetchUserBlends ()
```

Fetches the list of blends the current user participates in.

Returns

Vector of [BlendSummary](#) with blend IDs and titles.

Definition at line 1512 of file [AppController.cpp](#).

6.2.3.6 get_current_email()

```
std::string AppController::get_current_email ()
```

Retrieves the current user's email (if an email was used on signup)

Returns

string copy of user's email. "" if [User](#) DNE or they provided no email on signup

Definition at line 1506 of file [AppController.cpp](#).

6.2.3.7 get_current_user()

```
const User * AppController::get_current_user ()
```

Retrieves the current user from the AppSate

Returns

User shared pointer. nullptr if User DNE

Definition at line 1496 of file [AppController.cpp](#).

6.2.3.8 get_current_username()

```
std::string AppController::get_current_username ()
```

Retrieves the current users username

Returns

string copy of username. "" if User DNE

Definition at line 1500 of file [AppController.cpp](#).

6.2.3.9 getEventRouter()

```
EventRouter & AppController::getEventRouter ()
```

Returns

Reference to the [EventRouter](#) so panels can call dispatch().

Definition at line 399 of file [AppController.cpp](#).

6.2.3.10 handleCreateBlend()

```
void AppController::handleCreateBlend (
    const EventPayload & payload)
```

Runs the blend algorithm over the listed participants, stores the result, and creates a [ChatRoom](#) for the new blend.

Parameters

<i>payload</i>	{ "userID_0": "...", "userID_1": "...", ... }
----------------	---

Definition at line 946 of file [AppController.cpp](#).

6.2.3.11 handleDeleteAccount()

```
void AppController::handleDeleteAccount (
    const EventPayload & payload)
```

Deletes the current account after password re-authentication.

Parameters

<i>payload</i>	{ "userID": "...", "password": "..." }
----------------	--

Definition at line 669 of file [AppController.cpp](#).

6.2.3.12 handleLeaveBlend()

```
void AppController::handleLeaveBlend (  
    const EventPayload & payload)
```

Leaves a blend: notifies server to remove the user, posts a system message to the chat, and clears the active blend if it was the one left.

Parameters

<i>payload</i>	{ "blendID": "...", "blendTitle": "..." }
----------------	---

Definition at line 1556 of file [AppController.cpp](#).

6.2.3.13 handleLogin()

```
void AppController::handleLogin (  
    const EventPayload & payload)
```

Validates credentials and sets [AppState::currentUser](#) on success.

Parameters

<i>payload</i>	{ "userID": "...", "password": "..." }
----------------	--

Definition at line 490 of file [AppController.cpp](#).

6.2.3.14 handleLogout()

```
void AppController::handleLogout (  
    const EventPayload & payload)
```

Persists the current user, clears [AppState](#), and navigates to [LoginPanel](#).

Parameters

<i>payload</i>	{}
----------------	----

Definition at line 637 of file [AppController.cpp](#).

6.2.3.15 handleOpenChat()

```
void AppController::handleOpenChat (  
    const EventPayload & payload)
```

Opens the ChatPanel for the active blend's [ChatRoom](#). No-ops if there is no active blend or no room for it.

Parameters

<i>payload</i>	{ }
----------------	-----

Definition at line 1382 of file [AppController.cpp](#).

6.2.3.16 handlePlayVideo()

```
void AppController::handlePlayVideo (  
    const EventPayload & payload)
```

Opens the selected video (currently a stub).

Parameters

<i>payload</i>	{ "videoID": "..." }
----------------	----------------------

Definition at line 1285 of file [AppController.cpp](#).

6.2.3.17 handleRefresh()

```
void AppController::handleRefresh (  
    const EventPayload & payload)
```

Generates a new blend from the existing participants, excluding videos already in the current blend. Re-enriches any videos missing metadata via the YouTube API before sampling.

Parameters

<i>payload</i>	{ }
----------------	-----

Definition at line 1291 of file [AppController.cpp](#).

6.2.3.18 handleRegister()

```
void AppController::handleRegister (  
    const EventPayload & payload)
```

Creates a new account and logs the user in on success.

Parameters

<i>payload</i>	{ "userID": "...", "username": "...", "email": "...", "password": "..." }
----------------	---

Definition at line 423 of file [AppController.cpp](#).

6.2.3.19 handleSelectBlend()

```
void AppController::handleSelectBlend (  
    const EventPayload & payload)
```

Switches the active blend to the specified blend ID, loading participants and regenerating the video feed.

Parameters

<i>payload</i>	{ "blendID": "..." }
----------------	----------------------

Definition at line 1600 of file [AppController.cpp](#).

6.2.3.20 handleSendMessage()

```
void AppController::handleSendMessage (
    const EventPayload & payload)
```

Validates the sender is a blend participant, then adds the message.

Parameters

<i>payload</i>	{ "userID": "...", "text": "..." }
----------------	------------------------------------

Definition at line 1479 of file [AppController.cpp](#).

6.2.3.21 handleUploadData()

```
void AppController::handleUploadData (
    const EventPayload & payload)
```

Imports the file at filePath and persists the result to the database. Metadata enrichment is conditional and may be skipped for large HTML uploads to avoid blocking the UI path.

Parameters

<i>payload</i>	{ "filePath": "...", "userID": "..." }
----------------	--

Definition at line 709 of file [AppController.cpp](#).

6.2.3.22 loadParticipantsWithWatchLater()

```
std::list< User > AppController::loadParticipantsWithWatchLater (
    const std::vector< std::string > & participantIDs,
    std::vector< std::string > * missingData,
    bool enrichParticipantLists = false) [private]
```

Loads participant users and enriches each with watch-later data.

For each user ID, this method fetches watch-later videos from the server and constructs an in-memory [User](#) object containing the parsed [YouTubeData](#).

Parameters

<i>participantIDs</i>	Ordered list of user IDs to resolve.
<i>missingData</i>	Optional output vector populated with user IDs that could not be loaded or had no available watch-later data.

Returns

List of participant users that have usable watch-later data.

Definition at line 186 of file [AppController.cpp](#).

6.2.3.23 loadParticipantsWithWatchLaterParallel()

```
std::list< User > AppController::loadParticipantsWithWatchLaterParallel (
    const std::vector< std::string > & participantIDs,
    std::vector< std::string > * missingData,
    std::size_t workerThreads) [private]
```

Parallel variant used by login restore flows when many participants exist. Uses separate [HttpClient](#) instances per worker to avoid shared-client contention.

Definition at line 241 of file [AppController.cpp](#).

6.2.3.24 lookupUser()

```
bool AppController::lookupUser (
    const std::string & userID)
```

Checks whether a user with the given ID exists in the backend.

This uses the auth user-profile endpoint so account existence checks are independent from feature data (for example users with no uploaded watch-later list yet).

Parameters

<i>userID</i>	The user ID to look up.
---------------	-------------------------

Returns

true if the user was found, false otherwise.

Definition at line 1044 of file [AppController.cpp](#).

6.2.3.25 parseBlendID()

```
std::optional< std::string > AppController::parseBlendID (
    const std::string & response) [static], [private]
```

Extracts a blend identifier from a blend-related JSON response.

Parameters

<i>response</i>	Raw JSON response body expected to include <code>blend_id</code> .
-----------------	--

Returns

[Blend](#) ID when present and non-empty; otherwise `std::nullopt`.

Definition at line 114 of file [AppController.cpp](#).

6.2.3.26 parseChatHistory()

```
std::list< Message > AppController::parseChatHistory (
    const std::string & response) [static]
```

Parses the JSON response from GET /chat-history into a list of Messages. Expected format:

```
{"messages":[{"sender_id":"...", "content":"...", "sent_at":"..."},...]}
```

Exposed as public static so it can be unit-tested without a live server.

Parameters

<i>response</i>	Raw JSON string from the server.
-----------------	----------------------------------

Returns

Ordered list of Messages with original server timestamps.

Definition at line 1439 of file [AppController.cpp](#).

6.2.3.27 parseParticipantIDs()

```
std::vector< std::string > AppController::parseParticipantIDs (  
    const std::string & response) [static], [private]
```

Parses participant user IDs from a blend participant response.

Parameters

<i>response</i>	Raw JSON response body expected to include an array field named <code>participants</code> .
-----------------	---

Returns

Vector of parsed participant IDs. Returns an empty vector when payload is malformed or the field is absent.

Definition at line 128 of file [AppController.cpp](#).

6.2.3.28 parseUserFromAuthResponse()

```
std::optional< User > AppController::parseUserFromAuthResponse (  
    const std::string & response,  
    const std::string & fallbackPassword) [static], [private]
```

Parses an authenticated user object from the login response payload.

Expects a JSON object that includes at minimum a non-empty `user_id` field. Optional fields such as `username` and `email` are used when present.

Parameters

<i>response</i>	Raw JSON response body returned by the auth endpoint.
<i>fallbackPassword</i>	Password provided at login time, used to populate the in-memory <code>User</code> object when the response does not include one.

Returns

Parsed `User` on success, or `std::nullopt` when parsing fails or required fields are missing.

Definition at line 94 of file [AppController.cpp](#).

6.2.3.29 parseWatchLaterVideos()

```
std::list< Video > AppController::parseWatchLaterVideos (  
    const std::string & response) [static], [private]
```

Parses watch-later videos from a watch-later endpoint response.

Parameters

<i>response</i>	Raw JSON response body expected to include an array field named <code>videos</code> .
-----------------	---

Returns

List of parsed [Video](#) objects. Entries missing required `video_id` are skipped.

Definition at line 149 of file [AppController.cpp](#).

6.2.3.30 registerBlendOnServer()

```
bool AppController::registerBlendOnServer (
    const Blend & blend,
    const std::string & creatorID) [private]
```

POSTs the given blend to POST `/api/v1/blends` so the server provisions its chat room. No-op when offline.

Returns

true if the server accepted the blend (2xx), false otherwise.

Definition at line 1057 of file [AppController.cpp](#).

6.2.3.31 registerEvents()

```
void AppController::registerEvents () [private]
```

Registers all UI event handlers on the [EventRouter](#). Called once during construction.

Definition at line 380 of file [AppController.cpp](#).

6.2.3.32 reportProgress()

```
void AppController::reportProgress (
    double progress,
    const std::string & message) [private]
```

Sends a coarse-grained progress update to the active UI reporter.

Definition at line 411 of file [AppController.cpp](#).

6.2.3.33 setProgressReporter()

```
void AppController::setProgressReporter (
    std::function< void(double, const std::string &)> reporter)
```

Registers a UI progress sink used for long-running controller operations.

Definition at line 403 of file [AppController.cpp](#).

6.2.4 Member Data Documentation

6.2.4.1 appState

```
AppState& AppController::appState [private]
```

Definition at line 224 of file [AppController.hpp](#).

6.2.4.2 currentBlend

```
std::unique_ptr<Blend> AppController::currentBlend [private]
```

Owns the most recently generated [Blend](#) so [AppState](#) can hold a raw pointer.

Definition at line 232 of file [AppController.hpp](#).

6.2.4.3 eventRouter

```
EventRouter AppController::eventRouter [private]
```

Definition at line 225 of file [AppController.hpp](#).

6.2.4.4 http

```
HttpClient AppController::http [private]
```

Definition at line 227 of file [AppController.hpp](#).

6.2.4.5 isConnected

```
bool AppController::isConnected [private]
```

Definition at line 228 of file [AppController.hpp](#).

6.2.4.6 pendingAccountCreated_

```
bool AppController::pendingAccountCreated_ = false [private]
```

Definition at line 337 of file [AppController.hpp](#).

6.2.4.7 pendingRegistration_

```
std::optional<User> AppController::pendingRegistration_ [private]
```

Definition at line 336 of file [AppController.hpp](#).

6.2.4.8 progressReporter

```
std::function<void(double, const std::string&)> AppController::progressReporter [private]
```

Definition at line 229 of file [AppController.hpp](#).

6.2.4.9 server_url

```
const std::string AppController::server_url [private]
```

Definition at line 226 of file [AppController.hpp](#).

The documentation for this class was generated from the following files:

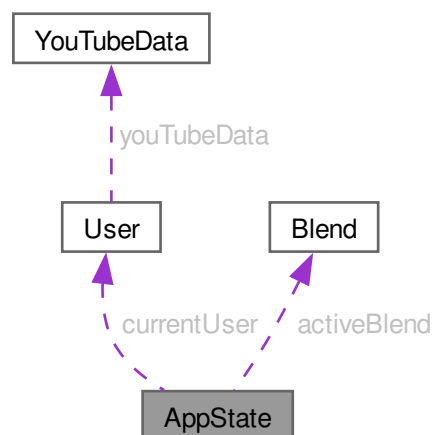
- [AppController.hpp](#)
- [AppController.cpp](#)

6.3 AppState Class Reference

[AppState](#) class declaration.

```
#include <AppState.hpp>
```

Collaboration diagram for AppState:



Public Member Functions

- [User](#) * [getCurrentUser](#) () const
- void [setCurrentUser](#) ([User](#) *user)
- [Blend](#) * [getActiveBlend](#) () const
- void [setActiveBlend](#) ([Blend](#) *blend)
- bool [isGeneratingBlend](#) () const
- void [setIsBlendGenerating](#) (bool flag)
- [ChatRoom](#) * [getActiveChatRoom](#) ()
- void [createChatRoom](#) (const [Blend](#) &blend)
- [ChatRoom](#) * [getChatRoom](#) (const std::string &blendID)
- void [addSessionUser](#) (const [User](#) &user)
- std::map< std::string, [User](#) > [getSessionUsers](#) () const
- void [clearSessionUsers](#) ()
- void [setUsersWithoutData](#) (const std::vector< std::string > &users)
- std::vector< std::string > [getUsersWithoutData](#) () const
- void [setPendingBlendMessage](#) (const std::string &msg)
- std::string [takePendingBlendMessage](#) ()
- void [setPendingUploadError](#) (const std::string &msg)
- std::string [takePendingUploadError](#) ()
- void [clearSession](#) ()

Static Public Member Functions

- static [AppState](#) & [getInstance](#) ()

Private Member Functions

- [AppState](#) (const [AppState](#) &)=delete
- [AppState](#) & [operator=](#) (const [AppState](#) &)=delete

Private Attributes

- [User](#) * [currentUser](#)
Logged-in user; null when no session is active.
- [Blend](#) * [activeBlend](#)
Most recently generated or loaded [Blend](#); null if none.
- bool [isBlendGenerating](#)
True while an [IBlendAlgorithm](#) is running.
- std::map< std::string, [ChatRoom](#) > [chatRooms](#)
- std::map< std::string, [User](#) > [sessionUsers](#)
- std::vector< std::string > [usersWithoutData](#)
- std::string [pendingBlendMessage](#)
- std::string [pendingUploadError](#)

6.3.1 Detailed Description

[AppState](#) class declaration.

Session-scoped singleton holding all runtime application state.

Definition at line 31 of file [AppState.hpp](#).

6.3.2 Constructor & Destructor Documentation

6.3.2.1 AppState()

```
AppState::AppState () [private]
```

Definition at line 11 of file [AppState.cpp](#).

6.3.3 Member Function Documentation

6.3.3.1 addSessionUser()

```
void AppState::addSessionUser (  
    const User & user)
```

Adds or replaces the session entry for this user.

Parameters

<i>user</i>	User who has uploaded blend data.
-------------	---

Definition at line 90 of file [AppState.cpp](#).

6.3.3.2 clearSession()

```
void AppState::clearSession ()
```

Resets all session state: clears currentUser, activeBlend, isBlendGenerating, chat rooms, and session users. Called by [AppController](#) on logout.

Definition at line 134 of file [AppState.cpp](#).

6.3.3.3 clearSessionUsers()

```
void AppState::clearSessionUsers ()
```

Removes all session participants (call before starting a new blend).

Definition at line 98 of file [AppState.cpp](#).

6.3.3.4 createChatRoom()

```
void AppState::createChatRoom (  
    const Blend & blend)
```

Creates a [ChatRoom](#) for the given [Blend](#). Replaces any existing room with the same blendID.

Parameters

<i>blend</i>	The Blend whose participants populate the new room.
--------------	---

Definition at line 67 of file [AppState.cpp](#).

6.3.3.5 getActiveBlend()

```
Blend * AppState::getActiveBlend () const
```

Returns

Pointer to the active [Blend](#), or nullptr if none exists.

Definition at line 42 of file [AppState.cpp](#).

6.3.3.6 getActiveChatRoom()

```
ChatRoom * AppState::getActiveChatRoom ()
```

Returns a pointer to the [ChatRoom](#) for the active [Blend](#).

Returns

Pointer to the room, or nullptr if no active blend or no room exists.

Definition at line 62 of file [AppState.cpp](#).

6.3.3.7 getChatRoom()

```
ChatRoom * AppState::getChatRoom (  
    const std::string & blendID)
```

Returns a pointer to the [ChatRoom](#) for the given blendID.

Parameters

<i>blendID</i>	ID of the desired blend.
----------------	--------------------------

Returns

Pointer to the room, or nullptr if none exists.

Definition at line 82 of file [AppState.cpp](#).

6.3.3.8 getCurrentUser()

```
User * AppState::getCurrentUser () const
```

Returns

Pointer to the logged-in user, or nullptr if no session is active.

Definition at line 26 of file [AppState.cpp](#).

6.3.3.9 getInstance()

```
AppState & AppState::getInstance () [static]
```

Returns the single global [AppState](#) instance. Creates it on first call; returns the same instance on every subsequent call.

Definition at line 18 of file [AppState.cpp](#).

6.3.3.10 getSessionUsers()

```
std::map< std::string, User > AppState::getSessionUsers () const
```

Returns

All users that have uploaded data in this session.

Definition at line 94 of file [AppState.cpp](#).

6.3.3.11 getUsersWithoutData()

```
std::vector< std::string > AppState::getUsersWithoutData () const
```

Returns

userIDs of participants without data from the last blend creation.

Definition at line 108 of file [AppState.cpp](#).

6.3.3.12 isGeneratingBlend()

```
bool AppState::isGeneratingBlend () const
```

Returns

True while an [IBlendAlgorithm](#) is running.

Definition at line 52 of file [AppState.cpp](#).

6.3.3.13 setActiveBlend()

```
void AppState::setActiveBlend (  
    Blend * blend)
```


Parameters

<i>blend</i>	Raw pointer to the active Blend (AppController owns it).
--------------	---

Definition at line 46 of file [AppState.cpp](#).

6.3.3.14 setCurrentUser()

```
void AppState::setCurrentUser (  
    User * user)
```

Parameters

<i>user</i>	Pointer to the newly logged-in User (AppController owns it).
-------------	---

Definition at line 30 of file [AppState.cpp](#).

6.3.3.15 setIsBlendGenerating()

```
void AppState::setIsBlendGenerating (  
    bool flag)
```

Parameters

<i>flag</i>	True to mark blend generation as in-progress.
-------------	---

Definition at line 56 of file [AppState.cpp](#).

6.3.3.16 setPendingBlendMessage()

```
void AppState::setPendingBlendMessage (  
    const std::string & msg)
```

Sets a one-time message shown by [LoginPanel](#) when the user is a blend participant but has not yet uploaded data.

Parameters

<i>msg</i>	Message to display.
------------	-------------------------------------

Definition at line 112 of file [AppState.cpp](#).

6.3.3.17 setPendingUploadError()

```
void AppState::setPendingUploadError (  
    const std::string & msg)
```

Sets a one-time message shown by [DataInstructionsPanel](#) when upload failed or produced no usable videos.

Parameters

<i>msg</i>	Message to display.
------------	---------------------

Definition at line 122 of file [AppState.cpp](#).

6.3.3.18 setUsersWithoutData()

```
void AppState::setUsersWithoutData (  
    const std::vector< std::string > & users)
```

Stores userIDs of participants who had no Watch Later data when the blend was last created.

Parameters

<i>users</i>	List of userIDs without data.
--------------	-------------------------------

Definition at line 104 of file [AppState.cpp](#).

6.3.3.19 takePendingBlendMessage()

```
std::string AppState::takePendingBlendMessage ()
```

Returns and clears the pending blend message.

Returns

The pending message string, then clears it.

Definition at line 116 of file [AppState.cpp](#).

6.3.3.20 takePendingUploadError()

```
std::string AppState::takePendingUploadError ()
```

Returns and clears the pending upload error message.

Returns

The pending upload error string, then clears it.

Definition at line 126 of file [AppState.cpp](#).

6.3.4 Member Data Documentation

6.3.4.1 activeBlend

```
Blend* AppState::activeBlend [private]
```

Most recently generated or loaded [Blend](#); null if none.

Definition at line 34 of file [AppState.hpp](#).

6.3.4.2 chatRooms

```
std::map<std::string, ChatRoom> AppState::chatRooms [private]
```

All chat rooms for this session, keyed by blendID.

Definition at line 38 of file [AppState.hpp](#).

6.3.4.3 currentUser

```
User* AppState::currentUser [private]
```

Logged-in user; null when no session is active.

Definition at line 33 of file [AppState.hpp](#).

6.3.4.4 isBlendGenerating

```
bool AppState::isBlendGenerating [private]
```

True while an [IBlendAlgorithm](#) is running.

Definition at line 35 of file [AppState.hpp](#).

6.3.4.5 pendingBlendMessage

```
std::string AppState::pendingBlendMessage [private]
```

One-time message set by `handleLogin` when the user belongs to a blend but has not yet uploaded data. Cleared after being read.

Definition at line 50 of file [AppState.hpp](#).

6.3.4.6 pendingUploadError

```
std::string AppState::pendingUploadError [private]
```

One-time upload failure message set by `handleUploadData` when parsing, enrichment, or persistence fails. Cleared after being read by UI.

Definition at line 56 of file [AppState.hpp](#).

6.3.4.7 sessionUsers

```
std::map<std::string, User> AppState::sessionUsers [private]
```

Participants that have uploaded Watch Later data, keyed by userID.

Definition at line 41 of file [AppState.hpp](#).

6.3.4.8 usersWithoutData

```
std::vector<std::string> AppState::usersWithoutData [private]
```

Participants who had no data when the last blend was created.

Definition at line 44 of file [AppState.hpp](#).

The documentation for this class was generated from the following files:

- [AppState.hpp](#)
- [AppState.cpp](#)

6.4 Blend Class Reference

[Blend](#) class declaration.

```
#include <Blend.hpp>
```

Public Member Functions

- [Blend](#) (const std::string &blendID, const std::string &title, const std::string &algorithmUsed, const std::list< [User](#) > &participants, const std::list< [Video](#) > &videoList)
- std::string [getBlendID](#) () const
- std::string [getTitle](#) () const
- std::string [getAlgorithmUsed](#) () const
- std::list< [User](#) > [getParticipants](#) () const
- std::list< [Video](#) > [getVideoList](#) () const
- void [setVideoList](#) (const std::list< [Video](#) > &videoList)
- [Video](#) [getVideo](#) (int index) const
- int [size](#) () const
- void [setBlendID](#) (const std::string &blendID)
- std::string [toString](#) () const

Private Attributes

- std::string [blendID](#)
- std::string [title](#)
- std::string [algorithmUsed](#)
- std::list< [User](#) > [participants](#)
- std::list< [Video](#) > [videoList](#)

6.4.1 Detailed Description

[Blend](#) class declaration.

Definition at line 24 of file [Blend.hpp](#).

6.4.2 Constructor & Destructor Documentation

6.4.2.1 Blend()

```
Blend::Blend (
    const std::string & blendID,
    const std::string & title,
    const std::string & algorithmUsed,
    const std::list< User > & participants,
    const std::list< Video > & videoList)
```

Constructs a [Blend](#) with all required fields.

Parameters

<i>blendID</i>	Unique identifier for this blend.
<i>title</i>	User-facing name for this blend.
<i>algorithmUsed</i>	Name of the IBlendAlgorithm that created it.
<i>participants</i>	Users whose Watch Later data was used as input.
<i>videoList</i>	Ordered list of videos selected by the algorithm.

Definition at line 14 of file [Blend.cpp](#).

6.4.3 Member Function Documentation

6.4.3.1 getAlgorithmUsed()

```
std::string Blend::getAlgorithmUsed () const
```

Returns

Name of the algorithm used to generate this blend.

Definition at line 35 of file [Blend.cpp](#).

6.4.3.2 getBlendID()

```
std::string Blend::getBlendID () const
```

Returns

Unique blend identifier.

Definition at line 27 of file [Blend.cpp](#).

6.4.3.3 getParticipants()

```
std::list< User > Blend::getParticipants () const
```

Returns

List of users who participated in this blend.

Definition at line 39 of file [Blend.cpp](#).

6.4.3.4 getTitle()

```
std::string Blend::getTitle () const
```

Returns

User-facing title of this blend.

Definition at line 31 of file [Blend.cpp](#).

6.4.3.5 getVideo()

```
Video Blend::getVideo (  
    int index) const
```

Returns the video at a given position, wrapping around the list.

Parameters

<i>index</i>	Zero-based index; behaviour is undefined for negative values.
--------------	---

Returns

[Video](#) at the requested position.

Definition at line 59 of file [Blend.cpp](#).

6.4.3.6 getVideoList()

```
std::list< Video > Blend::getVideoList () const
```

Returns

Ordered list of videos in this blend.

Definition at line 43 of file [Blend.cpp](#).

6.4.3.7 setBlendID()

```
void Blend::setBlendID (  
    const std::string & blendID)
```

Overrides the blend identifier (used when reconciling with server IDs).

Parameters

<i>blendID</i>	Server-authoritative blend identifier.
----------------	--

Definition at line 53 of file [Blend.cpp](#).

6.4.3.8 setVideoList()

```
void Blend::setVideoList (  
    const std::list< Video > & videoList)
```

Replaces the current video list (e.g. after API enrichment).

Parameters

<i>videoList</i>	New video list to store.
------------------	--------------------------

Definition at line 49 of file [Blend.cpp](#).

6.4.3.9 size()

```
int Blend::size () const
```

Returns

Number of videos in the blend.

Definition at line 69 of file [Blend.cpp](#).

6.4.3.10 toString()

```
std::string Blend::toString () const
```

Returns

JSON string representation of this blend.

Definition at line 73 of file [Blend.cpp](#).

6.4.4 Member Data Documentation

6.4.4.1 algorithmUsed

```
std::string Blend::algorithmUsed [private]
```

Definition at line 28 of file [Blend.hpp](#).

6.4.4.2 blendID

```
std::string Blend::blendID [private]
```

Definition at line 26 of file [Blend.hpp](#).

6.4.4.3 participants

```
std::list<User> Blend::participants [private]
```

Definition at line 29 of file [Blend.hpp](#).

6.4.4.4 title

```
std::string Blend::title [private]
```

Definition at line 27 of file [Blend.hpp](#).

6.4.4.5 videoList

```
std::list<Video> Blend::videoList [private]
```

Definition at line 30 of file [Blend.hpp](#).

The documentation for this class was generated from the following files:

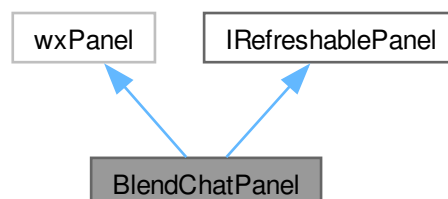
- [Blend.hpp](#)
- [Blend.cpp](#)

6.5 BlendChatPanel Class Reference

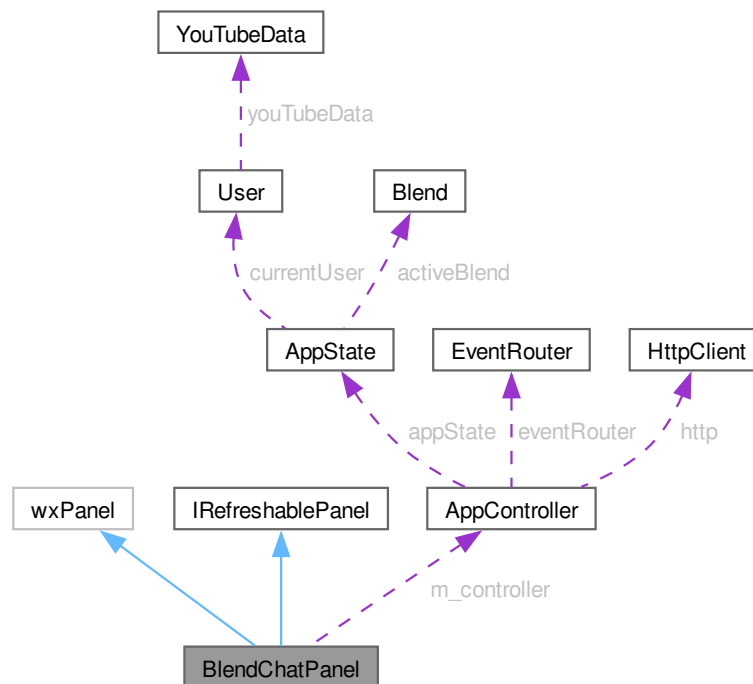
[BlendChatPanel](#) class declaration.

```
#include <BlendChatPanel.hpp>
```

Inheritance diagram for BlendChatPanel:



Collaboration diagram for BlendChatPanel:



Public Member Functions

- `BlendChatPanel` (`wxWindow *parent`, `AppController &controller`, `NavigateFn nav`)
- `void Refresh ()` override
- `void Reload ()`

Private Member Functions

- `void ConnectWebSocket` (`const std::string &blendID`, `const std::string &userID`)
- `void DoSend ()`
- `void OnSend` (`wxCommandEvent &evt`)
- `void OnInputEnter` (`wxCommandEvent &evt`)
- `void OnPollTimer` (`wxTimerEvent &evt`)
- `void OnParticipantsToggle` (`wxCommandEvent &evt`)

Private Attributes

- `AppController & m_controller`
- `NavigateFn m_nav`
- `wxScrolledWindow * m_msgScroll`
- `wxPanel * m_msgInner`
- `wxBoxSizer * m_msgSizer`
- `wxStaticText * m_noBlendLabel`

- wxButton * [m_participantsBtn](#)
- wxPanel * [m_participantsDropdown](#)
- wxBoxSizer * [m_participantsSizer](#)
- wxTextCtrl * [m_inputCtrl](#)
- wxPanel * [m_inputPanel](#)
- wxTimer * [m_pollTimer](#)
- std::string [m_wsBlendID](#)
- std::mutex [m_incomingMx](#)
- std::vector< std::pair< std::string, std::string > > [m_incoming](#)
- std::unique_ptr< [ChatWebSocket](#) > [m_ws](#)

6.5.1 Detailed Description

[BlendChatPanel](#) class declaration.

Definition at line 63 of file [BlendChatPanel.hpp](#).

6.5.2 Constructor & Destructor Documentation

6.5.2.1 BlendChatPanel()

```
BlendChatPanel::BlendChatPanel (
    wxWindow * parent,
    AppController & controller,
    NavigateFn nav)
```

Definition at line 44 of file [BlendChatPanel.cpp](#).

6.5.3 Member Function Documentation

6.5.3.1 ConnectWebSocket()

```
void BlendChatPanel::ConnectWebSocket (
    const std::string & blendID,
    const std::string & userID) [private]
```

Definition at line 368 of file [BlendChatPanel.cpp](#).

6.5.3.2 DoSend()

```
void BlendChatPanel::DoSend () [private]
```

Definition at line 317 of file [BlendChatPanel.cpp](#).

6.5.3.3 OnInputEnter()

```
void BlendChatPanel::OnInputEnter (
    wxCommandEvent & evt) [private]
```

Definition at line 354 of file [BlendChatPanel.cpp](#).

6.5.3.4 OnParticipantsToggle()

```
void BlendChatPanel::OnParticipantsToggle (
    wxCommandEvent & evt) [private]
```

Definition at line 359 of file [BlendChatPanel.cpp](#).

6.5.3.5 OnPollTimer()

```
void BlendChatPanel::OnPollTimer (
    wxTimerEvent & evt) [private]
```

Definition at line 396 of file [BlendChatPanel.cpp](#).

6.5.3.6 OnSend()

```
void BlendChatPanel::OnSend (
    wxCommandEvent & evt) [private]
```

Definition at line 349 of file [BlendChatPanel.cpp](#).

6.5.3.7 Refresh()

```
void BlendChatPanel::Refresh () [override], [virtual]
```

Refreshes the panel's display to reflect current application state. Called by [MainFrame](#) whenever the panel becomes active.

Implements [IRefreshablePanel](#).

Definition at line 154 of file [BlendChatPanel.cpp](#).

6.5.3.8 Reload()

```
void BlendChatPanel::Reload ()
```

Definition at line 159 of file [BlendChatPanel.cpp](#).

6.5.4 Member Data Documentation

6.5.4.1 m_controller

```
AppController& BlendChatPanel::m_controller [private]
```

Definition at line 73 of file [BlendChatPanel.hpp](#).

6.5.4.2 m_incoming

```
std::vector<std::pair<std::string, std::string> > BlendChatPanel::m_incoming [private]
```

Definition at line 112 of file [BlendChatPanel.hpp](#).

6.5.4.3 m_incomingMx

```
std::mutex BlendChatPanel::m_incomingMx [private]
```

Definition at line 107 of file [BlendChatPanel.hpp](#).

6.5.4.4 m_inputCtrl

```
wxTextCtrl* BlendChatPanel::m_inputCtrl [private]
```

Definition at line 93 of file [BlendChatPanel.hpp](#).

6.5.4.5 m_inputPanel

```
wxPanel* BlendChatPanel::m_inputPanel [private]
```

Definition at line 96 of file [BlendChatPanel.hpp](#).

6.5.4.6 m_msgInner

```
wxPanel* BlendChatPanel::m_msgInner [private]
```

Definition at line 79 of file [BlendChatPanel.hpp](#).

6.5.4.7 m_msgScroll

```
wxScrolledWindow* BlendChatPanel::m_msgScroll [private]
```

Definition at line 77 of file [BlendChatPanel.hpp](#).

6.5.4.8 m_msgSizer

```
wxBoxSizer* BlendChatPanel::m_msgSizer [private]
```

Definition at line 81 of file [BlendChatPanel.hpp](#).

6.5.4.9 m_nav

```
NavigateFn BlendChatPanel::m_nav [private]
```

Definition at line 74 of file [BlendChatPanel.hpp](#).

6.5.4.10 m_noBlendLabel

```
wxStaticText* BlendChatPanel::m_noBlendLabel [private]
```

Definition at line 84 of file [BlendChatPanel.hpp](#).

6.5.4.11 m_participantsBtn

```
wxButton* BlendChatPanel::m_participantsBtn [private]
```

Definition at line 87 of file [BlendChatPanel.hpp](#).

6.5.4.12 m_participantsDropdown

```
wxPanel* BlendChatPanel::m_participantsDropdown [private]
```

Definition at line 89 of file [BlendChatPanel.hpp](#).

6.5.4.13 m_participantsSizer

```
wxBoxSizer* BlendChatPanel::m_participantsSizer [private]
```

Definition at line 90 of file [BlendChatPanel.hpp](#).

6.5.4.14 m_pollTimer

```
wxTimer* BlendChatPanel::m_pollTimer [private]
```

Definition at line 101 of file [BlendChatPanel.hpp](#).

6.5.4.15 m_ws

```
std::unique_ptr<ChatWebSocket> BlendChatPanel::m_ws [private]
```

Definition at line 115 of file [BlendChatPanel.hpp](#).

6.5.4.16 m_wsBlendID

```
std::string BlendChatPanel::m_wsBlendID [private]
```

Definition at line 104 of file [BlendChatPanel.hpp](#).

The documentation for this class was generated from the following files:

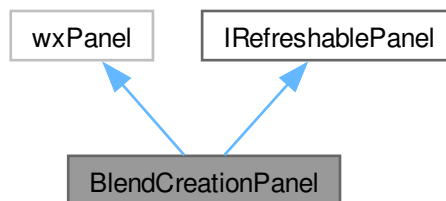
- [BlendChatPanel.hpp](#)
- [BlendChatPanel.cpp](#)

6.6 BlendCreationPanel Class Reference

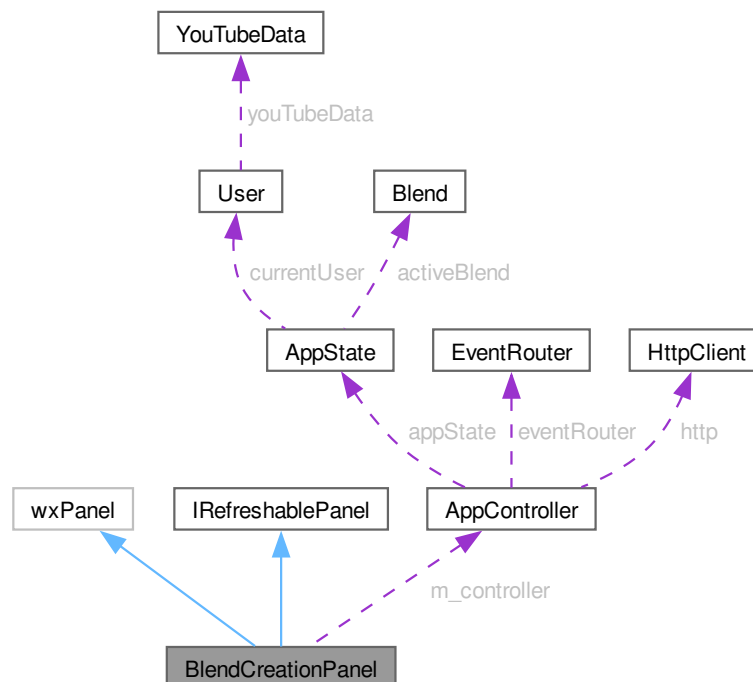
[BlendCreationPanel](#) class declaration.

```
#include <BlendCreationPanel.hpp>
```

Inheritance diagram for BlendCreationPanel:



Collaboration diagram for BlendCreationPanel:



Public Member Functions

- [BlendCreationPanel](#) (wxWindow *parent, [AppController](#) &controller, [NavigateFn](#) nav)
- void [Refresh](#) () override
- void [Reload](#) ()

Private Member Functions

- void [RebuildUserList](#) ()
- void [UpdateCountLabel](#) ()
- void [OnAdd](#) (wxCommandEvent &evt)
- void [OnCreate](#) (wxCommandEvent &evt)
- void [OnRemoveUser](#) (const std::string &username)

Private Attributes

- [AppController](#) & [m_controller](#)
- [NavigateFn](#) [m_nav](#)
- wxScrolledWindow * [m_userListScroll](#)
- wxPanel * [m_userListInner](#)
- wxBoxSizer * [m_userListSizer](#)
- wxStaticText * [m_emptyLabel](#)
- wxStaticText * [m_countLabel](#)
- wxTextCtrl * [m_blendNameCtrl](#)
- wxButton * [m_createBtn](#)
- std::vector< std::string > [m_addedUsers](#)

6.6.1 Detailed Description

[BlendCreationPanel](#) class declaration.

Definition at line 59 of file [BlendCreationPanel.hpp](#).

6.6.2 Constructor & Destructor Documentation

6.6.2.1 BlendCreationPanel()

```
BlendCreationPanel::BlendCreationPanel (  
    wxWindow * parent,  
    AppController & controller,  
    NavigateFn nav)
```

Definition at line 27 of file [BlendCreationPanel.cpp](#).

6.6.3 Member Function Documentation

6.6.3.1 OnAdd()

```
void BlendCreationPanel::OnAdd (  
    wxCommandEvent & evt) [private]
```

Definition at line 274 of file [BlendCreationPanel.cpp](#).

6.6.3.2 OnCreate()

```
void BlendCreationPanel::OnCreate (
    wxCommandEvent & evt) [private]
```

Definition at line 316 of file [BlendCreationPanel.cpp](#).

6.6.3.3 OnRemoveUser()

```
void BlendCreationPanel::OnRemoveUser (
    const std::string & username) [private]
```

Definition at line 372 of file [BlendCreationPanel.cpp](#).

6.6.3.4 RebuildUserList()

```
void BlendCreationPanel::RebuildUserList () [private]
```

Definition at line 206 of file [BlendCreationPanel.cpp](#).

6.6.3.5 Refresh()

```
void BlendCreationPanel::Refresh () [override], [virtual]
```

Refreshes the panel's display to reflect current application state. Called by [MainFrame](#) whenever the panel becomes active.

Implements [IRefreshablePanel](#).

Definition at line 179 of file [BlendCreationPanel.cpp](#).

6.6.3.6 Reload()

```
void BlendCreationPanel::Reload ()
```

Definition at line 184 of file [BlendCreationPanel.cpp](#).

6.6.3.7 UpdateCountLabel()

```
void BlendCreationPanel::UpdateCountLabel () [private]
```

Definition at line 265 of file [BlendCreationPanel.cpp](#).

6.6.4 Member Data Documentation

6.6.4.1 m_addedUsers

```
std::vector<std::string> BlendCreationPanel::m_addedUsers [private]
```

Definition at line 114 of file [BlendCreationPanel.hpp](#).

6.6.4.2 m_blendNameCtrl

```
wxTextCtrl* BlendCreationPanel::m_blendNameCtrl [private]
```

Definition at line 108 of file [BlendCreationPanel.hpp](#).

6.6.4.3 m_controller

```
AppController& BlendCreationPanel::m_controller [private]
```

Definition at line 91 of file [BlendCreationPanel.hpp](#).

6.6.4.4 m_countLabel

```
wxStaticText* BlendCreationPanel::m_countLabel [private]
```

Definition at line 105 of file [BlendCreationPanel.hpp](#).

6.6.4.5 m_createBtn

```
wxButton* BlendCreationPanel::m_createBtn [private]
```

Definition at line 111 of file [BlendCreationPanel.hpp](#).

6.6.4.6 m_emptyLabel

```
wxStaticText* BlendCreationPanel::m_emptyLabel [private]
```

Definition at line 102 of file [BlendCreationPanel.hpp](#).

6.6.4.7 m_nav

```
NavigateFn BlendCreationPanel::m_nav [private]
```

Definition at line 92 of file [BlendCreationPanel.hpp](#).

6.6.4.8 m_userListInner

```
wxPanel* BlendCreationPanel::m_userListInner [private]
```

Definition at line 97 of file [BlendCreationPanel.hpp](#).

6.6.4.9 m_userListScroll

```
wxScrolledWindow* BlendCreationPanel::m_userListScroll [private]
```

Definition at line 95 of file [BlendCreationPanel.hpp](#).

6.6.4.10 m_userListSizer

```
wxBoxSizer* BlendCreationPanel::m_userListSizer [private]
```

Definition at line 99 of file [BlendCreationPanel.hpp](#).

The documentation for this class was generated from the following files:

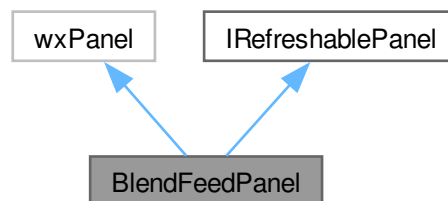
- [BlendCreationPanel.hpp](#)
- [BlendCreationPanel.cpp](#)

6.7 BlendFeedPanel Class Reference

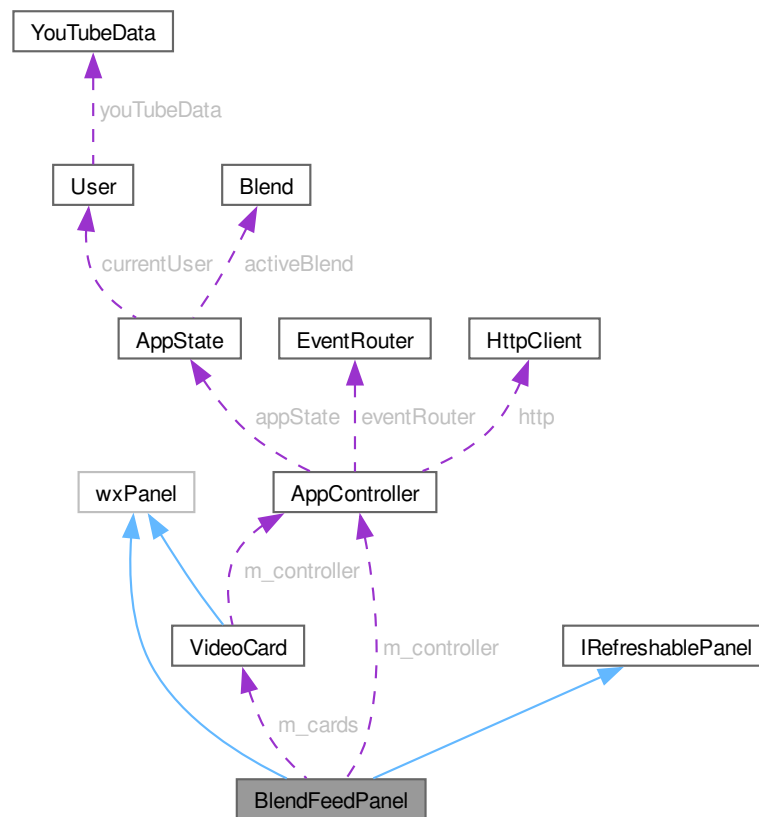
Panel that renders paged blend videos in a fixed 3x2 card layout.

```
#include <BlendFeedPanel.hpp>
```

Inheritance diagram for BlendFeedPanel:



Collaboration diagram for BlendFeedPanel:



Public Member Functions

- `BlendFeedPanel` (`wxWindow *parent`, `AppController &controller`)
Construct a `BlendFeedPanel`.
- `void Refresh ()` override
Refresh panel state from current application data.
- `void NextPage ()`
Advance to the next six videos, wrapping around the blend.
- `void LoadFromBlend ()`
Reload the current page from the active blend.

Private Attributes

- `AppController & m_controller`
- `VideoCard * m_cards [kPageSize]`
- `int m_offset`

Static Private Attributes

- `static constexpr int kPageSize = 6`

6.7.1 Detailed Description

Panel that renders paged blend videos in a fixed 3x2 card layout.

Definition at line 33 of file [BlendFeedPanel.hpp](#).

6.7.2 Constructor & Destructor Documentation

6.7.2.1 BlendFeedPanel()

```
BlendFeedPanel::BlendFeedPanel (
    wxWindow * parent,
    AppController & controller)
```

Construct a [BlendFeedPanel](#).

Parameters

<i>parent</i>	Parent wxWidgets window.
<i>controller</i>	Application controller used for refresh dispatches.

Definition at line 21 of file [BlendFeedPanel.cpp](#).

6.7.3 Member Function Documentation

6.7.3.1 LoadFromBlend()

```
void BlendFeedPanel::LoadFromBlend ()
```

Reload the current page from the active blend.

Clears all cards if there is no active blend.

Definition at line 76 of file [BlendFeedPanel.cpp](#).

6.7.3.2 NextPage()

```
void BlendFeedPanel::NextPage ()
```

Advance to the next six videos, wrapping around the blend.

Definition at line 53 of file [BlendFeedPanel.cpp](#).

6.7.3.3 Refresh()

```
void BlendFeedPanel::Refresh () [override], [virtual]
```

Refresh panel state from current application data.

Implements [IRefreshablePanel](#).

Definition at line 44 of file [BlendFeedPanel.cpp](#).

6.7.4 Member Data Documentation

6.7.4.1 kPageSize

```
int BlendFeedPanel::kPageSize = 6 [static], [constexpr], [private]
```

Definition at line 60 of file [BlendFeedPanel.hpp](#).

6.7.4.2 m_cards

```
VideoCard* BlendFeedPanel::m_cards[kPageSize] [private]
```

Definition at line 63 of file [BlendFeedPanel.hpp](#).

6.7.4.3 m_controller

```
AppController& BlendFeedPanel::m_controller [private]
```

Definition at line 62 of file [BlendFeedPanel.hpp](#).

6.7.4.4 m_offset

```
int BlendFeedPanel::m_offset [private]
```

Definition at line 64 of file [BlendFeedPanel.hpp](#).

The documentation for this class was generated from the following files:

- [BlendFeedPanel.hpp](#)
- [BlendFeedPanel.cpp](#)

6.8 AppController::BlendSummary Struct Reference

[BlendSummary](#) class declaration.

```
#include <AppController.hpp>
```

Public Attributes

- std::string [blendID](#)
- std::string [title](#)

6.8.1 Detailed Description

[BlendSummary](#) class declaration.

Holds summary info about a blend for display in the active blends list.

Definition at line 171 of file [AppController.hpp](#).

6.8.2 Member Data Documentation

6.8.2.1 blendID

```
std::string AppController::BlendSummary::blendID
```

Definition at line 172 of file [AppController.hpp](#).

6.8.2.2 title

```
std::string AppController::BlendSummary::title
```

Definition at line 173 of file [AppController.hpp](#).

The documentation for this struct was generated from the following file:

- [AppController.hpp](#)

6.9 BtnConfig Struct Reference

Width/height dimensions for a button preset.

```
#include <ButtonsConfig.hpp>
```

Public Attributes

- int [minWidth](#)
- int [minHeight](#)
- int [maxWidth](#)
- int [maxHeight](#)

6.9.1 Detailed Description

Width/height dimensions for a button preset.

A width of -1 means the control can expand horizontally in an expanding sizer while keeping the configured minimum height.

Definition at line 50 of file [ButtonsConfig.hpp](#).

6.9.2 Member Data Documentation

6.9.2.1 maxHeight

```
int BtnConfig::maxHeight
```

Definition at line 54 of file [ButtonsConfig.hpp](#).

6.9.2.2 maxWidth

```
int BtnConfig::maxWidth
```

Definition at line 53 of file [ButtonsConfig.hpp](#).

6.9.2.3 minHeight

```
int BtnConfig::minHeight
```

Definition at line 52 of file [ButtonsConfig.hpp](#).

6.9.2.4 minWidth

```
int BtnConfig::minWidth
```

Definition at line 51 of file [ButtonsConfig.hpp](#).

The documentation for this struct was generated from the following file:

- [ButtonsConfig.hpp](#)

6.10 Channel Class Reference

[Channel](#) class declaration.

```
#include <Channel.hpp>
```

Public Member Functions

- [Channel](#) (const std::string &channelID, const std::string &displayName, const std::list< std::string > &categories)
- std::string [getChannelID](#) () const
- std::string [getDisplayName](#) () const
- std::list< std::string > [getCategories](#) () const
- std::string [toString](#) () const

Private Attributes

- `std::string` [channelID](#)
- `std::string` [displayName](#)
- `std::list< std::string >` [categories](#)

6.10.1 Detailed Description

[Channel](#) class declaration.

Definition at line 17 of file [Channel.hpp](#).

6.10.2 Constructor & Destructor Documentation

6.10.2.1 Channel()

```
Channel::Channel (
    const std::string & channelID,
    const std::string & displayName,
    const std::list< std::string > & categories)
```

Constructs a [Channel](#).

Parameters

<i>channelID</i>	Unique YouTube channel identifier.
<i>displayName</i>	Human-readable channel name.
<i>categories</i>	List of content categories for this channel.

Definition at line 12 of file [Channel.cpp](#).

6.10.3 Member Function Documentation

6.10.3.1 getCategories()

```
std::list< std::string > Channel::getCategories () const
```

Returns

Content categories for this channel.

Definition at line 27 of file [Channel.cpp](#).

6.10.3.2 getChannelID()

```
std::string Channel::getChannelID () const
```

Returns

Unique YouTube channel identifier.

Definition at line 19 of file [Channel.cpp](#).

6.10.3.3 getDisplayName()

```
std::string Channel::getDisplayName () const
```

Returns

Human-readable channel name.

Definition at line 23 of file [Channel.cpp](#).

6.10.3.4 toString()

```
std::string Channel::toString () const
```

Definition at line 31 of file [Channel.cpp](#).

6.10.4 Member Data Documentation

6.10.4.1 categories

```
std::list<std::string> Channel::categories [private]
```

Definition at line 21 of file [Channel.hpp](#).

6.10.4.2 channelId

```
std::string Channel::channelID [private]
```

Definition at line 19 of file [Channel.hpp](#).

6.10.4.3 displayName

```
std::string Channel::displayName [private]
```

Definition at line 20 of file [Channel.hpp](#).

The documentation for this class was generated from the following files:

- [Channel.hpp](#)
- [Channel.cpp](#)

6.11 ChatRoom Class Reference

[ChatRoom](#) class declaration.

```
#include <ChatRoom.hpp>
```

Public Member Functions

- [ChatRoom](#) (const std::string &[blendID](#), const std::vector< std::string > &[participantIDs](#))
- std::string [getBlendID](#) () const
- std::vector< std::string > [getParticipantIDs](#) () const
- std::list< [Message](#) > [getMessages](#) () const
- void [addMessage](#) (const std::string &userID, const std::string &text)
- void [addMessage](#) (const [Message](#) &msg)
- bool [isParticipant](#) (const std::string &userID) const
- void [clearMessages](#) ()
- std::string [toString](#) () const

Private Attributes

- std::string [blendID](#)
ID of the owning [Blend](#).
- std::vector< std::string > [participantIDs](#)
userIDs pulled from the [Blend](#) on construction.
- std::list< [Message](#) > [messages](#)
In-memory history, oldest first.

6.11.1 Detailed Description

[ChatRoom](#) class declaration.

In-memory chat session associated with a [Blend](#).

Definition at line 28 of file [ChatRoom.hpp](#).

6.11.2 Constructor & Destructor Documentation

6.11.2.1 ChatRoom()

```
ChatRoom::ChatRoom (
    const std::string & blendID,
    const std::vector< std::string > & participantIDs)
```

Constructs a [ChatRoom](#) for a given blend.

Parameters

<i>blendID</i>	ID of the Blend this room belongs to.
<i>participantIDs</i>	userIDs of all blend participants.

Definition at line 13 of file [ChatRoom.cpp](#).

6.11.3 Member Function Documentation

6.11.3.1 addMessage() [1/2]

```
void ChatRoom::addMessage (  
    const Message & msg)
```

Appends a pre-constructed [Message](#) (preserving its timestamp). Used when loading history from the server so original sent_at is kept.

Parameters

<i>msg</i>	The Message to append.
------------	--

Definition at line 37 of file [ChatRoom.cpp](#).

6.11.3.2 addMessage() [2/2]

```
void ChatRoom::addMessage (  
    const std::string & userID,  
    const std::string & text)
```

Appends a new message to the room's history.

Parameters

<i>userID</i>	ID of the sending user.
<i>text</i>	Message content.

Definition at line 33 of file [ChatRoom.cpp](#).

6.11.3.3 clearMessages()

```
void ChatRoom::clearMessages ()
```

Clears all messages from memory. Has no effect on persistence since persistence is not implemented yet.

Definition at line 46 of file [ChatRoom.cpp](#).

6.11.3.4 getBlendID()

```
std::string ChatRoom::getBlendID () const
```

Returns

ID of the [Blend](#) this room belongs to.

Definition at line 19 of file [ChatRoom.cpp](#).

6.11.3.5 `getMessages()`

```
std::list< Message > ChatRoom::getMessages () const
```

Returns

In-memory message history, oldest first.

Definition at line 27 of file [ChatRoom.cpp](#).

6.11.3.6 `getParticipantIDs()`

```
std::vector< std::string > ChatRoom::getParticipantIDs () const
```

Returns

List of participant userIDs.

Definition at line 23 of file [ChatRoom.cpp](#).

6.11.3.7 `isParticipant()`

```
bool ChatRoom::isParticipant (
    const std::string & userID) const
```

Returns true if the given userID is a participant in this room. Used to validate that only blend participants can send messages.

Parameters

<i>userID</i>	ID to check.
---------------	--------------

Returns

true if the user is a participant.

Definition at line 41 of file [ChatRoom.cpp](#).

6.11.3.8 `toString()`

```
std::string ChatRoom::toString () const
```

Definition at line 50 of file [ChatRoom.cpp](#).

6.11.4 Member Data Documentation

6.11.4.1 blendID

```
std::string ChatRoom::blendID [private]
```

ID of the owning [Blend](#).

Definition at line 30 of file [ChatRoom.hpp](#).

6.11.4.2 messages

```
std::list<Message> ChatRoom::messages [private]
```

In-memory history, oldest first.

Definition at line 32 of file [ChatRoom.hpp](#).

6.11.4.3 participantIDs

```
std::vector<std::string> ChatRoom::participantIDs [private]
```

userIDs pulled from the [Blend](#) on construction.

Definition at line 31 of file [ChatRoom.hpp](#).

The documentation for this class was generated from the following files:

- [ChatRoom.hpp](#)
- [ChatRoom.cpp](#)

6.12 ChatWebSocket Class Reference

[ChatWebSocket](#) class declaration.

```
#include <ChatWebSocket.hpp>
```

Public Member Functions

- [ChatWebSocket](#) (const std::string url)
- [~ChatWebSocket](#) ()
- void [connect](#) ()
- void [setOnMessage](#) (std::function< void(const std::string &senderID, const std::string &content)> cb)
- void [set_blendID](#) (const std::string blendID)
- void [set_userID](#) (const std::string userID)
- std::string [get_fullUrl](#) ()
- void [send_message](#) (std::string msg)

Private Member Functions

- `std::string content_formatter` (`std::string content`)
- `void build_fullUrl` ()

Private Attributes

- `ix::WebSocket ws`
- `std::string url` = `kAppBackendWsPrefix`
- `std::string blendID`
- `std::string userID`
- `std::string fullUrl`
- `bool isStarted` = `false`
- `std::function< void(const std::string &, const std::string &)>` `m_onMessage`

6.12.1 Detailed Description

`ChatWebSocket` class declaration.

Definition at line 25 of file `ChatWebSocket.hpp`.

6.12.2 Constructor & Destructor Documentation

6.12.2.1 ChatWebSocket()

```
ChatWebSocket::ChatWebSocket (
    const std::string url)
```

Sets up WebSocket to perform specialized actions upon successful connection, message receiving, and errors. Given the WebSocket remains connected after starting up until its been told to stop listening for responses

Parameters

<code>url</code>	Is assigned to this->url
------------------	--------------------------

Definition at line 38 of file `ChatWebSocket.cpp`.

6.12.2.2 ~ChatWebSocket()

```
ChatWebSocket::~~ChatWebSocket ()
```

Stops the WebSocket from listening for responses.

Definition at line 68 of file `ChatWebSocket.cpp`.

6.12.3 Member Function Documentation

6.12.3.1 build_fullUrl()

```
void ChatWebSocket::build_fullUrl () [private]
```

helper function that builds the full url path for the socket to send messages to

Definition at line 100 of file [ChatWebSocket.cpp](#).

6.12.3.2 connect()

```
void ChatWebSocket::connect ()
```

Opens the WebSocket connection without sending a message. Must be called after [set_blendID\(\)](#) and [set_userID\(\)](#) are set. No-op if already connected.

Definition at line 78 of file [ChatWebSocket.cpp](#).

6.12.3.3 content_formatter()

```
string ChatWebSocket::content_formatter (
    std::string content) [private]
```

Formats string into appropriate JSON structure as needed by the server.

Parameters

<i>content</i>	String to be formatted
----------------	------------------------

Returns

the structure JSON form of content ("{"content": <content>}")

Definition at line 96 of file [ChatWebSocket.cpp](#).

6.12.3.4 get_fullUrl()

```
string ChatWebSocket::get_fullUrl ()
```

Retrieves the full websocket url

Returns

string of currently configured websocket url

Definition at line 76 of file [ChatWebSocket.cpp](#).

6.12.3.5 send_message()

```
void ChatWebSocket::send_message (
    std::string msg)
```

Sends a message through the WebSocket (given the url has been correctly configured with blend and user ID)

Parameters

<i>msg</i>	String message (leave as plain string, no formatting needed)
------------	--

Definition at line 91 of file [ChatWebSocket.cpp](#).

6.12.3.6 set_blendID()

```
void ChatWebSocket::set_blendID (  
    const std::string blendID)
```

Sets blendID

Parameters

<i>blendID</i>	the blend id to be set
----------------	------------------------

Definition at line 74 of file [ChatWebSocket.cpp](#).

6.12.3.7 set_userID()

```
void ChatWebSocket::set_userID (  
    const std::string userID)
```

Sets userID

Parameters

<i>userID</i>	the user id to be set
---------------	-----------------------

Definition at line 75 of file [ChatWebSocket.cpp](#).

6.12.3.8 setOnMessage()

```
void ChatWebSocket::setOnMessage (  
    std::function< void(const std::string &senderID, const std::string &content)> cb)
```

Registers a callback invoked (on the WebSocket background thread) whenever a chat message is received from the server.

Parameters

<i>cb</i>	Called with (senderID, messageContent).
-----------	---

Definition at line 86 of file [ChatWebSocket.cpp](#).

6.12.4 Member Data Documentation

6.12.4.1 blendID

```
std::string ChatWebSocket::blendID [private]
```

Definition at line 87 of file [ChatWebSocket.hpp](#).

6.12.4.2 fullUrl

```
std::string ChatWebSocket::fullUrl [private]
```

Definition at line 91 of file [ChatWebSocket.hpp](#).

6.12.4.3 isStarted

```
bool ChatWebSocket::isStarted = false [private]
```

Definition at line 93 of file [ChatWebSocket.hpp](#).

6.12.4.4 m_onMessage

```
std::function<void(const std::string&, const std::string&)> ChatWebSocket::m_onMessage [private]
```

Definition at line 95 of file [ChatWebSocket.hpp](#).

6.12.4.5 url

```
std::string ChatWebSocket::url = kAppBackendWsPrefix [private]
```

Definition at line 85 of file [ChatWebSocket.hpp](#).

6.12.4.6 userID

```
std::string ChatWebSocket::userID [private]
```

Definition at line 89 of file [ChatWebSocket.hpp](#).

6.12.4.7 ws

```
ix::WebSocket ChatWebSocket::ws [private]
```

Definition at line 83 of file [ChatWebSocket.hpp](#).

The documentation for this class was generated from the following files:

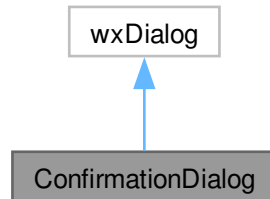
- [ChatWebSocket.hpp](#)
- [ChatWebSocket.cpp](#)

6.13 ConfirmationDialog Class Reference

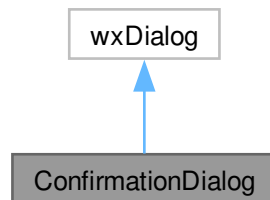
[ConfirmationDialog](#) class declaration.

```
#include <ConfirmationDialog.hpp>
```

Inheritance diagram for ConfirmationDialog:



Collaboration diagram for ConfirmationDialog:



Public Member Functions

- [ConfirmationDialog](#) (wxWindow *parent, const wxString &title, const wxString &message, const wxString &okLabel=wxT("OK"), const wxString &cancelLabel=wxT("Cancel"))

Static Public Member Functions

- static bool [Confirm](#) (wxWindow *parent, const wxString &title, const wxString &message)

6.13.1 Detailed Description

[ConfirmationDialog](#) class declaration.

Definition at line 43 of file [ConfirmationDialog.hpp](#).

6.13.2 Constructor & Destructor Documentation

6.13.2.1 ConfirmationDialog()

```
ConfirmationDialog::ConfirmationDialog (
    wxWindow * parent,
    const wxString & title,
    const wxString & message,
    const wxString & okLabel = wxT("OK"),
    const wxString & cancelLabel = wxT("Cancel"))
```

Definition at line 18 of file [ConfirmationDialog.cpp](#).

6.13.3 Member Function Documentation

6.13.3.1 Confirm()

```
bool ConfirmationDialog::Confirm (
    wxWindow * parent,
    const wxString & title,
    const wxString & message) [static]
```

Definition at line 107 of file [ConfirmationDialog.cpp](#).

The documentation for this class was generated from the following files:

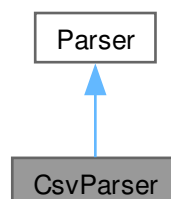
- [ConfirmationDialog.hpp](#)
- [ConfirmationDialog.cpp](#)

6.14 CsvParser Class Reference

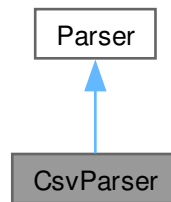
[CsvParser](#) class declaration.

```
#include <CsvParser.hpp>
```

Inheritance diagram for CsvParser:



Collaboration diagram for CsvParser:



Public Member Functions

- [CsvParser](#) (`std::list< std::string > raw_data`)
- `std::list< std::unordered_map< std::string, std::string > > parse ()` override
- `void setData (std::list< std::string > data)` override
- `int getParsed ()` override

Private Attributes

- `std::list< std::string > raw_data`
- `int parse_id`

6.14.1 Detailed Description

[CsvParser](#) class declaration.

CSV file parser implementing the [Parser](#) interface.

Definition at line 19 of file [CsvParser.hpp](#).

6.14.2 Constructor & Destructor Documentation

6.14.2.1 CsvParser() [1/2]

```
CsvParser::CsvParser ()
```

Definition at line 13 of file [CsvParser.cpp](#).

6.14.2.2 CsvParser() [2/2]

```
CsvParser::CsvParser (  
    std::list< std::string > raw_data)
```

Constructs a [CsvParser](#) pre-loaded with raw data.

Parameters

<code>raw_data</code>	Lines of raw CSV text to be parsed.
-----------------------	-------------------------------------

6.14.2.3 ~CsvParser()

```
CsvParser::~CsvParser ()
```

Definition at line 17 of file [CsvParser.cpp](#).

6.14.3 Member Function Documentation

6.14.3.1 getParseId()

```
int CsvParser::getParseId () [override], [virtual]
```

Returns

File_ID::CSV

Implements [Parser](#).

Definition at line 79 of file [CsvParser.cpp](#).

6.14.3.2 parse()

```
list< unordered_map< string, string > > CsvParser::parse () [override], [virtual]
```

Parses the loaded raw CSV data into a list of row dictionaries. The first row is used as column headers (keys).

Returns

List of maps from column name to cell value.

Implements [Parser](#).

Definition at line 19 of file [CsvParser.cpp](#).

6.14.3.3 setData()

```
void CsvParser::setData (
    std::list< std::string > data) [override], [virtual]
```

Replaces the raw data to be parsed on the next [parse\(\)](#) call.

Parameters

<i>data</i>	Lines of raw CSV text.
-------------	------------------------

Implements [Parser](#).

Definition at line 78 of file [CsvParser.cpp](#).

6.14.4 Member Data Documentation

6.14.4.1 parse_id

```
int CsvParser::parse_id [private]
```

Definition at line 47 of file [CsvParser.hpp](#).

6.14.4.2 raw_data

```
std::list<std::string> CsvParser::raw_data [private]
```

Definition at line 46 of file [CsvParser.hpp](#).

The documentation for this class was generated from the following files:

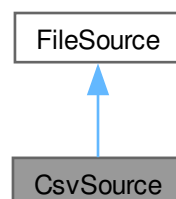
- [CsvParser.hpp](#)
- [CsvParser.cpp](#)

6.15 CsvSource Class Reference

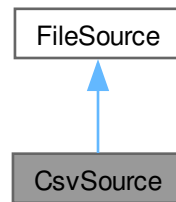
[CsvSource](#) class declaration.

```
#include <CsvSource.hpp>
```

Inheritance diagram for CsvSource:



Collaboration diagram for CsvSource:



Public Member Functions

- [CsvSource](#) (std::string src)
- std::list< std::string > [read](#) () override
- void [setSource](#) (std::string src) override
- int [getSourceId](#) () override

Private Attributes

- std::string [src](#)
- int [source_id](#)

6.15.1 Detailed Description

[CsvSource](#) class declaration.

Definition at line 18 of file [CsvSource.hpp](#).

6.15.2 Constructor & Destructor Documentation

6.15.2.1 CsvSource()

```
CsvSource::CsvSource (
    std::string src)
```

Constructs a [CsvSource](#) configured for the given file path.

Parameters

<code>src</code>	Path to the CSV file to read.
------------------	-------------------------------

Definition at line 11 of file [CsvSource.cpp](#).

6.15.3 Member Function Documentation

6.15.3.1 getSourceId()

```
int CsvSource::getSourceId () [override], [virtual]
```

Returns

File_ID::CSV

Implements [FileSource](#).

Definition at line 46 of file [CsvSource.cpp](#).

6.15.3.2 read()

```
list< string > CsvSource::read () [override], [virtual]
```

Reads the configured CSV file line-by-line.

Returns

List of raw text lines from the file.

Implements [FileSource](#).

Definition at line 15 of file [CsvSource.cpp](#).

6.15.3.3 setSource()

```
void CsvSource::setSource (
    std::string src) [override], [virtual]
```

Changes the file path used by [read\(\)](#).

Parameters

src	New file path.
-----	----------------

Implements [FileSource](#).

Definition at line 45 of file [CsvSource.cpp](#).

6.15.4 Member Data Documentation

6.15.4.1 source_id

```
int CsvSource::source_id [private]
```

Definition at line 44 of file [CsvSource.hpp](#).

6.15.4.2 src

```
std::string CsvSource::src [private]
```

Definition at line 43 of file [CsvSource.hpp](#).

The documentation for this class was generated from the following files:

- [CsvSource.hpp](#)
- [CsvSource.cpp](#)

6.16 DataExtractor Class Reference

[DataExtractor](#) class declaration.

```
#include <DataExtractor.hpp>
```

Public Member Functions

- [DataExtractor](#) (std::unique_ptr< [FileSource](#) > src, std::unique_ptr< [Parser](#) > p)
- [~DataExtractor](#) ()
- std::list< std::unordered_map< std::string, std::string > > [extract](#) ()
- void [setSource](#) (std::unique_ptr< [FileSource](#) > src)
- void [setParser](#) (std::unique_ptr< [Parser](#) > p)

Private Attributes

- std::unique_ptr< [FileSource](#) > [src_format](#)
- std::unique_ptr< [Parser](#) > [par_format](#)

6.16.1 Detailed Description

[DataExtractor](#) class declaration.

Coordinates [FileSource](#) and [Parser](#) to extract and parse file data.

Definition at line 35 of file [DataExtractor.hpp](#).

6.16.2 Constructor & Destructor Documentation

6.16.2.1 DataExtractor()

```
DataExtractor::DataExtractor (  
    std::unique_ptr< FileSource > src,  
    std::unique_ptr< Parser > p)
```

Constructs a [DataExtractor](#) with [FileSource](#) and [Parser](#).

Parameters

<i>src</i>	Unique pointer to FileSource implementation (must be configured)
<i>p</i>	Unique pointer to Parser implementation (must be configured)

Note

[FileSource](#) and [Parser](#) must have matching format IDs

Definition at line 14 of file [DataExtractor.cpp](#).

6.16.2.2 ~DataExtractor()

```
DataExtractor::~DataExtractor ()
```

Destructor. Cleans up owned [FileSource](#) and [Parser](#) objects.

Definition at line 20 of file [DataExtractor.cpp](#).

6.16.3 Member Function Documentation**6.16.3.1 extract()**

```
list< unordered_map< string, string > > DataExtractor::extract ()
```

Extracts and parses file data using configured [FileSource](#) and [Parser](#).

Reads raw data from [FileSource](#), validates format IDs match, feeds data to [Parser](#), and returns structured format.

Returns

List of dictionaries where each dict represents a data row

Exceptions

<i>Calls</i>	exit(1) if FileSource and Parser format IDs don't match
<i>Calls</i>	exit(1) if pointers are null or validation fails

Definition at line 22 of file [DataExtractor.cpp](#).

6.16.3.2 setParser()

```
void DataExtractor::setParser (
    std::unique_ptr< Parser > p)
```

Replaces the current [Parser](#) with a new one.

Parameters

<i>p</i>	Unique pointer to new Parser implementation
----------	---

Definition at line 60 of file [DataExtractor.cpp](#).

6.16.3.3 setSource()

```
void DataExtractor::setSource (
    std::unique_ptr< FileSource > src)
```

Replaces the current [FileSource](#) with a new one.

Parameters

<i>src</i>	Unique pointer to new FileSource implementation
------------	---

Definition at line 64 of file [DataExtractor.cpp](#).

6.16.4 Member Data Documentation

6.16.4.1 par_format

```
std::unique_ptr<Parser> DataExtractor::par_format [private]
```

Definition at line 70 of file [DataExtractor.hpp](#).

6.16.4.2 src_format

```
std::unique_ptr<FileSource> DataExtractor::src_format [private]
```

Definition at line 69 of file [DataExtractor.hpp](#).

The documentation for this class was generated from the following files:

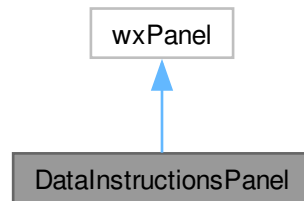
- [DataExtractor.hpp](#)
- [DataExtractor.cpp](#)

6.17 DataInstructionsPanel Class Reference

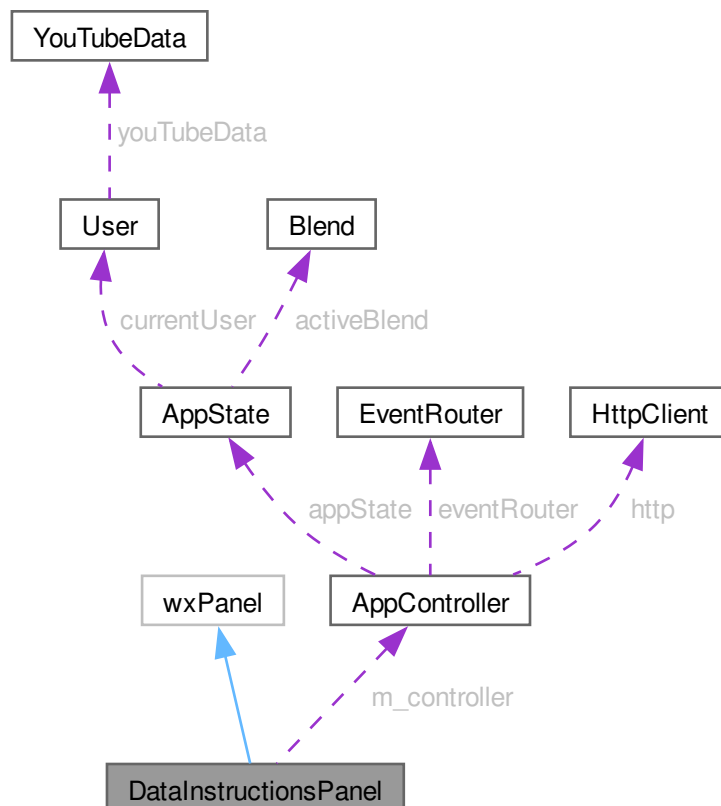
[DataInstructionsPanel](#) class declaration.

```
#include <DataInstructionsPanel.hpp>
```

Inheritance diagram for DataInstructionsPanel:



Collaboration diagram for DataInstructionsPanel:



Public Member Functions

- [DataInstructionsPanel](#) (wxWindow *parent, [AppController](#) &controller, [NavigateFn](#) nav)

Private Member Functions

- void [OnBrowse](#) (wxCommandEvent &)

Private Attributes

- [AppController](#) & m_controller
- [NavigateFn](#) m_nav

6.17.1 Detailed Description

[DataInstructionsPanel](#) class declaration.

Definition at line 50 of file [DataInstructionsPanel.hpp](#).

6.17.2 Constructor & Destructor Documentation

6.17.2.1 DataInstructionsPanel()

```
DataInstructionsPanel::DataInstructionsPanel (  
    wxWindow * parent,  
    AppController & controller,  
    NavigateFn nav)
```

Definition at line 23 of file [DataInstructionsPanel.cpp](#).

6.17.3 Member Function Documentation

6.17.3.1 OnBrowse()

```
void DataInstructionsPanel::OnBrowse (  
    wxCommandEvent & ) [private]
```

Definition at line 129 of file [DataInstructionsPanel.cpp](#).

6.17.4 Member Data Documentation

6.17.4.1 m_controller

```
AppController& DataInstructionsPanel::m_controller [private]
```

Definition at line 55 of file [DataInstructionsPanel.hpp](#).

6.17.4.2 m_nav

`NavigateFn` `DataInstructionsPanel::m_nav` [private]

Definition at line 56 of file [DataInstructionsPanel.hpp](#).

The documentation for this class was generated from the following files:

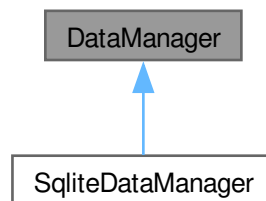
- [DataInstructionsPanel.hpp](#)
- [DataInstructionsPanel.cpp](#)

6.18 DataManager Class Reference

[DataManager](#) class declaration.

```
#include <DataManager.hpp>
```

Inheritance diagram for `DataManager`:



Public Member Functions

- virtual bool [createUser](#) (const [User](#) &user)=0
- virtual std::optional< [User](#) > [findUserByID](#) (const std::string &userID)=0
- virtual bool [validatePassword](#) (const std::string &userID, const std::string &password)=0
- virtual bool [saveWatchLater](#) (const std::string &userID, const std::list< [Video](#) > &videos)=0
- virtual std::list< [Video](#) > [loadWatchLater](#) (const std::string &userID)=0
- virtual bool [saveBlend](#) (const std::string &blendID, const std::string &creatorID, const std::string &algorithm, const std::vector< std::string > &participantIDs)=0
- virtual std::optional< std::string > [findBlendForUser](#) (const std::string &userID)=0
- virtual std::vector< std::string > [loadBlendParticipants](#) (const std::string &blendID)=0

6.18.1 Detailed Description

[DataManager](#) class declaration.

Definition at line 25 of file [DataManager.hpp](#).

6.18.2 Member Function Documentation

6.18.2.1 createUser()

```
virtual bool DataManager::createUser (
    const User & user) [pure virtual]
```

Persists a new user record.

Returns

true on success, false if the userID already exists or the insert otherwise fails.

Implemented in [SqliteDataManager](#).

6.18.2.2 findBlendForUser()

```
virtual std::optional< std::string > DataManager::findBlendForUser (
    const std::string & userID) [pure virtual]
```

Returns the blendID of the blend this user belongs to, if any.

Returns

The blendID string, or std::nullopt if the user has no blend.

Implemented in [SqliteDataManager](#).

6.18.2.3 findUserByID()

```
virtual std::optional< User > DataManager::findUserByID (
    const std::string & userID) [pure virtual]
```

Loads a user by their unique ID.

Returns

The [User](#) if found, std::nullopt otherwise.

Implemented in [SqliteDataManager](#).

6.18.2.4 loadBlendParticipants()

```
virtual std::vector< std::string > DataManager::loadBlendParticipants (
    const std::string & blendID) [pure virtual]
```

Returns all participant userIDs for a given blend.

Parameters

<i>blendID</i>	to be used for loading the respective participants
----------------	--

Returns

vector of strings representing userIDs

Implemented in [SqliteDataManager](#).

6.18.2.5 loadWatchLater()

```
virtual std::list< Video > DataManager::loadWatchLater (
    const std::string & userID) [pure virtual]
```

Loads the stored Watch Later video list for a user.

Returns

Empty list if no data has been saved for this user.

Implemented in [SqliteDataManager](#).

6.18.2.6 saveBlend()

```
virtual bool DataManager::saveBlend (
    const std::string & blendID,
    const std::string & creatorID,
    const std::string & algorithm,
    const std::vector< std::string > & participantIDs) [pure virtual]
```

Persists a blend and its participant list. Re-calling with the same blendID replaces the existing record.

Parameters

<i>creatorID</i>	userID of the user who initiated the blend.
<i>participantIDs</i>	All participant userIDs (including those without data).

Implemented in [SqliteDataManager](#).

6.18.2.7 saveWatchLater()

```
virtual bool DataManager::saveWatchLater (
    const std::string & userID,
    const std::list< Video > & videos) [pure virtual]
```

Persists a user's Watch Later video list, replacing any previously saved list.

Returns

true on success.

Implemented in [SqliteDataManager](#).

6.18.2.8 validatePassword()

```
virtual bool DataManager::validatePassword (
    const std::string & userID,
    const std::string & password) [pure virtual]
```

Checks whether the given plaintext password matches the stored record.

Returns

true if the userID exists and the password matches.

Implemented in [SqliteDataManager](#).

The documentation for this class was generated from the following file:

- [DataManager.hpp](#)

6.19 DataSaver Class Reference

[DataSaver](#) class declaration.

```
#include <DataSaver.hpp>
```

Public Member Functions

- [DataSaver](#) ([FileBin](#) b, [FileFormatter](#) ff)
- int [save](#) (std::list< std::unordered_map< std::string, std::string > > data)
- void [setFileBin](#) ([FileBin](#) b)
- void [setFileFormatter](#) ([FileFormatter](#) ff)

Private Attributes

- std::vector< std::unique_ptr< [FileBin](#) > > [bins](#)
Registered file output handlers.
- std::vector< std::unique_ptr< [FileFormatter](#) > > [formatters](#)
Registered data formatters.

6.19.1 Detailed Description

[DataSaver](#) class declaration.

Definition at line 22 of file [DataSaver.hpp](#).

6.19.2 Constructor & Destructor Documentation

6.19.2.1 DataSaver()

```
DataSaver::DataSaver (
    FileBin b,
    FileFormatter ff)
```

Constructs a [DataSaver](#) with an initial [FileBin](#) and [FileFormatter](#).

Parameters

<i>b</i>	The FileBin implementation for file output.
<i>ff</i>	The FileFormatter implementation for data formatting.

6.19.3 Member Function Documentation

6.19.3.1 `save()`

```
int DataSaver::save (
    std::list< std::unordered_map< std::string, std::string > > data)
```

Formats and writes data to file using the configured formatters and bins.

Parameters

<i>data</i>	List of row dictionaries to save.
-------------	-----------------------------------

Returns

0 on success, non-zero on failure.

6.19.3.2 `setFileBin()`

```
void DataSaver::setFileBin (
    FileBin b)
```

Adds or replaces the [FileBin](#) used for file output.

Parameters

<i>b</i>	New FileBin implementation.
----------	---

6.19.3.3 `setFileFormatter()`

```
void DataSaver::setFileFormatter (
    FileFormatter ff)
```

Adds or replaces the [FileFormatter](#) used for data formatting.

Parameters

<i>ff</i>	New FileFormatter implementation.
-----------	---

6.19.4 Member Data Documentation

6.19.4.1 bins

```
std::vector<std::unique_ptr<FileBin> > DataSaver::bins [private]
```

Registered file output handlers.

Definition at line 53 of file [DataSaver.hpp](#).

6.19.4.2 formatters

```
std::vector<std::unique_ptr<FileFormatter> > DataSaver::formatters [private]
```

Registered data formatters.

Definition at line 54 of file [DataSaver.hpp](#).

The documentation for this class was generated from the following file:

- [DataSaver.hpp](#)

6.20 EventRouter Class Reference

[EventRouter](#) class declaration.

```
#include <EventRouter.hpp>
```

Public Member Functions

- void [registerListener](#) (const std::string &eventName, [EventHandler](#) handler)
- void [dispatch](#) (const std::string &eventName, const [EventPayload](#) &payload={})
- void [deregisterAll](#) ()

Private Attributes

- std::map< std::string, [EventHandler](#) > [listeners](#)

6.20.1 Detailed Description

[EventRouter](#) class declaration.

Lightweight event-dispatch system connecting the UI layer to [AppController](#).

Definition at line 40 of file [EventRouter.hpp](#).

6.20.2 Constructor & Destructor Documentation

6.20.2.1 EventRouter()

```
EventRouter::EventRouter ()
```

Definition at line 10 of file [EventRouter.cpp](#).

6.20.3 Member Function Documentation

6.20.3.1 deregisterAll()

```
void EventRouter::deregisterAll ()
```

Removes all registered listeners. Called by [AppController](#)'s destructor during teardown.

Definition at line 25 of file [EventRouter.cpp](#).

6.20.3.2 dispatch()

```
void EventRouter::dispatch (  
    const std::string & eventName,  
    const EventPayload & payload = {})
```

Dispatches a named event to its registered handler. Logs a warning and does nothing if the event name is unrecognised.

Parameters

<i>eventName</i>	Name of the event to dispatch.
<i>payload</i>	Key-value data passed to the handler.

Definition at line 16 of file [EventRouter.cpp](#).

6.20.3.3 registerListener()

```
void EventRouter::registerListener (  
    const std::string & eventName,  
    EventHandler handler)
```

Registers a handler for the given event name. Replaces any existing handler registered under the same name.

Parameters

<i>eventName</i>	Unique name for this event.
<i>handler</i>	Callable invoked when the event is dispatched.

Definition at line 12 of file [EventRouter.cpp](#).

6.20.4 Member Data Documentation

6.20.4.1 listeners

```
std::map<std::string, EventHandler> EventRouter::listeners [private]
```

Definition at line 42 of file [EventRouter.hpp](#).

The documentation for this class was generated from the following files:

- [EventRouter.hpp](#)
- [EventRouter.cpp](#)

6.21 File_ID Class Reference

[File_ID](#) class declaration.

```
#include <File_ID.hpp>
```

Static Public Attributes

- static constexpr int [CSV](#) = 1
- static constexpr int [HTML](#) = 2
- static constexpr int [JSON](#) = 3

6.21.1 Detailed Description

[File_ID](#) class declaration.

File format ID constants for validating source-parser compatibility.

Definition at line 16 of file [File_ID.hpp](#).

6.21.2 Member Data Documentation

6.21.2.1 CSV

```
int File_ID::CSV = 1 [static], [constexpr]
```

Definition at line 18 of file [File_ID.hpp](#).

6.21.2.2 HTML

```
int File_ID::HTML = 2 [static], [constexpr]
```

Definition at line 19 of file [File_ID.hpp](#).

6.21.2.3 JSON

```
int File_ID::JSON = 3 [static], [constexpr]
```

Definition at line 20 of file [File_ID.hpp](#).

The documentation for this class was generated from the following file:

- [File_ID.hpp](#)

6.22 FileBin Class Reference

[FileBin](#) class declaration.

```
#include <FileBin.hpp>
```

Public Member Functions

- virtual [~FileBin](#) ()=default
- virtual int [write](#) ()=0
- virtual void [setOutputFile](#) (std::string output)=0
- virtual void [setRawData](#) (std::string raw)=0
- virtual void [setFileName](#) (std::string fn)=0

6.22.1 Detailed Description

[FileBin](#) class declaration.

Abstract interface for writing data to files.

Definition at line 22 of file [FileBin.hpp](#).

6.22.2 Constructor & Destructor Documentation

6.22.2.1 ~FileBin()

```
virtual FileBin::~~FileBin () [virtual], [default]
```

Virtual destructor for proper cleanup in derived classes.

6.22.3 Member Function Documentation

6.22.3.1 setFileName()

```
virtual void FileBin::setFileName (  
    std::string fn) [pure virtual]
```

Sets the output file name (without directory path).

Parameters

<i>fn</i>	File name for output
-----------	----------------------

6.22.3.2 setOutputFile()

```
virtual void FileBin::setOutputFile (  
    std::string output) [pure virtual]
```

Sets the output file directory path for [write\(\)](#).

Parameters

<i>output</i>	Directory path where file will be written
---------------	---

6.22.3.3 setRawData()

```
virtual void FileBin::setRawData (  
    std::string raw) [pure virtual]
```

Sets the raw data string to be written to file.

Parameters

<i>raw</i>	Formatted data string to write
------------	--------------------------------

6.22.3.4 write()

```
virtual int FileBin::write () [pure virtual]
```

Writes configured data to the specified output file.

Creates or overwrites the file at the configured output path and writes the configured raw data to it.

Returns

Status code (0 on success, non-zero on failure)

The documentation for this class was generated from the following file:

- [FileBin.hpp](#)

6.23 FileFormatter Class Reference

[FileFormatter](#) class declaration.

```
#include <FileFormatter.hpp>
```

Public Member Functions

- virtual [~FileFormatter](#) ()=default
- virtual std::string [format](#) ()=0
- virtual void [setData](#) (std::list< std::unordered_map< std::string, std::string > > data)=0

6.23.1 Detailed Description

[FileFormatter](#) class declaration.

Definition at line 23 of file [FileFormatter.hpp](#).

6.23.2 Constructor & Destructor Documentation

6.23.2.1 ~FileFormatter()

```
virtual FileFormatter::~~FileFormatter () [virtual], [default]
```

Virtual destructor for proper cleanup in derived classes.

6.23.3 Member Function Documentation

6.23.3.1 format()

```
virtual std::string FileFormatter::format () [pure virtual]
```

Formats structured data into format-specific string representation.

Converts list<Dict<str,str>> configured via [setData\(\)](#) into a properly formatted string for writing to file (e.g., CSV with commas and newlines).

Returns

Formatted string ready for file output

6.23.3.2 setData()

```
virtual void FileFormatter::setData (
    std::list< std::unordered_map< std::string, std::string > > data) [pure virtual]
```

Sets the structured data to be formatted by [format\(\)](#).

Parameters

<i>data</i>	List of dictionaries representing data rows
-------------	---

The documentation for this class was generated from the following file:

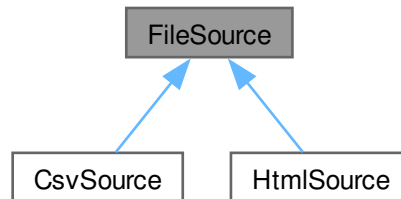
- [FileFormatter.hpp](#)

6.24 FileSource Class Reference

[FileSource](#) class declaration.

```
#include <FileSource.hpp>
```

Inheritance diagram for FileSource:



Public Member Functions

- virtual `std::list< std::string > read ()=0`
- virtual void `setSource (std::string src)=0`
- virtual int `getSourceId ()=0`

6.24.1 Detailed Description

[FileSource](#) class declaration.

Abstract interface for reading file source data.

Definition at line 22 of file [FileSource.hpp](#).

6.24.2 Member Function Documentation

6.24.2.1 getSourceId()

```
virtual int FileSource::getSourceId () [pure virtual]
```

Gets the file format ID associated with this source.

Returns

Integer ID representing the file format (e.g., `File_ID::CSV`)

Implemented in [CsvSource](#), and [HtmlSource](#).

6.24.2.2 read()

```
virtual std::list< std::string > FileSource::read () [pure virtual]
```

Reads the configured file source path and returns raw file data.

Returns

List of strings containing raw file data (line-by-line or complete content)

Implemented in [CsvSource](#), and [HtmlSource](#).

6.24.2.3 setSource()

```
virtual void FileSource::setSource (
    std::string src) [pure virtual]
```

Sets the file source path for [read\(\)](#) to use.

Parameters

<i>src</i>	File path to configure as the source for reading
------------	--

Implemented in [CsvSource](#).

The documentation for this class was generated from the following file:

- [FileSource.hpp](#)

6.25 Friend Class Reference

[Friend](#) class declaration.

```
#include <Friend.hpp>
```

Public Member Functions

- [Friend](#) (const std::string &userID, const std::string &displayName, const std::string &email)
- std::string [getUserID](#) () const
- std::string [getDisplayName](#) () const
- std::string [getEmail](#) () const
- void [setDisplayName](#) (const std::string &displayName)
- void [setEmail](#) (const std::string &email)
- std::string [toString](#) () const

Private Attributes

- `std::string` [userID](#)
- `std::string` [displayName](#)
- `std::string` [email](#)

6.25.1 Detailed Description

[Friend](#) class declaration.

A friend or contact entry stored on a [User](#)'s friends list.

Definition at line 17 of file [Friend.hpp](#).

6.25.2 Constructor & Destructor Documentation

6.25.2.1 Friend()

```
Friend::Friend (
    const std::string & userID,
    const std::string & displayName,
    const std::string & email)
```

Constructs a [Friend](#) entry.

Parameters

<i>userID</i>	Unique identifier of the friend's account.
<i>displayName</i>	Display name shown in the UI.
<i>email</i>	Email address of the friend.

Definition at line 11 of file [Friend.cpp](#).

6.25.3 Member Function Documentation

6.25.3.1 getDisplayName()

```
std::string Friend::getDisplayName () const
```

Returns

Display name of the friend.

Definition at line 22 of file [Friend.cpp](#).

6.25.3.2 getEmail()

```
std::string Friend::getEmail () const
```

Returns

Email address of the friend.

Definition at line 26 of file [Friend.cpp](#).

6.25.3.3 getUserID()

```
std::string Friend::getUserID () const
```

Returns

Unique identifier of the friend's account.

Definition at line 18 of file [Friend.cpp](#).

6.25.3.4 setDisplayName()

```
void Friend::setDisplayName (  
    const std::string & displayName)
```

Parameters

<i>displayName</i>	New display name.
--------------------	-------------------

Definition at line 32 of file [Friend.cpp](#).

6.25.3.5 setEmail()

```
void Friend::setEmail (  
    const std::string & email)
```

Parameters

<i>email</i>	New email address.
--------------	--------------------

Definition at line 36 of file [Friend.cpp](#).

6.25.3.6 toString()

```
std::string Friend::toString () const
```

String representation of [Friend](#)

Returns

string

Definition at line 40 of file [Friend.cpp](#).

6.25.4 Member Data Documentation

6.25.4.1 displayName

```
std::string Friend::displayName [private]
```

Definition at line 20 of file [Friend.hpp](#).

6.25.4.2 email

```
std::string Friend::email [private]
```

Definition at line 21 of file [Friend.hpp](#).

6.25.4.3 userID

```
std::string Friend::userID [private]
```

Definition at line 19 of file [Friend.hpp](#).

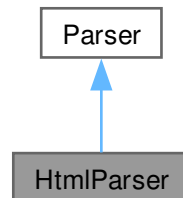
The documentation for this class was generated from the following files:

- [Friend.hpp](#)
- [Friend.cpp](#)

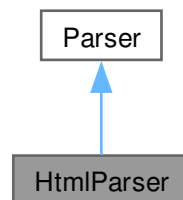
6.26 HtmlParser Class Reference

HTML file parser implementation using Gumbo.

Inheritance diagram for HtmlParser:



Collaboration diagram for HtmlParser:



Public Member Functions

- [HtmlParser](#) (list< string > raw_data)
- list< unordered_map< string, string > > [parse](#) ()
- void [setData](#) (list< string > data) override
- int [getParseId](#) () override

Public Member Functions inherited from [Parser](#)

- virtual void [setData](#) (std::list< std::string > data)=0

Private Member Functions

- void [extract_links](#) (GumboNode *node, list< unordered_map< string, string > > &out)
- string [get_text](#) (GumboNode *node)

Private Attributes

- list< string > [raw_data](#) = {}
- int const [parse_id](#) = File_ID::HTML

6.26.1 Detailed Description

HTML file parser implementation using Gumbo.

Definition at line 16 of file [HtmlParser.cpp](#).

6.26.2 Constructor & Destructor Documentation

6.26.2.1 HtmlParser() [1/2]

```
HtmlParser::HtmlParser (  
    list< string > raw_data) [inline]
```

Definition at line 18 of file [HtmlParser.cpp](#).

6.26.2.2 HtmlParser() [2/2]

```
HtmlParser::HtmlParser () [inline]
```

Definition at line 19 of file [HtmlParser.cpp](#).

6.26.3 Member Function Documentation

6.26.3.1 extract_links()

```
void HtmlParser::extract_links (  
    GumboNode * node,  
    list< unordered_map< string, string > > & out) [inline], [private]
```

Definition at line 56 of file [HtmlParser.cpp](#).

6.26.3.2 get_text()

```
string HtmlParser::get_text (  
    GumboNode * node) [inline], [private]
```

Definition at line 86 of file [HtmlParser.cpp](#).

6.26.3.3 getParseId()

```
int HtmlParser::getParseId () [inline], [override], [virtual]
```

Gets the file format ID associated with this parser.

Returns

Integer ID representing the file format (e.g., File_ID::CSV)

Implements [Parser](#).

Definition at line 48 of file [HtmlParser.cpp](#).

6.26.3.4 parse()

```
list< unordered_map< string, string > > HtmlParser::parse () [inline], [virtual]
```

Parses raw file data into a structured list of dictionaries.

Converts raw file data configured via setData() into a list of unordered string-to-string dictionaries. Each dictionary represents a row with column titles as keys and row contents as values.

Returns

List of unordered maps representing parsed file rows

Implements [Parser](#).

Definition at line 22 of file [HtmlParser.cpp](#).

6.26.3.5 setData()

```
void HtmlParser::setData (
    list< string > data) [inline], [override]
```

Definition at line 46 of file [HtmlParser.cpp](#).

6.26.4 Member Data Documentation

6.26.4.1 parse_id

```
int const HtmlParser::parse_id = File_ID::HTML [private]
```

Definition at line 53 of file [HtmlParser.cpp](#).

6.26.4.2 raw_data

```
list<string> HtmlParser::raw_data = {} [private]
```

Definition at line 52 of file [HtmlParser.cpp](#).

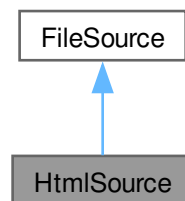
The documentation for this class was generated from the following file:

- [HtmlParser.cpp](#)

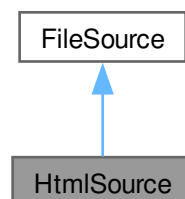
6.27 HtmlSource Class Reference

HTML file source/reader implementation.

Inheritance diagram for HtmlSource:



Collaboration diagram for HtmlSource:



Public Member Functions

- [HtmlSource](#) (std::string s)
- list< string > [read](#) () override
- int [getSourceId](#) ()
- void [setSource](#) (string new_source)

Public Member Functions inherited from [FileSource](#)

- virtual void [setSource](#) (std::string src)=0

Private Attributes

- string [src_path](#) = ""
- int const [source_id](#) = File_ID::HTML

6.27.1 Detailed Description

HTML file source/reader implementation.

Definition at line 16 of file [HtmlSource.cpp](#).

6.27.2 Constructor & Destructor Documentation

6.27.2.1 [HtmlSource\(\)](#)

```
HtmlSource::HtmlSource (
    std::string s) [inline]
```

Definition at line 18 of file [HtmlSource.cpp](#).

6.27.3 Member Function Documentation

6.27.3.1 [getSourceId\(\)](#)

```
int HtmlSource::getSourceId () [inline], [virtual]
```

Gets the file format ID associated with this source.

Returns

Integer ID representing the file format (e.g., File_ID::CSV)

Implements [FileSource](#).

Definition at line 42 of file [HtmlSource.cpp](#).

6.27.3.2 [read\(\)](#)

```
list< string > HtmlSource::read () [inline], [override], [virtual]
```

Reads the configured file source path and returns raw file data.

Returns

List of strings containing raw file data (line-by-line or complete content)

Implements [FileSource](#).

Definition at line 21 of file [HtmlSource.cpp](#).

6.27.3.3 setSource()

```
void HtmlSource::setSource (
    string new_source) [inline]
```

Definition at line 43 of file [HtmlSource.cpp](#).

6.27.4 Member Data Documentation

6.27.4.1 source_id

```
int const HtmlSource::source_id = File_ID::HTML [private]
```

Definition at line 47 of file [HtmlSource.cpp](#).

6.27.4.2 src_path

```
string HtmlSource::src_path = " " [private]
```

Definition at line 46 of file [HtmlSource.cpp](#).

The documentation for this class was generated from the following file:

- [HtmlSource.cpp](#)

6.28 HttpClient Class Reference

[HttpClient](#) class declaration.

```
#include <HttpClient.hpp>
```

Public Member Functions

- [HttpClient](#) (std::string base)
- long [getLastStatusCode](#) () const
Returns the HTTP status code from the most recent successful libcurl transfer initiated by this client instance.
- bool [wasLastRequestSuccessful](#) (const std::string &method) const
Convenience helper that applies [isRequestSuccessful\(\)](#) to the latest response code captured by this client.
- std::string [get](#) (const std::string &path)
- std::string [post](#) (const std::string &path, const std::string &json)
- std::string [del](#) (const std::string &path, const std::string &json)
- std::string [build_watch_later_endpoint](#) (std::string userID)
Builds the watch-later endpoint for a specific user.
- std::string [build_latest_blend_endpoint](#) (std::string userID)
Builds the endpoint used to fetch a user's latest blend.
- std::string [build_blend_participant_endpoint](#) (std::string blendID)
Builds the endpoint used to fetch participant IDs for a blend.
- std::string [build_chatroom_detail_endpoint](#) (std::string blendID)
Builds the endpoint used to fetch chat room details for a blend.
- std::string [build_auth_user_lookup_endpoint](#) (std::string userID)
Builds the auth user lookup endpoint for a specific user.
- std::string [build_delete_user_endpoint](#) (std::string userID)
Builds the account-delete endpoint for a specific user.
- std::string [build_user_blends_endpoint](#) (std::string userID)
- std::string [build_leave_blend_endpoint](#) (std::string blendID)
- std::string [build_chat_history_endpoint](#) (std::string blendID, std::string userID)

Static Public Member Functions

- static bool [isRequestSuccessful](#) (const std::string &method, long statusCode)
Evaluates whether an HTTP status code should be treated as a successful result for a specific request method.

Public Attributes

- const std::string [P](#) = "POST"
- const std::string [G](#) = "GET"
- const std::string [D](#) = "DELETE"
- const std::string [REG_USER](#) = "/api/v1/auth/register"
- const std::string [LOGIN](#) = "/api/v1/auth/login"
- const std::string [AUTH_USER_PREFIX](#) = "/api/v1/auth/users/"
- const std::string [DELETE_USER_PREFIX](#) = "/api/v1/auth/users/"
- const std::string [BLEND](#) = "/api/v1/blends"
- const int [SUCC](#) = 200
- const int [ALT_SUCC](#) = 201
- const int [MAL_REQ_ERR](#) = 400
- const int [INV_USR_LOG_ERR](#) = 401
- const int [UNAUTH_RES_REQ_ERR](#) = 403
- const int [MISS_RM_CNTXT](#) = 404
- const int [DUPLICATE](#) = 409
- const int [SERVER_ERR](#) = 500

Private Member Functions

- std::string [request](#) (const std::string &method, const std::string &path, const std::string &body)

Static Private Member Functions

- static size_t [write_callback](#) (void *contents, size_t size, size_t nmemb, void *out)

Private Attributes

- std::string [baseUrl](#)
- const std::string [hdr](#) = "Content-Type: application/json"
- long [lastStatusCode](#) = 0

6.28.1 Detailed Description

[HttpClient](#) class declaration.

Definition at line 22 of file [HttpClient.hpp](#).

6.28.2 Constructor & Destructor Documentation

6.28.2.1 HttpClient()

```
HttpClient::HttpClient (
    std::string base)
```

Basic constructor. Assigns base url to this instance

Parameters

<i>base</i>	string that will be saved to this instance
-------------	--

Definition at line 49 of file [HttpClient.cpp](#).

6.28.3 Member Function Documentation

6.28.3.1 build_auth_user_lookup_endpoint()

```
string HttpClient::build_auth_user_lookup_endpoint (  
    std::string userID)
```

Builds the auth user lookup endpoint for a specific user.

Result format: `/api/v1/auth/users/{userID}`

This endpoint is the canonical account-existence lookup in the frontend/backend contract and should be used instead of feature routes like watch-later when validating participant IDs.

Parameters

<i>userID</i>	User identifier used in the endpoint path.
---------------	--

Returns

Endpoint path string for user profile lookup.

Definition at line 103 of file [HttpClient.cpp](#).

6.28.3.2 build_blend_participant_endpoint()

```
string HttpClient::build_blend_participant_endpoint (  
    std::string blendID)
```

Builds the endpoint used to fetch participant IDs for a blend.

Result format: `/api/v1/blends/{blendID}/participants`

Parameters

<i>blendID</i>	Blend identifier used in the endpoint path.
----------------	---

Returns

Endpoint path string for participant lookup.

Definition at line 95 of file [HttpClient.cpp](#).

6.28.3.3 build_chat_history_endpoint()

```
string HttpClient::build_chat_history_endpoint (  
    std::string blendID,  
    std::string userID)
```

Builds the endpoint for fetching a blend's chat message history.

Parameters

<i>blendID</i>	ID of the blend whose chat history is requested.
<i>userID</i>	ID of the requesting user (passed as query param).

Returns

the structured string endpoint

Definition at line 192 of file [HttpClient.cpp](#).

6.28.3.4 build_chatroom_detail_endpoint()

```
string HttpClient::build_chatroom_detail_endpoint (  
    std::string blendID)
```

Builds the endpoint used to fetch chat room details for a blend.

Result format: /api/v1/blends/{blendID}/chatroom

Parameters

<i>blendID</i>	Blend identifier used as the chatroom context.
----------------	--

Returns

Endpoint path string for chatroom details.

Definition at line 99 of file [HttpClient.cpp](#).

6.28.3.5 build_delete_user_endpoint()

```
string HttpClient::build_delete_user_endpoint (  
    std::string userID)
```

Builds the account-delete endpoint for a specific user.

Result format: /api/v1/auth/users/{userID}

Parameters

<i>userID</i>	User identifier used in the endpoint path.
---------------	--

Returns

Endpoint path string for account deletion.

Definition at line 107 of file [HttpClient.cpp](#).

6.28.3.6 build_latest_blend_endpoint()

```
string HttpClient::build_latest_blend_endpoint (  
    std::string userID)
```

Builds the endpoint used to fetch a user's latest blend.

Result format: /api/v1/users/{userID}/blend

Parameters

<i>userID</i>	User identifier used in the endpoint path.
---------------	--

Returns

Endpoint path string for latest blend lookup.

Definition at line 91 of file [HttpClient.cpp](#).

6.28.3.7 build_leave_blend_endpoint()

```
string HttpClient::build_leave_blend_endpoint (  
    std::string blendID)
```

Builds the endpoint for leaving a blend. Result format: `/api/v1/blends/{blendID}/leave`

Parameters

<i>blendID</i>	Blend identifier.
----------------	-------------------

Returns

Endpoint path string for leaving a blend.

Definition at line 188 of file [HttpClient.cpp](#).

6.28.3.8 build_user_blends_endpoint()

```
string HttpClient::build_user_blends_endpoint (  
    std::string userID)
```

Builds the endpoint for fetching all blends a user participates in. Result format: `/api/v1/users/{userID}/blends`

Parameters

<i>userID</i>	User identifier.
---------------	------------------

Returns

Endpoint path string for listing user blends.

Definition at line 184 of file [HttpClient.cpp](#).

6.28.3.9 build_watch_later_endpoint()

```
string HttpClient::build_watch_later_endpoint (  
    std::string userID)
```

Builds the watch-later endpoint for a specific user.

Result format: `/api/v1/users/{userID}/watch-later`

Parameters

<i>userID</i>	User identifier used in the endpoint path.
---------------	--

Returns

Endpoint path string for persisting or retrieving watch-later data.

Definition at line 87 of file [HttpClient.cpp](#).

6.28.3.10 del()

```
string HttpClient::del (  
    const std::string & path,  
    const std::string & json)
```

Public facing delete method. Fires HTTP DELETE message to the server.

Parameters

<i>path</i>	endpoint string pointer
<i>json</i>	formatted json string pointer message to be sent

Returns

server response body

Definition at line 83 of file [HttpClient.cpp](#).

6.28.3.11 get()

```
string HttpClient::get (  
    const std::string & path)
```

Public facing get method. Acquires the formatted string based on the given path endpoint. Helps simplify pinging the server for data by providing the

Parameters

<i>path</i>	string endpoint path pointer the formatted string is for
-------------	--

Returns

formatted json string

Definition at line 75 of file [HttpClient.cpp](#).

6.28.3.12 getLastStatusCode()

```
long HttpClient::getLastStatusCode () const
```

Returns the HTTP status code from the most recent successful libcurl transfer initiated by this client instance.

This value is updated by [request\(\)](#) after `curl_easy_perform()` and is available to callers who need post-call diagnostics.

Returns

Last response status code, or 0 before any request has run.

Definition at line 67 of file [HttpClient.cpp](#).

6.28.3.13 isRequestSuccessful()

```
bool HttpClient::isRequestSuccessful (
    const std::string & method,
    long statusCode) [static]
```

Evaluates whether an HTTP status code should be treated as a successful result for a specific request method.

This is the central error-code handler for [HttpClient](#). Use it whenever you need to decide if a completed request succeeded semantically, not just transport-wise.

Success rules are intentionally aligned with the frontend/backend contract in [Docs/Front_End_Requirements_for_Server_Communication.md](#):

- GET is successful only for 200 OK.
- POST is successful for 200 OK or 201 Created.
- All other status codes are considered failures.

Parameters

<i>method</i>	HTTP verb string ("GET" or "POST").
<i>statusCode</i>	Numeric HTTP status code returned by the server.

Returns

`true` when the code represents success for the given method; `false` otherwise.

Exceptions

<i>std::invalid_argument</i>	if the method is unsupported.
------------------------------	-------------------------------

Definition at line 53 of file [HttpClient.cpp](#).

6.28.3.14 post()

```
string HttpClient::post (
    const std::string & path,
    const std::string & json)
```

Public facing post method. fires HTTP message to the server.

Parameters

<i>path</i>	the endpoint string pointer
<i>json</i>	formatted json string pointer message to be sent

Returns

Definition at line 79 of file [HttpClient.cpp](#).

6.28.3.15 request()

```
string HttpClient::request (  
    const std::string & method,  
    const std::string & path,  
    const std::string & body) [private]
```

Helper function. Does the grunt work of retrieving and posting data to the server

Parameters

<i>method</i>	Determines whether or not we POST or GET
<i>path</i>	determine the endpoint we need to use
<i>body</i>	the JSON body that needs to be POSTed. leave empty when GETting

Returns

the response from either operation

just have this here for completeness libcurl will by default request through GET. but with the above cmd, libcurl will switch to POST automatically

within the future can add extra blocks for other request methods

Definition at line 111 of file [HttpClient.cpp](#).

6.28.3.16 wasLastRequestSuccessful()

```
bool HttpClient::wasLastRequestSuccessful (  
    const std::string & method) const
```

Convenience helper that applies [isRequestSuccessful\(\)](#) to the latest response code captured by this client.

Parameters

<i>method</i>	HTTP verb to evaluate ("GET" or "POST").
---------------	--

Returns

true when the latest response code is successful for method.

Exceptions

<code>std::invalid_argument</code>	if the method is unsupported.
------------------------------------	-------------------------------

Definition at line 71 of file [HttpClient.cpp](#).

6.28.3.17 write_callback()

```
size_t HttpClient::write_callback (  
    void * contents,  
    size_t size,  
    size_t nmemb,  
    void * out) [static], [private]
```

Part of the libcurl design pattern. Used to determine how streamed data should be handled from the request response. (libcurl does not handle how to deal with streamed data)

Parameters

<i>contents</i>	Is the pointer to a chunk of raw bytes from the server
<i>size</i>	Describes how long the chunk is
<i>nmemb</i>	Aids in describing the size of the chunk (like typical malloc patterns)
<i>out</i>	A pointer to a string we would like to append the chunks to (in this case)

Returns

(size * nmemb) as part of the libcurl pattern to verify data handling from callback

Definition at line 196 of file [HttpClient.cpp](#).

6.28.4 Member Data Documentation

6.28.4.1 ALT_SUCC

```
const int HttpClient::ALT_SUCC = 201
```

Definition at line 47 of file [HttpClient.hpp](#).

6.28.4.2 AUTH_USER_PREFIX

```
const std::string HttpClient::AUTH_USER_PREFIX = "/api/v1/auth/users/"
```

Definition at line 38 of file [HttpClient.hpp](#).

6.28.4.3 baseUrl

```
std::string HttpClient::baseUrl [private]
```

Definition at line 220 of file [HttpClient.hpp](#).

6.28.4.4 BLEND

```
const std::string HttpClient::BLEND = "/api/v1/blends"
```

Definition at line 42 of file [HttpClient.hpp](#).

6.28.4.5 D

```
const std::string HttpClient::D = "DELETE"
```

Definition at line 29 of file [HttpClient.hpp](#).

6.28.4.6 DELETE_USER_PREFIX

```
const std::string HttpClient::DELETE_USER_PREFIX = "/api/v1/auth/users/"
```

Definition at line 40 of file [HttpClient.hpp](#).

6.28.4.7 DUPLICATE

```
const int HttpClient::DUPLICATE = 409
```

Definition at line 52 of file [HttpClient.hpp](#).

6.28.4.8 G

```
const std::string HttpClient::G = "GET"
```

Definition at line 27 of file [HttpClient.hpp](#).

6.28.4.9 hdr

```
const std::string HttpClient::hdr = "Content-Type: application/json" [private]
```

Definition at line 223 of file [HttpClient.hpp](#).

6.28.4.10 INV_USR_LOG_ERR

```
const int HttpClient::INV_USR_LOG_ERR = 401
```

Definition at line 49 of file [HttpClient.hpp](#).

6.28.4.11 lastStatusCode

```
long HttpClient::lastStatusCode = 0 [private]
```

Definition at line 226 of file [HttpClient.hpp](#).

6.28.4.12 LOGIN

```
const std::string HttpClient::LOGIN = "/api/v1/auth/login"
```

Definition at line 36 of file [HttpClient.hpp](#).

6.28.4.13 MAL_REQ_ERR

```
const int HttpClient::MAL_REQ_ERR = 400
```

Definition at line 48 of file [HttpClient.hpp](#).

6.28.4.14 MISS_RM_CNTXT

```
const int HttpClient::MISS_RM_CNTXT = 404
```

Definition at line 51 of file [HttpClient.hpp](#).

6.28.4.15 P

```
const std::string HttpClient::P = "POST"
```

Definition at line 25 of file [HttpClient.hpp](#).

6.28.4.16 REG_USER

```
const std::string HttpClient::REG_USER = "/api/v1/auth/register"
```

Definition at line 34 of file [HttpClient.hpp](#).

6.28.4.17 SERVER_ERR

```
const int HttpClient::SERVER_ERR = 500
```

Definition at line 53 of file [HttpClient.hpp](#).

6.28.4.18 SUCC

```
const int HttpClient::SUCC = 200
```

Definition at line 46 of file [HttpClient.hpp](#).

6.28.4.19 UNAUTH_RES_REQ_ERR

```
const int HttpClient::UNAUTH_RES_REQ_ERR = 403
```

Definition at line 50 of file [HttpClient.hpp](#).

The documentation for this class was generated from the following files:

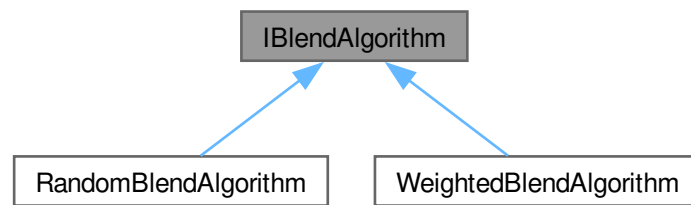
- [HttpClient.hpp](#)
- [HttpClient.cpp](#)

6.29 IBlendAlgorithm Class Reference

[IBlendAlgorithm](#) class declaration.

```
#include <IBlendAlgorithm.hpp>
```

Inheritance diagram for IBlendAlgorithm:



Public Member Functions

- virtual [Blend](#) [generateBlend](#) (const std::list< [User](#) > &participants, const std::string &title="")=0

6.29.1 Detailed Description

[IBlendAlgorithm](#) class declaration.

Abstract interface for blend-generation algorithms.

Definition at line 23 of file [IBlendAlgorithm.hpp](#).

6.29.2 Member Function Documentation

6.29.2.1 generateBlend()

```
virtual Blend IBlendAlgorithm::generateBlend (  
    const std::list< User > & participants,  
    const std::string & title = "") [pure virtual]
```

Generates a [Blend](#) from a collection of participants.

Parameters

<i>participants</i>	List of users whose Watch Later data is used as input.
<i>title</i>	User-facing name for the blend (default: empty).

Returns

A new [Blend](#) containing the algorithm's chosen video selection.

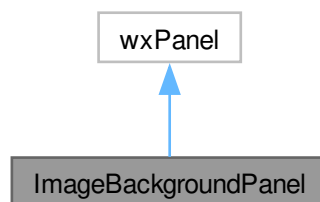
Implemented in [RandomBlendAlgorithm](#), and [WeightedBlendAlgorithm](#).

The documentation for this class was generated from the following file:

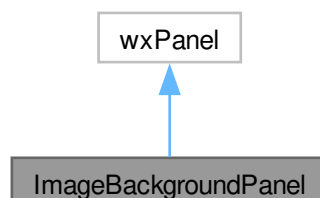
- [IBlendAlgorithm.hpp](#)

6.30 ImageBackgroundPanel Class Reference

Inheritance diagram for ImageBackgroundPanel:



Collaboration diagram for ImageBackgroundPanel:



Public Member Functions

- [ImageBackgroundPanel](#) (wxWindow *parent, const std::unordered_map< int, wxImage > &imageStore, int key)

Private Member Functions

- void [OnPaint](#) (wxPaintEvent &)

Private Attributes

- const std::unordered_map< int, wxImage > & [m_imageStore](#)
- int [m_key](#)

6.30.1 Detailed Description

Definition at line 62 of file [MainFrame.cpp](#).

6.30.2 Constructor & Destructor Documentation

6.30.2.1 ImageBackgroundPanel()

```
ImageBackgroundPanel::ImageBackgroundPanel (  
    wxWindow * parent,  
    const std::unordered_map< int, wxImage > & imageStore,  
    int key) [inline]
```

Definition at line 64 of file [MainFrame.cpp](#).

6.30.3 Member Function Documentation

6.30.3.1 OnPaint()

```
void ImageBackgroundPanel::OnPaint (  
    wxPaintEvent & ) [inline], [private]
```

Definition at line 79 of file [MainFrame.cpp](#).

6.30.4 Member Data Documentation

6.30.4.1 m_imageStore

```
const std::unordered_map<int, wxImage>& ImageBackgroundPanel::m_imageStore [private]
```

Definition at line 76 of file [MainFrame.cpp](#).

6.30.4.2 m_key

```
int ImageBackgroundPanel::m_key [private]
```

Definition at line 77 of file [MainFrame.cpp](#).

The documentation for this class was generated from the following file:

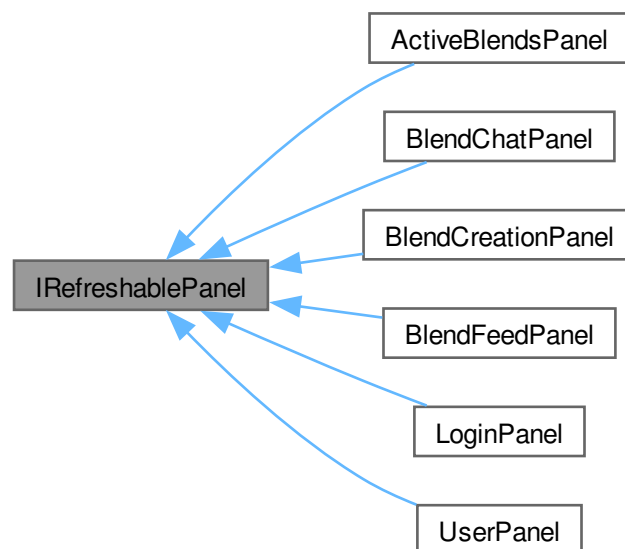
- [MainFrame.cpp](#)

6.31 IRefreshablePanel Class Reference

Interface for panels that support refresh operations.

```
#include <IRefreshablePanel.hpp>
```

Inheritance diagram for IRefreshablePanel:



Public Member Functions

- virtual void [Refresh](#) ()=0

6.31.1 Detailed Description

Interface for panels that support refresh operations.

[IRefreshablePanel](#) class declaration.

Definition at line 21 of file [IRefreshablePanel.hpp](#).

6.31.2 Member Function Documentation

6.31.2.1 Refresh()

```
virtual void IRefreshablePanel::Refresh () [pure virtual]
```

Refreshes the panel's display to reflect current application state. Called by [MainFrame](#) whenever the panel becomes active.

Implemented in [ActiveBlendsPanel](#), [BlendChatPanel](#), [BlendCreationPanel](#), [BlendFeedPanel](#), [LoginPanel](#), and [UserPanel](#).

The documentation for this class was generated from the following file:

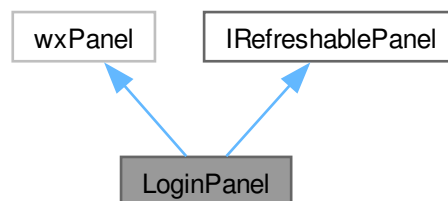
- [IRefreshablePanel.hpp](#)

6.32 LoginPanel Class Reference

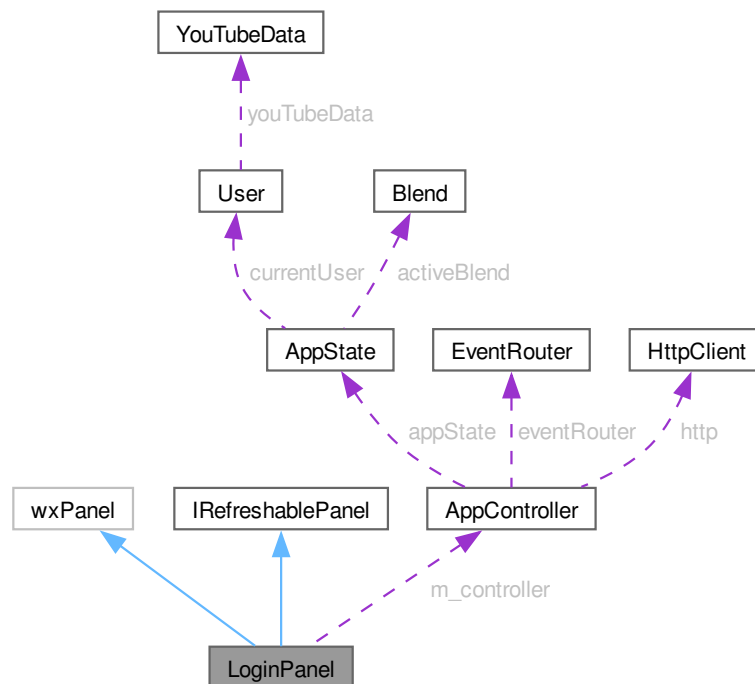
[LoginPanel](#) class declaration.

```
#include <LoginPanel.hpp>
```

Inheritance diagram for LoginPanel:



Collaboration diagram for LoginPanel:



Public Member Functions

- `LoginPanel` (`wxWindow *parent`, `AppController &controller`, `NavigateFn nav`, `const wxImage &bglImage=wxNullImage`)
- `void Refresh ()` override
- `void Reset ()`

Private Member Functions

- `void BuildSignInForm` (`wxWindow *parent`, `wxSizer *sizer`)
- `void BuildRegisterForm` (`wxWindow *parent`, `wxSizer *sizer`)
- `void ShowTab` (`int index`)
- `void OnSignIn` (`wxCommandEvent &`)
- `void OnRegister` (`wxCommandEvent &`)
- `void OnPaint` (`wxPaintEvent &`)
- `void ProceedAfterLogin` ()

Private Attributes

- `AppController & m_controller`
- `NavigateFn m_nav`
- `wxButton * m_signinTab`
- `wxButton * m_registerTab`

- wxSimplebook * [m_formBook](#)
- wxTextCtrl * [m_siUsername](#)
- wxTextCtrl * [m_siPassword](#)
- wxStaticText * [m_siError](#)
- wxTextCtrl * [m_regUsername](#)
- wxTextCtrl * [m_regEmail](#)
- wxTextCtrl * [m_regPassword](#)
- wxTextCtrl * [m_regConfirm](#)
- wxStaticText * [m_regError](#)
- wxImage [m_bgImage](#)

6.32.1 Detailed Description

[LoginPanel](#) class declaration.

Definition at line 52 of file [LoginPanel.hpp](#).

6.32.2 Constructor & Destructor Documentation

6.32.2.1 LoginPanel()

```
LoginPanel::LoginPanel (  
    wxWindow * parent,  
    AppController & controller,  
    NavigateFn nav,  
    const wxImage & bgImage = wxNullImage)
```

Definition at line 53 of file [LoginPanel.cpp](#).

6.32.3 Member Function Documentation

6.32.3.1 BuildRegisterForm()

```
void LoginPanel::BuildRegisterForm (  
    wxWindow * parent,  
    wxSizer * sizer) [private]
```

Definition at line 180 of file [LoginPanel.cpp](#).

6.32.3.2 BuildSignInForm()

```
void LoginPanel::BuildSignInForm (  
    wxWindow * parent,  
    wxSizer * sizer) [private]
```

Definition at line 157 of file [LoginPanel.cpp](#).

6.32.3.3 OnPaint()

```
void LoginPanel::OnPaint (
    wxPaintEvent & ) [private]
```

Definition at line 358 of file [LoginPanel.cpp](#).

6.32.3.4 OnRegister()

```
void LoginPanel::OnRegister (
    wxCommandEvent & ) [private]
```

Definition at line 272 of file [LoginPanel.cpp](#).

6.32.3.5 OnSignIn()

```
void LoginPanel::OnSignIn (
    wxCommandEvent & ) [private]
```

Definition at line 230 of file [LoginPanel.cpp](#).

6.32.3.6 ProceedAfterLogin()

```
void LoginPanel::ProceedAfterLogin () [private]
```

Definition at line 310 of file [LoginPanel.cpp](#).

6.32.3.7 Refresh()

```
void LoginPanel::Refresh () [override], [virtual]
```

Refreshes the panel's display to reflect current application state. Called by [MainFrame](#) whenever the panel becomes active.

Implements [IRefreshablePanel](#).

Definition at line 338 of file [LoginPanel.cpp](#).

6.32.3.8 Reset()

```
void LoginPanel::Reset ()
```

Definition at line 343 of file [LoginPanel.cpp](#).

6.32.3.9 ShowTab()

```
void LoginPanel::ShowTab (
    int index) [private]
```

Definition at line 206 of file [LoginPanel.cpp](#).

6.32.4 Member Data Documentation

6.32.4.1 m_bgImage

```
wxImage LoginPanel::m_bgImage [private]
```

Definition at line 85 of file [LoginPanel.hpp](#).

6.32.4.2 m_controller

```
AppController& LoginPanel::m_controller [private]
```

Definition at line 63 of file [LoginPanel.hpp](#).

6.32.4.3 m_formBook

```
wxSimplebook* LoginPanel::m_formBook [private]
```

Definition at line 71 of file [LoginPanel.hpp](#).

6.32.4.4 m_nav

```
NavigateFn LoginPanel::m_nav [private]
```

Definition at line 64 of file [LoginPanel.hpp](#).

6.32.4.5 m_regConfirm

```
wxTextCtrl* LoginPanel::m_regConfirm [private]
```

Definition at line 82 of file [LoginPanel.hpp](#).

6.32.4.6 m_regEmail

```
wxTextCtrl* LoginPanel::m_regEmail [private]
```

Definition at line 80 of file [LoginPanel.hpp](#).

6.32.4.7 m_regError

```
wxStaticText* LoginPanel::m_regError [private]
```

Definition at line 83 of file [LoginPanel.hpp](#).

6.32.4.8 m_registerTab

```
wxButton* LoginPanel::m_registerTab [private]
```

Definition at line 68 of file [LoginPanel.hpp](#).

6.32.4.9 m_regPassword

```
wxTextCtrl* LoginPanel::m_regPassword [private]
```

Definition at line 81 of file [LoginPanel.hpp](#).

6.32.4.10 m_regUsername

```
wxTextCtrl* LoginPanel::m_regUsername [private]
```

Definition at line 79 of file [LoginPanel.hpp](#).

6.32.4.11 m_siError

```
wxStaticText* LoginPanel::m_siError [private]
```

Definition at line 76 of file [LoginPanel.hpp](#).

6.32.4.12 m_signinTab

```
wxButton* LoginPanel::m_signinTab [private]
```

Definition at line 67 of file [LoginPanel.hpp](#).

6.32.4.13 m_siPassword

```
wxTextCtrl* LoginPanel::m_siPassword [private]
```

Definition at line 75 of file [LoginPanel.hpp](#).

6.32.4.14 m_siUsername

```
wxTextCtrl* LoginPanel::m_siUsername [private]
```

Definition at line 74 of file [LoginPanel.hpp](#).

The documentation for this class was generated from the following files:

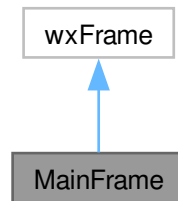
- [LoginPanel.hpp](#)
- [LoginPanel.cpp](#)

6.33 MainFrame Class Reference

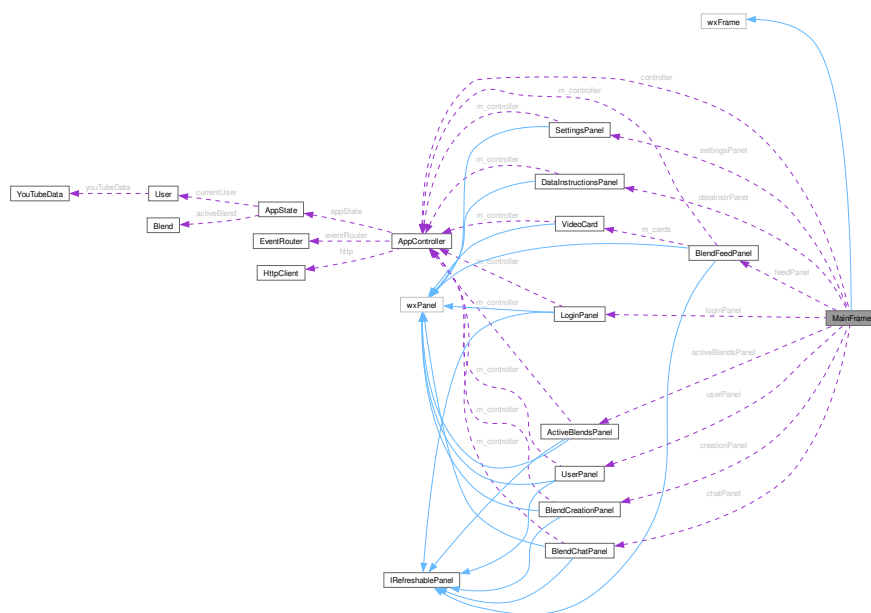
Root application window containing the full-window page switcher.

```
#include <MainFrame.hpp>
```

Inheritance diagram for MainFrame:



Collaboration diagram for MainFrame:



Public Member Functions

- [MainFrame](#) ([AppController](#) &controller)
- void [NavigateTo](#) ([Page](#) page)
- void [TriggerFeedRefresh](#) ()

Private Types

- enum **ImageKey** { **BG_MAIN** = 0 }

Private Member Functions

- wxPanel * **BuildHomePage** (wxWindow *parent)
- wxPanel * **BuildSettingsPage** (wxWindow *parent)
- void **OnSettings** (wxCommandEvent &)
Navigates to SETTINGS.
- void **OnUser** (wxCommandEvent &)
Navigates to USER.
- void **OnBlend** (wxCommandEvent &)
Navigates to BLEND_CHAT or BLEND_CREATION.
- void **OnRefresh** (wxCommandEvent &)
Advances the feed by one page.
- void **RecolorAll** (const **Palette** &oldPalette)
- void **RecolorWidget** (wxWindow *w, const **Palette** &oldPalette)
- bool **LoadImage** (int key, const wxString &path)

Private Attributes

- **AppController** & **controller**
- wxSimplebook * **book**
- **LoginPanel** * **loginPanel**
- **DataInstructionsPanel** * **dataInstrPanel**
- **BlendCreationPanel** * **creationPanel**
- **BlendFeedPanel** * **feedPanel**
- **UserPanel** * **userPanel**
- **SettingsPanel** * **settingsPanel**
- **BlendChatPanel** * **chatPanel**
- **ActiveBlendsPanel** * **activeBlendsPanel**
- wxButton * **refreshBtn**
- bool **refreshInProgress**
- std::unordered_map< int, wxImage > **images**

6.33.1 Detailed Description

Root application window containing the full-window page switcher.

[MainFrame](#) class declaration.

Definition at line 48 of file [MainFrame.hpp](#).

6.33.2 Member Enumeration Documentation

6.33.2.1 ImageKey

```
enum MainFrame::ImageKey [private]
```

Definition at line 71 of file [MainFrame.hpp](#).

6.33.3 Constructor & Destructor Documentation

6.33.3.1 MainFrame()

```
MainFrame::MainFrame (
    ApplicationController & controller) [explicit]
```

Constructs and shows the [MainFrame](#). All sub-panels are created and added to the wxSimplebook here.

Parameters

<i>controller</i>	Application controller shared by all panels.
-------------------	--

Definition at line 98 of file [MainFrame.cpp](#).

6.33.4 Member Function Documentation

6.33.4.1 BuildHomePage()

```
wxPanel * MainFrame::BuildHomePage (
    wxWindow * parent) [private]
```

Builds the home page composite (top bar + title + feed + refresh button).

Returns

Pointer to the constructed home page wxPanel.

Definition at line 281 of file [MainFrame.cpp](#).

6.33.4.2 LoadImage()

```
bool MainFrame::LoadImage (
    int key,
    const wxString & path) [private]
```

Definition at line 427 of file [MainFrame.cpp](#).

6.33.4.3 NavigateTo()

```
void MainFrame::NavigateTo (
    Page page)
```

Switches to the specified page and calls the target panel's Refresh() method where applicable so it always shows fresh data on arrival.

Parameters

<i>page</i>	Page to navigate to.
-------------	--------------------------------------

Definition at line 229 of file [MainFrame.cpp](#).

6.33.4.4 RecolorAll()

```
void MainFrame::RecolorAll (  
    const Palette & oldPalette) [private]
```

Recursively recolors every widget in the window tree by mapping old palette colours to the new palette after a theme switch.

Definition at line 369 of file [MainFrame.cpp](#).

6.33.4.5 RecolorWidget()

```
void MainFrame::RecolorWidget (  
    wxWindow * w,  
    const Palette & oldPalette) [private]
```

Helper: recursively recolor a widget and its children.

Definition at line 376 of file [MainFrame.cpp](#).

6.33.4.6 TriggerFeedRefresh()

```
void MainFrame::TriggerFeedRefresh ()
```

Advances the blend feed by one page of videos. Called by the Refresh button on the home screen.

Definition at line 239 of file [MainFrame.cpp](#).

6.33.5 Member Data Documentation

6.33.5.1 activeBlendsPanel

```
ActiveBlendsPanel* MainFrame::activeBlendsPanel [private]
```

Definition at line 85 of file [MainFrame.hpp](#).

6.33.5.2 book

```
wxSimplebook* MainFrame::book [private]
```

Definition at line 77 of file [MainFrame.hpp](#).

6.33.5.3 chatPanel

```
BlendChatPanel* MainFrame::chatPanel [private]
```

Definition at line 84 of file [MainFrame.hpp](#).

6.33.5.4 controller

```
AppController& MainFrame::controller [private]
```

Definition at line 75 of file [MainFrame.hpp](#).

6.33.5.5 creationPanel

```
BlendCreationPanel* MainFrame::creationPanel [private]
```

Definition at line 80 of file [MainFrame.hpp](#).

6.33.5.6 dataInstrPanel

```
DataInstructionsPanel* MainFrame::dataInstrPanel [private]
```

Definition at line 79 of file [MainFrame.hpp](#).

6.33.5.7 feedPanel

```
BlendFeedPanel* MainFrame::feedPanel [private]
```

Definition at line 81 of file [MainFrame.hpp](#).

6.33.5.8 images

```
std::unordered_map<int, wxImage> MainFrame::images [private]
```

Definition at line 90 of file [MainFrame.hpp](#).

6.33.5.9 loginPanel

```
LoginPanel* MainFrame::loginPanel [private]
```

Definition at line 78 of file [MainFrame.hpp](#).

6.33.5.10 refreshBtn

```
wxButton* MainFrame::refreshBtn [private]
```

Definition at line 87 of file [MainFrame.hpp](#).

6.33.5.11 refreshInProgress

```
bool MainFrame::refreshInProgress [private]
```

Definition at line 89 of file [MainFrame.hpp](#).

6.33.5.12 settingsPanel

```
SettingsPanel* MainFrame::settingsPanel [private]
```

Definition at line 83 of file [MainFrame.hpp](#).

6.33.5.13 userPanel

```
UserPanel* MainFrame::userPanel [private]
```

Definition at line 82 of file [MainFrame.hpp](#).

The documentation for this class was generated from the following files:

- [MainFrame.hpp](#)
- [MainFrame.cpp](#)

6.34 Message Struct Reference

A single chat message sent within a [ChatRoom](#).

```
#include <Message.hpp>
```

Public Member Functions

- [Message](#) (const std::string &[userID](#), const std::string &[text](#), std::time_t [timestamp](#)=std::time(nullptr))
- std::string [toString](#) () const

Public Attributes

- std::string [userID](#)
ID of the [User](#) who sent this message.
- std::string [text](#)
Content of the message.
- std::time_t [timestamp](#)
Unix timestamp of when the message was sent.

6.34.1 Detailed Description

A single chat message sent within a [ChatRoom](#).

[Message](#) class declaration.

Definition at line 22 of file [Message.hpp](#).

6.34.2 Constructor & Destructor Documentation

6.34.2.1 Message()

```
Message::Message (
    const std::string & userID,
    const std::string & text,
    std::time_t timestamp = std::time(nullptr)) [inline]
```

Constructs a [Message](#) with an optional timestamp.

Parameters

<i>userID</i>	ID of the sending user.
<i>text</i>	Message content.
<i>timestamp</i>	Unix timestamp; defaults to the current time.

Definition at line 33 of file [Message.hpp](#).

6.34.3 Member Function Documentation

6.34.3.1 toString()

```
std::string Message::toString () const
```

String representation of [Message](#)

Returns

```
string
```

Definition at line 10 of file [Message.cpp](#).

6.34.4 Member Data Documentation

6.34.4.1 text

```
std::string Message::text
```

Content of the message.

Definition at line 24 of file [Message.hpp](#).

6.34.4.2 timestamp

```
std::time_t Message::timestamp
```

Unix timestamp of when the message was sent.

Definition at line 25 of file [Message.hpp](#).

6.34.4.3 userID

```
std::string Message::userID
```

ID of the [User](#) who sent this message.

Definition at line 23 of file [Message.hpp](#).

The documentation for this struct was generated from the following files:

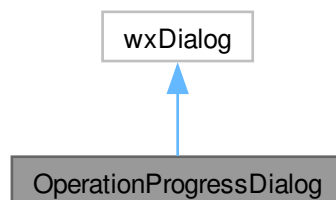
- [Message.hpp](#)
- [Message.cpp](#)

6.35 OperationProgressDialog Class Reference

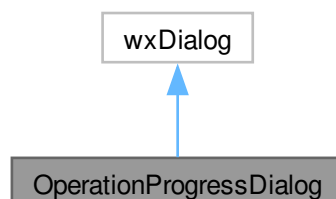
Non-modal progress dialog showing upload/processing progress.

```
#include <OperationProgressDialog.hpp>
```

Inheritance diagram for OperationProgressDialog:



Collaboration diagram for OperationProgressDialog:



Public Member Functions

- [OperationProgressDialog](#) (wxWindow *parent, const wxString &title)
Constructor with parent window and title.
- [~OperationProgressDialog](#) ()
Destructor.
- void [UpdateProgress](#) (double progress, const wxString &message=wxEmptyString)
Update progress bar and status message.
- virtual int [ShowModal](#) ()
Show dialog non-modally.

Private Member Functions

- void [UpdateProgressFill](#) ()

Private Attributes

- wxPanel * [progressTrack](#)
Progress track background.
- wxPanel * [progressFill](#)
Progress fill foreground.
- wxStaticText * [percentLabel](#)
Percentage display.
- wxStaticText * [messageLabel](#)
Status message.
- double [currentProgress](#) = 0.0

6.35.1 Detailed Description

Non-modal progress dialog showing upload/processing progress.

Definition at line 21 of file [OperationProgressDialog.hpp](#).

6.35.2 Constructor & Destructor Documentation

6.35.2.1 OperationProgressDialog()

```
OperationProgressDialog::OperationProgressDialog (  
    wxWindow * parent,  
    const wxString & title)
```

Constructor with parent window and title.

Definition at line 20 of file [OperationProgressDialog.cpp](#).

6.35.2.2 ~OperationProgressDialog()

```
OperationProgressDialog::~OperationProgressDialog ()
```

Destructor.

Definition at line 90 of file [OperationProgressDialog.cpp](#).

6.35.3 Member Function Documentation

6.35.3.1 ShowModal()

```
int OperationProgressDialog::ShowModal () [virtual]
```

Show dialog non-modally.

Definition at line 118 of file [OperationProgressDialog.cpp](#).

6.35.3.2 UpdateProgress()

```
void OperationProgressDialog::UpdateProgress (
    double progress,
    const wxString & message = wxEmptyString)
```

Update progress bar and status message.

Parameters

<i>progress</i>	Progress value 0.0 to 1.0.
<i>message</i>	Optional status message to display.

Definition at line 94 of file [OperationProgressDialog.cpp](#).

6.35.3.3 UpdateProgressFill()

```
void OperationProgressDialog::UpdateProgressFill () [private]
```

Definition at line 124 of file [OperationProgressDialog.cpp](#).

6.35.4 Member Data Documentation

6.35.4.1 currentProgress

```
double OperationProgressDialog::currentProgress = 0.0 [private]
```

Definition at line 52 of file [OperationProgressDialog.hpp](#).

6.35.4.2 messageLabel

```
wxStaticText* OperationProgressDialog::messageLabel [private]
```

Status message.

Definition at line 51 of file [OperationProgressDialog.hpp](#).

6.35.4.3 percentLabel

```
wxStaticText* OperationProgressDialog::percentLabel [private]
```

Percentage display.

Definition at line 50 of file [OperationProgressDialog.hpp](#).

6.35.4.4 progressFill

```
wxPanel* OperationProgressDialog::progressFill [private]
```

Progress fill foreground.

Definition at line 49 of file [OperationProgressDialog.hpp](#).

6.35.4.5 progressTrack

```
wxPanel* OperationProgressDialog::progressTrack [private]
```

Progress track background.

Track and fill are custom panels so Accent color is reliable across platforms.

Definition at line 48 of file [OperationProgressDialog.hpp](#).

The documentation for this class was generated from the following files:

- [OperationProgressDialog.hpp](#)
- [OperationProgressDialog.cpp](#)

6.36 Palette Struct Reference

[Palette](#) class declaration.

```
#include <UIColors.hpp>
```

Public Attributes

- wxColour [Background](#)
- wxColour [Surface](#)
- wxColour [SurfaceRaised](#)
- wxColour [Accent](#)
- wxColour [AccentHover](#)
- wxColour [TextPrimary](#)
- wxColour [TextSecondary](#)
- wxColour [TextMuted](#)
- wxColour [Separator](#)
- wxColour [Danger](#)

6.36.1 Detailed Description

[Palette](#) class declaration.

Definition at line [39](#) of file [UIColors.hpp](#).

6.36.2 Member Data Documentation

6.36.2.1 Accent

```
wxColour Palette::Accent
```

Definition at line [44](#) of file [UIColors.hpp](#).

6.36.2.2 AccentHover

```
wxColour Palette::AccentHover
```

Definition at line [45](#) of file [UIColors.hpp](#).

6.36.2.3 Background

```
wxColour Palette::Background
```

Definition at line [40](#) of file [UIColors.hpp](#).

6.36.2.4 Danger

```
wxColour Palette::Danger
```

Definition at line [52](#) of file [UIColors.hpp](#).

6.36.2.5 Separator

`wxColour Palette::Separator`

Definition at line 51 of file [UIColors.hpp](#).

6.36.2.6 Surface

`wxColour Palette::Surface`

Definition at line 41 of file [UIColors.hpp](#).

6.36.2.7 SurfaceRaised

`wxColour Palette::SurfaceRaised`

Definition at line 42 of file [UIColors.hpp](#).

6.36.2.8 TextMuted

`wxColour Palette::TextMuted`

Definition at line 49 of file [UIColors.hpp](#).

6.36.2.9 TextPrimary

`wxColour Palette::TextPrimary`

Definition at line 47 of file [UIColors.hpp](#).

6.36.2.10 TextSecondary

`wxColour Palette::TextSecondary`

Definition at line 48 of file [UIColors.hpp](#).

The documentation for this struct was generated from the following file:

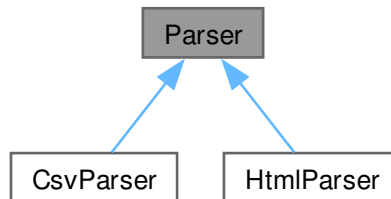
- [UIColors.hpp](#)

6.37 Parser Class Reference

[Parser](#) class declaration.

```
#include <Parser.hpp>
```

Inheritance diagram for Parser:



Public Member Functions

- virtual `std::list< std::unordered_map< std::string, std::string > >` [parse](#) ()=0
- virtual void [setData](#) (`std::list< std::string >` data)=0
- virtual int [getParseId](#) ()=0

6.37.1 Detailed Description

[Parser](#) class declaration.

Definition at line 28 of file [Parser.hpp](#).

6.37.2 Member Function Documentation

6.37.2.1 `getParseId()`

```
virtual int Parser::getParseId () [pure virtual]
```

Gets the file format ID associated with this parser.

Returns

Integer ID representing the file format (e.g., `File_ID::CSV`)

Implemented in [CsvParser](#), and [HtmlParser](#).

6.37.2.2 parse()

```
virtual std::list< std::unordered_map< std::string, std::string > > Parser::parse () [pure virtual]
```

Parses raw file data into a structured list of dictionaries.

Converts raw file data configured via [setData\(\)](#) into a list of unordered string-to-string dictionaries. Each dictionary represents a row with column titles as keys and row contents as values.

Returns

List of unordered maps representing parsed file rows

Implemented in [CsvParser](#), and [HtmlParser](#).

6.37.2.3 setData()

```
virtual void Parser::setData (
    std::list< std::string > data) [pure virtual]
```

Sets the raw file data to be parsed by [parse\(\)](#).

Parameters

<i>data</i>	List of strings containing raw file data
-------------	--

Implemented in [CsvParser](#).

The documentation for this class was generated from the following file:

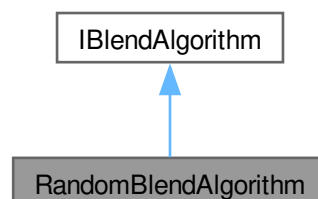
- [Parser.hpp](#)

6.38 RandomBlendAlgorithm Class Reference

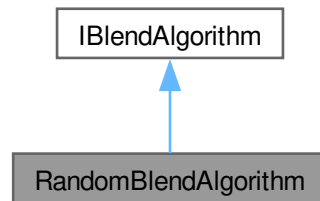
[RandomBlendAlgorithm](#) class declaration.

```
#include <RandomBlendAlgorithm.hpp>
```

Inheritance diagram for RandomBlendAlgorithm:



Collaboration diagram for RandomBlendAlgorithm:



Public Member Functions

- [RandomBlendAlgorithm](#) (int videosPerUser=5)
- [Blend generateBlend](#) (const std::list< [User](#) > &participants, const std::string &title="") override

Private Attributes

- int [videosPerUser](#)

6.38.1 Detailed Description

[RandomBlendAlgorithm](#) class declaration.

Definition at line 18 of file [RandomBlendAlgorithm.hpp](#).

6.38.2 Constructor & Destructor Documentation

6.38.2.1 RandomBlendAlgorithm()

```
RandomBlendAlgorithm::RandomBlendAlgorithm (
    int videosPerUser = 5) [explicit]
```

Parameters

<i>videosPerUser</i>	How many videos to sample from each participant. If a user has fewer videos than this, all of their videos are included.
----------------------	--

Definition at line 14 of file [RandomBlendAlgorithm.cpp](#).

6.38.3 Member Function Documentation

6.38.3.1 generateBlend()

```
Blend RandomBlendAlgorithm::generateBlend (
    const std::list< User > & participants,
    const std::string & title = "") [override], [virtual]
```

Generates a [Blend](#) from a collection of participants.

Parameters

<i>participants</i>	List of users whose Watch Later data is used as input.
<i>title</i>	User-facing name for the blend (default: empty).

Returns

A new [Blend](#) containing the algorithm's chosen video selection.

Implements [IBlendAlgorithm](#).

Definition at line 17 of file [RandomBlendAlgorithm.cpp](#).

6.38.4 Member Data Documentation

6.38.4.1 videosPerUser

```
int RandomBlendAlgorithm::videosPerUser [private]
```

Definition at line 34 of file [RandomBlendAlgorithm.hpp](#).

The documentation for this class was generated from the following files:

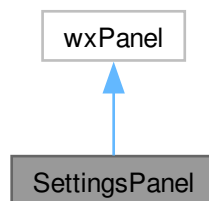
- [RandomBlendAlgorithm.hpp](#)
- [RandomBlendAlgorithm.cpp](#)

6.39 SettingsPanel Class Reference

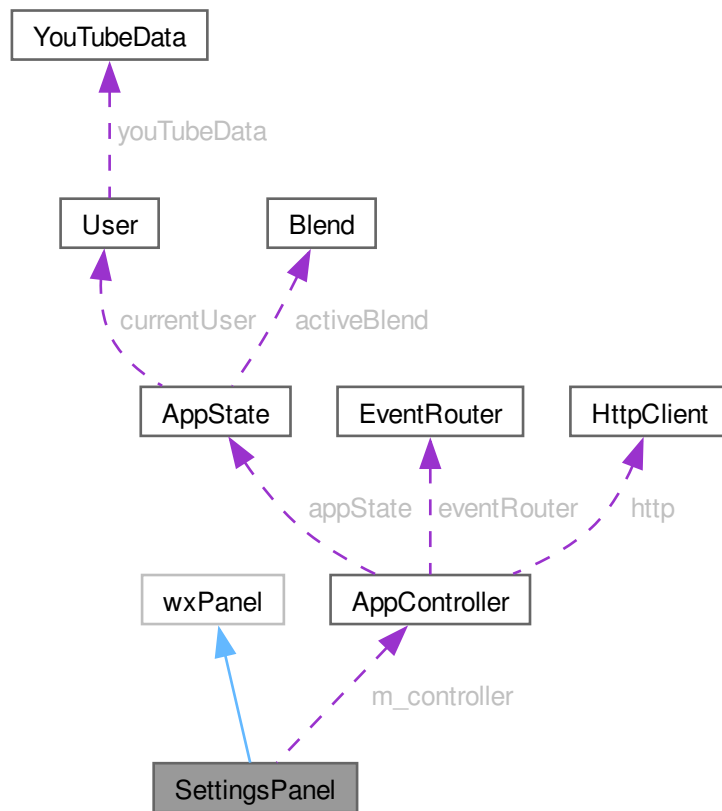
[SettingsPanel](#) class declaration.

```
#include <SettingsPanel.hpp>
```

Inheritance diagram for SettingsPanel:



Collaboration diagram for SettingsPanel:



Public Member Functions

- [SettingsPanel](#) (`wxWindow *parent`, [AppController](#) &controller, [NavigateFn](#) nav)

Private Member Functions

- void [RefreshUserInfo](#) ()
- void [OnShow](#) (`wxShowEvent &evt`)
- void [OnThemeChanged](#) (`wxCommandEvent &evt`)
- void [OnLogout](#) (`wxCommandEvent &evt`)

Private Attributes

- [AppController](#) & `m_controller`
- [NavigateFn](#) `m_nav`
- `wxChoice *` `m_themeChoice` = nullptr

6.39.1 Detailed Description

[SettingsPanel](#) class declaration.

Definition at line 21 of file [SettingsPanel.hpp](#).

6.39.2 Constructor & Destructor Documentation

6.39.2.1 SettingsPanel()

```
SettingsPanel::SettingsPanel (  
    wxWindow * parent,  
    ApplicationController & controller,  
    NavigateFn nav)
```

Definition at line 29 of file [SettingsPanel.cpp](#).

6.39.3 Member Function Documentation

6.39.3.1 OnLogout()

```
void SettingsPanel::OnLogout (  
    wxCommandEvent & evt) [private]
```

Definition at line 121 of file [SettingsPanel.cpp](#).

6.39.3.2 OnShow()

```
void SettingsPanel::OnShow (  
    wxShowEvent & evt) [private]
```

Definition at line 99 of file [SettingsPanel.cpp](#).

6.39.3.3 OnThemeChanged()

```
void SettingsPanel::OnThemeChanged (  
    wxCommandEvent & evt) [private]
```

Definition at line 106 of file [SettingsPanel.cpp](#).

6.39.3.4 RefreshUserInfo()

```
void SettingsPanel::RefreshUserInfo () [private]
```

Definition at line 92 of file [SettingsPanel.cpp](#).

6.39.4 Member Data Documentation

6.39.4.1 m_controller

[AppController](#)& `SettingsPanel::m_controller` [private]

Definition at line 26 of file [SettingsPanel.hpp](#).

6.39.4.2 m_nav

[NavigateFn](#) `SettingsPanel::m_nav` [private]

Definition at line 27 of file [SettingsPanel.hpp](#).

6.39.4.3 m_themeChoice

`wxChoice*` `SettingsPanel::m_themeChoice` = nullptr [private]

Definition at line 28 of file [SettingsPanel.hpp](#).

The documentation for this class was generated from the following files:

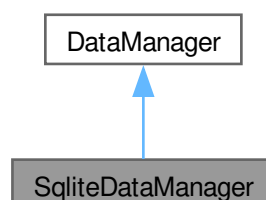
- [SettingsPanel.hpp](#)
- [SettingsPanel.cpp](#)

6.40 SqliteDataManager Class Reference

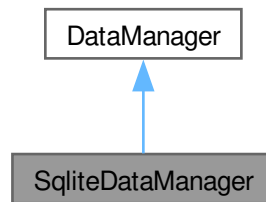
[SqliteDataManager](#) type declaration.

```
#include <SqliteDataManager.hpp>
```

Inheritance diagram for `SqliteDataManager`:



Collaboration diagram for `SqliteDataManager`:



Public Member Functions

- [SqliteDataManager](#) (const std::string &dbPath)
- bool [createUser](#) (const [User](#) &user) override
- std::optional< [User](#) > [findUserById](#) (const std::string &userID) override
- bool [validatePassword](#) (const std::string &userID, const std::string &password) override
- bool [saveWatchLater](#) (const std::string &userID, const std::list< [Video](#) > &videos) override
- std::list< [Video](#) > [loadWatchLater](#) (const std::string &userID) override
- bool [saveBlend](#) (const std::string &blendID, const std::string &creatorID, const std::string &algorithm, const std::vector< std::string > &participantIDs) override
- std::optional< std::string > [findBlendForUser](#) (const std::string &userID) override
- std::vector< std::string > [loadBlendParticipants](#) (const std::string &blendID) override

Private Member Functions

- void [initSchema](#) ()

Private Attributes

- sqlite3 * [db](#) = nullptr

6.40.1 Detailed Description

[SqliteDataManager](#) type declaration.

Definition at line 23 of file [SqliteDataManager.hpp](#).

6.40.2 Constructor & Destructor Documentation

6.40.2.1 SqliteDataManager()

```

SqliteDataManager::SqliteDataManager (
    const std::string & dbPath) [explicit]
  
```

Definition at line 13 of file [SqliteDataManager.cpp](#).

6.40.2.2 ~SqliteDataManager()

```
SqliteDataManager::~SqliteDataManager () [override]
```

Definition at line 25 of file [SqliteDataManager.cpp](#).

6.40.3 Member Function Documentation

6.40.3.1 createUser()

```
bool SqliteDataManager::createUser (
    const User & user) [override], [virtual]
```

Persists a new user record.

Returns

true on success, false if the userID already exists or the insert otherwise fails.

Implements [DataManager](#).

Definition at line 95 of file [SqliteDataManager.cpp](#).

6.40.3.2 findBlendForUser()

```
std::optional< std::string > SqliteDataManager::findBlendForUser (
    const std::string & userID) [override], [virtual]
```

Returns the blendID of the blend this user belongs to, if any.

Returns

The blendID string, or std::nullopt if the user has no blend.

Implements [DataManager](#).

Definition at line 304 of file [SqliteDataManager.cpp](#).

6.40.3.3 findUserByID()

```
std::optional< User > SqliteDataManager::findUserByID (
    const std::string & userID) [override], [virtual]
```

Loads a user by their unique ID.

Returns

The [User](#) if found, std::nullopt otherwise.

Implements [DataManager](#).

Definition at line 126 of file [SqliteDataManager.cpp](#).

6.40.3.4 initSchema()

```
void SqliteDataManager::initSchema () [private]
```

Definition at line 31 of file [SqliteDataManager.cpp](#).

6.40.3.5 loadBlendParticipants()

```
std::vector< std::string > SqliteDataManager::loadBlendParticipants (
    const std::string & blendID) [override], [virtual]
```

Returns all participant userIDs for a given blend.

Parameters

<i>blendID</i>	to be used for loading the respective participants
----------------	--

Returns

vector of strings representing userIDs

Implements [DataManager](#).

Definition at line 327 of file [SqliteDataManager.cpp](#).

6.40.3.6 loadWatchLater()

```
std::list< Video > SqliteDataManager::loadWatchLater (  
    const std::string & userID) [override], [virtual]
```

Loads the stored Watch Later video list for a user.

Returns

Empty list if no data has been saved for this user.

Implements [DataManager](#).

Definition at line 211 of file [SqliteDataManager.cpp](#).

6.40.3.7 saveBlend()

```
bool SqliteDataManager::saveBlend (  
    const std::string & blendID,  
    const std::string & creatorID,  
    const std::string & algorithm,  
    const std::vector< std::string > & participantIDs) [override], [virtual]
```

Persists a blend and its participant list. Re-calling with the same blendID replaces the existing record.

Parameters

<i>creatorID</i>	userID of the user who initiated the blend.
<i>participantIDs</i>	All participant userIDs (including those without data).

Implements [DataManager](#).

Definition at line 250 of file [SqliteDataManager.cpp](#).

6.40.3.8 saveWatchLater()

```
bool SqliteDataManager::saveWatchLater (
    const std::string & userID,
    const std::list< Video > & videos) [override], [virtual]
```

Persists a user's Watch Later video list, replacing any previously saved list.

Returns

true on success.

Implements [DataManager](#).

Definition at line 164 of file [SqliteDataManager.cpp](#).

6.40.3.9 validatePassword()

```
bool SqliteDataManager::validatePassword (
    const std::string & userID,
    const std::string & password) [override], [virtual]
```

Checks whether the given plaintext password matches the stored record.

Returns

true if the userID exists and the password matches.

Implements [DataManager](#).

Definition at line 155 of file [SqliteDataManager.cpp](#).

6.40.4 Member Data Documentation

6.40.4.1 db

```
sqlite3* SqliteDataManager::db = nullptr [private]
```

Definition at line 47 of file [SqliteDataManager.hpp](#).

The documentation for this class was generated from the following files:

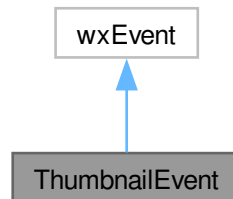
- [SqliteDataManager.hpp](#)
- [SqliteDataManager.cpp](#)

6.41 ThumbnailEvent Class Reference

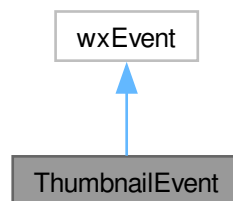
Event fired on the [VideoCard](#) when an async image fetch completes.

```
#include <VideoCard.hpp>
```

Inheritance diagram for ThumbnailEvent:



Collaboration diagram for ThumbnailEvent:



Public Member Functions

- [ThumbnailEvent](#) (wxEventType type, int id, wxImage img, bool ok, int gen)
- wxEvent * [Clone](#) () const override

Public Attributes

- wxImage [image](#)
Decoded image (valid only when success is true).
- bool [success](#)
True if the fetch and decode both succeeded.
- int [generation](#)
Must match [VideoCard::m_thumbGen](#) or [m_logoGen](#).

6.41.1 Detailed Description

Event fired on the [VideoCard](#) when an async image fetch completes.

Used for both the video thumbnail (EVT_THUMBNAIL_LOADED) and the channel logo (EVT_CHANNEL_LOGO_LOADED). The generation field must match the card's current counter or the event is treated as stale and discarded.

[ThumbnailEvent](#) class declaration.

Definition at line 37 of file [VideoCard.hpp](#).

6.41.2 Constructor & Destructor Documentation

6.41.2.1 ThumbnailEvent()

```
ThumbnailEvent::ThumbnailEvent (
    wxEventType type,
    int id,
    wxImage img,
    bool ok,
    int gen) [inline]
```

Definition at line 43 of file [VideoCard.hpp](#).

6.41.3 Member Function Documentation

6.41.3.1 Clone()

```
wxEvent * ThumbnailEvent::Clone () const [inline], [override]
```

Definition at line 46 of file [VideoCard.hpp](#).

6.41.4 Member Data Documentation

6.41.4.1 generation

```
int ThumbnailEvent::generation
```

Must match [VideoCard::m_thumbGen](#) or [m_logoGen](#).

Definition at line 41 of file [VideoCard.hpp](#).

6.41.4.2 image

```
wxImage ThumbnailEvent::image
```

Decoded image (valid only when success is true).

Definition at line 39 of file [VideoCard.hpp](#).

6.41.4.3 success

```
bool ThumbnailEvent::success
```

True if the fetch and decode both succeeded.

Definition at line 40 of file [VideoCard.hpp](#).

The documentation for this class was generated from the following file:

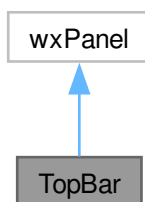
- [VideoCard.hpp](#)

6.42 TopBar Class Reference

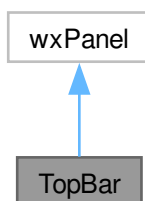
Reusable horizontal top bar for interior pages.

```
#include <TopBar.hpp>
```

Inheritance diagram for TopBar:



Collaboration diagram for TopBar:



Public Member Functions

- [TopBar](#) (wxWindow *parent, const wxString &title, [NavigateFn](#) nav, [Page](#) backDest)
Construct a [TopBar](#).

Private Member Functions

- void [OnBack](#) (wxCommandEvent &evt)
Handle clicks on the back button.

Private Attributes

- [NavigateFn](#) m_nav
- [Page](#) m_backDest

6.42.1 Detailed Description

Reusable horizontal top bar for interior pages.

Definition at line 19 of file [TopBar.hpp](#).

6.42.2 Constructor & Destructor Documentation

6.42.2.1 TopBar()

```
TopBar::TopBar (
    wxWindow * parent,
    const wxString & title,
    NavigateFn nav,
    Page backDest)
```

Construct a [TopBar](#).

Parameters

<i>parent</i>	Parent window that owns the bar.
<i>title</i>	Title text shown in the center.
<i>nav</i>	Navigation callback used for back navigation.
<i>backDest</i>	Destination page when the back button is clicked.

Definition at line 16 of file [TopBar.cpp](#).

6.42.3 Member Function Documentation

6.42.3.1 OnBack()

```
void TopBar::OnBack (
    wxCommandEvent & evt) [private]
```

Handle clicks on the back button.

Parameters

<code>evt</code>	Button click event.
------------------	---------------------

Definition at line 73 of file [TopBar.cpp](#).

6.42.4 Member Data Documentation

6.42.4.1 `m_backDest`

```
Page TopBar::m_backDest [private]
```

Destination page used when the back button is activated.

Definition at line 40 of file [TopBar.hpp](#).

6.42.4.2 `m_nav`

```
NavigateFn TopBar::m_nav [private]
```

Navigation callback forwarded from the parent panel.

Definition at line 38 of file [TopBar.hpp](#).

The documentation for this class was generated from the following files:

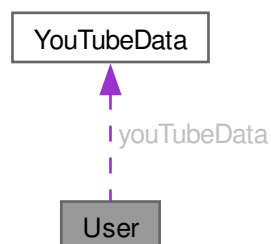
- [TopBar.hpp](#)
- [TopBar.cpp](#)

6.43 User Class Reference

[User](#) class declaration.

```
#include <User.hpp>
```

Collaboration diagram for User:



Public Member Functions

- [User](#) (const std::string &userID, const std::string &username, const std::string &email, const std::string &password)
- std::string [getUserID](#) () const
- std::string [getUsername](#) () const
- std::string [getEmail](#) () const
- std::string [getPassword](#) () const
- std::list< [Friend](#) > [getFriends](#) () const
- [YouTubeData](#) & [getYouTubeData](#) ()
- const [YouTubeData](#) & [getYouTubeData](#) () const
- void [setUsername](#) (const std::string &username)
- void [setEmail](#) (const std::string &email)
- void [setPassword](#) (const std::string &password)
- void [setYouTubeData](#) (const [YouTubeData](#) &youTubeData)
- void [addFriend](#) (const [User](#) &user)
- void [removeFriend](#) (const std::string &userID)
- std::string [toString](#) () const

Private Attributes

- std::string [userID](#)
 - std::string [username](#)
 - std::string [email](#)
 - std::string [password](#)
- Stored as plaintext - hashing is a future TODO.*
- std::list< [Friend](#) > [friends](#)
 - [YouTubeData](#) [youTubeData](#)

6.43.1 Detailed Description

[User](#) class declaration.

A registered application user.

Definition at line 24 of file [User.hpp](#).

6.43.2 Constructor & Destructor Documentation

6.43.2.1 User()

```
User::User (  
    const std::string & userID,  
    const std::string & username,  
    const std::string & email,  
    const std::string & password)
```

Constructs a [User](#) with identity fields.

Parameters

<i>userID</i>	Unique user identifier chosen at registration.
<i>username</i>	Display name shown in the UI.
<i>email</i>	User 's email address (may be empty).
<i>password</i>	Plaintext password.

Definition at line 13 of file [User.cpp](#).

6.43.3 Member Function Documentation

6.43.3.1 addFriend()

```
void User::addFriend (  
    const User & user)
```

Adds a friend entry for the given user.

Parameters

<i>user</i>	User to add as a friend.
-------------	--

Definition at line 43 of file [User.cpp](#).

6.43.3.2 getEmail()

```
std::string User::getEmail () const
```

Returns

Email address.

Definition at line 23 of file [User.cpp](#).

6.43.3.3 getFriends()

```
std::list< Friend > User::getFriends () const
```

Returns

List of this user's friends.

Definition at line 26 of file [User.cpp](#).

6.43.3.4 getPassword()

```
std::string User::getPassword () const
```

Returns

Plaintext password.

Definition at line 24 of file [User.cpp](#).

6.43.3.5 getUserID()

```
std::string User::getUserID () const
```

Returns

Unique user identifier.

Definition at line 21 of file [User.cpp](#).

6.43.3.6 getUsername()

```
std::string User::getUsername () const
```

Returns

Display name.

Definition at line 22 of file [User.cpp](#).

6.43.3.7 getYouTubeData() [1/2]

```
YouTubeData & User::getYouTubeData ()
```

Returns

Mutable reference to this user's YouTube data.

Definition at line 28 of file [User.cpp](#).

6.43.3.8 getYouTubeData() [2/2]

```
const YouTubeData & User::getYouTubeData () const
```

Returns

Const reference to this user's YouTube data.

Definition at line 29 of file [User.cpp](#).

6.43.3.9 removeFriend()

```
void User::removeFriend (  
    const std::string & userID)
```

Removes the friend entry with the given userID.

Parameters

<i>userID</i>	ID of the friend to remove.
---------------	-----------------------------

Definition at line 50 of file [User.cpp](#).

6.43.3.10 setEmail()

```
void User::setEmail (  
    const std::string & email)
```

Parameters

<i>email</i>	New email address.
--------------	--------------------

Definition at line 34 of file [User.cpp](#).

6.43.3.11 setPassword()

```
void User::setPassword (  
    const std::string & password)
```

Parameters

<i>password</i>	New plaintext password.
-----------------	-------------------------

Definition at line 35 of file [User.cpp](#).

6.43.3.12 setUsername()

```
void User::setUsername (  
    const std::string & username)
```

Parameters

<i>username</i>	New display name.
-----------------	-------------------

Definition at line 33 of file [User.cpp](#).

6.43.3.13 setYouTubeData()

```
void User::setYouTubeData (  
    const YouTubeData & youtubeData)
```

Parameters

<code>youTubeData</code>	Replaces the user's YouTube data in bulk.
--------------------------	---

Definition at line 37 of file [User.cpp](#).

6.43.3.14 toString()

```
std::string User::toString () const
```

String representation of [User](#)

Returns

```
string
```

Definition at line 56 of file [User.cpp](#).

6.43.4 Member Data Documentation

6.43.4.1 email

```
std::string User::email [private]
```

Definition at line 28 of file [User.hpp](#).

6.43.4.2 friends

```
std::list<Friend> User::friends [private]
```

Definition at line 30 of file [User.hpp](#).

6.43.4.3 password

```
std::string User::password [private]
```

Stored as plaintext - hashing is a future TODO.

Definition at line 29 of file [User.hpp](#).

6.43.4.4 userID

```
std::string User::userID [private]
```

Definition at line 26 of file [User.hpp](#).

6.43.4.5 username

```
std::string User::username [private]
```

Definition at line 27 of file [User.hpp](#).

6.43.4.6 youTubeData

```
YouTubeData User::youTubeData [private]
```

Definition at line 31 of file [User.hpp](#).

The documentation for this class was generated from the following files:

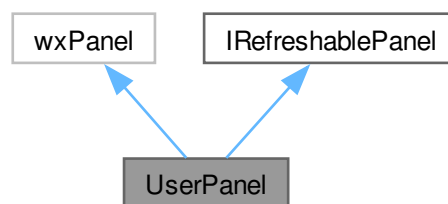
- [User.hpp](#)
- [User.cpp](#)

6.44 UserPanel Class Reference

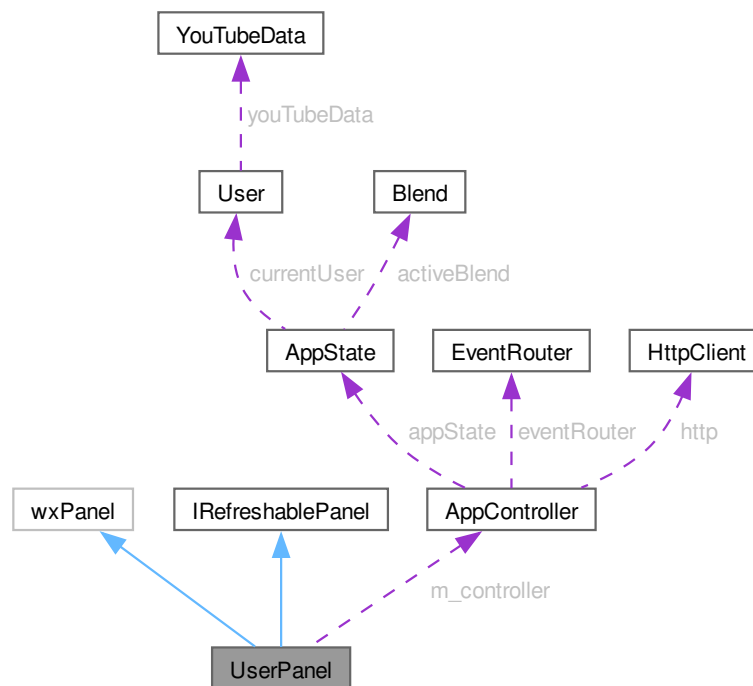
Panel for user account display and account actions.

```
#include <UserPanel.hpp>
```

Inheritance diagram for UserPanel:



Collaboration diagram for `UserPanel`:



Public Member Functions

- `UserPanel` (`wxWindow *parent`, `AppController` &controller, `NavigateFn` nav)
Construct a `UserPanel`.
- void `Refresh` () override
Refresh panel UI from current `AppState`.

Private Member Functions

- void `RefreshUserInfo` ()
- void `SetDeleteReauthVisible` (bool isVisible)
- void `OnDeleteRequest` (wxCommandEvent &evt)
- void `OnConfirmDelete` (wxCommandEvent &evt)
- void `OnReuploadCSV` (wxCommandEvent &evt)
- void `OnLogout` (wxCommandEvent &evt)
- void `OnShow` (wxShowEvent &evt)

Private Attributes

- `AppController` & `m_controller`
- `NavigateFn` `m_nav`
- `wxStaticText` * `m_usernameLabel`
- `wxStaticText` * `m_emailLabel`
- `wxTextCtrl` * `m_deletePasswordField`
- `wxStaticText` * `m_deleteErrorLabel`
- `wxButton` * `m_confirmDeleteBtn`

6.44.1 Detailed Description

Panel for user account display and account actions.

Definition at line 48 of file [UserPanel.hpp](#).

6.44.2 Constructor & Destructor Documentation

6.44.2.1 UserPanel()

```
UserPanel::UserPanel (  
    wxWindow * parent,  
    ApplicationController & controller,  
    NavigateFn nav)
```

Construct a [UserPanel](#).

Parameters

<i>parent</i>	Parent wxWidgets window.
<i>controller</i>	Application controller reference.
<i>nav</i>	Page navigation callback.

Definition at line 29 of file [UserPanel.cpp](#).

6.44.3 Member Function Documentation

6.44.3.1 OnConfirmDelete()

```
void UserPanel::OnConfirmDelete (  
    wxCommandEvent & evt) [private]
```

Handle delete-account confirmation click.

Definition at line 195 of file [UserPanel.cpp](#).

6.44.3.2 OnDeleteRequest()

```
void UserPanel::OnDeleteRequest (  
    wxCommandEvent & evt) [private]
```

Handle initial delete-account request button click.

Definition at line 179 of file [UserPanel.cpp](#).

6.44.3.3 OnLogout()

```
void UserPanel::OnLogout (
    wxCommandEvent & evt) [private]
```

Handle logout action.

Definition at line 266 of file [UserPanel.cpp](#).

6.44.3.4 OnReuploadCSV()

```
void UserPanel::OnReuploadCSV (
    wxCommandEvent & evt) [private]
```

Handle CSV re-upload action.

Definition at line 234 of file [UserPanel.cpp](#).

6.44.3.5 OnShow()

```
void UserPanel::OnShow (
    wxShowEvent & evt) [private]
```

Handle panel show event to trigger refresh.

Definition at line 172 of file [UserPanel.cpp](#).

6.44.3.6 Refresh()

```
void UserPanel::Refresh () [override], [virtual]
```

Refresh panel UI from current [AppState](#).

Implements [IRefreshablePanel](#).

Definition at line 128 of file [UserPanel.cpp](#).

6.44.3.7 RefreshUserInfo()

```
void UserPanel::RefreshUserInfo () [private]
```

Refresh user labels from [AppState](#).

Definition at line 133 of file [UserPanel.cpp](#).

6.44.3.8 SetDeleteReauthVisible()

```
void UserPanel::SetDeleteReauthVisible (  
    bool isVisible) [private]
```

Show or hide delete-account re-auth controls.

Definition at line 157 of file [UserPanel.cpp](#).

6.44.4 Member Data Documentation

6.44.4.1 m_confirmDeleteBtn

```
wxButton* UserPanel::m_confirmDeleteBtn [private]
```

Definition at line 72 of file [UserPanel.hpp](#).

6.44.4.2 m_controller

```
AppController& UserPanel::m_controller [private]
```

Definition at line 64 of file [UserPanel.hpp](#).

6.44.4.3 m_deleteErrorLabel

```
wxStaticText* UserPanel::m_deleteErrorLabel [private]
```

Definition at line 71 of file [UserPanel.hpp](#).

6.44.4.4 m_deletePasswordField

```
wxTextCtrl* UserPanel::m_deletePasswordField [private]
```

Definition at line 70 of file [UserPanel.hpp](#).

6.44.4.5 m_emailLabel

```
wxStaticText* UserPanel::m_emailLabel [private]
```

Definition at line 69 of file [UserPanel.hpp](#).

6.44.4.6 m_nav

```
NavigateFn UserPanel::m_nav [private]
```

Definition at line 65 of file [UserPanel.hpp](#).

6.44.4.7 m_usernameLabel

```
wxStaticText* UserPanel::m_usernameLabel [private]
```

Labels that display the logged-in user's info.

Definition at line 68 of file [UserPanel.hpp](#).

The documentation for this class was generated from the following files:

- [UserPanel.hpp](#)
- [UserPanel.cpp](#)

6.45 Video Class Reference

[Video](#) class declaration.

```
#include <Video.hpp>
```

Public Member Functions

- [Video](#) (const std::string &videoID, const std::string &title, const std::string &channelID, const std::string &thumbnailURL, int duration, const std::list< std::string > &tags, const std::string &channelName="", const std::string &channelLogoURL="")
- std::string [getVideoID](#) () const
- std::string [getTitle](#) () const
- std::string [getChannelID](#) () const
- std::string [getThumbnailURL](#) () const
- int [getDuration](#) () const
- std::list< std::string > [getTags](#) () const
- std::string [getChannelName](#) () const
- std::string [getChannelLogoURL](#) () const
- std::string [toString](#) () const

Private Attributes

- std::string [videoID](#)
- std::string [title](#)
- std::string [channelID](#)
- std::string [thumbnailURL](#)
- int [duration](#)
- std::list< std::string > [tags](#)
- std::string [channelName](#)
- std::string [channelLogoURL](#)

6.45.1 Detailed Description

[Video](#) class declaration.

Immutable data model for a single YouTube video.

Definition at line 21 of file [Video.hpp](#).

6.45.2 Constructor & Destructor Documentation

6.45.2.1 Video()

```
Video::Video (
    const std::string & videoID,
    const std::string & title,
    const std::string & channelID,
    const std::string & thumbnailURL,
    int duration,
    const std::list< std::string > & tags,
    const std::string & channelName = "",
    const std::string & channelLogoURL = "")
```

Constructs a [Video](#) with all metadata fields.

Parameters

<i>videoID</i>	YouTube video ID (e.g. "dQw4w9WgXcQ").
<i>title</i>	Video title returned by the Data API.
<i>channelID</i>	YouTube channel ID of the uploader.
<i>thumbnailURL</i>	URL of the best-available thumbnail image.
<i>duration</i>	Video duration in seconds (0 if unknown).
<i>tags</i>	List of tags associated with the video.
<i>channelName</i>	Display name of the uploading channel.
<i>channelLogoURL</i>	URL of the channel's profile picture.

Definition at line 12 of file [Video.cpp](#).

6.45.3 Member Function Documentation

6.45.3.1 getChannelID()

```
std::string Video::getChannelID () const
```

Returns

YouTube channel ID of the uploader.

Definition at line 37 of file [Video.cpp](#).

6.45.3.2 getChannelLogoURL()

```
std::string Video::getChannelLogoURL () const
```

Returns

URL of the channel's profile picture.

Definition at line 57 of file [Video.cpp](#).

6.45.3.3 `getChannelName()`

```
std::string Video::getChannelName () const
```

Returns

Display name of the uploading channel.

Definition at line 53 of file [Video.cpp](#).

6.45.3.4 `getDuration()`

```
int Video::getDuration () const
```

Returns

Duration in seconds (0 if unknown).

Definition at line 45 of file [Video.cpp](#).

6.45.3.5 `getTags()`

```
std::list< std::string > Video::getTags () const
```

Returns

List of tags associated with the video.

Definition at line 49 of file [Video.cpp](#).

6.45.3.6 `getThumbnailURL()`

```
std::string Video::getThumbnailURL () const
```

Returns

URL of the video thumbnail (empty if not yet enriched).

Definition at line 41 of file [Video.cpp](#).

6.45.3.7 `getTitle()`

```
std::string Video::getTitle () const
```

Returns

[Video](#) title (empty if not yet enriched by the API).

Definition at line 33 of file [Video.cpp](#).

6.45.3.8 `getVideoID()`

```
std::string Video::getVideoID () const
```

Returns

YouTube video ID string.

Definition at line 29 of file [Video.cpp](#).

6.45.3.9 `toString()`

```
std::string Video::toString () const
```

Returns

JSON string representation of this video.

Definition at line 61 of file [Video.cpp](#).

6.45.4 Member Data Documentation

6.45.4.1 `channelID`

```
std::string Video::channelID [private]
```

Definition at line 25 of file [Video.hpp](#).

6.45.4.2 `channelLogoURL`

```
std::string Video::channelLogoURL [private]
```

Definition at line 30 of file [Video.hpp](#).

6.45.4.3 `channelName`

```
std::string Video::channelName [private]
```

Definition at line 29 of file [Video.hpp](#).

6.45.4.4 `duration`

```
int Video::duration [private]
```

Definition at line 27 of file [Video.hpp](#).

6.45.4.5 tags

```
std::list<std::string> Video::tags [private]
```

Definition at line 28 of file [Video.hpp](#).

6.45.4.6 thumbnailURL

```
std::string Video::thumbnailURL [private]
```

Definition at line 26 of file [Video.hpp](#).

6.45.4.7 title

```
std::string Video::title [private]
```

Definition at line 24 of file [Video.hpp](#).

6.45.4.8 videoID

```
std::string Video::videoID [private]
```

Definition at line 23 of file [Video.hpp](#).

The documentation for this class was generated from the following files:

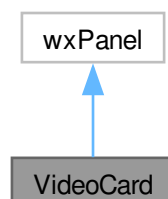
- [Video.hpp](#)
- [Video.cpp](#)

6.46 VideoCard Class Reference

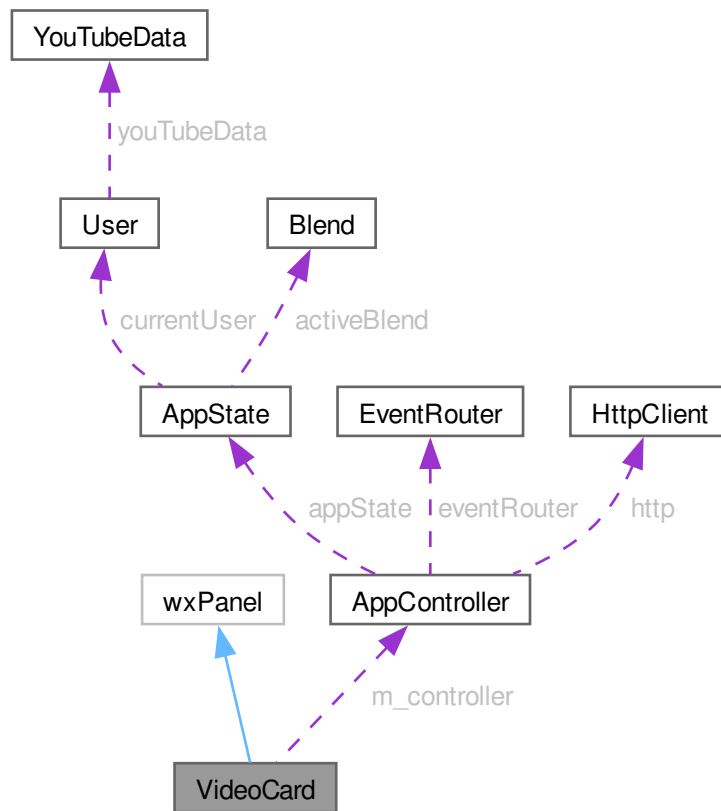
Custom wxPanel widget that displays a single YouTube video preview.

```
#include <VideoCard.hpp>
```

Inheritance diagram for VideoCard:



Collaboration diagram for VideoCard:



Public Member Functions

- **VideoCard** (wxWindow *parent, **AppController** &controller)
- void **SetVideo** (const **Video** &video)
- void **Clear** ()
- wxSize **DoGetBestSize** () const override

Private Member Functions

- void **FetchThumbnailAsync** ()
- void **FetchChannelLogoAsync** ()
- void **OnThumbnailLoaded** (**ThumbnailEvent** &evt)
- void **OnChannelLogoLoaded** (**ThumbnailEvent** &evt)
- void **OnPaint** (wxPaintEvent &evt)
- void **OnMouseEnter** (wxMouseEvent &evt)
- void **OnMouseLeave** (wxMouseEvent &evt)
- void **OnClick** (wxMouseEvent &evt)
- int **DrawWrappedText** (wxGraphicsContext *gc, const wxString &text, int maxLines, double x, double y, double maxWidth) const

Private Attributes

- [AppController](#) & [m_controller](#)
- wxString [m_videoID](#)
- wxString [m_title](#)
- wxString [m_channelID](#)
- wxString [m_channelName](#)
- wxString [m_thumbnailURL](#)
- wxString [m_channelLogoURL](#)
- wxBitmap [m_thumbnail](#)
- bool [m_thumbLoading](#) = false
- bool [m_thumbFailed](#) = false
- int [m_thumbGen](#) = 0
- Incremented on each SetVideo/Clear to discard stale fetches.*
- wxBitmap [m_channelLogo](#)
- bool [m_logoLoading](#) = false
- bool [m_logoFailed](#) = false
- int [m_logoGen](#) = 0
- Incremented on each SetVideo/Clear to discard stale fetches.*
- bool [m_hovered](#) = false

6.46.1 Detailed Description

Custom wxPanel widget that displays a single YouTube video preview.

[VideoCard](#) class declaration.

Definition at line 59 of file [VideoCard.hpp](#).

6.46.2 Constructor & Destructor Documentation

6.46.2.1 VideoCard()

```
VideoCard::VideoCard (
    wxWindow * parent,
    AppController & controller)
```

Constructs a [VideoCard](#).

Parameters

<i>parent</i>	Parent window.
<i>controller</i>	Application controller used to dispatch playVideo events.

Definition at line 72 of file [VideoCard.cpp](#).

6.46.3 Member Function Documentation

6.46.3.1 Clear()

```
void VideoCard::Clear ()
```

Resets the card to its empty placeholder state.

Definition at line 114 of file [VideoCard.cpp](#).

6.46.3.2 DoGetBestSize()

```
wxSize VideoCard::DoGetBestSize () const [inline], [override]
```

Definition at line 77 of file [VideoCard.hpp](#).

6.46.3.3 DrawWrappedText()

```
int VideoCard::DrawWrappedText (
    wxGraphicsContext * gc,
    const wxString & text,
    int maxLines,
    double x,
    double y,
    double maxWidth) const [private]
```

Draws word-wrapped text with ellipsis trimming on the last line.

Parameters

<i>gc</i>	Graphics context to draw into.
<i>text</i>	Text to render.
<i>maxLines</i>	Maximum number of lines before truncating.
<i>x</i>	Left edge of the text area.
<i>y</i>	Top edge of the first line.
<i>maxWidth</i>	Maximum pixel width of a line.

Returns

Y coordinate immediately below the last rendered line.

Definition at line 224 of file [VideoCard.cpp](#).

6.46.3.4 FetchChannelLogoAsync()

```
void VideoCard::FetchChannelLogoAsync () [private]
```

Spawns a background thread to fetch and decode the channel logo.

Definition at line 157 of file [VideoCard.cpp](#).

6.46.3.5 FetchThumbnailAsync()

```
void VideoCard::FetchThumbnailAsync () [private]
```

Spawns a background thread to fetch and decode the video thumbnail.

Definition at line 130 of file [VideoCard.cpp](#).

6.46.3.6 OnChannelLogoLoaded()

```
void VideoCard::OnChannelLogoLoaded (  
    ThumbnailEvent & evt) [private]
```

Definition at line 191 of file [VideoCard.cpp](#).

6.46.3.7 OnClick()

```
void VideoCard::OnClick (  
    wxMouseEvent & evt) [private]
```

Definition at line 206 of file [VideoCard.cpp](#).

6.46.3.8 OnMouseEnter()

```
void VideoCard::OnMouseEnter (  
    wxMouseEvent & evt) [private]
```

Definition at line 203 of file [VideoCard.cpp](#).

6.46.3.9 OnMouseLeave()

```
void VideoCard::OnMouseLeave (  
    wxMouseEvent & evt) [private]
```

Definition at line 204 of file [VideoCard.cpp](#).

6.46.3.10 OnPaint()

```
void VideoCard::OnPaint (  
    wxPaintEvent & evt) [private]
```

Definition at line 276 of file [VideoCard.cpp](#).

6.46.3.11 OnThumbnailLoaded()

```
void VideoCard::OnThumbnailLoaded (  
    ThumbnailEvent & evt) [private]
```

Definition at line 179 of file [VideoCard.cpp](#).

6.46.3.12 SetVideo()

```
void VideoCard::SetVideo (  
    const Video & video)
```

Loads a video into the card and starts async image fetches.

Parameters

<i>video</i>	Video whose metadata and images should be displayed.
--------------	--

Definition at line 95 of file [VideoCard.cpp](#).

6.46.4 Member Data Documentation

6.46.4.1 m_channelID

```
wxString VideoCard::m_channelID [private]
```

Definition at line 84 of file [VideoCard.hpp](#).

6.46.4.2 m_channelLogo

```
wxBitmap VideoCard::m_channelLogo [private]
```

Definition at line 94 of file [VideoCard.hpp](#).

6.46.4.3 m_channelLogoURL

```
wxString VideoCard::m_channelLogoURL [private]
```

Definition at line 87 of file [VideoCard.hpp](#).

6.46.4.4 m_channelName

```
wxString VideoCard::m_channelName [private]
```

Definition at line 85 of file [VideoCard.hpp](#).

6.46.4.5 m_controller

```
AppController& VideoCard::m_controller [private]
```

Definition at line 80 of file [VideoCard.hpp](#).

6.46.4.6 m_hovered

```
bool VideoCard::m_hovered = false [private]
```

Definition at line 99 of file [VideoCard.hpp](#).

6.46.4.7 m_logoFailed

```
bool VideoCard::m_logoFailed = false [private]
```

Definition at line 96 of file [VideoCard.hpp](#).

6.46.4.8 m_logoGen

```
int VideoCard::m_logoGen = 0 [private]
```

Incremented on each SetVideo/Clear to discard stale fetches.

Definition at line 97 of file [VideoCard.hpp](#).

6.46.4.9 m_logoLoading

```
bool VideoCard::m_logoLoading = false [private]
```

Definition at line 95 of file [VideoCard.hpp](#).

6.46.4.10 m_thumbFailed

```
bool VideoCard::m_thumbFailed = false [private]
```

Definition at line 91 of file [VideoCard.hpp](#).

6.46.4.11 m_thumbGen

```
int VideoCard::m_thumbGen = 0 [private]
```

Incremented on each SetVideo/Clear to discard stale fetches.

Definition at line 92 of file [VideoCard.hpp](#).

6.46.4.12 m_thumbLoading

```
bool VideoCard::m_thumbLoading = false [private]
```

Definition at line 90 of file [VideoCard.hpp](#).

6.46.4.13 m_thumbnail

```
wxBitmap VideoCard::m_thumbnail [private]
```

Definition at line 89 of file [VideoCard.hpp](#).

6.46.4.14 m_thumbnailURL

```
wxString VideoCard::m_thumbnailURL [private]
```

Definition at line 86 of file [VideoCard.hpp](#).

6.46.4.15 m_title

```
wxString VideoCard::m_title [private]
```

Definition at line 83 of file [VideoCard.hpp](#).

6.46.4.16 m_videoID

```
wxString VideoCard::m_videoID [private]
```

Definition at line 82 of file [VideoCard.hpp](#).

The documentation for this class was generated from the following files:

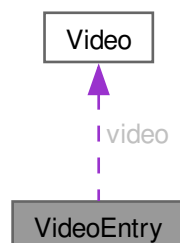
- [VideoCard.hpp](#)
- [VideoCard.cpp](#)

6.47 VideoEntry Class Reference

[VideoEntry](#) class declaration.

```
#include <VideoEntry.hpp>
```

Collaboration diagram for VideoEntry:



Public Member Functions

- [VideoEntry](#) (const [Video](#) &video, int watchCount, int lastWatched)
- [Video](#) [getVideo](#) () const
- int [getWatchCount](#) () const
- int [getLastWatched](#) () const
- void [setWatchCount](#) (int watchCount)
- void [setLastWatched](#) (int lastWatched)
- std::string [toString](#) () const

Private Attributes

- [Video](#) [video](#)
- int [watchCount](#)
- int [lastWatched](#)

6.47.1 Detailed Description

[VideoEntry](#) class declaration.

A [Video](#) decorated with watch-history metadata.

Definition at line 20 of file [VideoEntry.hpp](#).

6.47.2 Constructor & Destructor Documentation

6.47.2.1 VideoEntry()

```
VideoEntry::VideoEntry (  
    const Video & video,  
    int watchCount,  
    int lastWatched)
```

Constructs a [VideoEntry](#).

Parameters

<i>video</i>	The underlying Video object.
<i>watchCount</i>	Number of times the user has watched this video.
<i>lastWatched</i>	Unix timestamp of the most recent watch event.

Definition at line 9 of file [VideoEntry.cpp](#).

6.47.3 Member Function Documentation

6.47.3.1 getLastWatched()

```
int VideoEntry::getLastWatched () const
```

Returns

Unix timestamp of the most recent watch event.

Definition at line 22 of file [VideoEntry.cpp](#).

6.47.3.2 `getVideo()`

```
Video VideoEntry::getVideo () const
```

Returns

The underlying [Video](#).

Definition at line 14 of file [VideoEntry.cpp](#).

6.47.3.3 `getWatchCount()`

```
int VideoEntry::getWatchCount () const
```

Returns

Number of times the user has watched this video.

Definition at line 18 of file [VideoEntry.cpp](#).

6.47.3.4 `setLastWatched()`

```
void VideoEntry::setLastWatched (  
    int lastWatched)
```

Parameters

<i>lastWatched</i>	Updated last-watched Unix timestamp.
--------------------	--------------------------------------

Definition at line 31 of file [VideoEntry.cpp](#).

6.47.3.5 `setWatchCount()`

```
void VideoEntry::setWatchCount (  
    int watchCount)
```

Parameters

<i>watchCount</i>	Updated watch count.
-------------------	----------------------

Definition at line 27 of file [VideoEntry.cpp](#).

6.47.3.6 toString()

```
std::string VideoEntry::toString () const
```

String representation of [VideoEntry](#)

Returns

string

Definition at line 35 of file [VideoEntry.cpp](#).

6.47.4 Member Data Documentation

6.47.4.1 lastWatched

```
int VideoEntry::lastWatched [private]
```

Definition at line 24 of file [VideoEntry.hpp](#).

6.47.4.2 video

```
Video VideoEntry::video [private]
```

Definition at line 22 of file [VideoEntry.hpp](#).

6.47.4.3 watchCount

```
int VideoEntry::watchCount [private]
```

Definition at line 23 of file [VideoEntry.hpp](#).

The documentation for this class was generated from the following files:

- [VideoEntry.hpp](#)
- [VideoEntry.cpp](#)

6.48 YouTubeMetadataFetcher::VideoMeta Struct Reference

[VideoMeta](#) class declaration.

Public Attributes

- std::string [title](#)
- std::string [channelId](#)
- std::string [channelName](#)
- std::string [thumbnailUrl](#)

6.48.1 Detailed Description

[VideoMeta](#) class declaration.

Definition at line 70 of file [YouTubeMetadataFetcher.hpp](#).

6.48.2 Member Data Documentation

6.48.2.1 channelId

```
std::string YouTubeMetadataFetcher::VideoMeta::channelId
```

Definition at line 72 of file [YouTubeMetadataFetcher.hpp](#).

6.48.2.2 channelName

```
std::string YouTubeMetadataFetcher::VideoMeta::channelName
```

Definition at line 73 of file [YouTubeMetadataFetcher.hpp](#).

6.48.2.3 thumbnailUrl

```
std::string YouTubeMetadataFetcher::VideoMeta::thumbnailUrl
```

Definition at line 74 of file [YouTubeMetadataFetcher.hpp](#).

6.48.2.4 title

```
std::string YouTubeMetadataFetcher::VideoMeta::title
```

Definition at line 71 of file [YouTubeMetadataFetcher.hpp](#).

The documentation for this struct was generated from the following file:

- [YouTubeMetadataFetcher.hpp](#)

6.49 WatchHistoryParser Class Reference

[WatchHistoryParser](#) class declaration.

```
#include <WatchHistoryParser.hpp>
```

Public Member Functions

- [WatchHistoryParser](#) (const std::string &filePath)
- std::list< [Video](#) > [parse](#) ()
- std::list< [Video](#) > [parseMultiThreaded](#) (std::function< void(double)> progressCallback=nullptr)
- int [getParserId](#) () const

Static Private Member Functions

- static std::optional< std::string > [extractVideoIdFromHref](#) (const std::string &href)
- static std::list< [Video](#) > [parseRange](#) (const std::string &htmlContent, std::size_t startPos, std::size_t endPos)

Private Attributes

- std::string [filePath](#)
- int [parser_id](#)

6.49.1 Detailed Description

[WatchHistoryParser](#) class declaration.

Definition at line 24 of file [WatchHistoryParser.hpp](#).

6.49.2 Constructor & Destructor Documentation

6.49.2.1 WatchHistoryParser()

```
WatchHistoryParser::WatchHistoryParser (
    const std::string & filePath) [explicit]
```

Definition at line 18 of file [WatchHistoryParser.cpp](#).

6.49.3 Member Function Documentation

6.49.3.1 extractVideoIdFromHref()

```
std::optional< std::string > WatchHistoryParser::extractVideoIdFromHref (
    const std::string & href) [static], [private]
```

Pulls the YouTube video idea from href tags

Parameters

<i>href</i>	tag to extract the id from
-------------	----------------------------

Returns

`string`
of YouTube video id

Definition at line 22 of file [WatchHistoryParser.cpp](#).

6.49.3.2 getParserId()

```
int WatchHistoryParser::getParserId () const
```

Definition at line 109 of file [WatchHistoryParser.cpp](#).

6.49.3.3 parse()

```
std::list< Video > WatchHistoryParser::parse ()
```

Parses watch-history HTML and returns lightweight [Video](#) objects. Only videoID is populated.

Definition at line 65 of file [WatchHistoryParser.cpp](#).

6.49.3.4 parseMultiThreaded()

```
std::list< Video > WatchHistoryParser::parseMultiThreaded (  
    std::function< void(double)> progressCallback = nullptr)
```

Multi-threaded parse using 2-3 worker threads for faster processing. Each thread processes a section of the file to avoid duplicate work.

Parameters

<i>progressCallback</i>	Optional callback to report parsing progress (0.0 to 1.0)
-------------------------	---

Returns

List of unique [Video](#) objects extracted from all threads

Definition at line 113 of file [WatchHistoryParser.cpp](#).

6.49.3.5 parseRange()

```
std::list< Video > WatchHistoryParser::parseRange (  
    const std::string & htmlContent,  
    std::size_t startPos,  
    std::size_t endPos) [static], [private]
```

Parse a range of HTML content.

Parameters

<i>htmlContent</i>	Full HTML file content
<i>startPos</i>	Starting character position for this thread
<i>endPos</i>	Ending character position for this thread

Returns

List of [Video](#) objects found in this section

Definition at line 194 of file [WatchHistoryParser.cpp](#).

6.49.4 Member Data Documentation

6.49.4.1 filePath

```
std::string WatchHistoryParser::filePath [private]
```

Definition at line 45 of file [WatchHistoryParser.hpp](#).

6.49.4.2 parser_id

```
int WatchHistoryParser::parser_id [private]
```

Definition at line 46 of file [WatchHistoryParser.hpp](#).

The documentation for this class was generated from the following files:

- [WatchHistoryParser.hpp](#)
- [WatchHistoryParser.cpp](#)

6.50 WatchLaterParser Class Reference

[WatchLaterParser](#) class declaration.

```
#include <WatchLaterParser.hpp>
```

Public Member Functions

- [WatchLaterParser](#) (const std::string &filePath)
- std::list< [Video](#) > [parse](#) ()
- std::list< [Video](#) > [parseMultiThreaded](#) (std::function< void(double)> progressCallback=nullptr)
- int [getParserId](#) ()

Private Attributes

- std::string [filePath](#)
- int [parser_id](#)

6.50.1 Detailed Description

[WatchLaterParser](#) class declaration.

Specialized parser for YouTube Takeout Watch Later CSV files.

Definition at line 30 of file [WatchLaterParser.hpp](#).

6.50.2 Constructor & Destructor Documentation

6.50.2.1 WatchLaterParser()

```
WatchLaterParser::WatchLaterParser (  
    const std::string & filePath) [explicit]
```

Constructs [WatchLaterParser](#) for the given CSV file.

Parameters

<i>filePath</i>	Path to the YouTube Takeout Watch Later CSV file. Also ensures that the CSV parser id is assigned
-----------------	---

Definition at line 17 of file [WatchLaterParser.cpp](#).

6.50.3 Member Function Documentation

6.50.3.1 getParserId()

```
int WatchLaterParser::getParserId ()
```

Definition at line 47 of file [WatchLaterParser.cpp](#).

6.50.3.2 parse()

```
std::list< Video > WatchLaterParser::parse ()
```

Reads and parses the Watch Later CSV file into [Video](#) objects.

Each returned [Video](#) object has only the videoID field populated; all other fields (title, channelId, etc.) are empty/zero.

Returns

List of [Video](#) objects initialized from Watch Later CSV rows

Definition at line 21 of file [WatchLaterParser.cpp](#).

6.50.3.3 parseMultiThreaded()

```
std::list< Video > WatchLaterParser::parseMultiThreaded (  
    std::function< void(double)> progressCallback = nullptr)
```

Multi-threaded parse using 2-3 worker threads for faster CSV processing. Each thread processes a range of CSV rows to avoid duplicate work.

Parameters

<i>progressCallback</i>	Optional callback to report parsing progress (0.0 to 1.0)
-------------------------	---

Returns

List of [Video](#) objects extracted from all threads

Definition at line 49 of file [WatchLaterParser.cpp](#).

6.50.4 Member Data Documentation

6.50.4.1 filePath

```
std::string WatchLaterParser::filePath [private]
```

Definition at line 58 of file [WatchLaterParser.hpp](#).

6.50.4.2 parser_id

```
int WatchLaterParser::parser_id [private]
```

Definition at line 59 of file [WatchLaterParser.hpp](#).

The documentation for this class was generated from the following files:

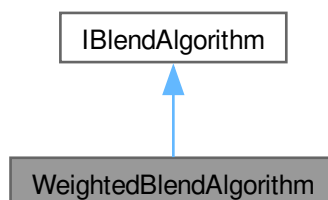
- [WatchLaterParser.hpp](#)
- [WatchLaterParser.cpp](#)

6.51 WeightedBlendAlgorithm Class Reference

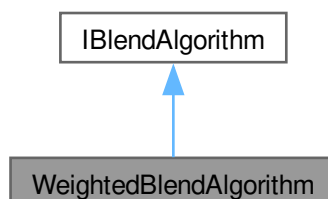
[WeightedBlendAlgorithm](#) class declaration.

```
#include <WeightedBlendAlgorithm.hpp>
```

Inheritance diagram for [WeightedBlendAlgorithm](#):



Collaboration diagram for [WeightedBlendAlgorithm](#):



Public Member Functions

- [WeightedBlendAlgorithm](#) (int [videosPerUser](#)=5)
- [WeightedBlendAlgorithm](#) (const std::vector< double > &weights, int [videosPerUser](#)=5)
- [Blend generateBlend](#) (const std::list< [User](#) > &participants, const std::string &title="") override

Private Member Functions

- void [normalizeWeights](#) (std::vector< double > &weights)
- std::list< [Video](#) > [selectWeightedVideos](#) (const [User](#) &user, double weight)

Private Attributes

- std::vector< double > [participantWeights](#)
- int [videosPerUser](#)

6.51.1 Detailed Description

[WeightedBlendAlgorithm](#) class declaration.

Definition at line 21 of file [WeightedBlendAlgorithm.hpp](#).

6.51.2 Constructor & Destructor Documentation

6.51.2.1 WeightedBlendAlgorithm() [1/2]

```
WeightedBlendAlgorithm::WeightedBlendAlgorithm (
    int videosPerUser = 5) [explicit]
```

Constructs a [WeightedBlendAlgorithm](#) with default weight distribution.

Parameters

<i>videosPerUser</i>	How many videos to select from each participant. If a user has fewer videos than this, all of their videos are included.
----------------------	--

6.51.2.2 WeightedBlendAlgorithm() [2/2]

```
WeightedBlendAlgorithm::WeightedBlendAlgorithm (
    const std::vector< double > & weights,
    int videosPerUser = 5)
```

Constructs a [WeightedBlendAlgorithm](#) with custom participant weights.

Parameters

<i>weights</i>	Vector of weights for each participant, in order. Weights should be normalized or will be normalized internally.
<i>videosPerUser</i>	How many videos to select from each participant.

6.51.3 Member Function Documentation

6.51.3.1 generateBlend()

```
Blend WeightedBlendAlgorithm::generateBlend (  
    const std::list< User > & participants,  
    const std::string & title = "") [override], [virtual]
```

Generates a weighted [Blend](#) from a collection of participants. Each participant contributes videos based on their assigned weight.

Parameters

<i>participants</i>	List of users whose Watch Later data is used as input.
<i>title</i>	User-facing name for the blend (default: empty).

Returns

A new [Blend](#) containing weighted video selections from participants.

Implements [IBlendAlgorithm](#).

6.51.3.2 normalizeWeights()

```
void WeightedBlendAlgorithm::normalizeWeights (  
    std::vector< double > & weights) [private]
```

Normalizes the weight vector so that weights sum to 1.0.

Parameters

<i>weights</i>	Vector of weights to normalize.
----------------	---------------------------------

6.51.3.3 selectWeightedVideos()

```
std::list< Video > WeightedBlendAlgorithm::selectWeightedVideos (  
    const User & user,  
    double weight) [private]
```

Selects videos from a participant based on their weight contribution.

Parameters

<i>user</i>	The participant to select videos from.
<i>weight</i>	The weight factor for this participant (0.0 to 1.0).

Returns

List of selected videos.

6.51.4 Member Data Documentation

6.51.4.1 participantWeights

```
std::vector<double> WeightedBlendAlgorithm::participantWeights [private]
```

Vector of weights assigned to participants.

Definition at line 66 of file [WeightedBlendAlgorithm.hpp](#).

6.51.4.2 videosPerUser

```
int WeightedBlendAlgorithm::videosPerUser [private]
```

Number of videos to attempt to select from each participant.

Definition at line 69 of file [WeightedBlendAlgorithm.hpp](#).

The documentation for this class was generated from the following file:

- [WeightedBlendAlgorithm.hpp](#)

6.52 YouTubeData Class Reference

[YouTubeData](#) class declaration.

```
#include <YouTubeData.hpp>
```

Public Member Functions

- `std::list< VideoEntry > getWatchHistory () const`
- `std::list< Video > getLikedVideos () const`
- `std::list< Channel > getSubscribedChannels () const`
- `std::list< Video > getWatchLaterVideos () const`
- `void setWatchHistory (const std::list< VideoEntry > &watchHistory)`
- `void setLikedVideos (const std::list< Video > &likedVideos)`
- `void setSubscribedChannels (const std::list< Channel > &subscribedChannels)`
- `void setWatchLaterVideos (const std::list< Video > &videos)`
- `std::list< Video > getMostWatchedVideos (int n) const`
- `std::list< Channel > getTopChannels (int n) const`
- `std::string toString () const`

Private Attributes

- `std::list< VideoEntry > watchHistory`
- `std::list< Video > likedVideos`
- `std::list< Channel > subscribedChannels`
- `std::list< Video > watchLaterVideos`

6.52.1 Detailed Description

[YouTubeData](#) class declaration.

Container for all YouTube data associated with a [User](#).

Definition at line 29 of file [YouTubeData.hpp](#).

6.52.2 Constructor & Destructor Documentation

6.52.2.1 [YouTubeData\(\)](#)

```
YouTubeData::YouTubeData ()
```

Definition at line 13 of file [YouTubeData.cpp](#).

6.52.3 Member Function Documentation

6.52.3.1 [getLikedVideos\(\)](#)

```
std::list< Video > YouTubeData::getLikedVideos () const
```

Returns

List of videos the user has liked.

Definition at line 20 of file [YouTubeData.cpp](#).

6.52.3.2 [getMostWatchedVideos\(\)](#)

```
std::list< Video > YouTubeData::getMostWatchedVideos (  
    int n) const
```

Returns the n most-replayed videos from the watch history.

Parameters

<i>n</i>	Maximum number of results to return.
----------	--------------------------------------

Returns

List of up to n Videos sorted by descending watch count.

Definition at line 52 of file [YouTubeData.cpp](#).

6.52.3.3 getSubscribedChannels()

```
std::list< Channel > YouTubeData::getSubscribedChannels () const
```

Returns

List of channels the user is subscribed to.

Definition at line 24 of file [YouTubeData.cpp](#).

6.52.3.4 getTopChannels()

```
std::list< Channel > YouTubeData::getTopChannels (  
    int n) const
```

Returns the n most frequently watched channels from the watch history.

Parameters

<i>n</i>	Maximum number of results to return.
----------	--------------------------------------

Returns

List of up to n Channels sorted by descending view count.

Definition at line 72 of file [YouTubeData.cpp](#).

6.52.3.5 getWatchHistory()

```
std::list< VideoEntry > YouTubeData::getWatchHistory () const
```

Returns

Full watch-history list ([VideoEntry](#) objects with watch counts).

Definition at line 16 of file [YouTubeData.cpp](#).

6.52.3.6 getWatchLaterVideos()

```
std::list< Video > YouTubeData::getWatchLaterVideos () const
```

Returns

Watch Later playlist videos.

Definition at line 42 of file [YouTubeData.cpp](#).

6.52.3.7 setLikedVideos()

```
void YouTubeData::setLikedVideos (  
    const std::list< Video > & likedVideos)
```

Parameters

<i>likedVideos</i>	Replaces the liked-videos list.
--------------------	---------------------------------

Definition at line 34 of file [YouTubeData.cpp](#).

6.52.3.8 setSubscribedChannels()

```
void YouTubeData::setSubscribedChannels (
    const std::list< Channel > & subscribedChannels)
```

Parameters

<i>subscribedChannels</i>	Replaces the subscribed-channels list.
---------------------------	--

Definition at line 38 of file [YouTubeData.cpp](#).

6.52.3.9 setWatchHistory()

```
void YouTubeData::setWatchHistory (
    const std::list< VideoEntry > & watchHistory)
```

Parameters

<i>watchHistory</i>	Replaces the full watch-history list.
---------------------	---------------------------------------

Definition at line 30 of file [YouTubeData.cpp](#).

6.52.3.10 setWatchLaterVideos()

```
void YouTubeData::setWatchLaterVideos (
    const std::list< Video > & videos)
```

Parameters

<i>videos</i>	Replaces the Watch Later playlist.
---------------	------------------------------------

Definition at line 46 of file [YouTubeData.cpp](#).

6.52.3.11 toString()

```
std::string YouTubeData::toString () const
```

Returns

JSON string representation of this [YouTubeData](#) object.

Definition at line 99 of file [YouTubeData.cpp](#).

6.52.4 Member Data Documentation

6.52.4.1 likedVideos

```
std::list<Video> YouTubeData::likedVideos [private]
```

Definition at line 32 of file [YouTubeData.hpp](#).

6.52.4.2 subscribedChannels

```
std::list<Channel> YouTubeData::subscribedChannels [private]
```

Definition at line 33 of file [YouTubeData.hpp](#).

6.52.4.3 watchHistory

```
std::list<VideoEntry> YouTubeData::watchHistory [private]
```

Definition at line 31 of file [YouTubeData.hpp](#).

6.52.4.4 watchLaterVideos

```
std::list<Video> YouTubeData::watchLaterVideos [private]
```

Definition at line 34 of file [YouTubeData.hpp](#).

The documentation for this class was generated from the following files:

- [YouTubeData.hpp](#)
- [YouTubeData.cpp](#)

6.53 YouTubeDataImporter Class Reference

[YouTubeDataImporter](#) class declaration.

```
#include <YouTubeDataImporter.hpp>
```

Public Member Functions

- `std::list< Video > import` (const std::string &filePath) const
- `std::list< Video > importMultiThreaded` (const std::string &filePath, std::function< void(double)> progress↔ Callback=nullptr) const

Static Private Member Functions

- static std::string `lowerExtension` (const std::string &filePath)

6.53.1 Detailed Description

[YouTubeDataImporter](#) class declaration.

Definition at line 21 of file [YouTubeDataImporter.hpp](#).

6.53.2 Member Function Documentation

6.53.2.1 import()

```
std::list< Video > YouTubeDataImporter::import (
    const std::string & filePath) const
```

Imports YouTube data from a supported file path.

Supported extensions:

- .csv : Watch Later export
- .html/.htm : watch-history export

Parameters

<i>filePath</i>	path to the specified file
-----------------	----------------------------

Returns

```
list<video>
extracted and formatted from
filePath
```

Definition at line 28 of file [YouTubeDataImporter.cpp](#).

6.53.2.2 importMultiThreaded()

```
std::list< Video > YouTubeDataImporter::importMultiThreaded (
    const std::string & filePath,
    std::function< void(double)> progressCallback = nullptr) const
```

Multi-threaded import with progress feedback. Uses 3 worker threads to process the file faster.

Parameters

<i>filePath</i>	Path to YouTube export file
<i>progressCallback</i>	Optional callback receiving progress 0.0-1.0

Returns

List of [Video](#) objects extracted from file (ID-only)

Definition at line 44 of file [YouTubeDataImporter.cpp](#).

6.53.2.3 lowerExtension()

```
std::string YouTubeDataImporter::lowerExtension (
    const std::string & filePath) [static], [private]
```

Helper to ensure extensions are always lowered

Parameters

<i>filePath</i>	desired file
-----------------	--------------

Returns

`string`
of the file path with the lowered extension

Definition at line 16 of file [YouTubeDataImporter.cpp](#).

The documentation for this class was generated from the following files:

- [YouTubeDataImporter.hpp](#)
- [YouTubeDataImporter.cpp](#)

6.54 YouTubeMetadataFetcher Class Reference

[YouTubeMetadataFetcher](#) class declaration.

```
#include <YouTubeMetadataFetcher.hpp>
```

Classes

- struct [VideoMeta](#)
[VideoMeta](#) class declaration.

Public Member Functions

- [YouTubeMetadataFetcher](#) (std::string apiKey)
- std::list< [Video](#) > [enrich](#) (const std::list< [Video](#) > &videos)

Private Member Functions

- std::unordered_map< std::string, [VideoMeta](#) > [fetchVideoBatch](#) (const std::vector< std::string > &ids)
- std::unordered_map< std::string, std::string > [fetchChannelLogos](#) (const std::vector< std::string > &channelIds)
- std::string [httpGet](#) (const std::string &url)

Static Private Member Functions

- static size_t [writeCallback](#) (void *contents, size_t size, size_t nmemb, std::string *out)

Private Attributes

- std::string [apiKey](#)

Static Private Attributes

- static constexpr int `kBatchSize` = 50

6.54.1 Detailed Description

[YouTubeMetadataFetcher](#) class declaration.

Definition at line 50 of file [YouTubeMetadataFetcher.hpp](#).

6.54.2 Constructor & Destructor Documentation

6.54.2.1 YouTubeMetadataFetcher()

```
YouTubeMetadataFetcher::YouTubeMetadataFetcher (  
    std::string apiKey) [explicit]
```

Definition at line 23 of file [YouTubeMetadataFetcher.cpp](#).

6.54.3 Member Function Documentation

6.54.3.1 enrich()

```
std::list< Video > YouTubeMetadataFetcher::enrich (  
    const std::list< Video > & videos)
```

Enriches a list of Videos that only have videoID populated. Returns a new list where each [Video](#) has title, channelID, channelName, thumbnailURL, and channelLogoURL filled in from the API.

Definition at line 28 of file [YouTubeMetadataFetcher.cpp](#).

6.54.3.2 fetchChannelLogos()

```
std::unordered_map< std::string, std::string > YouTubeMetadataFetcher::fetchChannelLogos (  
    const std::vector< std::string > & channelIds) [private]
```

Definition at line 163 of file [YouTubeMetadataFetcher.cpp](#).

6.54.3.3 fetchVideoBatch()

```
std::unordered_map< std::string, YouTubeMetadataFetcher::VideoMeta > YouTubeMetadataFetcher↔  
::fetchVideoBatch (  
    const std::vector< std::string > & ids) [private]
```

Definition at line 112 of file [YouTubeMetadataFetcher.cpp](#).

6.54.3.4 httpGet()

```
std::string YouTubeMetadataFetcher::httpGet (  
    const std::string & url) [private]
```

Definition at line 207 of file [YouTubeMetadataFetcher.cpp](#).

6.54.3.5 writeCallback()

```
size_t YouTubeMetadataFetcher::writeCallback (  
    void * contents,  
    size_t size,  
    size_t nmemb,  
    std::string * out) [static], [private]
```

Definition at line 234 of file [YouTubeMetadataFetcher.cpp](#).

6.54.4 Member Data Documentation

6.54.4.1 apiKey

```
std::string YouTubeMetadataFetcher::apiKey [private]
```

Definition at line 62 of file [YouTubeMetadataFetcher.hpp](#).

6.54.4.2 kBatchSize

```
int YouTubeMetadataFetcher::kBatchSize = 50 [static], [constexpr], [private]
```

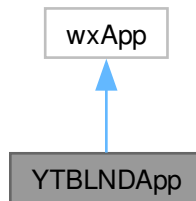
Definition at line 64 of file [YouTubeMetadataFetcher.hpp](#).

The documentation for this class was generated from the following files:

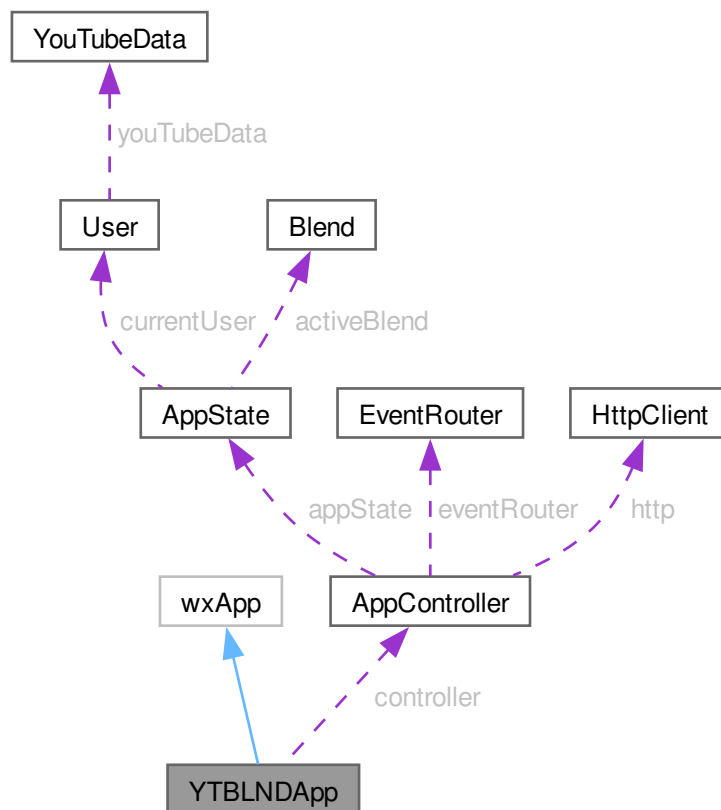
- [YouTubeMetadataFetcher.hpp](#)
- [YouTubeMetadataFetcher.cpp](#)

6.55 YTBLNDApp Class Reference

Inheritance diagram for YTBLNDApp:



Collaboration diagram for YTBLNDApp:



Public Member Functions

- bool [OnInit](#) () override

- int [OnExit](#) () override

Private Attributes

- [AppController](#) controller

6.55.1 Detailed Description

Definition at line 26 of file [main.cpp](#).

6.55.2 Member Function Documentation

6.55.2.1 OnExit()

```
int YTBLNDApp::OnExit () [override]
```

Definition at line 56 of file [main.cpp](#).

6.55.2.2 OnInit()

```
bool YTBLNDApp::OnInit () [override]
```

Definition at line 39 of file [main.cpp](#).

6.55.3 Member Data Documentation

6.55.3.1 controller

```
AppController YTBLNDApp::controller [private]
```

Definition at line 34 of file [main.cpp](#).

The documentation for this class was generated from the following file:

- [main.cpp](#)

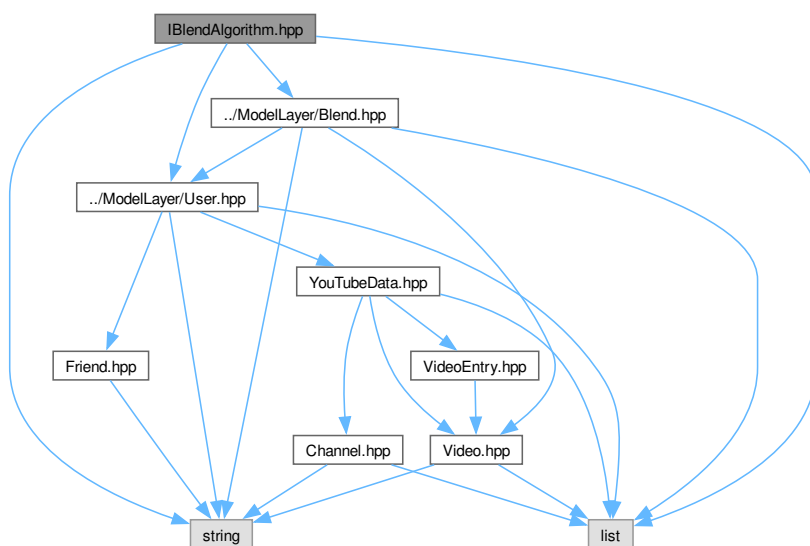
Chapter 7

File Documentation

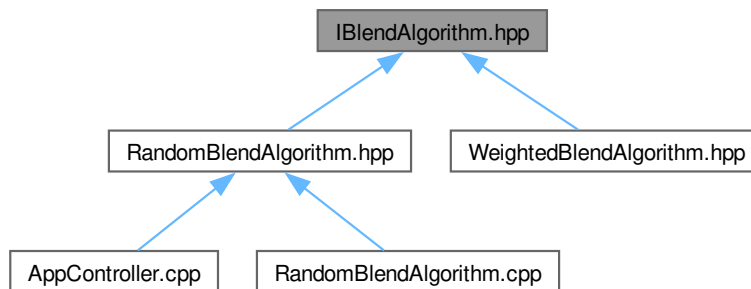
7.1 IBlendAlgorithm.hpp File Reference

Abstract interface for blend-generation algorithms.

```
#include <list>
#include <string>
#include "../ModelLayer/User.hpp"
#include "../ModelLayer/Blend.hpp"
Include dependency graph for IBlendAlgorithm.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [IBlendAlgorithm](#)
[IBlendAlgorithm](#) class declaration.

7.1.1 Detailed Description

Abstract interface for blend-generation algorithms.

Author

Jasmine Kumar

All blend algorithms must implement this interface so [AppController](#) can swap strategies without changing any other code.

Definition in file [IBlendAlgorithm.hpp](#).

7.2 IBlendAlgorithm.hpp

[Go to the documentation of this file.](#)

```

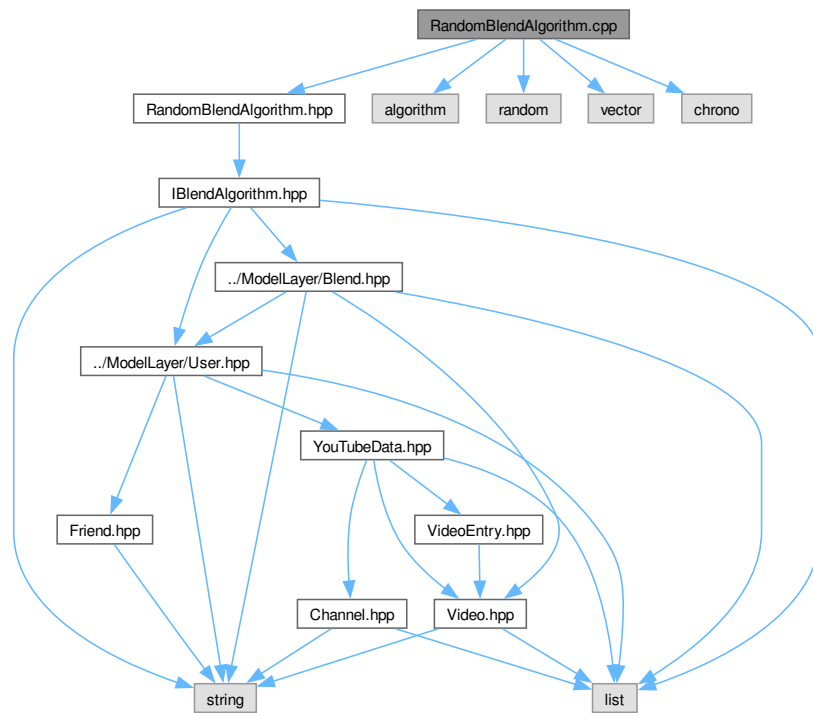
00001 #ifndef IBLEND_ALGORITHM_H
00002 #define IBLEND_ALGORITHM_H
00003
00012
00013 #include <list>
00014 #include <string>
00015 #include "../ModelLayer/User.hpp"
00016 #include "../ModelLayer/Blend.hpp"
00017
00023 class IBlendAlgorithm {
00024 public:
00025     virtual ~IBlendAlgorithm() = default;
00026
00033     virtual Blend generateBlend(const std::list<User>& participants,
00034                                const std::string& title = "") = 0;
00035 };
00036
00037 #endif
  
```

7.3 RandomBlendAlgorithm.cpp File Reference

Implementation for [RandomBlendAlgorithm](#).

```
#include "RandomBlendAlgorithm.hpp"
#include <algorithm>
#include <random>
#include <vector>
#include <chrono>
```

Include dependency graph for RandomBlendAlgorithm.cpp:



7.3.1 Detailed Description

Implementation for [RandomBlendAlgorithm](#).

Author

Jasmine Kumar

Definition in file [RandomBlendAlgorithm.cpp](#).

7.4 RandomBlendAlgorithm.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "RandomBlendAlgorithm.hpp"
00008
00009 #include <algorithm>
00010 #include <random>
00011 #include <vector>
00012 #include <chrono>
00013
00014 RandomBlendAlgorithm::RandomBlendAlgorithm(int videosPerUser)
00015     : videosPerUser(videosPerUser) {}
00016
00017 Blend RandomBlendAlgorithm::generateBlend(const std::list<User>& participants,
00018                                           const std::string& title) {
00019     std::list<Video> combined;
00020
00021     // Seed RNG from the system clock so each blend is different
00022     unsigned seed = static_cast<unsigned>(
00023         std::chrono::steady_clock::now().time_since_epoch().count());
00024     std::mt19937 rng(seed);
00025
00026     // collect a global pool of available videos (for filling empty slots)
00027     std::vector<Video> globalPool;
00028
00029     for (const User& user : participants) {
00030         std::list<Video> watchLater = user.getYouTubeData().getWatchLaterVideos();
00031
00032         // Copy to vector for random_shuffle / sample
00033         std::vector<Video> pool(watchLater.begin(), watchLater.end());
00034         std::shuffle(pool.begin(), pool.end(), rng);
00035
00036         int poolSize = static_cast<int>(pool.size());
00037         // add pool contents to global pool for later filling
00038         if (poolSize > 0) {
00039             globalPool.insert(globalPool.end(), pool.begin(), pool.end());
00040             // Fill videosPerUser slots for this user (allow duplicates by wrapping)
00041             for (int i = 0; i < videosPerUser; ++i) {
00042                 combined.push_back(pool[i % poolSize]);
00043             }
00044         }
00045     }
00046
00047     // If some users had no videos, fill remaining slots from the global pool
00048     int desiredTotal = static_cast<int>(participants.size()) * videosPerUser;
00049     if (static_cast<int>(combined.size()) < desiredTotal && !globalPool.empty()) {
00050         std::uniform_int_distribution<size_t> dist(0, globalPool.size() - 1);
00051         while (static_cast<int>(combined.size()) < desiredTotal) {
00052             combined.push_back(globalPool[dist(rng)]);
00053         }
00054     }
00055
00056     // Shuffle final selection so videos are mixed across users
00057     std::vector<Video> finalVec(combined.begin(), combined.end());
00058     std::shuffle(finalVec.begin(), finalVec.end(), rng);
00059     combined.assign(finalVec.begin(), finalVec.end());
00060
00061     // Build a simple blendID from participant IDs
00062     std::string blendID;
00063     for (const User& user : participants) {
00064         if (!blendID.empty()) blendID += "_";
00065         blendID += user.getUserID();
00066     }
00067     blendID += "_random";
00068
00069     return Blend(blendID, title, "random", participants, combined);
00070 }

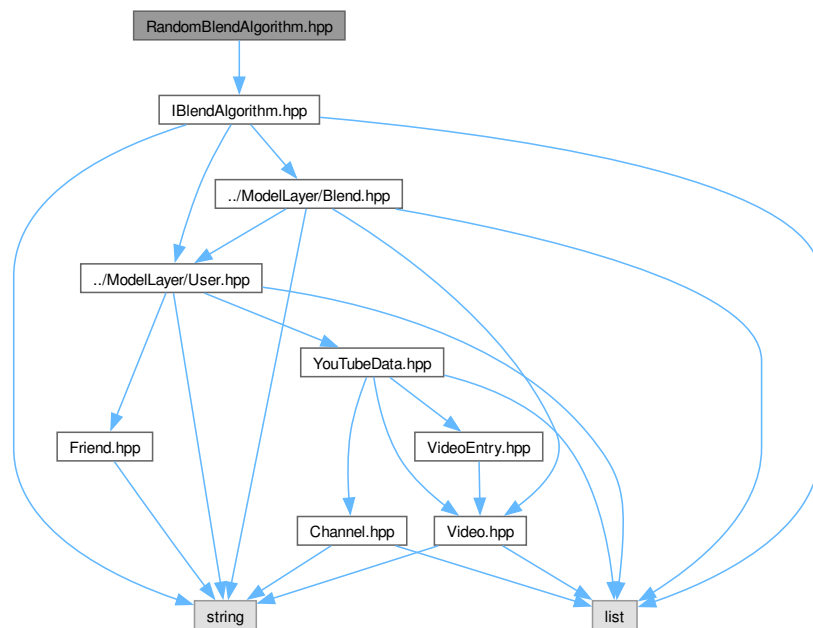
```

7.5 RandomBlendAlgorithm.hpp File Reference

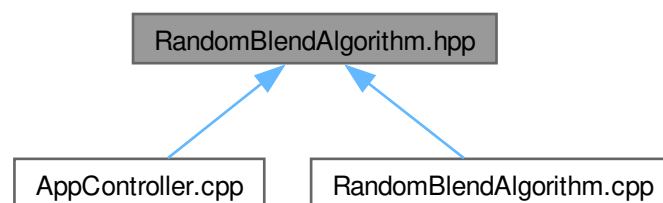
Random [Blend](#) implementation of [IBlendAlgorithm](#).

```
#include "IBlendAlgorithm.hpp"
```

Include dependency graph for RandomBlendAlgorithm.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [RandomBlendAlgorithm](#)
RandomBlendAlgorithm class declaration.

7.5.1 Detailed Description

Random [Blend](#) implementation of [IBlendAlgorithm](#).

Author

Jasmine Kumar

Blend algorithm that picks a random subset of videos from each participant's Watch Later list and combines them into one **Blend**.

Definition in file [RandomBlendAlgorithm.hpp](#).

7.6 RandomBlendAlgorithm.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef RANDOM_BLEND_ALGORITHM_H
00002 #define RANDOM_BLEND_ALGORITHM_H
00003
00004 #include "IBlendAlgorithm.hpp"
00005
00018 class RandomBlendAlgorithm : public IBlendAlgorithm {
00019 public:
00025     explicit RandomBlendAlgorithm(int videosPerUser = 5);
00026
00030     Blend generateBlend(const std::list<User>& participants,
00031                       const std::string& title = "") override;
00032
00033 private:
00034     int videosPerUser;
00035 };
00036
00037 #endif

```

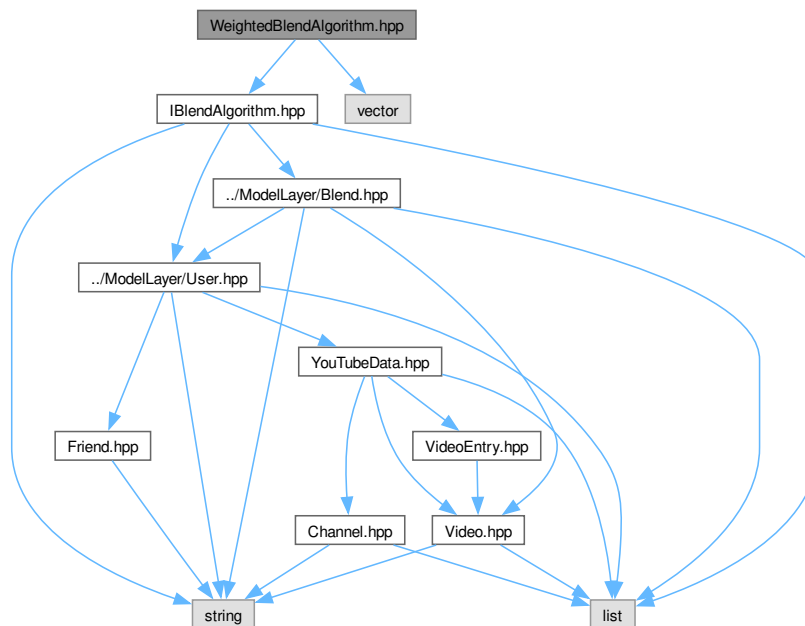
7.7 WeightedBlendAlgorithm.hpp File Reference

Weighted **Blend** implementation of [IBlendAlgorithm](#).

```
#include "IBlendAlgorithm.hpp"
```

```
#include <vector>
```

Include dependency graph for WeightedBlendAlgorithm.hpp:



Classes

- class [WeightedBlendAlgorithm](#)
WeightedBlendAlgorithm class declaration.

7.7.1 Detailed Description

Weighted [Blend](#) implementation of [IBlendAlgorithm](#).

Author

Shamar Pennant

Jasmine Kumar

[Blend](#) algorithm that generates a blend by weighing contributions from each participant. Participants with higher weights will have more of their videos selected for the final blend.

Definition in file [WeightedBlendAlgorithm.hpp](#).

7.8 WeightedBlendAlgorithm.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef WEIGHTED_BLEND_ALGORITHM_H
00002 #define WEIGHTED_BLEND_ALGORITHM_H
00003
00004 #include "IBlendAlgorithm.hpp"
00005 #include <vector>
00006
00021 class WeightedBlendAlgorithm : public IBlendAlgorithm {
00022 public:
00029     explicit WeightedBlendAlgorithm(int videosPerUser = 5);
00030
00037     WeightedBlendAlgorithm(const std::vector<double>& weights, int videosPerUser = 5);
00038
00047     Blend generateBlend(const std::list<User>& participants,
00048                       const std::string& title = "") override;
00049
00050 private:
00055     void normalizeWeights(std::vector<double>& weights);
00056
00063     std::list<Video> selectWeightedVideos(const User& user, double weight);
00064
00066     std::vector<double> participantWeights;
00067
00069     int videosPerUser;
00070 };
00071
00072 #endif

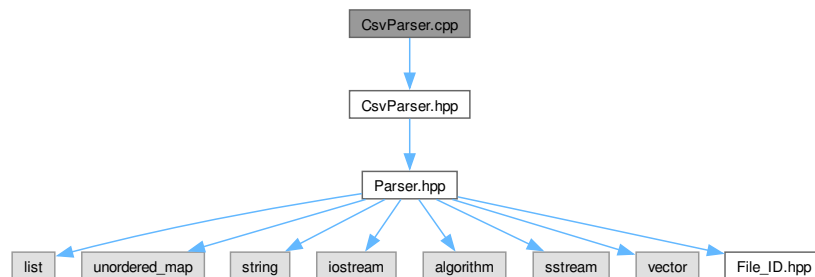
```

7.9 CsvParser.cpp File Reference

Implementation for [CsvParser](#).

```
#include "CsvParser.hpp"
```

Include dependency graph for CsvParser.cpp:



7.9.1 Detailed Description

Implementation for [CsvParser](#).

Author

Shamar Pennant

Definition in file [CsvParser.cpp](#).

7.10 CsvParser.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007
00008
00009 #include "CsvParser.hpp"
00010
00011 using namespace std;
00012
00013 CsvParser::CsvParser() {parse_id = File_ID::CSV;}
00014 CsvParser::CsvParser(list<string> raw_data) : raw_data(raw_data) {
00015     parse_id = File_ID::CSV;
00016 }
00017 CsvParser::~CsvParser() {}
00018
00019 list<unordered_map<string, string> > CsvParser::parse() {
00020     // check if raw_data is empty
00021     if (raw_data.empty()) {
00022         cerr << "error: raw data string is empty in CsvParser";
00023         exit(1);
00024     }
00025
00026     // returned value at end of function
00027     list<unordered_map<string, string> > ret;
00028
00029     // header contents
00030     vector<string> col_titles;
00031
00032     int line_num = 0;
00033     // loop through the raw_data list until it's empty
00034     while (!raw_data.empty()) {
00035         // grab the next line and advance
00036         string line = raw_data.front();
00037         raw_data.pop_front();
00038     }
  
```

```

00039         if (line_num == 0) {
00040             /*
00041              * When on the first line (the col titles)
00042              * save their names within col_titles to be
00043              * used later. formatting each dictionary
00044              * contents as key = col_title and value =
00045              * row_entry
00046              */
00047             stringstream temp_ss(line);
00048             string t = "";
00049             while (getline(temp_ss, t, ',')) {
00050                 // trim any white space/newline char
00051                 if (!t.empty() && t.back() == '\\n') {t.pop_back();}
00052                 // save col
00053                 col_titles.push_back(t);
00054             }
00055         } else { // for any other line, they're normal rows
00056             int num_col = 0;
00057             unordered_map<string, string> row_contents;
00058
00059             /* delimit the line/row and place key value pay
00060              * <"col_name", "row_value"> inside the dictionary*/
00061             stringstream temp_ss(line);
00062             string t = "";
00063             while (getline(temp_ss, t, ',')) {
00064                 if (!t.empty() && t.back() == '\\n') {t.pop_back();}
00065                 row_contents[col_titles.at(num_col++)] = t;
00066             }
00067
00068             // add the dictionary as a line to ret
00069             ret.push_back(row_contents);
00070         }
00071     }
00072     line_num++;
00073 }
00074
00075 return ret;
00076 }
00077
00078 void CsvParser::setData(list<string> data) {this->raw_data = data;}
00079 int CsvParser::getParseId() {return parse_id;}

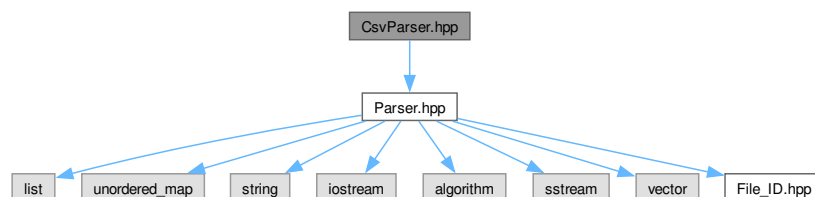
```

7.11 CsvParser.hpp File Reference

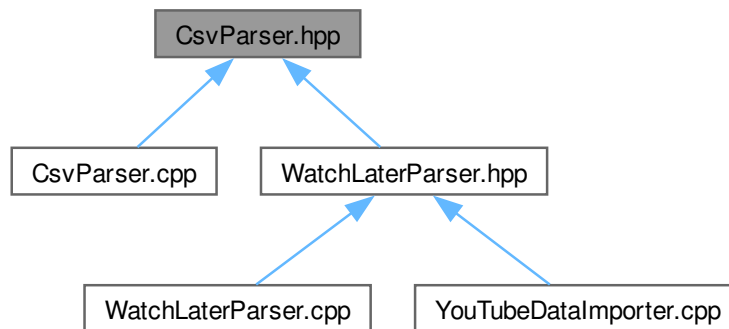
CSV file parser implementing the [Parser](#) interface.

```
#include "Parser.hpp"
```

Include dependency graph for CsvParser.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [CsvParser](#)
CsvParser class declaration.

7.11.1 Detailed Description

CSV file parser implementing the [Parser](#) interface.

Author

Shamar Pennant

Parses comma-separated values into a `list<Dict<string,string>>` format where the first row is treated as column headers.

Definition in file [CsvParser.hpp](#).

7.12 CsvParser.hpp

[Go to the documentation of this file.](#)

```

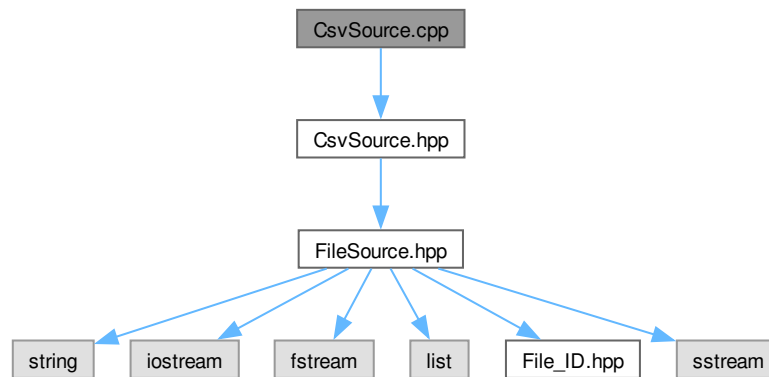
00001
00009
00010 #pragma once
00011
00012 #include "Parser.hpp"
00013
00019 class CsvParser : public Parser {
00020     public:
00021         CsvParser();
00026         CsvParser(std::list<std::string> raw_data);
00027         ~CsvParser();
00028
00034         std::list<std::unordered_map<std::string, std::string>> parse() override;
00035
00040         void setData(std::list<std::string> data) override;
00041
00043         int getParseId() override;
00044
00045     private:
00046         std::list<std::string> raw_data;
00047         int parse_id;
00048 };
  
```

7.13 CsvSource.cpp File Reference

Implementation for [CsvSource](#).

```
#include "CsvSource.hpp"
```

Include dependency graph for CsvSource.cpp:



7.13.1 Detailed Description

Implementation for [CsvSource](#).

Author

Shamar Pennant

Definition in file [CsvSource.cpp](#).

7.14 CsvSource.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "CsvSource.hpp"
00008
00009 using namespace std;
00010
00011 CsvSource::CsvSource(string src) : src(src) {
00012     source_id = File_ID::CSV;
00013 }
00014
00015 list<string> CsvSource::read() {
00016     if (src.empty()) { // in the event that src is empty
00017         cerr << "error: src string is empty in CsvSource";
00018         exit(1);
00019     }
00020
00021     // returned string container
00022     list<string> ret;
00023
00024     // creating (in) file stream with given file source path
  
```

```

00025     fstream read_file_in(src, ios::in);
00026     if (!read_file_in.is_open()) {
00027         cerr << "unable to open " << src << " within CsvSource" << endl;
00028         exit(1);
00029     }
00030
00031     if (!read_file_in.is_open()) {
00032         cerr << "error: could not open file: " << src;
00033         exit(1);
00034     }
00035
00036     // holds file line
00037     string line = "";
00038
00039     // reading the opened file stream into a temporary string
00040     while (getline(read_file_in, line)) {ret.push_back(line);}
00041
00042     return ret;
00043 }
00044
00045 void CsvSource::setSource(string src) {this->src = src;}
00046 int CsvSource::getSourceId() {return source_id;}

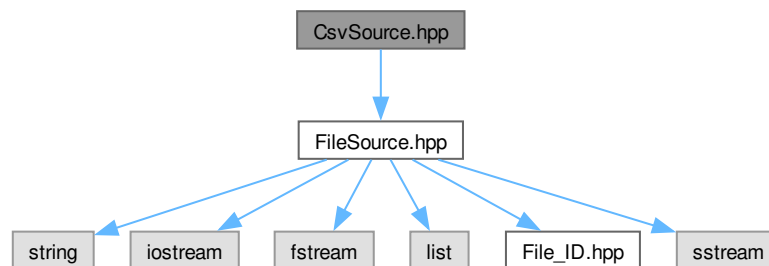
```

7.15 CsvSource.hpp File Reference

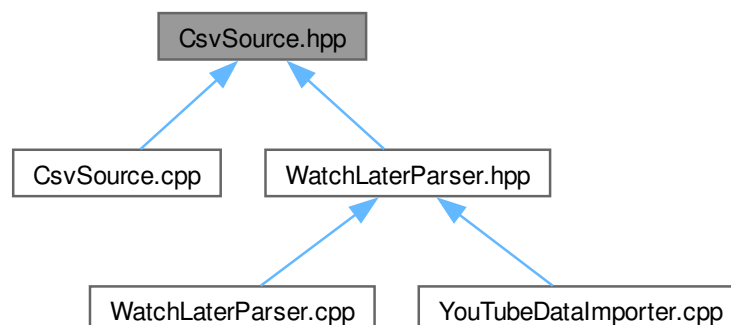
CSV file reader implementing the [FileSource](#) interface.

```
#include "FileSource.hpp"
```

Include dependency graph for CsvSource.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [CsvSource](#)
CsvSource class declaration.

7.15.1 Detailed Description

CSV file reader implementing the [FileSource](#) interface.

Author

Shamar Pennant

Reads a CSV file line-by-line and returns each line as a string in a list, ready to be handed to a [CsvParser](#).

Definition in file [CsvSource.hpp](#).

7.16 CsvSource.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #pragma once
00011
00012 #include "FileSource.hpp"
00013
00018 class CsvSource : public FileSource {
00019     public:
00024         CsvSource(std::string src);
00025         ~CsvSource() override = default;
00026
00031         std::list<std::string> read() override;
00032
00037         void setSource(std::string src) override;
00038
00040         int getSourceId() override;
00041
00042     private:
00043         std::string src;
00044         int source_id;
00045 };

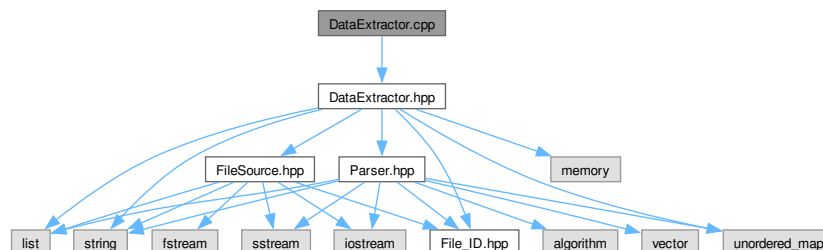
```

7.17 DataExtractor.cpp File Reference

[DataExtractor](#) class implementation.

```
#include "DataExtractor.hpp"
```

Include dependency graph for DataExtractor.cpp:



7.17.1 Detailed Description

[DataExtractor](#) class implementation.

Author

Shamar Pennant

Implements the [DataExtractor](#) class which coordinates [FileSource](#) and [Parser](#) objects to read and parse file data into structured format.

Definition in file [DataExtractor.cpp](#).

7.18 DataExtractor.cpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #include "DataExtractor.hpp"
00011
00012 using namespace std;
00013
00014 DataExtractor::DataExtractor(unique_ptr<FileSource> src, unique_ptr<Parser> p) {
00015     // change ownership of the param to this objects instance
00016     src_format = move(src);
00017     par_format = move(p);
00018 }
00019
00020 DataExtractor::~DataExtractor() {}
00021
00022 list<unordered_map<string, string>> DataExtractor::extract() {
00023     // grab the respective file format ids
00024     int source_id_in_use = src_format->getSourceId();
00025     int parse_id_in_use = par_format->getParseId();
00026
00027     // check if either id's are out of bounds (they shouldn't be)
00028     if (source_id_in_use <= -1 || parse_id_in_use <= -1) {
00029         cerr << "One of the provided ID's is out of bounds:\nsrc_id = " << source_id_in_use <<
00030             "\nparse_id = " << parse_id_in_use;
00031         exit(1);
00032     }
00033
00034     // check if id's match (how we know they match formats)
00035     if (source_id_in_use != parse_id_in_use) {
00036         cerr << "selected FileSource and Parser id's don't match: \nsrc_id = " << source_id_in_use <<
00037             "\nparse_id = " << parse_id_in_use;
00038         exit(1);
00039     }
00040
00041     // ensure neither pointers are null
00042     if (src_format == nullptr || par_format == nullptr) {
00043         cerr << "Unable to find FileSource or Parser in extract():\nsrc_id = "
00044             << source_id_in_use << ", " << (src_format == nullptr ? "contains nothing (nullptr)" : "has
obj")
00045             << "\nparse_id = "
00046             << parse_id_in_use << ", " << (par_format == nullptr ? "contains nothing (nullptr)" : "has
obj");
00047         exit(1);
00048     }
00049
00050     // --- assuming the both source and parser are already configured ---
00051
00052     // grab the raw data from the file
00053     auto raw_data = src_format->read();
00054     // feed the raw data into the given parser
00055     par_format->setData(raw_data);
00056     // format the data with the repsective parser
00057     list<unordered_map<string, string>> formatted_data = par_format->parse();
00058
00059     return formatted_data;
00060 }
00061
00062 void DataExtractor::setParser(unique_ptr<Parser> parser) {
00063     par_format = move(parser);
00064 }
00065
00066 void DataExtractor::setSource(unique_ptr<FileSource> src) {
00067     src_format = move(src);
00068 }

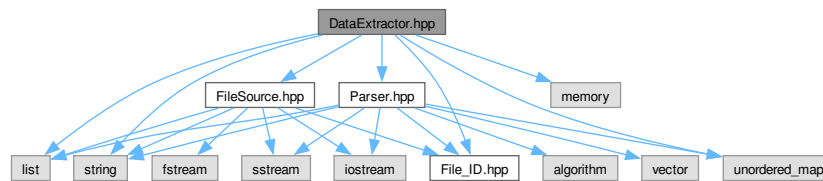
```


7.19 DataExtractor.hpp File Reference

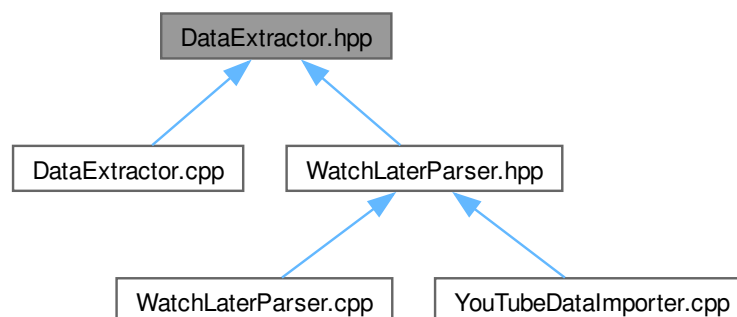
Extracts raw file data and parses it into structured format.

```
#include <list>
#include <memory>
#include <string>
#include <unordered_map>
#include "FileSource.hpp"
#include "Parser.hpp"
#include "File_ID.hpp"
```

Include dependency graph for DataExtractor.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [DataExtractor](#)
DataExtractor class declaration.

7.19.1 Detailed Description

Extracts raw file data and parses it into structured format.

Author

Shamar Pennant

Coordinates file reading (via [FileSource](#)) and data parsing (via [Parser](#)) to produce a consistent list<Dict<str,str>> format. Validates that [FileSource](#) and [Parser](#) objects have matching file format IDs before processing.

Example usage:

```
auto source = std::make_unique<CsvSource>("data.csv");
auto parser = std::make_unique<CsvParser>();
DataExtractor extractor(std::move(source), std::move(parser));
auto data = extractor.extract();
```

Definition in file [DataExtractor.hpp](#).

7.20 DataExtractor.hpp

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00003 #include <list>
00004 #include <memory>
00005 #include <string>
00006 #include <unordered_map>
00007
00008 #include "FileSource.hpp"
00009 #include "Parser.hpp"
00010 #include "File_ID.hpp"
00011 #include <memory>
00012
00035 class DataExtractor {
00036     public:
00042         DataExtractor(std::unique_ptr<FileSource> src, std::unique_ptr<Parser> p);
00043
00044         ~DataExtractor();
00045
00046         std::list<std::unordered_map<std::string, std::string>> extract();
00056
00057         void setSource(std::unique_ptr<FileSource> src);
00061
00062         void setParser(std::unique_ptr<Parser> p);
00066
00067     private:
00068         std::unique_ptr<FileSource> src_format;
00069         std::unique_ptr<Parser> par_format;
00070
00071 };
```

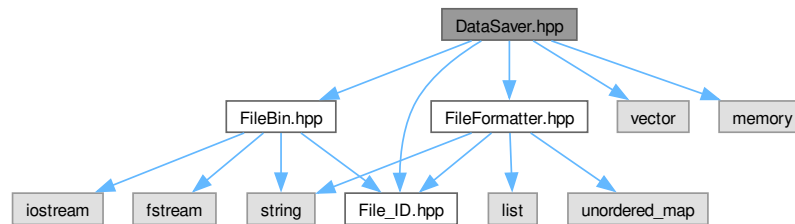
7.21 DataSaver.hpp File Reference

Coordinates [FileFormatter](#) and [FileBin](#) to persist structured data to files.

```
#include "FileBin.hpp"
#include "FileFormatter.hpp"
#include <vector>
#include <memory>
```

```
#include "File_ID.hpp"
```

Include dependency graph for DataSaver.hpp:



Classes

- class [DataSaver](#)
DataSaver class declaration.

7.21.1 Detailed Description

Coordinates [FileFormatter](#) and [FileBin](#) to persist structured data to files.

Author

Shamar Pennant

Maintains collections of [FileBin](#) and [FileFormatter](#) objects to support saving `list<Dict<string,string>>` data in multiple file formats simultaneously.

Definition in file [DataSaver.hpp](#).

7.22 DataSaver.hpp

[Go to the documentation of this file.](#)

```

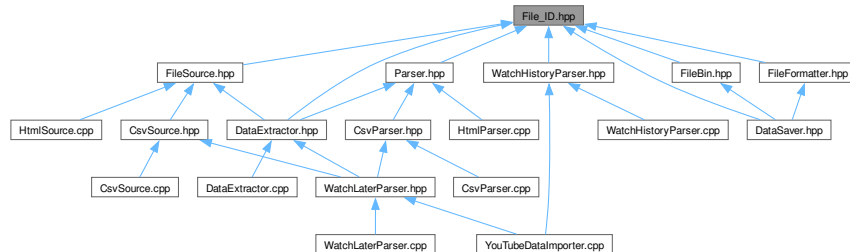
00001 #pragma once
00002
00011
00012 #include "FileBin.hpp"
00013 #include "FileFormatter.hpp"
00014 #include <vector>
00015 #include <memory>
00016 #include "File_ID.hpp"
00017
00022 class DataSaver {
00023     public:
00029         DataSaver(FileBin b, FileFormatter ff);
00030
00031         ~DataSaver();
00032
00038         int save(std::list<std::unordered_map<std::string, std::string>> data);
00039
00044         void setFileBin(FileBin b);
00045
00050         void setFileFormatter(FileFormatter ff);
00051
00052     private:
00053         std::vector<std::unique_ptr<FileBin>> bins;
00054         std::vector<std::unique_ptr<FileFormatter>> formatters;
00055 };

```

7.23 File_ID.hpp File Reference

File format ID constants.

This graph shows which files directly or indirectly include this file:



Classes

- class [File_ID](#)
[File_ID](#) class declaration.

7.23.1 Detailed Description

File format ID constants.

Author

Shamar Pennant

Defines static integer constants mapping file types to unique identifiers. Used to validate that [FileSource](#) and [Parser](#) objects are compatible.

Definition in file [File_ID.hpp](#).

7.24 File_ID.hpp

[Go to the documentation of this file.](#)

```

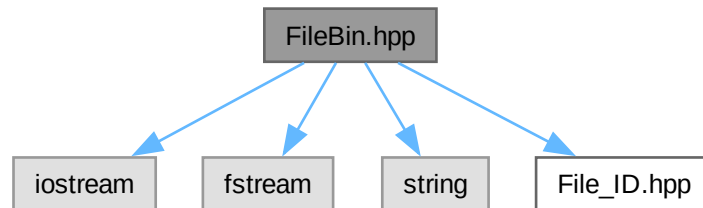
00001
00009 #pragma once
00010
00016 class File_ID {
00017     public:
00018         static constexpr int CSV = 1;
00019         static constexpr int HTML = 2;
00020         static constexpr int JSON = 3;
00021 };
  
```

7.25 FileBin.hpp File Reference

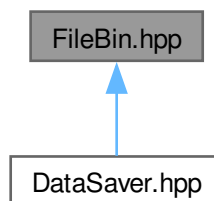
Abstract base class for writing data to files.

```
#include <iostream>
#include <fstream>
#include <string>
#include "File_ID.hpp"
```

Include dependency graph for FileBin.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [FileBin](#)
[FileBin](#) class declaration.

7.25.1 Detailed Description

Abstract base class for writing data to files.

Author

Shamar Pennant

Defines the interface for persisting formatted string data to files. Subclasses implement format-specific file writing operations (e.g., CSV files, HTML files, etc.).

Definition in file [FileBin.hpp](#).

7.26 FileBin.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include <iostream>
00004 #include <fstream>
00005 #include <string>
00006 #include "File_ID.hpp"
00007
00022 class FileBin {
00023     public:
00025         virtual ~FileBin() = default;
00026
00034         virtual int write() = 0;
00035
00039         virtual void setOutputFile(std::string output) = 0;
00040
00044         virtual void setRawData(std::string raw) = 0;
00045
00049         virtual void setFileName(std::string fn) = 0;
00050
00051     protected:
00052         FileBin() = default;
00053 };

```

7.27 FileFormatter.hpp File Reference

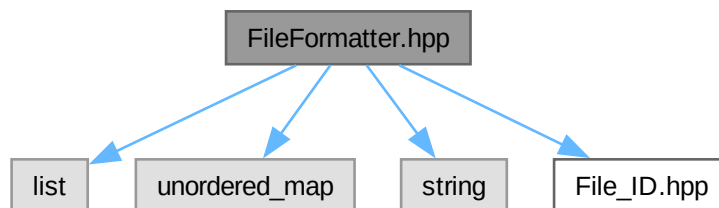
Abstract base class for formatting structured data to file format.

```

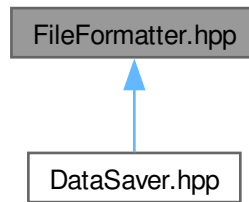
#include <list>
#include <unordered_map>
#include <string>
#include "File_ID.hpp"

```

Include dependency graph for FileFormatter.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [FileFormatter](#)
[FileFormatter](#) class declaration.

7.27.1 Detailed Description

Abstract base class for formatting structured data to file format.

Author

Shamar Pennant

Converts `list<Dict<str,str>>` structured data into format-specific string representation (e.g., CSV, HTML, JSON). Subclasses implement format-specific formatting logic for different file types. Abstract interface for formatting structured data to file format.

Definition in file [FileFormatter.hpp](#).

7.28 FileFormatter.hpp

[Go to the documentation of this file.](#)

```

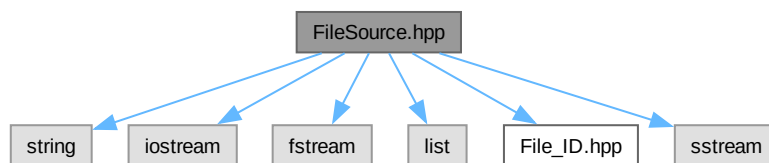
00001 #pragma once
00002
00003 #include <list>
00004 #include <unordered_map>
00005 #include <string>
00006 #include "File_ID.hpp"
00007
00018
00023 class FileFormatter {
00024     public:
00026         virtual ~FileFormatter() = default;
00027
00035         virtual std::string format() = 0;
00036
00040         virtual void setData(std::list<std::unordered_map<std::string, std::string> data) = 0;
00041
00042     protected:
00043         FileFormatter() = default;
00044 };
  
```

7.29 FileSource.hpp File Reference

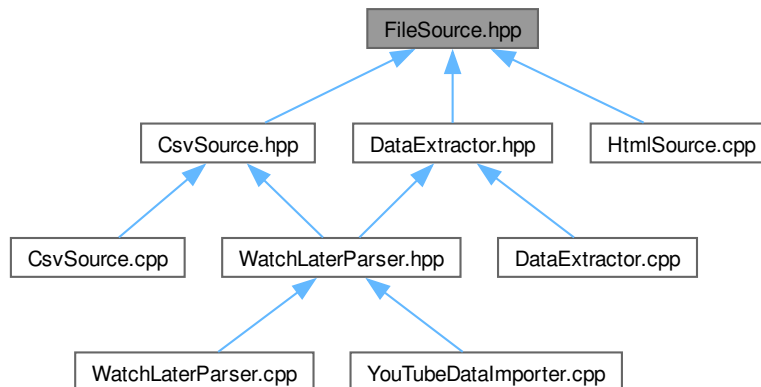
Abstract base class for reading file source data.

```
#include <string>
#include <iostream>
#include <fstream>
#include <list>
#include "File_ID.hpp"
#include <sstream>
```

Include dependency graph for FileSource.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [FileSource](#)
FileSource class declaration.

7.29.1 Detailed Description

Abstract base class for reading file source data.

Author

Shamar Pennant

Defines the interface for file reading operations across different file formats. Subclasses implement format-specific source reading (CSV, HTML, etc.).

Definition in file [FileSource.hpp](#).

7.30 FileSource.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include <string>
00003 #include <iostream>
00004 #include <fstream>
00005 #include <list>
00006 #include "File_ID.hpp"
00007 #include <sstream>
00008
00022 class FileSource {
00023     public:
00024         virtual ~FileSource() = default;
00025
00029         virtual std::list<std::string> read() = 0;
00030
00034         virtual void setSource(std::string src) = 0;
00035
00039         virtual int getSourceId() = 0;
00040
00041     protected:
00042         FileSource() = default;
00043 };

```

7.31 HtmlParser.cpp File Reference

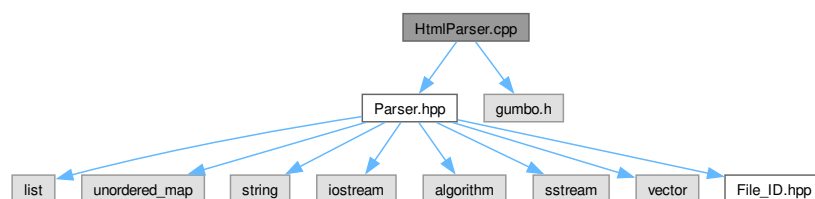
HTML file parser implementation using Gumbo.

```

#include "Parser.hpp"
#include <gumbo.h>

```

Include dependency graph for HtmlParser.cpp:



Classes

- class [HtmlParser](#)

HTML file parser implementation using Gumbo.

7.31.1 Detailed Description

HTML file parser implementation using Gumbo.

Author

Shamar Pennant

Implements the [Parser](#) interface for HTML files using the Gumbo HTML5 parser. Extracts HTML links and their text content into list<Dict<str,str>> format.

Definition in file [HtmlParser.cpp](#).

7.32 HtmlParser.cpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #include "Parser.hpp"
00011 #include <gumbo.h>
00012
00013 using namespace std;
00014
00016 class HtmlParser : public Parser {
00017     public:
00018         HtmlParser(list<string> raw_data) {this->raw_data = raw_data;}
00019         HtmlParser(){}
00020         ~HtmlParser() = default;
00021
00022         list<unordered_map<string,string>> parse() {
00023             if (raw_data.empty()) {
00024                 cerr << "HtmlParser: raw_data is empty" << endl;
00025                 exit(1);
00026             }
00027
00028             // given working with gumbo, everything is within a single item in the provided list
00029             string html = raw_data.front();
00030
00031             //grab the output through thr gumbo parser
00032             GumboOutput* output = gumbo_parse(html.c_str());
00033
00034             // container for returned results
00035             list<unordered_map<string,string>> results;
00036
00037             // place extracted links from the output into results
00038             extract_links(output->root, results);
00039
00040             // destroy ptr
00041             gumbo_destroy_output(&kGumboDefaultOptions, output);
00042
00043             return results;
00044         }
00045
00046         void setData(list<string> data) override {raw_data = data;}
00047
00048         int getParseId() override {return parse_id;}
00049
00050     private:
00051         list<string> raw_data = {};
00052         int const parse_id = File_ID::HTML;
00053
00054         // helper funtion to extracts links and tags using gumbo
00055

```

```

00056     void extract_links(GumboNode* node, list<unordered_map<string,string>& out) {
00057         if (node->type != GUMBO_NODE_ELEMENT)
00058             return;
00059
00060         GumboElement& el = node->v.element;
00061
00062         if (el.tag == GUMBO_TAG_A) {
00063             unordered_map<string,string> row;
00064
00065             // Extract href attribute
00066             GumboAttribute* href = gumbo_get_attribute(&el.attributes, "href");
00067             if (href) row["href"] = href->value;
00068
00069             // Extract visible text
00070             row["text"] = get_text(node);
00071
00072             // Tag name for debugging/consistency
00073             row["tag"] = "a";
00074
00075             out.push_back(row);
00076         }
00077
00078         // Recurse into children
00079         GumboVector* children = &el.children;
00080         for (unsigned i = 0; i < children->length; i++) {
00081             extract_links(static_cast<GumboNode*>(children->data[i]), out);
00082         }
00083     }
00084
00085     // Extract all text under a node
00086     string get_text(GumboNode* node) {
00087         if (node->type == GUMBO_NODE_TEXT)
00088             return node->v.text.text;
00089
00090         if (node->type != GUMBO_NODE_ELEMENT)
00091             return "";
00092
00093         string result;
00094         GumboVector* children = &node->v.element.children;
00095         for (unsigned i = 0; i < children->length; i++) {
00096             result += get_text(static_cast<GumboNode*>(children->data[i]));
00097         }
00098         return result;
00099     }
00100 };

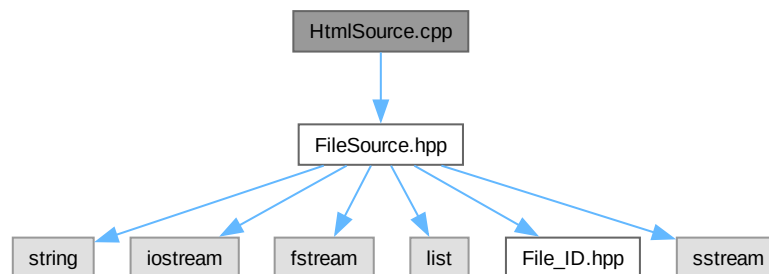
```

7.33 HtmlSource.cpp File Reference

HTML file source/reader implementation.

```
#include "FileSource.hpp"
```

Include dependency graph for HtmlSource.cpp:



Classes

- class [HtmlSource](#)

HTML file source/reader implementation.

7.33.1 Detailed Description

HTML file source/reader implementation.

Author

Shamar Pennant

Implements the [FileSource](#) interface for reading HTML files. Reads entire HTML file contents as a single string in `list<string>`. Note: Returns list with single item due to Gumbo parser requirements.

Definition in file [HtmlSource.cpp](#).

7.34 HtmlSource.cpp

[Go to the documentation of this file.](#)

```

00001
00010
00011 #include "FileSource.hpp"
00012
00013 using namespace std;
00014
00016 class HtmlSource : public FileSource {
00017     public:
00018         HtmlSource(std::string s) {src_path = s;}
00019         ~HtmlSource() = default;
00020
00021         list<string> read() override {
00022             // check if src is empty
00023             if (src_path.empty()) {
00024                 cerr << "html source path is empty" << endl;
00025                 exit(1);
00026             }
00027
00028             // open a file stream
00029             ifstream read_file_in(src_path, ios::in);
00030             if (!read_file_in.is_open()) {
00031                 cerr << "unable to open " << src_path << " within HtmlSource" << endl;
00032                 exit(1);
00033             }
00034
00035             // retrieve file contents
00036             stringstream buff;
00037             buff << read_file_in.rdbuf();
00038
00039             return {buff.str()};
00040         }
00041
00042         int getSourceId() {return source_id;}
00043         void setSource(string new_source) {src_path = new_source;}
00044
00045     private:
00046         string src_path = " ";
00047         int const source_id = File_ID::HTML;
00048 };

```

7.35 Parser.hpp File Reference

Abstract base class for parsing file data into structured format.

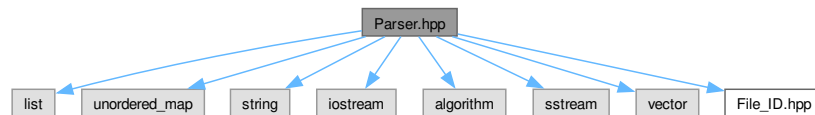
```

#include <list>
#include <unordered_map>
#include <string>
#include <iostream>

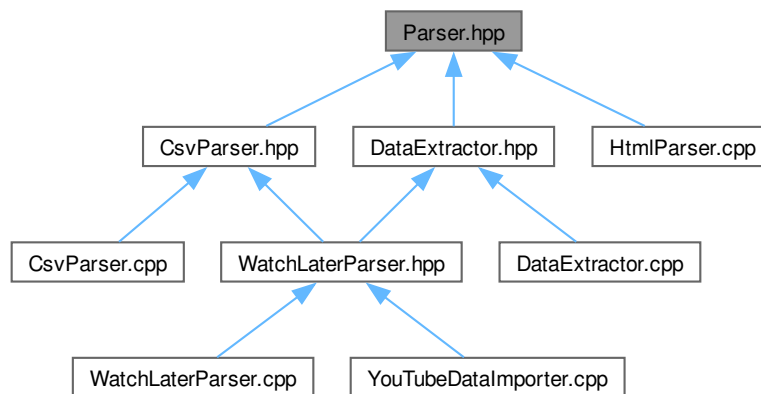
```

```
#include <algorithm>
#include <sstream>
#include <vector>
#include "File_ID.hpp"
```

Include dependency graph for Parser.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [Parser](#)
[Parser](#) class declaration.

7.35.1 Detailed Description

Abstract base class for parsing file data into structured format.

Author

Shamar Pennant

Defines the interface for converting raw file data into a consistent list of dictionaries format (`list<↔ Dict<string,string>>`). Each dictionary represents a row/entry with column headers as keys.

Definition in file [Parser.hpp](#).

7.36 Parser.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include <list>
00004 #include <unordered_map>
00005 #include <string>
00006
00007 #include <iostream>
00008 #include <algorithm>
00009 #include <sstream>
00010
00011 #include <vector>
00012 #include "File_ID.hpp"
00013
00023
00028 class Parser {
00029     public:
00030         virtual ~Parser() = default;
00031
00041         virtual std::list<std::unordered_map <std::string, std::string>> parse() = 0;
00042
00046         virtual void setData(std::list<std::string> data) = 0;
00047
00051         virtual int getParseId() = 0;
00052
00053     protected:
00054         Parser() = default;
00055 };

```

7.37 WatchHistoryParser.cpp File Reference

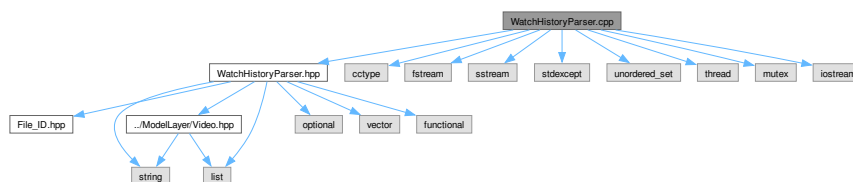
Implementation for [WatchHistoryParser](#).

```

#include "WatchHistoryParser.hpp"
#include <cctype>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <unordered_set>
#include <thread>
#include <mutex>
#include <iostream>

```

Include dependency graph for WatchHistoryParser.cpp:



7.37.1 Detailed Description

Implementation for [WatchHistoryParser](#).

Author

Shamar Pennant

Definition in file [WatchHistoryParser.cpp](#).

7.38 WatchHistoryParser.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "WatchHistoryParser.hpp"
00008
00009 #include <cctype>
00010 #include <fstream>
00011 #include <sstream>
00012 #include <stdexcept>
00013 #include <unordered_set>
00014 #include <thread>
00015 #include <mutex>
00016 #include <iostream>
00017
00018 WatchHistoryParser::WatchHistoryParser(const std::string& filePath) : filePath(filePath) {
00019     parser_id = File_ID::HTML;
00020 }
00021
00022 std::optional<std::string> WatchHistoryParser::extractVideoIdFromHref(const std::string& href) {
00023     std::string normalized = href;
00024     for (char& c : normalized) {
00025         c = static_cast<char>(std::tolower(static_cast<unsigned char>(c)));
00026     }
00027
00028     std::size_t watchPos = normalized.find("youtube.com/watch");
00029     if (watchPos != std::string::npos) {
00030         std::size_t queryPos = href.find('?', watchPos);
00031         if (queryPos == std::string::npos) {
00032             return std::nullopt;
00033         }
00034
00035         std::string query = href.substr(queryPos + 1);
00036         std::size_t start = 0;
00037         while (start < query.size()) {
00038             std::size_t amp = query.find('&', start);
00039             std::string token = query.substr(start, amp == std::string::npos ? std::string::npos : amp
- start);
00040
00041             if (token.rfind("v=", 0) == 0 && token.size() > 2) {
00042                 return token.substr(2);
00043             }
00044
00045             if (amp == std::string::npos) {
00046                 break;
00047             }
00048             start = amp + 1;
00049         }
00050     }
00051
00052     std::size_t shortPos = normalized.find("youtu.be/");
00053     if (shortPos != std::string::npos) {
00054         std::size_t idStart = shortPos + std::string("youtu.be/").size();
00055         std::size_t idEnd = href.find_first_of("?&#\"", idStart);
00056         std::string id = href.substr(idStart, idEnd == std::string::npos ? std::string::npos : idEnd -
idStart);
00057         if (!id.empty()) {
00058             return id;
00059         }
00060     }
00061
00062     return std::nullopt;
00063 }
00064
00065 std::list<Video> WatchHistoryParser::parse() {
00066     if (filePath.empty()) {
00067         throw std::invalid_argument("WatchHistoryParser: file path is empty");
00068     }
00069
00070     std::ifstream input(filePath, std::ios::in);
00071     if (!input.is_open()) {
00072         throw std::runtime_error("WatchHistoryParser: unable to open file " + filePath);
00073     }
00074
00075     std::stringstream buffer;
00076     buffer << input.rdbuf();
00077     std::string html = buffer.str();
00078
00079     std::list<Video> videos;
00080     std::unordered_set<std::string> seenVideoIds;
00081
00082     // Lightweight anchor scan to avoid hard dependency on external HTML parsers.
00083     std::size_t searchPos = 0;
00084     while (true) {

```

```

00085         std::size_t hrefPos = html.find("href=\"", searchPos);
00086         if (hrefPos == std::string::npos) {
00087             break;
00088         }
00089
00090         std::size_t valueStart = hrefPos + 6;
00091         std::size_t valueEnd = html.find("'", valueStart);
00092         if (valueEnd == std::string::npos) {
00093             break;
00094         }
00095
00096         std::string href = html.substr(valueStart, valueEnd - valueStart);
00097         auto videoId = extractVideoIdFromHref(href);
00098
00099         if (videoId.has_value() && seenVideoIds.insert(*videoId).second) {
00100             videos.emplace_back(*videoId, "", "", "", 0, std::list<std::string>{});
00101         }
00102
00103         searchPos = valueEnd + 1;
00104     }
00105
00106     return videos;
00107 }
00108
00109 int WatchHistoryParser::getParserId() const {
00110     return parser_id;
00111 }
00112
00113 std::list<Video> WatchHistoryParser::parseMultiThreaded(std::function<void(double)> progressCallback)
00114 {
00115     // Multi-threaded parsing: spawn 3 worker threads to process the file in parallel.
00116     // Each thread processes a distinct section to avoid duplicate work and improve
00117     // throughput on large HTML exports (e.g., 30K+ video IDs).
00118
00119     if (filePath.empty()) {
00120         throw std::invalid_argument("WatchHistoryParser: file path is empty");
00121     }
00122
00123     std::ifstream input(filePath, std::ios::in | std::ios::binary | std::ios::ate);
00124     if (!input.is_open()) {
00125         throw std::runtime_error("WatchHistoryParser: unable to open file " + filePath);
00126     }
00127
00128     // Read entire file at once for consistent parsing across threads.
00129     std::size_t fileSize = input.tellg();
00130     input.seekg(0);
00131
00132     std::string htmlContent(fileSize, '\0');
00133     input.read(&htmlContent[0], fileSize);
00134     input.close();
00135
00136     if (progressCallback) progressCallback(0.1);
00137
00138     // Divide file into 3 sections for parallel processing.
00139     const std::size_t section1End = fileSize / 3;
00140     const std::size_t section2End = (2 * fileSize) / 3;
00141     const std::size_t section3End = fileSize;
00142
00143     std::list<Video> videos1, videos2, videos3;
00144     std::mutex videoMutex;
00145     std::vector<std::thread> threads;
00146
00147     auto parseThreadFunc = [&](std::size_t startPos, std::size_t endPos, std::list<Video>&
00148     resultVideos) {
00149         // Each thread scans its section of the HTML file independently.
00150         // Results are collected into a separate list to avoid synchronization overhead.
00151         resultVideos = parseRange(htmlContent, startPos, endPos);
00152     };
00153
00154     // Spawn 3 threads to process file sections simultaneously.
00155     threads.emplace_back(parseThreadFunc, 0, section1End, std::ref(videos1));
00156     threads.emplace_back(parseThreadFunc, section1End, section2End, std::ref(videos2));
00157     threads.emplace_back(parseThreadFunc, section2End, section3End, std::ref(videos3));
00158
00159     if (progressCallback) progressCallback(0.3);
00160
00161     // Wait for all threads to complete.
00162     for (auto& thread : threads) {
00163         thread.join();
00164     }
00165
00166     if (progressCallback) progressCallback(0.7);
00167
00168     // Merge results from all threads, deduplicating by video ID.
00169     std::unordered_set<std::string> seenVideoIds;
00170     std::list<Video> mergedVideos;

```



```

00170     for (const auto& video : videos1) {
00171         if (seenVideoIds.insert(video.getVideoID()).second) {
00172             mergedVideos.push_back(video);
00173         }
00174     }
00175     for (const auto& video : videos2) {
00176         if (seenVideoIds.insert(video.getVideoID()).second) {
00177             mergedVideos.push_back(video);
00178         }
00179     }
00180     for (const auto& video : videos3) {
00181         if (seenVideoIds.insert(video.getVideoID()).second) {
00182             mergedVideos.push_back(video);
00183         }
00184     }
00185
00186     if (progressCallback) progressCallback(1.0);
00187
00188     std::cout << "[WatchHistoryParser] Multi-threaded parse complete: \""
00189               << filePath << "\" extracted " << mergedVideos.size() << " unique videos\\n";
00190
00191     return mergedVideos;
00192 }
00193
00194 std::list<Video> WatchHistoryParser::parseRange(const std::string& htmlContent, std::size_t startPos,
std::size_t endPos) {
00195     // Parse a range of HTML content, starting from startPos up to (but not including) endPos.
00196     // To avoid missing hrefs that span section boundaries, we search backwards from startPos
00197     // to find the previous href boundary and forwards from endPos to find the next boundary.
00198
00199     std::list<Video> videos;
00200     std::unordered_set<std::string> seenVideoIds;
00201
00202     // Adjust boundaries to avoid splitting href attributes.
00203     if (startPos > 0) {
00204         // Back up to previous href=" boundary.
00205         while (startPos > 0 && htmlContent.substr(startPos, 6) != "href=\"") {
00206             startPos--;
00207         }
00208     }
00209
00210     if (endPos < htmlContent.size()) {
00211         // Move forward to the next href=" boundary or end of file.
00212         while (endPos < htmlContent.size() && htmlContent.substr(endPos, 6) != "href=\"") {
00213             endPos++;
00214         }
00215     }
00216
00217     // Scan for href attributes within this adjusted range.
00218     std::size_t searchPos = startPos;
00219     while (searchPos < endPos) {
00220         std::size_t hrefPos = htmlContent.find("href=\"", searchPos);
00221         if (hrefPos == std::string::npos || hrefPos >= endPos) {
00222             break;
00223         }
00224
00225         std::size_t valueStart = hrefPos + 6;
00226         std::size_t valueEnd = htmlContent.find("'", valueStart);
00227         if (valueEnd == std::string::npos || valueEnd > endPos) {
00228             break;
00229         }
00230
00231         std::string href = htmlContent.substr(valueStart, valueEnd - valueStart);
00232         auto videoId = extractVideoIdFromHref(href);
00233
00234         if (videoId.has_value() && seenVideoIds.insert(*videoId).second) {
00235             videos.emplace_back(*videoId, "", "", "", 0, std::list<std::string>{});
00236         }
00237
00238         searchPos = valueEnd + 1;
00239     }
00240
00241     return videos;
00242 }

```

7.39 WatchHistoryParser.hpp File Reference

Specialized parser for YouTube watch-history HTML exports.

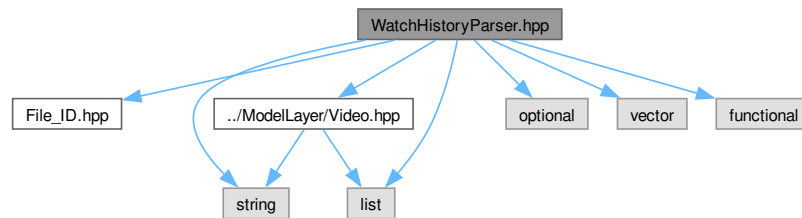
```

#include "File_ID.hpp"
#include "../ModelLayer/Video.hpp"

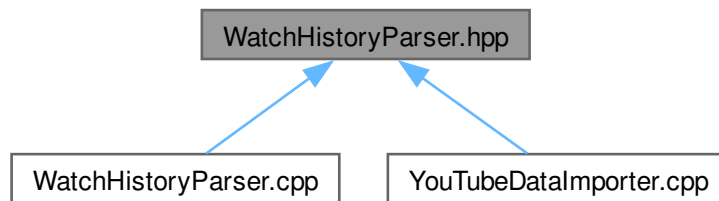
```

```
#include <list>
#include <optional>
#include <string>
#include <vector>
#include <functional>
```

Include dependency graph for WatchHistoryParser.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [WatchHistoryParser](#)
[WatchHistoryParser](#) class declaration.

7.39.1 Detailed Description

Specialized parser for YouTube watch-history HTML exports.

Author

Shamar Pennant

Reads a Google Takeout watch-history HTML file and extracts YouTube video IDs from watched-video links. Supports both single-threaded and multi-threaded parsing.

Definition in file [WatchHistoryParser.hpp](#).

7.40 WatchHistoryParser.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include "File_ID.hpp"
00004 #include "../ModelLayer/Video.hpp"
00005
00006 #include <list>
00007 #include <optional>
00008 #include <string>
00009 #include <vector>
00010 #include <functional>
00011
00024 class WatchHistoryParser {
00025 public:
00026     explicit WatchHistoryParser(const std::string& filePath);
00027
00032     std::list<Video> parse();
00033
00040     std::list<Video> parseMultiThreaded(std::function<void(double)> progressCallback = nullptr);
00041
00042     int getParserId() const;
00043
00044 private:
00045     std::string filePath;
00046     int parser_id;
00047
00053     static std::optional<std::string> extractVideoIdFromHref(const std::string& href);
00054
00062     static std::list<Video> parseRange(const std::string& htmlContent, std::size_t startPos,
std::size_t endPos);
00063 };

```

7.41 WatchLaterParser.cpp File Reference

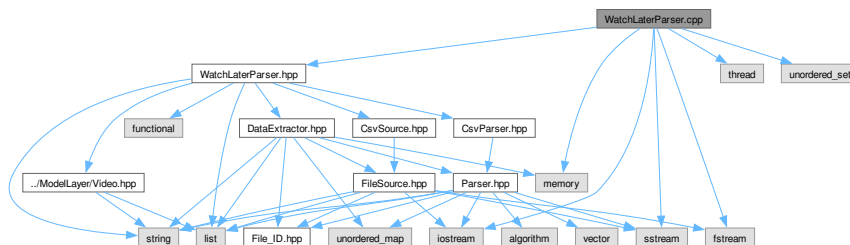
Implementation for [WatchLaterParser](#).

```

#include "WatchLaterParser.hpp"
#include <memory>
#include <fstream>
#include <sstream>
#include <thread>
#include <unordered_set>
#include <iostream>

```

Include dependency graph for WatchLaterParser.cpp:



7.41.1 Detailed Description

Implementation for [WatchLaterParser](#).

Author

Shamar Pennant

Jasmine Kumar

Definition in file [WatchLaterParser.cpp](#).

7.42 WatchLaterParser.cpp

[Go to the documentation of this file.](#)

```

00001
00002
00003 #include "WatchLaterParser.hpp"
00004
00005 #include <memory>
00006 #include <fstream>
00007 #include <sstream>
00008 #include <thread>
00009 #include <unordered_set>
00010 #include <iostream>
00011
00012 WatchLaterParser::WatchLaterParser(const std::string& filePath) : filePath(filePath) {
00013     parser_id = File_ID::CSV;
00014 }
00015
00016 std::list<Video> WatchLaterParser::parse() {
00017     DataExtractor extractor(
00018         std::make_unique<CsvSource>(filePath),
00019         std::make_unique<CsvParser>()
00020     );
00021
00022     auto rows = extractor.extract();
00023     std::list<Video> videos;
00024
00025     for (const auto& row : rows) {
00026         auto it = row.find("Video ID");
00027         if (it != row.end() && !it->second.empty()) {
00028             videos.emplace_back(
00029                 it->second,
00030                 "",
00031                 "",
00032                 "",
00033                 0,
00034                 std::list<std::string>{}
00035             );
00036         }
00037     }
00038     return videos;
00039 }
00040
00041 int WatchLaterParser::getParserId() {return parser_id;}
00042
00043 std::list<Video> WatchLaterParser::parseMultiThreaded(std::function<void(double)> progressCallback) {
00044     // Multi-threaded CSV parsing: spawn 3 worker threads to process rows in parallel.
00045     // Each thread processes a range of rows to maximize throughput.
00046
00047     std::ifstream input(filePath);
00048     if (!input.is_open()) {
00049         throw std::runtime_error("WatchLaterParser: unable to open file " + filePath);
00050     }
00051
00052     if (progressCallback) progressCallback(0.1);
00053
00054     // Read all lines into memory for parallel processing.
00055     std::vector<std::string> lines;
00056     std::string line;
00057     while (std::getline(input, line)) {
00058         lines.push_back(line);
00059     }
00060     input.close();
00061
00062     if (lines.empty()) {
00063         if (progressCallback) progressCallback(1.0);
00064         return std::list<Video>();
00065     }
00066
00067     if (progressCallback) progressCallback(0.2);

```

```

00074
00075 // First line is typically the header; process starting from line 1.
00076 std::size_t dataStartIdx = (lines.size() > 0 && lines[0].find("Video ID") != std::string::npos) ?
1 : 0;
00077 std::size_t totalRows = lines.size() - dataStartIdx;
00078
00079 if (totalRows == 0) {
00080     if (progressCallback) progressCallback(1.0);
00081     return std::list<Video>();
00082 }
00083
00084 // Divide rows into 3 sections for parallel processing.
00085 const std::size_t section1Start = dataStartIdx;
00086 const std::size_t section1End = dataStartIdx + (totalRows / 3);
00087 const std::size_t section2Start = section1End;
00088 const std::size_t section2End = dataStartIdx + (2 * totalRows / 3);
00089 const std::size_t section3Start = section2End;
00090 const std::size_t section3End = lines.size();
00091
00092 std::list<Video> videos1, videos2, videos3;
00093 std::vector<std::thread> threads;
00094
00095 auto parseRowsFunc = [&lines](std::size_t startIdx, std::size_t endIdx) -> std::list<Video> {
00096     // Each thread parses its range of CSV rows independently.
00097     // Extracts the Video ID (first column, comma-separated).
00098     std::list<Video> result;
00099     if (lines.empty() || startIdx >= lines.size()) {
00100         return result;
00101     }
00102
00103     for (std::size_t i = startIdx; i < endIdx && i < lines.size(); ++i) {
00104         const std::string& row = lines[i];
00105         if (row.empty()) {
00106             continue;
00107         }
00108
00109         // CSV format: "Video ID, Added on ..."
00110         std::size_t commaPos = row.find(',');
00111         std::string videoID = (commaPos != std::string::npos) ? row.substr(0, commaPos) : row;
00112
00113         // Trim quotes if present.
00114         if (!videoID.empty() && videoID.front() == '"' && videoID.back() == '"') {
00115             videoID = videoID.substr(1, videoID.size() - 2);
00116         }
00117
00118         if (!videoID.empty()) {
00119             result.emplace_back(videoID, "", "", "", 0, std::list<std::string>{});
00120         }
00121     }
00122     return result;
00123 };
00124
00125 // Spawn 3 threads.
00126 threads.emplace_back([&]() { videos1 = parseRowsFunc(section1Start, section1End); });
00127 threads.emplace_back([&]() { videos2 = parseRowsFunc(section2Start, section2End); });
00128 threads.emplace_back([&]() { videos3 = parseRowsFunc(section3Start, section3End); });
00129
00130 if (progressCallback) progressCallback(0.5);
00131
00132 // Wait for all threads.
00133 for (auto& thread : threads) {
00134     thread.join();
00135 }
00136
00137 if (progressCallback) progressCallback(0.8);
00138
00139 // Merge results, deduplicating by video ID.
00140 std::unordered_set<std::string> seenVideoIDs;
00141 std::list<Video> mergedVideos;
00142
00143 for (const auto& video : videos1) {
00144     if (seenVideoIDs.insert(video.getVideoID()).second) {
00145         mergedVideos.push_back(video);
00146     }
00147 }
00148 for (const auto& video : videos2) {
00149     if (seenVideoIDs.insert(video.getVideoID()).second) {
00150         mergedVideos.push_back(video);
00151     }
00152 }
00153 for (const auto& video : videos3) {
00154     if (seenVideoIDs.insert(video.getVideoID()).second) {
00155         mergedVideos.push_back(video);
00156     }
00157 }
00158
00159 if (progressCallback) progressCallback(1.0);

```

```

00160
00161     std::cout << "[WatchLaterParser] Multi-threaded parse complete: \"\"
00162               << filePath << "\" extracted \" \" mergedVideos.size() << \" unique videos\n";
00163
00164     return mergedVideos;
00165 }

```

7.43 WatchLaterParser.hpp File Reference

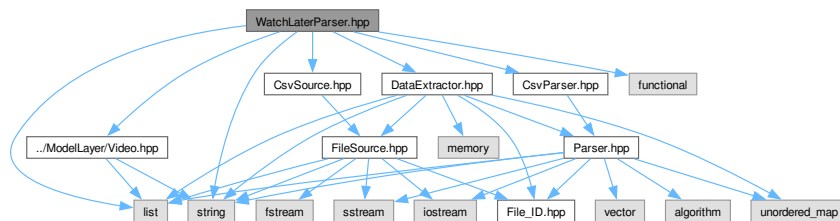
Specialized parser for YouTube Takeout Watch Later CSV files.

```

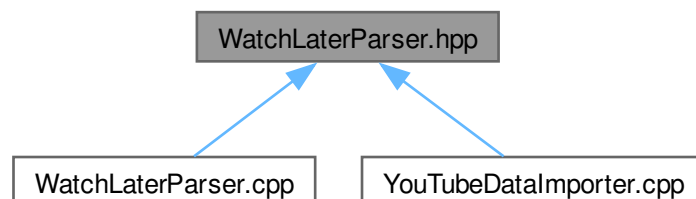
#include "DataExtractor.hpp"
#include "CsvSource.hpp"
#include "CsvParser.hpp"
#include "../ModelLayer/Video.hpp"
#include <list>
#include <string>
#include <functional>

```

Include dependency graph for WatchLaterParser.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [WatchLaterParser](#)
WatchLaterParser class declaration.

7.43.1 Detailed Description

Specialized parser for YouTube Takeout Watch Later CSV files.

Author

Shamar Pennant

Jasmine Kumar

Parses YouTube Takeout 'Watch later-videos.csv' files into [Video](#) objects with only the videoID field populated. Internally uses [CsvSource](#) for reading and [CsvParser](#) for parsing. The resulting Videos have empty strings/zero for all other field since Watch Later CSV only provides ID and timestamp. Supports both single-threaded and multi-threaded parsing.

Definition in file [WatchLaterParser.hpp](#).

7.44 WatchLaterParser.hpp

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00003 #include "DataExtractor.hpp"
00004 #include "CsvSource.hpp"
00005 #include "CsvParser.hpp"
00006 #include "../ModelLayer/Video.hpp"
00007
00008 #include <list>
00009 #include <string>
00010 #include <functional>
00011
00024
00030 class WatchLaterParser {
00031 public:
00036     explicit WatchLaterParser(const std::string& filePath);
00037
00045     std::list<Video> parse();
00046
00053     std::list<Video> parseMultiThreaded(std::function<void(double)> progressCallback = nullptr);
00054
00055     int getParserId();
00056
00057 private:
00058     std::string filePath;
00059     int parser_id;
00060 };
```

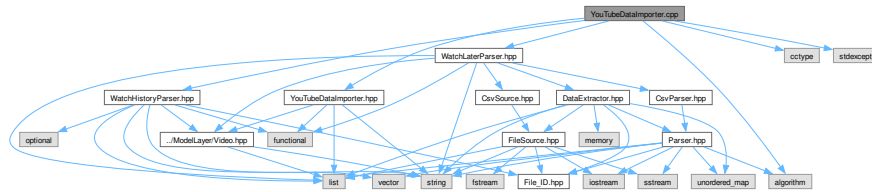
7.45 YouTubeDataImporter.cpp File Reference

Implementation for [YouTubeDataImporter](#).

```
#include "YouTubeDataImporter.hpp"
#include "WatchHistoryParser.hpp"
#include "WatchLaterParser.hpp"
#include <algorithm>
#include <cctype>
```

```
#include <stdexcept>
```

Include dependency graph for YouTubeDataImporter.cpp:



7.45.1 Detailed Description

Implementation for [YouTubeDataImporter](#).

Author

Shamar Pennant

Definition in file [YouTubeDataImporter.cpp](#).

7.46 YouTubeDataImporter.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #include "YouTubeDataImporter.hpp"
00008
00009 #include "WatchHistoryParser.hpp"
00010 #include "WatchLaterParser.hpp"
00011
00012 #include <algorithm>
00013 #include <cctype>
00014 #include <stdexcept>
00015
00016 std::string YouTubeDataImporter::lowerExtension(const std::string& filePath) {
00017     std::size_t dotPos = filePath.find_last_of('.');
00018     if (dotPos == std::string::npos) {
00019         return "";
00020     }
00021
00022     std::string ext = filePath.substr(dotPos);
00023     std::transform(ext.begin(), ext.end(), ext.begin(),
00024         [](unsigned char c) { return static_cast<char>(std::tolower(c)); });
00025     return ext;
00026 }
00027
00028 std::list<Video> YouTubeDataImporter::import(const std::string& filePath) const {
00029     const std::string ext = lowerExtension(filePath);
00030
00031     if (ext == ".csv") {
00032         return WatchLaterParser(filePath).parse();
00033     }
00034
00035     if (ext == ".html" || ext == ".htm") {
00036         return WatchHistoryParser(filePath).parse();
00037     }
00038
00039     throw std::invalid_argument(
00040         "YouTubeDataImporter: unsupported file extension for path: " + filePath
00041     );
00042 }
00043
00044 std::list<Video> YouTubeDataImporter::importMultiThreaded(
00045     const std::string& filePath,
```



```

00046     std::function<void(double)> progressCallback) const {
00047         // Multi-threaded import that leverages 3 worker threads for faster file processing.
00048         // Routes to the appropriate parser based on file extension, then invokes
00049         // that parser's multi-threaded parse method with progress feedback.
00050
00051         const std::string ext = lowerExtension(filePath);
00052
00053         if (ext == ".csv") {
00054             return WatchLaterParser(filePath).parseMultiThreaded(progressCallback);
00055         }
00056
00057         if (ext == ".html" || ext == ".htm") {
00058             return WatchHistoryParser(filePath).parseMultiThreaded(progressCallback);
00059         }
00060
00061         throw std::invalid_argument(
00062             "YouTubeDataImporter: unsupported file extension for path: " + filePath
00063         );
00064     }

```

7.47 YouTubeDataImporter.hpp File Reference

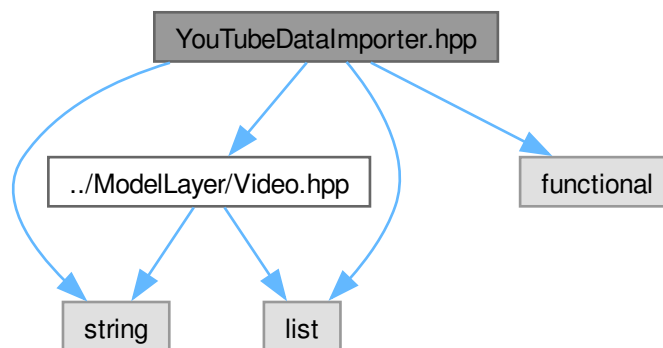
Entry-point importer that routes YouTube exports to the right parser.

```

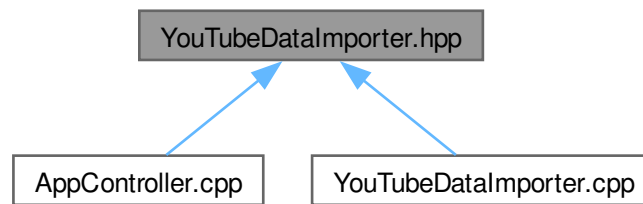
#include "../ModelLayer/Video.hpp"
#include <list>
#include <string>
#include <functional>

```

Include dependency graph for YouTubeDataImporter.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [YouTubeDataImporter](#)
[YouTubeDataImporter](#) class declaration.

7.47.1 Detailed Description

Entry-point importer that routes YouTube exports to the right parser.

Author

Shamar Pennant

Keeps [AppController](#) clean by hiding CSV-vs-HTML parser selection. Supports both single-threaded and multi-threaded import with progress tracking.

Definition in file [YouTubeDataImporter.hpp](#).

7.48 YouTubeDataImporter.hpp

[Go to the documentation of this file.](#)

```

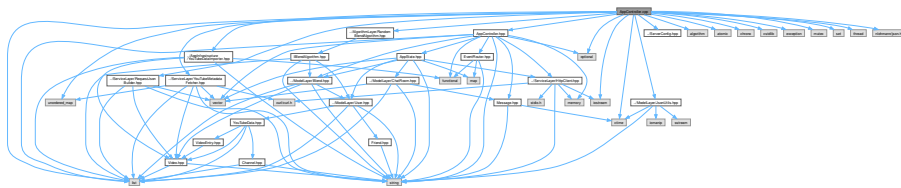
00001 #pragma once
00002
00003 #include "../ModelLayer/Video.hpp"
00004
00005 #include <list>
00006 #include <string>
00007 #include <functional>
00008
00021 class YouTubeDataImporter {
00022 public:
00033     std::list<Video> import(const std::string& filePath) const;
00034
00042     std::list<Video> importMultiThreaded(const std::string& filePath,
00043                                         std::function<void(double)> progressCallback = nullptr)
00044     const;
00045 private:
00051     static std::string lowerExtension(const std::string& filePath);
00052 };
  
```

7.49 AppController.cpp File Reference

Implementation for [AppController](#).

```
#include "AppController.hpp"
#include "../ServerConfig.hpp"
#include "../AppInfrastructure/YouTubeDataImporter.hpp"
#include "../AlgorithmLayer/RandomBlendAlgorithm.hpp"
#include "../ModelLayer/JsonUtils.hpp"
#include "../ServiceLayer/YouTubeMetadataFetcher.hpp"
#include "../ServiceLayer/RequestJsonBuilder.hpp"
#include <algorithm>
#include <atomic>
#include <chrono>
#include <cstdlib>
#include <ctime>
#include <exception>
#include <iostream>
#include <list>
#include <mutex>
#include <set>
#include <thread>
#include <unordered_map>
#include <nlohmann/json.hpp>
#include <optional>
```

Include dependency graph for AppController.cpp:



Typedefs

- using [Json](#) = nlohmann::json

Variables

- static const std::string [kYouTubeAPIKey](#) = "AlzaSyBDzH4_T9NSaAh_s59sFYrHtNnKvHmyARM"

7.49.1 Detailed Description

Implementation for [AppController](#).

Author

Jasmine Kumar
Shamar Pennant

Definition in file [AppController.cpp](#).

7.49.2 Typedef Documentation

7.49.2.1 Json

```
using Json = nlohmann::json
```

Definition at line 30 of file [AppController.cpp](#).

7.49.3 Variable Documentation

7.49.3.1 kYouTubeAPIKey

```
const std::string kYouTubeAPIKey = "AIzaSyBDzH4_T9NSaAh_s59sFYrHtNnKvHmyARM" [static]
```

Definition at line 323 of file [AppController.cpp](#).

7.50 AppController.cpp

[Go to the documentation of this file.](#)

```
00001
00007
00008 #include "AppController.hpp"
00009 #include "../ServerConfig.hpp"
00010 #include "../AppInfrastructure/YouTubeDataImporter.hpp"
00011 #include "../AlgorithmLayer/RandomBlendAlgorithm.hpp"
00012 #include "../ModelLayer/JsonUtils.hpp"
00013 #include "../ServiceLayer/YouTubeMetadataFetcher.hpp"
00014 #include "../ServiceLayer/RequestJsonBuilder.hpp"
00015 #include <algorithm>
00016 #include <atomic>
00017 #include <chrono>
00018 #include <cstdlib>
00019 #include <ctime>
00020 #include <exception>
00021 #include <iostream>
00022 #include <list>
00023 #include <mutex>
00024 #include <set>
00025 #include <thread>
00026 #include <unordered_map>
00027 #include <nlohmann/json.hpp>
00028 #include <optional>
00029
00030 using Json = nlohmann::json;
00031 using namespace std;
00032
00033 namespace {
00034
00035 std::string resolveBackendBaseUrl() {
00036     return kAppBackendBaseUrl;
00037 }
00038
00039 constexpr std::size_t kHtmlUploadMetadataThreshold = 500;
00040
00041 constexpr std::size_t kMaxUploadEnrichmentCount = 50;
00042
00043 constexpr std::size_t kMaxBlendFeedEnrichmentCount = 50;
00044
00045 constexpr std::size_t kBlendMetadataWorkerThreads = 3;
00046
00047 constexpr std::size_t kLoginParticipantLoadWorkerThreads = 3;
00048
00049 constexpr std::size_t kMaxLoginMetadataEnrichmentCount = 50;
00050
00051 bool isMetadataIncomplete(const Video& v) {
00052     return v.getTitle().empty() ||
00053         v.getChannelID().empty() ||
00054         v.getChannelName().empty() ||
```

```

00068         v.getThumbnailURL().empty() ||
00069         v.getChannelLogoURL().empty();
00070     }
00071
00072     bool hasCompleteMetadata(const Video& v) {
00073         return !v.getTitle().empty() &&
00074             !v.getChannelID().empty() &&
00075             !v.getChannelName().empty() &&
00076             !v.getThumbnailURL().empty() &&
00077             !v.getChannelLogoURL().empty();
00078     }
00079
00080     std::string lowerExtension(const std::string& filePath) {
00081         const std::size_t dotPos = filePath.find_last_of('.');
00082         if (dotPos == std::string::npos) {
00083             return "";
00084         }
00085
00086         std::string ext = filePath.substr(dotPos);
00087         std::transform(ext.begin(), ext.end(), ext.begin(),
00088             [](unsigned char c) { return static_cast<char>(std::tolower(c)); });
00089         return ext;
00090     }
00091
00092 }
00093
00094 std::optional<User> AppController::parseUserFromAuthResponse(const std::string& response,
00095                                                             const std::string& fallbackPassword) {
00096     Json parsed = Json::parse(response, nullptr, false);
00097     if (parsed.is_discarded() || !parsed.is_object() || !parsed.contains("user_id")) {
00098         return std::nullopt;
00099     }
00100
00101     const std::string userID = parsed.value("user_id", "");
00102     if (userID.empty()) {
00103         return std::nullopt;
00104     }
00105
00106     return User(
00107         userID,
00108         parsed.value("username", userID),
00109         parsed.value("email", ""),
00110         fallbackPassword
00111     );
00112 }
00113
00114 std::optional<std::string> AppController::parseBlendID(const std::string& response) {
00115     Json parsed = Json::parse(response, nullptr, false);
00116     if (parsed.is_discarded() || !parsed.is_object()) {
00117         return std::nullopt;
00118     }
00119
00120     const std::string blendID = parsed.value("blend_id", "");
00121     if (blendID.empty()) {
00122         return std::nullopt;
00123     }
00124
00125     return blendID;
00126 }
00127
00128 std::vector<std::string> AppController::parseParticipantIDs(const std::string& response) {
00129     std::vector<std::string> participantIDs;
00130
00131     Json parsed = Json::parse(response, nullptr, false);
00132     if (parsed.is_discarded() || !parsed.is_object() || !parsed.contains("participants")) {
00133         return participantIDs;
00134     }
00135
00136     if (!parsed["participants"].is_array()) {
00137         return participantIDs;
00138     }
00139
00140     for (const auto& participant : parsed["participants"]) {
00141         if (participant.is_string()) {
00142             participantIDs.push_back(participant.get<std::string>());
00143         }
00144     }
00145
00146     return participantIDs;
00147 }
00148
00149 std::list<Video> AppController::parseWatchLaterVideos(const std::string& response) {
00150     std::list<Video> videos;
00151
00152     Json parsed = Json::parse(response, nullptr, false);
00153     if (parsed.is_discarded() || !parsed.is_object() || !parsed.contains("videos")) {
00154         return videos;

```

```

00155     }
00156
00157     if (!parsed["videos"].is_array()) {
00158         return videos;
00159     }
00160
00161     for (const auto& item : parsed["videos"]) {
00162         if (!item.is_object()) {
00163             continue;
00164         }
00165
00166         const std::string videoID = item.value("video_id", "");
00167         if (videoID.empty()) {
00168             continue;
00169         }
00170
00171         videos.emplace_back(
00172             videoID,
00173             item.value("title", ""),
00174             item.value("channel_id", ""),
00175             item.value("thumbnail_url", ""),
00176             item.value("duration", 0),
00177             std::list<std::string>{},
00178             item.value("channel_name", ""),
00179             item.value("channel_logo_url", "")
00180         );
00181     }
00182
00183     return videos;
00184 }
00185
00186 std::list<User> AppController::loadParticipantsWithWatchLater(const std::vector<std::string>&
participantIDs,
00187                                                             std::vector<std::string>* missingData,
00188                                                             bool enrichParticipantLists) {
00189     std::list<User> participants;
00190
00191     for (const auto& uid : participantIDs) {
00192         std::string watchLaterResponse;
00193         try {
00194             watchLaterResponse = http.get(http.build_watch_later_endpoint(uid));
00195         } catch (const std::exception& ex) {
00196             std::cerr << "[AppController] loadParticipantsWithWatchLater: failed to fetch watch-later
for '"
00197                 << uid << "' - " << ex.what() << "\n";
00198             if (missingData != nullptr) {
00199                 missingData->push_back(uid);
00200             }
00201             continue;
00202         }
00203
00204         if (!http.wasLastRequestSuccessful(http.G)) {
00205             if (missingData != nullptr) {
00206                 missingData->push_back(uid);
00207             }
00208             continue;
00209         }
00210
00211         std::list<Video> watchLater = parseWatchLaterVideos(watchLaterResponse);
00212         if (watchLater.empty()) {
00213             Json debug = Json::parse(watchLaterResponse, nullptr, false);
00214             if (debug.is_discarded() || !debug.is_object() || !debug.contains("videos")) {
00215                 std::cerr << "[AppController] loadParticipantsWithWatchLater: malformed watch-later
payload for '"
00216                     << uid << "'\n";
00217             }
00218             if (missingData != nullptr) {
00219                 missingData->push_back(uid);
00220             }
00221             continue;
00222         }
00223
00224         // Optional: enrich participant pools. For blend-create/select paths this
00225         // is intentionally disabled to avoid expensive full-list metadata pulls.
00226         if (enrichParticipantLists) {
00227             enrichIfMissingMetadata(uid, watchLater);
00228         }
00229
00230         // create a local representation of the user
00231         User user(uid, uid, "", "");
00232         YouTubeData yt;
00233         yt.setWatchLaterVideos(watchLater);
00234         user.setYouTubeData(yt);
00235         participants.push_back(user);
00236     }
00237
00238     return participants;

```

```

00239 }
00240
00241 std::list<User> AppController::loadParticipantsWithWatchLaterParallel(
00242     const std::vector<std::string>& participantIDs,
00243     std::vector<std::string>* missingData,
00244     std::size_t workerThreads) {
00245     std::list<User> participants;
00246     if (participantIDs.empty() || workerThreads == 0) {
00247         return participants;
00248     }
00249
00250     const std::size_t workerCount = std::min<std::size_t>(workerThreads, participantIDs.size());
00251     std::vector<std::optional<User>> usersByIndex(participantIDs.size());
00252     std::mutex missingMutex;
00253     std::atomic<std::size_t> completed{0};
00254     std::vector<std::thread> workers;
00255     workers.reserve(workerCount);
00256
00257     for (std::size_t workerIndex = 0; workerIndex < workerCount; ++workerIndex) {
00258         workers.emplace_back([&, workerIndex]() {
00259             HttpClient threadHttp(server_url);
00260
00261             for (std::size_t i = workerIndex; i < participantIDs.size(); i += workerCount) {
00262                 const std::string& uid = participantIDs[i];
00263                 try {
00264                     const std::string response =
threadHttp.get(threadHttp.build_watch_later_endpoint(uid));
00265                     if (!threadHttp.wasLastRequestSuccessful(threadHttp.G)) {
00266                         if (missingData != nullptr) {
00267                             std::lock_guard<std::mutex> lock(missingMutex);
00268                             missingData->push_back(uid);
00269                         }
00270                         completed.fetch_add(1, std::memory_order_relaxed);
00271                         continue;
00272                     }
00273
00274                     std::list<Video> watchLater = parseWatchLaterVideos(response);
00275                     if (watchLater.empty()) {
00276                         if (missingData != nullptr) {
00277                             std::lock_guard<std::mutex> lock(missingMutex);
00278                             missingData->push_back(uid);
00279                         }
00280                         completed.fetch_add(1, std::memory_order_relaxed);
00281                         continue;
00282                     }
00283
00284                     User user(uid, uid, "", "");
00285                     YouTubeData yt;
00286                     yt.setWatchLaterVideos(watchLater);
00287                     user.setYouTubeData(yt);
00288                     usersByIndex[i] = user;
00289                 } catch (const std::exception&) {
00290                     if (missingData != nullptr) {
00291                         std::lock_guard<std::mutex> lock(missingMutex);
00292                         missingData->push_back(uid);
00293                     }
00294                 }
00295
00296                 completed.fetch_add(1, std::memory_order_relaxed);
00297             }
00298         });
00299     }
00300
00301     while (completed.load(std::memory_order_relaxed) < participantIDs.size()) {
00302         const double ratio = static_cast<double>(completed.load(std::memory_order_relaxed)) /
00303             static_cast<double>(participantIDs.size());
00304         reportProgress(0.6 + (ratio * 0.22), "Loading blend participant data...");
00305         std::this_thread::sleep_for(std::chrono::milliseconds(20));
00306     }
00307
00308     for (auto& worker : workers) {
00309         if (worker.joinable()) {
00310             worker.join();
00311         }
00312     }
00313
00314     for (const auto& maybeUser : usersByIndex) {
00315         if (maybeUser.has_value()) {
00316             participants.push_back(*maybeUser);
00317         }
00318     }
00319
00320     return participants;
00321 }
00322
00323 static const std::string kYouTubeAPIKey = "AIzaSyBDzH4_T9NSaAh_s59sFYrHtNnKvHmyARM";
00324

```

```

00325 // -- Constructor / Destructor -----
00326
00327 ApplicationController::AppController():
00328     appState(AppState::getInstance()),
00329     server_url(resolveBackendBaseUrl()),
00330     http(server_url),
00331     isConnected(false) {
00332     registerEvents();
00333
00334     std::string resp = http.get("/ping");
00335     try {
00336         if (resp.find("pong") != std::string::npos) {
00337             std::cout << "[AppState] Successful connection to the server" << std::endl;
00338             isConnected = true;
00339         } else {
00340             std::cout << "[AppState] Server reachable but unexpected response..."
00341                 << resp
00342                 << " "
00343                 << endl;
00344         }
00345     } catch (const std::exception& e) {
00346         std::cout << "[AppState] Server unavailable, running offline: " << e.what() << std::endl;
00347         cout << "[AppState] Server response: " << resp << endl;
00348     }
00349 }
00350
00351 ApplicationController::AppController(const std::string& /*dbPath*/)
00352     : AppController() {}
00353
00354 ApplicationController::~AppController() {
00355     // If application shutting down and there's a server-created pending account,
00356     // attempt to delete it so username isn't left reserved.
00357     if (pendingAccountCreated_ && pendingRegistration_.has_value() && isConnected) {
00358         try {
00359             const std::string delResp = http.del(
00360                 http.build_delete_user_endpoint(pendingRegistration_>getUserID()),
00361                 JsonRequestBuilder::buildDeleteAccountJson(pendingRegistration_>getPassword())
00362             );
00363             if (http.wasLastRequestSuccessful(http.D)) {
00364                 std::cout << "[AppController] Deleted pending account '" <<
00365                     pendingRegistration_>getUserID()
00366                     << "' on shutdown\n";
00367             } else {
00368                 std::cerr << "[AppController] Failed to delete pending account on shutdown, HTTP "
00369                     << http.getLastStatusCode() << " response=" << delResp << "\n";
00370             }
00371         } catch (...) {
00372             // best-effort; swallow errors
00373         }
00374     }
00375     eventRouter.deregisterAll();
00376 }
00377
00378 // -- Event Registration -----
00379
00380 void ApplicationController::registerEvents() {
00381     // Binds each named event to the corresponding handler on this controller.
00382     // Panels dispatch events by name; EventRouter calls the handler below.
00383     eventRouter.registerListener("register", [this](const EventPayload& p){ handleRegister(p); });
00384     eventRouter.registerListener("login", [this](const EventPayload& p){ handleLogin(p); });
00385     eventRouter.registerListener("logout", [this](const EventPayload& p){ handleLogout(p); });
00386     eventRouter.registerListener("deleteAccount", [this](const EventPayload& p){
00387         handleDeleteAccount(p); });
00387     eventRouter.registerListener("uploadData", [this](const EventPayload& p){ handleUploadData(p);
00388 });
00388     eventRouter.registerListener("createBlend", [this](const EventPayload& p){ handleCreateBlend(p);
00389 });
00389     eventRouter.registerListener("playVideo", [this](const EventPayload& p){ handlePlayVideo(p); });
00390     eventRouter.registerListener("refresh", [this](const EventPayload& p){ handleRefresh(p); });
00391     eventRouter.registerListener("openChat", [this](const EventPayload& p){ handleOpenChat(p); });
00392     eventRouter.registerListener("sendMessage", [this](const EventPayload& p){ handleSendMessage(p);
00393 });
00393     eventRouter.registerListener("leaveBlend", [this](const EventPayload& p){ handleLeaveBlend(p);
00394 });
00394     eventRouter.registerListener("selectBlend", [this](const EventPayload& p){ handleSelectBlend(p);
00395 });
00396
00397 // -- Getter -----
00398
00399 EventRouter& ApplicationController::getEventRouter() {
00400     return eventRouter;
00401 }
00402
00403 void ApplicationController::setProgressReporter(std::function<void(double, const std::string&)> reporter) {
00404     progressReporter = std::move(reporter);

```



```

00405 }
00406
00407 void AppController::clearProgressReporter() {
00408     progressReporter = nullptr;
00409 }
00410
00411 void AppController::reportProgress(double progress, const std::string& message) {
00412     if (!progressReporter) {
00413         return;
00414     }
00415
00416     if (progress < 0.0) progress = 0.0;
00417     if (progress > 1.0) progress = 1.0;
00418     progressReporter(progress, message);
00419 }
00420
00421 // -- Event Handlers -----
00422
00423 void AppController::handleRegister(const EventPayload& payload) {
00424     // verifying user data structure
00425     auto idIt = payload.find("userID");
00426     auto unIt = payload.find("username");
00427     auto emIt = payload.find("email");
00428     auto pwIt = payload.find("password");
00429
00430     if (idIt == payload.end() || unIt == payload.end() ||
00431         pwIt == payload.end()) { // if user data is malformed, notify
00432         std::cerr << "[AppController] handleRegister: missing required fields\n";
00433         return;
00434     }
00435
00436     // Build a local User object
00437     User newUser(idIt->second,
00438                 unIt->second,
00439                 emIt != payload.end() ? emIt->second : "",
00440                 pwIt->second);
00441
00442     // Attempt to create the account on the server immediately so we can delete it if
00443     // the user cancels or exits before uploading their data (backend requires it).
00444     if (isConnected) {
00445         try {
00446             const std::string registerResponse = http.post(
00447                 http.REG_USER,
00448                 RequestJsonBuilder::buildRegisterJson(
00449                     newUser.getUserID(),
00450                     newUser.getUsername(),
00451                     newUser.getEmail(),
00452                     newUser.getPassword()
00453                 )
00454             );
00455
00456             if (!http.wasLastRequestSuccessful(http.P)) {
00457                 if (http.getLastStatusCode() == http.DUPLICATE) {
00458                     std::cerr << "[AppController] handleRegister: userID '"
00459                         << newUser.getUserID() << "' already exists\n";
00460                 } else {
00461                     std::cerr << "[AppController] handleRegister: registration failed, HTTP "
00462                         << http.getLastStatusCode() << " response=" << registerResponse << "\n";
00463                 }
00464                 return;
00465             }
00466
00467             // success - keep pendingRegistration_ so we can delete it if upload never happens
00468             pendingRegistration_ = newUser;
00469             pendingAccountCreated_ = true;
00470             std::cout << "[AppController] Account created on server for '" << newUser.getUserID()
00471                 << "' (pending upload)\n";
00472         } catch (const std::exception& ex) {
00473             std::cerr << "[AppController] handleRegister: HTTP error creating account: "
00474                 << ex.what() << "\n";
00475             return;
00476         }
00477     } else {
00478         // Offline: still store pendingRegistration_ locally (account not created on server).
00479         pendingRegistration_ = newUser;
00480         pendingAccountCreated_ = false;
00481         std::cout << "[AppController] Pending local registration stored for '" << newUser.getUserID()
00482             << "' (offline)\n";
00483     }
00484
00485     // Set the current user locally so the UI can continue to the DataInstructionsPanel.
00486     appState.setCurrentUser(new User(newUser));
00487     return;
00488 }
00489
00490 void AppController::handleLogin(const EventPayload& payload) {
00491     // verifying incorrect userID and password

```

```

00492     auto idIt = payload.find("userID");
00493     auto pwIt = payload.find("password");
00494
00495     if (idIt == payload.end() || pwIt == payload.end()) {
00496         std::cerr << "[AppController] handleLogin: missing 'userID' or 'password'\n";
00497         return;
00498     }
00499
00500     reportProgress(0.05, "Authenticating account...");
00501
00502     const auto loginResponse = http.post(
00503         http.LOGIN,
00504         JsonRequestBuilder::buildLoginJson(idIt->second, pwIt->second)
00505     );
00506
00507     // retrieve the status of the request
00508     const bool req_status = http.isRequestSuccessful(http.P, http.getLastStatusCode());
00509
00510     if (!req_status) {
00511         cout << "[AppController] handleLogin: invalid credentials or '"
00512             << idIt->second
00513             << "' does not exist..."
00514             << endl;
00515         return;
00516     }
00517
00518     reportProgress(0.2, "Loading your saved data...");
00519
00520     std::optional<User> user = parseUserFromAuthResponse(loginResponse, pwIt->second);
00521     if (!user.has_value()) {
00522         std::cerr << "[AppController] handleLogin: malformed login response\n";
00523         return;
00524     }
00525
00526     string watchLaterResponse;
00527     // Restore persisted watch-later data from the backend
00528     list<Video> watchLater;
00529     try {
00530         watchLaterResponse = http.get(http.build_watch_later_endpoint(idIt->second));
00531     } catch (exception& e) {
00532         std::cerr << "[AppController] handleRegister: HTTP error, unable to move forward: "
00533             << e.what() << "\n";
00534         cout << "Server response: "
00535             << watchLaterResponse
00536             << endl;
00537
00538         return;
00539     }
00540
00541     if (http.wasLastRequestSuccessful(http.G)) {
00542         watchLater = parseWatchLaterVideos(watchLaterResponse);
00543         reportProgress(0.3, "Refreshing metadata subset...");
00544
00545         // Keep login responsive by enriching only a bounded subset and skipping
00546         // DB write-back on this path. Full enrichment can happen later during
00547         // upload/blend-display workflows.
00548         watchLater = enrichMissingMetadataSubsetMultithreaded(
00549             watchLater,
00550             kMaxLoginMetadataEnrichmentCount,
00551             kBlendMetadataWorkerThreads,
00552             [](const std::list<Video>& chunk) {
00553                 return YouTubeMetadataFetcher(kYouTubeAPIKey).enrich(chunk);
00554             }
00555         );
00556         reportProgress(0.42, "Metadata step complete");
00557     }
00558
00559     if (!watchLater.empty()) {
00560         YouTubeData yd;
00561         yd.setWatchLaterVideos(watchLater);
00562         user->setYouTubeData(yd);
00563         appState.addSessionUser(*user);
00564     }
00565
00566     appState.setCurrentUser(new User(*user));
00567     std::cout << "[AppController] Logged in as '" << user->getUsername() << "'\n";
00568     reportProgress(0.45, "Checking blend memberships...");
00569
00570     // Check if this user belongs to a previously saved blend
00571     const std::string latestBlendResponse = http.get(http.build_latest_blend_endpoint(idIt->second));
00572     if (!http.wasLastRequestSuccessful(http.G)) {
00573         return;
00574     }
00575     std::optional<std::string> blendID = parseBlendID(latestBlendResponse);
00576     if (!blendID.has_value()) {
00577         return;
00578     }

```

```

00579
00580     const std::string participantResponse = http.get(http.build_blend_participant_endpoint(*blendID));
00581     if (!http.wasLastRequestSuccessful(http.G)) {
00582         return;
00583     }
00584
00585     const std::vector<std::string> participantIDs = parseParticipantIDs(participantResponse);
00586     reportProgress(0.6, "Loading blend participant data...");
00587     std::list<User> participants = loadParticipantsWithWatchLaterParallel(
00588         participantIDs,
00589         nullptr,
00590         kLoginParticipantLoadWorkerThreads
00591     );
00592
00593     if (participants.empty()) return; // no one has data yet - nothing to show
00594
00595     // Try to recover the blend title from the server
00596     std::string restoredTitle;
00597     try {
00598         const std::string blendsResp = http.get(http.build_user_blends_endpoint(idIt->second));
00599         if (http.wasLastRequestSuccessful(http.G)) {
00600             Json blendsParsed = Json::parse(blendsResp, nullptr, false);
00601             if (!blendsParsed.is_discarded() && blendsParsed.contains("blends") &&
blendsParsed["blends"].is_array()) {
00602                 for (const auto& item : blendsParsed["blends"]) {
00603                     if (item.value("blend_id", "") == *blendID) {
00604                         restoredTitle = item.value("title", "");
00605                         break;
00606                     }
00607                 }
00608             }
00609         } catch (...) {}
00610     } catch (...) {}
00611
00612     // Re-run the algorithm to produce a fresh blend from current persisted data
00613     reportProgress(0.85, "Generating blend feed...");
00614     currentBlend = std::make_unique<Blend>(
00615         RandomBlendAlgorithm(5).generateBlend(participants, restoredTitle)
00616     );
00617     currentBlend->setBlendID(*blendID);
00618     // Attempt to fill in missing metadata for the restored blend's feed videos.
00619     // If this blend contains videos from a large HTML import, they may lack title/channel
00620     // data until this point. Metadata enrichment here is deferred and happens on-demand.
00621     enrichActiveBlendFeedIfMissingMetadata();
00622     appState.setActiveBlend(currentBlend.get());
00623     appState.createChatRoom(*currentBlend);
00624
00625     // If this user has no data, leave a message for LoginPanel to display
00626     if (watchLater.empty()) {
00627         appState.setPendingBlendMessage(
00628             "You have been added to a blend. Upload your YouTube playlist "
00629             "data to contribute your videos to it.");
00630     }
00631
00632     std::cout << "[AppController] Restored blend '" << *blendID
00633         << "' with " << currentBlend->size() << " videos\n";
00634     reportProgress(1.0, "Login complete");
00635 }
00636
00637 void AppController::handleLogout(const EventPayload& payload) {
00638     // If there is a server-created pending account and the user is logging out
00639     // before uploading, attempt to delete it.
00640     if (pendingAccountCreated_ && pendingRegistration_.has_value() && isConnected) {
00641         try {
00642             const std::string delResp = http.del(
00643                 http.build_delete_user_endpoint(pendingRegistration_->getUserID()),
00644                 JsonRequestBuilder::buildDeleteAccountJson(pendingRegistration_->getPassword())
00645             );
00646             if (http.wasLastRequestSuccessful(http.D)) {
00647                 std::cout << "[AppController] Deleted pending account '"
00648                     << pendingRegistration_->getUserID() << "' on logout\n";
00649             } else {
00650                 std::cerr << "[AppController] Failed to delete pending account on logout, HTTP "
00651                     << http.getLastStatusCode() << " response=" << delResp << "\n";
00652             }
00653         } catch (const std::exception& ex) {
00654             std::cerr << "[AppController] handleLogout: failed to delete pending account: " << ex.what()
00655                 << "\n";
00656         }
00657         pendingRegistration_.reset();
00658         pendingAccountCreated_ = false;
00659     }
00660
00661     User* u = appState.getCurrentUser();
00662     if (u) {
00663         std::cout << "[AppController] Logging out '" << u->getUsername() << "'\n";
00664         delete u;

```

```

00664     }
00665     appState.setCurrentUser(nullptr);
00666     appState.clearSession();
00667 }
00668
00669 void ApplicationController::handleDeleteAccount(const EventPayload& payload) {
00670     auto userIt = payload.find("userID");
00671     auto pwIt = payload.find("password");
00672
00673     if (userIt == payload.end() || pwIt == payload.end()) {
00674         std::cerr << "[AppController] handleDeleteAccount: missing 'userID' or 'password'\n";
00675         return;
00676     }
00677
00678     if (!isConnected) {
00679         std::cerr << "[AppController] handleDeleteAccount: server unavailable in offline mode\n";
00680         return;
00681     }
00682
00683     try {
00684         const std::string response = http.del(
00685             http.build_delete_user_endpoint(userIt->second),
00686             JsonRequestBuilder::buildDeleteAccountJson(pwIt->second)
00687         );
00688
00689         if (!http.wasLastRequestSuccessful(http.D)) {
00690             std::cerr << "[AppController] handleDeleteAccount: delete failed, HTTP "
00691                 << http.getLastStatusCode() << " response=" << response << "\n";
00692             return;
00693         }
00694
00695         User* currentUser = appState.getCurrentUser();
00696         if (currentUser != nullptr && currentUser->getUserID() == userIt->second) {
00697             handleLogout({});
00698         }
00699
00700         std::cout << "[AppController] handleDeleteAccount: deleted user '"
00701             << userIt->second << "'\n";
00702
00703     } catch (const std::exception& e) {
00704         std::cerr << "[AppController] handleDeleteAccount: HTTP failed - "
00705             << e.what() << "\n";
00706     }
00707 }
00708
00709 void ApplicationController::handleUploadData(const EventPayload& payload) {
00710     auto fileIt = payload.find("filePath");
00711     auto userIt = payload.find("userID");
00712     if (fileIt == payload.end() || userIt == payload.end()) {
00713         std::cerr << "[AppController] handleUploadData: missing 'filePath' or 'userID' in payload\n";
00714         appState.setPendingUploadError("Upload failed: missing file path or user ID.");
00715         return;
00716     }
00717
00718     const std::string& filePath = fileIt->second;
00719     const std::string& userID = userIt->second;
00720
00721     // If there's a pending registration for this user, create the server account first.
00722     if (pendingRegistration_.has_value() && pendingRegistration_->getUserID() == userID) {
00723         if (!isConnected) {
00724             appState.setPendingUploadError("Cannot create account while offline. Please connect and
try again.");
00725             return;
00726         }
00727         std::string registerResponse;
00728         try {
00729             registerResponse = http.post(
00730                 http.REG_USER,
00731                 JsonRequestBuilder::buildRegisterJson(
00732                     pendingRegistration_->getUserID(),
00733                     pendingRegistration_->getUsername(),
00734                     pendingRegistration_->getEmail(),
00735                     pendingRegistration_->getPassword()
00736                 )
00737             );
00738             if (!http.wasLastRequestSuccessful(http.P)) {
00739                 if (http.getLastStatusCode() == http.DUPLICATE) {
00740                     appState.setPendingUploadError("Registration failed: username already taken.
Choose a different username.");
00741                 } else {
00742                     appState.setPendingUploadError("Registration failed: server returned an error.
Please try again.");
00743                 }
00744                 // clear pending to allow retry or new username flow
00745                 pendingRegistration_.reset();
00746                 return;
00747             }

```

```

00748         // success - clear pendingRegistration_
00749         pendingRegistration_.reset();
00750         std::cout << "[AppController] Account created on server for '" << userID << "'\n";
00751     } catch (const std::exception& ex) {
00752         appState.setPendingUploadError("Registration failed: network error while creating account.
Please try again.");
00753         std::cerr << "[AppController] handleUploadData: failed to create account for '" << userID <<
"' - '" << ex.what() << "\n";
00754         return;
00755     }
00756 }
00757
00758 std::cout << "[AppController] Starting file upload process for '" << filePath << "'\n";
00759
00760 std::list<Video> watchLater;
00761 try {
00762     // Multi-threaded import with progress feedback via logging.
00763     // Divides file processing across 3 worker threads for ~3x faster parsing of large files.
00764     // Progress callback logs to stdout so users can monitor progress.
00765     auto progressCallback = [](double progress) {
00766         int percent = static_cast<int>(progress * 100);
00767         std::cout << "[AppController] File parsing progress: " << percent << "%\n";
00768     };
00769
00770     std::cout << "[AppController] Starting multi-threaded file import...\n";
00771     watchLater = YouTubeDataImporter().importMultiThreaded(filePath, progressCallback);
00772     std::cout << "[AppController] File import complete: " << watchLater.size()
<< " videos extracted\n";
00773 } catch (const std::exception& ex) {
00774     std::cerr << "[AppController] handleUploadData: import failed for '"
<< filePath << "' - '" << ex.what() << "\n";
00775     appState.setPendingUploadError(
00776         "Upload failed: could not parse this file. Please select a supported Google Takeout export
(.csv, .html, or .htm).");
00777     return;
00778 }
00779
00780 if (watchLater.empty()) {
00781     std::cerr << "[AppController] handleUploadData: no videos found in '"
<< filePath << "'\n";
00782     appState.setPendingUploadError(
00783         "No videos were found in the selected file. Please verify you selected a valid YouTube
playlist CSV or watch-history HTML export from Google Takeout.");
00784     return;
00785 }
00786
00787 const std::string fileExt = lowerExtension(filePath);
00788 const bool isCsvUpload = (fileExt == ".csv");
00789 const bool isHtmlUpload = (fileExt == ".html" || fileExt == ".htm");
00790
00791 // Smart metadata enrichment strategy:
00792 // - On upload, enrich ONLY the first kMaxUploadEnrichmentCount videos to keep upload fast.
00793 // - CSV: always enrich first 50
00794 // - HTML <=500 videos: enrich all
00795 // - HTML >500 videos: enrich first 50, rest at display time
00796 //
00797 // This avoids blocking the UI while still providing good metadata for the most important videos.
00798
00799 std::list<Video> videosToEnrich;
00800 std::list<Video> videosNotEnriched;
00801
00802 if (isHtmlUpload && watchLater.size() > kHtmlUploadMetadataThreshold) {
00803     // Large HTML: enrich only first 50
00804     auto it = watchLater.begin();
00805     std::advance(it, std::min(watchLater.size(), kMaxUploadEnrichmentCount));
00806     videosToEnrich.splice(videosToEnrich.end(), watchLater, watchLater.begin(), it);
00807     videosNotEnriched.splice(videosNotEnriched.end(), watchLater);
00808     std::cout << "[AppController] Large HTML upload (" << (videosToEnrich.size() +
videosNotEnriched.size())
<< " videos): enriching first " << videosToEnrich.size() << ", deferring "
<< videosNotEnriched.size() << "\n";
00809 } else if (isCsvUpload && watchLater.size() > kMaxUploadEnrichmentCount) {
00810     // CSV with many videos: enrich first 50
00811     auto it = watchLater.begin();
00812     std::advance(it, std::min(watchLater.size(), kMaxUploadEnrichmentCount));
00813     videosToEnrich.splice(videosToEnrich.end(), watchLater, watchLater.begin(), it);
00814     videosNotEnriched.splice(videosNotEnriched.end(), watchLater);
00815     std::cout << "[AppController] CSV with " << (videosToEnrich.size() + videosNotEnriched.size())
<< " videos: enriching first " << videosToEnrich.size() << "\n";
00816 } else {
00817     // Small HTML or CSV: enrich all
00818     videosToEnrich = watchLater;
00819     std::cout << "[AppController] Small file (" << videosToEnrich.size()
<< " videos): enriching all\n";
00820 }

```

```

00830
00831 // Execute metadata enrichment only on the manageable subset.
00832 bool enriched = false;
00833 if (!videosToEnrich.empty()) {
00834     std::cout << "[AppController] Starting metadata enrichment for "
00835               << videosToEnrich.size() << " videos...\n";
00836     try {
00837         videosToEnrich = YouTubeMetadataFetcher(kYouTubeAPIKey).enrich(videosToEnrich);
00838         enriched = !kYouTubeAPIKey.empty();
00839         std::cout << "[AppController] Metadata enrichment complete\n";
00840     } catch (const std::exception& ex) {
00841         std::cerr << "[AppController] handleUploadData: metadata fetch failed: "
00842               << ex.what() << "\n";
00843         // Non-fatal - continue with whatever data we have
00844     }
00845 }
00846
00847 // Recombine enriched and non-enriched videos.
00848 videosToEnrich.splice(videosToEnrich.end(), videosNotEnriched);
00849 watchLater = videosToEnrich;
00850
00851 // Drop videos the API couldn't find - they are deleted, private, or unavailable.
00852 if (enriched) {
00853     watchLater.remove_if([](const Video& v) { return v.getTitle().empty(); });
00854     std::cout << "[AppController] " << watchLater.size()
00855               << " videos available after filtering unavailable entries\n";
00856 }
00857
00858 std::cout << "[AppController] Uploading " << watchLater.size()
00859           << " videos to server...\n";
00860
00861 // Attempt to post data to the server
00862 std::string saveWatchLaterResponse;
00863 try {
00864     saveWatchLaterResponse =
00865         http.post(
00866             http.build_watch_later_endpoint(userID),
00867             RequestJsonBuilder::buildWatchLaterJson(watchLater)
00868         );
00869     if (!http.wasLastRequestSuccessful(http.P)) {
00870         std::cerr << "[AppController] handleUploadData: failed to persist watch-later list, HTTP "
00871               << http.getLastStatusCode() << " response=" << saveWatchLaterResponse << "\n";
00872         appState.setPendingUploadError("Upload failed: server could not save your data. Please try
again.");
00873         return;
00874     }
00875
00876     cout << "[AppController] handleUploadData: Successful upload" << endl;
00877 } catch (exception& e) {
00878     std::cerr << "[AppController] handleUploadData: upload failed." << endl;
00879     cerr << "[AppController] handleUploadData - server response: " << saveWatchLaterResponse << endl;
00880     appState.setPendingUploadError("Upload failed: unable to reach the server. Please try
again.");
00881     return;
00882 }
00883
00884 // Clear pending registration state (account created + upload complete)
00885 if (pendingRegistration_.has_value() && pendingRegistration_>getUserID() == userID) {
00886     pendingRegistration_.reset();
00887     pendingAccountCreated_ = false;
00888 }
00889
00890 // Update the current user's in-memory YouTubeData
00891 User* currentUser = appState.getCurrentUser();
00892 if (currentUser && currentUser->getUserID() == userID) {
00893     YouTubeData yd;
00894     yd.setWatchLaterVideos(watchLater);
00895     currentUser->setYouTubeData(yd);
00896     appState.addSessionUser(*currentUser);
00897 }
00898
00899 std::cout << "[AppController] Saved " << watchLater.size()
00900           << " watch-later videos for user '" << userID << "'\n";
00901 appState.setPendingUploadError("");
00902
00903 // If this user is already a blend participant, re-run the algorithm
00904 // so their data is included and the blend stays up to date.
00905 const std::string latestBlendResponse = http.get(http.build_latest_blend_endpoint(userID));
00906 if (!http.wasLastRequestSuccessful(http.G)) {
00907     return;
00908 }
00909
00910 std::optional<std::string> blendID = parseBlendID(latestBlendResponse);
00911 if (!blendID.has_value()) {
00912     return;
00913 }
00914

```

```

00915     const std::string participantResponse = http.get(http.build_blend_participant_endpoint(*blendID));
00916     if (!http.wasLastRequestSuccessful(http.G)) {
00917         return;
00918     }
00919
00920     const std::vector<std::string> participantIDs = parseParticipantIDs(participantResponse);
00921     std::list<User> participants = loadParticipantsWithWatchLater(participantIDs, nullptr, false);
00922
00923     if (participants.empty()) {
00924         return;
00925     }
00926
00927     // Preserve existing blend title if available
00928     std::string existingTitle;
00929     Blend* active = appState.getActiveBlend();
00930     if (active) existingTitle = active->getTitle();
00931
00932     currentBlend = std::make_unique<Blend>(
00933         RandomBlendAlgorithm(5).generateBlend(participants, existingTitle)
00934     );
00935     currentBlend->setBlendID(*blendID);
00936     // After regenerating the blend with new participant data, try to enrich
00937     // any videos that may be missing metadata (e.g., from large HTML uploads).
00938     enrichActiveBlendFeedIfMissingMetadata();
00939     appState.setActiveBlend(currentBlend.get());
00940     appState.createChatRoom(*currentBlend);
00941
00942     std::cout << "[AppController] Re-generated blend '" << *blendID
00943               << "' after data upload - now " << currentBlend->size() << " videos\n";
00944 }
00945
00946 void AppController::handleCreateBlend(const EventPayload& payload) {
00947     // Extract blend title
00948     auto titleIt = payload.find("blendTitle");
00949     std::string blendTitle = (titleIt != payload.end()) ? titleIt->second : "";
00950
00951     // Collect participant userIDs from the payload (userID_0, userID_1, ...)
00952     std::vector<std::string> allParticipantIDs;
00953     for (int i = 0; ; ++i) {
00954         auto it = payload.find("userID_" + std::to_string(i));
00955         if (it == payload.end()) break;
00956         allParticipantIDs.push_back(it->second);
00957     }
00958
00959     if (allParticipantIDs.empty()) {
00960         std::cerr << "[AppController] handleCreateBlend: no participant IDs in payload\n";
00961         return;
00962     }
00963
00964     reportProgress(0.1, "Loading participant data...");
00965     std::vector<std::string> missingData;
00966     std::list<User> participants =
00967         loadParticipantsWithWatchLater(allParticipantIDs, &missingData, false);
00968
00969     // Report users without uploaded data back to the UI via AppState
00970     // (non-existent users are excluded - they should have been rejected at the UI level)
00971     appState.setUsersWithoutData(missingData);
00972
00973     // Enforce upload: if any participant is missing Watch Later data, refuse to create the blend.
00974     if (!missingData.empty()) {
00975         appState.setIsBlendGenerating(false);
00976         std::cerr << "[AppController] handleCreateBlend: cannot create blend - the following
participants have not uploaded Watch Later data:\n";
00977         for (const auto& uid : missingData) std::cerr << " " << uid << "\n";
00978         // UI should show users without data via appState; do not proceed.
00979         return;
00980     }
00981
00982     if (participants.empty()) {
00983         std::cerr << "[AppController] handleCreateBlend: no participants have data\n";
00984         return;
00985     }
00986
00987     appState.setIsBlendGenerating(true);
00988     reportProgress(0.3, "Generating blend...");
00989
00990     currentBlend = std::make_unique<Blend>(
00991         RandomBlendAlgorithm(5).generateBlend(participants, blendTitle)
00992     );
00993
00994     // Persist the blend and all valid participants (existing users, even those
00995     // without data yet, so they are found on their next login).
00996     // Non-existent users (notFound) are excluded - they have no DB row.
00997     std::vector<std::string> validParticipantIDs;
00998     for (const auto& uid : allParticipantIDs) {
00999         if (lookupUser(uid)) {
01000             validParticipantIDs.push_back(uid);

```

```

01001     }
01002 }
01003
01004 if (validParticipantIDs.empty()) {
01005     appState.setIsBlendGenerating(false);
01006     std::cerr << "[AppController] handleCreateBlend: no valid participants found\n";
01007     return;
01008 }
01009
01010 reportProgress(0.55, "Saving blend to server...");
01011 User* creator = appState.getCurrentUser();
01012 std::string creatorID = creator ? creator->getUserID() : allParticipantIDs[0];
01013 const std::string createBlendResponse =
01014     http.post(http.BLEND,
01015               RequestJsonBuilder::buildBlendJson(currentBlend->getBlendID(),
01016                                                  currentBlend->getTitle(),
01017                                                  creatorID,
01018                                                  currentBlend->getAlgorithmUsed(),
01019                                                  validParticipantIDs));
01020
01021 if (!http.wasLastRequestSuccessful(http.P)) {
01022     appState.setIsBlendGenerating(false);
01023     std::cerr << "[AppController] handleCreateBlend: failed to persist blend, HTTP "
01024               << http.getLastStatusCode() << " response=" << createBlendResponse << "\n";
01025     return;
01026 }
01027
01028 // Attempt to fill in missing metadata for the newly created blend's videos.
01029 // If participants have large HTML-imported watch lists, their contribution videos
01030 // may lack enriched metadata until now.
01031 reportProgress(0.8, "Enriching video metadata...");
01032 enrichActiveBlendFeedIfMissingMetadata();
01033 appState.setActiveBlend(currentBlend.get());
01034 appState.createChatRoom(*currentBlend);
01035 appState.setIsBlendGenerating(false);
01036 reportProgress(1.0, "Blend created!");
01037
01038 // Register with server so the chat room is provisioned there too.
01039 registerBlendOnServer(*currentBlend, creatorID);
01040
01041 std::cout << "[AppController] Blend created with "
01042           << currentBlend->size() << " videos\n";
01043 }
01044
01045 bool AppController::lookupUser(const std::string& userID) {
01046     try {
01047         // User existence is validated via auth profile lookup. This avoids
01048         // false negatives for valid users who have not uploaded watch-later yet.
01049         http.get(http.build_auth_user_lookup_endpoint(userID));
01050         return http.wasLastRequestSuccessful(http.G);
01051     } catch (const std::exception&) {
01052         return false;
01053     }
01054 }
01055 // -- Private helpers -----
01056
01057 bool AppController::registerBlendOnServer(const Blend& blend,
01058                                          const std::string& creatorID) {
01059     if (!isConnected) return false;
01060
01061     // Build participant array: ["id1","id2",...]
01062     std::string participantsJson = "[";
01063     bool first = true;
01064     for (const User& p : blend.getParticipants()) {
01065         if (!first) participantsJson += ",";
01066         participantsJson += ModelJson::quote(p.getUserID());
01067         first = false;
01068     }
01069     participantsJson += "];";
01070
01071     const std::string body =
01072         "{ \"blend_id\": \"" + ModelJson::quote(blend.getBlendID()) + "\", "
01073         + "\"title\": \"" + ModelJson::quote(blend.getTitle()) + "\", "
01074         + "\"creator_id\": \"" + ModelJson::quote(creatorID) + "\", "
01075         + "\"algorithm\": \"" + ModelJson::quote(blend.getAlgorithmUsed()) + "\", "
01076         + "\"participants\": " + participantsJson + " }";
01077
01078     try {
01079         http.post(http.BLEND, body);
01080         if (http.wasLastRequestSuccessful(http.P)) {
01081             std::cout << "[AppController] registerBlendOnServer: blend '"
01082                   << blend.getBlendID() << "' registered\n";
01083             return true;
01084         }
01085         // 409 Conflict means it already exists - treat as success.
01086         if (http.getLastStatusCode() == http.DUPLICATE) {
01087             std::cout << "[AppController] registerBlendOnServer: blend '"

```



```

01088         « blend.getBlendID() « " " already on server\n";
01089         return true;
01090     }
01091     std::cerr « "[AppController] registerBlendOnServer: server returned "
01092               « http.getLastStatusCode() « "\n";
01093 } catch (const std::exception& e) {
01094     std::cerr « "[AppController] registerBlendOnServer: HTTP failed - "
01095               « e.what() « "\n";
01096 }
01097 return false;
01098 }
01099
01100 void AppController::enrichIfMissingMetadata(const std::string& userID,
01101                                             std::list<Video>& videos) {
01102     bool needsEnrichment = std::any_of(videos.begin(), videos.end(),
01103         [](const Video& v) {
01104             return v.getTitle().empty() ||
01105                    v.getChannelID().empty() ||
01106                    v.getChannelName().empty() ||
01107                    v.getThumbnailURL().empty() ||
01108                    v.getChannelLogoURL().empty();
01109         });
01110     if (!needsEnrichment) return;
01111
01112     std::cout « "[AppController] enrichIfMissingMetadata: re-enriching "
01113              « videos.size() « " videos for '" « userID « "'\n";
01114     try {
01115         videos = YouTubeMetadataFetcher(kYouTubeAPIKey).enrich(videos);
01116         // Drop any video still missing a title - confirmed unavailable on YouTube
01117         if (!kYouTubeAPIKey.empty())
01118             videos.remove_if([](const Video& v) { return v.getTitle().empty(); });
01119
01120         const std::string saveResponse =
01121             http.post(http.build_watch_later_endpoint(userID),
01122                      RequestJsonBuilder::buildWatchLaterJson(videos));
01123         if (!http.wasLastRequestSuccessful(http.P)) {
01124             std::cerr « "[AppController] enrichIfMissingMetadata: failed to persist refreshed metadata
for '"
01125                     « userID « "'", HTTP " « http.getLastStatusCode()
01126                     « " response=" « saveResponse « "\n";
01127         }
01128     } catch (const std::exception& ex) {
01129         std::cerr « "[AppController] enrichIfMissingMetadata failed for '"
01130                 « userID « "': " « ex.what() « "\n";
01131     }
01132 }
01133
01134 std::list<Video> AppController::enrichMissingMetadataSubsetMultithreaded(
01135     const std::list<Video>& videos,
01136     std::size_t maxVideosToEnrich,
01137     std::size_t workerThreads,
01138     const std::function<std::list<Video>(const std::list<Video>&)>& enrichChunkFn) {
01139     if (videos.empty() || maxVideosToEnrich == 0 || workerThreads == 0) {
01140         return videos;
01141     }
01142
01143     std::list<Video> updatedVideos = videos;
01144
01145     std::vector<Video> pending;
01146     pending.reserve(std::min(maxVideosToEnrich, videos.size()));
01147     for (const auto& video : videos) {
01148         if (!isMetadataIncomplete(video)) {
01149             continue;
01150         }
01151         pending.push_back(video);
01152         if (pending.size() >= maxVideosToEnrich) {
01153             break;
01154         }
01155     }
01156
01157     if (pending.empty()) {
01158         return updatedVideos;
01159     }
01160
01161     const std::size_t workerCount = std::max<std::size_t>(
01162         1,
01163         std::min<std::size_t>(workerThreads, pending.size())
01164     );
01165
01166     std::unordered_map<std::string, Video> enrichedByID;
01167     std::mutex resultMutex;
01168     std::vector<std::thread> workers;
01169     workers.reserve(workerCount);
01170
01171     for (std::size_t workerIndex = 0; workerIndex < workerCount; ++workerIndex) {
01172         const std::size_t start = workerIndex * pending.size() / workerCount;
01173         const std::size_t end = (workerIndex + 1) * pending.size() / workerCount;

```

```

01174         if (start >= end) {
01175             continue;
01176         }
01177
01178         workers.emplace_back([&, start, end]() {
01179             std::list<Video> chunk;
01180             for (std::size_t i = start; i < end; ++i) {
01181                 chunk.push_back(pending[i]);
01182             }
01183
01184             std::list<Video> chunkEnriched;
01185             try {
01186                 chunkEnriched = enrichChunkFn(chunk);
01187             } catch (...) {
01188                 return;
01189             }
01190
01191             std::lock_guard<std::mutex> lock(resultMutex);
01192             for (const auto& enriched : chunkEnriched) {
01193                 if (hasCompleteMetadata(enriched)) {
01194                     enrichedByID.insert_or_assign(enriched.getVideoID(), enriched);
01195                 }
01196             }
01197         });
01198     }
01199
01200     for (auto& worker : workers) {
01201         if (worker.joinable()) {
01202             worker.join();
01203         }
01204     }
01205
01206     for (auto& video : updatedVideos) {
01207         auto enrichedIt = enrichedByID.find(video.getVideoID());
01208         if (enrichedIt != enrichedByID.end()) {
01209             video = enrichedIt->second;
01210         }
01211     }
01212
01213     return updatedVideos;
01214 }
01215
01216 void AppController::enrichActiveBlendFeedIfMissingMetadata() {
01217     // Re-enriches only the currently displayed blend's video feed when any metadata
01218     // fields are incomplete. This keeps display quality high without re-processing
01219     // full watch-history sized lists on the UI path.
01220     //
01221     // This complements handleUploadData which may skip enrichment for large HTML
01222     // uploads. By deferring metadata fetch to blend-display time, we:
01223     // - Keep upload fast (avoid blocking the UI during file ingestion).
01224     // - Enrich only a small subset of videos (the active blend feed).
01225     // - Still provide rich metadata when users view their blend.
01226
01227     if (!currentBlend) {
01228         return;
01229     }
01230
01231     std::list<Video> feedVideos = currentBlend->getVideoList();
01232     const bool needsEnrichment =
01233         std::any_of(feedVideos.begin(), feedVideos.end(), isMetadataIncomplete);
01234
01235     if (!needsEnrichment) {
01236         return;
01237     }
01238
01239     std::size_t incompleteCount = 0;
01240     for (const auto& video : feedVideos) {
01241         if (isMetadataIncomplete(video)) {
01242             ++incompleteCount;
01243         }
01244     }
01245     const std::size_t deferredCount = incompleteCount > kMaxBlendFeedEnrichmentCount
01246         ? (incompleteCount - kMaxBlendFeedEnrichmentCount)
01247         : 0;
01248
01249     std::cout << "[AppController] enrichActiveBlendFeedIfMissingMetadata: enriching up to "
01250         << kMaxBlendFeedEnrichmentCount << " blend videos ("
01251         << deferredCount << " deferred)\n";
01252
01253     try {
01254         const std::list<Video> enrichedVideos = enrichMissingMetadataSubsetMultithreaded(
01255             feedVideos,
01256             kMaxBlendFeedEnrichmentCount,
01257             kBlendMetadataWorkerThreads,
01258             [](const std::list<Video>& chunk) {
01259                 return YouTubeMetadataFetcher(kYouTubeAPIKey).enrich(chunk);
01260             }

```

```

01261     };
01262
01263     std::size_t appliedCount = 0;
01264     auto oldIt = feedVideos.begin();
01265     auto newIt = enrichedVideos.begin();
01266     for (; oldIt != feedVideos.end() && newIt != enrichedVideos.end(); ++oldIt, ++newIt) {
01267         if (isMetadataIncomplete(*oldIt) && hasCompleteMetadata(*newIt)) {
01268             ++appliedCount;
01269         }
01270     }
01271
01272     if (appliedCount > 0) {
01273         currentBlend->setVideoList(enrichedVideos);
01274     }
01275
01276     std::cout << "[AppController] enrichActiveBlendFeedIfMissingMetadata: completed "
01277               << appliedCount << " updates using "
01278               << kBlendMetadataWorkerThreads << " workers\n";
01279 } catch (const std::exception& ex) {
01280     std::cerr << "[AppController] enrichActiveBlendFeedIfMissingMetadata failed: "
01281               << ex.what() << "\n";
01282 }
01283 }
01284
01285 void AppController::handlePlayVideo(const EventPayload& payload) {
01286     // TODO: read "videoID" from payload
01287     // TODO: open the video - exact mechanism (browser, in-app) TBD
01288     std::cout << "[AppController] handlePlayVideo called (stub)\n";
01289 }
01290
01291 void AppController::handleRefresh(const EventPayload& /*payload*/) {
01292     reportProgress(0.05, "Preparing blend refresh...");
01293
01294     Blend* existing = appState.getActiveBlend();
01295     if (!existing) {
01296         std::cerr << "[AppController] handleRefresh: no active blend\n";
01297         return;
01298     }
01299
01300     // Build the set of video IDs already shown in the current blend
01301     std::set<std::string> shownIDs;
01302     for (const auto& v : existing->getVideoList())
01303         shownIDs.insert(v.getVideoID());
01304
01305     // Reload each participant's full video pool from the backend, then exclude shown videos
01306     std::vector<std::string> participantIDs;
01307     for (const User& participant : existing->getParticipants()) {
01308         participantIDs.push_back(participant.getUserID());
01309     }
01310     reportProgress(0.2, "Loading participant video pools...");
01311
01312     // Lightweight load for refresh: fetch each participant's watch-later list
01313     // but DO NOT call enrichIfMissingMetadata (that can trigger slow network calls).
01314     std::list<User> participants;
01315     for (std::size_t i = 0; i < participantIDs.size(); ++i) {
01316         const auto& uid = participantIDs[i];
01317         const std::string watchLaterResponse = http.get(http.build_watch_later_endpoint(uid));
01318         if (!http.wasLastRequestSuccessful(http.G)) continue;
01319
01320         std::list<Video> allVideos = parseWatchLaterVideos(watchLaterResponse);
01321         if (allVideos.empty()) continue;
01322
01323         // Build a fresh pool of videos not yet shown
01324         std::list<Video> freshPool;
01325         for (const auto& v : allVideos) {
01326             if (shownIDs.find(v.getVideoID()) == shownIDs.end()) {
01327                 freshPool.push_back(v);
01328             }
01329         }
01330         // If all of this user's videos were already shown, reset to the full pool
01331         if (freshPool.empty()) {
01332             freshPool = allVideos;
01333         }
01334
01335         YouTubeData pyd;
01336         pyd.setWatchLaterVideos(freshPool);
01337         User p(uid, uid, "", "");
01338         p.setYouTubeData(pyd);
01339         participants.push_back(p);
01340
01341         if (!participantIDs.empty()) {
01342             const double ratio = static_cast<double>(i + 1) /
01343                               static_cast<double>(participantIDs.size());
01344             reportProgress(0.2 + (ratio * 0.35), "Loading participant video pools...");
01345         }
01346     }

```

```

01347     if (participants.empty()) {
01348         std::cerr << "[AppController] handleRefresh: no participants have data\n";
01349         return;
01350     }
01351
01352     std::string refreshTitle = existing->getTitle();
01353     std::string refreshID = existing->getBlendID();
01354     reportProgress(0.62, "Generating refreshed blend...");
01355     currentBlend = std::make_unique<Blend>(
01356         RandomBlendAlgorithm(5).generateBlend(participants, refreshTitle)
01357     );
01358     currentBlend->setBlendID(refreshID);
01359
01360     // Prevent duplicate videos within the same refreshed blend payload.
01361     std::set<std::string> seenVideoIDs;
01362     std::list<Video> uniqueVideos;
01363     for (const auto& video : currentBlend->getVideoList()) {
01364         if (seenVideoIDs.insert(video.getVideoID()).second) {
01365             uniqueVideos.push_back(video);
01366         }
01367     }
01368     currentBlend->setVideoList(uniqueVideos);
01369     reportProgress(0.8, "Enriching missing metadata...");
01370     // Attempt to enrich any missing metadata in the refreshed blend feed.
01371     // After filtering and re-sampling, some videos may lack enriched fields, especially
01372     // if the participant data originated from large HTML imports.
01373     enrichActiveBlendFeedIfMissingMetadata();
01374
01375     appState.setActiveBlend(currentBlend.get());
01376     reportProgress(1.0, "Refresh complete");
01377
01378     std::cout << "[AppController] Blend refreshed with "
01379               << currentBlend->size() << " new videos\n";
01380 }
01381
01382 void AppController::handleOpenChat(const EventPayload& payload) {
01383     auto blendIt = payload.find("blendID");
01384     auto userIt = payload.find("userID");
01385     if (blendIt == payload.end() || userIt == payload.end()) {
01386         std::cerr << "[AppController] handleOpenChat: missing 'blendID' or 'userID'\n";
01387         return;
01388     }
01389
01390     ChatRoom* room = appState.getActiveChatRoom();
01391     if (!room) {
01392         std::cerr << "[AppController] handleOpenChat: no active ChatRoom\n";
01393         return;
01394     }
01395
01396     if (!isConnected) {
01397         std::cout << "[AppController] handleOpenChat: offline - skipping history fetch\n";
01398         return;
01399     }
01400
01401     try {
01402         const std::string endpoint =
01403             http.build_chat_history_endpoint(blendIt->second, userIt->second);
01404         std::string response = http.get(endpoint);
01405
01406         if (http.getLastStatusCode() == http.MISS_RM_CNTXT) {
01407             // Blend not registered on server yet - provision it now and retry.
01408             Blend* blend = appState.getActiveBlend();
01409             if (blend) {
01410                 User* creator = appState.getCurrentUser();
01411                 const std::string creatorID = creator->getUserID() : userIt->second;
01412                 if (registerBlendOnServer(*blend, creatorID)) {
01413                     response = http.get(endpoint);
01414                 }
01415             }
01416         }
01417
01418         if (!http.wasLastRequestSuccessful(http.G)) {
01419             std::cerr << "[AppController] handleOpenChat: server returned "
01420                   << http.getLastStatusCode() << " - leaving ChatRoom empty\n";
01421             return;
01422         }
01423
01424         std::list<Message> history = parseChatHistory(response);
01425         room->clearMessages();
01426         for (const Message& msg : history) {
01427             room->addMessage(msg);
01428         }
01429         std::cout << "[AppController] handleOpenChat: loaded "
01430               << history.size() << " messages for blend '"
01431               << blendIt->second << "'\n";
01432     } catch (const std::exception& e) {
01433

```

```

01434         std::cerr << "[AppController] handleOpenChat: HTTP failed - "
01435                     << e.what() << "\n";
01436     }
01437 }
01438
01439 std::list<Message> AppController::parseChatHistory(const std::string& response) {
01440     std::list<Message> result;
01441
01442     Json parsed = Json::parse(response, nullptr, false);
01443     if (parsed.is_discarded() || !parsed.is_object() || !parsed.contains("messages")) {
01444         return result;
01445     }
01446
01447     const Json& messages = parsed["messages"];
01448     if (!messages.is_array()) {
01449         return result;
01450     }
01451
01452     for (const auto& item : messages) {
01453         if (!item.is_object()) {
01454             continue;
01455         }
01456
01457         const std::string sender = item.value("sender_id", "");
01458         const std::string content = item.value("content", "");
01459         const std::string sentAt = item.value("sent_at", "");
01460
01461         if (sender.empty() || content.empty()) {
01462             continue;
01463         }
01464
01465         std::time_t ts = std::time(nullptr);
01466         if (!sentAt.empty()) {
01467             std::tm tm{};
01468             if (strptime(sentAt.c_str(), "%Y-%m-%dT%H:%M:%SZ", &tm) != nullptr) {
01469                 ts = timegm(&tm);
01470             }
01471         }
01472
01473         result.emplace_back(sender, content, ts);
01474     }
01475
01476     return result;
01477 }
01478
01479 void AppController::handleSendMessage(const EventPayload& payload) {
01480     auto idIt = payload.find("userID");
01481     auto textIt = payload.find("text");
01482     if (idIt == payload.end() || textIt == payload.end()) return;
01483
01484     ChatRoom* room = appState.getActiveChatRoom();
01485     if (!room) return;
01486
01487     if (!room->isParticipant(idIt->second)) {
01488         std::cerr << "[AppController] handleSendMessage: '"
01489                 << idIt->second << "' is not a participant\n";
01490         return;
01491     }
01492
01493     room->addMessage(idIt->second, textIt->second);
01494 }
01495
01496 const User* AppController::get_current_user() {
01497     return appState.getCurrentUser();
01498 }
01499
01500 std::string AppController::get_current_username() {
01501     if (get_current_user() == nullptr) return "";
01502
01503     return appState.getCurrentUser()->getUsername();
01504 }
01505
01506 std::string AppController::get_current_email() {
01507     if (get_current_user() == nullptr) return "";
01508
01509     return appState.getCurrentUser()->getEmail();
01510 }
01511
01512 std::vector<AppController::BlendSummary> AppController::fetchUserBlends() {
01513     std::vector<BlendSummary> blends;
01514     User* user = appState.getCurrentUser();
01515     if (!user || !isConnected) return blends;
01516
01517     try {
01518         // Try the new /users/:userID/blends endpoint first (returns all blends).
01519         const std::string response = http.get(http.build_user_blends_endpoint(user->getUserID()));
01520         if (http.wasLastRequestSuccessful(http.G)) {

```

```

01521         Json parsed = Json::parse(response, nullptr, false);
01522         if (!parsed.is_discarded() && parsed.contains("blends") && parsed["blends"].is_array()) {
01523             for (const auto& item : parsed["blends"]) {
01524                 if (!item.is_object()) continue;
01525                 BlendSummary s;
01526                 s.blendID = item.value("blend_id", "");
01527                 s.title = item.value("title", s.blendID);
01528                 if (!s.blendID.empty())
01529                     blends.push_back(std::move(s));
01530             }
01531             return blends;
01532         }
01533     }
01534
01535     // Fallback: old server only has GET /users/:userID/blend (single latest blend).
01536     const std::string fallback = http.get(http.build_latest_blend_endpoint(user->getUserID()));
01537     if (http.wasLastRequestSuccessful(http.G)) {
01538         std::optional<std::string> blendID = parseBlendID(fallback);
01539         if (blendID.has_value() && !blendID->empty()) {
01540             BlendSummary s;
01541             s.blendID = *blendID;
01542             // Try to use the active blend's title if it matches
01543             Blend* active = appState.getActiveBlend();
01544             s.title = (active && active->getBlendID() == s.blendID)
01545                 ? active->getTitle() : s.blendID;
01546             if (s.title.empty()) s.title = s.blendID;
01547             blends.push_back(std::move(s));
01548         }
01549     }
01550     } catch (const std::exception& e) {
01551         std::cerr << "[AppController] fetchUserBlends: " << e.what() << "\n";
01552     }
01553     return blends;
01554 }
01555
01556 void AppController::handleLeaveBlend(const EventPayload& payload) {
01557     auto blendIt = payload.find("blendID");
01558     auto titleIt = payload.find("blendTitle");
01559     if (blendIt == payload.end()) return;
01560
01561     User* user = appState.getCurrentUser();
01562     if (!user) return;
01563
01564     const std::string& blendID = blendIt->second;
01565     const std::string blendTitle = (titleIt != payload.end()) ? titleIt->second : blendID;
01566
01567     // Send system message to chat before leaving
01568     if (isConnected) {
01569         try {
01570             const std::string systemMsg = user->getUsername() + " has left " + blendTitle + ".";
01571             // Post via the WebSocket-compatible chat message endpoint
01572             ChatRoom* room = appState.getChatRoom(blendID);
01573             if (room) {
01574                 room->addMessage("system", systemMsg);
01575             }
01576         } catch (...) {}
01577     }
01578
01579     // Notify server
01580     if (isConnected) {
01581         try {
01582             const std::string body =
01583                 "{\"user_id\":\"" + ModelJson::quote(user->getUserID()) + "\"}";
01584             http.post(http.build_leave_blend_endpoint(blendID), body);
01585         } catch (const std::exception& e) {
01586             std::cerr << "[AppController] handleLeaveBlend: " << e.what() << "\n";
01587         }
01588     }
01589
01590     // Clear local state if this was the active blend
01591     Blend* active = appState.getActiveBlend();
01592     if (active && active->getBlendID() == blendID) {
01593         currentBlend.reset();
01594         appState.setActiveBlend(nullptr);
01595     }
01596
01597     std::cout << "[AppController] Left blend '" << blendTitle << "'\n";
01598 }
01599
01600 void AppController::handleSelectBlend(const EventPayload& payload) {
01601     auto blendIt = payload.find("blendID");
01602     if (blendIt == payload.end()) return;
01603
01604     const std::string& blendID = blendIt->second;
01605
01606     // Fetch participants for this blend
01607     reportProgress(0.1, "Fetching participants...");

```

```

01608     const std::string participantResponse = http.get(http.build_blend_participant_endpoint(blendID));
01609     if (!http.wasLastRequestSuccessful(http.G)) {
01610         std::cerr << "[AppController] handleSelectBlend: failed to fetch participants\n";
01611         return;
01612     }
01613
01614     reportProgress(0.3, "Loading participant data...");
01615     const std::vector<std::string> participantIDs = parseParticipantIDs(participantResponse);
01616     std::list<User> participants = loadParticipantsWithWatchLater(participantIDs, nullptr, false);
01617
01618     if (participants.empty()) {
01619         std::cerr << "[AppController] handleSelectBlend: no participants have data\n";
01620         return;
01621     }
01622
01623     // Extract title from payload if available
01624     auto titleIt = payload.find("blendTitle");
01625     std::string title = (titleIt != payload.end()) ? titleIt->second : "";
01626
01627     reportProgress(0.6, "Generating blend feed...");
01628     currentBlend = std::make_unique<Blend>(
01629         RandomBlendAlgorithm(5).generateBlend(participants, title)
01630     );
01631     currentBlend->setBlendID(blendID);
01632     // Attempt to fill in missing metadata for the selected blend's feed.
01633     // If the blend contains videos from an HTML import that was saved without enrichment,
01634     // metadata will be fetched here to provide a complete display experience.
01635     reportProgress(0.8, "Enriching video metadata...");
01636     enrichActiveBlendFeedIfMissingMetadata();
01637     appState.setActiveBlend(currentBlend.get());
01638     appState.createChatRoom(*currentBlend);
01639     reportProgress(1.0, "Blend loaded!");
01640
01641     std::cout << "[AppController] Switched to blend '" << blendID
01642               << "' with " << currentBlend->size() << " videos\n";
01643 }

```

7.51 AppController.hpp File Reference

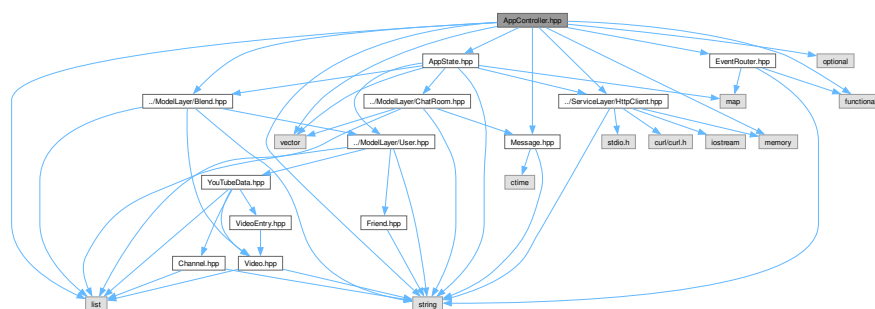
Central application coordinator.

```

#include <memory>
#include <list>
#include <functional>
#include <optional>
#include <string>
#include <vector>
#include "AppState.hpp"
#include "EventRouter.hpp"
#include "../ModelLayer/Blend.hpp"
#include "../ModelLayer/Message.hpp"
#include "../ServiceLayer/HttpClient.hpp"

```

Include dependency graph for AppController.hpp:




```

00044     explicit AppController();
00045
00050     explicit AppController(const std::string& dbPath);
00051
00056     ~AppController();
00057
00059     EventRouter& getEventRouter();
00060
00062     void setProgressReporter(std::function<void(double, const std::string&)> reporter);
00063
00065     void clearProgressReporter();
00066
00071     void handleRegister(const EventPayload& payload);
00072
00077     void handleLogin(const EventPayload& payload);
00078
00083     void handleLogout(const EventPayload& payload);
00084
00089     void handleDeleteAccount(const EventPayload& payload);
00090
00097     void handleUploadData(const EventPayload& payload);
00098
00104     void handleCreateBlend(const EventPayload& payload);
00105
00110     void handlePlayVideo(const EventPayload& payload);
00111
00118     void handleRefresh(const EventPayload& payload);
00119
00125     void handleOpenChat(const EventPayload& payload);
00126
00136     bool lookupUser(const std::string& userID);
00137
00142     void handleSendMessage(const EventPayload& payload);
00143
00144     // --- helpers for GUI to easily access user information such as their
00145
00150     const User* get_current_user();
00151
00156     std::string get_current_username();
00157
00162     std::string get_current_email();
00163
00171     struct BlendSummary {
00172         std::string blendID;
00173         std::string title;
00174     };
00175
00180     std::vector<BlendSummary> fetchUserBlends();
00181
00187     void handleLeaveBlend(const EventPayload& payload);
00188
00194     void handleSelectBlend(const EventPayload& payload);
00195
00203     static std::list<Message> parseChatHistory(const std::string& response);
00204
00217     static std::list<Video> enrichMissingMetadataSubsetMultithreaded(
00218         const std::list<Video>& videos,
00219         std::size_t maxVideosToEnrich,
00220         std::size_t workerThreads,
00221         const std::function<std::list<Video>(const std::list<Video>&)>& enrichChunkFn
00222     );
00223 private:
00224     AppState& appState; // Singleton session state
00225     EventRouter eventRouter; // Messenger between UI and controller
00226     const std::string server_url; // url and port to the server
00227     HttpClient http; // http client wrapper for server communications
00228     bool isConnected; // tracks whether AppState has a connection to the server
00229     std::function<void(double, const std::string&)> progressReporter; // optional UI progress sink
00230
00232     std::unique_ptr<Blend> currentBlend;
00233
00238     void registerEvents();
00239
00247     void enrichIfMissingMetadata(const std::string& userID,
00248                                 std::list<Video>& videos);
00249
00255     void enrichActiveBlendFeedIfMissingMetadata();
00256
00269     static std::optional<User> parseUserFromAuthResponse(const std::string& response,
00270                                                         const std::string& fallbackPassword);
00271
00278     static std::optional<std::string> parseBlendID(const std::string& response);
00279
00288     static std::vector<std::string> parseParticipantIDs(const std::string& response);
00289
00298     static std::list<Video> parseWatchLaterVideos(const std::string& response);
00299

```

```

00311         std::list<User> loadParticipantsWithWatchLater(const std::vector<std::string>& participantIDs,
00312             std::vector<std::string>* missingData,
00313             bool enrichParticipantLists = false);
00314
00315     std::list<User> loadParticipantsWithWatchLaterParallel(
00316         const std::vector<std::string>& participantIDs,
00317         std::vector<std::string>* missingData,
00318         std::size_t workerThreads
00319     );
00320
00321     void reportProgress(double progress, const std::string& message);
00322
00323     bool registerBlendOnServer(const Blend& blend, const std::string& creatorID);
00324
00325     // Holds a pending registration until upload completes (or is cancelled).
00326     std::optional<User> pendingRegistration_;
00327     bool pendingAccountCreated_ = false;
00328
00329 };
00330
00331 #endif // APPCONTROLLER_HPP

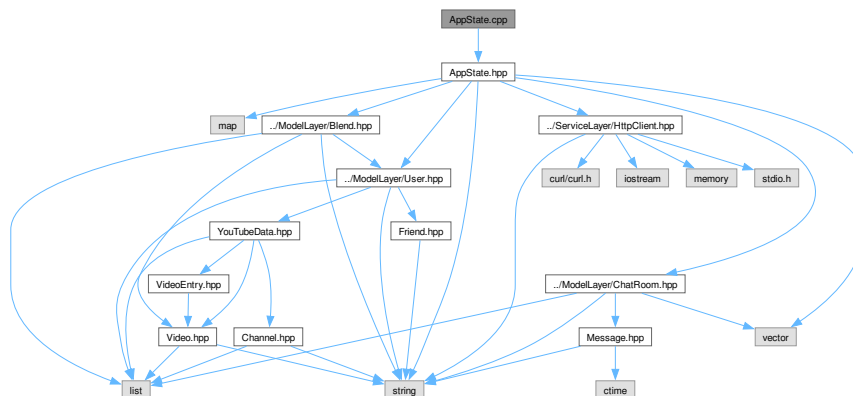
```

7.53 AppState.cpp File Reference

Implementation for [AppState](#).

```
#include "AppState.hpp"
```

Include dependency graph for AppState.cpp:



7.53.1 Detailed Description

Implementation for [AppState](#).

Author

Jasmine Kumar

Definition in file [AppState.cpp](#).

7.54 AppState.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "AppState.hpp"
00008
00009 using namespace std;
00010
00011 AppState::AppState():
00012     currentUser(nullptr),
00013     activeBlend(nullptr),
00014     isBlendGenerating(false) {}
00015
00016 // -- Singleton -----
00017
00018 AppState& AppState::getInstance() {
00019     // Constructed once on first call; guaranteed by C++11 to be thread-safe.
00020     static AppState instance;
00021     return instance;
00022 }
00023
00024 // -- Current User -----
00025
00026 User* AppState::getCurrentUser() const {
00027     return currentUser;
00028 }
00029
00030 void AppState::setCurrentUser(User* user) {
00031     // If a different user is logging in, the active blend is no longer valid.
00032     if (currentUser != nullptr && user != nullptr &&
00033         currentUser->getUserID() != user->getUserID()) {
00034         activeBlend = nullptr;
00035         chatRooms.clear();
00036     }
00037     currentUser = user;
00038 }
00039
00040 // -- Active Blend -----
00041
00042 Blend* AppState::getActiveBlend() const {
00043     return activeBlend;
00044 }
00045
00046 void AppState::setActiveBlend(Blend* blend) {
00047     activeBlend = blend;
00048 }
00049
00050 // -- Blend Generation Flag -----
00051
00052 bool AppState::isGeneratingBlend() const {
00053     return isBlendGenerating;
00054 }
00055
00056 void AppState::setIsBlendGenerating(bool flag) {
00057     isBlendGenerating = flag;
00058 }
00059
00060 // -- Chat Rooms -----
00061
00062 ChatRoom* AppState::getActiveChatRoom() {
00063     if (activeBlend == nullptr) return nullptr;
00064     return getChatRoom(activeBlend->getBlendID());
00065 }
00066
00067 void AppState::createChatRoom(const Blend& blend) {
00068     // Build participant ID list from the Blend's User list
00069     std::vector<std::string> participantIDs;
00070     for (const auto& user : blend.getParticipants()) {
00071         participantIDs.push_back(user.getUserID());
00072     }
00073
00074     // Insert or replace the room for this blendID
00075     chatRooms.emplace(
00076         std::piecewise_construct,
00077         std::forward_as_tuple(blend.getBlendID()),
00078         std::forward_as_tuple(blend.getBlendID(), participantIDs)
00079     );
00080 }
00081
00082 ChatRoom* AppState::getChatRoom(const std::string& blendID) {
00083     auto it = chatRooms.find(blendID);
00084     if (it == chatRooms.end()) return nullptr;
00085     return &it->second;
00086 }

```

```

00087
00088 // -- Session Users -----
00089
00090 void AppState::addSessionUser(const User& user) {
00091     sessionUsers.insert_or_assign(user.getUserID(), user);
00092 }
00093
00094 std::map<std::string, User> AppState::getSessionUsers() const {
00095     return sessionUsers;
00096 }
00097
00098 void AppState::clearSessionUsers() {
00099     sessionUsers.clear();
00100 }
00101
00102 // -- Blend creation feedback -----
00103
00104 void AppState::setUsersWithoutData(const std::vector<std::string>& users) {
00105     usersWithoutData = users;
00106 }
00107
00108 std::vector<std::string> AppState::getUsersWithoutData() const {
00109     return usersWithoutData;
00110 }
00111
00112 void AppState::setPendingBlendMessage(const std::string& msg) {
00113     pendingBlendMessage = msg;
00114 }
00115
00116 std::string AppState::takePendingBlendMessage() {
00117     std::string msg = pendingBlendMessage;
00118     pendingBlendMessage.clear();
00119     return msg;
00120 }
00121
00122 void AppState::setPendingUploadError(const std::string& msg) {
00123     pendingUploadError = msg;
00124 }
00125
00126 std::string AppState::takePendingUploadError() {
00127     std::string msg = pendingUploadError;
00128     pendingUploadError.clear();
00129     return msg;
00130 }
00131
00132 // -- Session -----
00133
00134 void AppState::clearSession() {
00135     currentUser = nullptr;
00136     activeBlend = nullptr;
00137     isBlendGenerating = false;
00138     chatRooms.clear();
00139     sessionUsers.clear();
00140     usersWithoutData.clear();
00141     pendingBlendMessage.clear();
00142     pendingUploadError.clear();
00143 }

```

7.55 AppState.hpp File Reference

Session-scoped singleton holding all runtime application state.

```

#include <map>
#include <string>
#include <vector>
#include "../ModelLayer/User.hpp"
#include "../ModelLayer/Blend.hpp"
#include "../ModelLayer/ChatRoom.hpp"
#include "../ServiceLayer/HttpClient.hpp"

```


7.56 AppState.hpp

[Go to the documentation of this file.](#)

```

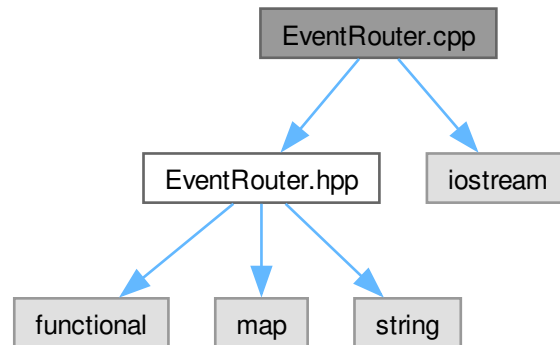
00001 #ifndef APPSTATE_H
00002 #define APPSTATE_H
00003
00016
00017 #include <map>
00018 #include <string>
00019 #include <vector>
00020
00021 #include "../ModelLayer/User.hpp"
00022 #include "../ModelLayer/Blend.hpp"
00023 #include "../ModelLayer/ChatRoom.hpp"
00024 #include "../ServiceLayer/HttpClient.hpp"
00025
00031 class AppState {
00032     private:
00033         User*   currentUser;
00034         Blend*  activeBlend;
00035         bool    isBlendGenerating;
00036
00038         std::map<std::string, ChatRoom> chatRooms;
00039
00041         std::map<std::string, User> sessionUsers;
00042
00044         std::vector<std::string> usersWithoutData;
00045
00050         std::string pendingBlendMessage;
00051
00056         std::string pendingUploadError;
00057
00058         AppState();
00059         AppState(const AppState&) = delete;
00060         AppState& operator=(const AppState&) = delete;
00061
00062     public:
00067         static AppState& getInstance();
00068
00070         User*   getCurrentUser() const;
00072         void    setCurrentUser(User* user);
00073
00075         Blend*  getActiveBlend() const;
00077         void    setActiveBlend(Blend* blend);
00078
00080         bool    isGeneratingBlend() const;
00082         void    setIsBlendGenerating(bool flag);
00083
00088         ChatRoom* getActiveChatRoom();
00089
00095         void    createChatRoom(const Blend& blend);
00096
00102         ChatRoom* getChatRoom(const std::string& blendID);
00103
00108         void    addSessionUser(const User& user);
00109
00111         std::map<std::string, User> getSessionUsers() const;
00112
00114         void    clearSessionUsers();
00115
00121         void    setUsersWithoutData(const std::vector<std::string>& users);
00122
00124         std::vector<std::string> getUsersWithoutData() const;
00125
00131         void    setPendingBlendMessage(const std::string& msg);
00132
00137         std::string takePendingBlendMessage();
00138
00144         void    setPendingUploadError(const std::string& msg);
00145
00150         std::string takePendingUploadError();
00151
00157         void    clearSession();
00158 };
00159
00160 #endif

```

7.57 EventRouter.cpp File Reference

Implementation for [EventRouter](#).

```
#include "EventRouter.hpp"
#include <iostream>
Include dependency graph for EventRouter.cpp:
```



7.57.1 Detailed Description

Implementation for [EventRouter](#).

Author

Jasmine Kumar

Definition in file [EventRouter.cpp](#).

7.58 EventRouter.cpp

[Go to the documentation of this file.](#)

```

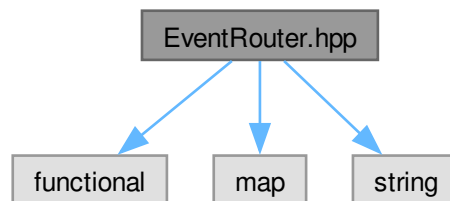
00001
00006
00007 #include "EventRouter.hpp"
00008 #include <iostream>
00009
00010 EventRouter::EventRouter() {}
00011
00012 void EventRouter::registerListener(const std::string& eventName, EventHandler handler) {
00013     listeners[eventName] = std::move(handler);
00014 }
00015
00016 void EventRouter::dispatch(const std::string& eventName, const EventPayload& payload) {
00017     auto it = listeners.find(eventName);
00018     if (it == listeners.end()) {
00019         std::cerr << "[EventRouter] Unknown event: " << eventName << "\n";
00020         return;
00021     }
00022     it->second(payload);
00023 }
00024
00025 void EventRouter::deregisterAll() {
00026     listeners.clear();
00027 }
```

7.59 EventRouter.hpp File Reference

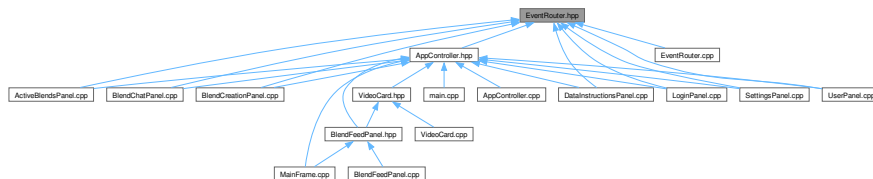
Lightweight event-dispatch system connecting the UI layer to [AppController](#).

```
#include <functional>
#include <map>
#include <string>
```

Include dependency graph for EventRouter.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [EventRouter](#)
[EventRouter](#) class declaration.

Typedefs

- using [EventPayload](#) = `std::map<std::string, std::string>`
- using [EventHandler](#) = `std::function<void(const EventPayload&)>`

7.59.1 Detailed Description

Lightweight event-dispatch system connecting the UI layer to [AppController](#).

Author

Jasmine Kumar

UI panels dispatch named events with a string-to-string payload map. [AppController](#) registers a handler for each event name during construction. Panels never call [AppController](#) directly; [AppController](#) never polls the UI.

Payload conventions by event name:

- "login" ->
`{ "userID": "...", "password": "..." }`
- "logout" -> {}
- "deleteAccount" ->
`{ "userID": "...", "password": "..." }`
- "uploadData" ->
`{ "filePath": "...", "userID": "..." }`
- "createBlend" ->
`{ "userID_0": "...", "userID_1": "..." }`
- "playVideo" ->
`{ "videoID": "..." }`
- "refresh" -> {}
- "openChat" -> {}
- "sendMessage" ->
`{ "userID": "...", "text": "..." }`

Definition in file [EventRouter.hpp](#).

7.59.2 Typedef Documentation

7.59.2.1 EventHandler

```
using EventHandler = std::function<void(const EventPayload&)>
```

Any callable that accepts an [EventPayload](#).

Definition at line 33 of file [EventRouter.hpp](#).

7.59.2.2 EventPayload

```
using EventPayload = std::map<std::string, std::string>
```

String-to-string map passed with every dispatched event.

Definition at line 30 of file [EventRouter.hpp](#).

7.60 EventRouter.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef EVENTROUTER_H
00002 #define EVENTROUTER_H
00003
00004
00005 #include <functional>
00006 #include <map>
00007 #include <string>
00008
00009 using EventPayload = std::map<std::string, std::string>;
00010
00011 using EventHandler = std::function<void(const EventPayload&);>;
00012
00013 class EventRouter {
00014     private:
00015         std::map<std::string, EventHandler> listeners;
00016     public:
00017         EventRouter();
00018
00019         void registerListener(const std::string& eventName, EventHandler handler);
00020
00021         void dispatch(const std::string& eventName, const EventPayload& payload = {});
00022
00023         void deregisterAll();
00024 };
00025 #endif

```

7.61 main.cpp File Reference

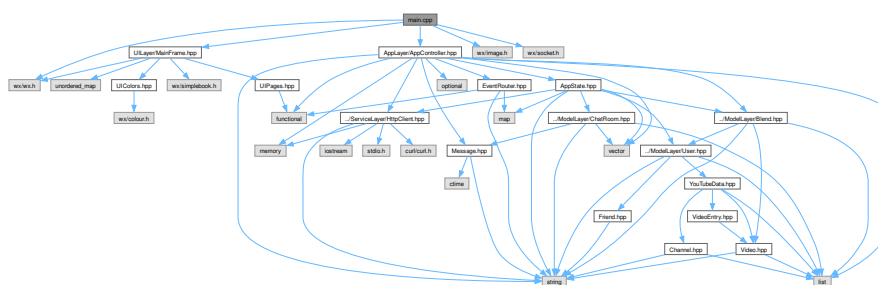
Implementation for main.

```

#include <wx/wx.h>
#include <wx/image.h>
#include <wx/socket.h>
#include "UILayer/MainFrame.hpp"
#include "AppLayer/AppController.hpp"

```

Include dependency graph for main.cpp:



Classes

- class [YTBLNDApp](#)

Functions

- `wxIMPLEMENT_APP` ([YTBLNDApp](#))

7.61.1 Detailed Description

Implementation for main.

Author

Jasmine Kumar

Definition in file [main.cpp](#).

7.62 main.cpp

[Go to the documentation of this file.](#)

```

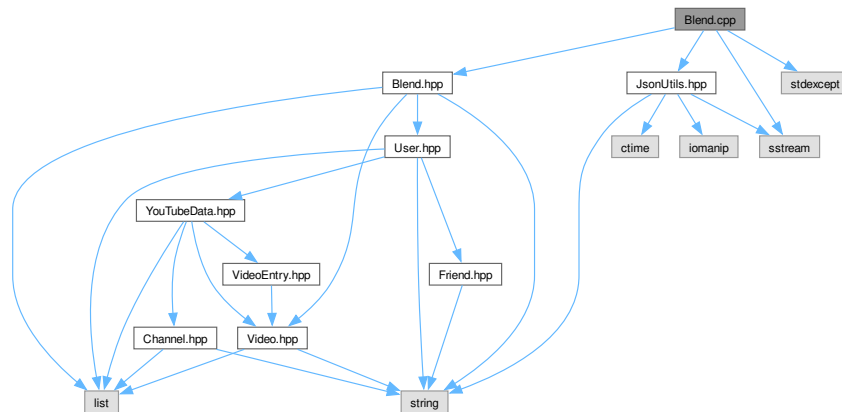
00001
00006
00007 // =====
00008 // main.cpp - Application entry point
00009 //
00010 // Creates the single AppController (which owns AppState, EventRouter, and
00011 // SqliteDataManager), initialises wxWidgets image handlers for thumbnail
00012 // loading, then shows the MainFrame starting at the LOGIN page.
00013 //
00014 // Design note: AppController is a member of YTBNDApp so its lifetime covers
00015 // the entire run of the app. MainFrame receives a reference and forwards it
00016 // to every sub-panel via its constructor.
00017 // =====
00018
00019 #include <wx/wx.h>
00020 #include <wx/image.h>
00021 #include <wx/socket.h>
00022
00023 #include "UILayer/MainFrame.hpp"
00024 #include "AppLayer/AppController.hpp"
00025
00026 class YTBNDApp : public wxApp {
00027 public:
00028     bool OnInit() override;
00029     int OnExit() override;
00030
00031 private:
00032     // AppController owns AppState (singleton), EventRouter, and SqliteDataManager.
00033     // Declared here so its lifetime matches the application.
00034     AppController controller;
00035 };
00036
00037 wxIMPLEMENT_APP(YTBNDApp);
00038
00039 bool YTBNDApp::OnInit() {
00040     // Enable JPEG and PNG handlers for VideoCard thumbnail loading.
00041     // Must be called before any wxImage::LoadFile() / wxURL usage.
00042     wxInitAllImageHandlers();
00043
00044     // Initialise wxSocket on the main thread before any background thread
00045     // calls wxURL::GetInputStream(). wxWidgets requires socket init to
00046     // happen on the main thread; failing to do this causes the
00047     // "wxWidgets sockets must be initialized in the main thread" error.
00048     wxSocketBase::Initialize();
00049
00050     // MainFrame starts maximised and shows the LOGIN page.
00051     auto* frame = new MainFrame(controller);
00052     frame->Show(true);
00053     return true;
00054 }
00055
00056 int YTBNDApp::OnExit() {
00057     wxSocketBase::Shutdown();
00058     return wxApp::OnExit();
00059 }

```

7.63 Blend.cpp File Reference

Implementation for [Blend](#).

```
#include "Blend.hpp"
#include "JsonUtils.hpp"
#include <stdexcept>
#include <sstream>
Include dependency graph for Blend.cpp:
```



7.63.1 Detailed Description

Implementation for [Blend](#).

Author

Jasmine Kumar

Definition in file [Blend.cpp](#).

7.64 Blend.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #include "Blend.hpp"
00008 #include "JsonUtils.hpp"
00009
00010 #include <stdexcept>
00011
00012 #include <sstream>
00013
00014 Blend::Blend(const std::string& blendID,
00015             const std::string& title,
00016             const std::string& algorithmUsed,
00017             const std::list<User>& participants,
00018             const std::list<Video>& videoList)
00019 : blendID(blendID),
00020   title(title),
00021   algorithmUsed(algorithmUsed),
```

```

00022     participants(participants),
00023     videoList(videoList) {}
00024
00025 // Getters
00026
00027 std::string Blend::getBlendID() const {
00028     return blendID;
00029 }
00030
00031 std::string Blend::getTitle() const {
00032     return title;
00033 }
00034
00035 std::string Blend::getAlgorithmUsed() const {
00036     return algorithmUsed;
00037 }
00038
00039 std::list<User> Blend::getParticipants() const {
00040     return participants;
00041 }
00042
00043 std::list<Video> Blend::getVideoList() const {
00044     return videoList;
00045 }
00046
00047 // Setters
00048
00049 void Blend::setVideoList(const std::list<Video>& videoList) {
00050     this->videoList = videoList;
00051 }
00052
00053 void Blend::setBlendID(const std::string& blendID) {
00054     this->blendID = blendID;
00055 }
00056
00057 // Indexed Access
00058
00059 Video Blend::getVideo(int index) const {
00060     if (index < 0 || index >= static_cast<int>(videoList.size())) {
00061         throw std::out_of_range("Blend::getVideo - index " + std::to_string(index) +
00062             " is out of range (size = " + std::to_string(videoList.size()) + ")");
00063     }
00064     auto it = videoList.begin();
00065     std::advance(it, index);
00066     return *it;
00067 }
00068
00069 int Blend::size() const {
00070     return static_cast<int>(videoList.size());
00071 }
00072
00073 std::string Blend::toString() const {
00074     std::ostringstream participantsJson;
00075     participantsJson << "[";
00076     bool firstParticipant = true;
00077     for (const auto& participant : participants) {
00078         if (!firstParticipant) {
00079             participantsJson << ",";
00080         }
00081         participantsJson << ModelJson::quote(participant.getUserID());
00082         firstParticipant = false;
00083     }
00084     participantsJson << "]";
00085
00086     std::ostringstream videosJson;
00087     videosJson << "[";
00088     bool firstVideo = true;
00089     for (const auto& video : videoList) {
00090         if (!firstVideo) {
00091             videosJson << ",";
00092         }
00093         videosJson << video.toString();
00094         firstVideo = false;
00095     }
00096     videosJson << "]";
00097
00098     return "{"
00099         "\"blend_id\":\"" + ModelJson::quote(blendID) + ","
00100         "\"title\":\"" + ModelJson::quote(title) + ","
00101         "\"chat_room_id\":\"" + ModelJson::quote(blendID) + ","
00102         "\"algorithm\":\"" + ModelJson::quote(algorithmUsed) + ","
00103         "\"participants\":\"" + participantsJson.str() + ","
00104         "\"videos\":\"" + videosJson.str()
00105         + "}";
00106 }

```


7.66 Blend.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef BLEND_H
00002 #define BLEND_H
00003
00013
00014 #include <string>
00015 #include <list>
00016
00017 #include "User.hpp"
00018 #include "Video.hpp"
00019
00024 class Blend {
00025     private:
00026         std::string blendID;           // indentifying blend method
00027         std::string title;             // title of the blend
00028         std::string algorithmUsed;     // algorithm to be used for parsing through the blend
00029         std::list<User> participants;  // users included in the blend
00030         std::list<Video> videoList;    // the video list included within the blend
00031
00032     public:
00041         Blend(const std::string& blendID,
00042               const std::string& title,
00043               const std::string& algorithmUsed,
00044               const std::list<User>& participants,
00045               const std::list<Video>& videoList);
00046
00048         std::string      getBlendID()      const;
00050         std::string      getTitle()         const;
00052         std::string      getAlgorithmUsed() const;
00054         std::list<User>  getParticipants()  const;
00056         std::list<Video> getVideoList()     const;
00057
00062         void setVideoList(const std::list<Video>& videoList);
00063
00069         Video getVideo(int index) const;
00070
00072         int size() const;
00073
00078         void setBlendID(const std::string& blendID);
00079
00081         std::string toString() const;
00082 };
00083
00084 #endif

```

7.67 Channel.cpp File Reference

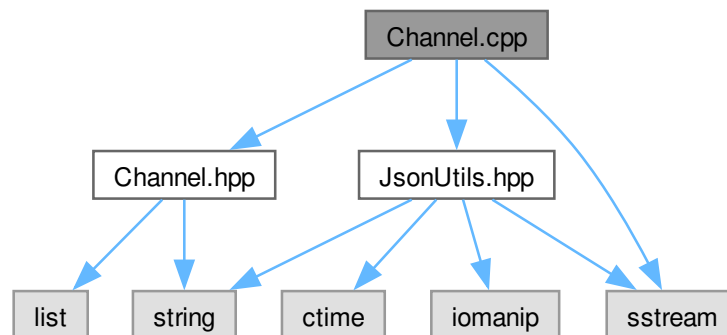
Implementation for [Channel](#).

```

#include "Channel.hpp"
#include "JsonUtils.hpp"
#include <sstream>

```

Include dependency graph for Channel.cpp:



7.67.1 Detailed Description

Implementation for [Channel](#).

Author

Jasmine Kumar

Definition in file [Channel.cpp](#).

7.68 Channel.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "Channel.hpp"
00008 #include "JsonUtils.hpp"
00009
00010 #include <sstream>
00011
00012 Channel::Channel(const std::string& channelId,
00013                 const std::string& displayName,
00014                 const std::list<std::string>& categories)
00015     : channelId(channelId), displayName(displayName), categories(categories) {}
00016
00017 // Getters
00018
00019 std::string Channel::getChannelID() const {
00020     return channelId;
00021 }
00022
00023 std::string Channel::getDisplayName() const {
00024     return displayName;
00025 }
00026
00027 std::list<std::string> Channel::getCategories() const {
00028     return categories;
00029 }
00030
00031 std::string Channel::toString() const {
00032     std::ostringstream categoriesJson;

```



```
00033     categoriesJson << "[";
00034     bool first = true;
00035     for (const auto& category : categories) {
00036         if (!first) {
00037             categoriesJson << ",";
00038         }
00039         categoriesJson << ModelJson::quote(category);
00040         first = false;
00041     }
00042     categoriesJson << "];";
00043
00044     return "{"
00045         "\"channel_id\":\"" + ModelJson::quote(channelID) + ","
00046         "\"display_name\":\"" + ModelJson::quote(displayName) + ","
00047         "\"categories\":\"" + categoriesJson.str()
00048         + "\"";
00049 }
```

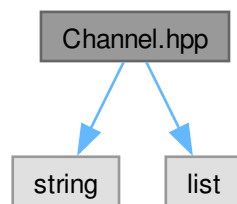
7.69 Channel.hpp File Reference

Model representing a YouTube channel.

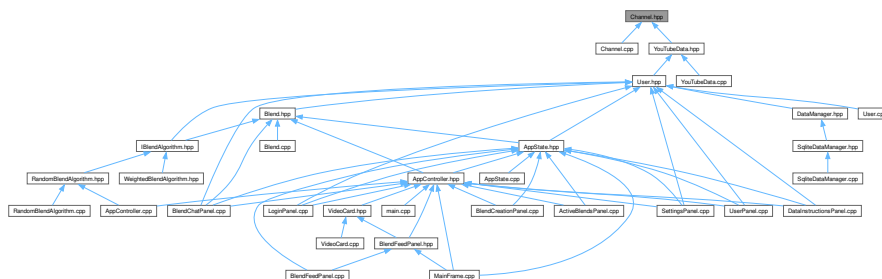
```
#include <string>
```

```
#include <list>
```

Include dependency graph for Channel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class Channel
Channel class declaration.

7.69.1 Detailed Description

Model representing a YouTube channel.

Author

Jasmine Kumar

Definition in file [Channel.hpp](#).

7.70 Channel.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef CHANNEL_H
00002 #define CHANNEL_H
00003
00009
00010 #include <string>
00011 #include <list>
00012
00017 class Channel {
00018     private:
00019         std::string channelID;           // identifying channel method
00020         std::string displayName;         // the channel front facing name (as seen on YouTube)
00021         std::list<std::string> categories; // categories/themes of the channel
00022
00023     public:
00030         Channel(const std::string& channelID,
00031                const std::string& displayName,
00032                const std::list<std::string>& categories);
00033
00035         std::string getChannelID() const;
00037         std::string getDisplayName() const;
00039         std::list<std::string> getCategories() const;
00040
00041         std::string toString() const;
00042 };
00043
00044 #endif

```

7.71 ChatRoom.cpp File Reference

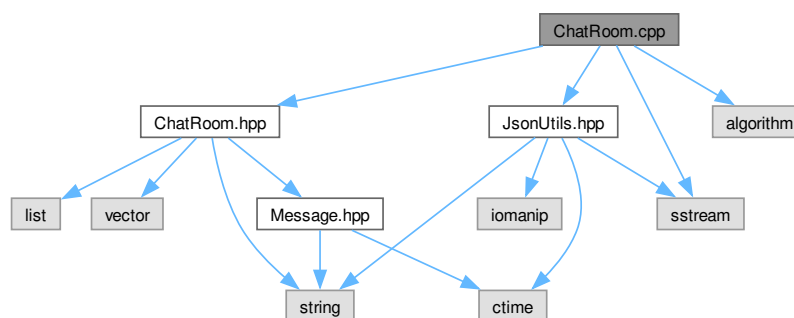
Implementation for [ChatRoom](#).

```

#include "ChatRoom.hpp"
#include "JsonUtils.hpp"
#include <algorithm>
#include <sstream>

```

Include dependency graph for ChatRoom.cpp:



7.71.1 Detailed Description

Implementation for [ChatRoom](#).

Author

Jasmine Kumar

Definition in file [ChatRoom.cpp](#).

7.72 ChatRoom.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "ChatRoom.hpp"
00008 #include "JsonUtils.hpp"
00009
00010 #include <algorithm>
00011 #include <sstream>
00012
00013 ChatRoom::ChatRoom(const std::string& blendID,
00014                   const std::vector<std::string>& participantIDs)
00015     : blendID(blendID), participantIDs(participantIDs) {}
00016
00017 // -- Getters -----
00018
00019 std::string ChatRoom::getBlendID() const {
00020     return blendID;
00021 }
00022
00023 std::vector<std::string> ChatRoom::getParticipantIDs() const {
00024     return participantIDs;
00025 }
00026
00027 std::list<Message> ChatRoom::getMessages() const {
00028     return messages;
00029 }
00030
00031 // -- Messaging -----
00032
00033 void ChatRoom::addMessage(const std::string& userID, const std::string& text) {
00034     messages.emplace_back(userID, text);
00035 }
00036
00037 void ChatRoom::addMessage(const Message& msg) {
00038     messages.push_back(msg);
00039 }
00040
00041 bool ChatRoom::isParticipant(const std::string& userID) const {
00042     return std::find(participantIDs.begin(), participantIDs.end(), userID)
00043         != participantIDs.end();
00044 }
00045
00046 void ChatRoom::clearMessages() {
00047     messages.clear();
00048 }
00049
00050 std::string ChatRoom::toString() const {
00051     std::ostringstream membersJson;
00052     membersJson << "[";
00053     bool firstMember = true;
00054     for (const auto& participantID : participantIDs) {
00055         if (!firstMember) {
00056             membersJson << ",";
00057         }
00058         membersJson << ModelJson::quote(participantID);
00059         firstMember = false;
00060     }
00061     membersJson << "]";
00062
00063     std::ostringstream messagesJson;
00064     messagesJson << "[";
00065     bool firstMessage = true;
00066     for (const auto& message : messages) {

```

```

00067         if (!firstMessage) {
00068             messagesJson < " , ";
00069         }
00070         messagesJson < " { "
00071             < " \"type\": \"chat_message\", "
00072             < " \"room_id\": \" < ModelJson::quote(blendID) < " , "
00073             < " \"sender_id\": \" < ModelJson::quote(message.userID) < " , "
00074             < " \"content\": \" < ModelJson::quote(message.text) < " , "
00075             < " \"sent_at\": \" < ModelJson::quote(ModelJson::toIso8601(message.timestamp))
00076             < " } ";
00077         firstMessage = false;
00078     }
00079     messagesJson < " ] ";
00080
00081     return " { "
00082         " \"blend_id\": \" < ModelJson::quote(blendID) < " , "
00083         " \"chat_room_id\": \" < ModelJson::quote(blendID) < " , "
00084         " \"member_users\": \" < membersJson.str() < " , "
00085         " \"messages\": \" < messagesJson.str()
00086         + " } ";
00087 }

```

7.73 ChatRoom.hpp File Reference

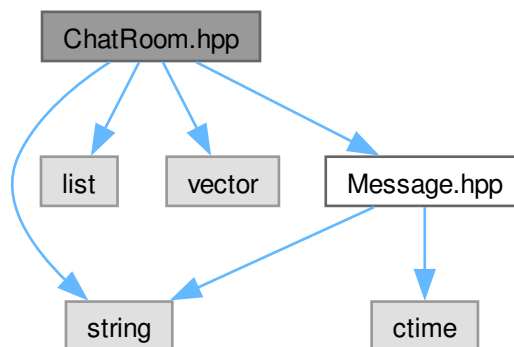
In-memory chat session tied to a specific [Blend](#).

```

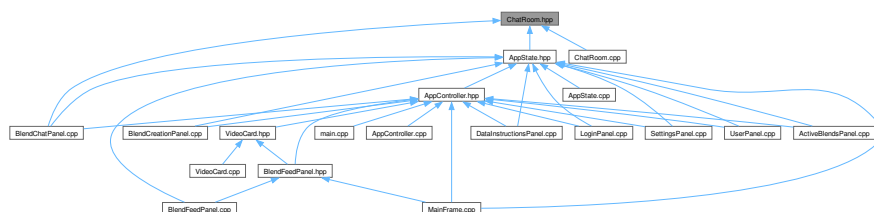
#include <string>
#include <list>
#include <vector>
#include "Message.hpp"

```

Include dependency graph for ChatRoom.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [ChatRoom](#)
[ChatRoom](#) class declaration.

7.73.1 Detailed Description

In-memory chat session tied to a specific [Blend](#).

Author

Jasmine Kumar

Identified by the blendID of the [Blend](#) it belongs to. Messages are not persisted in this version - all chat history is lost when the session ends.

Currently only one [ChatRoom](#) is active at a time (the one matching the active [Blend](#) in [AppState](#)), but [AppState](#) holds a map<string, ChatRoom> keyed by blendID, so supporting multiple simultaneous rooms requires no changes here.

Definition in file [ChatRoom.hpp](#).

7.74 ChatRoom.hpp

[Go to the documentation of this file.](#)

```

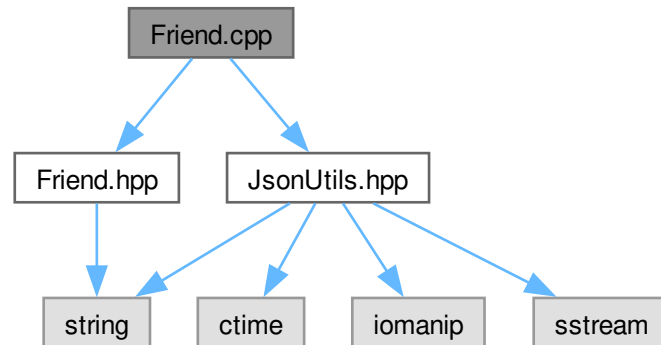
00001 #ifndef CHATROOM_H
00002 #define CHATROOM_H
00003
00016
00017 #include <string>
00018 #include <list>
00019 #include <vector>
00020
00021 #include "Message.hpp"
00022
00028 class ChatRoom {
00029     private:
00030         std::string blendID;
00031         std::vector<std::string> participantIDs;
00032         std::list<Message> messages;
00033
00034     public:
00040         ChatRoom(const std::string& blendID,
00041                 const std::vector<std::string>& participantIDs);
00042
00044         std::string getBlendID() const;
00046         std::vector<std::string> getParticipantIDs() const;
00048         std::list<Message> getMessages() const;
00049
00055         void addMessage(const std::string& userID, const std::string& text);
00056
00062         void addMessage(const Message& msg);
00063
00070         bool isParticipant(const std::string& userID) const;
00071
00076         void clearMessages();
00077
00078         std::string toString() const;
00079 };
00080
00081 #endif

```

7.75 Friend.cpp File Reference

Implementation for [Friend](#).

```
#include "Friend.hpp"
#include "JsonUtils.hpp"
Include dependency graph for Friend.cpp:
```



7.75.1 Detailed Description

Implementation for [Friend](#).

Author

Jasmine Kumar

Definition in file [Friend.cpp](#).

7.76 Friend.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "Friend.hpp"
00008 #include "JsonUtils.hpp"
00009
00010 //
00011 Friend::Friend(const std::string& userID,
00012               const std::string& displayName,
00013               const std::string& email)
00014     : userID(userID), displayName(displayName), email(email) {}
00015
00016 // Getters
00017
00018 std::string Friend::getUserID() const {
00019     return userID;
00020 }
00021
```

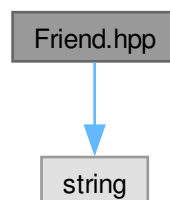
```
00022 std::string Friend::getDisplayName() const {
00023     return displayName;
00024 }
00025
00026 std::string Friend::getEmail() const {
00027     return email;
00028 }
00029
00030 //Setters (userID is intentionally omitted - it is immutable after construction)
00031
00032 void Friend::setDisplayName(const std::string& displayName) {
00033     this->displayName = displayName;
00034 }
00035
00036 void Friend::setEmail(const std::string& email) {
00037     this->email = email;
00038 }
00039
00040 std::string Friend::toString() const {
00041     return "{"
00042         "\"user_id\": " + ModelJson::quote(userID) + ", "
00043         "\"username\": " + ModelJson::quote(displayName) + ", "
00044         "\"email\": " + ModelJson::quote(email)
00045         + "}";
00046 }
```

7.77 Friend.hpp File Reference

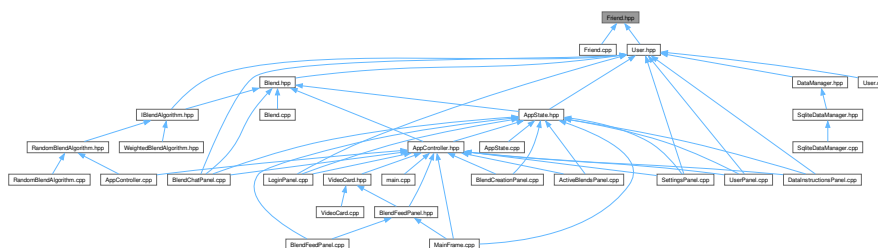
Model representing a friend/contact entry on a [User's](#) list.

```
#include <string>
```

Include dependency graph for Friend.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [Friend](#)

[Friend](#) class declaration.

7.77.1 Detailed Description

Model representing a friend/contact entry on a [User](#)'s list.

Author

Jasmine Kumar

Definition in file [Friend.hpp](#).

7.78 Friend.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef FRIEND_H
00002 #define FRIEND_H
00003
00009
00010 #include <string>
00011
00017 class Friend {
00018     private:
00019         std::string userID;
00020         std::string displayName;
00021         std::string email;
00022
00023     public:
00030         Friend(const std::string& userID,
00031               const std::string& displayName,
00032               const std::string& email);
00033
00035         std::string getUserID() const;
00037         std::string getDisplayName() const;
00039         std::string getEmail() const;
00040
00042         void setDisplayName(const std::string& displayName);
00044         void setEmail(const std::string& email);
00045
00050         std::string toString() const;
00051 };
00052
00053 #endif
```

7.79 JsonUtils.hpp File Reference

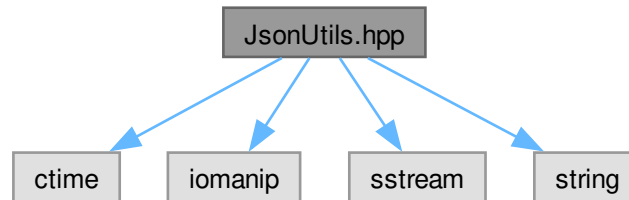
Utility functions for JSON string formatting and serialization.

```
#include <ctime>
#include <iomanip>
#include <sstream>
```

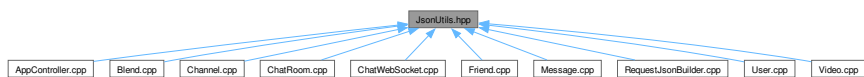


```
#include <string>
```

Include dependency graph for JsonUtils.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [ModelJson](#)
JSON formatting utilities for model serialization.

Functions

- `std::string` [ModelJson::escape](#) (const std::string &input)
- `std::string` [ModelJson::quote](#) (const std::string &input)
- `std::string` [ModelJson::toISOString8601](#) (std::time_t timestamp)

7.79.1 Detailed Description

Utility functions for JSON string formatting and serialization.

Author

Shamar Pennant

Provides helper functions for escaping special characters in JSON strings, quoting values, and converting timestamps to ISO8601 format.

Definition in file [JsonUtils.hpp](#).

7.80 JsonUtils.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef MODEL_JSON_UTILS_H
00011 #define MODEL_JSON_UTILS_H
00012
00013 #include <ctime>
00014 #include <iomanip>
00015 #include <sstream>
00016 #include <string>
00017
00022 namespace ModelJson {
00023
00024
00035 inline std::string escape(const std::string& input) {
00036     std::ostringstream out;
00037     for (unsigned char c : input) {
00038         switch (c) {
00039             case '"': out << "\\\""; break;
00040             case '\\': out << "\\\""; break;
00041             case 'b': out << "\\b"; break;
00042             case 'f': out << "\\f"; break;
00043             case 'n': out << "\\n"; break;
00044             case 'r': out << "\\r"; break;
00045             case 't': out << "\\t"; break;
00046             default:
00047                 if (c < 0x20) {
00048                     out << "\\u"
00049                         << std::hex << std::uppercase << std::setw(4)
00050                         << std::setfill('0') << static_cast<int>(c)
00051                         << std::dec;
00052                 } else {
00053                     out << static_cast<char>(c);
00054                 }
00055             }
00056         }
00057     }
00058     return out.str();
00059 }
00069 inline std::string quote(const std::string& input) {
00070     return std::string("\"") + escape(input) + "\"";
00071 }
00072
00082 inline std::string toIso8601(std::time_t timestamp) {
00083     std::tm utc{};
00084     #if defined(_WIN32)
00085         gmtime_s(&utc, &timestamp);
00086     #else
00087         gmtime_r(&timestamp, &utc);
00088     #endif
00089
00090     std::ostringstream out;
00091     out << std::put_time(&utc, "%Y-%m-%dT%H:%M:%SZ");
00092     return out.str();
00093 }
00094
00095 } // namespace ModelJson
00096
00097 #endif

```

7.81 Message.cpp File Reference

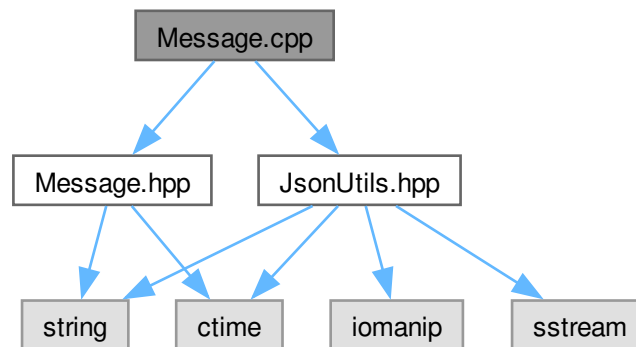
Implementation for [Message](#).

```

#include "Message.hpp"
#include "JsonUtils.hpp"

```

Include dependency graph for Message.cpp:



7.81.1 Detailed Description

Implementation for [Message](#).

Author

Jasmine Kumar

Definition in file [Message.cpp](#).

7.82 Message.cpp

[Go to the documentation of this file.](#)

```

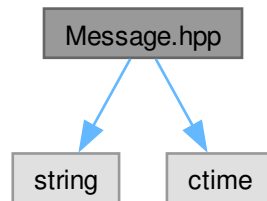
00001
00006
00007 #include "Message.hpp"
00008 #include "JsonUtils.hpp"
00009
00010 std::string Message::toString() const {
00011     return "{"
00012         "\"sender_id\": " + ModelJson::quote(userID) + ", "
00013         "\"content\": " + ModelJson::quote(text) + ", "
00014         "\"sent_at\": " + ModelJson::quote(ModelJson::toIso8601(timestamp))
00015         + "}";
00016 }

```

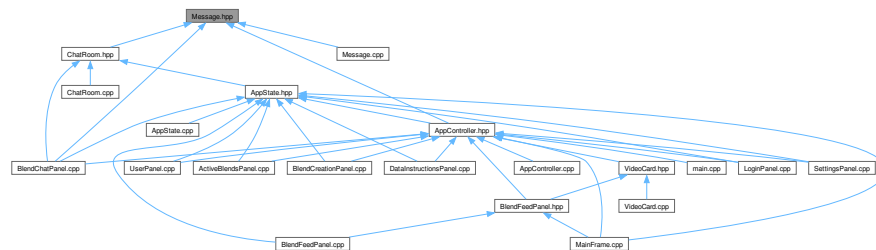
7.83 Message.hpp File Reference

A single chat message sent within a [ChatRoom](#).

```
#include <string>
#include <ctime>
Include dependency graph for Message.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- struct [Message](#)
A single chat message sent within a [ChatRoom](#).

7.83.1 Detailed Description

A single chat message sent within a [ChatRoom](#).

Author

Jasmine Kumar

Kept as a plain struct since it is pure data with no behaviour.

Definition in file [Message.hpp](#).

7.84 Message.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef MESSAGE_H
00002 #define MESSAGE_H
00003
00011
00012 #include <string>
00013 #include <ctime>
00014
00022 struct Message {
00023     std::string userID;
00024     std::string text;
00025     std::time_t timestamp;
00026
00033     Message(const std::string& userID,
00034             const std::string& text,
00035             std::time_t timestamp = std::time(nullptr))
00036         : userID(userID), text(text), timestamp(timestamp) {}
00037
00042     std::string toString() const;
00043 };
00044
00045 #endif

```

7.85 User.cpp File Reference

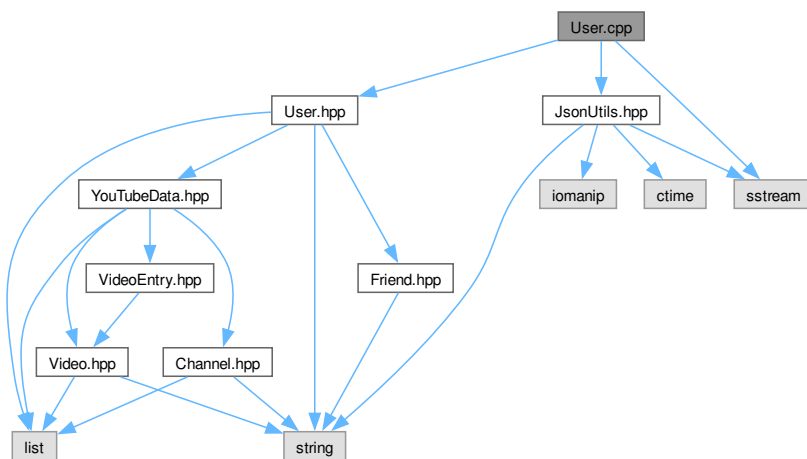
Implementation for [User](#).

```

#include "User.hpp"
#include "JsonUtils.hpp"
#include <sstream>

```

Include dependency graph for User.cpp:



7.85.1 Detailed Description

Implementation for [User](#).

Author

Jasmine Kumar

Definition in file [User.cpp](#).

7.86 User.cpp

[Go to the documentation of this file.](#)

```

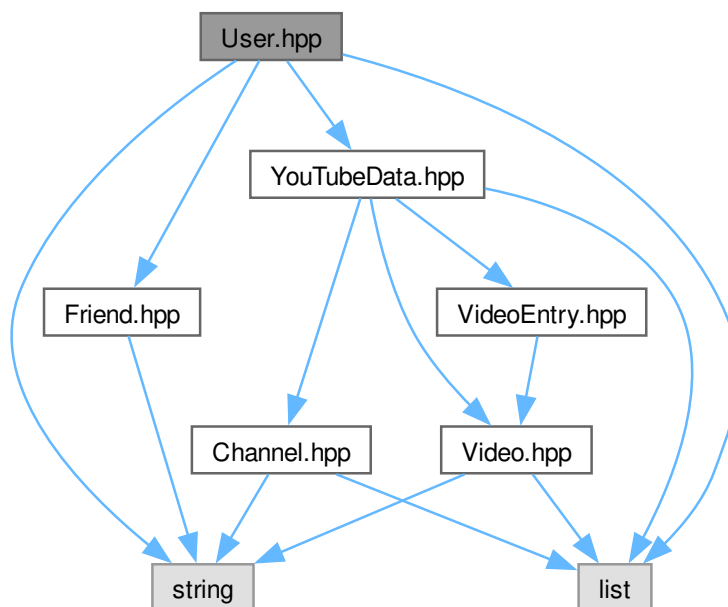
00001
00006
00007 #include "User.hpp"
00008
00009 #include "JsonUtils.hpp"
00010
00011 #include <sstream>
00012
00013 User::User(const std::string& userID,
00014            const std::string& username,
00015            const std::string& email,
00016            const std::string& password)
00017     : userID(userID), username(username), email(email), password(password) {}
00018
00019 // Getters
00020
00021 std::string User::getUserID() const { return userID; }
00022 std::string User::getUsername() const { return username; }
00023 std::string User::getEmail() const { return email; }
00024 std::string User::getPassword() const { return password; }
00025
00026 std::list<Friend> User::getFriends() const { return friends; }
00027
00028 YouTubeData& User::getYouTubeData() { return youtubeData; }
00029 const YouTubeData& User::getYouTubeData() const { return youtubeData; }
00030
00031 // Setters
00032
00033 void User::setUsername(const std::string& username) { this->username = username; }
00034 void User::setEmail(const std::string& email) { this->email = email; }
00035 void User::setPassword(const std::string& password) { this->password = password; }
00036
00037 void User::setYouTubeData(const YouTubeData& youtubeData) {
00038     this->youtubeData = youtubeData;
00039 }
00040
00041 // Friend management
00042
00043 void User::addFriend(const User& user) {
00044     for (const auto& f : friends) {
00045         if (f.getUserID() == user.getUserID()) return; // no duplicates
00046     }
00047     friends.emplace_back(user.getUserID(), user.getUsername(), user.getEmail());
00048 }
00049
00050 void User::removeFriend(const std::string& userID) {
00051     friends.remove_if([&userID](const Friend& f) {
00052         return f.getUserID() == userID;
00053     });
00054 }
00055
00056 std::string User::toString() const {
00057     std::ostringstream friendsJson;
00058     friendsJson << "[";
00059     bool firstFriend = true;
00060     for (const auto& friendUser : friends) {
00061         if (!firstFriend) {
00062             friendsJson << ",";
00063         }
00064         friendsJson << friendUser.toString();
00065         firstFriend = false;
00066     }
00067     friendsJson << "];";
00068
00069     return "{
00070         \"user_id\": \" + ModelJson::quote(userID) + \",
00071         \"username\": \" + ModelJson::quote(username) + \",
00072         \"email\": \" + ModelJson::quote(email) + \",
00073         \"friends\": \" + friendsJson.str() + \",
00074         \"youtube_data\": \" + youtubeData.toString()
00075         + \"}";
00076 }

```

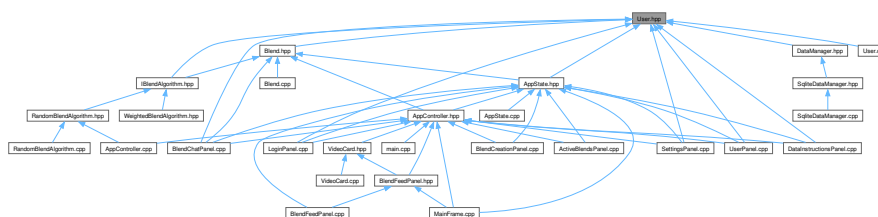
7.87 User.hpp File Reference

Model representing a registered application user.

```
#include <string>
#include <list>
#include "Friend.hpp"
#include "YouTubeData.hpp"
Include dependency graph for User.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class `User`
`User` class declaration.

7.87.1 Detailed Description

Model representing a registered application user.

Author

Jasmine Kumar

Holds identity fields and the user's associated YouTube data. Passwords are stored as plaintext at this stage of development.

Definition in file [User.hpp](#).

7.88 User.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef USER_H
00002 #define USER_H
00003
00012
00013 #include <string>
00014 #include <list>
00015
00016 #include "Friend.hpp"
00017 #include "YouTubeData.hpp"
00018
00024 class User {
00025     private:
00026         std::string userID; // generated id
00027         std::string username; // provided username
00028         std::string email; // provided user email
00029         std::string password;
00030         std::list<Friend> friends; // list of friends -- to be fully implemented as a
        backend/client/UI feature in the future
        YouTubeData youTubeData; // provided YouTube Data on file upload
00031
00032     public:
00033         User(const std::string& userID,
00041             const std::string& username,
00042             const std::string& email,
00043             const std::string& password);
00044
00045         std::string      getUserID() const;
00047         std::string      getUsername() const;
00049         std::string      getEmail() const;
00051         std::string      getPassword() const;
00053         std::list<Friend> getFriends() const;
00055         YouTubeData&      getYouTubeData();
00057         const YouTubeData& getYouTubeData() const;
00059
00060         void setUsername(const std::string& username);
00062         void setEmail(const std::string& email);
00064         void setPassword(const std::string& password);
00066         void setYouTubeData(const YouTubeData& youTubeData);
00068
00069         void addFriend(const User& user);
00074
00075         void removeFriend(const std::string& userID);
00080
00081         std::string toString() const;
00086
00087 };
00088
00089 #endif

```

7.89 Video.cpp File Reference

Implementation for [Video](#).

```

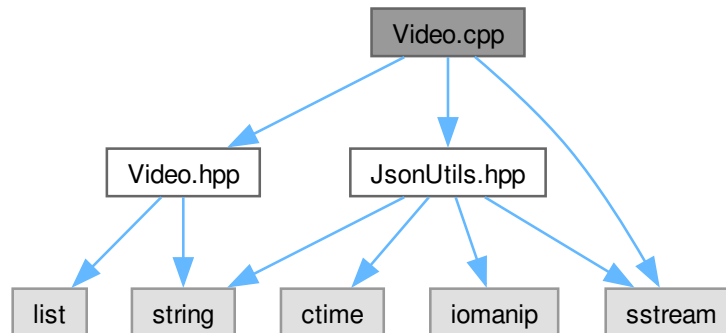
#include "Video.hpp"
#include "JsonUtils.hpp"

```



```
#include <sstream>
```

Include dependency graph for Video.cpp:



7.89.1 Detailed Description

Implementation for [Video](#).

Author

Jasmine Kumar

Definition in file [Video.cpp](#).

7.90 Video.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "Video.hpp"
00008 #include "JsonUtils.hpp"
00009
00010 #include <sstream>
00011
00012 Video::Video(const std::string& videoID,
00013             const std::string& title,
00014             const std::string& channelID,
00015             const std::string& thumbnailURL,
00016             int duration,
00017             const std::list<std::string>& tags,
00018             const std::string& channelName,
00019             const std::string& channelLogoURL)
00020 : videoID(videoID),
00021   title(title),
00022   channelID(channelID),
00023   thumbnailURL(thumbnailURL),
00024   duration(duration),
00025   tags(tags),
00026   channelName(channelName),
00027   channelLogoURL(channelLogoURL) {}
00028
00029 std::string Video::getVideoID() const {
00030     return videoID;
00031 }
  
```

```

00032
00033 std::string Video::getTitle() const {
00034     return title;
00035 }
00036
00037 std::string Video::getChannelID() const {
00038     return channelID;
00039 }
00040
00041 std::string Video::getThumbnailURL() const {
00042     return thumbnailURL;
00043 }
00044
00045 int Video::getDuration() const {
00046     return duration;
00047 }
00048
00049 std::list<std::string> Video::getTags() const {
00050     return tags;
00051 }
00052
00053 std::string Video::getChannelName() const {
00054     return channelName;
00055 }
00056
00057 std::string Video::getChannelLogoURL() const {
00058     return channelLogoURL;
00059 }
00060
00061 std::string Video::toString() const {
00062     std::ostringstream tagsJson;
00063     tagsJson << "[";
00064     bool first = true;
00065     for (const auto& tag : tags) {
00066         if (!first) {
00067             tagsJson << ",";
00068         }
00069         tagsJson << ModelJson::quote(tag);
00070         first = false;
00071     }
00072     tagsJson << "]";
00073
00074     return "{"
00075         "\"video_id\": " + ModelJson::quote(videoID) + ","
00076         "\"title\": " + ModelJson::quote(title) + ","
00077         "\"channel_id\": " + ModelJson::quote(channelID) + ","
00078         "\"thumbnail_url\": " + ModelJson::quote(thumbnailURL) + ","
00079         "\"duration\": " + std::to_string(duration) + ","
00080         "\"tags\": " + tagsJson.str()
00081         + "}";
00082 }

```

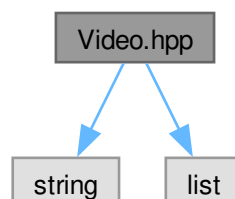
7.91 Video.hpp File Reference

Immutable data model for a single YouTube video.

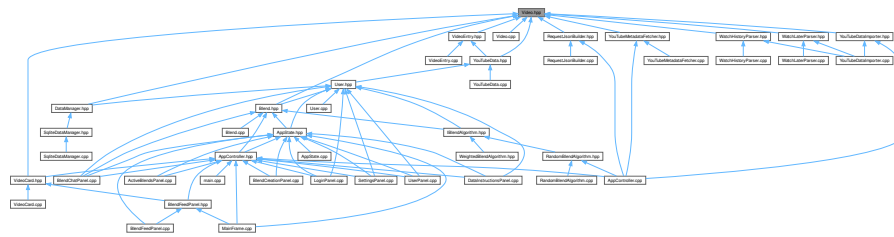
```
#include <string>
```

```
#include <list>
```

Include dependency graph for Video.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class **Video**
Video class declaration.

7.91.1 Detailed Description

Immutable data model for a single YouTube video.

Author

Jasmine Kumar

Populated initially with only a videoID from the Watch Later CSV; the remaining fields are filled in by [YouTubeMetadataFetcher](#) after an API call.

Definition in file [Video.hpp](#).

7.92 Video.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef VIDEO_H
00002 #define VIDEO_H
00003
00012
00013 #include <string>
00014 #include <list>
00015
00021 class Video {
00022     private:
00023         std::string videoID;           // YouTube video id
00024         std::string title;             // YouTube video title
00025         std::string channelID;         // YouTube channel ID
00026         std::string thumbnailURL;      // URL to video thumbnail
00027         int duration;                 // length of video
00028         std::list<std::string> tags;    // tags associated with video themes/categories -- to be
implemented later on
00029         std::string channelName;       // YouTube channel name
00030         std::string channelLogoURL;    // YouTube channel icon/logo
00031
00032     public:
00044         Video(const std::string& videoID,
00045             const std::string& title,
00046             const std::string& channelID,
00047             const std::string& thumbnailURL,
00048             int duration,
00049             const std::list<std::string>& tags,
00050             const std::string& channelName = "",

```

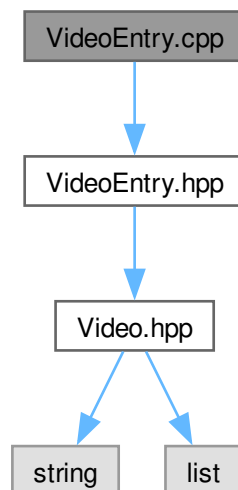
```
00051         const std::string& channelLogoURL = "");
00052
00053     std::string getVideoID() const;
00054     std::string getTitle() const;
00055     std::string getChannelID() const;
00056     std::string getThumbnailURL() const;
00057     int getDuration() const;
00058     std::list<std::string> getTags() const;
00059     std::string getChannelName() const;
00060     std::string getChannelLogoURL() const;
00061
00062     std::string toString() const;
00063 };
00064 #endif
```

7.93 VideoEntry.cpp File Reference

Implementation for [VideoEntry](#).

```
#include "VideoEntry.hpp"
```

Include dependency graph for VideoEntry.cpp:



7.93.1 Detailed Description

Implementation for [VideoEntry](#).

Author

Jasmine Kumar

Definition in file [VideoEntry.cpp](#).

7.94 VideoEntry.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "VideoEntry.hpp"
00008
00009 VideoEntry::VideoEntry(const Video& video, int watchCount, int lastWatched)
00010     : video(video), watchCount(watchCount), lastWatched(lastWatched) {}
00011
00012 // Getters
00013
00014 Video VideoEntry::getVideo() const {
00015     return video;
00016 }
00017
00018 int VideoEntry::getWatchCount() const {
00019     return watchCount;
00020 }
00021
00022 int VideoEntry::getLastWatched() const {
00023     return lastWatched;
00024 }
00025
00026 // Setters
00027 void VideoEntry::setWatchCount(int watchCount) {
00028     this->watchCount = watchCount;
00029 }
00030
00031 void VideoEntry::setLastWatched(int lastWatched) {
00032     this->lastWatched = lastWatched;
00033 }
00034
00035 std::string VideoEntry::toString() const {
00036     return "{" +
00037         "\"video\": " + video.toString() + ", "
00038         "\"watch_count\": " + std::to_string(watchCount) + ", "
00039         "\"last_watched\": " + std::to_string(lastWatched)
00040         + "}";
00041 }

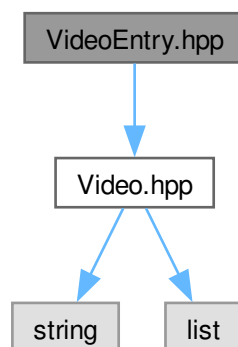
```

7.95 VideoEntry.hpp File Reference

A [Video](#) decorated with watch-history metadata.

```
#include "Video.hpp"
```

Include dependency graph for VideoEntry.hpp:

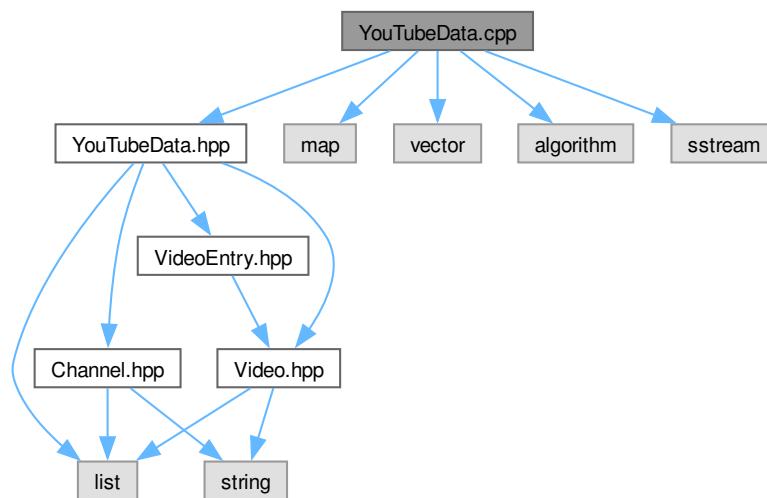


7.97 YouTubeData.cpp File Reference

Implementation for [YouTubeData](#).

```
#include "YouTubeData.hpp"
#include <map>
#include <vector>
#include <algorithm>
#include <sstream>
```

Include dependency graph for YouTubeData.cpp:



7.97.1 Detailed Description

Implementation for [YouTubeData](#).

Author

Jasmine Kumar

Definition in file [YouTubeData.cpp](#).

7.98 YouTubeData.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #include "YouTubeData.hpp"
00008 #include <map>
00009 #include <vector>
00010 #include <algorithm>
00011 #include <sstream>
00012
```

```

00013 YouTubeData::YouTubeData() {}
00014
00015 // Getters
00016 std::list<VideoEntry> YouTubeData::getWatchHistory() const {
00017     return watchHistory;
00018 }
00019
00020 std::list<Video> YouTubeData::getLikedVideos() const {
00021     return likedVideos;
00022 }
00023
00024 std::list<Channel> YouTubeData::getSubscribedChannels() const {
00025     return subscribedChannels;
00026 }
00027
00028 //Setters
00029
00030 void YouTubeData::setWatchHistory(const std::list<VideoEntry>& watchHistory) {
00031     this->watchHistory = watchHistory;
00032 }
00033
00034 void YouTubeData::setLikedVideos(const std::list<Video>& likedVideos) {
00035     this->likedVideos = likedVideos;
00036 }
00037
00038 void YouTubeData::setSubscribedChannels(const std::list<Channel>& subscribedChannels) {
00039     this->subscribedChannels = subscribedChannels;
00040 }
00041
00042 std::list<Video> YouTubeData::getWatchLaterVideos() const {
00043     return watchLaterVideos;
00044 }
00045
00046 void YouTubeData::setWatchLaterVideos(const std::list<Video>& videos) {
00047     this->watchLaterVideos = videos;
00048 }
00049
00050 // Computed Queries
00051
00052 std::list<Video> YouTubeData::getMostWatchedVideos(int n) const {
00053     // Copy watch history into a vector for sorting
00054     std::vector<VideoEntry> entries(watchHistory.begin(), watchHistory.end());
00055
00056     // Sort copied vector
00057     std::sort(entries.begin(), entries.end(),
00058         [](const VideoEntry& a, const VideoEntry& b) {
00059             return a.getWatchCount() > b.getWatchCount();
00060         });
00061     // Add sorted results to a list
00062     std::list<Video> result;
00063     int count = 0;
00064     for (const auto& entry : entries) {
00065         if (count >= n) break;
00066         result.push_back(entry.getVideo());
00067         count++;
00068     }
00069     return result;
00070 }
00071
00072 std::list<Channel> YouTubeData::getTopChannels(int n) const {
00073     // Count how many watched videos belong to each channelId
00074     std::map<std::string, int> channelWatchCounts;
00075     for (const auto& entry : watchHistory) {
00076         const std::string& cid = entry.getVideo().getChannelID();
00077         channelWatchCounts[cid] += entry.getWatchCount();
00078     }
00079
00080     // Sort subscribedChannels by their accumulated watch count
00081     std::vector<Channel> channels(subscribedChannels.begin(), subscribedChannels.end());
00082     std::sort(channels.begin(), channels.end(),
00083         [&channelWatchCounts](const Channel& a, const Channel& b) {
00084             int countA = channelWatchCounts.count(a.getChannelID()) ?
channelWatchCounts.at(a.getChannelID()) : 0;
00085             int countB = channelWatchCounts.count(b.getChannelID()) ?
channelWatchCounts.at(b.getChannelID()) : 0;
00086             return countA > countB;
00087         });
00088
00089     std::list<Channel> result;
00090     int count = 0;
00091     for (const auto& channel : channels) {
00092         if (count >= n) break;
00093         result.push_back(channel);
00094         count++;
00095     }
00096     return result;
00097 }

```



```

00098
00099 std::string YouTubeData::toString() const {
00100     std::ostringstream watchHistoryJson;
00101     watchHistoryJson << "[";
00102     bool firstWatch = true;
00103     for (const auto& entry : watchHistory) {
00104         if (!firstWatch) {
00105             watchHistoryJson << ",";
00106         }
00107         watchHistoryJson << entry.toString();
00108         firstWatch = false;
00109     }
00110     watchHistoryJson << "]";
00111
00112     std::ostringstream likedJson;
00113     likedJson << "[";
00114     bool firstLiked = true;
00115     for (const auto& video : likedVideos) {
00116         if (!firstLiked) {
00117             likedJson << ",";
00118         }
00119         likedJson << video.toString();
00120         firstLiked = false;
00121     }
00122     likedJson << "]";
00123
00124     std::ostringstream channelsJson;
00125     channelsJson << "[";
00126     bool firstChannel = true;
00127     for (const auto& channel : subscribedChannels) {
00128         if (!firstChannel) {
00129             channelsJson << ",";
00130         }
00131         channelsJson << channel.toString();
00132         firstChannel = false;
00133     }
00134     channelsJson << "]";
00135
00136     std::ostringstream watchLaterJson;
00137     watchLaterJson << "[";
00138     bool firstLater = true;
00139     for (const auto& video : watchLaterVideos) {
00140         if (!firstLater) {
00141             watchLaterJson << ",";
00142         }
00143         watchLaterJson << video.toString();
00144         firstLater = false;
00145     }
00146     watchLaterJson << "]";
00147
00148     return "{ \"watch_history\": " + watchHistoryJson.str() + ", \"
00149         \"liked_videos\": " + likedJson.str() + ", \"
00150         \"subscribed_channels\": " + channelsJson.str() + ", \"
00151         \"watch_later\": {" + "\"videos\": " + watchLaterJson.str() + "}"
00152         + " }";
00153 }
00154 }

```

7.99 YouTubeData.hpp File Reference

Container for all YouTube data associated with a [User](#).

```

#include <list>
#include "VideoEntry.hpp"
#include "Channel.hpp"
#include "Video.hpp"

```


7.100 YouTubeData.hpp

[Go to the documentation of this file.](#)

```

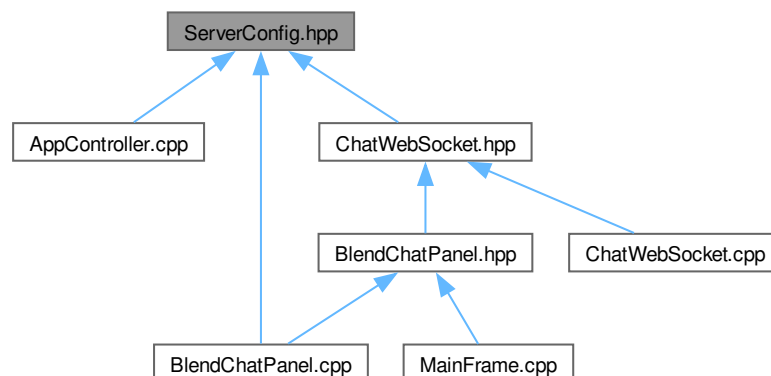
00001 #ifndef YOUTUBEDATA_H
00002 #define YOUTUBEDATA_H
00003
00017
00018 #include <list>
00019
00020 #include "VideoEntry.hpp"
00021 #include "Channel.hpp"
00022 #include "Video.hpp"
00023
00029 class YouTubeData {
00030     private:
00031         std::list<VideoEntry> watchHistory; // further enriched list of videos from users watch
00032         history
00033         std::list<Video> likedVideos; // list of videos from users liked playlist
00034         std::list<Channel> subscribedChannels; // list of users subscribed channels
00035         std::list<Video> watchLaterVideos; // list of videos from users watch later (any
00036         playlist)
00037     public:
00038         YouTubeData();
00039
00040         std::list<VideoEntry> getWatchHistory() const;
00041         std::list<Video> getLikedVideos() const;
00042         std::list<Channel> getSubscribedChannels() const;
00043         std::list<Video> getWatchLaterVideos() const;
00044
00045         void setWatchHistory(const std::list<VideoEntry>& watchHistory);
00046         void setLikedVideos(const std::list<Video>& likedVideos);
00047         void setSubscribedChannels(const std::list<Channel>& subscribedChannels);
00048         void setWatchLaterVideos(const std::list<Video>& videos);
00049
00050         std::list<Video> getMostWatchedVideos(int n) const;
00051
00052         std::list<Channel> getTopChannels(int n) const;
00053
00054         std::string toString() const;
00055 };
00056
00057 #endif

```

7.101 ServerConfig.hpp File Reference

Centralized backend server addresses.

This graph shows which files directly or indirectly include this file:



Variables

- `constexpr const char * kAppBackendBaseUrl = "http://137.220.58.22:8080"`
HTTP base URL used by [AppController](#) and other app-side HTTP calls.
- `constexpr const char * kAppBackendWsPrefix = "ws://137.220.58.22:8080/api/v1/ws/chats/"`
WebSocket endpoint prefix used by [BlendChatPanel](#).
- `constexpr const char * kTestBackendBaseUrl = "http://137.220.58.22:8080"`
HTTP base URL used by the test suite.
- `constexpr const char * kTestBackendWsPrefix = "ws://137.220.58.22:8080/api/v1/ws/chats/"`
WebSocket endpoint prefix used by tests that exercise WS connectivity.

7.101.1 Detailed Description

Centralized backend server addresses.

Author

Shamar Pennant

Edit the constants below to retarget the application and/or the test suite to a different server. No other file needs to change.

- `kApp*` constants are used by the running application ([AppController](#), [UILayer](#)).
- `kTest*` constants are used by the test suite and can point at a separate staging instance if needed.

Definition in file [ServerConfig.hpp](#).

7.101.2 Variable Documentation

7.101.2.1 kAppBackendBaseUrl

```
const char* kAppBackendBaseUrl = "http://137.220.58.22:8080" [inline], [constexpr]
```

HTTP base URL used by [AppController](#) and other app-side HTTP calls.

Definition at line 17 of file [ServerConfig.hpp](#).

7.101.2.2 kAppBackendWsPrefix

```
const char* kAppBackendWsPrefix = "ws://137.220.58.22:8080/api/v1/ws/chats/" [inline], [constexpr]
```

WebSocket endpoint prefix used by [BlendChatPanel](#).

Definition at line 20 of file [ServerConfig.hpp](#).

7.101.2.3 kTestBackendBaseUrl

```
const char* kTestBackendBaseUrl = "http://137.220.58.22:8080" [inline], [constexpr]
```

HTTP base URL used by the test suite.

Definition at line 23 of file [ServerConfig.hpp](#).

7.101.2.4 kTestBackendWsPrefix

```
const char* kTestBackendWsPrefix = "ws://137.220.58.22:8080/api/v1/ws/chats/" [inline], [constexpr]
```

WebSocket endpoint prefix used by tests that exercise WS connectivity.

Definition at line 26 of file [ServerConfig.hpp](#).

7.102 ServerConfig.hpp

[Go to the documentation of this file.](#)

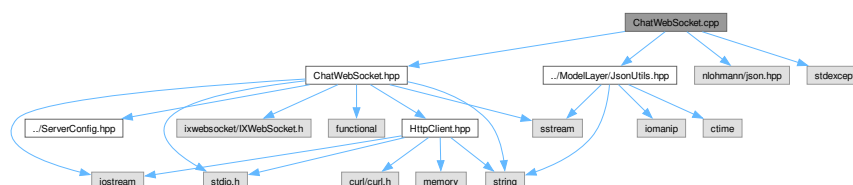
```
00001 #pragma once
00002
00015
00017 inline constexpr const char* kAppBackendBaseUrl = "http://137.220.58.22:8080";
00018
00020 inline constexpr const char* kAppBackendWsPrefix = "ws://137.220.58.22:8080/api/v1/ws/chats/";
00021
00023 inline constexpr const char* kTestBackendBaseUrl = "http://137.220.58.22:8080";
00024
00026 inline constexpr const char* kTestBackendWsPrefix = "ws://137.220.58.22:8080/api/v1/ws/chats/";
```

7.103 ChatWebSocket.cpp File Reference

Implementation for [ChatWebSocket](#).

```
#include "ChatWebSocket.hpp"
#include "../ModelLayer/JsonUtils.hpp"
#include <nlohmann/json.hpp>
#include <stdexcept>
```

Include dependency graph for ChatWebSocket.cpp:



Functions

- static std::string [urlEncode](#) (const std::string &value)

7.103.1 Detailed Description

Implementation for [ChatWebSocket](#).

Author

Shamar Pennant

Definition in file [ChatWebSocket.cpp](#).

7.103.2 Function Documentation

7.103.2.1 urlEncode()

```
std::string urlEncode (
    const std::string & value) [static]
```

Definition at line 16 of file [ChatWebSocket.cpp](#).

7.104 ChatWebSocket.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #include "ChatWebSocket.hpp"
00008 #include "../ModelLayer/JsonUtils.hpp"
00009
00010 #include <nlohmann/json.hpp>
00011 #include <stdexcept>
00012
00013 using namespace std;
00014 using Json = nlohmann::json;
00015
00016 static std::string urlEncode(const std::string& value) {
00017     if (value.empty()) {
00018         return value;
00019     }
00020
00021     CURL* encoder = curl_easy_init();
00022     if (encoder == nullptr) {
00023         return value;
00024     }
00025
00026     char* escaped = curl_easy_escape(encoder, value.c_str(), static_cast<int>(value.size()));
00027     if (escaped == nullptr) {
00028         curl_easy_cleanup(encoder);
00029         return value;
00030     }
00031
00032     std::string encoded(escaped);
00033     curl_free(escaped);
00034     curl_easy_cleanup(encoder);
00035     return encoded;
00036 }
00037
00038 ChatWebSocket::ChatWebSocket(const string url) {
00039     this->url = url;
00040
00041     // setting up specialized broadcasting and functions
00042     ws.setOnMessageCallback([this](const ix::WebSocketMessagePtr& msg) {
00043         if (msg->type == ix::WebSocketMessageType::Message) {
00044             std::cout << "WS Received: " << msg->str << "\n";
00045             // Parse sender/content from contract-shaped outbound server messages.
00046             if (m_onMessage) {
00047                 Json parsed = Json::parse(msg->str, nullptr, false);
00048                 if (parsed.is_discarded() || !parsed.is_object()) {
00049                     return;
```

```

00050         }
00051
00052         std::string sender = parsed.value("sender_id", "");
00053         std::string content = parsed.value("content", "");
00054         if (!sender.empty() && !content.empty()) {
00055             m_onMessage(sender, content);
00056         }
00057     }
00058 }
00059 else if (msg->type == ix::WebSocketMessageType::Open) {
00060     std::cout << "WS Connected\n";
00061 }
00062 else if (msg->type == ix::WebSocketMessageType::Error) {
00063     std::cerr << "WS Error: " << msg->errorInfo.reason << "\n";
00064 }
00065 });
00066 }
00067
00068 ChatWebSocket::~ChatWebSocket() {
00069     if (isStarted) {
00070         ws.stop();
00071     }
00072 }
00073
00074 void ChatWebSocket::set_blendID(const string blendID) { this->blendID = blendID; }
00075 void ChatWebSocket::set_userID(const string userID) { this->userID = userID; }
00076 string ChatWebSocket::get_fullUrl() { return fullUrl; }
00077
00078 void ChatWebSocket::connect() {
00079     if (isStarted) return;
00080     build_fullUrl();
00081     ws.setUrl(fullUrl);
00082     ws.start();
00083     isStarted = true;
00084 }
00085
00086 void ChatWebSocket::setOnMessage(
00087     std::function<void(const std::string&, const std::string&)> cb) {
00088     m_onMessage = std::move(cb);
00089 }
00090
00091 void ChatWebSocket::send_message(string msg) {
00092     connect(); // no-op if already started
00093     ws.send(content_formatter(msg));
00094 }
00095
00096 string ChatWebSocket::content_formatter(string content) {
00097     return "{\"content\":\"" + ModelJson::quote(content) + "\"}";
00098 }
00099
00100 void ChatWebSocket::build_fullUrl() {
00101     // checks if blendID and userID have been correctly configured
00102     if (blendID.empty() || userID.empty()) {
00103         throw invalid_argument("ChatWebSocket requires blendID and userID before sending messages");
00104     }
00105
00106     stringstream fu;
00107     fu << url << urlEncode(blendID) << "?user_id=" << urlEncode(userID);
00108     fullUrl = fu.str();
00109 }

```

7.105 ChatWebSocket.hpp File Reference

Simple WebSocket wrapper.

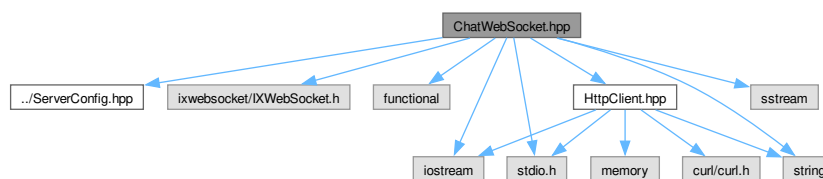
```

#include "../ServerConfig.hpp"
#include <ixwebsocket/IXWebSocket.h>
#include <functional>
#include <iostream>
#include <stdio.h>
#include <sstream>
#include <string>

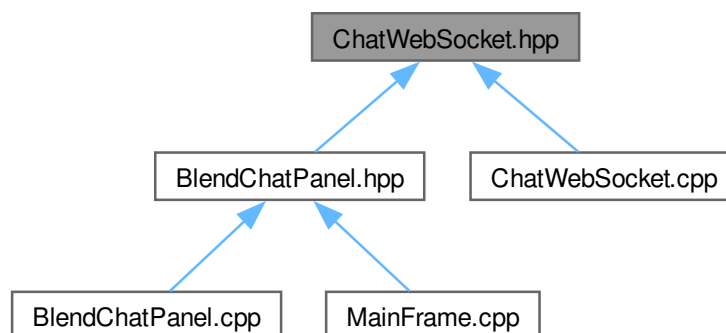
```

```
#include "HttpClient.hpp"
```

Include dependency graph for ChatWebSocket.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [ChatWebSocket](#)
[ChatWebSocket](#) class declaration.

7.105.1 Detailed Description

Simple WebSocket wrapper.

Author

Shamar Pennant

Simple websocket wrapper to ease communications with the server in regards to chatrooms

Definition in file [ChatWebSocket.hpp](#).

7.106 ChatWebSocket.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include "../ServerConfig.hpp"
00004 #include <ixwebsocket/IXWebSocket.h>
00005 #include <functional>
00006 #include <iostream>
00007 #include <stdio.h>
00008 #include <sstream>
00009 #include <string>
00010 #include "HttpClient.hpp"
00011
00020
00025 class ChatWebSocket {
00026     public:
00034         ChatWebSocket(const std::string url);
00035
00039         ~ChatWebSocket();
00040
00046         void connect();
00047
00053         void setOnMessage(std::function<void(const std::string& senderID,
00054                                             const std::string& content)> cb);
00055
00060         void set_blendID(const std::string blendID);
00061
00066         void set_userID(const std::string userID);
00067
00072         std::string get_fullUrl();
00073
00079         void send_message(std::string msg);
00080
00081     private:
00082         // WebSocket object
00083         ix::WebSocket ws;
00084         // copy of the last given url
00085         std::string url = kAppBackendWsPrefix;
00086         // copy of the last given blendID for the socket path
00087         std::string blendID;
00088         // copy of the last given userID for the socket path
00089         std::string userID;
00090         // full websocket url
00091         std::string fullUrl;
00092         // Tracks whether the socket loop has been started.
00093         bool isStarted = false;
00094         // Optional callback invoked on incoming chat messages.
00095         std::function<void(const std::string&, const std::string&)> m_onMessage;
00096
00103         std::string content_formatter(std::string content);
00104
00109         void build_fullUrl();
00110 };

```

7.107 DataManager.hpp File Reference

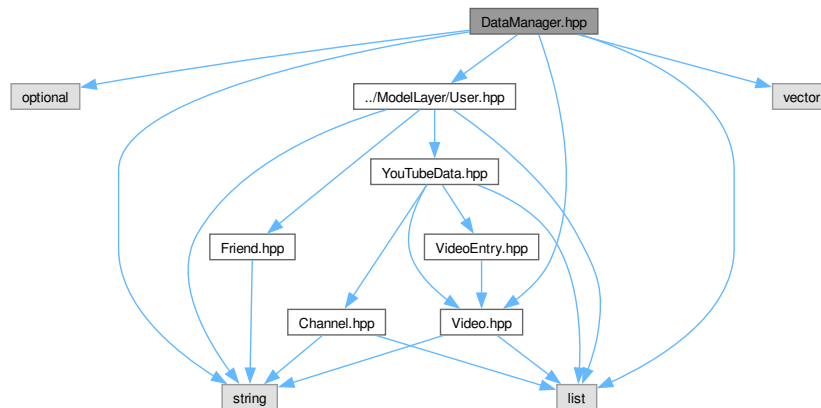
Interface for all user-persistence operations.

```

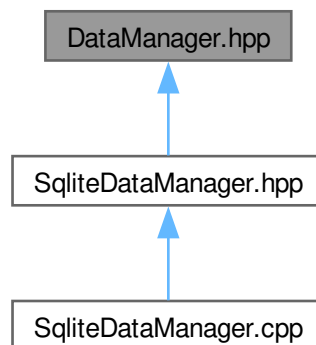
#include <optional>
#include <string>
#include <vector>
#include <list>
#include "../ModelLayer/User.hpp"
#include "../ModelLayer/Video.hpp"

```

Include dependency graph for DataManager.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [DataManager](#)
[DataManager](#) class declaration.

7.107.1 Detailed Description

Interface for all user-persistence operations.

Author

Shamar Pennant
Jasmine Kumar

[AppController](#) depends only on this interface, keeping it decoupled from the concrete storage back-end (SQLite today, something else tomorrow). Tests can inject a mock implementation without touching the real DB.

Definition in file [DataManager.hpp](#).

7.108 DataManager.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef DATA_MANAGER_H
00002 #define DATA_MANAGER_H
00003
00004 #include <optional>
00005 #include <string>
00006 #include <vector>
00007 #include <list>
00008 #include "../ModelLayer/User.hpp"
00009 #include "../ModelLayer/Video.hpp"
00010
00025 class DataManager {
00026 public:
00027     virtual ~DataManager() = default;
00028
00034     virtual bool createUser(const User& user) = 0;
00035
00040     virtual std::optional<User> findUserByID(const std::string& userID) = 0;
00041
00046     virtual bool validatePassword(const std::string& userID,
00047                                   const std::string& password) = 0;
00048
00053     virtual bool saveWatchLater(const std::string& userID,
00054                                   const std::list<Video>& videos) = 0;
00055
00060     virtual std::list<Video> loadWatchLater(const std::string& userID) = 0;
00061
00068     virtual bool saveBlend(const std::string& blendID,
00069                             const std::string& creatorID,
00070                             const std::string& algorithm,
00071                             const std::vector<std::string>& participantIDs) = 0;
00072
00077     virtual std::optional<std::string> findBlendForUser(const std::string& userID) = 0;
00078
00084     virtual std::vector<std::string> loadBlendParticipants(const std::string& blendID) = 0;
00085 };
00086
00087 #endif

```

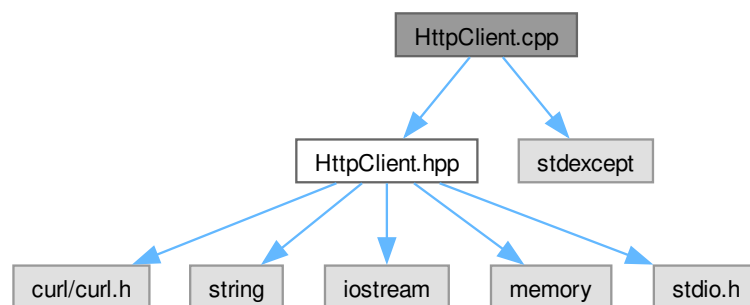
7.109 HttpClient.cpp File Reference

Implementation for [HttpClient](#).

```
#include "HttpClient.hpp"
```

```
#include <stdexcept>
```

Include dependency graph for HttpClient.cpp:



7.109.1 Detailed Description

Implementation for [HttpClient](#).

Author

Shamar Pennant

Definition in file [HttpClient.cpp](#).

7.110 HttpClient.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00012
00013 #include "HttpClient.hpp"
00014
00015 #include <stdexcept>
00016
00017 using namespace std;
00018
00019 namespace {
00020
00021 string urlEncodeComponent(const string& value) {
00022     if (value.empty()) {
00023         return value;
00024     }
00025
00026     CURL* encoder = curl_easy_init();
00027     if (encoder == nullptr) {
00028         return value;
00029     }
00030
00031     char* escaped = curl_easy_escape(encoder, value.c_str(), static_cast<int>(value.size()));
00032     if (escaped == nullptr) {
00033         curl_easy_cleanup(encoder);
00034         return value;
00035     }
00036
00037     string encoded(escaped);
00038     curl_free(escaped);
00039     curl_easy_cleanup(encoder);
00040     return encoded;
00041 }
00042
00043 string sanitizePathSegment(const string& value) {
00044     return urlEncodeComponent(value);
00045 }
00046
00047 }
00048
00049 HttpClient::HttpClient(string base) {
00050     baseUrl = move(base);
00051 }
00052
00053 bool HttpClient::isRequestSuccessful(const string& method, long statusCode) {
00054     if (method == "GET") {
00055         return statusCode == 200;
00056     }
00057     if (method == "POST") {
00058         return statusCode == 200 || statusCode == 201;
00059     }
00060     if (method == "DELETE") {
00061         return statusCode == 200 || statusCode == 204;
00062     }
00063
00064     throw invalid_argument("HttpClient::isRequestSuccessful received unsupported method: " + method);
00065 }
00066
00067 long HttpClient::getLastStatusCode() const {
00068     return lastStatusCode;
00069 }
00070
00071 bool HttpClient::wasLastRequestSuccessful(const string& method) const {

```

```

00072     return isRequestSuccessful(method, lastStatusCode);
00073 }
00074
00075 string HttpClient::get(const string& path) {
00076     return request(G, path, "");
00077 }
00078
00079 string HttpClient::post(const string& path, const string& json) {
00080     return request(P, path, json);
00081 }
00082
00083 string HttpClient::del(const string& path, const string& json) {
00084     return request(D, path, json);
00085 }
00086
00087 string HttpClient::build_watch_later_endpoint(string userID) {
00088     return "/api/v1/users/" + sanitizePathSegment(userID) + "/watch-later";
00089 }
00090
00091 string HttpClient::build_latest_blend_endpoint(string userID) {
00092     return "/api/v1/users/" + sanitizePathSegment(userID) + "/blend";
00093 }
00094
00095 string HttpClient::build_blend_participant_endpoint(string blendID) {
00096     return "/api/v1/blends/" + sanitizePathSegment(blendID) + "/participants";
00097 }
00098
00099 string HttpClient::build_chatroom_detail_endpoint(string blendID) {
00100     return "/api/v1/blends/" + sanitizePathSegment(blendID) + "/chatroom";
00101 }
00102
00103 string HttpClient::build_auth_user_lookup_endpoint(string userID) {
00104     return AUTH_USER_PREFIX + sanitizePathSegment(userID);
00105 }
00106
00107 string HttpClient::build_delete_user_endpoint(string userID) {
00108     return DELETE_USER_PREFIX + sanitizePathSegment(userID);
00109 }
00110
00111 string HttpClient::request(const string& method, const string& path, const string&body) {
00112     // grabbing a new curl object
00113     CURL* curl = curl_easy_init();
00114     if (!curl) {
00115         throw runtime_error("HttpClient failed to initialize CURL handle");
00116     }
00117
00118     // string containing the response from the HTTP method
00119     string response;
00120     // full path to endpoint
00121     const string fullUrl = baseUrl + path;
00122
00123     // --- setting up curl for requests ---
00124
00125     // instruct curl where to send the the request
00126     curl_easy_setopt(curl, CURLOPT_URL, fullUrl.c_str());
00127     // curl doesnt return the call directly, so we need write_callback()
00128     // before being able to grab the response
00129     curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, write_callback);
00130     curl_easy_setopt(curl, CURLOPT_WRITEDATA, &response);
00131
00132     // create the header for our request (following curl design patterns)
00133     struct curl_slist* headers = nullptr;
00134     headers = curl_slist_append(headers, hdr.c_str());
00135     curl_easy_setopt(curl, CURLOPT_HTTPHEADER, headers);
00136
00137     // --- handling given method ---
00138     if (method == P) { // POST
00139         // send the given body
00140         curl_easy_setopt(curl, CURLOPT_POST, 1L);
00141         curl_easy_setopt(curl, CURLOPT_POSTFIELDS, body.c_str());
00142         curl_easy_setopt(curl, CURLOPT_POSTFIELDSIZE, static_cast<long>(body.size()));
00143     } else if (method == G) { // GET
00144     } else if (method == D) { // DELETE
00145         curl_easy_setopt(curl, CURLOPT_HTTPGET, 1L);
00146     } else if (method == D) { // DELETE
00147         curl_easy_setopt(curl, CURLOPT_CUSTOMREQUEST, D.c_str());
00148         curl_easy_setopt(curl, CURLOPT_POSTFIELDS, body.c_str());
00149         curl_easy_setopt(curl, CURLOPT_POSTFIELDSIZE, static_cast<long>(body.size()));
00150     } else {
00151         curl_easy_cleanup(curl);
00152         curl_slist_free_all(headers);
00153         throw invalid_argument("HttpClient::request received unsupported method: " + method);
00154     }
00155 }
00156
00157 // firing off the request and clean up all pointers

```

```

00168     CURLcode result = curl_easy_perform(curl);
00169
00170     long statusCode = 0;
00171     curl_easy_getinfo(curl, CURLINFO_RESPONSE_CODE, &statusCode);
00172     lastStatusCode = statusCode;
00173
00174     curl_easy_cleanup(curl);
00175     curl_slist_free_all(headers);
00176
00177     if (result != CURLE_OK) {
00178         throw runtime_error(string("HttpClient request failed: ") + curl_easy_strerror(result));
00179     }
00180
00181     return response;
00182 }
00183
00184 string HttpClient::build_user_blends_endpoint(string userID) {
00185     return "/api/v1/users/" + sanitizePathSegment(userID) + "/blends";
00186 }
00187
00188 string HttpClient::build_leave_blend_endpoint(string blendID) {
00189     return "/api/v1/blends/" + sanitizePathSegment(blendID) + "/leave";
00190 }
00191
00192 string HttpClient::build_chat_history_endpoint(string blendID, string userID) {
00193     return "/api/v1/blends/" + sanitizePathSegment(blendID) + "/chat-history?user_id=" +
        sanitizePathSegment(userID);
00194 }
00195
00196 size_t HttpClient::write_callback(void* contents, size_t size, size_t nmemb, void* out) {
00197     auto* response = static_cast<string*>(out);
00198     response->append(static_cast<char*>(contents), size * nmemb);
00199
00200     return size * nmemb;
00201 }

```

7.111 HttpClient.hpp File Reference

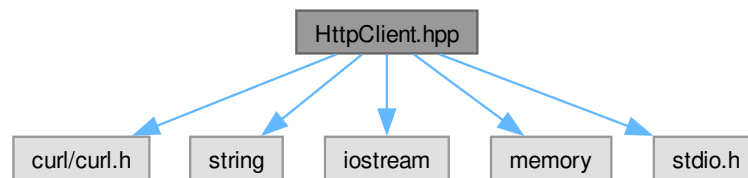
Simple HTTP client wrapper.

```

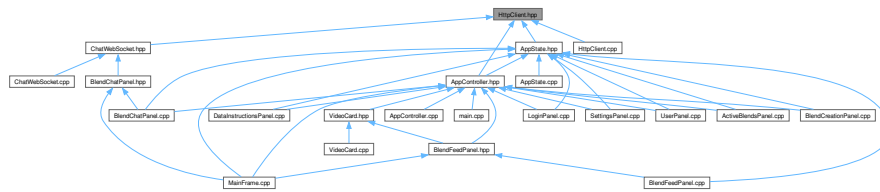
#include <curl/curl.h>
#include <string>
#include <iostream>
#include <memory>
#include <stdio.h>

```

Include dependency graph for HttpClient.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `HttpClient`
HttpClient class declaration.

7.111.1 Detailed Description

Simple HTTP client wrapper.

Author

Shamar Pennant

Handles sending and receiving HTTP messages to our custom server. Using custom formatting and structure as defined in the `route` layer/

Definition in file [HttpClient.hpp](#).

7.112 HttpClient.hpp

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00003 #include <curl/curl.h>
00004 #include <string>
00005 #include <iostream>
00006 #include <memory>
00007 #include <stdio.h>
00008
00009
00010
00011
00022 class HttpClient {
00023 public:
00024     // Constant for posting to eliminate typing errors
00025     const std::string P = "POST";
00026     // Constant for getting to eliminate typing errors
00027     const std::string G = "GET";
00028     // Constant for deleting to eliminate typing errors
00029     const std::string D = "DELETE";
00030
00031     // --- endpoint constants ---
00032
00033     // user registration endpoint
00034     const std::string REG_USER = "/api/v1/auth/register";
00035     // user login endpoint
00036     const std::string LOGIN = "/api/v1/auth/login";
00037     // user profile lookup endpoint prefix
00038     const std::string AUTH_USER_PREFIX = "/api/v1/auth/users/";
00039     // user delete endpoint prefix
00040     const std::string DELETE_USER_PREFIX = "/api/v1/auth/users/";
00041     // endpoint pertaining to creating a blend
```

```

00042         const std::string BLEND = "/api/v1/blends";
00043
00044         // --- status codes ---
00045
00046         const int SUCC = 200; // success
00047         const int ALT_SUCC = 201; // alt success
00048         const int MAL_REQ_ERR = 400; // malformed request error
00049         const int INV_USR_LOG_ERR = 401; // invalid login credentials
00050         const int UNAUTH_RES_REQ_ERR = 403; // user not authorized for request resource
00051         const int MISS_RM_CNTXT = 404; // missing user or blend room context
00052         const int DUPLICATE = 409; // duplicate entry
00053         const int SERVER_ERR = 500; // server error
00054         //
00055
00060         HttpClient(std::string base);
00061         ~HttpClient() = default;
00062
00063         static bool isRequestSuccessful(const std::string& method, long statusCode);
00064
00065         long getLastStatusCode() const;
00066
00067         bool wasLastRequestSuccessful(const std::string& method) const;
00068
00069         std::string get(const std::string& path);
00070         std::string post(const std::string& path, const std::string& json);
00071
00072         std::string del(const std::string& path, const std::string& json);
00073
00074         std::string build_watch_later_endpoint(std::string userID);
00075
00076         std::string build_latest_blend_endpoint(std::string userID);
00077
00078         std::string build_blend_participant_endpoint(std::string blendID);
00079
00080         std::string build_chatroom_detail_endpoint(std::string blendID);
00081
00082         std::string build_auth_user_lookup_endpoint(std::string userID);
00083
00084         std::string build_delete_user_endpoint(std::string userID);
00085
00086         std::string build_user_blends_endpoint(std::string userID);
00087
00088         std::string build_leave_blend_endpoint(std::string blendID);
00089
00090         std::string build_chat_history_endpoint(std::string blendID, std::string userID);
00091
00092     private:
00093         // Holds the base url for server communication
00094         std::string baseUrl;
00095
00096         // used as a header internally (specifically for our server)
00097         const std::string hdr = "Content-Type: application/json";
00098
00099         // HTTP status code from the latest response (0 means not set yet).
00100         long lastStatusCode = 0;
00101
00102         std::string request(const std::string& method, const std::string& path, const std::string&
00103             body);
00104
00105         static size_t write_callback(void* contents, size_t size, size_t nmemb, void* out);
00106     };

```

7.113 RequestJsonBuilder.cpp File Reference

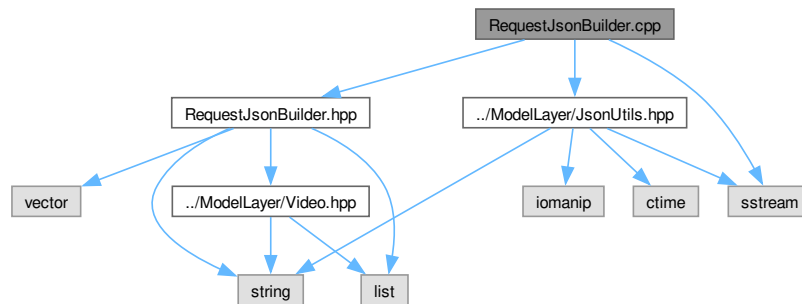
Implementation for RequestJsonBuilder.

```

#include "RequestJsonBuilder.hpp"
#include "../ModelLayer/JsonUtils.hpp"
#include <sstream>

```


Include dependency graph for RequestJsonBuilder.cpp:



Functions

- `std::string RequestJsonBuilder::buildRegisterJson` (const `std::string` &userID, const `std::string` &username, const `std::string` &email, const `std::string` &password)
Builds the registration request body for the auth/register endpoint.
- `std::string RequestJsonBuilder::buildLoginJson` (const `std::string` &userID, const `std::string` &password)
Builds the login request body for the auth/login endpoint.
- `std::string RequestJsonBuilder::buildDeleteAccountJson` (const `std::string` &password)
Builds the delete-account request body.
- `std::string RequestJsonBuilder::buildBlendJson` (const `std::string` &blendID, const `std::string` &title, const `std::string` &creatorID, const `std::string` &algorithm, const `std::vector`< `std::string` > &participants)
Builds the blend creation request body.
- `std::string RequestJsonBuilder::buildWatchLaterJson` (const `std::list`< `Video` > &videos)
Builds the watch-later persistence request body.

7.113.1 Detailed Description

Implementation for RequestJsonBuilder.

Author

Shamar Pennant

Definition in file [RequestJsonBuilder.cpp](#).

7.113.2 Function Documentation

7.113.2.1 buildBlendJson()

```

std::string RequestJsonBuilder::buildBlendJson (
    const std::string & blendID,
    const std::string & title,
    const std::string & creatorID,

```

```
const std::string & algorithm,
const std::vector< std::string > & participants)
```

Builds the blend creation request body.

Output format:

```
{
  "blend_id": "...",
  "creator_id": "...",
  "algorithm": "...",
  "participants": ["...", "..."]
}
```

Parameters

<i>blendID</i>	Unique blend identifier generated client-side.
<i>creatorID</i>	User ID of the blend creator.
<i>algorithm</i>	Name of the algorithm used to generate the blend.
<i>participants</i>	Ordered participant user IDs for this blend.

Returns

JSON string safe for HTTP POST to the blend endpoint.

Definition at line 40 of file [RequestJsonBuilder.cpp](#).

7.113.2.2 buildDeleteAccountJson()

```
std::string RequestJsonBuilder::buildDeleteAccountJson (
    const std::string & password)
```

Builds the delete-account request body.

Output format:

```
{ "password": "..."
```

Parameters

<i>password</i>	Plain-text password entered by the user for re-authentication.
-----------------	--

Returns

JSON string safe for HTTP DELETE to the user-delete endpoint.

Definition at line 34 of file [RequestJsonBuilder.cpp](#).

7.113.2.3 buildLoginJson()

```
std::string RequestJsonBuilder::buildLoginJson (
    const std::string & userID,
    const std::string & password)
```

Builds the login request body for the auth/login endpoint.

Output format:

```
{ "user_id": "...", "password": "..."
```

Parameters

<i>userID</i>	User identifier provided by the client.
<i>password</i>	Plain-text password entered by the user.

Returns

JSON string safe for HTTP POST to the login endpoint.

Definition at line 27 of file [RequestJsonBuilder.cpp](#).

7.113.2.4 buildRegisterJson()

```
std::string RequestJsonBuilder::buildRegisterJson (  
    const std::string & userID,  
    const std::string & username,  
    const std::string & email,  
    const std::string & password)
```

Builds the registration request body for the auth/register endpoint.

Output format:

```
{  
  "user_id": "...",  
  "username": "...",  
  "email": "...",  
  "password": "..."  
}
```

Parameters

<i>userID</i>	User identifier chosen by the registering user.
<i>username</i>	Display name to store with the account.
<i>email</i>	Optional email value (can be empty).
<i>password</i>	Plain-text password entered during registration.

Returns

JSON string safe for HTTP POST to the register endpoint.

Definition at line 15 of file [RequestJsonBuilder.cpp](#).

7.113.2.5 buildWatchLaterJson()

```
std::string RequestJsonBuilder::buildWatchLaterJson (  
    const std::list< Video > & videos)
```

Builds the watch-later persistence request body.

Output format:

```
{  
  "videos": [  
    { "video_id": "...", "title": "..." }  
  ]  
}
```

Parameters

<code>videos</code>	Watch-later videos to serialize.
---------------------	----------------------------------

Returns

JSON string safe for HTTP POST to the watch-later endpoint.

Definition at line 64 of file [RequestJsonBuilder.cpp](#).

7.114 RequestJsonBuilder.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "RequestJsonBuilder.hpp"
00008
00009 #include "../ModelLayer/JsonUtils.hpp"
00010
00011 #include <sstream>
00012
00013 namespace RequestJsonBuilder {
00014
00015 std::string buildRegisterJson(const std::string& userID,
00016                             const std::string& username,
00017                             const std::string& email,
00018                             const std::string& password) {
00019     return "{"
00020         "\"user_id\": " + ModelJson::quote(userID) + ", "
00021         "\"username\": " + ModelJson::quote(username) + ", "
00022         "\"email\": " + ModelJson::quote(email) + ", "
00023         "\"password\": " + ModelJson::quote(password)
00024         + "}";
00025 }
00026
00027 std::string buildLoginJson(const std::string& userID, const std::string& password) {
00028     return "{"
00029         "\"user_id\": " + ModelJson::quote(userID) + ", "
00030         "\"password\": " + ModelJson::quote(password)
00031         + "}";
00032 }
00033
00034 std::string buildDeleteAccountJson(const std::string& password) {
00035     return "{"
00036         "\"password\": " + ModelJson::quote(password)
00037         + "}";
00038 }
00039
00040 std::string buildBlendJson(const std::string& blendID,
00041                           const std::string& title,
00042                           const std::string& creatorID,
00043                           const std::string& algorithm,
00044                           const std::vector<std::string>& participants) {
00045     std::ostringstream participantsJson;
00046     participantsJson << "[";
00047     for (size_t i = 0; i < participants.size(); ++i) {
00048         if (i > 0) {
00049             participantsJson << ", ";
00050         }
00051         participantsJson << ModelJson::quote(participants[i]);
00052     }
00053     participantsJson << "]";
00054
00055     return "{"
00056         "\"blend_id\": " + ModelJson::quote(blendID) + ", "
00057         "\"title\": " + ModelJson::quote(title) + ", "
00058         "\"creator_id\": " + ModelJson::quote(creatorID) + ", "
00059         "\"algorithm\": " + ModelJson::quote(algorithm) + ", "
00060         "\"participants\": " + participantsJson.str()
00061         + "}";
00062 }
00063
00064 std::string buildWatchLaterJson(const std::list<Video>& videos) {
00065     std::ostringstream videosJson;

```

```

00066     videosJson « "[";
00067
00068     bool first = true;
00069     for (const auto& video : videos) {
00070         if (!first) {
00071             videosJson « ",";
00072         }
00073         first = false;
00074
00075         videosJson « "{"
00076             « "\"video_id\": \" « ModelJson::quote(video.getVideoID()) « ","
00077             « "\"title\": \" « ModelJson::quote(video.getTitle()) « ","
00078             « "\"channel_id\": \" « ModelJson::quote(video.getChannelID()) « ","
00079             « "\"channel_name\": \" « ModelJson::quote(video.getChannelName()) « ","
00080             « "\"thumbnail_url\": \" « ModelJson::quote(video.getThumbnailURL()) « ","
00081             « "\"channel_logo_url\": \" « ModelJson::quote(video.getChannelLogoURL())
00082             « "}"
00083     }
00084
00085     videosJson « "]";
00086     return "{\"videos\": " + videosJson.str() + "}";
00087 }
00088
00089 } // namespace RequestJsonBuilder

```

7.115 RequestJsonBuilder.hpp File Reference

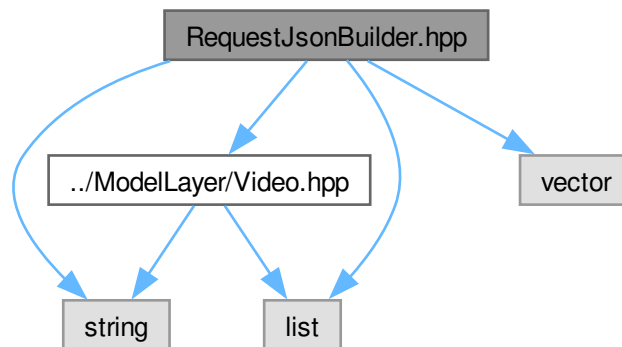
JSON request body builders used by ServiceLayer HTTP calls.

```

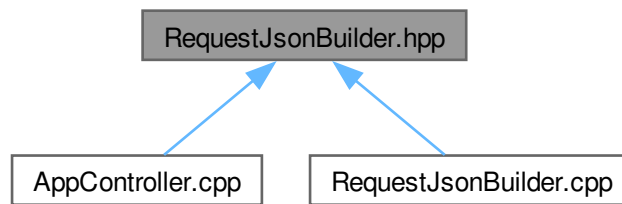
#include "../ModelLayer/Video.hpp"
#include <list>
#include <string>
#include <vector>

```

Include dependency graph for RequestJsonBuilder.hpp:



This graph shows which files directly or indirectly include this file:



Functions

- `std::string RequestJsonBuilder::buildRegisterJson` (`const std::string &userID`, `const std::string &username`, `const std::string &email`, `const std::string &password`)
Builds the registration request body for the auth/register endpoint.
- `std::string RequestJsonBuilder::buildLoginJson` (`const std::string &userID`, `const std::string &password`)
Builds the login request body for the auth/login endpoint.
- `std::string RequestJsonBuilder::buildDeleteAccountJson` (`const std::string &password`)
Builds the delete-account request body.
- `std::string RequestJsonBuilder::buildBlendJson` (`const std::string &blendID`, `const std::string &title`, `const std::string &creatorID`, `const std::string &algorithm`, `const std::vector< std::string > &participants`)
Builds the blend creation request body.
- `std::string RequestJsonBuilder::buildWatchLaterJson` (`const std::list< Video > &videos`)
Builds the watch-later persistence request body.

7.115.1 Detailed Description

JSON request body builders used by ServiceLayer HTTP calls.

Author

Shamar Pennant

Definition in file [RequestJsonBuilder.hpp](#).

7.115.2 Function Documentation

7.115.2.1 buildBlendJson()

```

std::string RequestJsonBuilder::buildBlendJson (
    const std::string & blendID,
    const std::string & title,
    const std::string & creatorID,

```

```
const std::string & algorithm,  
const std::vector< std::string > & participants)
```

Builds the blend creation request body.

Output format:

```
{  
  "blend_id": "...",  
  "creator_id": "...",  
  "algorithm": "...",  
  "participants": ["...", "..."]  
}
```

Parameters

<i>blendID</i>	Unique blend identifier generated client-side.
<i>creatorID</i>	User ID of the blend creator.
<i>algorithm</i>	Name of the algorithm used to generate the blend.
<i>participants</i>	Ordered participant user IDs for this blend.

Returns

JSON string safe for HTTP POST to the blend endpoint.

Definition at line 40 of file [RequestJsonBuilder.cpp](#).

7.115.2.2 buildDeleteAccountJson()

```
std::string JsonRequestBuilder::buildDeleteAccountJson (  
    const std::string & password)
```

Builds the delete-account request body.

Output format:

```
{ "password": "..."
```

Parameters

<i>password</i>	Plain-text password entered by the user for re-authentication.
-----------------	--

Returns

JSON string safe for HTTP DELETE to the user-delete endpoint.

Definition at line 34 of file [RequestJsonBuilder.cpp](#).

7.115.2.3 buildLoginJson()

```
std::string JsonRequestBuilder::buildLoginJson (  
    const std::string & userID,  
    const std::string & password)
```

Builds the login request body for the auth/login endpoint.

Output format:

```
{ "user_id": "...", "password": "..."
```

Parameters

<i>userID</i>	User identifier provided by the client.
<i>password</i>	Plain-text password entered by the user.

Returns

JSON string safe for HTTP POST to the login endpoint.

Definition at line 27 of file [RequestJsonBuilder.cpp](#).

7.115.2.4 buildRegisterJson()

```
std::string RequestJsonBuilder::buildRegisterJson (
    const std::string & userID,
    const std::string & username,
    const std::string & email,
    const std::string & password)
```

Builds the registration request body for the auth/register endpoint.

Output format:

```
{
  "user_id": "...",
  "username": "...",
  "email": "...",
  "password": "..."
}
```

Parameters

<i>userID</i>	User identifier chosen by the registering user.
<i>username</i>	Display name to store with the account.
<i>email</i>	Optional email value (can be empty).
<i>password</i>	Plain-text password entered during registration.

Returns

JSON string safe for HTTP POST to the register endpoint.

Definition at line 15 of file [RequestJsonBuilder.cpp](#).

7.115.2.5 buildWatchLaterJson()

```
std::string RequestJsonBuilder::buildWatchLaterJson (
    const std::list< Video > & videos)
```

Builds the watch-later persistence request body.

Output format:

```
{
  "videos": [
    { "video_id": "...", "title": "..." }
  ]
}
```


Parameters

<i>videos</i>	Watch-later videos to serialize.
---------------	----------------------------------

Returns

JSON string safe for HTTP POST to the watch-later endpoint.

Definition at line 64 of file [RequestJsonBuilder.cpp](#).

7.116 RequestJsonBuilder.hpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #pragma once
00008
00009 #include "../ModelLayer/Video.hpp"
00010
00011 #include <list>
00012 #include <string>
00013 #include <vector>
00014
00015 namespace RequestJsonBuilder {
00016
00036 std::string buildRegisterJson(const std::string& userID,
00037                               const std::string& username,
00038                               const std::string& email,
00039                               const std::string& password);
00040
00053 std::string buildLoginJson(const std::string& userID, const std::string& password);
00054
00066 std::string buildDeleteAccountJson(const std::string& password);
00067
00087 std::string buildBlendJson(const std::string& blendID,
00088                             const std::string& title,
00089                             const std::string& creatorID,
00090                             const std::string& algorithm,
00091                             const std::vector<std::string>& participants);
00092
00108 std::string buildWatchLaterJson(const std::list<Video>& videos);
00109
00110 } // namespace RequestJsonBuilder

```

7.117 SqliteDataManager.cpp File Reference

Implementation for [SqliteDataManager](#).

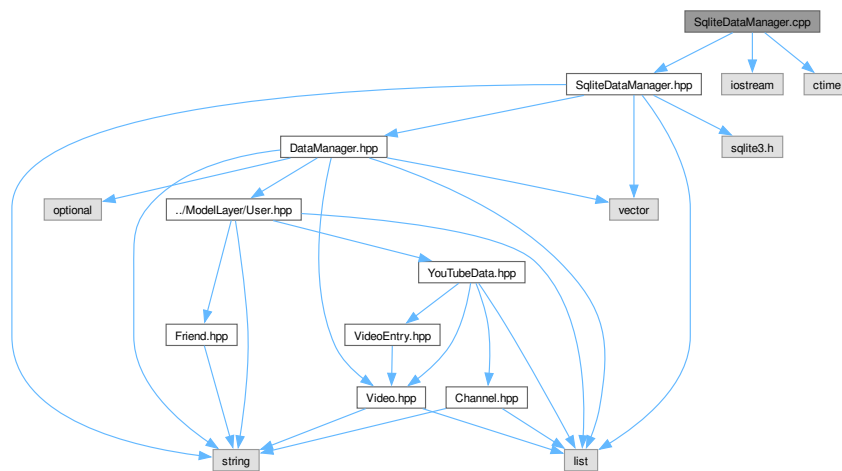
```

#include "SqliteDataManager.hpp"
#include <iostream>

```

```
#include <ctime>
```

Include dependency graph for `SqliteDataManager.cpp`:



7.117.1 Detailed Description

Implementation for [SqliteDataManager](#).

Author

Jasmine Kumar

Definition in file [SqliteDataManager.cpp](#).

7.118 SqliteDataManager.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "SqliteDataManager.hpp"
00008 #include <iostream>
00009 #include <ctime>
00010
00011 // -- Constructor / Destructor -----
00012
00013 SqliteDataManager::SqliteDataManager(const std::string& dbPath) {
00014     int rc = sqlite3_open(dbPath.c_str(), &db);
00015     if (rc != SQLITE_OK) {
00016         std::cerr << "[SqliteDataManager] Failed to open DB: "
00017                 << sqlite3_errmsg(db) << "\n";
00018         sqlite3_close(db);
00019         db = nullptr;
00020         return;
00021     }
00022     initSchema();
00023 }
00024
00025 SqliteDataManager::~SqliteDataManager() {
00026     if (db) sqlite3_close(db);
00027 }
00028
00029 // -- Private helpers -----
00030

```

```

00031 void SqliteDataManager::initSchema() {
00032     const char* sql =
00033         // -- Users -----
00034         "CREATE TABLE IF NOT EXISTS users ("
00035         "    user_id TEXT PRIMARY KEY,"
00036         "    username TEXT NOT NULL,"
00037         "    email TEXT,"
00038         "    password TEXT NOT NULL"
00039         ");"
00040
00041         // -- Watch Later videos per user -----
00042         // Stores each parsed + API-enriched video as a row; replaced in bulk
00043         // on re-upload. position preserves original playlist order for display.
00044         "CREATE TABLE IF NOT EXISTS user_watch_later ("
00045         "    user_id TEXT NOT NULL,"
00046         "    video_id TEXT NOT NULL,"
00047         "    title TEXT,"
00048         "    channel_id TEXT,"
00049         "    channel_name TEXT,"
00050         "    channel_logo_url TEXT,"
00051         "    thumbnail_url TEXT,"
00052         "    position INTEGER NOT NULL DEFAULT 0,"
00053         "    PRIMARY KEY (user_id, video_id)"
00054         ");"
00055
00056         // -- Blend metadata -----
00057         // creator_id kept separate for future delete/manage privileges.
00058         "CREATE TABLE IF NOT EXISTS blends ("
00059         "    blend_id TEXT PRIMARY KEY,"
00060         "    creator_id TEXT NOT NULL,"
00061         "    algorithm TEXT NOT NULL,"
00062         "    created_at TEXT NOT NULL"
00063         ");"
00064
00065         // -- Blend participants -----
00066         // One row per (blend, user) pair - enables fast lookup in both directions.
00067         "CREATE TABLE IF NOT EXISTS blend_participants ("
00068         "    blend_id TEXT NOT NULL,"
00069         "    user_id TEXT NOT NULL,"
00070         "    PRIMARY KEY (blend_id, user_id)"
00071         ");";
00072
00073     char* errMsg = nullptr;
00074     int rc = sqlite3_exec(db, sql, nullptr, nullptr, &errMsg);
00075     if (rc != SQLITE_OK) {
00076         std::cerr << "[SqliteDataManager] Schema init failed: " << errMsg << "\n";
00077         sqlite3_free(errMsg);
00078     }
00079
00080     // Migration: add new columns to pre-existing databases that have the old
00081     // user_watch_later schema (SQLite ignores "duplicate column" errors).
00082     const char* migrations[] = {
00083         "ALTER TABLE user_watch_later ADD COLUMN channel_id TEXT;",
00084         "ALTER TABLE user_watch_later ADD COLUMN channel_name TEXT;",
00085         "ALTER TABLE user_watch_later ADD COLUMN channel_logo_url TEXT;",
00086         "ALTER TABLE user_watch_later ADD COLUMN thumbnail_url TEXT;",
00087     };
00088     for (const char* m : migrations) {
00089         sqlite3_exec(db, m, nullptr, nullptr, nullptr); // ignore error if column exists
00090     }
00091 }
00092
00093 // -- DataManager interface -----
00094
00095 bool SqliteDataManager::createUser(const User& user) {
00096     if (!db) return false;
00097
00098     const char* sql =
00099         "INSERT INTO users (user_id, username, email, password) "
00100         "VALUES (?, ?, ?, ?);";
00101
00102     sqlite3_stmt* stmt = nullptr;
00103     if (sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr) != SQLITE_OK) {
00104         std::cerr << "[SqliteDataManager] createUser prepare failed: "
00105             << sqlite3_errmsg(db) << "\n";
00106         return false;
00107     }
00108
00109     sqlite3_bind_text(stmt, 1, user.getUserID().c_str(), -1, SQLITE_TRANSIENT);
00110     sqlite3_bind_text(stmt, 2, user.getUsername().c_str(), -1, SQLITE_TRANSIENT);
00111     sqlite3_bind_text(stmt, 3, user.getEmail().c_str(), -1, SQLITE_TRANSIENT);
00112     sqlite3_bind_text(stmt, 4, user.getPassword().c_str(), -1, SQLITE_TRANSIENT);
00113
00114     int rc = sqlite3_step(stmt);
00115     sqlite3_finalize(stmt);
00116
00117     if (rc != SQLITE_DONE) {

```

```

00118         // SQLITE_CONSTRAINT means the PRIMARY KEY (user_id) already exists
00119         std::cerr << "[SqliteDataManager] createUser failed: "
00120             << sqlite3_errmsg(db) << "\n";
00121         return false;
00122     }
00123     return true;
00124 }
00125
00126 std::optional<User> SqliteDataManager::findUserByID(const std::string& userID) {
00127     if (!db) return std::nullopt;
00128
00129     const char* sql =
00130         "SELECT user_id, username, email, password "
00131         "FROM users WHERE user_id = ?;";
00132
00133     sqlite3_stmt* stmt = nullptr;
00134     if (sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr) != SQLITE_OK) {
00135         std::cerr << "[SqliteDataManager] findUserByID prepare failed: "
00136             << sqlite3_errmsg(db) << "\n";
00137         return std::nullopt;
00138     }
00139
00140     sqlite3_bind_text(stmt, 1, userID.c_str(), -1, SQLITE_TRANSIENT);
00141
00142     std::optional<User> result = std::nullopt;
00143     if (sqlite3_step(stmt) == SQLITE_ROW) {
00144         std::string id = reinterpret_cast<const char*>(sqlite3_column_text(stmt, 0));
00145         std::string uname = reinterpret_cast<const char*>(sqlite3_column_text(stmt, 1));
00146         std::string email = reinterpret_cast<const char*>(sqlite3_column_text(stmt, 2));
00147         std::string password = reinterpret_cast<const char*>(sqlite3_column_text(stmt, 3));
00148         result = User(id, uname, email, password);
00149     }
00150
00151     sqlite3_finalize(stmt);
00152     return result;
00153 }
00154
00155 bool SqliteDataManager::validatePassword(const std::string& userID,
00156     const std::string& password) {
00157     std::optional<User> user = findUserByID(userID);
00158     if (!user.has_value()) return false;
00159     return user->getPassword() == password;
00160 }
00161
00162 // -- Watch Later -----
00163
00164 bool SqliteDataManager::saveWatchLater(const std::string& userID,
00165     const std::list<Video>& videos) {
00166     if (!db) return false;
00167
00168     // Delete existing rows for this user then re-insert - handles re-uploads cleanly.
00169     const char* delSql = "DELETE FROM user_watch_later WHERE user_id = ?;";
00170     sqlite3_stmt* del = nullptr;
00171     if (sqlite3_prepare_v2(db, delSql, -1, &del, nullptr) != SQLITE_OK) {
00172         std::cerr << "[SqliteDataManager] saveWatchLater delete prepare failed: "
00173             << sqlite3_errmsg(db) << "\n";
00174         return false;
00175     }
00176     sqlite3_bind_text(del, 1, userID.c_str(), -1, SQLITE_TRANSIENT);
00177     sqlite3_step(del);
00178     sqlite3_finalize(del);
00179
00180     const char* insSql =
00181         "INSERT OR REPLACE INTO user_watch_later "
00182         "(user_id, video_id, title, channel_id, channel_name, channel_logo_url, thumbnail_url,
00183 position) "
00184         "VALUES (?, ?, ?, ?, ?, ?, ?, ?);";
00185     sqlite3_stmt* ins = nullptr;
00186     if (sqlite3_prepare_v2(db, insSql, -1, &ins, nullptr) != SQLITE_OK) {
00187         std::cerr << "[SqliteDataManager] saveWatchLater insert prepare failed: "
00188             << sqlite3_errmsg(db) << "\n";
00189         return false;
00190     }
00191
00192     int pos = 0;
00193     for (const Video& v : videos) {
00194         sqlite3_bind_text(ins, 1, userID.c_str(), -1, SQLITE_TRANSIENT);
00195         sqlite3_bind_text(ins, 2, v.getVideoID().c_str(), -1, SQLITE_TRANSIENT);
00196         sqlite3_bind_text(ins, 3, v.getTitle().c_str(), -1, SQLITE_TRANSIENT);
00197         sqlite3_bind_text(ins, 4, v.getChannelID().c_str(), -1, SQLITE_TRANSIENT);
00198         sqlite3_bind_text(ins, 5, v.getChannelName().c_str(), -1, SQLITE_TRANSIENT);
00199         sqlite3_bind_text(ins, 6, v.getChannelLogoURL().c_str(), -1, SQLITE_TRANSIENT);
00200         sqlite3_bind_text(ins, 7, v.getThumbnailURL().c_str(), -1, SQLITE_TRANSIENT);
00201         sqlite3_bind_int(ins, 8, pos++);
00202         if (sqlite3_step(ins) != SQLITE_DONE) {
00203             std::cerr << "[SqliteDataManager] saveWatchLater insert failed: "
00204                 << sqlite3_errmsg(db) << "\n";

```

```

00204     }
00205     sqlite3_reset(ins);
00206 }
00207 sqlite3_finalize(ins);
00208 return true;
00209 }
00210
00211 std::list<Video> SqliteDataManager::loadWatchLater(const std::string& userID) {
00212     std::list<Video> result;
00213     if (!db) return result;
00214
00215     const char* sql =
00216         "SELECT video_id, title, channel_id, channel_name, channel_logo_url, thumbnail_url "
00217         "FROM user_watch_later "
00218         "WHERE user_id = ? ORDER BY position ASC;";
00219     sqlite3_stmt* stmt = nullptr;
00220     if (sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr) != SQLITE_OK) {
00221         std::cerr << "[SqliteDataManager] loadWatchLater prepare failed: "
00222             << sqlite3_errmsg(db) << "\n";
00223         return result;
00224     }
00225     sqlite3_bind_text(stmt, 1, userID.c_str(), -1, SQLITE_TRANSIENT);
00226
00227     auto colText = [&](int col) -> std::string {
00228         const unsigned char* p = sqlite3_column_text(stmt, col);
00229         return p ? reinterpret_cast<const char*>(p) : "";
00230     };
00231
00232     while (sqlite3_step(stmt) == SQLITE_ROW) {
00233         result.emplace_back(
00234             colText(0), // videoID
00235             colText(1), // title
00236             colText(2), // channelID
00237             colText(5), // thumbnailURL
00238             0,
00239             std::list<std::string>{},
00240             colText(3), // channelName
00241             colText(4) // channelLogoURL
00242         );
00243     }
00244     sqlite3_finalize(stmt);
00245     return result;
00246 }
00247
00248 // -- Blend persistence -----
00249
00250 bool SqliteDataManager::saveBlend(const std::string& blendID,
00251     const std::string& creatorID,
00252     const std::string& algorithm,
00253     const std::vector<std::string>& participantIDs) {
00254     if (!db) return false;
00255
00256     // ISO timestamp for created_at
00257     std::time_t now = std::time(nullptr);
00258     char timeBuf[32];
00259     std::strftime(timeBuf, sizeof(timeBuf), "%Y-%m-%dT%H:%M:%S", std::gmtime(&now));
00260
00261     // Upsert blend row
00262     const char* blendSql =
00263         "INSERT OR REPLACE INTO blends (blend_id, creator_id, algorithm, created_at) "
00264         "VALUES (?, ?, ?, ?);";
00265     sqlite3_stmt* bs = nullptr;
00266     if (sqlite3_prepare_v2(db, blendSql, -1, &bs, nullptr) != SQLITE_OK) {
00267         std::cerr << "[SqliteDataManager] saveBlend blend prepare failed: "
00268             << sqlite3_errmsg(db) << "\n";
00269         return false;
00270     }
00271     sqlite3_bind_text(bs, 1, blendID.c_str(), -1, SQLITE_TRANSIENT);
00272     sqlite3_bind_text(bs, 2, creatorID.c_str(), -1, SQLITE_TRANSIENT);
00273     sqlite3_bind_text(bs, 3, algorithm.c_str(), -1, SQLITE_TRANSIENT);
00274     sqlite3_bind_text(bs, 4, timeBuf, -1, SQLITE_TRANSIENT);
00275     sqlite3_step(bs);
00276     sqlite3_finalize(bs);
00277
00278     // Replace participant rows - delete first so removed users are cleaned up
00279     const char* delSql = "DELETE FROM blend_participants WHERE blend_id = ?;";
00280     sqlite3_stmt* del = nullptr;
00281     sqlite3_prepare_v2(db, delSql, -1, &del, nullptr);
00282     sqlite3_bind_text(del, 1, blendID.c_str(), -1, SQLITE_TRANSIENT);
00283     sqlite3_step(del);
00284     sqlite3_finalize(del);
00285
00286     const char* partSql =
00287         "INSERT OR IGNORE INTO blend_participants (blend_id, user_id) VALUES (?, ?);";
00288     sqlite3_stmt* ps = nullptr;
00289     if (sqlite3_prepare_v2(db, partSql, -1, &ps, nullptr) != SQLITE_OK) {
00290         std::cerr << "[SqliteDataManager] saveBlend participants prepare failed: "

```

```

00291         « sqlite3_errmsg(db) « "\n";
00292     return false;
00293 }
00294 for (const auto& uid : participantIDs) {
00295     sqlite3_bind_text(ps, 1, blendID.c_str(), -1, SQLITE_TRANSIENT);
00296     sqlite3_bind_text(ps, 2, uid.c_str(), -1, SQLITE_TRANSIENT);
00297     sqlite3_step(ps);
00298     sqlite3_reset(ps);
00299 }
00300 sqlite3_finalize(ps);
00301 return true;
00302 }
00303
00304 std::optional<std::string> SqliteDataManager::findBlendForUser(const std::string& userID) {
00305     if (!db) return std::nullopt;
00306
00307     const char* sql =
00308         "SELECT bp.blend_id FROM blend_participants bp "
00309         "INNER JOIN blends b ON bp.blend_id = b.blend_id "
00310         "WHERE bp.user_id = ? "
00311         "ORDER BY b.created_at DESC LIMIT 1;";
00312     sqlite3_stmt* stmt = nullptr;
00313     if (sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr) != SQLITE_OK) {
00314         std::cerr « "[SqliteDataManager] findBlendForUser prepare failed: "
00315                 « sqlite3_errmsg(db) « "\n";
00316         return std::nullopt;
00317     }
00318     sqlite3_bind_text(stmt, 1, userID.c_str(), -1, SQLITE_TRANSIENT);
00319
00320     std::optional<std::string> result = std::nullopt;
00321     if (sqlite3_step(stmt) == SQLITE_ROW)
00322         result = reinterpret_cast<const char*>(sqlite3_column_text(stmt, 0));
00323     sqlite3_finalize(stmt);
00324     return result;
00325 }
00326
00327 std::vector<std::string> SqliteDataManager::loadBlendParticipants(const std::string& blendID) {
00328     std::vector<std::string> result;
00329     if (!db) return result;
00330
00331     const char* sql =
00332         "SELECT user_id FROM blend_participants WHERE blend_id = ?;";
00333     sqlite3_stmt* stmt = nullptr;
00334     if (sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr) != SQLITE_OK) {
00335         std::cerr « "[SqliteDataManager] loadBlendParticipants prepare failed: "
00336                 « sqlite3_errmsg(db) « "\n";
00337         return result;
00338     }
00339     sqlite3_bind_text(stmt, 1, blendID.c_str(), -1, SQLITE_TRANSIENT);
00340
00341     while (sqlite3_step(stmt) == SQLITE_ROW)
00342         result.emplace_back(reinterpret_cast<const char*>(sqlite3_column_text(stmt, 0)));
00343     sqlite3_finalize(stmt);
00344     return result;
00345 }

```

7.119 SqliteDataManager.hpp File Reference

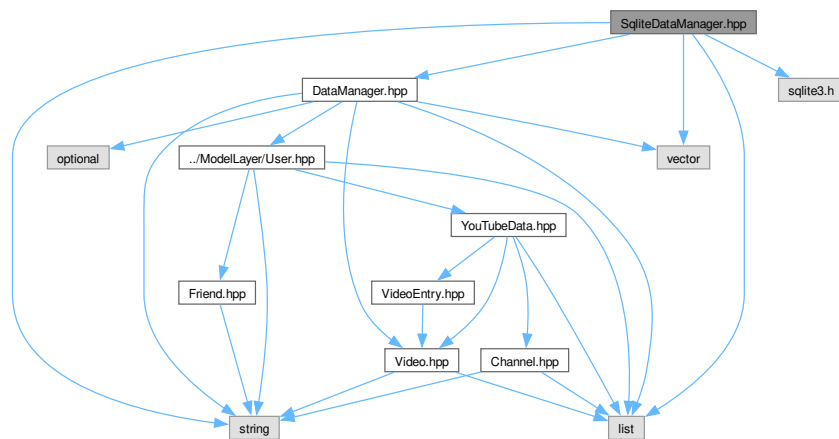
SQLite-backed implementation of [DataManager](#).

```

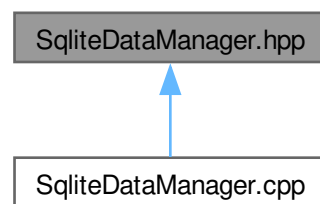
#include "DataManager.hpp"
#include <sqlite3.h>
#include <string>
#include <vector>
#include <list>

```

Include dependency graph for SqliteDataManager.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [SqliteDataManager](#)
SqliteDataManager type declaration.

7.119.1 Detailed Description

SQLite-backed implementation of [DataManager](#).

Author

Jasmine Kumar

Opens (or creates) a .db file on construction and runs the schema migration automatically. Pass ":memory:" as the path to get an in-memory database - useful for unit tests.

Definition in file [SqliteDataManager.hpp](#).

7.120 SqliteDataManager.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef SQLITE_DATA_MANAGER_H
00002 #define SQLITE_DATA_MANAGER_H
00003
00004 #include "DataManager.hpp"
00005 #include <sqlite3.h>
00006 #include <string>
00007 #include <vector>
00008 #include <list>
00009
00023 class SqliteDataManager : public DataManager {
00024 public:
00025     explicit SqliteDataManager(const std::string& dbPath);
00026     ~SqliteDataManager() override;
00027
00028     bool createUser(const User& user) override;
00029     std::optional<User> findUserByID(const std::string& userID) override;
00030     bool validatePassword(const std::string& userID,
00031                          const std::string& password) override;
00032
00033     bool saveWatchLater(const std::string& userID,
00034                        const std::list<Video>& videos) override;
00035     std::list<Video> loadWatchLater(const std::string& userID) override;
00036
00037     bool saveBlend(const std::string& blendID,
00038                  const std::string& creatorID,
00039                  const std::string& algorithm,
00040                  const std::vector<std::string>& participantIDs) override;
00041     std::optional<std::string> findBlendForUser(
00042         const std::string& userID) override;
00043     std::vector<std::string> loadBlendParticipants(
00044         const std::string& blendID) override;
00045
00046 private:
00047     sqlite3* db = nullptr;
00048
00049     // Runs CREATE TABLE IF NOT EXISTS for all tables on startup.
00050     void initSchema();
00051 };
00052
00053 #endif

```

7.121 YouTubeMetadataFetcher.cpp File Reference

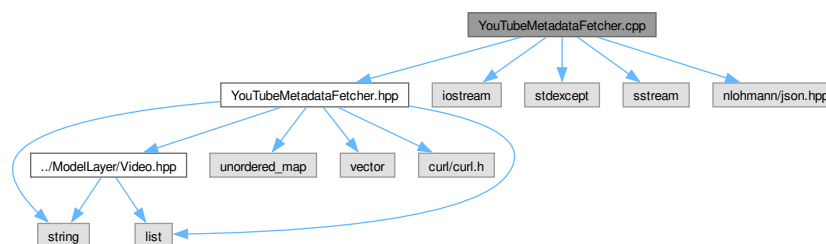
Implementation for [YouTubeMetadataFetcher](#).

```

#include "YouTubeMetadataFetcher.hpp"
#include <iostream>
#include <stdexcept>
#include <sstream>
#include <nlohmann/json.hpp>

```

Include dependency graph for YouTubeMetadataFetcher.cpp:



Typedefs

- using `json` = `nlohmann::json`

7.121.1 Detailed Description

Implementation for [YouTubeMetadataFetcher](#).

Author

Jasmine Kumar

Definition in file [YouTubeMetadataFetcher.cpp](#).

7.121.2 Typedef Documentation

7.121.2.1 json

```
using json = nlohmann::json
```

Definition at line 19 of file [YouTubeMetadataFetcher.cpp](#).

7.122 YouTubeMetadataFetcher.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 // =====
00008 // YouTubeMetadataFetcher.cpp
00009 // =====
00010
00011 #include "YouTubeMetadataFetcher.hpp"
00012
00013 #include <iostream>
00014 #include <stdexcept>
00015 #include <sstream>
00016
00017 #include <nlohmann/json.hpp>
00018
00019 using json = nlohmann::json;
00020
00021 // -- Constructor -----
00022
00023 YouTubeMetadataFetcher::YouTubeMetadataFetcher(std::string apiKey)
00024     : apiKey(std::move(apiKey)) {}
00025
00026 // -- Public: enrich -----
00027
00028 std::list<Video> YouTubeMetadataFetcher::enrich(const std::list<Video>& videos) {
00029     if (apiKey.empty()) {
00030         std::cerr << "[YouTubeMetadataFetcher] No API key - returning videos unchanged\n";
00031         return videos;
00032     }
00033
00034     // Collect all video IDs in order
00035     std::vector<std::string> allIds;
00036     allIds.reserve(videos.size());
00037     for (const auto& v : videos) allIds.push_back(v.getVideoID());
00038
00039     // Fetch video metadata in batches of kBatchSize
00040     std::unordered_map<std::string, VideoMeta> metaMap;
00041     for (int i = 0; i < static_cast<int>(allIds.size()); i += kBatchSize) {
00042         int end = std::min(i + kBatchSize, static_cast<int>(allIds.size()));
```

```

00043         std::vector<std::string> batch(allIds.begin() + i, allIds.begin() + end);
00044     try {
00045         auto batchResult = fetchVideoBatch(batch);
00046         metaMap.insert(batchResult.begin(), batchResult.end());
00047     } catch (const std::exception& ex) {
00048         std::cerr << "[YouTubeMetadataFetcher] fetchVideoBatch failed: "
00049             << ex.what() << "\n";
00050     }
00051 }
00052
00053 // Collect unique channel IDs from the fetched metadata
00054 std::vector<std::string> channelIdList;
00055 {
00056     std::unordered_map<std::string, bool> seen;
00057     for (const auto& [vid, meta] : metaMap) {
00058         if (!meta.channelId.empty() && seen.find(meta.channelId) == seen.end()) {
00059             channelIdList.push_back(meta.channelId);
00060             seen[meta.channelId] = true;
00061         }
00062     }
00063 }
00064
00065 // Fetch channel logos in batches of kBatchSize
00066 std::unordered_map<std::string, std::string> logoMap;
00067 for (int i = 0; i < static_cast<int>(channelIdList.size()); i += kBatchSize) {
00068     int end = std::min(i + kBatchSize, static_cast<int>(channelIdList.size()));
00069     std::vector<std::string> batch(channelIdList.begin() + i,
00070                                     channelIdList.begin() + end);
00071     try {
00072         auto batchLogos = fetchChannelLogos(batch);
00073         logoMap.insert(batchLogos.begin(), batchLogos.end());
00074     } catch (const std::exception& ex) {
00075         std::cerr << "[YouTubeMetadataFetcher] fetchChannelLogos failed: "
00076             << ex.what() << "\n";
00077     }
00078 }
00079
00080 // Rebuild the video list with enriched data
00081 std::list<Video> enriched;
00082 for (const auto& v : videos) {
00083     auto it = metaMap.find(v.getVideoID());
00084     if (it == metaMap.end()) {
00085         // API didn't return this video (private/deleted) - keep as-is
00086         enriched.push_back(v);
00087         continue;
00088     }
00089     const VideoMeta& m = it->second;
00090
00091     std::string logoUrl;
00092     auto logoIt = logoMap.find(m.channelId);
00093     if (logoIt != logoMap.end()) logoUrl = logoIt->second;
00094
00095     enriched.emplace_back(
00096         v.getVideoID(),
00097         m.title,
00098         m.channelId,
00099         m.thumbnailUrl,
00100         v.getDuration(),
00101         v.getTags(),
00102         m.channelName,
00103         logoUrl
00104     );
00105 }
00106 return enriched;
00107 }
00108
00109 // -- Private: fetchVideoBatch -----
00110
00111 std::unordered_map<std::string, YouTubeMetadataFetcher::VideoMeta>
00112 YouTubeMetadataFetcher::fetchVideoBatch(const std::vector<std::string>& ids) {
00113     // Build comma-separated ID list
00114     std::ostringstream idParam;
00115     for (size_t i = 0; i < ids.size(); ++i) {
00116         if (i) idParam << ',';
00117         idParam << ids[i];
00118     }
00119
00120     std::string url =
00121         "https://www.googleapis.com/youtube/v3/videos"
00122         "?part=snippet"
00123         "&id=" + idParam.str() +
00124         "&key=" + apiKey;
00125
00126     std::string body = httpGet(url);
00127
00128     std::unordered_map<std::string, VideoMeta> result;
00129     try {

```

```

00130     auto root = json::parse(body);
00131     for (const auto& item : root.value("items", json::array())) {
00132         std::string videoId = item.value("id", "");
00133         if (videoId.empty()) continue;
00134
00135         const auto& snippet = item.value("snippet", json::object());
00136
00137         VideoMeta m;
00138         m.title = snippet.value("title", "");
00139         m.channelId = snippet.value("channelId", "");
00140         m.channelName = snippet.value("channelTitle", "");
00141
00142         // Prefer highest-resolution thumbnail available
00143         const auto& thumbs = snippet.value("thumbnails", json::object());
00144         for (const char* key : {"maxres", "standard", "high", "medium", "default"}) {
00145             if (thumbs.contains(key)) {
00146                 m.thumbnailUrl = thumbs[key].value("url", "");
00147                 if (!m.thumbnailUrl.empty()) break;
00148             }
00149         }
00150
00151         result[videoId] = std::move(m);
00152     }
00153 } catch (const json::parse_error& ex) {
00154     throw std::runtime_error(
00155         std::string("YouTubeMetadataFetcher: videos JSON parse error: ") + ex.what());
00156 }
00157 return result;
00158 }
00159
00160 // -- Private: fetchChannelLogos -----
00161
00162 std::unordered_map<std::string, std::string>
00163 YouTubeMetadataFetcher::fetchChannelLogos(const std::vector<std::string>& channelIds) {
00164     std::ostringstream idParam;
00165     for (size_t i = 0; i < channelIds.size(); ++i) {
00166         if (i) idParam << ',';
00167         idParam << channelIds[i];
00168     }
00169
00170     std::string url =
00171         "https://www.googleapis.com/youtube/v3/channels"
00172         "?part=snippet"
00173         "&id=" + idParam.str() +
00174         "&key=" + apiKey;
00175
00176     std::string body = httpGet(url);
00177
00178     std::unordered_map<std::string, std::string> result;
00179     try {
00180         auto root = json::parse(body);
00181         for (const auto& item : root.value("items", json::array())) {
00182             std::string channelId = item.value("id", "");
00183             if (channelId.empty()) continue;
00184
00185             const auto& snippet = item.value("snippet", json::object());
00186             const auto& thumbs = snippet.value("thumbnails", json::object());
00187
00188             // Channel logos: prefer medium (240x240) over default (88x88)
00189             std::string logoUrl;
00190             for (const char* key : {"medium", "default"}) {
00191                 if (thumbs.contains(key)) {
00192                     logoUrl = thumbs[key].value("url", "");
00193                     if (!logoUrl.empty()) break;
00194                 }
00195             }
00196             if (!logoUrl.empty()) result[channelId] = std::move(logoUrl);
00197         }
00198     } catch (const json::parse_error& ex) {
00199         throw std::runtime_error(
00200             std::string("YouTubeMetadataFetcher: channels JSON parse error: ") + ex.what());
00201     }
00202     return result;
00203 }
00204
00205 // -- Private: httpGet -----
00206
00207 std::string YouTubeMetadataFetcher::httpGet(const std::string& url) {
00208     CURL* curl = curl_easy_init();
00209     if (!curl) throw std::runtime_error("YouTubeMetadataFetcher: curl_easy_init failed");
00210
00211     std::string response;
00212     curl_easy_setopt(curl, CURLOPT_URL, url.c_str());
00213     curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, writeCallback);
00214     curl_easy_setopt(curl, CURLOPT_WRITEDATA, &response);
00215     curl_easy_setopt(curl, CURLOPT_FOLLOWLOCATION, 1L);
00216     curl_easy_setopt(curl, CURLOPT_HTTPGET, 1L);

```

```

00217 // Aggressive timeouts to prevent hanging on slow/unavailable API endpoints.
00218 // 3s connect timeout ensures socket is reaching the server quickly.
00219 // 10s total timeout prevents long waits for slow API responses.
00220 curl_easy_setopt(curl, CURLOPT_CONNECTTIMEOUT, 3L);
00221 curl_easy_setopt(curl, CURLOPT_TIMEOUT, 10L);
00222
00223 CURLcode res = curl_easy_perform(curl);
00224 curl_easy_cleanup(curl);
00225
00226 if (res != CURLE_OK) {
00227     throw std::runtime_error(
00228         std::string("YouTubeMetadataFetcher: HTTP GET failed: ") +
00229         curl_easy_strerror(res));
00230 }
00231 return response;
00232 }
00233
00234 size_t YouTubeMetadataFetcher::writeCallback(void* contents, size_t size,
00235                                             size_t nmemb, std::string* out) {
00236     out->append(static_cast<char*>(contents), size * nmemb);
00237     return size * nmemb;
00238 }

```

7.123 YouTubeMetadataFetcher.hpp File Reference

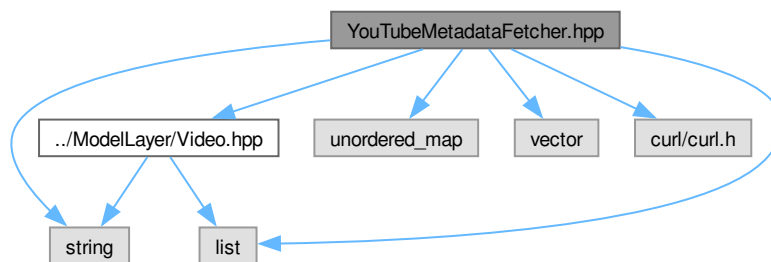
Enriches a list of [Video](#) objects (video-ID only) with full snippet metadata from the YouTube Data API v3.

```

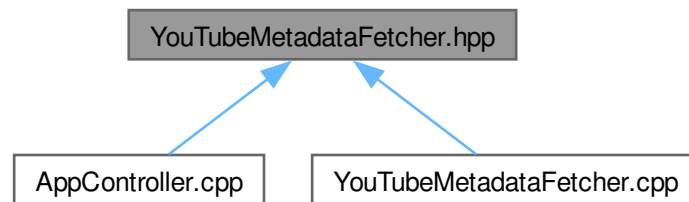
#include <string>
#include <list>
#include <unordered_map>
#include <vector>
#include <curl/curl.h>
#include "../ModelLayer/Video.hpp"

```

Include dependency graph for YouTubeMetadataFetcher.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [YouTubeMetadataFetcher](#)
YouTubeMetadataFetcher class declaration.
- struct [YouTubeMetadataFetcher::VideoMeta](#)
VideoMeta class declaration.

7.123.1 Detailed Description

Enriches a list of [Video](#) objects (video-ID only) with full snippet metadata from the YouTube Data API v3.

Author

Jasmine Kumar

videos.list?part=snippet&id=<ids>&key=<apiKey> -> title, channelId, channelTitle, thumbnailUrl channels.↵
list?part=snippet&id=<channelIds>&key=<apiKey> -> channel logo URL (snippet.thumbnails.default.url)

IDs are batched at most 50 per request (YouTube API limit). Videos not returned by the API (private, deleted) are kept unchanged. Any network or parse error is logged to stderr; the remaining videos are still returned with whatever data was successfully retrieved.

Definition in file [YouTubeMetadataFetcher.hpp](#).

7.124 YouTubeMetadataFetcher.hpp

[Go to the documentation of this file.](#)

```

00001 // =====
00002 // YouTubeMetadataFetcher.hpp
00003 //
00004 // Enriches a list of Video objects (video-ID only) with full snippet metadata
00005 // from the YouTube Data API v3:
00006 //
00007 //   videos.list?part=snippet&id=<ids>&key=<apiKey>
00008 //       -> title, channelId, channelTitle, thumbnailUrl
00009 //
00010 //   channels.list?part=snippet&id=<channelIds>&key=<apiKey>
00011 //       -> channel logo URL (snippet.thumbnails.default.url)
  
```

```

00012 //
00013 // IDs are batched at most 50 per request (YouTube API limit).
00014 // Videos not returned by the API (private, deleted) are kept unchanged.
00015 // Any network or parse error is logged to stderr; the remaining videos are
00016 // still returned with whatever data was successfully retrieved.
00017 // =====
00018
00019 #pragma once
00020
00021 #include <string>
00022 #include <list>
00023 #include <unordered_map>
00024 #include <vector>
00025
00026 #include <curl/curl.h>
00027
00028 #include "../ModelLayer/Video.hpp"
00029
00045
00050 class YouTubeMetadataFetcher {
00051 public:
00052     explicit YouTubeMetadataFetcher(std::string apiKey);
00053
00059     std::list<Video> enrich(const std::list<Video>& videos);
00060
00061 private:
00062     std::string apiKey;
00063
00064     static constexpr int kBatchSize = 50;
00065
00070     struct VideoMeta {
00071         std::string title;
00072         std::string channelId;
00073         std::string channelName;
00074         std::string thumbnailUrl;
00075     };
00076
00077     // Fetches snippet data for up to kBatchSize video IDs.
00078     // Returns a map of videoId -> VideoMeta.
00079     std::unordered_map<std::string, VideoMeta>
00080     fetchVideoBatch(const std::vector<std::string>& ids);
00081
00082     // Fetches the default thumbnail URL for each channel ID.
00083     // Returns a map of channelId -> logoUrl.
00084     std::unordered_map<std::string, std::string>
00085     fetchChannelLogos(const std::vector<std::string>& channelIds);
00086
00087     // Performs a GET request and returns the response body.
00088     // Throws std::runtime_error on curl failure.
00089     std::string httpGet(const std::string& url);
00090
00091     static size_t writeCallback(void* contents, size_t size, size_t nmemb,
00092                                std::string* out);
00093 };

```

7.125 ActiveBlendsPanel.cpp File Reference

Implementation for [ActiveBlendsPanel](#).

```

#include "ActiveBlendsPanel.hpp"
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include "TopBar.hpp"
#include "ConfirmationDialog.hpp"
#include "OperationProgressDialog.hpp"
#include "../AppLayer/AppController.hpp"
#include "../AppLayer/AppState.hpp"
#include "../AppLayer/EventRouter.hpp"

```



```

00046     m_listScroll->SetSizer(scrollSizer);
00047
00048     root->Add(m_listScroll, 1, wxEXPAND | wxALL, 8);
00049
00050     // Empty state label
00051     m_emptyLabel = new wxStaticText(m_listInner, wxID_ANY,
00052                                     "No active blends.",
00053                                     wxDefaultPosition, wxDefaultSize,
00054                                     wxALIGN_CENTER_HORIZONTAL);
00055     m_emptyLabel->SetForegroundColour(UIColors::TextMuted);
00056     m_listSizer->Add(m_emptyLabel, 0, wxALIGN_CENTER_HORIZONTAL | wxTOP, 24);
00057
00058     SetSizer(root);
00059 }
00060
00061 void ActiveBlendsPanel::Refresh()
00062 {
00063     RebuildList();
00064 }
00065
00066 void ActiveBlendsPanel::RebuildList()
00067 {
00068     m_listSizer->Clear(true);
00069
00070     // Re-create empty label (cleared above)
00071     m_emptyLabel = new wxStaticText(m_listInner, wxID_ANY,
00072                                     "No active blends.",
00073                                     wxDefaultPosition, wxDefaultSize,
00074                                     wxALIGN_CENTER_HORIZONTAL);
00075     m_emptyLabel->SetForegroundColour(UIColors::TextMuted);
00076
00077     auto blends = m_controller.fetchUserBlends();
00078
00079     if (blends.empty()) {
00080         m_emptyLabel->Show(true);
00081         m_listSizer->Add(m_emptyLabel, 0, wxALIGN_CENTER_HORIZONTAL | wxTOP, 24);
00082     } else {
00083         m_emptyLabel->Show(false);
00084         m_listSizer->Add(m_emptyLabel, 0);
00085
00086         for (const auto& blend : blends) {
00087             auto* row = new wxPanel(m_listInner, wxID_ANY);
00088             row->SetBackgroundColour(UIColors::SurfaceRaised);
00089             row->SetMinSize(wxSize(-1, 52));
00090
00091             auto* rowSizer = new wxBoxSizer(wxHORIZONTAL);
00092
00093             // Blend title label
00094             auto* titleLabel = new wxStaticText(row, wxID_ANY,
00095                                                 wxString::FromUTF8(blend.title));
00096             titleLabel->SetForegroundColour(UIColors::TextPrimary);
00097             wxFont f = titleLabel->GetFont();
00098             f.SetPointSize(12);
00099             f.SetWeight(wxFONTWEIGHT_BOLD);
00100             titleLabel->SetFont(f);
00101
00102             // Open button
00103             auto* openBtn = new wxButton(row, wxID_ANY, "Open");
00104             UIButtons::ApplySizeBounds(openBtn, ButtonType::Compact);
00105             openBtn->SetBackgroundColour(UIColors::Accent);
00106             openBtn->SetForegroundColour(UIColors::TextPrimary);
00107
00108             // Leave button
00109             auto* leaveBtn = new wxButton(row, wxID_ANY, "Leave");
00110             UIButtons::ApplySizeBounds(leaveBtn, ButtonType::Compact);
00111             leaveBtn->SetBackgroundColour(UIColors::Danger);
00112             leaveBtn->SetForegroundColour(UIColors::TextPrimary);
00113
00114             std::string capID = blend.blendID;
00115             std::string capTitle = blend.title;
00116             openBtn->Bind(wxEVT_BUTTON, [this, capID, capTitle](wxCommandEvent&) {
00117                 OnSelectBlend(capID, capTitle);
00118             });
00119             leaveBtn->Bind(wxEVT_BUTTON, [this, capID, capTitle](wxCommandEvent&) {
00120                 OnLeaveBlend(capID, capTitle);
00121             });
00122
00123             rowSizer->Add(titleLabel, 1, wxALIGN_CENTER_VERTICAL | wxLEFT, 16);
00124             rowSizer->Add(openBtn, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 8);
00125             rowSizer->Add(leaveBtn, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 16);
00126             row->SetSizer(rowSizer);
00127
00128             m_listSizer->Add(row, 0, wxEXPAND | wxLEFT | wxRIGHT | wxTOP, 6);
00129         }
00130     }
00131
00132     m_listInner->Layout();

```



```

00133     m_listScroll->FitInside();
00134     m_listScroll->Layout();
00135     Layout();
00136 }
00137
00138 void ActiveBlendsPanel::OnSelectBlend(const std::string& blendID,
00139                                     const std::string& title)
00140 {
00141     OperationProgressDialog progress(this, "Loading Blend");
00142     progress.ShowModal();
00143     progress.UpdateProgress(0.02, "Preparing...");
00144
00145     // Wire up the progress reporter so AppController updates the bar as it works
00146     m_controller.setProgressReporter([&progress](double ratio, const std::string& message) {
00147         progress.UpdateProgress(ratio, wxString::FromUTF8(message));
00148     });
00149
00150     m_controller.getEventRouter().dispatch("selectBlend",
00151     {"blendID", blendID}, {"blendTitle", title});
00152     m_controller.clearProgressReporter();
00153
00154     progress.UpdateProgress(1.0, "Blend ready");
00155     progress.Destroy();
00156     m_nav(Page::HOME);
00157 }
00158
00159 void ActiveBlendsPanel::OnLeaveBlend(const std::string& blendID,
00160                                    const std::string& title)
00161 {
00162     bool confirmed = ConfirmationDialog::Confirm(
00163         this, "Leave Blend",
00164         "Are you sure you want to leave \"" + title + "\"?");
00165     if (!confirmed) return;
00166
00167     m_controller.getEventRouter().dispatch("leaveBlend",
00168     {"blendID", blendID}, {"blendTitle", title});
00169
00170     RebuildList();
00171 }

```

7.127 ActiveBlendsPanel.hpp File Reference

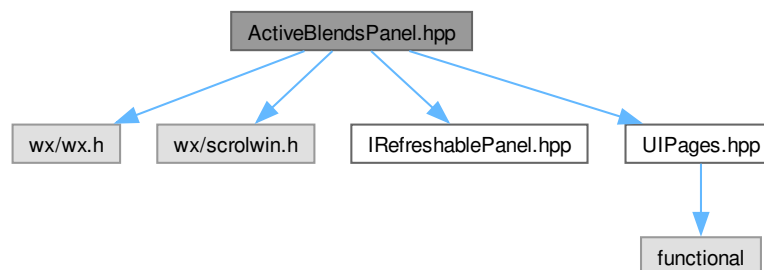
Panel for viewing and managing active blends.

```

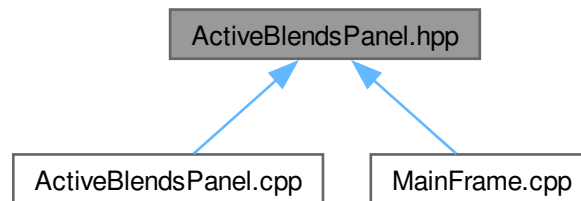
#include <wx/wx.h>
#include <wx/scrolwin.h>
#include "IRefreshablePanel.hpp"
#include "UIPages.hpp"

```

Include dependency graph for ActiveBlendsPanel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [ActiveBlendsPanel](#)
ActiveBlendsPanel class declaration.

7.127.1 Detailed Description

Panel for viewing and managing active blends.

Author

Jasmine Kumar

Definition in file [ActiveBlendsPanel.hpp](#).

7.128 ActiveBlendsPanel.hpp

[Go to the documentation of this file.](#)

```

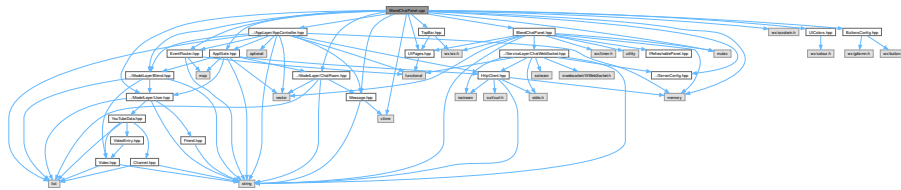
00001
00006
00007 #pragma once
00008
00009 #include <wx/wx.h>
00010 #include <wx/scrolwin.h>
00011
00012 #include "IRefreshablePanel.hpp"
00013 #include "UIPages.hpp"
00014
00015 class AppController;
00016
00021 class ActiveBlendsPanel : public wxPanel, public IRefreshablePanel {
00022 public:
00023     ActiveBlendsPanel(wxWindow* parent, AppController& controller, NavigateFn nav);
00024     void Refresh() override;
00025
00026 private:
00027     AppController& m_controller;
00028     NavigateFn     m_nav;
00029
00030     wxScrolledWindow* m_listScroll;
00031     wxPanel*          m_listInner;
00032     wxBoxSizer*       m_listSizer;
00033     wxStaticText*     m_emptyLabel;
00034
00035     void RebuildList();
00036     void OnSelectBlend(const std::string& blendID, const std::string& title);
00037     void OnLeaveBlend(const std::string& blendID, const std::string& title);
00038 };
  
```

7.129 BlendChatPanel.cpp File Reference

Implementation for BlendChatPanel.

```
#include "BlendChatPanel.hpp"
#include "../ServerConfig.hpp"
#include <wx/scrolwin.h>
#include <ctime>
#include <memory>
#include <mutex>
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include "UIPages.hpp"
#include "TopBar.hpp"
#include "../AppLayer/AppController.hpp"
#include "../AppLayer/AppState.hpp"
#include "../AppLayer/EventRouter.hpp"
#include "../ModelLayer/ChatRoom.hpp"
#include "../ModelLayer/Message.hpp"
#include "../ModelLayer/Blend.hpp"
#include "../ModelLayer/User.hpp"
```

Include dependency graph for BlendChatPanel.cpp:



Functions

- static wxString **FormatTimestamp** (std::time_t ts)

7.129.1 Detailed Description

Implementation for [BlendChatPanel](#).

Author

Jasmine Kumar

Definition in file [BlendChatPanel.cpp](#).

7.129.2 Function Documentation

7.129.2.1 FormatTimestamp()

```
wxString FormatTimestamp (
    std::time_t ts) [static]
```

Definition at line 33 of file BlendChatPanel.cpp.

7.130 BlendChatPanel.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 // =====
00008 // BlendChatPanel.cpp - Blend chat screen implementation
00009 // =====
00010
00011 #include "BlendChatPanel.hpp"
00012
00013 #include "../ServerConfig.hpp"
00014 #include <wx/scrolwin.h>
00015 #include <ctime>
00016 #include <memory>
00017 #include <mutex>
00018
00019 #include "UIColors.hpp"
00020 #include "ButtonsConfig.hpp"
00021 #include "UIPages.hpp"
00022 #include "TopBar.hpp"
00023 #include "../AppLayer/AppController.hpp"
00024 #include "../AppLayer/AppState.hpp"
00025 #include "../AppLayer/EventRouter.hpp"
00026 #include "../ModelLayer/ChatRoom.hpp"
00027 #include "../ModelLayer/Message.hpp"
00028 #include "../ModelLayer/Blend.hpp"
00029 #include "../ModelLayer/User.hpp"
00030
00031 // -- Helper: format a Unix timestamp as HH:MM -----
00032
00033 static wxString FormatTimestamp(std::time_t ts)
00034 {
00035     std::tm* tm = std::localtime(&ts);
00036     if (!tm) return "??:??";
00037     char buf[9]; // "12:34 PM\0"
00038     std::strftime(buf, sizeof(buf), "%I:%M %p", tm);
00039     return wxString::FromAscii(buf);
00040 }
00041
00042 // -- Construction -----
00043
00044 BlendChatPanel::BlendChatPanel(wxWindow* parent,
00045                                AppController& controller,
00046                                NavigateFn nav)
00047 : wxPanel(parent, wxID_ANY)
00048 , m_controller(controller)
00049 , m_nav(std::move(nav))
00050 {
00051     SetBackgroundColour(UIColors::Background);
00052
00053     auto* root = new wxBoxSizer(wxVERTICAL);
00054
00055     // -- TopBar (title updated in Reload) -----
00056     auto* topBar = new TopBar(this, "Blend Chat", m_nav, Page::HOME);
00057     root->Add(topBar, 0, wxEXPAND);
00058
00059     // -- "No active blend." placeholder -----
00060     m_noBlendLabel = new wxStaticText(this, wxID_ANY, "No active blend.",
00061                                     wxDefaultPosition, wxDefaultSize,
00062                                     wxALIGN_CENTER_HORIZONTAL);
00063     m_noBlendLabel->SetForegroundColour(UIColors::TextMuted);
00064     root->Add(m_noBlendLabel, 1, wxALIGN_CENTER | wxALL, 20);
00065
00066     // -- Participants toggle button + dropdown -----
00067     m_participantsBtn = new wxButton(this, wxID_ANY, "Participants",
00068                                     wxDefaultPosition, wxSize(-1, 48),
00069                                     wxBU_LEFT);
00070     m_participantsBtn->SetBackgroundColour(UIColors::Accent);
00071     m_participantsBtn->SetForegroundColour(*wxWHITE);
00072     {
00073         wxFont f = m_participantsBtn->GetFont();
00074         f.SetPointSize(14);
00075         f.SetWeight(wxFONTWEIGHT_BOLD);
00076         m_participantsBtn->SetFont(f);
00077     }
00078     m_participantsBtn->Hide(); // shown in Reload() when blend is active
00079     root->Add(m_participantsBtn, 0, wxEXPAND);
00080
00081     m_participantsDropdown = new wxPanel(this, wxID_ANY);
00082     m_participantsDropdown->SetBackgroundColour(UIColors::SurfaceRaised);
00083     m_participantsSizer = new wxBoxSizer(wxVERTICAL);
00084     m_participantsDropdown->SetSizer(m_participantsSizer);
00085     m_participantsDropdown->Hide();
00086     root->Add(m_participantsDropdown, 0, wxEXPAND);

```

```

00087
00088     m_participantsBtn->Bind(wxEVT_BUTTON, &BlendChatPanel::OnParticipantsToggle, this);
00089
00090     // -- Message scroll area -----
00091     m_msgScroll = new wxScrolledWindow(this, wxID_ANY,
00092                                     wxDefaultPosition, wxDefaultSize,
00093                                     wxVSCROLL | wxBORDER_NONE);
00094     m_msgScroll->SetBackgroundColour(UIColors::Surface);
00095     m_msgScroll->SetScrollRate(0, 12);
00096
00097     m_msgInner = new wxPanel(m_msgScroll, wxID_ANY);
00098     m_msgInner->SetBackgroundColour(UIColors::Surface);
00099     m_msgSizer = new wxBoxSizer(wxVERTICAL);
00100     m_msgInner->SetSizer(m_msgSizer);
00101
00102     auto* msgScrollSizer = new wxBoxSizer(wxVERTICAL);
00103     msgScrollSizer->Add(m_msgInner, 1, wxEXPAND);
00104     m_msgScroll->SetSizer(msgScrollSizer);
00105
00106     root->Add(m_msgScroll, 1, wxEXPAND);
00107
00108     // -- Input panel -----
00109     m_inputPanel = new wxPanel(this, wxID_ANY);
00110     m_inputPanel->SetBackgroundColour(UIColors::Surface);
00111     m_inputPanel->SetMinSize(wxSize(-1, 60));
00112
00113     auto* inputSizer = new wxBoxSizer(wxHORIZONTAL);
00114
00115     m_inputCtrl = new wxTextCtrl(m_inputPanel, wxID_ANY, wxEmptyString,
00116                                wxDefaultPosition, wxDefaultSize,
00117                                wxTE_PROCESS_ENTER | wxBORDER_SIMPLE);
00118     m_inputCtrl->SetBackgroundColour(UIColors::SurfaceRaised);
00119     m_inputCtrl->SetForegroundColour(UIColors::TextPrimary);
00120     m_inputCtrl->SetHint("Type a message...");
00121
00122     auto* sendBtn = new wxButton(m_inputPanel, wxID_ANY, "Send");
00123     UIButtons::ApplySizeBounds(sendBtn, ButtonType::Nav);
00124     sendBtn->SetBackgroundColour(UIColors::Accent);
00125     sendBtn->SetForegroundColour(UIColors::TextPrimary);
00126
00127     inputSizer->Add(m_inputCtrl, 1, wxALIGN_CENTER_VERTICAL | wxLEFT | wxTOP | wxBOTTOM, 10);
00128     inputSizer->Add(sendBtn, 0, wxALIGN_CENTER_VERTICAL | wxALL, 10);
00129     m_inputPanel->SetSizer(inputSizer);
00130
00131     root->Add(m_inputPanel, 0, wxEXPAND);
00132
00133     SetSizer(root);
00134
00135     // -- Bindings -----
00136     sendBtn->Bind(wxEVT_BUTTON, &BlendChatPanel::OnSend, this);
00137     m_inputCtrl->Bind(wxEVT_TEXT_ENTER, &BlendChatPanel::OnInputEnter, this);
00138
00139     // -- Poll timer - drains incoming WebSocket messages on the main thread ----
00140     m_pollTimer = new wxTimer(this);
00141     Bind(wxEVT_TIMER, &BlendChatPanel::OnPollTimer, this, m_pollTimer->GetId());
00142     m_pollTimer->Start(500 /* ms */);
00143
00144     // -- Logout listener - close WebSocket immediately when user logs out -----
00145     m_controller.getEventRouter().registerListener("logout",
00146         [this](const EventPayload&) {
00147             m_ws.reset();
00148             m_wsBlendID.clear();
00149         });
00150 }
00151
00152 // -- Public -----
00153
00154 void BlendChatPanel::Refresh()
00155 {
00156     Reload();
00157 }
00158
00159 void BlendChatPanel::Reload()
00160 {
00161     AppState& state = AppState::getInstance();
00162     Blend* blend = state.getActiveBlend();
00163     ChatRoom* room = state.getActiveChatRoom();
00164
00165     bool hasBlend = (blend != nullptr && room != nullptr);
00166
00167     m_noBlendLabel ->Show(!hasBlend);
00168     m_participantsBtn ->Show(hasBlend);
00169     m_msgScroll ->Show(hasBlend);
00170     m_inputPanel ->Show(hasBlend);
00171
00172     if (!hasBlend) {
00173         m_participantsDropdown->Hide();

```

```

00174         m_ws.reset();
00175         m_wsBlendID.clear();
00176         Layout();
00177         return;
00178     }
00179
00180     // -- Connect WebSocket if blend changed or not yet connected -----
00181     User* currentUser = state.getCurrentUser();
00182     if (currentUser && room->getBlendID() != m_wsBlendID) {
00183         // Load history first (synchronous - ChatRoom is populated before UI rebuild)
00184         m_controller.getEventRouter().dispatch("openChat",
00185             {"blendID", room->getBlendID()},
00186             {"userID", currentUser->getUserID()});
00187         ConnectWebSocket(room->getBlendID(), currentUser->getUserID());
00188     }
00189
00190     // -- Populate participants dropdown -----
00191     m_participantsBtn->Show();
00192     m_participantsSizer->Clear(true /* delete_windows */);
00193
00194     for (const User& p : blend->getParticipants()) {
00195         auto* row = new wxPanel(m_participantsDropdown, wxID_ANY);
00196         row->SetBackgroundColour(UIColors::SurfaceRaised);
00197
00198         // Username - bold, slightly larger
00199         auto* nameLabel = new wxStaticText(row, wxID_ANY,
00200             wxString::FromUTF8(p.getUsername()));
00201         nameLabel->SetForegroundColour(UIColors::TextPrimary);
00202         {
00203             wxFont f = nameLabel->GetFont();
00204             f.SetPointSize(11);
00205             f.SetWeight(wxFONTWEIGHT_BOLD);
00206             nameLabel->SetFont(f);
00207         }
00208
00209         // User ID - normal weight, smaller
00210         auto* idLbl = new wxStaticText(row, wxID_ANY,
00211             wxString::FromUTF8(" (" + p.getUserID() + ")"));
00212         idLbl->SetForegroundColour(UIColors::TextSecondary);
00213         {
00214             wxFont f = idLbl->GetFont();
00215             f.SetPointSize(9);
00216             idLbl->SetFont(f);
00217         }
00218
00219         auto* rowSizer = new wxBoxSizer(wxHORIZONTAL);
00220         rowSizer->Add(nameLabel, 0, wxALIGN_CENTER_VERTICAL | wxTOP | wxBOTTOM | wxLEFT, 8);
00221         rowSizer->Add(idLbl, 0, wxALIGN_CENTER_VERTICAL | wxTOP | wxBOTTOM, 8);
00222         row->SetSizer(rowSizer);
00223
00224         m_participantsSizer->Add(row, 0, wxEXPAND | wxBOTTOM, 1);
00225     }
00226
00227     m_participantsDropdown->Layout();
00228
00229     // -- Rebuild message list -----
00230     m_msgSizer->Clear(true /* delete_windows */);
00231
00232     const std::list<Message>& msgs = room->getMessages();
00233
00234     if (msgs.empty()) {
00235         auto* placeholder = new wxStaticText(m_msgInner, wxID_ANY,
00236             "No messages yet.",
00237             wxDefaultPosition, wxDefaultSize,
00238             wxALIGN_CENTER_HORIZONTAL);
00239         placeholder->SetForegroundColour(UIColors::TextMuted);
00240         m_msgSizer->Add(placeholder, 0, wxALIGN_CENTER_HORIZONTAL | wxTOP, 20);
00241     } else {
00242         static const wxColour kHoverBg(50, 50, 50);
00243
00244         for (const Message& msg : msgs) {
00245             // Message row container
00246             auto* rowPanel = new wxPanel(m_msgInner, wxID_ANY);
00247             rowPanel->SetBackgroundColour(UIColors::Surface);
00248
00249             auto* rowSizer = new wxBoxSizer(wxVERTICAL);
00250
00251             // -- Header line: [username bold] [spacer] [HH:MM AM/PM small grey] --
00252             auto* headerLine = new wxBoxSizer(wxHORIZONTAL);
00253
00254             auto* userLabel = new wxStaticText(rowPanel, wxID_ANY,
00255                 wxString::FromUTF8(msg.userID));
00256             userLabel->SetForegroundColour(UIColors::TextSecondary);
00257             wxFont uf = userLabel->GetFont();
00258             uf.SetWeight(wxFONTWEIGHT_BOLD);
00259             userLabel->SetFont(uf);
00260

```

```

00261         auto* tsLabel = new wxStaticText(rowPanel, wxID_ANY,
00262                                         FormatTimestamp(msg.timestamp));
00263         tsLabel->SetForegroundColour(UIColors::TextMuted);
00264         wxFont tf = tsLabel->GetFont();
00265         tf.SetPointSize(tf.GetPointSize() - 1);
00266         tsLabel->SetFont(tf);
00267
00268         headerLine->Add(userLabel, 0, wxALIGN_CENTER_VERTICAL);
00269         headerLine->AddStretchSpacer(1);
00270         headerLine->Add(tsLabel, 0, wxALIGN_CENTER_VERTICAL);
00271         rowSizer->Add(headerLine, 0, wxEXPAND | wxLEFT | wxRIGHT | wxTOP, 8);
00272
00273         // -- Message text below the header -----
00274         auto* textLabel = new wxStaticText(rowPanel, wxID_ANY,
00275                                         wxString::FromUTF8(msg.text));
00276         textLabel->SetForegroundColour(UIColors::TextPrimary);
00277         textLabel->Wrap(600);
00278         rowSizer->Add(textLabel, 0, wxEXPAND | wxLEFT | wxRIGHT | wxBOTTOM, 8);
00279
00280         rowPanel->SetSizer(rowSizer);
00281         m_msgSizer->Add(rowPanel, 0, wxEXPAND | wxLEFT | wxRIGHT, 4);
00282
00283         // -- Hover highlight -----
00284         auto applyHover = [=](bool hover) {
00285             wxColour bg = hover ? kHoverBg : UIColors::Surface;
00286             rowPanel->SetBackgroundColour(bg);
00287             userLabel->SetBackgroundColour(bg);
00288             tsLabel->SetBackgroundColour(bg);
00289             textLabel->SetBackgroundColour(bg);
00290             rowPanel->Refresh();
00291         };
00292         for (wxWindow* w : std::initializer_list<wxWindow*>{
00293             rowPanel, userLabel, tsLabel, textLabel}) {
00294             w->Bind(wxEVT_ENTER_WINDOW, [applyHover](wxMouseEvent& e){ applyHover(true);
e.Skip(); });
00295             w->Bind(wxEVT_LEAVE_WINDOW, [applyHover](wxMouseEvent& e){ applyHover(false);
e.Skip(); });
00296         }
00297     }
00298 }
00299
00300 m_msgInner->Layout();
00301 m_msgScroll->FitInside();
00302 m_msgScroll->Layout();
00303
00304 // Scroll to bottom
00305 int xUnit, yUnit;
00306 m_msgScroll->GetScrollPixelsPerUnit(&xUnit, &yUnit);
00307 if (yUnit > 0) {
00308     int vH = m_msgScroll->GetVirtualSize().GetHeight();
00309     m_msgScroll->Scroll(0, vH / yUnit);
00310 }
00311
00312 Layout();
00313 }
00314
00315 // -- Private -----
00316
00317 void BlendChatPanel::DoSend()
00318 {
00319     wxString text = m_inputCtrl->GetValue().Trim().Trim(false);
00320     if (text.IsEmpty()) return;
00321
00322     AppState& state = AppState::getInstance();
00323     User* user = state.getCurrentUser();
00324     ChatRoom* room = state.getActiveChatRoom();
00325
00326     if (!room) return;
00327
00328     std::string userID = user ? user->getUserID() : "anonymous";
00329     std::string textStr = text.ToStdString();
00330
00331     // Dispatch through AppController - handleMessage validates the sender
00332     // and calls room->addMessage(), making the message available for Reload().
00333     m_controller.getEventRouter().dispatch("sendMessage",
00334     {"userID", userID}, {"text", textStr});
00335
00336     // Send to server via WebSocket so other participants receive it.
00337     if (m_ws) {
00338         try {
00339             m_ws->send_message(textStr);
00340         } catch (const std::exception& e) {
00341             std::cerr << "[BlendChatPanel] WebSocket send failed: " << e.what() << "\n";
00342         }
00343     }
00344
00345     m_inputCtrl->Clear();

```

```

00346     Reload();
00347 }
00348
00349 void BlendChatPanel::OnSend(wxCommandEvent& /*evt*/)
00350 {
00351     DoSend();
00352 }
00353
00354 void BlendChatPanel::OnInputEnter(wxCommandEvent& /*evt*/)
00355 {
00356     DoSend();
00357 }
00358
00359 void BlendChatPanel::OnParticipantsToggle(wxCommandEvent& /*evt*/)
00360 {
00361     bool nowVisible = !m_participantsDropdown->IsShown();
00362     m_participantsDropdown->Show(nowVisible);
00363     Layout();
00364 }
00365
00366 // -- WebSocket -----
00367
00368 void BlendChatPanel::ConnectWebSocket(const std::string& blendID,
00369                                       const std::string& userID)
00370 {
00371     // Reset any existing connection first.
00372     m_ws.reset();
00373     m_wsBlendID.clear();
00374
00375     try {
00376         m_ws = std::make_unique<ChatWebSocket>(kAppBackendWsPrefix);
00377         m_ws->set_blendID(blendID);
00378         m_ws->set_userID(userID);
00379
00380         // Register incoming-message callback - runs on the WS background thread.
00381         // Push to m_incoming; OnPollTimer drains it on the main thread.
00382         m_ws->setOnMessage([this](const std::string& sender,
00383                                   const std::string& content) {
00384             std::lock_guard<std::mutex> lk(m_incomingMx);
00385             m_incoming.emplace_back(sender, content);
00386         });
00387
00388         m_ws->connect();
00389         m_wsBlendID = blendID;
00390     } catch (const std::exception& e) {
00391         std::cerr << "[BlendChatPanel] WebSocket connect failed: " << e.what() << "\n";
00392         m_ws.reset();
00393     }
00394 }
00395
00396 void BlendChatPanel::OnPollTimer(wxTimerEvent& /*evt*/)
00397 {
00398     // Drain incoming messages pushed by the WebSocket background thread.
00399     std::vector<std::pair<std::string, std::string>> batch;
00400     {
00401         std::lock_guard<std::mutex> lk(m_incomingMx);
00402         std::swap(batch, m_incoming);
00403     }
00404
00405     if (batch.empty()) return;
00406
00407     ChatRoom* room = AppState::GetInstance().getActiveChatRoom();
00408     if (!room) return;
00409
00410     for (auto& [sender, content] : batch) {
00411         // Skip echoes of our own messages (already added in DoSend).
00412         User* me = AppState::GetInstance().getCurrentUser();
00413         if (me && sender == me->getUserID()) continue;
00414         room->addMessage(sender, content);
00415     }
00416
00417     Reload();
00418 }

```

7.131 BlendChatPanel.hpp File Reference

Blend chat screen.

```

#include <wx/wx.h>
#include <wx/timer.h>

```

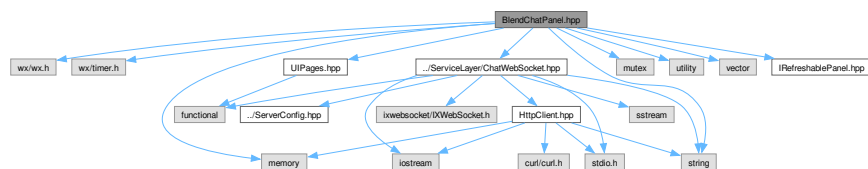


```

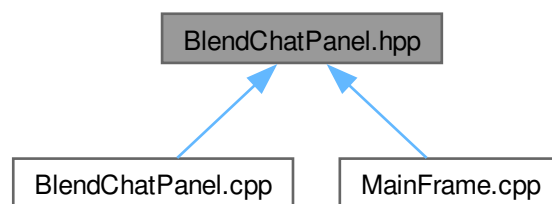
#include <memory>
#include <mutex>
#include <string>
#include <utility>
#include <vector>
#include "IRefreshablePanel.hpp"
#include "UIPages.hpp"
#include "../ServiceLayer/ChatWebSocket.hpp"

```

Include dependency graph for BlendChatPanel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [BlendChatPanel](#)
[BlendChatPanel](#) class declaration.

7.131.1 Detailed Description

[Blend](#) chat screen.

Author

Jasmine Kumar

Displays the chat room associated with the active blend and lets the user send new messages.

7.131.1.1 LAYOUT

[TopBar](#) ("Blend Chat", back -> HOME) [if no active blend] - "No active blend." label (full height) [if blend active] - scrollable message list + input row

7.131.1.2 MESSAGE LIST

Reload() reads [AppState::getActiveChatRoom\(\)](#)->getMessages() and rebuilds all message rows from scratch. Each row shows: "[userID]: message text" (secondary + primary colour) HH:MM timestamp (muted, smaller font) Messages are separated by thin wxStaticLine rules. The list auto-scrolls to the bottom after each rebuild.

7.131.1.3 SENDING

DoSend() dispatches "sendMessage" with the current user's ID and the trimmed input text, then calls Reload() so the new message appears. Both the Send button and pressing Enter in the input field call DoSend().

TODO: "sendMessage" is currently a stub in [AppController](#) - messages are added to the in-memory [ChatRoom](#) but may not be persisted to SQLite. Wire up [AppController::handleSendMessage](#) to save to the database. TODO: Real-time updates - currently the list only refreshes on send or manual navigation. Add a polling timer or network event listener to pick up messages from other users. TODO: Show the sender's display name instead of raw userID.

Definition in file [BlendChatPanel.hpp](#).

7.132 BlendChatPanel.hpp

[Go to the documentation of this file.](#)

```
00001
00038
00039 #pragma once
00040
00041 #include <wx/wx.h>
00042 #include <wx/timer.h>
00043
00044 #include <memory>
00045 #include <mutex>
00046 #include <string>
00047 #include <utility>
00048 #include <vector>
00049
00050 #include "IRefreshablePanel.hpp"
00051 #include "UIPages.hpp"
00052 #include "../ServiceLayer/ChatWebSocket.hpp"
00053
00054 class AppController;
00055 class wxScrolledWindow;
00056 class wxTextCtrl;
00057 class wxStaticText;
00058
00063 class BlendChatPanel : public wxPanel, public IRefreshablePanel {
00064 public:
00065     BlendChatPanel(wxWindow* parent, AppController& controller, NavigateFn nav);
00066
00067     void Refresh() override;
00068
00069     // Refreshes message list from AppState's active ChatRoom.
00070     void Reload();
00071
00072 private:
00073     AppController& m_controller;
00074     NavigateFn m_nav;
00075
00076     // Scrolled window containing the message rows
00077     wxScrolledWindow* m_msgScroll;
00078     // Inner panel inside the scroll window
```

```

00079     wxPanel*         m_msgInner;
00080     //Sizer for the inner panel
00081     wxBoxSizer*      m_msgSizer;
00082
00083     // "No active blend." placeholder (shown when no blend is active)
00084     wxStaticText*    m_noBlendLabel;
00085
00086     // Purple toggle button that shows/hides the participants dropdown
00087     wxButton*         m_participantsBtn;
00088     // Dropdown panel containing the participant list (hidden by default)
00089     wxPanel*          m_participantsDropdown;
00090     wxBoxSizer*       m_participantsSizer;
00091
00092     // Text input for typing messages
00093     wxTextCtrl*       m_inputCtrl;
00094
00095     // Panel that wraps the input row (hidden when no blend active)
00096     wxPanel*          m_inputPanel;
00097
00098     // -- WebSocket / incoming message handling -----
00099
00100     // Timer that drains m_incoming on the main thread (~500 ms interval).
00101     wxTimer*          m_pollTimer;
00102
00103     // Blend ID the WebSocket is currently connected to (empty = not connected).
00104     std::string        m_wsBlendID;
00105
00106     // Mutex protecting m_incoming (written by WS background thread, read by timer).
00107     std::mutex         m_incomingMx;
00108
00109     // Incoming messages from other participants waiting to be added to ChatRoom.
00110     // Declared after m_incomingMx; m_ws declared last so it is destroyed first,
00111     // stopping the background thread before the mutex/vector are torn down.
00112     std::vector<std::pair<std::string, std::string>> m_incoming;
00113
00114     // Active WebSocket connection (null when no blend is open).
00115     std::unique_ptr<ChatWebSocket> m_ws;
00116
00117     // Connects (or reconnects) the WebSocket for the given blend/user.
00118     void ConnectWebSocket(const std::string& blendID, const std::string& userID);
00119
00120     // Send logic (shared between button click and Enter key)
00121     void DoSend();
00122
00123     // Event handlers
00124     void OnSend(wxCommandEvent& evt);
00125     void OnInputEnter(wxCommandEvent& evt);
00126     void OnPollTimer(wxTimerEvent& evt);
00127     void OnParticipantsToggle(wxCommandEvent& evt);
00128 };

```

7.133 BlendCreationPanel.cpp File Reference

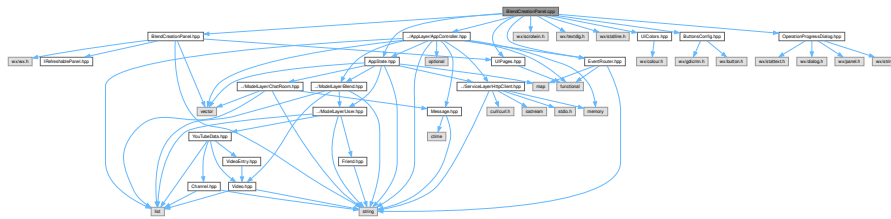
Implementation for [BlendCreationPanel](#).

```

#include "BlendCreationPanel.hpp"
#include <wx/scrolwin.h>
#include <wx/textdlg.h>
#include <wx/statline.h>
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include "UIPages.hpp"
#include "OperationProgressDialog.hpp"
#include "../AppLayer/AppController.hpp"
#include "../AppLayer/AppState.hpp"
#include "../AppLayer/EventRouter.hpp"

```

Include dependency graph for BlendCreationPanel.cpp:



7.133.1 Detailed Description

Implementation for [BlendCreationPanel](#).

Author

Jasmine Kumar

Definition in file [BlendCreationPanel.cpp](#).

7.134 BlendCreationPanel.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 // =====
00008 // BlendCreationPanel.cpp - New-blend setup screen implementation
00009 // =====
00010
00011 #include "BlendCreationPanel.hpp"
00012
00013 #include <wx/scrolwin.h>
00014 #include <wx/textdlg.h>
00015 #include <wx/statline.h>
00016
00017 #include "UIColors.hpp"
00018 #include "ButtonsConfig.hpp"
00019 #include "UIPages.hpp"
00020 #include "OperationProgressDialog.hpp"
00021 #include "../AppLayer/AppController.hpp"
00022 #include "../AppLayer/AppState.hpp"
00023 #include "../AppLayer/EventRouter.hpp"
00024
00025 // -- Construction -----
00026
00027 BlendCreationPanel::BlendCreationPanel(wxWindow* parent,
00028                                       AppController& controller,
00029                                       NavigateFn nav)
00030 : wxPanel(parent, wxID_ANY)
00031 , m_controller(controller)
00032 , m_nav(std::move(nav))
00033 {
00034     SetBackgroundColour(UIColors::Background);
00035
00036     auto* root = new wxBoxSizer(wxVERTICAL);
00037
00038     // -- TopBar (dynamic back: LOGIN if no blend yet, HOME otherwise) -----
00039     auto* topBar = new wxPanel(this, wxID_ANY);
00040     topBar->SetBackgroundColour(UIColors::Surface);
00041     topBar->SetMinSize(wxSize(-1, 48));
00042     {
00043         auto* backBtn = new wxButton(topBar, wxID_ANY, wxT("< Back"));
00044         backBtn->SetBackgroundColour(UIColors::SurfaceRaised);
00045         backBtn->SetForegroundColor(UIColors::TextPrimary);
00046         UIButtons::ApplySizeBounds(backBtn, ButtonType::TopBarBack);
00047     }
00048 }
```

```

00047
00048     backBtn->Bind(wx.EVT_BUTTON, [this](wxCommandEvent&) {
00049         if (AppState::getInstance().getActiveBlend() != nullptr)
00050             m_nav(Page::HOME);
00051         else
00052             m_nav(Page::LOGIN);
00053     });
00054
00055     auto* titleLabel = new wxStaticText(topBar, wxID_ANY, "New Blend",
00056                                         wxDefaultPosition, wxDefaultSize,
00057                                         wxALIGN_CENTER_HORIZONTAL);
00058     titleLabel->SetForegroundColour(UIColors::TextPrimary);
00059     wxFont tf = titleLabel->GetFont();
00060     tf.SetPointSize(14);
00061     tf.SetWeight(wxFontWeight_BOLD);
00062     titleLabel->SetFont(tf);
00063
00064     auto* barSizer = new wxBoxSizer(wxHORIZONTAL);
00065     barSizer->Add(backBtn, 0, wxALIGN_CENTER_VERTICAL | wxLEFT, 10);
00066     barSizer->AddStretchSpacer(1);
00067     barSizer->Add(titleLabel, 0, wxALIGN_CENTER_VERTICAL);
00068     barSizer->AddStretchSpacer(1);
00069     barSizer->Add(UIButtons::GetSize(ButtonType::TopBarBack).GetWidth() + 10, 0);
00070
00071     auto* outerBar = new wxBoxSizer(wxVERTICAL);
00072     outerBar->Add(barSizer, 1, wxEXPAND | wxTOP | wxBOTTOM, 8);
00073     topBar->SetSizer(outerBar);
00074 }
00075 root->Add(topBar, 0, wxEXPAND);
00076
00077 // -- Blend name row -----
00078 auto* namePanel = new wxPanel(this, wxID_ANY);
00079 namePanel->SetBackgroundColour(UIColors::Surface);
00080 auto* nameSizer = new wxBoxSizer(wxHORIZONTAL);
00081
00082 auto* nameLabel = new wxStaticText(namePanel, wxID_ANY, "Blend Name:");
00083 nameLabel->SetForegroundColour(UIColors::TextPrimary);
00084 wxFont nlf = nameLabel->GetFont();
00085 nlf.SetWeight(wxFontWeight_BOLD);
00086 nlf.SetPointSize(11);
00087 nameLabel->SetFont(nlf);
00088
00089 m_blendNameCtrl = new wxTextCtrl(namePanel, wxID_ANY, "",
00090                                   wxDefaultPosition, wxSize(-1, 36),
00091                                   wxTE_PROCESS_ENTER);
00092 m_blendNameCtrl->SetHint("Enter a name for your blend...");
00093 m_blendNameCtrl->SetBackgroundColour(UIColors::SurfaceRaised);
00094 m_blendNameCtrl->SetForegroundColour(UIColors::TextPrimary);
00095
00096 nameSizer->Add(nameLabel, 0, wxALIGN_CENTER_VERTICAL | wxLEFT, 16);
00097 nameSizer->Add(m_blendNameCtrl, 1, wxALIGN_CENTER_VERTICAL | wxLEFT | wxRIGHT, 12);
00098 namePanel->SetSizer(nameSizer);
00099 namePanel->SetMinSize(wxSize(-1, 48));
00100 root->Add(namePanel, 0, wxEXPAND);
00101
00102 // -- Action row -----
00103 auto* actionPanel = new wxPanel(this, wxID_ANY);
00104 actionPanel->SetBackgroundColour(UIColors::Surface);
00105 auto* actionSizer = new wxBoxSizer(wxHORIZONTAL);
00106
00107 auto* blendLabel = new wxStaticText(actionPanel, wxID_ANY, "PARTICIPANTS");
00108 blendLabel->SetForegroundColour(UIColors::TextPrimary);
00109 wxFont lf = blendLabel->GetFont();
00110 lf.SetWeight(wxFontWeight_BOLD);
00111 lf.SetPointSize(13);
00112 blendLabel->SetFont(lf);
00113
00114 auto* addBtn = new wxButton(actionPanel, wxID_ANY, "Add");
00115 UIButtons::ApplySizeBounds(addBtn, ButtonType::Compact);
00116 addBtn->SetBackgroundColour(UIColors::SurfaceRaised);
00117 addBtn->SetForegroundColour(UIColors::TextPrimary);
00118
00119 m_createBtn = new wxButton(actionPanel, wxID_ANY, "Create Blend");
00120 UIButtons::ApplySizeBounds(m_createBtn, ButtonType::Medium);
00121 m_createBtn->SetBackgroundColour(UIColors::Accent);
00122 m_createBtn->SetForegroundColour(UIColors::TextPrimary);
00123 m_createBtn->Disable();
00124
00125 actionSizer->Add(blendLabel, 0, wxALIGN_CENTER_VERTICAL | wxLEFT, 16);
00126 actionSizer->AddStretchSpacer(1);
00127 actionSizer->Add(addBtn, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 8);
00128 actionSizer->Add(m_createBtn, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 16);
00129 actionPanel->SetSizer(actionSizer);
00130 actionPanel->SetMinSize(wxSize(-1, 52));
00131
00132 root->Add(actionPanel, 0, wxEXPAND);
00133

```

```

00134 // -- User list area -----
00135 m_userListScroll = new wxScrolledWindow(this, wxID_ANY,
00136                                         wxDefaultPosition, wxDefaultSize,
00137                                         wxVSCROLL | wxBORDER_NONE);
00138 m_userListScroll->SetBackgroundColour(UIColors::Background);
00139 m_userListScroll->SetScrollRate(0, 12);
00140
00141 m_userListInner = new wxPanel(m_userListScroll, wxID_ANY);
00142 m_userListInner->SetBackgroundColour(UIColors::Background);
00143 m_userListSizer = new wxBoxSizer(wxVERTICAL);
00144
00145 // "No users added" placeholder
00146 m_emptyLabel = new wxStaticText(m_userListInner, wxID_ANY,
00147                                 "No users added",
00148                                 wxDefaultPosition, wxDefaultSize,
00149                                 wxALIGN_CENTER_HORIZONTAL);
00150 m_emptyLabel->SetForegroundColour(UIColors::TextMuted);
00151 m_userListSizer->Add(m_emptyLabel, 0,
00152                    wxALIGN_CENTER_HORIZONTAL | wxTOP, 24);
00153
00154 m_userListInner->SetSizer(m_userListSizer);
00155
00156 // Wire the inner panel into the scroll window
00157 auto* scrollSizer = new wxBoxSizer(wxVERTICAL);
00158 scrollSizer->Add(m_userListInner, 1, wxEXPAND);
00159 m_userListScroll->SetSizer(scrollSizer);
00160
00161 root->Add(m_userListScroll, 1, wxEXPAND | wxALL, 8);
00162
00163 // -- Count indicator -----
00164 m_countLabel = new wxStaticText(this, wxID_ANY, "0 / 8 users",
00165                                 wxDefaultPosition, wxDefaultSize,
00166                                 wxALIGN_CENTER_HORIZONTAL);
00167 m_countLabel->SetForegroundColour(UIColors::TextSecondary);
00168 root->Add(m_countLabel, 0, wxALIGN_CENTER_HORIZONTAL | wxBOTTOM, 12);
00169
00170 SetSizer(root);
00171
00172 // -- Bindings -----
00173 addBtn->Bind(wx.EVT_BUTTON, &BlendCreationPanel::OnAdd, this);
00174 m_createBtn->Bind(wx.EVT_BUTTON, &BlendCreationPanel::OnCreate, this);
00175 }
00176
00177 // -- Public -----
00178
00179 void BlendCreationPanel::Refresh()
00180 {
00181     Reload();
00182 }
00183
00184 void BlendCreationPanel::Reload()
00185 {
00186     m_addedUsers.clear();
00187     m_blendNameCtrl->Clear();
00188
00189     // Pre-populate with the logged-in user so they don't have to add themselves
00190     User* current = AppState::getInstance().getCurrentUser();
00191     if (current)
00192         m_addedUsers.push_back(current->getUserID());
00193
00194     RebuildUserList();
00195     UpdateCountLabel();
00196
00197     // Button enabled once there are >= 2 users (current user + at least one other)
00198     if (m_addedUsers.size() >= 2)
00199         m_createBtn->Enable();
00200     else
00201         m_createBtn->Disable();
00202 }
00203
00204 // -- Private helpers -----
00205
00206 void BlendCreationPanel::RebuildUserList()
00207 {
00208     // Destroy all children of the inner panel except the empty label.
00209     // We recreate the inner sizer from scratch.
00210     m_userListSizer->Clear(true /* delete_windows */);
00211
00212     // Re-create the empty label (it was deleted above)
00213     m_emptyLabel = new wxStaticText(m_userListInner, wxID_ANY,
00214                                     "No users added",
00215                                     wxDefaultPosition, wxDefaultSize,
00216                                     wxALIGN_CENTER_HORIZONTAL);
00217     m_emptyLabel->SetForegroundColour(UIColors::TextMuted);
00218
00219     if (m_addedUsers.empty()) {
00220         m_emptyLabel->Show(true);
00221     }

```

```

00221         m_userListSizer->Add(m_emptyLabel, 0,
00222                               wxALIGN_CENTER_HORIZONTAL | wxTOP, 24);
00223     } else {
00224         m_emptyLabel->Show(false);
00225         m_userListSizer->Add(m_emptyLabel, 0); // keep in sizer but hidden
00226
00227         for (const auto& username : m_addedUsers) {
00228             // Row panel
00229             auto* row = new wxPanel(m_userListInner, wxID_ANY);
00230             row->SetBackgroundColour(UIColors::SurfaceRaised);
00231             row->SetMinSize(wxSize(-1, 44));
00232
00233             auto* rowSizer = new wxBoxSizer(wxHORIZONTAL);
00234
00235             auto* nameLabel = new wxStaticText(row, wxID_ANY,
00236                                                wxString::FromUTF8(username));
00237             nameLabel->SetForegroundColour(UIColors::TextPrimary);
00238
00239             auto* removeBtn = new wxButton(row, wxID_ANY, wxT("\xc3\x97"));
00240             UIButtons::ApplySizeBounds(removeBtn, ButtonType::Icon);
00241             removeBtn->SetBackgroundColour(UIColors::SurfaceRaised);
00242             removeBtn->SetForegroundColour(UIColors::Danger);
00243
00244             // Capture username by value for the lambda
00245             std::string cap = username;
00246             removeBtn->Bind(wxEVT_BUTTON, [this, cap](wxCommandEvent&) {
00247                 OnRemoveUser(cap);
00248             });
00249
00250             rowSizer->Add(nameLabel, 0, wxALIGN_CENTER_VERTICAL | wxLEFT, 16);
00251             rowSizer->AddStretchSpacer(1);
00252             rowSizer->Add(removeBtn, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 8);
00253             row->SetSizer(rowSizer);
00254
00255             m_userListSizer->Add(row, 0, wxEXPAND | wxLEFT | wxRIGHT | wxTOP,
00256                                   6);
00257         }
00258     }
00259
00260     m_userListInner->Layout();
00261     m_userListScroll->FitInside();
00262     m_userListScroll->Layout();
00263 }
00264
00265 void BlendCreationPanel::UpdateCountLabel()
00266 {
00267     wxString txt = wxString::Format("%d / 8 users",
00268                                     static_cast<int>(m_addedUsers.size()));
00269     m_countLabel->SetLabel(txt);
00270 }
00271
00272 // -- Event handlers -----
00273
00274 void BlendCreationPanel::OnAdd(wxCommandEvent& /*evt*/)
00275 {
00276     if (static_cast<int>(m_addedUsers.size()) >= 8) {
00277         wxMessageBox("Maximum 8 users reached.", "Limit Reached",
00278                     wxOK | wxICON_INFORMATION, this);
00279         return;
00280     }
00281
00282     wxTextEntryDialog dlg(this, "Enter username:", "Add User");
00283     dlg.SetBackgroundColour(UIColors::Surface);
00284     if (dlg.ShowModal() != wxID_OK) return;
00285
00286     std::string entered = dlg.GetValue().Trim().ToStdString();
00287
00288     if (entered.empty()) return;
00289
00290     // Check for duplicate
00291     for (const auto& u : m_addedUsers) {
00292         if (u == entered) {
00293             wxMessageBox("User \"" + entered + "\" is already in the list.",
00294                         "Duplicate User", wxOK | wxICON_WARNING, this);
00295             return;
00296         }
00297     }
00298
00299     if (!m_controller.lookupUser(entered)) {
00300         wxMessageBox("User \"" + entered + "\" does not exist.\n"
00301                     "Please ask them to create an account first.",
00302                     "User Not Found", wxOK | wxICON_ERROR, this);
00303         return;
00304     }
00305
00306     m_addedUsers.push_back(entered);
00307     RebuildUserList();

```

```

00308     UpdateCountLabel();
00309
00310     if (m_addedUsers.size() >= 2)
00311         m_createBtn->Enable();
00312     else
00313         m_createBtn->Disable();
00314 }
00315
00316 void BlendCreationPanel::OnCreate(wxCommandEvent& /*evt*/)
00317 {
00318     if (m_addedUsers.size() < 2) return;
00319
00320     std::string blendName = m_blendNameCtrl->GetValue().ToStdString();
00321     if (blendName.empty()) {
00322         wxMessageBox("Please enter a name for your blend.",
00323             "Name Required", wxOK | wxICON_INFORMATION, this);
00324         m_blendNameCtrl->SetFocus();
00325         return;
00326     }
00327
00328     // Build payload: {"blendTitle": "...", "userID_0": "alice", "userID_1": "bob", ...}
00329     EventPayload payload;
00330     payload["blendTitle"] = blendName;
00331     for (std::size_t i = 0; i < m_addedUsers.size(); ++i)
00332         payload["userID_" + std::to_string(i)] = m_addedUsers[i];
00333
00334     OperationProgressDialog progress(this, "Creating Blend");
00335     progress.ShowModal();
00336     progress.UpdateProgress(0.02, "Preparing...");
00337
00338     // Wire up the progress reporter so AppController updates the bar as it works
00339     m_controller.setProgressReporter([&progress](double ratio, const std::string& message) {
00340         progress.UpdateProgress(ratio, wxString::FromUTF8(message));
00341     });
00342
00343     m_controller.getEventRouter().dispatch("createBlend", payload);
00344     m_controller.clearProgressReporter();
00345
00346     progress.UpdateProgress(1.0, "Blend ready");
00347     progress.Destroy();
00348
00349     // Check if any participants were missing data
00350     std::vector<std::string> missing = AppState::getInstance().getUsersWithoutData();
00351     if (!missing.empty()) {
00352         std::string msg = "The following users have no Watch Later data uploaded:\n";
00353         for (const auto& uid : missing)
00354             msg += "    \u2022 " + uid + "\n";
00355
00356         if (AppState::getInstance().getActiveBlend() != nullptr) {
00357             // Blend was still created from whoever had data - warn and continue
00358             msg += "\nTheir videos were not included. The blend was created with the remaining
00359 users.";
00360             wxMessageBox(msg, "Some Users Missing Data", wxOK | wxICON_WARNING, this);
00361             m_nav(Page::HOME);
00362         } else {
00363             // No one had data - cannot create a blend at all
00364             msg += "\nNo blend could be created. Please ensure at least one user has uploaded their
00365 data.";
00366             wxMessageBox(msg, "Cannot Create Blend", wxOK | wxICON_ERROR, this);
00367         }
00368         return;
00369     }
00370     m_nav(Page::HOME);
00371 }
00372 void BlendCreationPanel::OnRemoveUser(const std::string& username)
00373 {
00374     auto it = std::find(m_addedUsers.begin(), m_addedUsers.end(), username);
00375     if (it != m_addedUsers.end())
00376         m_addedUsers.erase(it);
00377
00378     RebuildUserList();
00379     UpdateCountLabel();
00380
00381     if (m_addedUsers.size() >= 2)
00382         m_createBtn->Enable();
00383     else
00384         m_createBtn->Disable();
00385 }

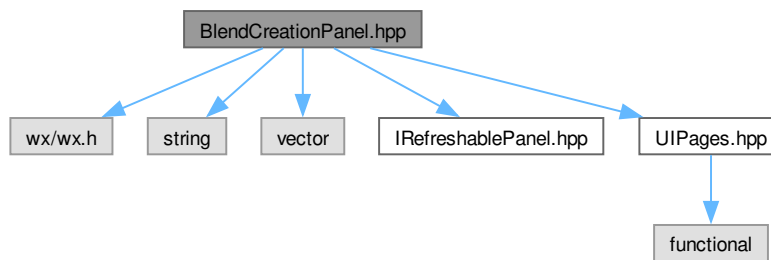
```


7.135 BlendCreationPanel.hpp File Reference

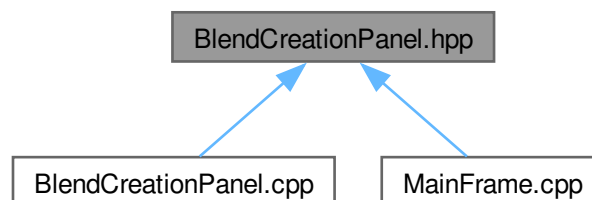
New-blend setup screen.

```
#include <wx/wx.h>
#include <string>
#include <vector>
#include "IRefreshablePanel.hpp"
#include "UIPages.hpp"
```

Include dependency graph for BlendCreationPanel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [BlendCreationPanel](#)
BlendCreationPanel class declaration.

7.135.1 Detailed Description

New-blend setup screen.

Author

Jasmine Kumar

Lets the logged-in user assemble a group of up to 8 participants (by username) and then create a blend from their combined YouTube data.

7.135.1.1 LAYOUT

[TopBar](#) ("New Blend", back -> HOME) Action row: "BLEND" label | Add button | Create [Blend](#) button Scrollable user list (wxScrolledWindow) "X / 8 users" count label

7.135.1.2 USER-ADD FLOW

1. [User](#) clicks Add -> wxTextEntryDialog prompts for a username.
2. Duplicate check against m_addedUsers.
3. "lookupUser" event dispatched to [AppController](#) (optimistic - user is added to the list regardless of whether the backend finds them). TODO: Wire up AppController::handleLookupUser to validate that the username exists in the database before adding.
4. "Create Blend" button is enabled once ≥ 2 users are present.

7.135.1.3 BLEND CREATION

OnCreate builds a payload {"userID_0": ..., "userID_1": ..., ...} and dispatches "createBlend". On return it navigates to BLEND_CHAT. TODO: If createBlend fails (e.g. one of the users has no data), display an error instead of navigating away.

7.135.1.4 RELOAD

Reload() is called by MainFrame::NavigateTo(BLEND_CREATION). It clears m_addedUsers so the panel starts fresh each time it is shown. TODO: Pre-populate the current user's own username so they do not have to add themselves manually.

Definition in file [BlendCreationPanel.hpp](#).

7.136 BlendCreationPanel.hpp

[Go to the documentation of this file.](#)

```
00001
00040
00041 #pragma once
00042
00043 #include <wx/wx.h>
00044 #include <string>
00045 #include <vector>
00046
00047 #include "IRefreshablePanel.hpp"
00048 #include "UIPages.hpp"
00049
00050 class AppController;
00051 class wxScrolledWindow;
00052 class wxStaticText;
00053 class wxButton;
00054
00059 class BlendCreationPanel : public wxPanel, public IRefreshablePanel {
00060 public:
00061     BlendCreationPanel(wxWindow* parent, AppController& controller, NavigateFn nav);
00062
00063     void Refresh() override;
00064
00065     // Clears user list and resets to empty state.
00066     void Reload();
00067
```

```

00068     // Test-friendly accessors
00069     #ifdef TESTING_MODE
00070     const std::vector<std::string>& GetAddedUsersForTesting() const {
00071         return m_addedUsers;
00072     }
00073     wxButton* GetCreateBtnForTesting() const { return m_createBtn; }
00074     wxStaticText* GetCountLabelForTesting() const { return m_countLabel; }
00075     void AddUserForTesting(const std::string& username) {
00076         m_addedUsers.push_back(username);
00077         RebuildUserList();
00078         UpdateCountLabel();
00079         if (m_addedUsers.size() >= 2) m_createBtn->Enable();
00080     }
00081     void RemoveUserForTesting(const std::string& username) {
00082         OnRemoveUser(username);
00083     }
00084     void CallOnCreateForTesting() {
00085         wxCommandEvent evt(wxEVT_BUTTON);
00086         OnCreate(evt);
00087     }
00088     #endif
00089 private:
00090     AppController& m_controller;
00091     NavigateFn      m_nav;
00092
00093     // The scrolled container that holds user rows
00094     wxScrolledWindow* m_userListScroll;
00095     // Inner panel inside the scroll window where rows are placed
00096     wxPanel*          m_userListInner;
00097     //Sizer for the inner panel
00098     wxBoxSizer*       m_userListSizer;
00099
00100     // "No users added" placeholder label
00101     wxStaticText*     m_emptyLabel;
00102
00103     // "X / 8 users" count label
00104     wxStaticText*     m_countLabel;
00105
00106     // Blend name input
00107     wxTextCtrl*       m_blendNameCtrl;
00108
00109     // "Create Blend" button - enabled when >= 2 users added
00110     wxButton*         m_createBtn;
00111
00112     // Currently added usernames
00113     std::vector<std::string> m_addedUsers;
00114
00115     // Rebuilds the user-row display from m_addedUsers.
00116     void RebuildUserList();
00117
00118     // Updates the count label text.
00119     void UpdateCountLabel();
00120
00121     // Event handlers
00122     void OnAdd(wxCommandEvent& evt);
00123     void OnCreate(wxCommandEvent& evt);
00124     void OnRemoveUser(const std::string& username);
00125 };

```

7.137 BlendFeedPanel.cpp File Reference

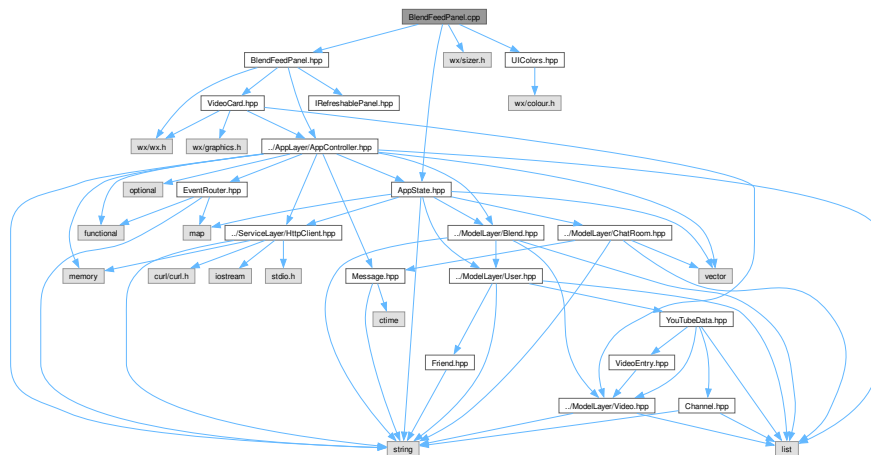
Implementation for [BlendFeedPanel](#).

```

#include "BlendFeedPanel.hpp"
#include <wx/sizer.h>
#include "UIColors.hpp"
#include "../AppLayer/AppState.hpp"

```

Include dependency graph for BlendFeedPanel.cpp:



7.137.1 Detailed Description

Implementation for [BlendFeedPanel](#).

Author

Jasmine Kumar

Definition in file [BlendFeedPanel.cpp](#).

7.138 BlendFeedPanel.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 // =====
00008 // BlendFeedPanel.cpp - 3x2 video grid for the Home page implementation
00009 // =====
00010
00011 #include "BlendFeedPanel.hpp"
00012
00013 #include <wx/sizer.h>
00014
00015 #include "UIColors.hpp"
00016 #include "../AppLayer/AppState.hpp"
00017
00018 // -----
00019 // Constructor
00020 // -----
00021 BlendFeedPanel::BlendFeedPanel(wxWindow* parent, AppController& controller)
00022     : wxPanel(parent, wxID_ANY)
00023     , m_controller(controller)
00024     , m_offset(0)
00025 {
00026     SetBackgroundColour(UIColors::Background);
00027
00028     // 2 rows x 3 cols, 16 px gaps in both directions
00029     wxGridSizer* grid = new wxGridSizer(2, 3, 16, 16);
00030
00031     for (int i = 0; i < kPageSize; ++i) {
00032         m_cards[i] = new VideoCard(this, m_controller);
00033         grid->Add(m_cards[i], 0, wxEXPAND);
00034     }

```

```

00035
00036 // Outer sizer adds a uniform margin around the grid
00037 wxBoxSizer* outer = new wxBoxSizer(wxVERTICAL);
00038 outer->Add(grid, 1, wxEXPAND | wxALL, 16);
00039 SetSizer(outer);
00040
00041 LoadFromBlend();
00042 }
00043
00044 void BlendFeedPanel::Refresh()
00045 {
00046     m_offset = 0;
00047     LoadFromBlend();
00048 }
00049
00050 // -----
00051 // NextPage
00052 // -----
00053 void BlendFeedPanel::NextPage()
00054 {
00055     Blend* blend = AppState::getInstance().getActiveBlend();
00056     if (!blend || blend->size() == 0) {
00057         m_offset = 0;
00058         LoadFromBlend();
00059         return;
00060     }
00061
00062     int nextOffset = m_offset + kPageSize;
00063     if (blend->size() - nextOffset < kPageSize) {
00064         // Not enough videos remaining to fill a full page - refresh with unseen videos
00065         m_controller.getEventRouter().dispatch("refresh");
00066         m_offset = 0;
00067     } else {
00068         m_offset = nextOffset;
00069     }
00070     LoadFromBlend();
00071 }
00072
00073 // -----
00074 // LoadFromBlend
00075 // -----
00076 void BlendFeedPanel::LoadFromBlend()
00077 {
00078     Blend* blend = AppState::getInstance().getActiveBlend();
00079
00080     if (!blend || blend->size() == 0) {
00081         // No active blend - show all cards in placeholder state
00082         for (int i = 0; i < kPageSize; ++i) {
00083             m_cards[i]->Clear();
00084         }
00085         return;
00086     }
00087
00088     const int total = blend->size();
00089
00090     // Clamp offset in case the blend shrank since last NextPage()
00091     if (m_offset >= total) {
00092         m_offset = 0;
00093     }
00094
00095     for (int i = 0; i < kPageSize; ++i) {
00096         int idx = m_offset + i;
00097         if (idx < total)
00098             m_cards[i]->SetVideo(blend->getVideo(idx));
00099         else
00100             m_cards[i]->Clear();
00101     }
00102 }

```

7.139 BlendFeedPanel.hpp File Reference

Home-page video feed panel that displays paged blend results.

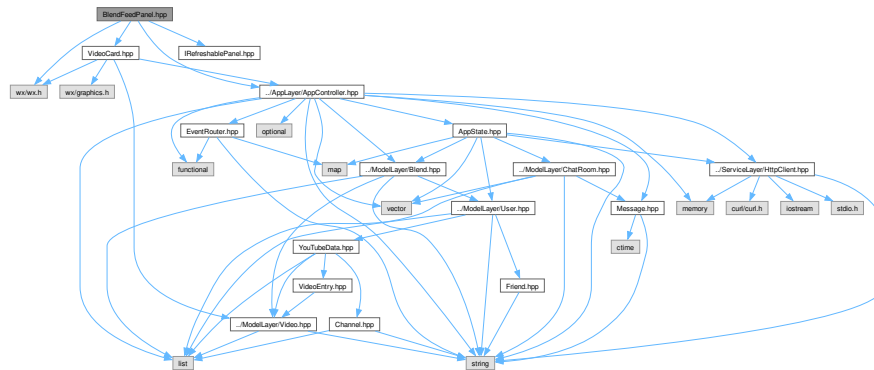
```

#include <wx/wx.h>
#include "VideoCard.hpp"
#include "IRefreshablePanel.hpp"

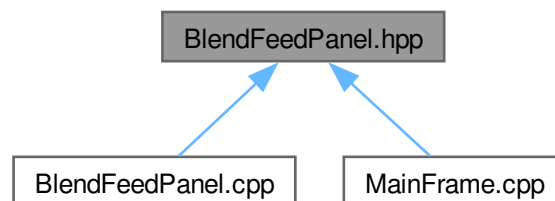
```

```
#include "../AppLayer/AppController.hpp"
```

Include dependency graph for BlendFeedPanel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [BlendFeedPanel](#)
Panel that renders paged blend videos in a fixed 3x2 card layout.

7.139.1 Detailed Description

Home-page video feed panel that displays paged blend results.

Author

Jasmine Kumar

Displays six [VideoCard](#) widgets in a fixed 3x2 grid sourced from the active blend in [AppState](#).

7.139.1.1 PAGING

m_offset tracks which six-video slice of the blend is visible. NextPage() advances by six entries (wrapping as needed), then refreshes via LoadFromBlend().

7.139.1.2 LOADING

LoadFromBlend() reads [AppState](#) directly so it always reflects the current active blend.

Definition in file [BlendFeedPanel.hpp](#).

7.140 BlendFeedPanel.hpp

[Go to the documentation of this file.](#)

```

00001
00020
00021 #pragma once
00022
00023 #include <wx/wx.h>
00024
00025 #include "VideoCard.hpp"
00026 #include "IRefreshablePanel.hpp"
00027 #include "../AppLayer/AppController.hpp"
00028
00033 class BlendFeedPanel : public wxPanel, public IRefreshablePanel {
00034 public:
00040     BlendFeedPanel(wxWindow* parent, AppController& controller);
00041
00045     void Refresh() override;
00046
00050     void NextPage();
00051
00057     void LoadFromBlend();
00058
00059 private:
00060     static constexpr int kPageSize = 6;    // 3 cols x 2 rows
00061
00062     AppController& m_controller;
00063     VideoCard*     m_cards[kPageSize];
00064     int            m_offset;    // index of the first card in the current page
00065 };

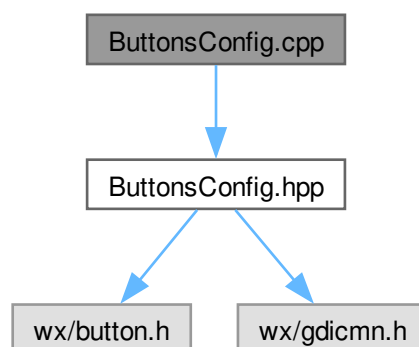
```

7.141 ButtonsConfig.cpp File Reference

Implements shared button sizing presets used across UILayer.

```
#include "ButtonsConfig.hpp"
```

Include dependency graph for ButtonsConfig.cpp:



Functions

- const [BtnConfig](#) & [UIButtons::GetConfig](#) ([ButtonType](#) type)
Returns the raw dimension preset for a button type.
- wxSize [UIButtons::GetSize](#) ([ButtonType](#) type)
Converts a preset into its minimum-size wxSize.
- wxSize [UIButtons::GetMinSize](#) ([ButtonType](#) type)
Returns the configured minimum size for a preset.
- wxSize [UIButtons::GetMaxSize](#) ([ButtonType](#) type)
Returns the configured maximum size for a preset.
- void [UIButtons::ApplySizeBounds](#) (wxButton *button, [ButtonType](#) type)
Applies the selected preset's min and max size bounds.

7.141.1 Detailed Description

Implements shared button sizing presets used across UILayer.

Author

Shamar Pennant

Definition in file [ButtonsConfig.cpp](#).

7.141.2 Function Documentation

7.141.2.1 ApplySizeBounds()

```
void UIButtons::ApplySizeBounds (  
    wxButton * button,  
    ButtonType type)
```

Applies the selected preset's min and max size bounds.

Parameters

<i>button</i>	Target button.
<i>type</i>	Preset key.

Definition at line 67 of file [ButtonsConfig.cpp](#).

7.141.2.2 GetConfig()

```
const BtnConfig & UIButtons::GetConfig (  
    ButtonType type)
```

Returns the raw dimension preset for a button type.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Constant preset dimensions.

Definition at line 30 of file [ButtonsConfig.cpp](#).

7.141.2.3 GetMaxSize()

```
wxSize UIButtons::GetMaxSize (  
    ButtonType type)
```

Returns the configured maximum size for a preset.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Maximum size constraints. (-1, -1) means no max bound.

Definition at line 61 of file [ButtonsConfig.cpp](#).

7.141.2.4 GetMinSize()

```
wxSize UIButtons::GetMinSize (  
    ButtonType type)
```

Returns the configured minimum size for a preset.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Minimum size constraints.

Definition at line 55 of file [ButtonsConfig.cpp](#).

7.141.2.5 GetSize()

```
wxSize UIButtons::GetSize (  
    ButtonType type)
```

Converts a preset into its minimum-size wxSize.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Equivalent minimum-size wxSize for the selected preset.

Definition at line 50 of file [ButtonsConfig.cpp](#).

7.142 ButtonsConfig.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "ButtonsConfig.hpp"
00008
00009 namespace {
00014 constexpr BtnConfig kTopBarBack = {80, 30, 180, 36};
00015 constexpr BtnConfig kNav = {96, 36, 220, 44};
00016 constexpr BtnConfig kCompact = {80, 34, 160, 40};
00017 constexpr BtnConfig kSmall = {100, 34, 180, 40};
00018 constexpr BtnConfig kMedium = {120, 34, 260, 42};
00019 constexpr BtnConfig kLarge = {140, 34, 280, 42};
00020 constexpr BtnConfig kFormTab = {120, 36, 260, 44};
00021 constexpr BtnConfig kFormSubmit = {140, 40, 320, 48};
00022 constexpr BtnConfig kFullWidthPrimary = {160, 40, 560, 48};
00023 constexpr BtnConfig kFullWidthSecondary = {160, 32, 560, 40};
00024 constexpr BtnConfig kFullWidthDanger = {160, 40, 560, 48};
00025 constexpr BtnConfig kIcon = {30, 30, 44, 44};
00026 }
00027
00028 namespace UIButtons {
00029
00030 const BtnConfig& GetConfig(ButtonType type)
00031 {
00032     switch (type) {
00033     case ButtonType::TopBarBack:      return kTopBarBack;
00034     case ButtonType::Nav:             return kNav;
00035     case ButtonType::Compact:         return kCompact;
00036     case ButtonType::Small:           return kSmall;
00037     case ButtonType::Medium:          return kMedium;
00038     case ButtonType::Large:           return kLarge;
00039     case ButtonType::FormTab:         return kFormTab;
00040     case ButtonType::FormSubmit:      return kFormSubmit;
00041     case ButtonType::FullWidthPrimary: return kFullWidthPrimary;
00042     case ButtonType::FullWidthSecondary: return kFullWidthSecondary;
00043     case ButtonType::FullWidthDanger: return kFullWidthDanger;
00044     case ButtonType::Icon:            return kIcon;
00045     }
00046
00047     return kNav;
00048 }
00049
00050 wxSize GetSize(ButtonType type)
00051 {
00052     return GetMinSize(type);
00053 }
00054
00055 wxSize GetMinSize(ButtonType type)
00056 {
00057     const BtnConfig& cfg = GetConfig(type);
00058     return wxSize(cfg.minWidth, cfg.minHeight);
00059 }
00060
00061 wxSize GetMaxSize(ButtonType type)
00062 {
00063     const BtnConfig& cfg = GetConfig(type);
00064     return wxSize(cfg.maxWidth, cfg.maxHeight);
00065 }
00066
00067 void ApplySizeBounds(wxButton* button, ButtonType type)
00068 {
00069     if (!button) {

```

```

00070         return;
00071     }
00072
00073     button->SetMinSize(GetMinSize(type));
00074     button->SetMaxSize(GetMaxSize(type));
00075 }
00076
00077 } // namespace UIButtons

```

7.143 ButtonsConfig.hpp File Reference

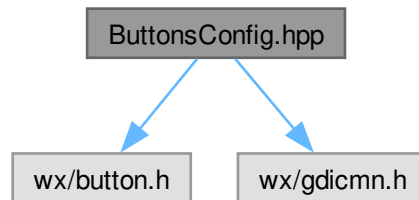
Centralized button size presets for UI layer widgets.

```

#include <wx/button.h>
#include <wx/gdicmn.h>

```

Include dependency graph for ButtonsConfig.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [BtnConfig](#)
Width/height dimensions for a button preset.

Enumerations

- enum class [ButtonType](#) {
[TopBarBack](#) = 0 , [Nav](#) , [Compact](#) , [Small](#) ,
[Medium](#) , [Large](#) , [FormTab](#) , [FormSubmit](#) ,
[FullWidthPrimary](#) , [FullWidthSecondary](#) , [FullWidthDanger](#) , [Icon](#) }
Named presets used to keep button sizing consistent across panels.

Functions

- const [BtnConfig](#) & [UIButtons::GetConfig](#) ([ButtonType](#) type)
Returns the raw dimension preset for a button type.
- wxSize [UIButtons::GetMinSize](#) ([ButtonType](#) type)
Returns the configured minimum size for a preset.
- wxSize [UIButtons::GetMaxSize](#) ([ButtonType](#) type)
Returns the configured maximum size for a preset.
- wxSize [UIButtons::GetSize](#) ([ButtonType](#) type)
Converts a preset into its minimum-size wxSize.
- void [UIButtons::ApplySizeBounds](#) (wxButton *button, [ButtonType](#) type)
Applies the selected preset's min and max size bounds.

7.143.1 Detailed Description

Centralized button size presets for UI layer widgets.

Author

Shamar Pennant

Presets are organized by usage area:

- Navigation: [TopBarBack](#), [Nav](#)
- Forms: [FormTab](#), [FormSubmit](#)
- Full-width actions: [FullWidthPrimary](#), [FullWidthSecondary](#), [FullWidthDanger](#)
- Utility/dialog actions: [Compact](#), [Small](#), [Medium](#), [Icon](#)

Use [UIButtons::ApplySizeBounds\(\)](#) for direct control styling, or [UIButtons::GetSize\(\)](#) when a wxSize is required.

Note: [ApplySizeBounds\(\)](#) also applies an optional max size bound from config.

Definition in file [ButtonsConfig.hpp](#).

7.143.2 Enumeration Type Documentation

7.143.2.1 ButtonType

```
enum class ButtonType [strong]
```

Named presets used to keep button sizing consistent across panels.

Enumerator

TopBarBack	Navigation back button (TopBar).
Nav	Primary nav buttons used in page headers/home top rows.
Compact	Small utility action (e.g., Add button).
Small	Dialog action button size.
Medium	Medium action button size.
FormTab	Login/Register tab buttons.
FormSubmit	Form submit actions (Sign In / Create Account).
FullWidthPrimary	Full-width prominent action.
FullWidthSecondary	Full-width secondary action.
FullWidthDanger	Full-width destructive action (Log Out).
Icon	Square icon-style action (e.g., remove-row button).

Definition at line 28 of file [ButtonsConfig.hpp](#).

7.143.3 Function Documentation

7.143.3.1 ApplySizeBounds()

```
void UIButtons::ApplySizeBounds (
    wxButton * button,
    ButtonType type)
```

Applies the selected preset's min and max size bounds.

Parameters

<i>button</i>	Target button.
<i>type</i>	Preset key.

Definition at line 67 of file [ButtonsConfig.cpp](#).

7.143.3.2 GetConfig()

```
const BtnConfig & UIButtons::GetConfig (
    ButtonType type)
```

Returns the raw dimension preset for a button type.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Constant preset dimensions.

Definition at line 30 of file [ButtonsConfig.cpp](#).

7.143.3.3 GetMaxSize()

```
wxSize UIButtons::GetMaxSize (
    ButtonType type)
```

Returns the configured maximum size for a preset.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Maximum size constraints. (-1, -1) means no max bound.

Definition at line 61 of file [ButtonsConfig.cpp](#).

7.143.3.4 GetMinSize()

```
wxSize UIButtons::GetMinSize (
    ButtonType type)
```

Returns the configured minimum size for a preset.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Minimum size constraints.

Definition at line 55 of file [ButtonsConfig.cpp](#).

7.143.3.5 GetSize()

```
wxSize UIButtons::GetSize (
    ButtonType type)
```

Converts a preset into its minimum-size wxSize.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Equivalent minimum-size wxSize for the selected preset.

Definition at line 50 of file [ButtonsConfig.cpp](#).

7.144 ButtonsConfig.hpp

[Go to the documentation of this file.](#)

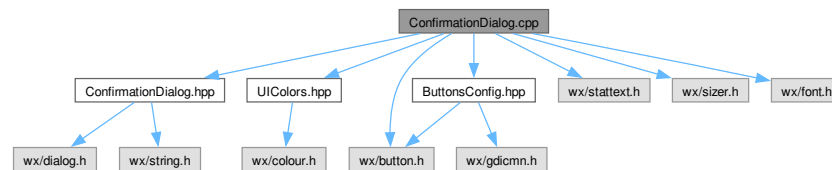
```
00001
00018
00019 #pragma once
00020
00021 #include <wx/button.h>
00022 #include <wx/gdicmn.h>
00023
00028 enum class ButtonType {
00029     TopBarBack = 0,
00030     Nav,
00031     Compact,
00032     Small,
00033     Medium,
00034     Large,
00035     FormTab,
00036     FormSubmit,
00037     FullWidthPrimary,
00038     FullWidthSecondary,
00039     FullWidthDanger,
00040     Icon
00041 };
00042
00050 struct BtnConfig {
00051     int minWidth;
00052     int minHeight;
00053     int maxWidth;
00054     int maxHeight;
00055 };
00056
00057 namespace UIButtons {
00063     const BtnConfig& GetConfig(ButtonType type);
00064
00070     wxSize GetMinSize(ButtonType type);
00071
00077     wxSize GetMaxSize(ButtonType type);
00078
00084     wxSize GetSize(ButtonType type);
00085
00091     void ApplySizeBounds(wxButton* button, ButtonType type);
00092 }
```

7.145 ConfirmationDialog.cpp File Reference

Implementation for [ConfirmationDialog](#).

```
#include "ConfirmationDialog.hpp"
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include <wx/stattext.h>
#include <wx/button.h>
#include <wx/sizer.h>
#include <wx/font.h>
```

Include dependency graph for ConfirmationDialog.cpp:



7.145.1 Detailed Description

Implementation for [ConfirmationDialog](#).

Author

Jasmine Kumar

Definition in file [ConfirmationDialog.cpp](#).

7.146 ConfirmationDialog.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #include "ConfirmationDialog.hpp"
00008 #include "UIColors.hpp"
00009 #include "ButtonsConfig.hpp"
00010 #include <wx/stattext.h>
00011 #include <wx/button.h>
00012 #include <wx/sizer.h>
00013 #include <wx/font.h>
00014
00015 // -----
00016 // Constructor
00017 // -----
00018 ConfirmationDialog::ConfirmationDialog(wxWindow* parent,
00019                                       const wxString& title,
00020                                       const wxString& message,
00021                                       const wxString& okLabel,
00022                                       const wxString& cancelLabel)
00023 : wxDialog(parent, wxID_ANY, title,
00024            wxDefaultPosition, wxDefaultSize,
00025            wxDEFAULT_DIALOG_STYLE | wxSTAY_ON_TOP)
00026 {
00027     // -- Overall panel background -----
```

```

00028     SetBackgroundColour(UIColors::Background);
00029
00030     // -- Message label -----
00031     wxStaticText* msgLabel = new wxStaticText(this, wxID_ANY, message,
00032                                             wxDefaultPosition, wxDefaultSize,
00033                                             wxALIGN_CENTER_HORIZONTAL);
00034     msgLabel->SetForegroundColour(UIColors::TextPrimary);
00035     msgLabel->SetBackgroundColour(UIColors::Background);
00036
00037     wxFont msgFont = msgLabel->GetFont();
00038     msgFont.SetPointSize(11);
00039     msgLabel->SetFont(msgFont);
00040
00041     // -- OK button (Accent colour) -----
00042     wxButton* okBtn = new wxButton(this, wxID_OK, okLabel,
00043                                   wxDefaultPosition, wxDefaultSize,
00044                                   wxBORDER_NONE);
00045     UIButtons::ApplySizeBounds(okBtn, ButtonType::Small);
00046     okBtn->SetBackgroundColour(UIColors::Accent);
00047     okBtn->SetForegroundColour(UIColors::TextPrimary);
00048
00049     okBtn->Bind(wxEVT_ENTER_WINDOW, [okBtn](wxMouseEvent& e) {
00050         okBtn->SetBackgroundColour(UIColors::AccentHover);
00051         okBtn->Refresh();
00052         e.Skip();
00053     });
00054     okBtn->Bind(wxEVT_LEAVE_WINDOW, [okBtn](wxMouseEvent& e) {
00055         okBtn->SetBackgroundColour(UIColors::Accent);
00056         okBtn->Refresh();
00057         e.Skip();
00058     });
00059
00060     // -- Cancel button (SurfaceRaised) -----
00061     wxButton* cancelBtn = new wxButton(this, wxID_CANCEL, cancelLabel,
00062                                       wxDefaultPosition, wxDefaultSize,
00063                                       wxBORDER_NONE);
00064     UIButtons::ApplySizeBounds(cancelBtn, ButtonType::Small);
00065     cancelBtn->SetBackgroundColour(UIColors::SurfaceRaised);
00066     cancelBtn->SetForegroundColour(UIColors::TextPrimary);
00067
00068     cancelBtn->Bind(wxEVT_ENTER_WINDOW, [cancelBtn](wxMouseEvent& e) {
00069         cancelBtn->SetBackgroundColour(wxColour(55, 55, 55));
00070         cancelBtn->Refresh();
00071         e.Skip();
00072     });
00073     cancelBtn->Bind(wxEVT_LEAVE_WINDOW, [cancelBtn](wxMouseEvent& e) {
00074         cancelBtn->SetBackgroundColour(UIColors::SurfaceRaised);
00075         cancelBtn->Refresh();
00076         e.Skip();
00077     });
00078
00079     // -- Button row -----
00080     wxBoxSizer* btnSizer = new wxBoxSizer(wxHORIZONTAL);
00081     btnSizer->Add(cancelBtn, 0, wxRIGHT, 12);
00082     btnSizer->Add(okBtn, 0);
00083
00084     // -- Root sizer with generous padding for "rounded" feel -----
00085     wxBoxSizer* rootSizer = new wxBoxSizer(wxVERTICAL);
00086     rootSizer->AddSpacer(24);
00087     rootSizer->Add(msgLabel, 0, wxEXPAND | wxLEFT | wxRIGHT, 32);
00088     rootSizer->AddSpacer(28);
00089     rootSizer->Add(btnSizer, 0, wxALIGN_RIGHT | wxRIGHT, 32);
00090     rootSizer->AddSpacer(24);
00091
00092     SetSizer(rootSizer);
00093     Fit();
00094     SetMinSize(wxSize(340, -1));
00095     Fit(); // refit after applying min size
00096
00097     if (parent) {
00098         CentreOnParent();
00099     } else {
00100         CentreOnScreen();
00101     }
00102 }
00103
00104 // -----
00105 // Static convenience helper
00106 // -----
00107 bool ConfirmationDialog::Confirm(wxWindow* parent,
00108                                 const wxString& title,
00109                                 const wxString& message)
00110 {
00111     ConfirmationDialog dlg(parent, title, message);
00112     return dlg.ShowModal() == wxID_OK;
00113 }

```

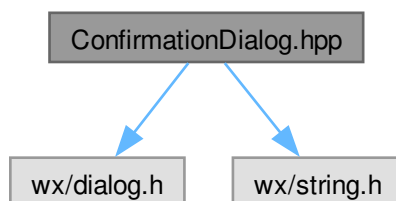

7.147 ConfirmationDialog.hpp File Reference

Dark-themed modal confirmation dialog.

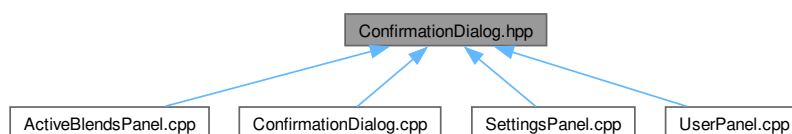
```
#include <wx/dialog.h>
```

```
#include <wx/string.h>
```

Include dependency graph for ConfirmationDialog.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [ConfirmationDialog](#)
ConfirmationDialog class declaration.

7.147.1 Detailed Description

Dark-themed modal confirmation dialog.

Author

Jasmine Kumar

Definition in file [ConfirmationDialog.hpp](#).

7.148 ConfirmationDialog.hpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 // A simple two-button (OK / Cancel) dialog that matches the YTBIND dark theme.
00008 // Use the static Confirm() helper for the common one-shot pattern, or
00009 // construct manually when you need custom button labels.
00010 //
00011 // Typical usage (static helper):
00012 //   if (ConfirmationDialog::Confirm(this, "Log Out", "Are you sure?")) { ... }
00013 //
00014 // Custom labels:
00015 //   ConfirmationDialog dlg(this, "Delete Blend", "This cannot be undone.",
00016 //                           "Delete", "Keep");
00017 //   if (dlg.ShowModal() == wxID_OK) { ... }
00018 //
00019 // Styling notes:
00020 //   - OK button uses UIColors::Accent (purple)
00021 //   - Cancel button uses UIColors::SurfaceRaised (grey)
00022 //   - Both buttons lighten slightly on hover
00023 //
00024 // TODO: Add a wxIcon / warning symbol to the left of the message for
00025 //       destructive-action dialogs (delete, logout, etc.).
00026
00027 #pragma once
00028 #include <wx/dialog.h>
00029 #include <wx/string.h>
00030
00031 // A dark-themed modal confirmation dialog.
00032 //
00033 // Typical usage via the static helper:
00034 //   if (ConfirmationDialog::Confirm(parent, "Delete blend?", "This cannot be undone.)) { ... }
00035 //
00036 // Or construct manually for custom button labels:
00037 //   ConfirmationDialog dlg(parent, "Title", "Message", "Delete", "Keep");
00038 //   if (dlg.ShowModal() == wxID_OK) { ... }
00043 class ConfirmationDialog : public wxDialog {
00044 public:
00045     ConfirmationDialog(wxWindow*      parent,
00046                       const wxString& title,
00047                       const wxString& message,
00048                       const wxString& okLabel      = wxT("OK"),
00049                       const wxString& cancelLabel = wxT("Cancel"));
00050
00051     // Convenience: create, show, destroy - returns true if OK was clicked.
00052     static bool Confirm(wxWindow*      parent,
00053                       const wxString& title,
00054                       const wxString& message);
00055 };

```

7.149 DataInstructionsPanel.cpp File Reference

Implementation for [DataInstructionsPanel](#).

```

#include "DataInstructionsPanel.hpp"
#include <wx/filedlg.h>
#include <wx/statline.h>
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include "../AppLayer/AppController.hpp"
#include "../AppLayer/AppState.hpp"
#include "../AppLayer/EventRouter.hpp"
#include "../ModelLayer/User.hpp"

```

[illegible]

Implementation for `DataInstructionsPanel`.

Jasmine Kumar

7.150 DataInstructionsPanel.cpp

```

00001
00006
00007 // =====
00008 // DataInstructionsPanel.cpp - Google Takeout data setup screen implementation
00009 // =====
00010
00011 #include "DataInstructionsPanel.hpp"
00012
00013 #include <wx/filedlg.h>
00014 #include <wx/statline.h>
00015
00016 #include "UIColors.hpp"
00017 #include "ButtonsConfig.hpp"
00018 #include "../AppLayer/AppController.hpp"
00019 #include "../AppLayer/AppState.hpp"
00020 #include "../AppLayer/EventRouter.hpp"
00021 #include "../ModelLayer/User.hpp"
00022
00023 DataInstructionsPanel::DataInstructionsPanel(wxWindow* parent,
00024                                             ApplicationController& controller,
00025                                             NavigateFn nav)
00026 : wxPanel(parent, wxID_ANY)
00027 , m_controller(controller)
00028 , m_nav(std::move(nav))
00029 {
00030     SetBackgroundColour(UIColors::Background);
00031
00032     auto* outer = new wxBoxSizer(wxVERTICAL);
00033
00034     // -- Heading -----
00035     auto* heading = new wxStaticText(this, wxID_ANY,
00036                                     "Set Up Your YouTube Data",
00037                                     wxDefaultPosition, wxDefaultSize,
00038                                     wxALIGN_CENTRE_HORIZONTAL);
00039
00040     {
00041         wxFont f = heading->GetFont();
00042         f.SetPointSize(22); f.SetWeight(wxFONTWEIGHT_BOLD);
00043         heading->SetFont(f);
00044     }

```

```

00044     heading->SetForegroundColour(UIColors::TextPrimary);
00045
00046     auto* sub = new wxStaticText(this, wxID_ANY,
00047         "YTBLND needs your YouTube export data to create a blend.",
00048         wxDefaultPosition, wxDefaultSize, wxALIGN_CENTRE_HORIZONTAL);
00049     sub->SetForegroundColour(UIColors::TextSecondary);
00050
00051     outer->AddStretchSpacer(1);
00052     outer->Add(heading, 0, wxALIGN_CENTER | wxBOTTOM, 8);
00053     outer->Add(sub, 0, wxALIGN_CENTER | wxBOTTOM, 32);
00054
00055     // -- Steps card -----
00056     auto* card = new wxPanel(this, wxID_ANY);
00057     card->SetBackgroundColour(UIColors::Surface);
00058     card->SetMinSize(wxSize(520, -1));
00059     auto* cardSizer = new wxBoxSizer(wxVERTICAL);
00060
00061     const wxString steps =
00062         "1. Go to takeout.google.com\n"
00063         "2. Click \"Deselect all\", then tick YouTube and YouTube Music\n"
00064         "3. Export and download the archive\n"
00065         "4. Inside the zip, choose one of these files:\n"
00066         "    - playlists / <any playlist>.csv (e.g. Watch later videos.csv)\n"
00067         "    - history / watch-history.html\n"
00068         "5. Click \"Browse for Data File...\" below and select that file";
00069
00070     auto* stepsLabel = new wxStaticText(card, wxID_ANY, steps);
00071     stepsLabel->SetForegroundColour(UIColors::TextPrimary);
00072     {
00073         wxFont f = stepsLabel->GetFont(); f.SetPointSize(11);
00074         stepsLabel->SetFont(f);
00075     }
00076
00077     auto* div = new wxStaticLine(card, wxID_ANY);
00078     div->SetBackgroundColour(UIColors::Separator);
00079
00080     auto* browseBtn = new wxButton(card, wxID_ANY, "Browse for CSV...");
00081     UIButtons::ApplySizeBounds(browseBtn, ButtonType::FullWidthPrimary);
00082     browseBtn->SetBackgroundColour(UIColors::Accent);
00083     browseBtn->SetForegroundColour(UIColors::TextPrimary);
00084
00085     // Add a Cancel button so users who decline to upload return to the login screen
00086     auto* cancelBtn = new wxButton(card, wxID_ANY, "Cancel");
00087     UIButtons::ApplySizeBounds(cancelBtn, ButtonType::FullWidthSecondary);
00088     cancelBtn->SetBackgroundColour(UIColors::Surface);
00089     cancelBtn->SetForegroundColour(UIColors::TextPrimary);
00090
00091     // Place buttons side-by-side (browse = primary, cancel = secondary)
00092     auto* btnRow = new wxBoxSizer(wxHORIZONTAL);
00093     btnRow->Add(browseBtn, 1, wxRIGHT, 8);
00094     btnRow->Add(cancelBtn, 0);
00095
00096     cardSizer->Add(stepsLabel, 0, wxALL, 24);
00097     cardSizer->Add(div, 0, wxEXPAND | wxLEFT | wxRIGHT, 0);
00098     cardSizer->Add(btnRow, 0, wxEXPAND | wxALL, 20);
00099     card->SetSizer(cardSizer);
00100
00101     auto* hCentre = new wxBoxSizer(wxHORIZONTAL);
00102     hCentre->AddStretchSpacer(1);
00103     hCentre->Add(card, 0, wxEXPAND);
00104     hCentre->AddStretchSpacer(1);
00105
00106     outer->Add(hCentre, 0, wxEXPAND);
00107     outer->AddStretchSpacer(2);
00108     SetSizer(outer);
00109
00110     browseBtn->Bind(wxEVT_BUTTON, &DataInstructionsPanel::OnBrowse, this);
00111     // If the user cancels (does not upload), abort registration flow and return to login.
00112     cancelBtn->Bind(wxEVT_BUTTON, [this](wxCommandEvent&) {
00113         User* user = AppState::getInstance().getCurrentUser();
00114         if (user) {
00115             // If the user has no uploaded watch-later videos, remove the server account
00116             // so the username isn't left reserved.
00117             if (user->getYouTubeData().getWatchLaterVideos().empty()) {
00118                 // deleteAccount expects userID and password in payload
00119                 m_controller.getEventRouter().dispatch("deleteAccount",
00120                     {"userID", user->getUserID()}, {"password", user->getPassword()});
00121             }
00122         }
00123         // Clear local session and return to login
00124         m_controller.getEventRouter().dispatch("logout", {});
00125         m_nav(Page::LOGIN);
00126     });
00127 }
00128
00129 void DataInstructionsPanel::OnBrowse(wxCommandEvent& /*evt*/)
00130 {

```

```

00131     wxFileDialog dlg(this, "Select YouTube Export Data", "", "",
00132                     "Supported files (*.csv;*.html;*.htm)|*.csv;*.html;*.htm|"
00133                     "CSV files (*.csv)|*.csv|"
00134                     "HTML files (*.html;*.htm)|*.html;*.htm|"
00135                     "All files (*.*)|*.*",
00136                     wxFD_OPEN | wxFD_FILE_MUST_EXIST);
00137     if (dlg.ShowModal() != wxID_OK) return;
00138
00139     std::string path = dlg.GetPath().ToStdString();
00140     User* user = AppState::getInstance().getCurrentUser();
00141     if (!user) return;
00142
00143     m_controller.getEventRouter().dispatch("uploadData",
00144     {"filePath", path}, {"userID", user->getUserID()});
00145
00146     std::string uploadError = AppState::getInstance().takePendingUploadError();
00147     if (!uploadError.empty()) {
00148         wxMessageBox(uploadError, "Upload Failed", wxOK | wxICON_ERROR, this);
00149         return;
00150     }
00151
00152     if (user->getYouTubeData().getWatchLaterVideos().empty()) {
00153         wxMessageBox(
00154             "Upload did not produce any usable videos. Please try another file.",
00155             "Upload Incomplete",
00156             wxOK | wxICON_WARNING,
00157             this
00158         );
00159         return;
00160     }
00161
00162     m_nav(Page::BLEND_CREATION);
00163 }
00164

```

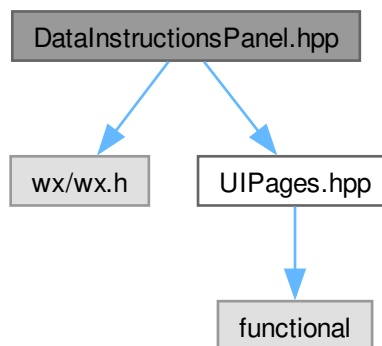
7.151 DataInstructionsPanel.hpp File Reference

Setup instructions panel for importing Google Takeout CSV data.

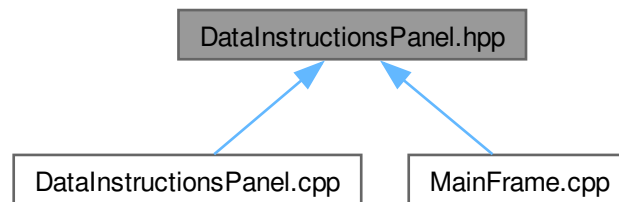
```
#include <wx/wx.h>
```

```
#include "UIPages.hpp"
```

Include dependency graph for DataInstructionsPanel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [DataInstructionsPanel](#)
[DataInstructionsPanel](#) class declaration.

7.151.1 Detailed Description

Setup instructions panel for importing Google Takeout CSV data.

Author

Jasmine Kumar

Definition in file [DataInstructionsPanel.hpp](#).

7.152 DataInstructionsPanel.hpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 // Shown after login when the user has no YouTube Watch Later data.
00008 // Walks them through exporting from Google Takeout and loading the CSV.
00009 //
00010 // LAYOUT
00011 // -----
00012 // "Set Up Your YouTube Data" heading
00013 // Subtitle explaining why data is needed
00014 // Steps card (Surface, centred):
00015 //   Numbered step-by-step instructions for Google Takeout export
00016 //   Horizontal divider
00017 //   "Browse for CSV..." button (Accent)
00018 //   "Skip for now" button (SurfaceRaised / muted)
00019 //
00020 // CSV UPLOAD
00021 // -----
00022 // OnBrowse: opens wxFileDialog (CSV filter), dispatches "uploadData" with
00023 // {filePath, userID} from AppState, then navigates to BLEND_CREATION.
00024 // TODO: Display a loading indicator while parsing - large CSV files may
00025 // take a noticeable amount of time.
00026 // TODO: Show an error/success message if the upload fails or the CSV is
00027 // malformed, rather than always navigating to BLEND_CREATION.
00028 //
00029 // SKIP
  
```

```

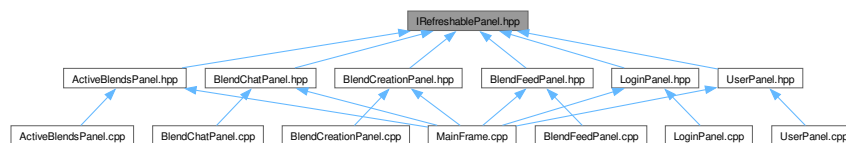
00030 // ----
00031 //   OnSkip: navigates directly to BLEND_CREATION without uploading.
00032 //   The user can still create a blend (the algorithm will have less/no
00033 //   personal data to work with).
00034 //   TODO: Remind the user on the blend creation screen that they skipped
00035 //   data upload, so the blend may not be personalised.
00036
00037 #pragma once
00038 #include <wx/wx.h>
00039 #include "UIPages.hpp"
00040
00041 class AppController;
00042
00043 // Shown after login when the user has no YouTube data loaded yet.
00044 // Explains how to obtain and place the Watch Later CSV, then lets the
00045 // user browse for the file or skip and go straight to blend creation.
00050 class DataInstructionsPanel : public wxPanel {
00051 public:
00052     DataInstructionsPanel(wxWindow* parent, AppController& controller, NavigateFn nav);
00053
00054 private:
00055     AppController& m_controller;
00056     NavigateFn     m_nav;
00057
00058     void OnBrowse(wxCommandEvent&);
00059 };

```

7.153 IRefreshablePanel.hpp File Reference

Interface for panels that support refresh operations.

This graph shows which files directly or indirectly include this file:



Classes

- class [IRefreshablePanel](#)
Interface for panels that support refresh operations.

7.153.1 Detailed Description

Interface for panels that support refresh operations.

Author

Shamar Pennant

[IRefreshablePanel](#) is a mixin interface that enables UI panels to receive refresh notifications when they become visible or when data changes. Implementing panels can update their display to reflect current application state whenever `Refresh()` is called.

Definition in file [IRefreshablePanel.hpp](#).

7.154 IRefreshablePanel.hpp

[Go to the documentation of this file.](#)

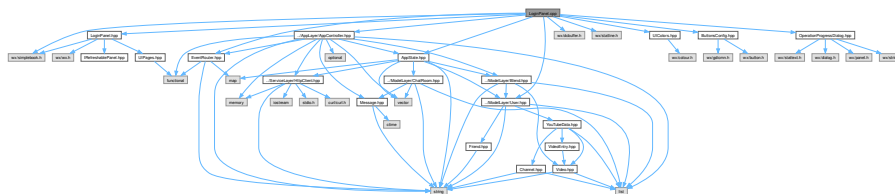
```
00001
00011
00012 #pragma once
00013
00021 class IRefreshablePanel {
00022 public:
00023     virtual ~IRefreshablePanel() = default;
00024
00029     virtual void Refresh() = 0;
00030 };
```

7.155 LoginPanel.cpp File Reference

Implementation for [LoginPanel](#).

```
#include "LoginPanel.hpp"
#include <wx/dcbuffer.h>
#include <wx/simplebook.h>
#include <wx/statline.h>
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include "OperationProgressDialog.hpp"
#include "../AppLayer/AppController.hpp"
#include "../AppLayer/AppState.hpp"
#include "../AppLayer/EventRouter.hpp"
#include "../ModelLayer/User.hpp"
```

Include dependency graph for LoginPanel.cpp:



Functions

- static wxTextCtrl * [MakeField](#) (wxWindow *parent, const wxString &hint, bool password=false)
- static wxStaticText * [MakeError](#) (wxWindow *parent)

7.155.1 Detailed Description

Implementation for [LoginPanel](#).

Author

Jasmine Kumar
Shamar Pennant

Definition in file [LoginPanel.cpp](#).

7.155.2 Function Documentation

7.155.2.1 MakeError()

```
wxStaticText * MakeError (
    wxWindow * parent) [static]
```

Definition at line 41 of file [LoginPanel.cpp](#).

7.155.2.2 MakeField()

```
wxTextCtrl * MakeField (
    wxWindow * parent,
    const wxString & hint,
    bool password = false) [static]
```

Definition at line 28 of file [LoginPanel.cpp](#).

7.156 LoginPanel.cpp

[Go to the documentation of this file.](#)

```
00001
00007
00008 // =====
00009 // LoginPanel.cpp - Sign In / Register screen implementation
00010 // =====
00011
00012 #include "LoginPanel.hpp"
00013
00014 #include <wx/dcbuffer.h>
00015 #include <wx/simplebook.h>
00016 #include <wx/statline.h>
00017
00018 #include "UIColors.hpp"
00019 #include "ButtonsConfig.hpp"
00020 #include "OperationProgressDialog.hpp"
00021 #include "../AppLayer/AppController.hpp"
00022 #include "../AppLayer/AppState.hpp"
00023 #include "../AppLayer/EventRouter.hpp"
00024 #include "../ModelLayer/User.hpp"
00025
00026 // -- Helpers -----
00027
00028 static wxTextCtrl* MakeField(wxWindow* parent,
00029                             const wxString& hint,
00030                             bool password = false)
00031 {
00032     long style = wxTE_PROCESS_ENTER | (password ? wxTE_PASSWORD : 0);
00033     auto* ctrl = new wxTextCtrl(parent, wxID_ANY, "", wxDefaultPosition,
00034                                wxSize(-1, 36), style);
00035     ctrl->SetHint(hint);
00036     ctrl->SetBackgroundColour(UIColors::SurfaceRaised);
00037     ctrl->SetForegroundColour(UIColors::TextPrimary);
00038     return ctrl;
00039 }
00040
00041 static wxStaticText* MakeError(wxWindow* parent)
00042 {
00043     auto* lbl = new wxStaticText(parent, wxID_ANY, "",
00044                                 wxDefaultPosition, wxDefaultSize,
00045                                 wxALIGN_CENTER_HORIZONTAL);
00046     lbl->SetForegroundColour(UIColors::Danger);
00047     lbl->Hide();
00048     return lbl;
00049 }
00050
00051 // -- Construction -----
00052
```

```

00053 LoginPanel::LoginPanel(wxWindow* parent, AppController& controller, NavigateFn nav,
00054                          const wxImage& bgImage)
00055     : wxPanel(parent, wxID_ANY)
00056     , m_controller(controller)
00057     , m_nav(std::move(nav))
00058     , m_bgImage(bgImage)
00059 {
00060     SetBackgroundStyle(wxBG_STYLE_PAINT);
00061     Bind(wxEVT_PAINT, &LoginPanel::OnPaint, this);
00062
00063     // -- Outer centering sizer -----
00064     auto* outer = new wxBoxSizer(wxVERTICAL);
00065
00066     // -- YTB LND title -----
00067     auto* titleLabel = new wxStaticText(this, wxID_ANY, "YTB LND",
00068                                         wxDefaultPosition, wxDefaultSize,
00069                                         wxALIGN_CENTRE_HORIZONTAL);
00070     wxFont tf = titleLabel->GetFont();
00071     tf.SetPointSize(42);
00072     tf.SetWeight(wxFONTWEIGHT_BOLD);
00073     titleLabel->SetFont(tf);
00074     titleLabel->SetForegroundColour(UIColors::Accent);
00075
00076     auto* subtitle = new wxStaticText(this, wxID_ANY, "Your blend, your music.",
00077                                       wxDefaultPosition, wxDefaultSize,
00078                                       wxALIGN_CENTRE_HORIZONTAL);
00079     subtitle->SetForegroundColour(UIColors::TextSecondary);
00080
00081     outer->AddStretchSpacer(2);
00082     outer->Add(titleLabel, 0, wxALIGN_CENTER | wxBOTTOM, 4);
00083     outer->Add(subtitle, 0, wxALIGN_CENTER | wxBOTTOM, 32);
00084
00085     // -- Form card -----
00086     auto* card = new wxPanel(this, wxID_ANY);
00087     card->SetBackgroundColour(UIColors::Surface);
00088     card->SetMinSize(wxSize(400, -1));
00089     auto* cardSizer = new wxBoxSizer(wxVERTICAL);
00090
00091     // Tab strip
00092     auto* tabRow = new wxBoxSizer(wxHORIZONTAL);
00093     m_signinTab = new wxButton(card, wxID_ANY, "Sign In",
00094                               wxDefaultPosition, wxDefaultSize);
00095     m_registerTab = new wxButton(card, wxID_ANY, "Register",
00096                                  wxDefaultPosition, wxDefaultSize);
00097     for (auto* btn : {m_signinTab, m_registerTab}) {
00098         UIButtons::ApplySizeBounds(btn, ButtonType::FormTab);
00099         btn->SetBackgroundColour(UIColors::SurfaceRaised);
00100         btn->SetForegroundColour(UIColors::TextSecondary);
00101     }
00102     tabRow->Add(m_signinTab, 1, wxEXPAND);
00103     tabRow->Add(m_registerTab, 1, wxEXPAND);
00104     cardSizer->Add(tabRow, 0, wxEXPAND);
00105
00106     // Divider below tabs
00107     auto* div = new wxStaticLine(card, wxID_ANY);
00108     div->SetBackgroundColour(UIColors::Separator);
00109     cardSizer->Add(div, 0, wxEXPAND);
00110
00111     // Form pages
00112     m_formBook = new wxSimplebook(card, wxID_ANY);
00113     m_formBook->SetBackgroundColour(UIColors::Surface);
00114
00115     // Page 0: Sign In
00116     {
00117         auto* page = new wxPanel(m_formBook, wxID_ANY);
00118         page->SetBackgroundColour(UIColors::Surface);
00119         auto* sz = new wxBoxSizer(wxVERTICAL);
00120         BuildSignInForm(page, sz);
00121         page->SetSizer(sz);
00122         m_formBook->AddPage(page, "SignIn");
00123     }
00124
00125     // Page 1: Register
00126     {
00127         auto* page = new wxPanel(m_formBook, wxID_ANY);
00128         page->SetBackgroundColour(UIColors::Surface);
00129         auto* sz = new wxBoxSizer(wxVERTICAL);
00130         BuildRegisterForm(page, sz);
00131         page->SetSizer(sz);
00132         m_formBook->AddPage(page, "Register");
00133     }
00134
00135     cardSizer->Add(m_formBook, 0, wxEXPAND);
00136     card->SetSizer(cardSizer);
00137
00138     // Centre the card horizontally
00139     auto* hCentre = new wxBoxSizer(wxHORIZONTAL);

```

```

00140     hCentre->AddStretchSpacer(1);
00141     hCentre->Add(card, 0, wxEXPAND);
00142     hCentre->AddStretchSpacer(1);
00143
00144     outer->Add(hCentre, 0, wxEXPAND);
00145     outer->AddStretchSpacer(3);
00146     SetSizer(outer);
00147
00148     // -- Tab bindings -----
00149     m_signinTab ->Bind(wxEVT_BUTTON, [this](wxCommandEvent&){ ShowTab(0); });
00150     m_registerTab->Bind(wxEVT_BUTTON, [this](wxCommandEvent&){ ShowTab(1); });
00151
00152     ShowTab(0); // start on Sign In
00153 }
00154
00155 // -- Form builders -----
00156
00157 void LoginPanel::BuildSignInForm(wxWindow* parent, wxSizer* sizer)
00158 {
00159     m_siUsername = MakeField(parent, "Username");
00160     m_siPassword = MakeField(parent, "Password", true);
00161     m_siError    = MakeError(parent);
00162
00163     auto* btn = new wxButton(parent, wxID_ANY, "Sign In",
00164                             wxDefaultPosition, wxDefaultSize);
00165     UIButtons::ApplySizeBounds(btn, ButtonType::FormSubmit);
00166     btn->SetBackgroundColour(UIColors::Accent);
00167     btn->SetForegroundColour(UIColors::TextPrimary);
00168
00169     sizer->Add(m_siUsername, 0, wxEXPAND | wxCENTRE | wxALL, 8);
00170     sizer->Add(m_siPassword, 0, wxEXPAND | wxCENTRE | wxALL, 8);
00171     sizer->Add(m_siError,    0, wxEXPAND | wxTOP | wxBOTTOM, 8);
00172     sizer->Add(btn,          0, wxEXPAND | wxALL | wxCENTER, 20);
00173
00174     btn->Bind(wxEVT_BUTTON, &LoginPanel::OnSignIn, this);
00175     // Also allow Enter in password field to submit
00176     m_siPassword->Bind(wxEVT_TEXT_ENTER,
00177                      [this](wxCommandEvent& e){ OnSignIn(e); });
00178 }
00179
00180 void LoginPanel::BuildRegisterForm(wxWindow* parent, wxSizer* sizer)
00181 {
00182     m_regUsername = MakeField(parent, "Username (this will be your login ID)");
00183     m_regEmail    = MakeField(parent, "Email (optional)");
00184     m_regPassword = MakeField(parent, "Password", true);
00185     m_regConfirm  = MakeField(parent, "Confirm password", true);
00186     m_regError    = MakeError(parent);
00187
00188     auto* btn = new wxButton(parent, wxID_ANY, "Create Account",
00189                             wxDefaultPosition, wxDefaultSize);
00190     UIButtons::ApplySizeBounds(btn, ButtonType::FormSubmit);
00191     btn->SetBackgroundColour(UIColors::Accent);
00192     btn->SetForegroundColour(UIColors::TextPrimary);
00193
00194     sizer->Add(m_regUsername, 0, wxEXPAND | wxALL, 8);
00195     sizer->Add(m_regEmail,    0, wxEXPAND | wxALL, 8);
00196     sizer->Add(m_regPassword, 0, wxEXPAND | wxALL, 8);
00197     sizer->Add(m_regConfirm,  0, wxEXPAND | wxALL, 8);
00198     sizer->Add(m_regError,    0, wxEXPAND | wxTOP | wxBOTTOM, 8);
00199     sizer->Add(btn,           0, wxEXPAND | wxALL | wxCENTER, 20);
00200
00201     btn->Bind(wxEVT_BUTTON, &LoginPanel::OnRegister, this);
00202 }
00203
00204 // -- Tab switcher -----
00205
00206 void LoginPanel::ShowTab(int index)
00207 {
00208     m_formBook->SetSelection(index);
00209
00210     // Active tab: accent colour; inactive: muted
00211     auto styleActive = [](wxButton* btn) {
00212         btn->SetBackgroundColour(UIColors::Accent);
00213         btn->SetForegroundColour(UIColors::TextPrimary);
00214     };
00215     auto styleInactive = [](wxButton* btn) {
00216         btn->SetBackgroundColour(UIColors::SurfaceRaised);
00217         btn->SetForegroundColour(UIColors::TextSecondary);
00218     };
00219
00220     if (index == 0) { styleActive(m_signinTab); styleInactive(m_registerTab); }
00221     else { styleInactive(m_signinTab); styleActive(m_registerTab); }
00222
00223     m_siError ->Hide();
00224     m_regError->Hide();
00225     Layout();
00226 }

```

```

00227
00228 // -- Event handlers -----
00229
00230 void LoginPanel::OnSignIn(wxCommandEvent& /*evt*/)
00231 {
00232     std::string username = m_siUsername->GetValue().Trim().ToStdString();
00233     std::string password = m_siPassword->GetValue().ToStdString();
00234
00235     if (username.empty() || password.empty()) {
00236         m_siError->SetLabel("Please enter your username and password.");
00237         m_siError->Show();
00238         m_siError->GetParent()->Layout();
00239         Layout();
00240         return;
00241     }
00242
00243     OperationProgressDialog progress(this, "Signing In");
00244     progress.ShowModal();
00245     progress.UpdateProgress(0.02, "Preparing login...");
00246
00247     m_controller.setProgressReporter([&progress](double ratio, const std::string& message) {
00248         progress.UpdateProgress(ratio, wxString::FromUTF8(message));
00249     });
00250
00251     // Dispatch uses userID - username IS the userID in this app
00252     m_controller.getEventRouter().dispatch("login",
00253         {"userID", username}, {"password", password});
00254     m_controller.clearProgressReporter();
00255
00256     progress.UpdateProgress(1.0, "Login finished");
00257     progress.Destroy();
00258
00259     if (AppState::getInstance().getCurrentUser() == nullptr) {
00260         m_siError->SetLabel("Incorrect username or password.");
00261         m_siError->Show();
00262         m_siError->GetParent()->Layout();
00263         m_siPassword->Clear();
00264         Layout();
00265         return;
00266     }
00267
00268     m_siError->Hide();
00269     ProceedAfterLogin();
00270 }
00271
00272 void LoginPanel::OnRegister(wxCommandEvent& /*evt*/)
00273 {
00274     std::string username = m_regUsername->GetValue().Trim().ToStdString();
00275     std::string email = m_regEmail->GetValue().Trim().ToStdString();
00276     std::string password = m_regPassword->GetValue().ToStdString();
00277     std::string confirm = m_regConfirm->GetValue().ToStdString();
00278
00279     if (username.empty() || password.empty()) {
00280         m_regError->SetLabel("Username and password are required.");
00281         m_regError->Show(); Layout(); return;
00282     }
00283     if (password != confirm) {
00284         m_regError->SetLabel("Passwords do not match.");
00285         m_regError->Show(); Layout(); return;
00286     }
00287     if (password.size() < 6) {
00288         m_regError->SetLabel("Password must be at least 6 characters.");
00289         m_regError->Show(); Layout(); return;
00290     }
00291
00292     // userID == username in this app
00293     m_controller.getEventRouter().dispatch("register",
00294         {"userID", username},
00295         {"username", username},
00296         {"email", email},
00297         {"password", password});
00298
00299     if (AppState::getInstance().getCurrentUser() == nullptr) {
00300         m_regError->SetLabel("Username already taken. Choose another.");
00301         m_regError->Show(); Layout(); return;
00302     }
00303
00304     m_regError->Hide();
00305     ProceedAfterLogin();
00306 }
00307
00308 // -- Post-login routing -----
00309
00310 void LoginPanel::ProceedAfterLogin()
00311 {
00312     User* user = AppState::getInstance().getCurrentUser();
00313     if (!user) return;

```

```

00314
00315 // If a blend was restored by handleLogin, go straight to the feed
00316 if (AppState::getInstance().getActiveBlend() != nullptr) {
00317     m_nav(Page::HOME);
00318     return;
00319 }
00320
00321 // If the user has no Watch Later data, send them to the upload screen.
00322 // Show an info dialog first if they were added to a blend by someone else.
00323 bool hasData = !user->getYouTubeData().getWatchLaterVideos().empty();
00324 if (!hasData) {
00325     std::string msg = AppState::getInstance().takePendingBlendMessage();
00326     if (!msg.empty())
00327         wxMessageBox(msg, "You're in a Blend!", wxOK | wxICON_INFORMATION, nullptr);
00328     m_nav(Page::DATA_INSTRUCTIONS);
00329     return;
00330 }
00331
00332 // Has data but no blend yet - let them create one
00333 m_nav(Page::BLEND_CREATION);
00334 }
00335
00336 // -- Reset -----
00337
00338 void LoginPanel::Refresh()
00339 {
00340     Reset();
00341 }
00342
00343 void LoginPanel::Reset()
00344 {
00345     m_siUsername->Clear();
00346     m_siPassword->Clear();
00347     m_regUsername->Clear();
00348     m_regEmail->Clear();
00349     m_regPassword->Clear();
00350     m_regConfirm->Clear();
00351     m_siError->Hide();
00352     m_regError->Hide();
00353     ShowTab(0);
00354 }
00355
00356 // -- Background painting -----
00357
00358 void LoginPanel::OnPaint(wxPaintEvent&)
00359 {
00360     wxBufferedPaintDC dc(this);
00361     if (m_bgImage.IsOk()) {
00362         wxSize sz = GetClientSize();
00363         if (sz.x > 0 && sz.y > 0) {
00364             wxBitmap scaled(m_bgImage.Scale(sz.x, sz.y, wxIMAGE_QUALITY_HIGH));
00365             dc.DrawBitmap(scaled, 0, 0, false);
00366             return;
00367         }
00368     }
00369     dc.SetBackground(wxBrush(UIColors::Background));
00370     dc.Clear();
00371 }

```

7.157 LoginPanel.hpp File Reference

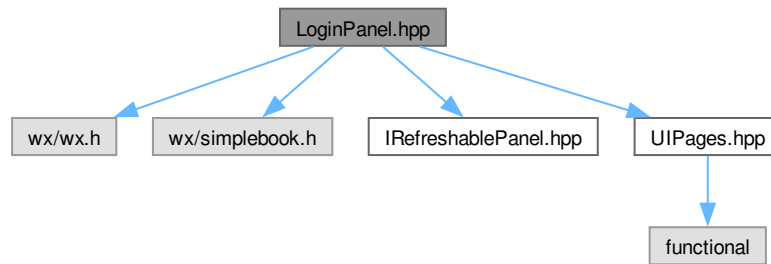
Sign In / Register screen.

```

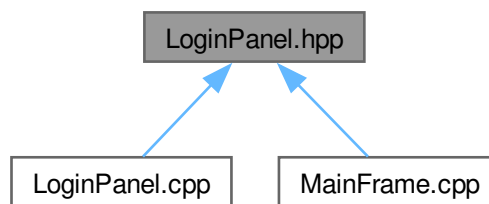
#include <wx/wx.h>
#include <wx/simplebook.h>
#include "IRefreshablePanel.hpp"
#include "UIPages.hpp"

```

Include dependency graph for LoginPanel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [LoginPanel](#)
[LoginPanel](#) class declaration.

7.157.1 Detailed Description

Sign In / Register screen.

Author

Jasmine Kumar
Shamar Pennant

The first screen the user sees. Contains a two-tab form card (Sign In and Register) rendered on top of the dark background with the YTBLND title.

7.157.1.1 SIGN IN

Dispatches "login" with {userID, password}. userID == username in this app. Checks [AppState::getCurrentUser\(\)](#) immediately after dispatch (the call is synchronous). On success, calls [ProceedAfterLogin\(\)](#). Displays an inline error label on failure (wrong credentials).

7.157.1.2 REGISTER

Validates locally (non-empty, passwords match, >=6 chars), then dispatches "register" with {userID, username, email, password}. On success, calls [ProceedAfterLogin\(\)](#). Displays an inline error label if the username is already taken.

7.157.1.3 POST-LOGIN ROUTING

[ProceedAfterLogin\(\)](#) checks whether the user's Watch Later list is empty. Empty -> DATA_INSTRUCTIONS (upload CSV first). Non-empty -> BLEND_CREATION (data already loaded, skip upload). TODO: This check queries the in-memory [YouTubeData](#) object. If data is stored in SQLite and lazy-loaded, the list may appear empty even for returning users. Verify that [SqliteDataManager](#) loads data into the [User](#) object before this check runs.

7.157.1.4 RESET

[Reset\(\)](#) is called by [MainFrame::NavigateTo\(LOGIN\)](#) (i.e. on logout). It clears all fields and returns to the Sign In tab.

Definition in file [LoginPanel.hpp](#).

7.158 LoginPanel.hpp

[Go to the documentation of this file.](#)

```
00001
00039
00040 #pragma once
00041 #include <wx/wx.h>
00042 #include <wx/simplebook.h>
00043 #include "IRefreshablePanel.hpp"
00044 #include "UIPages.hpp"
00045
00046 class AppController;
00047
00052 class LoginPanel : public wxPanel, public IRefreshablePanel {
00053 public:
00054     LoginPanel(wxWindow* parent, AppController& controller, NavigateFn nav,
00055               const wxImage& bgImage = wxNullImage);
00056
00057     void Refresh() override;
00058
00059     // Clear all fields and reset to Sign In tab. Called by MainFrame on logout.
00060     void Reset();
00061
00062 private:
00063     AppController& m_controller;
00064     NavigateFn m_nav;
00065
00066     // Tab toggle buttons
00067     wxButton* m_signinTab;
00068     wxButton* m_registerTab;
00069
00070     // Inner simplebook switches between sign-in and register forms
00071     wxSimplebook* m_formBook;
00072
```


7.159.1 Detailed Description

Implementation for [MainFrame](#).

Author

Jasmine Kumar

Shamar Pennant

Definition in file [MainFrame.cpp](#).

7.160 MainFrame.cpp

[Go to the documentation of this file.](#)

```

00001
00007
00008 #include "MainFrame.hpp"
00009 #include <wx/wx.h>
00010 #include <wx/simplebook.h>
00011 #include <wx/dcbuffer.h>
00012 #include <wx/filename.h>
00013 #include <wx/stdpaths.h>
00014
00015 #include "UIColors.hpp"
00016 #include "ButtonsConfig.hpp"
00017 #include "LoginPanel.hpp"
00018 #include "DataInstructionsPanel.hpp"
00019 #include "BlendFeedPanel.hpp"
00020 #include "UserPanel.hpp"
00021 #include "SettingsPanel.hpp"
00022 #include "BlendCreationPanel.hpp"
00023 #include "BlendChatPanel.hpp"
00024 #include "ActiveBlendsPanel.hpp"
00025 #include "OperationProgressDialog.hpp"
00026 #include "../AppLayer/AppController.hpp"
00027 #include "../AppLayer/AppState.hpp"
00028
00029 namespace {
00030 wxString ResolveResourcePath(const wxString& fileName)
00031 {
00032     /* only search for resources located within src. though structured this way in the event
00033     that more resource organization is needed in the future with extra directories
00034     */
00035     const wxArrayString relativeCandidates = {
00036         "YTBLND_src/resources/" + fileName
00037     };
00038
00039     for (const auto& candidate : relativeCandidates) {
00040         if (wxFileExists(candidate)) {
00041             return candidate;
00042         }
00043     }
00044
00045     wxString exePath(wxStandardPaths::Get().GetExecutablePath());
00046     const wxString exeDir = exePath.GetPath();
00047     const wxArrayString exeRelativeCandidates = {
00048         exeDir + "/../../YTBLND_src/resources/" + fileName
00049     };
00050
00051     for (const auto& candidate : exeRelativeCandidates) {
00052         if (wxFileExists(candidate)) {
00053             return candidate;
00054         }
00055     }
00056
00057     return "";
00058 }
00059 }
00060
00061 // -- Background Panel -----
00062 class ImageBackgroundPanel : public wxPanel {
00063 public:
00064     ImageBackgroundPanel(wxWindow* parent,
00065         const std::unordered_map<int, wxImage>& imageStore,

```

```

00066             int key)
00067             : wxPanel(parent)
00068             , m_imageStore(imageStore)
00069             , m_key(key)
00070         {
00071             SetBackgroundStyle(wxBG_STYLE_PAINT);
00072             Bind(wxEVT_PAINT, &ImageBackgroundPanel::OnPaint, this);
00073         }
00074
00075 private:
00076     const std::unordered_map<int, wxImage>& m_imageStore;
00077     int m_key;
00078
00079     void OnPaint(wxPaintEvent&) {
00080         wxBufferedPaintDC dc(this);
00081         dc.Clear();
00082         auto it = m_imageStore.find(m_key);
00083         if (it == m_imageStore.end() || !it->second.IsOk()) {
00084             return;
00085         }
00086
00087         wxSize sz = GetClientSize();
00088         if (sz.x <= 0 || sz.y <= 0) {
00089             return;
00090         }
00091
00092         wxBitmap scaled(it->second.Scale(sz.x, sz.y, wxIMAGE_QUALITY_HIGH));
00093         dc.DrawBitmap(scaled, 0, 0, true);
00094     }
00095 };
00096
00097 // -- Construction -----
00098 MainFrame::MainFrame(AppController& controller)
00099     : wxFrame(nullptr, wxID_ANY, "YTBLND",
00100               wxDefaultPosition, wxSize(1280, 800),
00101               wxDEFAULT_FRAME_STYLE | wxMAXIMIZE),
00102     controller(controller),
00103     refreshBtn(nullptr),
00104     refreshInProgress(false)
00105 {
00106     // resolve the path to the background image and load it as a wxImage
00107     const wxString bgPath = ResolveResourcePath("checkered_wave_background.jpg");
00108     if (bgPath.empty() || !LoadImage(BG_MAIN, bgPath)) {
00109         wxLogWarning("Background image not found withing resources directory...");
00110     }
00111
00112     // create the outer container panel with the app background colour
00113     auto* bgPanel = new wxPanel(this);
00114     bgPanel->SetBackgroundColour(UIColors::Background);
00115
00116     // placing our new background panel in the book
00117     book = new wxSimplebook(bgPanel, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxBORDER_NONE);
00118
00119     NavigateFn nav = [this](Page p) { NavigateTo(p); };
00120
00121     // --- Initialize pages and wrap each in a panel that paints the background image --
00122
00123     // the login panel - paints its own background image
00124     loginPanel = new LoginPanel(book, controller, nav, images[BG_MAIN]);
00125     book->AddPage(loginPanel, "Login");
00126
00127     auto* dataPageBg = new wxPanel(book);
00128     dataPageBg->SetBackgroundColour(UIColors::Background);
00129     dataInstrPanel = new DataInstructionsPanel(dataPageBg, controller, nav);
00130     {
00131         auto* s = new wxBoxSizer(wxVERTICAL);
00132         s->Add(dataInstrPanel, 1, wxEXPAND);
00133         dataPageBg->SetSizer(s);
00134     }
00135     book->AddPage(dataPageBg, "DataInstructions");
00136
00137     auto* creationPageBg = new wxPanel(book);
00138     creationPageBg->SetBackgroundColour(UIColors::Background);
00139     creationPanel = new BlendCreationPanel(creationPageBg, controller, nav);
00140     {
00141         auto* s = new wxBoxSizer(wxVERTICAL);
00142         s->Add(creationPanel, 1, wxEXPAND);
00143         creationPageBg->SetSizer(s);
00144     }
00145     book->AddPage(creationPageBg, "BlendCreation");
00146
00147     auto* homePageBg = new wxPanel(book);
00148     homePageBg->SetBackgroundColour(UIColors::Background);
00149     feedPanel = new BlendFeedPanel(homePageBg, controller);
00150     {
00151         auto* homePage = BuildHomePage(homePageBg);
00152     }

```

```

00153         auto* s = new wxBoxSizer(wxVERTICAL);
00154         s->Add(homePage, 1, wxEXPAND);
00155         homePageBg->SetSizer(s);
00156     }
00157     book->AddPage(homePageBg, "Home");
00158
00159     auto* userPageBg = new wxPanel(book);
00160     userPageBg->SetBackgroundColour(UIColors::Background);
00161     userPanel = new UserPanel(userPageBg, controller, nav);
00162     {
00163         auto* s = new wxBoxSizer(wxVERTICAL);
00164         s->Add(userPanel, 1, wxEXPAND);
00165         userPageBg->SetSizer(s);
00166     }
00167     book->AddPage(userPageBg, "User");
00168
00169     auto* settingsPageBg = new wxPanel(book);
00170     settingsPageBg->SetBackgroundColour(UIColors::Background);
00171     settingsPanel = new SettingsPanel(settingsPageBg, controller, nav);
00172     {
00173         auto* s = new wxBoxSizer(wxVERTICAL);
00174         s->Add(settingsPanel, 1, wxEXPAND);
00175         settingsPageBg->SetSizer(s);
00176     }
00177     book->AddPage(settingsPageBg, "Settings");
00178
00179     auto* chatPageBg = new wxPanel(book);
00180     chatPageBg->SetBackgroundColour(UIColors::Background);
00181     chatPanel = new BlendChatPanel(chatPageBg, controller, nav);
00182     {
00183         auto* s = new wxBoxSizer(wxVERTICAL);
00184         s->Add(chatPanel, 1, wxEXPAND);
00185         chatPageBg->SetSizer(s);
00186     }
00187     book->AddPage(chatPageBg, "BlendChat");
00188
00189     auto* activeBlendsPageBg = new wxPanel(book);
00190     activeBlendsPageBg->SetBackgroundColour(UIColors::Background);
00191     activeBlendsPanel = new ActiveBlendsPanel(activeBlendsPageBg, controller, nav);
00192     {
00193         auto* s = new wxBoxSizer(wxVERTICAL);
00194         s->Add(activeBlendsPanel, 1, wxEXPAND);
00195         activeBlendsPageBg->SetSizer(s);
00196     }
00197     book->AddPage(activeBlendsPageBg, "ActiveBlends");
00198
00199     // 6. Layout: Book expands inside BackgroundPanel
00200     auto* rootSizer = new wxBoxSizer(wxVERTICAL);
00201     rootSizer->Add(book, 1, wxEXPAND);
00202     bgPanel->SetSizer(rootSizer);
00203
00204     // Layout: BackgroundPanel expands inside the Frame
00205     auto* frameSizer = new wxBoxSizer(wxVERTICAL);
00206     frameSizer->Add(bgPanel, 1, wxEXPAND);
00207     SetSizer(frameSizer);
00208
00209     // -- Theme-change listener -----
00210     controller.getEventRouter().registerListener("theme_updated",
00211         [this](const EventPayload& payload) {
00212             auto it = payload.find("old_theme_index");
00213             if (it == payload.end()) return;
00214
00215             int oldIdx = std::stoi(it->second);
00216             const Palette* palettes[] = { &UIColors::DarkMode, &UIColors::LightMode,
&UIColors::NeonMode };
00217             if (oldIdx < 0 || oldIdx > 2) return;
00218
00219             RecolorAll(*palettes[oldIdx]);
00220         });
00221
00222     Maximize(true);
00223     book->SetSelection(static_cast<int>(Page::HOME));
00224     Layout();
00225     book->SetSelection(static_cast<int>(Page::LOGIN));
00226 }
00227
00228 // -- Navigation -----
00229 void MainFrame::NavigateTo(Page page) {
00230     if (page == Page::LOGIN) loginPanel->Refresh();
00231     if (page == Page::BLEND_CREATION) creationPanel->Refresh();
00232     if (page == Page::HOME) feedPanel->Refresh();
00233     if (page == Page::USER) userPanel->Refresh();
00234     if (page == Page::BLEND_CHAT) chatPanel->Refresh();
00235     if (page == Page::ACTIVE_BLENDS) activeBlendsPanel->Refresh();
00236     book->SetSelection(static_cast<int>(page));
00237 }
00238

```

```

00239 void MainFrame::TriggerFeedRefresh() {
00240     if (refreshInProgress) {
00241         return;
00242     }
00243
00244     refreshInProgress = true;
00245     if (refreshBtn != nullptr) {
00246         refreshBtn->Disable();
00247     }
00248
00249     OperationProgressDialog progress(this, "Refreshing Feed");
00250     progress.ShowModal();
00251     progress.UpdateProgress(0.08, "Preparing refresh...");
00252
00253     controller.setProgressReporter([&progress](double ratio, const std::string& message) {
00254         progress.UpdateProgress(ratio, wxString::FromUTF8(message));
00255     });
00256
00257     // NextPage may either advance the local page offset immediately or trigger
00258     // controller-level refresh work when a new blend sample is required.
00259     progress.UpdateProgress(0.2, "Updating feed...");
00260     try {
00261         feedPanel->NextPage();
00262         progress.UpdateProgress(1.0, "Feed refreshed");
00263     } catch (...) {
00264         controller.clearProgressReporter();
00265         if (refreshBtn != nullptr) {
00266             refreshBtn->Enable();
00267         }
00268         refreshInProgress = false;
00269         throw;
00270     }
00271
00272     controller.clearProgressReporter();
00273     progress.Destroy();
00274     if (refreshBtn != nullptr) {
00275         refreshBtn->Enable();
00276     }
00277     refreshInProgress = false;
00278 }
00279
00280 // -- Build Home Page -----
00281 wxPanel* MainFrame::BuildHomePage(wxWindow* parent) {
00282     auto* page = new wxPanel(parent, wxID_ANY);
00283
00284     auto* vbox = new wxBoxSizer(wxVERTICAL);
00285
00286     auto* topBar = new wxPanel(page, wxID_ANY);
00287     auto* hbox = new wxBoxSizer(wxHORIZONTAL);
00288
00289     auto makeBtn = [&](wxPanel* parent, const wxString& label) {
00290         auto* btn = new wxButton(parent, wxID_ANY, label);
00291         UIButtons::ApplySizeBounds(btn, ButtonType::Nav);
00292         btn->SetBackgroundColour(UIColors::SurfaceRaised);
00293         btn->SetForegroundColour(UIColors::TextPrimary);
00294         return btn;
00295     };
00296
00297     auto* settingsBtn = makeBtn(topBar, "Settings");
00298     auto* userBtn = makeBtn(topBar, "User");
00299     auto* activeBlendsBtn = makeBtn(topBar, "My Blends");
00300     auto* chatBtn = makeBtn(topBar, "Chat");
00301     auto* newBlendBtn = makeBtn(topBar, "New Blend");
00302
00303     settingsBtn->Bind(wxEVT_BUTTON, [this](wxCommandEvent&){ NavigateTo(Page::SETTINGS); });
00304     userBtn->Bind(wxEVT_BUTTON, [this](wxCommandEvent&){ NavigateTo(Page::USER); });
00305     activeBlendsBtn->Bind(wxEVT_BUTTON, [this](wxCommandEvent&){ NavigateTo(Page::ACTIVE_BLENDS); });
00306     chatBtn->Bind(wxEVT_BUTTON, [this](wxCommandEvent&){
00307         if (AppState::getInstance().getActiveBlend() != nullptr)
00308             NavigateTo(Page::BLEND_CHAT);
00309         else
00310             NavigateTo(Page::BLEND_CREATION);
00311     });
00312     newBlendBtn->Bind(wxEVT_BUTTON, [this](wxCommandEvent&){ NavigateTo(Page::BLEND_CREATION); });
00313
00314     hbox->Add(settingsBtn, 1, wxALL|wxEXPAND, 8);
00315     hbox->AddStretchSpacer(1);
00316     hbox->Add(userBtn, 1, wxALL|wxEXPAND, 8);
00317     hbox->Add(activeBlendsBtn, 1, wxALL|wxEXPAND, 8);
00318     hbox->Add(chatBtn, 1, wxALL|wxEXPAND, 8);
00319     hbox->Add(newBlendBtn, 1, wxALL|wxEXPAND, 8);
00320     topBar->SetSizer(hbox);
00321
00322     auto* titlePanel = new wxPanel(page, wxID_ANY);
00323     auto* titleBox = new wxBoxSizer(wxHORIZONTAL);
00324     auto* titleLabel = new wxStaticText(titlePanel, wxID_ANY, "YTBLND");
00325     wxFont titleFont = titleLabel->GetFont();

```

```

00326     titleFont.SetPointSize(32);
00327     titleFont.SetWeight(wxFONTWEIGHT_BOLD);
00328     titleLabel->SetFont(titleFont);
00329     titleLabel->SetForegroundColour(UIColors::Accent);
00330     titleBox->AddStretchSpacer(1);
00331     titleBox->Add(titleLabel, 0, wxALL|wxALIGN_CENTER_VERTICAL, 12);
00332     titleBox->AddStretchSpacer(1);
00333     titlePanel->SetSizer(titleBox);
00334
00335     feedPanel->Reparent(page);
00336
00337     auto* refreshPanel = new wxPanel(page, wxID_ANY);
00338     auto* rbox = new wxBoxSizer(wxHORIZONTAL);
00339     // Store this button on MainFrame so TriggerFeedRefresh can disable/enable it.
00340     refreshBtn = new wxButton(refreshPanel, wxID_ANY, "Refresh");
00341     UIButtons::ApplySizeBounds(refreshBtn, ButtonType::Nav);
00342     const wxString refreshPath = ResolveResourcePath("refresh.png");
00343     if (!refreshPath.empty()) {
00344         wxBitmap refreshBmp(refreshPath, wxBITMAP_TYPE_PNG);
00345         if (refreshBmp.IsOk()) {
00346             refreshBmp = wxBitmap(refreshBmp.ConvertToImage().Rescale(18, 18));
00347             refreshBtn->SetBitmap(refreshBmp);
00348         }
00349     } else {
00350         wxLogWarning("Refresh icon not found; using text-only button.");
00351     }
00352     refreshBtn->Bind(wxEVT_BUTTON, [this](wxCommandEvent&) { TriggerFeedRefresh(); });
00353     rbox->AddStretchSpacer(1);
00354     rbox->Add(refreshBtn, 0, wxALL, 12);
00355     rbox->AddStretchSpacer(1);
00356     refreshPanel->SetSizer(rbox);
00357
00358     vbox->Add(topBar, 0, wxEXPAND);
00359     vbox->Add(titlePanel, 0, wxEXPAND);
00360     vbox->Add(feedPanel, 1, wxEXPAND | wxALL, 16);
00361     vbox->Add(refreshPanel, 0, wxEXPAND);
00362     page->SetSizer(vbox);
00363
00364     return page;
00365 }
00366
00367 // -- Theme recoloring -----
00368
00369 void MainFrame::RecolorAll(const Palette& oldPalette)
00370 {
00371     RecolorWidget(this, oldPalette);
00372     Refresh();
00373     Update();
00374 }
00375
00376 void MainFrame::RecolorWidget(wxWindow* w, const Palette& oldPalette)
00377 {
00378     if (!w) return;
00379
00380     // Map old palette bg colours to new palette equivalents
00381     struct ColourMapping {
00382         wxColour old_col;
00383         wxColour new_col;
00384     };
00385
00386     const Palette& np = *UIColors::Current;
00387     const ColourMapping bgMap[] = {
00388         { oldPalette.Background, np.Background },
00389         { oldPalette.Surface, np.Surface },
00390         { oldPalette.SurfaceRaised, np.SurfaceRaised },
00391         { oldPalette.Accent, np.Accent },
00392         { oldPalette.AccentHover, np.AccentHover },
00393         { oldPalette.Danger, np.Danger },
00394         { oldPalette.Separator, np.Separator },
00395     };
00396
00397     const ColourMapping fgMap[] = {
00398         { oldPalette.TextPrimary, np.TextPrimary },
00399         { oldPalette.TextSecondary, np.TextSecondary },
00400         { oldPalette.TextMuted, np.TextMuted },
00401         { oldPalette.Accent, np.Accent },
00402         { oldPalette.Danger, np.Danger },
00403     };
00404
00405     wxColour bg = w->GetBackgroundColour();
00406     for (const auto& m : bgMap) {
00407         if (bg == m.old_col) {
00408             w->SetBackgroundColour(m.new_col);
00409             break;
00410         }
00411     }
00412 }

```

```

00413     wxColour fg = w->GetForegroundColour();
00414     for (const auto& m : fgMap) {
00415         if (fg == m.old_col) {
00416             w->SetForegroundColour(m.new_col);
00417             break;
00418         }
00419     }
00420
00421     // Recurse into children
00422     for (wxWindow* child : w->GetChildren()) {
00423         RecolorWidget(child, oldPalette);
00424     }
00425 }
00426
00427 bool MainFrame::LoadImage(int key, const wxString& path)
00428 {
00429     wxImage image;
00430     if (!image.LoadFile(path, wxBITMAP_TYPE_ANY) || !image.IsOk()) {
00431         return false;
00432     }
00433
00434     images[key] = image;
00435     return true;
00436 }

```

7.161 MainFrame.hpp File Reference

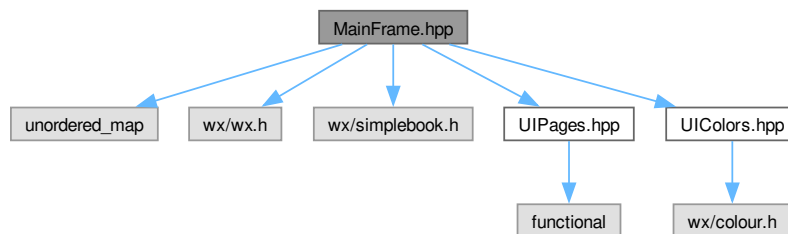
Root application window containing the full-window page switcher.

```

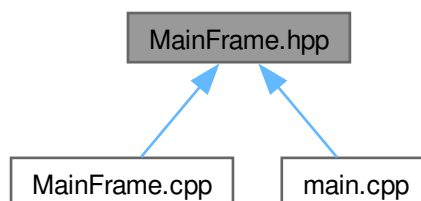
#include <unordered_map>
#include <wx/wx.h>
#include <wx/simplebook.h>
#include "UIPages.hpp"
#include "UIColors.hpp"

```

Include dependency graph for MainFrame.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [MainFrame](#)

Root application window containing the full-window page switcher.

7.161.1 Detailed Description

Root application window containing the full-window page switcher.

Author

Jasmine Kumar

Shamar Pennant

[MainFrame](#) is the single top-level wxFrame. It contains a wxSimplebook that acts as a full-window page switcher (no visible tabs). Every panel is created once at startup and kept alive for the application's lifetime; navigation simply changes which page is visible.

7.161.1.0.1 Page order (must match the Page enum in UIPages.hpp exactly)

Index	Enum	Panel
0	LOGIN	LoginPanel
1	DATA_INSTRUCTIONS	DataInstructionsPanel
2	BLEND_CREATION	BlendCreationPanel
3	HOME	BlendFeedPanel wrapper
4	USER	UserPanel
5	SETTINGS	SettingsPanel
6	BLEND_CHAT	BlendChatPanel

Definition in file [MainFrame.hpp](#).

7.162 MainFrame.hpp

[Go to the documentation of this file.](#)

```

00001
00023
00024 #pragma once
00025 #include <unordered_map>
00026 #include <wx/wx.h>
00027 #include <wx/simplebook.h>
00028 #include "UIPages.hpp"
00029 #include "UIColors.hpp"
00030
00031 class AppController;
00032 class LoginPanel;
00033 class DataInstructionsPanel;
00034 class BlendFeedPanel;
00035 class UserPanel;
00036 class SettingsPanel;
00037 class BlendCreationPanel;
00038 class BlendChatPanel;
00039 class ActiveBlendsPanel;
00040
00048 class MainFrame : public wxFrame {
00049 public:

```

```

00055     explicit MainFrame(AppController& controller);
00056
00062     void NavigateTo(Page page);
00063
00068     void TriggerFeedRefresh();
00069
00070 private:
00071     enum ImageKey {
00072         BG_MAIN = 0,
00073     };
00074
00075     AppController& controller;
00076
00077     wxSimplebook* book;
00078     LoginPanel* loginPanel;
00079     DataInstructionsPanel* dataInstrPanel;
00080     BlendCreationPanel* creationPanel;
00081     BlendFeedPanel* feedPanel;
00082     UserPanel* userPanel;
00083     SettingsPanel* settingsPanel;
00084     BlendChatPanel* chatPanel;
00085     ActiveBlendsPanel* activeBlendsPanel;
00086     // Home-page Refresh button reference so we can disable it during long refreshes.
00087     wxButton* refreshBtn;
00088     // Guards against duplicate refresh dispatches from double-clicks.
00089     bool refreshInProgress;
00090     std::unordered_map<int, wxImage> images; // images
00091
00096     wxPanel* BuildHomePage(wxWindow* parent);
00097     wxPanel* BuildSettingsPage(wxWindow* parent);
00098
00099     void OnSettings(wxCommandEvent&);
00100     void OnUser (wxCommandEvent&);
00101     void OnBlend (wxCommandEvent&);
00102     void OnRefresh (wxCommandEvent&);
00103
00108     void RecolorAll(const Palette& oldPalette);
00109
00111     void RecolorWidget(wxWindow* w, const Palette& oldPalette);
00112
00113     bool LoadImage(int key, const wxString& path);
00114 };

```

7.163 OperationProgressDialog.cpp File Reference

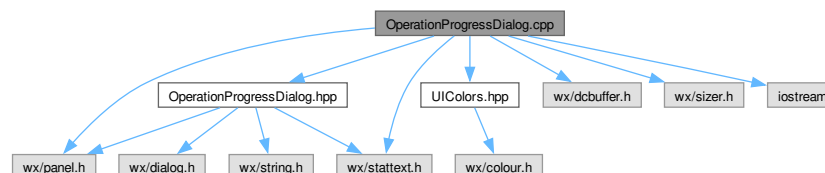
Implementation of non-modal progress dialog.

```

#include "OperationProgressDialog.hpp"
#include "UIColors.hpp"
#include <wx/dcbuffer.h>
#include <wx/sizer.h>
#include <wx/stattext.h>
#include <wx/panel.h>
#include <iostream>

```

Include dependency graph for OperationProgressDialog.cpp:



7.163.1 Detailed Description

Implementation of non-modal progress dialog.

Author

Shamar Pennant

Definition in file [OperationProgressDialog.cpp](#).

7.164 OperationProgressDialog.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "OperationProgressDialog.hpp"
00008 #include "UIColors.hpp"
00009
00010 #include <wx/dcbuffer.h>
00011 #include <wx/sizer.h>
00012 #include <wx/stattext.h>
00013 #include <wx/panel.h>
00014 #include <iostream>
00015
00016 namespace {
00017 constexpr int kProgressBorderPx = 1;
00018 }
00019
00020 OperationProgressDialog::OperationProgressDialog(wxWindow* parent, const wxString& title)
00021 : wxDialog(parent, wxID_ANY, title,
00022           wxDefaultPosition, wxSize(420, 180),
00023           wxDEFAULT_DIALOG_STYLE | wxRESIZE_BORDER) {
00024
00025     // Create main vertical sizer
00026     auto* mainSizer = new wxBoxSizer(wxVERTICAL);
00027     mainSizer->AddSpacer(10);
00028
00029     // Status message label
00030     messageLabel = new wxStaticText(this, wxID_ANY, wxT("Initializing..."));
00031     wxFont messageFont = messageLabel->GetFont();
00032     messageFont.SetPointSize(10);
00033     messageLabel->SetFont(messageFont);
00034     messageLabel->SetForegroundColour(UIColors::TextPrimary);
00035     mainSizer->Add(messageLabel, 0, wxEXPAND | wxLEFT | wxRIGHT, 20);
00036     mainSizer->AddSpacer(8);
00037
00038     // Custom progress bar (track + fill) to ensure theme colors render on GTK.
00039     progressTrack = new wxPanel(this, wxID_ANY, wxDefaultPosition, wxSize(-1, 14));
00040     progressTrack->SetMinSize(wxSize(-1, 14));
00041     progressTrack->SetBackgroundStyle(wxBG_STYLE_PAINT);
00042     progressTrack->SetBackgroundColour(UIColors::SurfaceRaised);
00043
00044     // Paint a consistent border for visibility in all themes.
00045     progressTrack->Bind(wxEVT_PAINT, [this](wxPaintEvent&) {
00046         wxAutoBufferedPaintDC dc(progressTrack);
00047         const wxSize sz = progressTrack->GetClientSize();
00048
00049         dc.SetPen(*wxTRANSPARENT_PEN);
00050         dc.SetBrush(wxBrush(UIColors::Separator));
00051         dc.DrawRectangle(0, 0, sz.GetWidth(), sz.GetHeight());
00052
00053         const int innerW = std::max(0, sz.GetWidth() - (2 * kProgressBorderPx));
00054         const int innerH = std::max(0, sz.GetHeight() - (2 * kProgressBorderPx));
00055         dc.SetBrush(wxBrush(UIColors::SurfaceRaised));
00056         dc.DrawRectangle(kProgressBorderPx, kProgressBorderPx, innerW, innerH);
00057     });
00058
00059     progressFill = new wxPanel(progressTrack, wxID_ANY, wxDefaultPosition, wxSize(0, 14));
00060     progressFill->SetBackgroundColour(UIColors::Accent);
00061
00062     progressTrack->Bind(wxEVT_SIZE, [this](wxSizeEvent& evt) {
00063         UpdateProgressFill();
00064         evt.Skip();
00065     });
00066
00067     mainSizer->Add(progressTrack, 0, wxEXPAND | wxLEFT | wxRIGHT, 20);
00068
00069     // Percentage label centred below the progress bar.
00070     percentLabel = new wxStaticText(this, wxID_ANY, wxT("0%"),
00071                                   wxDefaultPosition, wxDefaultSize,
00072                                   wxALIGN_CENTRE_HORIZONTAL);
00073     wxFont percentFont = percentLabel->GetFont();
00074     percentFont.SetPointSize(9);

```

```

00075     percentLabel->SetFont(percentFont);
00076     percentLabel->SetForegroundColour(UIColors::TextSecondary);
00077     mainSizer->AddSpacer(6);
00078     mainSizer->Add(percentLabel, 0, wxALIGN_CENTER_HORIZONTAL);
00079     mainSizer->AddStretchSpacer(1);
00080
00081     SetSizer(mainSizer);
00082     SetBackgroundColour(UIColors::Surface);
00083     SetWindowStyle(wxDEFAULT_DIALOG_STYLE | wxRESIZE_BORDER);
00084     UpdateProgressFill();
00085
00086     std::cout << "[OperationProgressDialog] Created progress dialog: "
00087               << std::string(title.mb_str()) << "\n";
00088 }
00089
00090 OperationProgressDialog::~OperationProgressDialog() {
00091     std::cout << "[OperationProgressDialog] Destroyed progress dialog\n";
00092 }
00093
00094 void OperationProgressDialog::UpdateProgress(double progress, const wxString& message) {
00095     // Clamp progress to 0.0-1.0 range
00096     if (progress < 0.0) progress = 0.0;
00097     if (progress > 1.0) progress = 1.0;
00098
00099     // Update custom progress fill
00100     currentProgress = progress;
00101     UpdateProgressFill();
00102
00103     // Update percentage label
00104     int percent = static_cast<int>(progress * 100);
00105     percentLabel->SetLabel(wxString::Format(wxT("%d%%"), percent));
00106
00107     // Update message if provided
00108     if (!message.IsEmpty()) {
00109         messageLabel->SetLabel(message);
00110     }
00111
00112     // Redraw dialog
00113     Layout();
00114     Refresh();
00115     wxYield();
00116 }
00117
00118 int OperationProgressDialog::ShowModal() {
00119     // Display non-modally to keep application responsive
00120     Show(true);
00121     return wxID_OK;
00122 }
00123
00124 void OperationProgressDialog::UpdateProgressFill() {
00125     if (progressTrack == nullptr || progressFill == nullptr) {
00126         return;
00127     }
00128
00129     const wxSize trackSize = progressTrack->GetClientSize();
00130     const int innerW = std::max(0, trackSize.GetWidth() - (2 * kProgressBorderPx));
00131     const int innerH = std::max(0, trackSize.GetHeight() - (2 * kProgressBorderPx));
00132     const int fillWidth = static_cast<int>(innerW * currentProgress);
00133
00134     progressFill->SetSize(kProgressBorderPx,
00135                          kProgressBorderPx,
00136                          std::max(0, fillWidth),
00137                          innerH);
00138
00139     progressTrack->Refresh();
00140     progressFill->Refresh();
00141 }

```

7.165 OperationProgressDialog.hpp File Reference

Dark-themed non-modal progress dialog for file upload/processing.

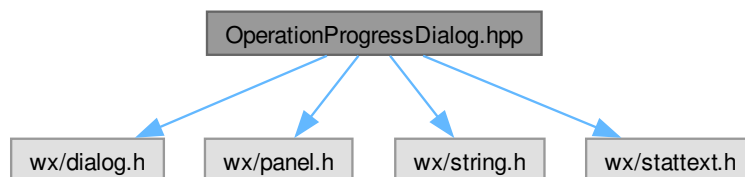
```

#include <wx/dialog.h>
#include <wx/panel.h>
#include <wx/string.h>

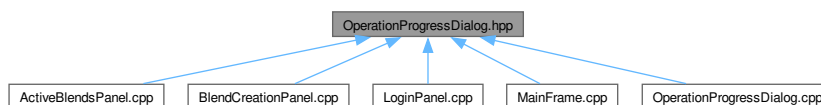
```

```
#include <wx/stattext.h>
```

Include dependency graph for OperationProgressDialog.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [OperationProgressDialog](#)
Non-modal progress dialog showing upload/processing progress.

7.165.1 Detailed Description

Dark-themed non-modal progress dialog for file upload/processing.

Author

Shamar Pennant

Displays progress bar, percentage, and status message during long operations. Helping to indicate to the user the progress of file uploading

Definition in file [OperationProgressDialog.hpp](#).

7.167.1 Detailed Description

Implementation for [SettingsPanel](#).

Author

Jasmine Kumar

Xavier Wah-Huen Lusetti

Definition in file [SettingsPanel.cpp](#).

7.168 SettingsPanel.cpp

[Go to the documentation of this file.](#)

```

00001
00007
00008 // =====
00009 // SettingsPanel.cpp
00010 // =====
00011
00012 #include "SettingsPanel.hpp"
00013
00014 #include <wx/statline.h>
00015 #include <wx/choice.h>
00016
00017 #include "UIColors.hpp"
00018 #include "ButtonsConfig.hpp"
00019 #include "UIPages.hpp"
00020 #include "TopBar.hpp"
00021 #include "ConfirmationDialog.hpp"
00022 #include "../AppLayer/AppController.hpp"
00023 #include "../AppLayer/AppState.hpp"
00024 #include "../AppLayer/EventRouter.hpp"
00025 #include "../ModelLayer/User.hpp"
00026
00027 // -- Construction -----
00028
00029 SettingsPanel::SettingsPanel(wxWindow* parent, AppController& controller, NavigateFn nav)
00030 : wxPanel(parent, wxID_ANY)
00031 , m_controller(controller)
00032 , m_nav(std::move(nav))
00033 {
00034     SetBackgroundColour(UIColors::Current->Background);
00035
00036     auto* root = new wxBoxSizer(wxVERTICAL);
00037
00038     // -- TopBar -----
00039     auto* topBar = new TopBar(this, "Settings", m_nav, Page::HOME);
00040     root->Add(topBar, 0, wxEXPAND);
00041
00042     // -- Theme Selection -----
00043     auto* themePanel = new wxPanel(this, wxID_ANY);
00044     themePanel->SetBackgroundColour(UIColors::Current->Surface);
00045     auto* themeSizer = new wxBoxSizer(wxHORIZONTAL);
00046
00047     auto* themeLabel = new wxStaticText(themePanel, wxID_ANY, "App Theme:");
00048     themeLabel->SetForegroundColour(UIColors::Current->TextPrimary);
00049
00050     // Theme Options
00051     wxArrayString choices;
00052     choices.Add("Dark Mode");
00053     choices.Add("Light Mode");
00054     choices.Add("Neon Mode");
00055
00056     m_themeChoice = new wxChoice(themePanel, wxID_ANY, wxDefaultPosition, wxDefaultSize, choices);
00057     m_themeChoice->SetSelection(0); // Default to first item
00058     m_themeChoice->Bind(wxEVT_CHOICE, &SettingsPanel::OnThemeChanged, this);
00059
00060     themeSizer->Add(themeLabel, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 10);
00061     themeSizer->Add(m_themeChoice, 1, wxEXPAND);
00062
00063     themePanel->SetSizer(themeSizer);
00064
00065     // Add it to the root sizer with padding

```

```

00066     root->Add(themePanel, 0, wxEXPAND | wxLEFT | wxRIGHT | wxTOP, 16);
00067
00068     // -- Divider -----
00069     auto* divider = new wxStaticLine(this, wxID_ANY);
00070     divider->SetBackgroundColour(UIColors::Current->Separator);
00071     root->Add(divider, 0, wxEXPAND);
00072
00073     // -- Spacer pushes logout to bottom -----
00074     root->AddStretchSpacer(1);
00075
00076     // -- Logout -----
00077     auto* logoutBtn = new wxButton(this, wxID_ANY, "Log Out");
00078     UIButtons::ApplySizeBounds(logoutBtn, ButtonType::FullWidthDanger);
00079     logoutBtn->SetBackgroundColour(UIColors::Current->Danger);
00080     logoutBtn->SetForegroundColour(UIColors::Current->TextPrimary);
00081     root->Add(logoutBtn, 0, wxEXPAND | wxLEFT | wxRIGHT | wxBOTTOM, 16);
00082
00083     SetSizer(root);
00084
00085     logoutBtn->Bind(wxEVT_BUTTON, &SettingsPanel::OnLogout, this);
00086     Bind(wxEVT_SHOW, &SettingsPanel::OnShow, this);
00087
00088 }
00089
00090 // -- Private helpers -----
00091
00092 void SettingsPanel::RefreshUserInfo()
00093 {
00094     User* user = AppState::getInstance().getCurrentUser();
00095 }
00096
00097 // -- Event handlers -----
00098
00099 void SettingsPanel::OnShow(wxShowEvent& evt)
00100 {
00101     if (evt.IsShown())
00102         RefreshUserInfo();
00103     evt.Skip();
00104 }
00105
00106 void SettingsPanel::OnThemeChanged(wxCommandEvent& /*evt*/) {
00107     int sel = m_themeChoice->GetSelection();
00108     ThemeType newTheme = static_cast<ThemeType>(sel);
00109
00110     // Record the old theme index before switching
00111     int oldIndex = static_cast<int>(UIColors::GetCurrentTheme());
00112
00113     // Swap the global color palette
00114     UIColors::SetTheme(newTheme);
00115
00116     // Notify MainFrame to recolor all panels, passing the old theme index
00117     m_controller.getEventRouter().dispatch("theme_updated",
00118         {"old_theme_index", std::to_string(oldIndex)});
00119 }
00120
00121 void SettingsPanel::OnLogout(wxCommandEvent& /*evt*/)
00122 {
00123     bool confirmed = ConfirmationDialog::Confirm(this, "Log Out", "Log out?");
00124     if (!confirmed) return;
00125
00126     m_controller.getEventRouter().dispatch("logout", {});
00127     m_nav(Page::LOGIN);
00128 }

```

7.169 SettingsPanel.hpp File Reference

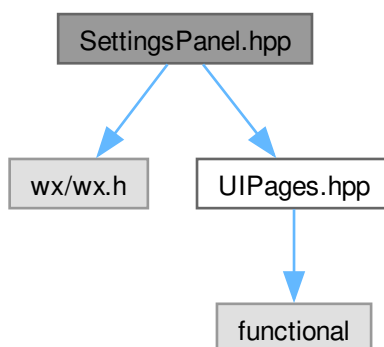
Theme selection and logout panel.

```

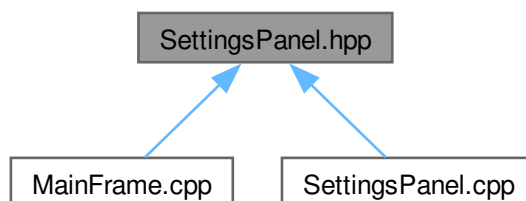
#include <wx/wx.h>
#include "UIPages.hpp"

```

Include dependency graph for SettingsPanel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [SettingsPanel](#)
[SettingsPanel](#) class declaration.

7.169.1 Detailed Description

Theme selection and logout panel.

Author

Jasmine Kumar
Xavier Wah-Huen Lusetti

Definition in file [SettingsPanel.hpp](#).

7.170 SettingsPanel.hpp

[Go to the documentation of this file.](#)

```

00001
00007
00008 #pragma once
00009
00010 #include <wx/wx.h>
00011
00012 #include "UIPages.hpp"
00013
00014 class AppController;
00015 class wxChoice;
00016
00021 class SettingsPanel : public wxPanel {
00022 public:
00023     SettingsPanel(wxWindow* parent, AppController& controller, NavigateFn nav);
00024
00025 private:
00026     AppController& m_controller;
00027     NavigateFn     m_nav;
00028     wxChoice*      m_themeChoice = nullptr;
00029
00030     void RefreshUserInfo();
00031
00032     void OnShow(wxShowEvent& evt);
00033     void OnThemeChanged(wxCommandEvent& evt);
00034     void OnLogout(wxCommandEvent& evt);
00035 };

```

7.171 TopBar.cpp File Reference

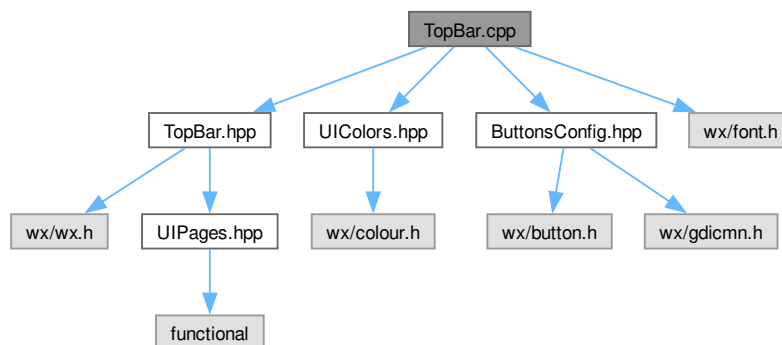
Implementation for [TopBar](#).

```

#include "TopBar.hpp"
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include <wx/font.h>

```

Include dependency graph for TopBar.cpp:



7.171.1 Detailed Description

Implementation for [TopBar](#).

Author

Jasmine Kumar

Definition in file [TopBar.cpp](#).

7.172 TopBar.cpp

[Go to the documentation of this file.](#)

```

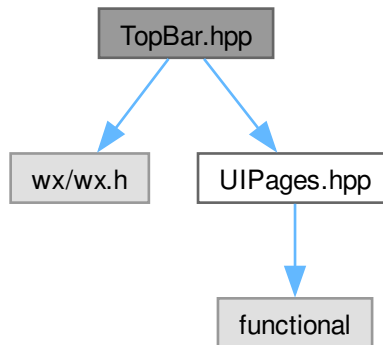
00001
00006
00007 // =====
00008 // TopBar.cpp - Reusable interior-page header bar implementation
00009 // =====
00010
00011 #include "TopBar.hpp"
00012 #include "UIColors.hpp"
00013 #include "ButtonsConfig.hpp"
00014 #include <wx/font.h>
00015
00016 TopBar::TopBar(wxWindow* parent, const wxString& title, NavigateFn nav, Page backDest)
00017     : wxPanel(parent, wxID_ANY)
00018     , m_nav(std::move(nav))
00019     , m_backDest(backDest)
00020 {
00021     SetBackgroundColour(UIColors::Surface);
00022
00023     // -- Back button -----
00024     wxButton* backBtn = new wxButton(this, wxID_ANY, wxT("< Back"));
00025     backBtn->SetBackgroundColour(UIColors::SurfaceRaised);
00026     backBtn->SetForegroundColour(UIColors::TextPrimary);
00027     UIButtons::ApplySizeBounds(backBtn, ButtonType::TopBarBack);
00028
00029     backBtn->Bind(wxEVT_BUTTON, &TopBar::OnBack, this);
00030
00031     // Lighten on hover for subtle feedback
00032     backBtn->Bind(wxEVT_ENTER_WINDOW, [backBtn](wxMouseEvent& e) {
00033         backBtn->SetBackgroundColour(wxColour(55, 55, 55));
00034         backBtn->Refresh();
00035         e.Skip();
00036     });
00037     backBtn->Bind(wxEVT_LEAVE_WINDOW, [backBtn](wxMouseEvent& e) {
00038         backBtn->SetBackgroundColour(UIColors::SurfaceRaised);
00039         backBtn->Refresh();
00040         e.Skip();
00041     });
00042
00043     // -- Title label -----
00044     wxStaticText* titleLabel = new wxStaticText(this, wxID_ANY, title,
00045                                                 wxDefaultPosition, wxDefaultSize,
00046                                                 wxALIGN_CENTER_HORIZONTAL);
00047     titleLabel->SetForegroundColour(UIColors::TextPrimary);
00048
00049     wxFont titleFont = titleLabel->GetFont();
00050     titleFont.SetPointSize(14);
00051     titleFont.SetWeight(wxFONTWEIGHT_BOLD);
00052     titleLabel->SetFont(titleFont);
00053
00054     // -- Layout -----
00055     // Three-column sizer: [back btn | title (stretch) | right spacer]
00056     // The right spacer mirrors the back button width so the title stays centred.
00057     wxBoxSizer* sizer = new wxBoxSizer(wxHORIZONTAL);
00058
00059     sizer->Add(backBtn, 0, wxALIGN_CENTER_VERTICAL | wxLEFT, 10);
00060     sizer->AddStretchSpacer(1);
00061     sizer->Add(titleLabel, 0, wxALIGN_CENTER_VERTICAL);
00062     sizer->AddStretchSpacer(1);
00063     // Invisible right-side spacer equal to back-button width + left margin
00064     sizer->Add(UIButtons::GetSize(ButtonType::TopBarBack).GetWidth() + 10, 0);
00065
00066     wxBoxSizer* outerSizer = new wxBoxSizer(wxVERTICAL);
00067     outerSizer->Add(sizer, 1, wxEXPAND | wxTOP | wxBOTTOM, 8);
00068
00069     SetSizer(outerSizer);
00070     SetMinSize(wxSize(-1, 48));
00071 }
00072
00073 void TopBar::OnBack(wxCommandEvent& /*evt*/)
00074 {
00075     if (m_nav) {
00076         m_nav(m_backDest);
00077     }
00078 }

```

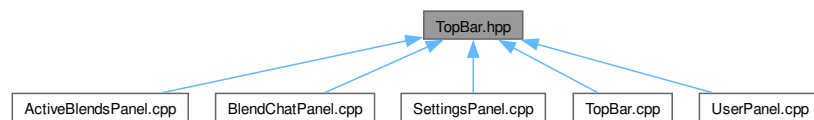
7.173 TopBar.hpp File Reference

Reusable interior-page header bar.

```
#include <wx/wx.h>
#include "UIPages.hpp"
Include dependency graph for TopBar.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [TopBar](#)
Reusable horizontal top bar for interior pages.

7.173.1 Detailed Description

Reusable interior-page header bar.

Author

Jasmine Kumar

Displays a left-aligned back button and a centred title label for interior pages. The back button triggers the provided [NavigateFn](#) callback with the configured destination page.

Definition in file [TopBar.hpp](#).

7.174 TopBar.hpp

[Go to the documentation of this file.](#)

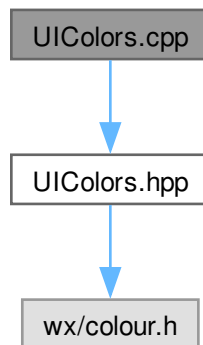
```
00001
00010
00011 #pragma once
00012 #include <wx/wx.h>
00013 #include "UIPages.hpp"
00014
00019 class TopBar : public wxPanel {
00020 public:
00028     TopBar(wxWindow* parent, const wxString& title, NavigateFn nav, Page backDest);
00029
00030 private:
00035     void OnBack(wxCommandEvent& evt);
00036
00038     NavigateFn m_nav;
00040     Page      m_backDest;
00041 };
```

7.175 UIColors.cpp File Reference

Implementation for [UIColors](#).

```
#include "UIColors.hpp"
```

Include dependency graph for UIColors.cpp:



Variables

- static ThemeType [sCurrentTheme](#) = ThemeType::Dark

7.175.1 Detailed Description

Implementation for [UIColors](#).

Author

Jasmine Kumar

Xavier Wah-Huen Lusetti

Definition in file [UIColors.cpp](#).

7.175.2 Variable Documentation

7.175.2.1 sCurrentTheme

ThemeType sCurrentTheme = ThemeType::Dark [static]

Definition at line 89 of file [UIColors.cpp](#).

7.176 UIColors.cpp

[Go to the documentation of this file.](#)

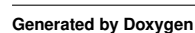
```

00001
00002
00003 // UIColors.cpp
00004 #include "UIColors.hpp"
00005
00006 namespace {
00007 void ApplyPalette(const Palette& p) {
00008     UIColors::Background = p.Background;
00009     UIColors::Surface = p.Surface;
00010     UIColors::SurfaceRaised = p.SurfaceRaised;
00011     UIColors::Accent = p.Accent;
00012     UIColors::AccentHover = p.AccentHover;
00013     UIColors::TextPrimary = p.TextPrimary;
00014     UIColors::TextSecondary = p.TextSecondary;
00015     UIColors::TextMuted = p.TextMuted;
00016     UIColors::Separator = p.Separator;
00017     UIColors::Danger = p.Danger;
00018 }
00019 }
00020
00021 Palette* UIColors::Current = &UIColors::DarkMode;
00022
00023 Palette UIColors::DarkMode = {
00024     wxColour(13,13,13),
00025     wxColour(25,25,25),
00026     wxColour(38,38,38),
00027     wxColour(138,0,255),
00028     wxColour(160,40,255),
00029     wxColour(255,255,255),
00030     wxColour(160,160,160),
00031     wxColour(100,100,100),
00032     wxColour(50,50,50),
00033     wxColour(211, 47, 47)
00034 };
00035
00036 Palette UIColors::LightMode = {
00037     wxColour(245, 245, 245),
00038     wxColour(255, 255, 255),
00039     wxColour(230, 230, 230),
00040     wxColour(0, 102, 204),
00041     wxColour(0, 120, 215),
00042     wxColour(33, 33, 33),
00043     wxColour(117, 117, 117),
00044     wxColour(189, 189, 189),
00045     wxColour(224, 224, 224),
00046     wxColour(211, 47, 47)
00047 };
00048
00049 Palette UIColors::NeonMode = {
00050     wxColour(0, 0, 0),
00051     wxColour(15, 15, 15),
00052     wxColour(30, 30, 30),
00053     wxColour(57, 255, 20),
00054     wxColour(102, 255, 0),
00055     wxColour(255, 255, 255),
00056

```

7.177 UIColors.hpp File Reference

Include dependency graph for UIColors.hpp:



Classes

- struct [Palette](#)
Palette class declaration.

Namespaces

- namespace [UIColors](#)
Shared dark-theme colour constants for the YTB LND UI.

Enumerations

- enum class **ThemeType** { **Dark** = 0 , **Light** = 1 , **Neon** = 2 }

Functions

- void [UIColors::SetTheme](#) (ThemeType theme)
- ThemeType [UIColors::GetCurrentTheme](#) ()

Variables

- [Palette](#) [UIColors::DarkMode](#)
- [Palette](#) [UIColors::LightMode](#)
- [Palette](#) [UIColors::NeonMode](#)
- [Palette](#) * [UIColors::Current](#) = &[UIColors::DarkMode](#)
- wxColour [UIColors::Background](#) = [UIColors::DarkMode.Background](#)
- wxColour [UIColors::Surface](#) = [UIColors::DarkMode.Surface](#)
- wxColour [UIColors::SurfaceRaised](#) = [UIColors::DarkMode.SurfaceRaised](#)
- wxColour [UIColors::Accent](#) = [UIColors::DarkMode.Accent](#)
- wxColour [UIColors::AccentHover](#) = [UIColors::DarkMode.AccentHover](#)
- wxColour [UIColors::TextPrimary](#) = [UIColors::DarkMode.TextPrimary](#)
- wxColour [UIColors::TextSecondary](#) = [UIColors::DarkMode.TextSecondary](#)
- wxColour [UIColors::TextMuted](#) = [UIColors::DarkMode.TextMuted](#)
- wxColour [UIColors::Separator](#) = [UIColors::DarkMode.Separator](#)
- wxColour [UIColors::Danger](#) = [UIColors::DarkMode.Danger](#)

7.177.1 Detailed Description

Shared dark-theme colour palette used throughout the UI.

Author

Jasmine Kumar
Xavier Wah-Huen Lusetti

All wxColour constants live here so the visual style is consistent and easy to retheme. Include this header instead of hard-coding colours in individual panels.

[Palette](#) (dark OLED-friendly theme):

Name	Hex	Usage
Background	#0D0D0D	Window / page background
Surface	#191919	Card / panel backgrounds
SurfaceRaised	#262626	Inputs, raised elements
Accent	#8A00FF	Brand purple - buttons, hover
AccentHover	#A028FF	Lighter purple for hover feedback
TextPrimary	#FFFFFF	Body text
TextSecondary	#A0A0A0	Secondary / caption text
TextMuted	#646464	Placeholder / disabled text
Separator	#323232	Rule lines
Danger	#DC3232	Destructive-action buttons

Definition in file [UIColors.hpp](#).

7.177.2 Enumeration Type Documentation

7.177.2.1 ThemeType

```
enum class ThemeType [strong]
```

Definition at line 29 of file [UIColors.hpp](#).

7.178 UIColors.hpp

[Go to the documentation of this file.](#)

```

00001
00025
00026 #pragma once
00027 #include <wx/colour.h>
00028
00029 enum class ThemeType {
00030     Dark = 0,
00031     Light = 1,
00032     Neon = 2
00033 };
00034
00039 struct Palette {
00040     wxColour Background;
00041     wxColour Surface;
00042     wxColour SurfaceRaised;
00043
00044     wxColour Accent;
00045     wxColour AccentHover;
00046
00047     wxColour TextPrimary;
00048     wxColour TextSecondary;
00049     wxColour TextMuted;
00050
00051     wxColour Separator;
00052     wxColour Danger;
00053 };
00054
00058 namespace UIColors {
00059     extern Palette DarkMode;
00060     extern Palette LightMode;
00061     extern Palette NeonMode;
00062     extern Palette* Current;
00063
00064     extern wxColour Background;
00065     extern wxColour Surface;
00066     extern wxColour SurfaceRaised;
00067     extern wxColour Accent;

```

```

00068     extern wxColour AccentHover;
00069     extern wxColour TextPrimary;
00070     extern wxColour TextSecondary;
00071     extern wxColour TextMuted;
00072     extern wxColour Separator;
00073     extern wxColour Danger;
00074
00075     void SetTheme(ThemeType theme);
00076
00080     ThemeType GetCurrentTheme();
00081 }

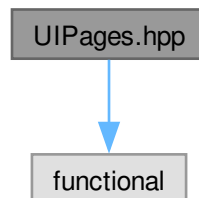
```

7.179 UIPages.hpp File Reference

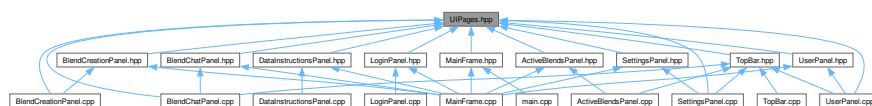
[Page](#) enum and [NavigateFn](#) callback type for the [MainFrame](#) page switcher.

```
#include <functional>
```

Include dependency graph for UIPages.hpp:



This graph shows which files directly or indirectly include this file:



Typedefs

- using [NavigateFn](#) = std::function<void([Page](#))>
Callback type passed to every sub-panel for initiating page transitions.

Enumerations

- enum class [Page](#) {
[LOGIN](#) = 0 , [DATA_INSTRUCTIONS](#) = 1 , [BLEND_CREATION](#) = 2 , [HOME](#) = 3 ,
[USER](#) = 4 , [SETTINGS](#) = 5 , [BLEND_CHAT](#) = 6 , [ACTIVE_BLENDS](#) = 7 }
All top-level pages in the [MainFrame](#) wxSimplebook.

7.179.1 Detailed Description

[Page](#) enum and [NavigateFn](#) callback type for the [MainFrame](#) page switcher.

Author

Jasmine Kumar

Defines every top-level page hosted in the [MainFrame](#) wxSimplebook and the [NavigateFn](#) callback type passed to sub-panels for page switching.

Pages must be added to the wxSimplebook in the same order as the [Page](#) enum values (0, 1, 2, ...). Sub-panels call `m_nav(Page::TARGET)` to navigate without depending on [MainFrame](#) directly.

7.179.1.0.1 Adding a new page

1. Add a new enumerator at the end of [Page](#) (keep values contiguous).
2. Create and add the corresponding wxPanel in `MainFrame::BuildPages()` at the matching position.
3. If the page needs reset/reload on arrival, add the call in `MainFrame::NavigateTo()`.

Definition in file [UIPages.hpp](#).

7.179.2 Typedef Documentation

7.179.2.1 NavigateFn

```
using NavigateFn = std::function<void(Page)>
```

Callback type passed to every sub-panel for initiating page transitions.

Call `m_nav(Page::SOME_PAGE)` to trigger a page switch in [MainFrame](#).

Definition at line 45 of file [UIPages.hpp](#).

7.179.3 Enumeration Type Documentation

7.179.3.1 Page

```
enum class Page [strong]
```

All top-level pages in the [MainFrame](#) wxSimplebook.

Numeric values must match the order panels are added in `MainFrame::BuildPages()`.

Enumerator

LOGIN	Sign-in / register screen.
DATA_INSTRUCTIONS	Shown after login when the user has no YouTube data.
BLEND_CREATION	Add participants and create a new blend.
HOME	Blend feed (3x2 grid + refresh).
USER	Account info and logout.
SETTINGS	Settings (stub - back button only).
BLEND_CHAT	Chat tied to the active blend.
ACTIVE_BLENDS	List of blends the user participates in.

Definition at line 29 of file [UIPages.hpp](#).

7.180 UIPages.hpp

[Go to the documentation of this file.](#)

```

00001
00020
00021 #pragma once
00022 #include <functional>
00023
00029 enum class Page {
00030     LOGIN                = 0,
00031     DATA_INSTRUCTIONS  = 1,
00032     BLEND_CREATION      = 2,
00033     HOME                = 3,
00034     USER               = 4,
00035     SETTINGS            = 5,
00036     BLEND_CHAT         = 6,
00037     ACTIVE_BLENDS      = 7
00038 };
00039
00045 using NavigateFn = std::function<void(Page)>;

```

7.181 UserPanel.cpp File Reference

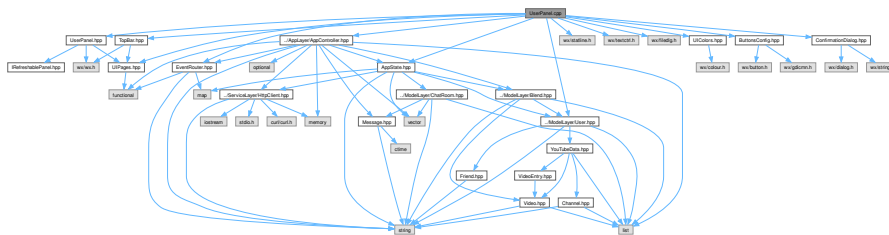
Implementation for [UserPanel](#).

```

#include "UserPanel.hpp"
#include <wx/statline.h>
#include <wx/textctrl.h>
#include <wx/filedlg.h>
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include "UIPages.hpp"
#include "TopBar.hpp"
#include "ConfirmationDialog.hpp"
#include "../AppLayer/AppController.hpp"
#include "../AppLayer/AppState.hpp"
#include "../AppLayer/EventRouter.hpp"
#include "../ModelLayer/User.hpp"

```

Include dependency graph for UserPanel.cpp:



7.181.1 Detailed Description

Implementation for [UserPanel](#).

Author

Jasmine Kumar

Definition in file [UserPanel.cpp](#).

7.182 UserPanel.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 // =====
00008 // UserPanel.cpp - Account info and logout screen implementation
00009 // =====
00010
00011 #include "UserPanel.hpp"
00012
00013 #include <wx/statline.h>
00014 #include <wx/textctrl.h>
00015 #include <wx/filedlg.h>
00016
00017 #include "UIColors.hpp"
00018 #include "ButtonsConfig.hpp"
00019 #include "UIPages.hpp"
00020 #include "TopBar.hpp"
00021 #include "ConfirmationDialog.hpp"
00022 #include "../AppLayer/AppController.hpp"
00023 #include "../AppLayer/AppState.hpp"
00024 #include "../AppLayer/EventRouter.hpp"
00025 #include "../ModelLayer/User.hpp"
00026
00027 // -- Construction -----
00028
00029 UserPanel::UserPanel(wxWindow* parent, AppController& controller, NavigateFn nav)
00030 : wxPanel(parent, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTRANSPARENT_WINDOW)
00031 , m_controller(controller)
00032 , m_nav(std::move(nav))
00033 {
00034     auto* root = new wxBoxSizer(wxVERTICAL);
00035
00036     // -- TopBar -----
00037     auto* topBar = new TopBar(this, "My Account", m_nav, Page::HOME);
00038     root->Add(topBar, 0, wxEXPAND);
00039
00040     // -- User info section -----
00041     auto* infoPanel = new wxPanel(this, wxID_ANY);
00042     infoPanel->SetBackgroundColour(UIColors::Surface);
00043     auto* infoSizer = new wxBoxSizer(wxVERTICAL);
00044
00045     m_usernameLabel = new wxStaticText(infoPanel, wxID_ANY, "Username: Not logged in");
00046     m_usernameLabel->SetForegroundColour(UIColors::TextPrimary);
00047     wxFont uf = m_usernameLabel->GetFont(); // grab a font
00048     uf.SetPointSize(24);
00049     m_usernameLabel->SetFont(uf);
00050
00051     m_emailLabel = new wxStaticText(infoPanel, wxID_ANY, "Email: ");
00052     m_emailLabel->SetForegroundColour(UIColors::TextSecondary);
00053     uf = m_emailLabel->GetFont();
00054     uf.SetPointSize(16);
00055     m_emailLabel->SetFont(uf);
00056
00057     // -- Divider -----
00058     auto* divider = new wxStaticLine(this, wxID_ANY);
00059     divider->SetBackgroundColour(UIColors::Separator);
00060     root->Add(divider, 0, wxEXPAND);
00061
00062     // -- Spacer pushes buttons to bottom -----
00063     root->AddStretchSpacer(2);
00064
00065     infoSizer->Add(m_usernameLabel, 0, wxCENTER | wxTOP, 16);
00066     infoSizer->Add(m_emailLabel, 0, wxCENTER | wxBOTTOM, 16);
00067     infoPanel->SetSizer(infoSizer);
00068     root->Add(infoPanel, 0, wxCENTER | wxBOTTOM);
00069
00070     root->AddStretchSpacer(1); // space out the buttons and the labels
00071
00072     // Re-upload YouTube data
00073     auto* reuploadBtn = new wxButton(this, wxID_ANY, "Re-upload YouTube Data");
00074     UIButtons::ApplySizeBounds(reuploadBtn, ButtonType::Large);
00075     reuploadBtn->SetBackgroundColour(UIColors::Accent);
00076     reuploadBtn->SetForegroundColour(UIColors::TextPrimary);
00077     root->Add(reuploadBtn, 0, wxEXPAND | wxCENTER | wxBOTTOM, 16);
00078
00079     // Delete Account
00080     auto* del_acc = new wxButton(this, wxID_ANY, "Delete Account");
00081     UIButtons::ApplySizeBounds(del_acc, ButtonType::Large);
00082     del_acc->SetBackgroundColour(UIColors::Danger);
00083     del_acc->SetForegroundColour(UIColors::TextPrimary);
00084     root->Add(del_acc, 0, wxEXPAND | wxCENTER | wxBOTTOM, 16); // keep the button in the center
00085
00086     m_deletePasswordField = new wxTextCtrl(this, wxID_ANY, "", wxDefaultPosition,

```

```

00087         wxDefaultSize, wxTE_PASSWORD | wxTE_PROCESS_ENTER);
00088     m_deletePasswordField->SetHint("Re-enter password to delete account");
00089     m_deletePasswordField->SetBackgroundColour(UIColors::SurfaceRaised);
00090     m_deletePasswordField->SetForegroundColour(UIColors::TextPrimary);
00091     root->Add(m_deletePasswordField, 0, wxEXPAND | wxCENTER | wxBOTTOM, 8);
00092
00093     m_confirmDeleteBtn = new wxButton(this, wxID_ANY, "Confirm Delete");
00094     UIButtons::ApplySizeBounds(m_confirmDeleteBtn, ButtonType::Large);
00095     m_confirmDeleteBtn->SetBackgroundColour(UIColors::Danger);
00096     m_confirmDeleteBtn->SetForegroundColour(UIColors::TextPrimary);
00097     root->Add(m_confirmDeleteBtn, 0, wxEXPAND | wxCENTER | wxBOTTOM, 8);
00098
00099     m_deleteErrorLabel = new wxStaticText(this, wxID_ANY, "");
00100     m_deleteErrorLabel->SetForegroundColour(UIColors::Danger);
00101     root->Add(m_deleteErrorLabel, 0, wxEXPAND | wxCENTER | wxBOTTOM, 12);
00102
00103     // -- Logout -----
00104     auto* logoutBtn = new wxButton(this, wxID_ANY, "Log Out");
00105     UIButtons::ApplySizeBounds(logoutBtn, ButtonType::Large);
00106     logoutBtn->SetBackgroundColour(UIColors::Danger);
00107     logoutBtn->SetForegroundColour(UIColors::TextPrimary);
00108     root->Add(logoutBtn, 0, wxEXPAND | wxCENTER | wxBOTTOM, 16); // keep the button in the bottom
center as well
00109
00110     // add some extra padding space beneath the buttons
00111     root->AddStretchSpacer(1);
00112
00113     SetSizer(root);
00114
00115     reuploadBtn->Bind(wxEVT_BUTTON, &UserPanel::OnReuploadCSV, this);
00116     del_acc->Bind(wxEVT_BUTTON, &UserPanel::OnDeleteRequest, this);
00117     m_confirmDeleteBtn->Bind(wxEVT_BUTTON, &UserPanel::OnConfirmDelete, this);
00118     m_deletePasswordField->Bind(wxEVT_TEXT_ENTER,
00119         &UserPanel::OnConfirmDelete, this);
00120     logoutBtn->Bind(wxEVT_BUTTON, &UserPanel::OnLogout, this);
00121     Bind(wxEVT_SHOW, &UserPanel::OnShow, this);
00122
00123     SetDeleteReauthVisible(false);
00124 }
00125
00126 // -- Private helpers -----
00127
00128 void UserPanel::Refresh()
00129 {
00130     RefreshUserInfo();
00131 }
00132
00133 void UserPanel::RefreshUserInfo()
00134 {
00135     SetDeleteReauthVisible(false);
00136
00137     User* user = AppState::getInstance().getCurrentUser();
00138     if (user) {
00139         m_usernameLabel->SetLabel("Username: " +
00140             wxString::FromUTF8(user->getUsername()));
00141
00142         // verify email presence
00143         std::string tmp = user->getEmail();
00144         std::string email_str;
00145         if (tmp == "") {email_str = "no provided email at registration...";}
00146         else {email_str = tmp;}
00147
00148         m_emailLabel->SetLabel("Email: " +
00149             wxString::FromUTF8(email_str));
00150     } else {
00151         m_usernameLabel->SetLabel("Not logged in");
00152         m_emailLabel->SetLabel("");
00153     }
00154     Layout();
00155 }
00156
00157 void UserPanel::SetDeleteReauthVisible(bool isVisible)
00158 {
00159     if (!isVisible) {
00160         m_deletePasswordField->Clear();
00161         m_deleteErrorLabel->SetLabel("");
00162     }
00163
00164     m_deletePasswordField->Show(isVisible);
00165     m_confirmDeleteBtn->Show(isVisible);
00166     m_deleteErrorLabel->Show(isVisible && !m_deleteErrorLabel->GetLabel().IsEmpty());
00167     Layout();
00168 }
00169
00170 // -- Event handlers -----
00171
00172 void UserPanel::OnShow(wxShowEvent& evt)

```

```

00173 {
00174     if (evt.IsShown())
00175         Refresh();
00176     evt.Skip();
00177 }
00178
00179 void UserPanel::OnDeleteRequest(wxCommandEvent& /*evt*/)
00180 {
00181     ConfirmationDialog dlg(this,
00182         "Delete Account",
00183         "Are you sure you want to delete your account? This cannot be undone.",
00184         "Yes",
00185         "Nevermind");
00186     if (dlg.ShowModal() != wxID_OK) {
00187         SetDeleteReauthVisible(false);
00188         return;
00189     }
00190
00191     SetDeleteReauthVisible(true);
00192     m_deletePasswordField->SetFocus();
00193 }
00194
00195 void UserPanel::OnConfirmDelete(wxCommandEvent& /*evt*/)
00196 {
00197     User* user = AppState::getInstance().getCurrentUser();
00198     if (user == nullptr) {
00199         m_deleteErrorLabel->SetLabel("No active user session.");
00200         m_deleteErrorLabel->Show();
00201         Layout();
00202         return;
00203     }
00204
00205     const std::string password = m_deletePasswordField->GetValue().ToStdString();
00206     if (password.empty()) {
00207         m_deleteErrorLabel->SetLabel("Password is required to delete your account.");
00208         m_deleteErrorLabel->Show();
00209         Layout();
00210         return;
00211     }
00212
00213     m_controller.getEventRouter().dispatch(
00214         "deleteAccount",
00215         {"userID", user->getUserID()}, {"password", password});
00216 );
00217
00218     if (AppState::getInstance().getCurrentUser() == nullptr) {
00219         SetDeleteReauthVisible(false);
00220         wxMessageBox("Your account has been deleted successfully.",
00221             "Account Deleted",
00222             wxOK | wxICON_INFORMATION,
00223             this);
00224         m_nav(Page::LOGIN);
00225         return;
00226     }
00227
00228     m_deletePasswordField->Clear();
00229     m_deleteErrorLabel->SetLabel("Delete failed. Check password and try again.");
00230     m_deleteErrorLabel->Show();
00231     Layout();
00232 }
00233
00234 void UserPanel::OnReuploadCSV(wxCommandEvent& /*evt*/)
00235 {
00236     User* user = AppState::getInstance().getCurrentUser();
00237     if (!user) {
00238         wxMessageBox("No active user session.", "Error", wxOK | wxICON_ERROR, this);
00239         return;
00240     }
00241
00242     wxFileDialog dlg(this, "Select YouTube Export Data", "", "",
00243         "Supported files (*.csv;*.html;*.htm)|*.csv;*.html;*.htm|"
00244         "CSV files (*.csv)|*.csv|"
00245         "HTML files (*.html;*.htm)|*.html;*.htm|"
00246         "All files (*.*)|*.*",
00247         wxFD_OPEN | wxFD_FILE_MUST_EXIST);
00248     if (dlg.ShowModal() != wxID_OK) return;
00249
00250     std::string filePath = dlg.GetPath().ToStdString();
00251
00252     m_controller.getEventRouter().dispatch("uploadData",
00253         {"filePath", filePath}, {"userID", user->getUserID()});
00254
00255     std::string uploadError = AppState::getInstance().takePendingUploadError();
00256     if (!uploadError.empty()) {
00257         wxMessageBox(uploadError, "Upload Failed", wxOK | wxICON_ERROR, this);
00258         return;
00259     }

```

```

00260
00261     wxMessageBox("Your YouTube data has been re-uploaded.\n"
00262                 "Active blends will be updated with your new data.",
00263                 "Data Updated", wxOK | wxICON_INFORMATION, this);
00264 }
00265
00266 void UserPanel::OnLogout(wxCommandEvent& /*evt*/)
00267 {
00268     bool confirmed = ConfirmationDialog::Confirm(this, "Log Out", "Log out?");
00269     if (!confirmed) return;
00270
00271     m_controller.getEventRouter().dispatch("logout", {});
00272     m_nav(Page::LOGIN);
00273 }

```

7.183 UserPanel.hpp File Reference

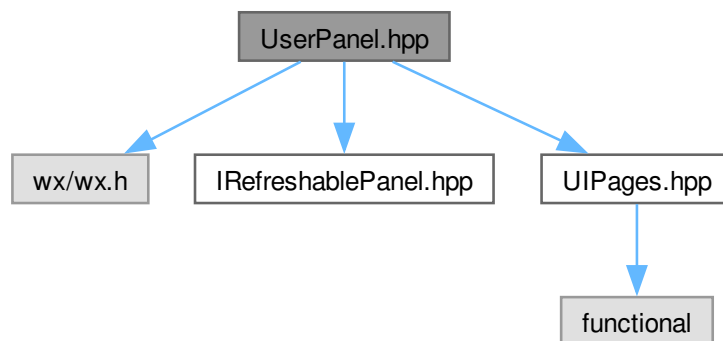
Account info and logout panel.

```

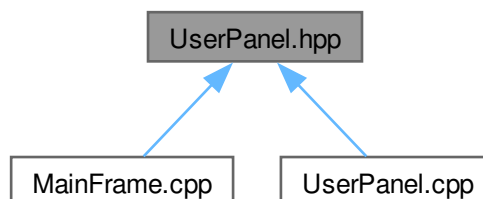
#include <wx/wx.h>
#include "IRefreshablePanel.hpp"
#include "UIPages.hpp"

```

Include dependency graph for UserPanel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [UserPanel](#)

Panel for user account display and account actions.

7.183.1 Detailed Description

Account info and logout panel.

Author

Jasmine Kumar

A panel that shows the current user's username and email, plus logout and account-deletion controls.

7.183.1.1 LAYOUT

[TopBar](#) ("My Account", back -> HOME) Info panel with username and email labels Separator and spacer Action buttons (logout and delete-account flow)

7.183.1.2 REFRESH ON SHOW

RefreshUserInfo() runs whenever the panel is shown (wxEVT_SHOW) so labels always reflect current [AppState](#).

7.183.1.3 LOGOUT

OnLogout: [ConfirmationDialog](#) -> dispatch("logout") -> nav(LOGIN).

7.183.1.4 FUTURE FEATURES

TODO: Add a "Change Password" form. TODO: Show YouTube data status and a CSV re-upload action. TODO: Display blend history.

Definition in file [UserPanel.hpp](#).

7.184 UserPanel.hpp

[Go to the documentation of this file.](#)

```

00001
00031
00032 #pragma once
00033
00034 #include <wx/wx.h>
00035
00036 #include "IRefreshablePanel.hpp"
00037 #include "UIPages.hpp"
00038
00039 class AppController;
00040 class wxStaticText;
00041 class wxTextCtrl;
00042 class wxButton;
00043
00048 class UserPanel : public wxPanel, public IRefreshablePanel {
00049 public:
00056     UserPanel(wxWindow* parent, AppController& controller, NavigateFn nav);
00057
00061     void Refresh() override;
00062
00063 private:
00064     AppController& m_controller;
00065     NavigateFn      m_nav;
00066
00068     wxStaticText* m_usernameLabel;
00069     wxStaticText* m_emailLabel;
00070     wxTextCtrl*   m_deletePasswordField;
00071     wxStaticText* m_deleteErrorLabel;
00072     wxButton*     m_confirmDeleteBtn;
00073
00075     void RefreshUserInfo();
00076
00078     void SetDeleteReauthVisible(bool isVisible);
00079
00081     void OnDeleteRequest(wxCommandEvent& evt);
00083     void OnConfirmDelete(wxCommandEvent& evt);
00085     void OnReuploadCSV(wxCommandEvent& evt);
00087     void OnLogout      (wxCommandEvent& evt);
00089     void OnShow         (wxShowEvent&    evt);
00090 };

```

7.185 VideoCard.cpp File Reference

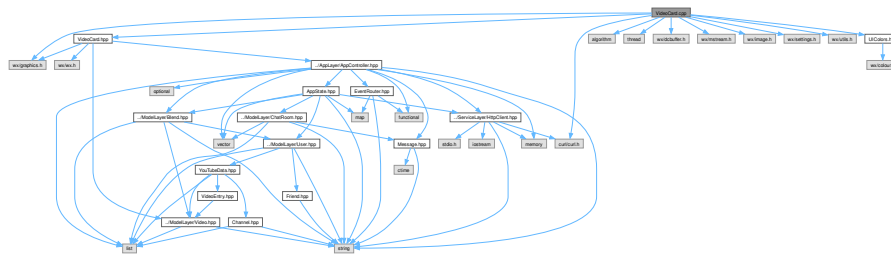
Implementation for [VideoCard](#).

```

#include "VideoCard.hpp"
#include <algorithm>
#include <thread>
#include <wx/dcbuffer.h>
#include <wx/graphics.h>
#include <wx/mstream.h>
#include <wx/image.h>
#include <wx/settings.h>
#include <wx/utils.h>
#include <curl/curl.h>
#include "UIColors.hpp"

```


Include dependency graph for VideoCard.cpp:



Functions

- **wxDEFINE_EVENT** (EVT_THUMBNAIL_LOADED, [ThumbnailEvent](#))
- **wxDEFINE_EVENT** (EVT_CHANNEL_LOGO_LOADED, [ThumbnailEvent](#))
- static double [LineHeight](#) (wxGraphicsContext *gc)

7.185.1 Detailed Description

Implementation for [VideoCard](#).

Author

Jasmine Kumar

Definition in file [VideoCard.cpp](#).

7.185.2 Function Documentation

7.185.2.1 LineHeight()

```
double LineHeight (
    wxGraphicsContext * gc) [static]
```

Definition at line 218 of file [VideoCard.cpp](#).

7.186 VideoCard.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 // =====
00008 // VideoCard.cpp - Custom video preview card widget implementation
00009 // =====
00010
00011 #include "VideoCard.hpp"
00012
00013 #include <algorithm>
00014 #include <thread>
00015
00016 #include <wx/dcbuffer.h>
```

```

00017 #include <wx/graphics.h>
00018 #include <wx/mstream.h>
00019 #include <wx/image.h>
00020 #include <wx/settings.h> // wxSystemSettings
00021 #include <wx/utils.h> // wxLaunchDefaultBrowser
00022
00023 #include <curl/curl.h>
00024
00025 #include "UIColors.hpp"
00026
00027 // -----
00028 // Event definitions
00029 // -----
00030 wxDEFINE_EVENT(EVT_THUMBNAIL_LOADED, ThumbnailEvent);
00031 wxDEFINE_EVENT(EVT_CHANNEL_LOGO_LOADED, ThumbnailEvent);
00032
00033 // -----
00034 // Internal curl helper - downloads a URL into memory and returns a wxImage.
00035 // Called only from background threads.
00036 // -----
00037 namespace {
00038
00039 size_t curlWrite(void* ptr, size_t size, size_t nmemb, std::string* out) {
00040     out->append(static_cast<char*>(ptr), size * nmemb);
00041     return size * nmemb;
00042 }
00043
00044 wxImage fetchImageViaCurl(const std::string& url) {
00045     CURL* curl = curl_easy_init();
00046     if (!curl) return wxImage();
00047
00048     std::string buf;
00049     curl_easy_setopt(curl, CURLOPT_URL, url.c_str());
00050     curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, curlWrite);
00051     curl_easy_setopt(curl, CURLOPT_WRITEDATA, &buf);
00052     curl_easy_setopt(curl, CURLOPT_FOLLOWLOCATION, 1L);
00053     curl_easy_setopt(curl, CURLOPT_TIMEOUT, 15L);
00054
00055     CURLcode res = curl_easy_perform(curl);
00056     curl_easy_cleanup(curl);
00057
00058     if (res != CURLE_OK || buf.empty()) return wxImage();
00059
00060     wxMemoryInputStream stream(buf.data(), buf.size());
00061     wxImage img;
00062     // wxBITMAP_TYPE_ANY lets wxWidgets detect JPEG / PNG / WebP automatically
00063     img.LoadFile(stream, wxBITMAP_TYPE_ANY);
00064     return img;
00065 }
00066
00067 } // namespace
00068
00069 // -----
00070 // Constructor
00071 // -----
00072 VideoCard::VideoCard(wxWindow* parent, AppController& controller)
00073 : wxPanel(parent, wxID_ANY, wxDefaultPosition, wxSize(260, 250))
00074 , m_controller(controller)
00075 {
00076     SetBackgroundColour(UIColors::Surface);
00077     SetBackgroundStyle(wxBG_STYLE_PAINT);
00078
00079     Bind(wxEVT_PAINT, &VideoCard::OnPaint, this);
00080     Bind(wxEVT_ENTER_WINDOW, &VideoCard::OnMouseEnter, this);
00081     Bind(wxEVT_LEAVE_WINDOW, &VideoCard::OnMouseLeave, this);
00082     Bind(wxEVT_LEFT_UP, &VideoCard::OnClick, this);
00083     Bind(EVT_THUMBNAIL_LOADED, &VideoCard::OnThumbnailLoaded, this);
00084     Bind(EVT_CHANNEL_LOGO_LOADED, &VideoCard::OnChannelLogoLoaded, this);
00085
00086     if (!wxImage::FindHandler(wxBITMAP_TYPE_JPEG))
00087         wxImage::AddHandler(new wxJPEGHandler());
00088     if (!wxImage::FindHandler(wxBITMAP_TYPE_PNG))
00089         wxImage::AddHandler(new wxPNGHandler());
00090 }
00091
00092 // -----
00093 // Public API
00094 // -----
00095 void VideoCard::SetVideo(const Video& video) {
00096     m_videoID = wxString::FromUTF8(video.GetVideoID());
00097     m_title = wxString::FromUTF8(video.GetTitle());
00098     m_channelID = wxString::FromUTF8(video.GetChannelID());
00099     m_channelName = wxString::FromUTF8(video.GetChannelName());
00100     m_thumbnailURL = wxString::FromUTF8(video.GetThumbnailURL());
00101     m_channelLogoURL = wxString::FromUTF8(video.GetChannelLogoURL());
00102
00103     m_thumbnail = wxBitmap();

```

```

00104     m_thumbLoading = false; m_thumbFailed = false;
00105
00106     m_channelLogo = wxBitmap();
00107     m_logoLoading = false; m_logoFailed = false;
00108
00109     FetchThumbnailAsync();
00110     FetchChannelLogoAsync();
00111     Refresh();
00112 }
00113
00114 void VideoCard::Clear() {
00115     m_videoID = m_title = m_channelID = m_channelName
00116         = m_thumbnailURL = m_channelLogoURL = wxEmptyString;
00117
00118     m_thumbnail = wxBitmap();
00119     m_thumbLoading = m_thumbFailed = false;
00120
00121     m_channelLogo = wxBitmap();
00122     m_logoLoading = m_logoFailed = false;
00123
00124     Refresh();
00125 }
00126
00127 // -----
00128 // Async fetches
00129 // -----
00130 void VideoCard::FetchThumbnailAsync() {
00131     if (m_videoID.IsEmpty()) return;
00132
00133     m_thumbLoading = true;
00134     ++m_thumbGen;
00135
00136     // Build the URL: prefer the stored thumbnailURL, fall back to YouTube CDN
00137     std::string url;
00138     if (!m_thumbnailURL.IsEmpty()) {
00139         url = std::string(m_thumbnailURL.ToUTF8());
00140     } else {
00141         url = "https://img.youtube.com/vi/" +
00142             std::string(m_videoID.ToUTF8()) + "/hqdefault.jpg";
00143     }
00144
00145     VideoCard* card = this;
00146     int gen = m_thumbGen;
00147
00148     std::thread([url, card, gen]() {
00149         wxImage img = fetchImageViaCurl(url);
00150         bool ok = img.IsOk();
00151         auto* evt = new ThumbnailEvent(EVT_THUMBNAIL_LOADED, wxID_ANY,
00152             std::move(img), ok, gen);
00153         wxQueueEvent(card, evt);
00154     }).detach();
00155 }
00156
00157 void VideoCard::FetchChannelLogoAsync() {
00158     if (m_channelLogoURL.IsEmpty()) return;
00159
00160     m_logoLoading = true;
00161     ++m_logoGen;
00162
00163     std::string url = std::string(m_channelLogoURL.ToUTF8());
00164     VideoCard* card = this;
00165     int gen = m_logoGen;
00166
00167     std::thread([url, card, gen]() {
00168         wxImage img = fetchImageViaCurl(url);
00169         bool ok = img.IsOk();
00170         auto* evt = new ThumbnailEvent(EVT_CHANNEL_LOGO_LOADED, wxID_ANY,
00171             std::move(img), ok, gen);
00172         wxQueueEvent(card, evt);
00173     }).detach();
00174 }
00175
00176 // -----
00177 // Event handlers
00178 // -----
00179 void VideoCard::OnThumbnailLoaded(ThumbnailEvent& evt) {
00180     if (evt.generation != m_thumbGen) return;
00181     m_thumbLoading = false;
00182     if (evt.success && evt.image.IsOk()) {
00183         m_thumbnail = wxBitmap(evt.image);
00184         m_thumbFailed = false;
00185     } else {
00186         m_thumbFailed = true;
00187     }
00188     Refresh();
00189 }
00190

```

```

00191 void VideoCard::OnChannelLogoLoaded(ThumbnailEvent& evt) {
00192     if (evt.generation != m_logoGen) return;
00193     m_logoLoading = false;
00194     if (evt.success && evt.image.IsOk()) {
00195         m_channelLogo = wxBitmap(evt.image);
00196         m_logoFailed = false;
00197     } else {
00198         m_logoFailed = true;
00199     }
00200     Refresh();
00201 }
00202
00203 void VideoCard::OnMouseEnter(wxMouseEvent& /*evt*/) { m_hovered = true; Refresh(); }
00204 void VideoCard::OnMouseLeave(wxMouseEvent& /*evt*/) { m_hovered = false; Refresh(); }
00205
00206 void VideoCard::OnClick(wxMouseEvent& /*evt*/) {
00207     if (m_videoID.IsEmpty()) return;
00208     wxLaunchDefaultBrowser(
00209         wxString::Format("https://www.youtube.com/watch?v=%s", m_videoID));
00210     EventPayload payload;
00211     payload["videoID"] = std::string(m_videoID.ToUTF8());
00212     m_controller.getEventRouter().dispatch("playVideo", payload);
00213 }
00214
00215 // -----
00216 // Painting helpers
00217 // -----
00218 static double LineHeight(wxGraphicsContext* gc) {
00219     double tw, th, td, tex;
00220     gc->GetTextExtent("Wg", &tw, &th, &td, &tex);
00221     return th;
00222 }
00223
00224 int VideoCard::DrawWrappedText(wxGraphicsContext* gc,
00225                                const wxString& text,
00226                                int maxLines,
00227                                double x, double y,
00228                                double maxWidth) const {
00229     if (text.IsEmpty()) return static_cast<int>(y);
00230
00231     double tw, th, td, tex;
00232     gc->GetTextExtent("Wg", &tw, &th, &td, &tex);
00233     double lineH = th + 2.0;
00234
00235     wxStringArray words = wxSplit(text, ' ');
00236     wxString currentLine;
00237     int lineCount = 0;
00238
00239     auto flush = [&](const wxString& line, bool isLast) {
00240         if (lineCount >= maxLines) return;
00241         wxString toDraw = line;
00242         gc->GetTextExtent(toDraw, &tw, &th, &td, &tex);
00243         if (!isLast || tw > maxWidth) {
00244             while (!toDraw.IsEmpty()) {
00245                 wxString candidate = toDraw + L"\u2026";
00246                 gc->GetTextExtent(candidate, &tw, &th, &td, &tex);
00247                 if (tw <= maxWidth) { toDraw = candidate; break; }
00248                 toDraw.RemoveLast();
00249             }
00250         }
00251         gc->DrawText(toDraw, x, y + lineCount * lineH);
00252         ++lineCount;
00253     };
00254
00255     for (size_t i = 0; i < words.GetCount(); ++i) {
00256         wxString candidate = currentLine.IsEmpty()
00257             ? words[i]
00258             : currentLine + " " + words[i];
00259         gc->GetTextExtent(candidate, &tw, &th, &td, &tex);
00260         if (tw > maxWidth && !currentLine.IsEmpty()) {
00261             flush(currentLine, false);
00262             if (lineCount >= maxLines) goto done;
00263             currentLine = words[i];
00264         } else {
00265             currentLine = candidate;
00266         }
00267     }
00268     if (!currentLine.IsEmpty()) flush(currentLine, true);
00269 done:
00270     return static_cast<int>(y + lineCount * lineH);
00271 }
00272
00273 // -----
00274 // OnPaint
00275 // -----
00276 void VideoCard::OnPaint(wxPaintEvent& /*evt*/) {
00277     wxAutoBufferedPaintDC dc(this);

```

```

00278 wxGraphicsContext* gc = wxGraphicsContext::Create(dc);
00279 if (!gc) return;
00280
00281 const wxSize sz = GetClientSize();
00282 const double W = sz.GetWidth();
00283 const double H = sz.GetHeight();
00284
00285 // -- Card background -----
00286 gc->SetBrush(wxBrush(UIColors::Surface));
00287 gc->SetPen(*wxTRANSPARENT_PEN);
00288 gc->DrawRoundedRectangle(0, 0, W, H, 10.0);
00289
00290 // -- Thumbnail area - 16:9 capped at 145 px so the text area never
00291 // gets squeezed out when the grid sizer stretches cards wider.
00292 const double thumbH = std::min(W * 9.0 / 16.0, 300.0);
00293
00294 if (m_thumbnail.IsOk()) {
00295     wxImage scaled = m_thumbnail.ConvertToImage()
00296         .Scale(static_cast<int>(W),
00297             static_cast<int>(thumbH),
00298             wxIMAGE_QUALITY_BILINEAR);
00299     gc->DrawBitmap(wxBitmap(scaled), 0, 0, W, thumbH);
00300 } else {
00301     gc->SetBrush(wxBrush(UIColors::SurfaceRaised));
00302     gc->SetPen(*wxTRANSPARENT_PEN);
00303     gc->DrawRectangle(0, 0, W, thumbH);
00304
00305     wxString label;
00306     if (m_thumbLoading) label = "Loading...";
00307     else if (m_thumbFailed) label = L"\u25B6";
00308     // else: no video set - blank
00309
00310     if (!label.IsEmpty()) {
00311         wxFont lf = wxSystemSettings::GetFont(wxSYS_DEFAULT_GUI_FONT);
00312         lf.SetPointSize(m_thumbFailed ? 26 : 11);
00313         gc->SetFont(lf, UIColors::TextMuted);
00314         double tw, th, td, tex;
00315         gc->GetTextExtent(label, &tw, &th, &td, &tex);
00316         gc->DrawText(label, (W - tw) / 2.0, (thumbH - th) / 2.0);
00317     }
00318 }
00319
00320 // -- Title -----
00321 const double padX = 8.0;
00322 const double textW = W - padX * 2.0;
00323 double curY = thumbH + 8.0;
00324
00325 if (!m_title.IsEmpty()) {
00326     wxFont titleFont = wxSystemSettings::GetFont(wxSYS_DEFAULT_GUI_FONT);
00327     titleFont.SetPointSize(10);
00328     titleFont.SetWeight(wxFONTWEIGHT_SEMIBOLD);
00329     gc->SetFont(titleFont, UIColors::TextPrimary);
00330     curY = DrawWrappedText(gc, m_title, 2, padX, curY, textW);
00331     curY += 4.0;
00332 }
00333
00334 // -- Channel row: round logo + channel name -----
00335 // Only render when we have at least a channel name or ID to show.
00336 // Avoids an orphaned grey circle for videos not yet enriched by the API.
00337 wxString chanLabel = m_channelName.IsEmpty() ? m_channelID : m_channelName;
00338 const bool hasChannelData = !chanLabel.IsEmpty() || m_channelLogo.IsOk()
00339     || m_logoLoading;
00340
00341 if (hasChannelData) {
00342     const double logoSize = 24.0;
00343     const double rowH = logoSize;
00344
00345     if (m_channelLogo.IsOk()) {
00346         // Punch a circle into the alpha channel - renders as a disc.
00347         int sz = static_cast<int>(logoSize);
00348         wxImage scaled = m_channelLogo.ConvertToImage()
00349             .Scale(sz, sz, wxIMAGE_QUALITY_BILINEAR);
00350         if (!scaled.HasAlpha()) scaled.InitAlpha();
00351
00352         const double cx = sz / 2.0, cy = sz / 2.0, r = sz / 2.0;
00353         for (int py = 0; py < sz; ++py)
00354             for (int px = 0; px < sz; ++px) {
00355                 double dx = px + 0.5 - cx, dy = py + 0.5 - cy;
00356                 if (dx * dx + dy * dy > r * r)
00357                     scaled.SetAlpha(px, py, wxALPHA_TRANSPARENT);
00358             }
00359         gc->DrawBitmap(wxBitmap(scaled), padX, curY, logoSize, logoSize);
00360     } else {
00361         // Placeholder circle (shown while logo is loading)
00362         gc->SetBrush(wxBrush(UIColors::SurfaceRaised));
00363         gc->SetPen(*wxTRANSPARENT_PEN);
00364         gc->DrawEllipse(padX, curY, logoSize, logoSize);
00365     }
00366 }

```

```

00365     }
00366
00367     if (!chanLabel.IsEmpty()) {
00368         wxFont chanFont = wxSystemSettings::GetFont(wxSYS_DEFAULT_GUI_FONT);
00369         chanFont.SetPointSize(9);
00370         gc->SetFont(chanFont, UIColors::TextSecondary);
00371
00372         const double textX      = padX + logoSize + 5.0;
00373         const double chanTextW = W - textX - padX;
00374
00375         double tw, th, td, tex;
00376         gc->GetTextExtent(chanLabel, &tw, &th, &td, &tex);
00377         if (tw > chanTextW) {
00378             while (!chanLabel.IsEmpty()) {
00379                 wxString cand = chanLabel + L"\u2026";
00380                 gc->GetTextExtent(cand, &tw, &th, &td, &tex);
00381                 if (tw <= chanTextW) { chanLabel = cand; break; }
00382                 chanLabel.RemoveLast();
00383             }
00384         }
00385         gc->DrawText(chanLabel, textX, curY + (rowH - th) / 2.0);
00386     }
00387 }
00388
00389 // -- Hover border -----
00390 if (m_hovered) {
00391     gc->SetBrush(*wxTRANSPARENT_BRUSH);
00392     gc->SetPen(wxPen(UIColors::Accent, 3));
00393     gc->DrawRoundedRectangle(1.5, 1.5, W - 3.0, H - 3.0, 14.0);
00394 }
00395
00396 delete gc;
00397 }

```

7.187 VideoCard.hpp File Reference

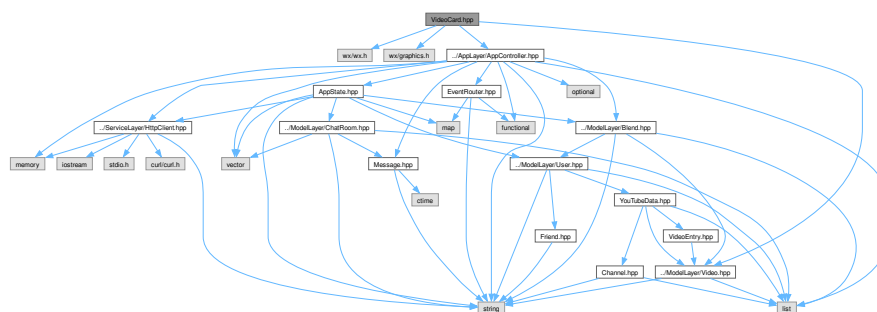
Custom wxPanel widget that displays a single YouTube video preview.

```

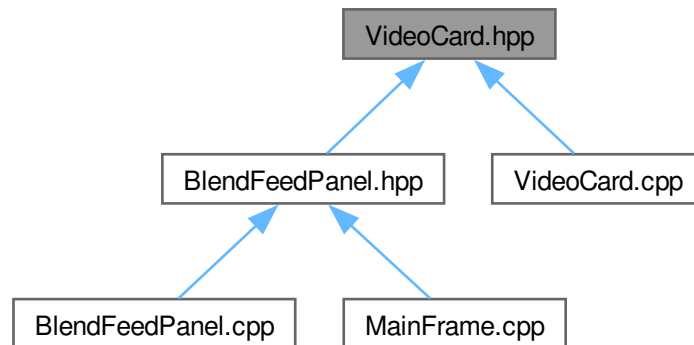
#include <wx/wx.h>
#include <wx/graphics.h>
#include "../ModelLayer/Video.hpp"
#include "../AppLayer/AppController.hpp"

```

Include dependency graph for VideoCard.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [ThumbnailEvent](#)
Event fired on the [VideoCard](#) when an async image fetch completes.
- class [VideoCard](#)
Custom wxPanel widget that displays a single YouTube video preview.

Functions

- **wxDECLARE_EVENT** (EVT_THUMBNAIL_LOADED, [ThumbnailEvent](#))
Fired when the video thumbnail is ready.
- **wxDECLARE_EVENT** (EVT_CHANNEL_LOGO_LOADED, [ThumbnailEvent](#))
Fired when the channel logo is ready.

7.187.1 Detailed Description

Custom wxPanel widget that displays a single YouTube video preview.

Author

Jasmine Kumar

Layout (top -> bottom):

- 16:9 thumbnail - fetched asynchronously via libcurl from thumbnailURL, falling back to [img.youtube.com/vi/<id>/hqdefault.jpg](#) if the URL is empty.
- [Video](#) title (up to 2 wrapped lines).
- [Channel](#) row: 24 px round channel logo + channel name. The logo is fetched asynchronously from [channelLogoURL](#).

Both async fetches use a generation counter so stale results from an earlier `SetVideo()` call are silently discarded. Clicking the card opens the video in the system browser and dispatches a "playVideo" event.

Definition in file [VideoCard.hpp](#).

7.188 VideoCard.hpp

[Go to the documentation of this file.](#)

```

00001
00017
00018 #pragma once
00019
00020 #include <wx/wx.h>
00021 #include <wx/graphics.h>
00022
00023 #include "../ModelLayer/Video.hpp"
00024 #include "../AppLayer/AppController.hpp"
00025
00037 class ThumbnailEvent : public wxEvent {
00038 public:
00039     wxImage image;
00040     bool success;
00041     int generation;
00042
00043     ThumbnailEvent(wxEventType type, int id, wxImage img, bool ok, int gen)
00044         : wxEvent(id, type), image(std::move(img)), success(ok), generation(gen) {}
00045
00046     wxEvent* Clone() const override { return new ThumbnailEvent(*this); }
00047 };
00048
00049 wxDECLARE_EVENT(EVT_THUMBNAIL_LOADED, ThumbnailEvent);
00050 wxDECLARE_EVENT(EVT_CHANNEL_LOGO_LOADED, ThumbnailEvent);
00051
00059 class VideoCard : public wxPanel {
00060 public:
00066     VideoCard(wxWindow* parent, AppController& controller);
00067
00072     void SetVideo(const Video& video);
00073
00075     void Clear();
00076
00077     wxSize DoGetBestSize() const override { return wxSize(260, 250); }
00078
00079 private:
00080     AppController& m_controller;
00081
00082     wxString m_videoID;
00083     wxString m_title;
00084     wxString m_channelID;
00085     wxString m_channelName;
00086     wxString m_thumbnailURL;
00087     wxString m_channelLogoURL;
00088
00089     wxBitmap m_thumbnail;
00090     bool m_thumbLoading = false;
00091     bool m_thumbFailed = false;
00092     int m_thumbGen = 0;
00093
00094     wxBitmap m_channelLogo;
00095     bool m_logoLoading = false;
00096     bool m_logoFailed = false;
00097     int m_logoGen = 0;
00098
00099     bool m_hovered = false;
00100
00102     void FetchThumbnailAsync();
00104     void FetchChannelLogoAsync();
00105
00106     void OnThumbnailLoaded (ThumbnailEvent& evt);
00107     void OnChannelLogoLoaded(ThumbnailEvent& evt);
00108     void OnPaint (wxPaintEvent& evt);
00109     void OnMouseEnter(wxMouseEvent& evt);
00110     void OnMouseLeave(wxMouseEvent& evt);
00111     void OnClick (wxMouseEvent& evt);
00112
00123     int DrawWrappedText (wxGraphicsContext* gc,
00124                         const wxString& text,
00125                         int maxLines,
00126                         double x, double y,
00127                         double maxWidth) const;
00128 };

```


Index

- ~AppController
 - AppController, [25](#)
- ~ChatWebSocket
 - ChatWebSocket, [70](#)
- ~CsvParser
 - CsvParser, [77](#)
- ~DataExtractor
 - DataExtractor, [82](#)
- ~FileBin
 - FileBin, [94](#)
- ~FileFormatter
 - FileFormatter, [96](#)
- ~OperationProgressDialog
 - OperationProgressDialog, [136](#)
- ~SqliteDataManager
 - SqliteDataManager, [148](#)
- Accent
 - Palette, [139](#)
 - UIColors, [15](#)
- AccentHover
 - Palette, [139](#)
 - UIColors, [15](#)
- ACTIVE_BLENDS
 - UIPages.hpp, [417](#)
- activeBlend
 - AppState, [43](#)
- ActiveBlendsPanel, [19](#)
 - ActiveBlendsPanel, [21](#)
 - m_controller, [22](#)
 - m_emptyLabel, [22](#)
 - m_listInner, [22](#)
 - m_listScroll, [22](#)
 - m_listSizer, [22](#)
 - m_nav, [22](#)
 - OnLeaveBlend, [21](#)
 - OnSelectBlend, [21](#)
 - RebuildList, [21](#)
 - Refresh, [21](#)
- activeBlendsPanel
 - MainFrame, [131](#)
- ActiveBlendsPanel.cpp, [342](#), [343](#)
- ActiveBlendsPanel.hpp, [345](#), [346](#)
- addFriend
 - User, [159](#)
- addMessage
 - ChatRoom, [67](#)
- addSessionUser
 - AppState, [38](#)
- algorithmUsed
 - Blend, [47](#)
- ALT_SUCC
 - HttpClient, [115](#)
- apiKey
 - YouTubeMetadataFetcher, [199](#)
- AppController, [23](#)
 - ~AppController, [25](#)
 - AppController, [25](#)
 - appState, [35](#)
 - clearProgressReporter, [25](#)
 - currentBlend, [35](#)
 - enrichActiveBlendFeedIfMissingMetadata, [25](#)
 - enrichIfMissingMetadata, [26](#)
 - enrichMissingMetadataSubsetMultithreaded, [27](#)
 - eventRouter, [35](#)
 - fetchUserBlends, [27](#)
 - get_current_email, [27](#)
 - get_current_user, [27](#)
 - get_current_username, [28](#)
 - getEventRouter, [28](#)
 - handleCreateBlend, [28](#)
 - handleDeleteAccount, [28](#)
 - handleLeaveBlend, [29](#)
 - handleLogin, [29](#)
 - handleLogout, [29](#)
 - handleOpenChat, [29](#)
 - handlePlayVideo, [30](#)
 - handleRefresh, [30](#)
 - handleRegister, [30](#)
 - handleSelectBlend, [30](#)
 - handleSendMessage, [31](#)
 - handleUploadData, [31](#)
 - http, [35](#)
 - isConnected, [35](#)
 - loadParticipantsWithWatchLater, [31](#)
 - loadParticipantsWithWatchLaterParallel, [31](#)
 - lookupUser, [32](#)
 - parseBlendID, [32](#)
 - parseChatHistory, [32](#)
 - parseParticipantIDs, [33](#)
 - parseUserFromAuthResponse, [33](#)
 - parseWatchLaterVideos, [33](#)
 - pendingAccountCreated_, [35](#)
 - pendingRegistration_, [35](#)
 - progressReporter, [35](#)
 - registerBlendOnServer, [34](#)
 - registerEvents, [34](#)
 - reportProgress, [34](#)
 - server_url, [36](#)

- setProgressReporter, 34
- AppController.cpp, 243, 244
 - Json, 244
 - kYouTubeAPIKey, 244
- AppController.hpp, 263, 264
- AppController::BlendSummary, 61
 - blendID, 62
 - title, 62
- ApplySizeBounds
 - ButtonsConfig.cpp, 368
 - ButtonsConfig.hpp, 373
- AppState, 36
 - activeBlend, 43
 - addSessionUser, 38
 - AppState, 38
 - chatRooms, 43
 - clearSession, 38
 - clearSessionUsers, 38
 - createChatRoom, 38
 - currentUser, 43
 - getActiveBlend, 39
 - getActiveChatRoom, 39
 - getChatRoom, 39
 - getCurrentUser, 39
 - getInstance, 40
 - getSessionUsers, 40
 - getUsersWithoutData, 40
 - isBlendGenerating, 43
 - isGeneratingBlend, 40
 - pendingBlendMessage, 43
 - pendingUploadError, 43
 - sessionUsers, 43
 - setActiveBlend, 40
 - setCurrentUser, 41
 - setIsBlendGenerating, 41
 - setPendingBlendMessage, 41
 - setPendingUploadError, 41
 - setUsersWithoutData, 42
 - takePendingBlendMessage, 42
 - takePendingUploadError, 42
 - usersWithoutData, 44
- appState
 - AppController, 35
- AppState.cpp, 266, 267
- AppState.hpp, 268, 270
- AUTH_USER_PREFIX
 - HttpClient, 115
- Background
 - Palette, 139
 - UIColors, 16
- baseUrl
 - HttpClient, 115
- bins
 - DataSaver, 91
- BLEND
 - HttpClient, 115
- Blend, 44
 - algorithmUsed, 47
- Blend, 45
 - blendID, 47
 - getAlgorithmUsed, 45
 - getBlendID, 45
 - getParticipants, 45
 - getTitle, 46
 - getVideo, 46
 - getVideoList, 46
 - participants, 48
 - setBlendID, 46
 - setVideoList, 47
 - size, 47
 - title, 48
 - toString, 47
 - videoList, 48
- Blend.cpp, 276
- Blend.hpp, 278, 279
- BLEND_CHAT
 - UIPages.hpp, 417
- BLEND_CREATION
 - UIPages.hpp, 417
- BlendChatPanel, 48
 - BlendChatPanel, 50
 - ConnectWebSocket, 50
 - DoSend, 50
 - m_controller, 51
 - m_incoming, 51
 - m_incomingMx, 52
 - m_inputCtrl, 52
 - m_inputPanel, 52
 - m_msgInner, 52
 - m_msgScroll, 52
 - m_msgSizer, 52
 - m_nav, 52
 - m_noBlendLabel, 52
 - m_participantsBtn, 53
 - m_participantsDropdown, 53
 - m_participantsSizer, 53
 - m_pollTimer, 53
 - m_ws, 53
 - m_wsBlendID, 53
 - OnInputEnter, 50
 - OnParticipantsToggle, 50
 - OnPollTimer, 51
 - OnSend, 51
 - Refresh, 51
 - Reload, 51
- BlendChatPanel.cpp, 347, 348
 - FormatTimestamp, 347
- BlendChatPanel.hpp, 352, 354
- BlendCreationPanel, 54
 - BlendCreationPanel, 55
 - m_addedUsers, 56
 - m_blendNameCtrl, 56
 - m_controller, 57
 - m_countLabel, 57
 - m_createBtn, 57
 - m_emptyLabel, 57

- m_nav, [57](#)
- m_userListInner, [57](#)
- m_userListScroll, [57](#)
- m_userListSizer, [57](#)
- OnAdd, [55](#)
- OnCreate, [55](#)
- OnRemoveUser, [56](#)
- RebuildUserList, [56](#)
- Refresh, [56](#)
- Reload, [56](#)
- UpdateCountLabel, [56](#)
- BlendCreationPanel.cpp, [355](#), [356](#)
- BlendCreationPanel.hpp, [361](#), [362](#)
- BlendFeedPanel, [58](#)
 - BlendFeedPanel, [60](#)
 - kPageSize, [61](#)
 - LoadFromBlend, [60](#)
 - m_cards, [61](#)
 - m_controller, [61](#)
 - m_offset, [61](#)
 - NextPage, [60](#)
 - Refresh, [60](#)
- BlendFeedPanel.cpp, [363](#), [364](#)
- BlendFeedPanel.hpp, [365](#), [367](#)
- blendID
 - AppController::BlendSummary, [62](#)
 - Blend, [47](#)
 - ChatRoom, [69](#)
 - ChatWebSocket, [73](#)
- book
 - MainFrame, [131](#)
- BtnConfig, [62](#)
 - maxHeight, [63](#)
 - maxWidth, [63](#)
 - minHeight, [63](#)
 - minWidth, [63](#)
- build_auth_user_lookup_endpoint
 - HttpClient, [109](#)
- build_blend_participant_endpoint
 - HttpClient, [109](#)
- build_chat_history_endpoint
 - HttpClient, [109](#)
- build_chatroom_detail_endpoint
 - HttpClient, [110](#)
- build_delete_user_endpoint
 - HttpClient, [110](#)
- build_fullUrl
 - ChatWebSocket, [71](#)
- build_latest_blend_endpoint
 - HttpClient, [110](#)
- build_leave_blend_endpoint
 - HttpClient, [111](#)
- build_user_blends_endpoint
 - HttpClient, [111](#)
- build_watch_later_endpoint
 - HttpClient, [111](#)
- buildBlendJson
 - RequestJsonBuilder.cpp, [321](#)
 - RequestJsonBuilder.hpp, [326](#)
- buildDeleteAccountJson
 - RequestJsonBuilder.cpp, [322](#)
 - RequestJsonBuilder.hpp, [327](#)
- BuildHomePage
 - MainFrame, [130](#)
- buildLoginJson
 - RequestJsonBuilder.cpp, [322](#)
 - RequestJsonBuilder.hpp, [327](#)
- BuildRegisterForm
 - LoginPanel, [124](#)
- buildRegisterJson
 - RequestJsonBuilder.cpp, [323](#)
 - RequestJsonBuilder.hpp, [328](#)
- BuildSignInForm
 - LoginPanel, [124](#)
- buildWatchLaterJson
 - RequestJsonBuilder.cpp, [323](#)
 - RequestJsonBuilder.hpp, [328](#)
- ButtonsConfig.cpp, [367](#), [370](#)
 - ApplySizeBounds, [368](#)
 - GetConfig, [368](#)
 - GetMaxSize, [369](#)
 - GetMinSize, [369](#)
 - GetSize, [369](#)
- ButtonsConfig.hpp, [371](#), [374](#)
 - ApplySizeBounds, [373](#)
 - ButtonType, [372](#)
 - Compact, [372](#)
 - FormSubmit, [372](#)
 - FormTab, [372](#)
 - FullWidthDanger, [372](#)
 - FullWidthPrimary, [372](#)
 - FullWidthSecondary, [372](#)
 - GetConfig, [373](#)
 - GetMaxSize, [373](#)
 - GetMinSize, [373](#)
 - GetSize, [374](#)
 - Icon, [372](#)
 - Medium, [372](#)
 - Nav, [372](#)
 - Small, [372](#)
 - TopBarBack, [372](#)
- ButtonType
 - ButtonsConfig.hpp, [372](#)
- categories
 - Channel, [65](#)
- Channel, [63](#)
 - categories, [65](#)
 - Channel, [64](#)
 - channelID, [65](#)
 - displayName, [65](#)
 - getCategories, [64](#)
 - getChannelID, [64](#)
 - getDisplayName, [65](#)
 - toString, [65](#)
- Channel.cpp, [279](#), [280](#)
- Channel.hpp, [281](#), [282](#)

- channelID
 - Channel, 65
 - Video, 171
- channelId
 - YouTubeMetadataFetcher::VideoMeta, 183
- channelLogoURL
 - Video, 171
- channelName
 - Video, 171
 - YouTubeMetadataFetcher::VideoMeta, 183
- chatPanel
 - MainFrame, 131
- ChatRoom, 66
 - addMessage, 67
 - blendID, 69
 - ChatRoom, 66
 - clearMessages, 67
 - getBlendID, 67
 - getMessages, 67
 - getParticipantIDs, 68
 - isParticipant, 68
 - messages, 69
 - participantIDs, 69
 - toString, 68
- ChatRoom.cpp, 282, 283
- ChatRoom.hpp, 284, 285
- chatRooms
 - AppState, 43
- ChatWebSocket, 69
 - ~ChatWebSocket, 70
 - blendID, 73
 - build_fullUrl, 71
 - ChatWebSocket, 70
 - connect, 71
 - content_formatter, 71
 - fullUrl, 73
 - get_fullUrl, 71
 - isStarted, 73
 - m_onMessage, 73
 - send_message, 71
 - set_blendID, 72
 - set_userID, 72
 - setOnMessage, 72
 - url, 73
 - userID, 73
 - ws, 73
- ChatWebSocket.cpp, 309, 310
 - urlEncode, 310
- ChatWebSocket.hpp, 311, 313
- Clear
 - VideoCard, 175
- clearMessages
 - ChatRoom, 67
- clearProgressReporter
 - AppController, 25
- clearSession
 - AppState, 38
- clearSessionUsers
 - AppState, 38
- Clone
 - ThumbnailEvent, 154
- Compact
 - ButtonsConfig.hpp, 372
- Confirm
 - ConfirmationDialog, 75
- ConfirmationDialog, 74
 - Confirm, 75
 - ConfirmationDialog, 75
- ConfirmationDialog.cpp, 375
- ConfirmationDialog.hpp, 377, 378
- connect
 - ChatWebSocket, 71
- ConnectWebSocket
 - BlendChatPanel, 50
- content_formatter
 - ChatWebSocket, 71
- controller
 - MainFrame, 132
 - YTBLNDApp, 201
- createChatRoom
 - AppState, 38
- createUser
 - DataManager, 87
 - SqliteDataManager, 149
- creationPanel
 - MainFrame, 132
- CSV
 - File_ID, 93
- CsvParser, 75
 - ~CsvParser, 77
 - CsvParser, 76
 - getParseId, 77
 - parse, 77
 - parse_id, 78
 - raw_data, 78
 - setData, 77
- CsvParser.cpp, 209, 210
- CsvParser.hpp, 211, 212
- CsvSource, 78
 - CsvSource, 79
 - getSourceId, 80
 - read, 80
 - setSource, 80
 - source_id, 80
 - src, 80
- CsvSource.cpp, 213
- CsvSource.hpp, 214, 215
- Current
 - UIColors, 16
- currentBlend
 - AppController, 35
- currentProgress
 - OperationProgressDialog, 137
- currentUser
 - AppState, 43

D

- HttpClient, [116](#)
- Danger
 - Palette, [139](#)
 - UIColors, [16](#)
- DarkMode
 - UIColors, [16](#)
- DATA_INSTRUCTIONS
 - UIPages.hpp, [417](#)
- DataExtractor, [81](#)
 - ~DataExtractor, [82](#)
 - DataExtractor, [81](#)
 - extract, [82](#)
 - par_format, [83](#)
 - setParser, [82](#)
 - setSource, [83](#)
 - src_format, [83](#)
- DataExtractor.cpp, [215](#), [216](#)
- DataExtractor.hpp, [217](#), [218](#)
- dataInstrPanel
 - MainFrame, [132](#)
- DataInstructionsPanel, [84](#)
 - DataInstructionsPanel, [85](#)
 - m_controller, [85](#)
 - m_nav, [85](#)
 - OnBrowse, [85](#)
- DataInstructionsPanel.cpp, [378](#), [379](#)
- DataInstructionsPanel.hpp, [381](#), [382](#)
- DataManager, [86](#)
 - createUser, [87](#)
 - findBlendForUser, [87](#)
 - findUserById, [87](#)
 - loadBlendParticipants, [87](#)
 - loadWatchLater, [88](#)
 - saveBlend, [88](#)
 - saveWatchLater, [88](#)
 - validatePassword, [88](#)
- DataManager.hpp, [313](#), [315](#)
- DataSaver, [89](#)
 - bins, [91](#)
 - DataSaver, [89](#)
 - formatters, [91](#)
 - save, [90](#)
 - setFileBin, [90](#)
 - setFileFormatter, [90](#)
- DataSaver.hpp, [218](#), [219](#)
- db
 - SqliteDataManager, [152](#)
- del
 - HttpClient, [112](#)
- DELETE_USER_PREFIX
 - HttpClient, [116](#)
- deregisterAll
 - EventRouter, [92](#)
- dispatch
 - EventRouter, [92](#)
- displayName
 - Channel, [65](#)
 - Friend, [101](#)
- DoGetBestSize
 - VideoCard, [175](#)
- DoSend
 - BlendChatPanel, [50](#)
- DrawWrappedText
 - VideoCard, [175](#)
- DUPLICATE
 - HttpClient, [116](#)
- duration
 - Video, [171](#)
- email
 - Friend, [101](#)
 - User, [162](#)
- enrich
 - YouTubeMetadataFetcher, [198](#)
- enrichActiveBlendFeedIfMissingMetadata
 - AppController, [25](#)
- enrichIfMissingMetadata
 - AppController, [26](#)
- enrichMissingMetadataSubsetMultithreaded
 - AppController, [27](#)
- escape
 - ModelJson, [13](#)
- EventHandler
 - EventRouter.hpp, [273](#)
- EventPayload
 - EventRouter.hpp, [273](#)
- EventRouter, [91](#)
 - deregisterAll, [92](#)
 - dispatch, [92](#)
 - EventRouter, [92](#)
 - listeners, [93](#)
 - registerListener, [92](#)
- eventRouter
 - AppController, [35](#)
- EventRouter.cpp, [270](#), [271](#)
- EventRouter.hpp, [272](#), [274](#)
 - EventHandler, [273](#)
 - EventPayload, [273](#)
- extract
 - DataExtractor, [82](#)
- extract_links
 - HtmlParser, [103](#)
- extractVideoIdFromHref
 - WatchHistoryParser, [184](#)
- feedPanel
 - MainFrame, [132](#)
- FetchChannelLogoAsync
 - VideoCard, [175](#)
- fetchChannelLogos
 - YouTubeMetadataFetcher, [198](#)
- FetchThumbnailAsync
 - VideoCard, [175](#)
- fetchUserBlends
 - AppController, [27](#)
- fetchVideoBatch
 - YouTubeMetadataFetcher, [198](#)

- File_ID, 93
 - CSV, 93
 - HTML, 93
 - JSON, 93
- File_ID.hpp, 220
- FileBin, 94
 - ~FileBin, 94
 - setFileName, 94
 - setOutputFile, 95
 - setRawData, 95
 - write, 95
- FileBin.hpp, 221, 222
- FileFormatter, 95
 - ~FileFormatter, 96
 - format, 96
 - setData, 96
- FileFormatter.hpp, 222, 223
- filePath
 - WatchHistoryParser, 186
 - WatchLaterParser, 188
- FileSource, 97
 - getSourceId, 97
 - read, 97
 - setSource, 98
- FileSource.hpp, 224, 225
- findBlendForUser
 - DataManager, 87
 - SqliteDataManager, 149
- findUserById
 - DataManager, 87
 - SqliteDataManager, 149
- format
 - FileFormatter, 96
- formatters
 - DataSaver, 91
- FormatTimestamp
 - BlendChatPanel.cpp, 347
- FormSubmit
 - ButtonsConfig.hpp, 372
- FormTab
 - ButtonsConfig.hpp, 372
- Friend, 98
 - displayName, 101
 - email, 101
 - Friend, 99
 - getDisplayName, 99
 - getEmail, 99
 - getUserID, 100
 - setDisplayName, 100
 - setEmail, 100
 - toString, 100
 - userID, 101
- Friend.cpp, 286
- Friend.hpp, 287, 288
- friends
 - User, 162
- fullUrl
 - ChatWebSocket, 73
- FullWidthDanger
 - ButtonsConfig.hpp, 372
- FullWidthPrimary
 - ButtonsConfig.hpp, 372
- FullWidthSecondary
 - ButtonsConfig.hpp, 372
- G
 - HttpClient, 116
- generateBlend
 - IBlendAlgorithm, 118
 - RandomBlendAlgorithm, 143
 - WeightedBlendAlgorithm, 190
- generation
 - ThumbnailEvent, 154
- get
 - HttpClient, 112
- get_current_email
 - AppController, 27
- get_current_user
 - AppController, 27
- get_current_username
 - AppController, 28
- get_fullUrl
 - ChatWebSocket, 71
- get_text
 - HtmlParser, 103
- getActiveBlend
 - AppState, 39
- getActiveChatRoom
 - AppState, 39
- getAlgorithmUsed
 - Blend, 45
- getBlendID
 - Blend, 45
 - ChatRoom, 67
- getCategories
 - Channel, 64
- getChannelID
 - Channel, 64
 - Video, 169
- getChannelLogoURL
 - Video, 169
- getChannelName
 - Video, 169
- getChatRoom
 - AppState, 39
- GetConfig
 - ButtonsConfig.cpp, 368
 - ButtonsConfig.hpp, 373
- GetCurrentTheme
 - UIColors, 15
- getCurrentUser
 - AppState, 39
- getDisplayName
 - Channel, 65
 - Friend, 99
- getDuration
 - Video, 170

- getEmail
 - Friend, 99
 - User, 159
- getEventRouter
 - AppController, 28
- getFriends
 - User, 159
- getInstance
 - AppState, 40
- getLastStatusCode
 - HttpClient, 112
- getLastWatched
 - VideoEntry, 180
- getLikedVideos
 - YouTubeData, 192
- GetMaxSize
 - ButtonsConfig.cpp, 369
 - ButtonsConfig.hpp, 373
- getMessages
 - ChatRoom, 67
- GetMinSize
 - ButtonsConfig.cpp, 369
 - ButtonsConfig.hpp, 373
- getMostWatchedVideos
 - YouTubeData, 192
- getParseld
 - CsvParser, 77
 - HtmlParser, 103
 - Parser, 141
- getParserId
 - WatchHistoryParser, 184
 - WatchLaterParser, 187
- getParticipantIDs
 - ChatRoom, 68
- getParticipants
 - Blend, 45
- getPassword
 - User, 159
- getSessionUsers
 - AppState, 40
- GetSize
 - ButtonsConfig.cpp, 369
 - ButtonsConfig.hpp, 374
- getSourceId
 - CsvSource, 80
 - FileSource, 97
 - HtmlSource, 106
- getSubscribedChannels
 - YouTubeData, 192
- getTags
 - Video, 170
- getThumbnailURL
 - Video, 170
- getTitle
 - Blend, 46
 - Video, 170
- getTopChannels
 - YouTubeData, 193
- getUserID
 - Friend, 100
 - User, 160
- getUsername
 - User, 160
- getUsersWithoutData
 - AppState, 40
- getVideo
 - Blend, 46
 - VideoEntry, 180
- getVideoId
 - Video, 170
- getVideoList
 - Blend, 46
- getWatchCount
 - VideoEntry, 181
- getWatchHistory
 - YouTubeData, 193
- getWatchLaterVideos
 - YouTubeData, 193
- getYouTubeData
 - User, 160
- handleCreateBlend
 - AppController, 28
- handleDeleteAccount
 - AppController, 28
- handleLeaveBlend
 - AppController, 29
- handleLogin
 - AppController, 29
- handleLogout
 - AppController, 29
- handleOpenChat
 - AppController, 29
- handlePlayVideo
 - AppController, 30
- handleRefresh
 - AppController, 30
- handleRegister
 - AppController, 30
- handleSelectBlend
 - AppController, 30
- handleSendMessage
 - AppController, 31
- handleUploadData
 - AppController, 31
- hdr
 - HttpClient, 116
- HOME
 - UIPages.hpp, 417
- HTML
 - File_ID, 93
- HtmlParser, 102
 - extract_links, 103
 - get_text, 103
 - getParseld, 103
 - HtmlParser, 103
 - parse, 104

- parse_id, 104
 - raw_data, 104
 - setData, 104
- HtmlParser.cpp, 225, 226
- HtmlSource, 105
 - getSourceId, 106
 - HtmlSource, 106
 - read, 106
 - setSource, 106
 - source_id, 107
 - src_path, 107
- HtmlSource.cpp, 227, 228
- http
 - AppController, 35
- HttpClient, 107
 - ALT_SUCC, 115
 - AUTH_USER_PREFIX, 115
 - baseUrl, 115
 - BLEND, 115
 - build_auth_user_lookup_endpoint, 109
 - build_blend_participant_endpoint, 109
 - build_chat_history_endpoint, 109
 - build_chatroom_detail_endpoint, 110
 - build_delete_user_endpoint, 110
 - build_latest_blend_endpoint, 110
 - build_leave_blend_endpoint, 111
 - build_user_blends_endpoint, 111
 - build_watch_later_endpoint, 111
 - D, 116
 - del, 112
 - DELETE_USER_PREFIX, 116
 - DUPLICATE, 116
 - G, 116
 - get, 112
 - getLastStatusCode, 112
 - hdr, 116
 - HttpClient, 108
 - INV_USR_LOG_ERR, 116
 - isRequestSuccessful, 113
 - lastStatusCode, 116
 - LOGIN, 116
 - MAL_REQ_ERR, 117
 - MISS_RM_CNTXT, 117
 - P, 117
 - post, 113
 - REG_USER, 117
 - request, 114
 - SERVER_ERR, 117
 - SUCC, 117
 - UNAUTH_RES_REQ_ERR, 117
 - wasLastRequestSuccessful, 114
 - write_callback, 115
- HttpClient.cpp, 315, 316
- HttpClient.hpp, 318, 319
- httpGet
 - YouTubeMetadataFetcher, 198
- IBlendAlgorithm, 118
 - generateBlend, 118
- IBlendAlgorithm.hpp, 203, 204
- Icon
 - ButtonsConfig.hpp, 372
- image
 - ThumbnailEvent, 154
- ImageBackgroundPanel, 119
 - ImageBackgroundPanel, 120
 - m_imageStore, 120
 - m_key, 120
 - OnPaint, 120
- ImageKey
 - MainFrame, 129
- images
 - MainFrame, 132
- import
 - YouTubeDataImporter, 196
- importMultiThreaded
 - YouTubeDataImporter, 196
- initSchema
 - SqliteDataManager, 149
- INV_USR_LOG_ERR
 - HttpClient, 116
- IRefreshablePanel, 121
 - Refresh, 122
- IRefreshablePanel.hpp, 383, 384
- isBlendGenerating
 - AppState, 43
- isConnected
 - AppController, 35
- isGeneratingBlend
 - AppState, 40
- isParticipant
 - ChatRoom, 68
- isRequestSuccessful
 - HttpClient, 113
- isStarted
 - ChatWebSocket, 73
- JSON
 - File_ID, 93
- Json
 - AppController.cpp, 244
- json
 - YouTubeMetadataFetcher.cpp, 337
- JsonUtils.hpp, 288, 290
- kAppBackendBaseUrl
 - ServerConfig.hpp, 308
- kAppBackendWsPrefix
 - ServerConfig.hpp, 308
- kBatchSize
 - YouTubeMetadataFetcher, 199
- kPageSize
 - BlendFeedPanel, 61
- kTestBackendBaseUrl
 - ServerConfig.hpp, 308
- kTestBackendWsPrefix
 - ServerConfig.hpp, 309
- kYouTubeAPIKey

- AppController.cpp, 244
- lastStatusCode
 - HttpClient, 116
- lastWatched
 - VideoEntry, 182
- LightMode
 - UIColors, 16
- likedVideos
 - YouTubeData, 195
- LineHeight
 - VideoCard.cpp, 425
- listeners
 - EventRouter, 93
- loadBlendParticipants
 - DataManager, 87
 - SqliteDataManager, 150
- LoadFromBlend
 - BlendFeedPanel, 60
- LoadImage
 - MainFrame, 130
- loadParticipantsWithWatchLater
 - AppController, 31
- loadParticipantsWithWatchLaterParallel
 - AppController, 31
- loadWatchLater
 - DataManager, 88
 - SqliteDataManager, 151
- LOGIN
 - HttpClient, 116
 - UIPages.hpp, 417
- LoginPanel, 122
 - BuildRegisterForm, 124
 - BuildSignInForm, 124
 - LoginPanel, 124
 - m_bgImage, 126
 - m_controller, 126
 - m_formBook, 126
 - m_nav, 126
 - m_regConfirm, 126
 - m_regEmail, 126
 - m_regError, 126
 - m_registerTab, 126
 - m_regPassword, 127
 - m_regUsername, 127
 - m_siError, 127
 - m_signinTab, 127
 - m_siPassword, 127
 - m_siUsername, 127
 - OnPaint, 124
 - OnRegister, 125
 - OnSignIn, 125
 - ProceedAfterLogin, 125
 - Refresh, 125
 - Reset, 125
 - ShowTab, 125
- loginPanel
 - MainFrame, 132
- LoginPanel.cpp, 384, 385
 - MakeError, 385
 - MakeField, 385
- LoginPanel.hpp, 389, 391
- lookupUser
 - AppController, 32
- lowerExtension
 - YouTubeDataImporter, 196
- m_addedUsers
 - BlendCreationPanel, 56
- m_backDest
 - TopBar, 157
- m_bgImage
 - LoginPanel, 126
- m_blendNameCtrl
 - BlendCreationPanel, 56
- m_cards
 - BlendFeedPanel, 61
- m_channelID
 - VideoCard, 177
- m_channelLogo
 - VideoCard, 177
- m_channelLogoURL
 - VideoCard, 177
- m_channelName
 - VideoCard, 177
- m_confirmDeleteBtn
 - UserPanel, 167
- m_controller
 - ActiveBlendsPanel, 22
 - BlendChatPanel, 51
 - BlendCreationPanel, 57
 - BlendFeedPanel, 61
 - DataInstructionsPanel, 85
 - LoginPanel, 126
 - SettingsPanel, 147
 - UserPanel, 167
 - VideoCard, 177
- m_countLabel
 - BlendCreationPanel, 57
- m_createBtn
 - BlendCreationPanel, 57
- m_deleteErrorLabel
 - UserPanel, 167
- m_deletePasswordField
 - UserPanel, 167
- m_emailLabel
 - UserPanel, 167
- m_emptyLabel
 - ActiveBlendsPanel, 22
 - BlendCreationPanel, 57
- m_formBook
 - LoginPanel, 126
- m_hovered
 - VideoCard, 177
- m_imageStore
 - ImageBackgroundPanel, 120
- m_incoming
 - BlendChatPanel, 51

- m_incomingMx
 - BlendChatPanel, 52
- m_inputCtrl
 - BlendChatPanel, 52
- m_inputPanel
 - BlendChatPanel, 52
- m_key
 - ImageBackgroundPanel, 120
- m_listInner
 - ActiveBlendsPanel, 22
- m_listScroll
 - ActiveBlendsPanel, 22
- m_listSizer
 - ActiveBlendsPanel, 22
- m_logoFailed
 - VideoCard, 177
- m_logoGen
 - VideoCard, 178
- m_logoLoading
 - VideoCard, 178
- m_msgInner
 - BlendChatPanel, 52
- m_msgScroll
 - BlendChatPanel, 52
- m_msgSizer
 - BlendChatPanel, 52
- m_nav
 - ActiveBlendsPanel, 22
 - BlendChatPanel, 52
 - BlendCreationPanel, 57
 - DataInstructionsPanel, 85
 - LoginPanel, 126
 - SettingsPanel, 147
 - TopBar, 157
 - UserPanel, 167
- m_noBlendLabel
 - BlendChatPanel, 52
- m_offset
 - BlendFeedPanel, 61
- m_onMessage
 - ChatWebSocket, 73
- m_participantsBtn
 - BlendChatPanel, 53
- m_participantsDropdown
 - BlendChatPanel, 53
- m_participantsSizer
 - BlendChatPanel, 53
- m_pollTimer
 - BlendChatPanel, 53
- m_regConfirm
 - LoginPanel, 126
- m_regEmail
 - LoginPanel, 126
- m_regError
 - LoginPanel, 126
- m_registerTab
 - LoginPanel, 126
- m_regPassword
 - LoginPanel, 127
- m_regUsername
 - LoginPanel, 127
- m_siError
 - LoginPanel, 127
- m_signinTab
 - LoginPanel, 127
- m_siPassword
 - LoginPanel, 127
- m_siUsername
 - LoginPanel, 127
- m_themeChoice
 - SettingsPanel, 147
- m_thumbFailed
 - VideoCard, 178
- m_thumbGen
 - VideoCard, 178
- m_thumbLoading
 - VideoCard, 178
- m_thumbnail
 - VideoCard, 178
- m_thumbnailURL
 - VideoCard, 178
- m_title
 - VideoCard, 179
- m_userListInner
 - BlendCreationPanel, 57
- m_userListScroll
 - BlendCreationPanel, 57
- m_userListSizer
 - BlendCreationPanel, 57
- m_usernameLabel
 - UserPanel, 167
- m_videoID
 - VideoCard, 179
- m_ws
 - BlendChatPanel, 53
- m_wsBlendID
 - BlendChatPanel, 53
- main.cpp, 274
- MainFrame, 128
 - activeBlendsPanel, 131
 - book, 131
 - BuildHomePage, 130
 - chatPanel, 131
 - controller, 132
 - creationPanel, 132
 - dataInstrPanel, 132
 - feedPanel, 132
 - ImageKey, 129
 - images, 132
 - LoadImage, 130
 - loginPanel, 132
 - MainFrame, 130
 - NavigateTo, 130
 - RecolorAll, 131
 - RecolorWidget, 131
 - refreshBtn, 132

- refreshInProgress, 132
- settingsPanel, 133
- TriggerFeedRefresh, 131
- userPanel, 133
- MainFrame.cpp, 392, 393
- MainFrame.hpp, 398, 399
- MakeError
 - LoginPanel.cpp, 385
- MakeField
 - LoginPanel.cpp, 385
- MAL_REQ_ERR
 - HttpClient, 117
- maxHeight
 - BtnConfig, 63
- maxWidth
 - BtnConfig, 63
- Medium
 - ButtonsConfig.hpp, 372
- Message, 133
 - Message, 134
 - text, 134
 - timestamp, 134
 - toString, 134
 - userID, 135
- Message.cpp, 290, 291
- Message.hpp, 291, 293
- messageLabel
 - OperationProgressDialog, 137
- messages
 - ChatRoom, 69
- minHeight
 - BtnConfig, 63
- minWidth
 - BtnConfig, 63
- MISS_RM_CNTXT
 - HttpClient, 117
- ModelJson, 13
 - escape, 13
 - quote, 13
 - toIso8601, 14
- Nav
 - ButtonsConfig.hpp, 372
- NavigateFn
 - UIPages.hpp, 417
- NavigateTo
 - MainFrame, 130
- NeonMode
 - UIColors, 17
- NextPage
 - BlendFeedPanel, 60
- normalizeWeights
 - WeightedBlendAlgorithm, 190
- OnAdd
 - BlendCreationPanel, 55
- OnBack
 - TopBar, 156
- OnBrowse
 - DataInstructionsPanel, 85
- OnChannelLogoLoaded
 - VideoCard, 176
- OnClick
 - VideoCard, 176
- OnConfirmDelete
 - UserPanel, 165
- OnCreate
 - BlendCreationPanel, 55
- OnDeleteRequest
 - UserPanel, 165
- OnExit
 - YTBLNDAApp, 201
- OnInit
 - YTBLNDAApp, 201
- OnInputEnter
 - BlendChatPanel, 50
- OnLeaveBlend
 - ActiveBlendsPanel, 21
- OnLogout
 - SettingsPanel, 146
 - UserPanel, 165
- OnMouseEnter
 - VideoCard, 176
- OnMouseLeave
 - VideoCard, 176
- OnPaint
 - ImageBackgroundPanel, 120
 - LoginPanel, 124
 - VideoCard, 176
- OnParticipantsToggle
 - BlendChatPanel, 50
- OnPollTimer
 - BlendChatPanel, 51
- OnRegister
 - LoginPanel, 125
- OnRemoveUser
 - BlendCreationPanel, 56
- OnReuploadCSV
 - UserPanel, 166
- OnSelectBlend
 - ActiveBlendsPanel, 21
- OnSend
 - BlendChatPanel, 51
- OnShow
 - SettingsPanel, 146
 - UserPanel, 166
- OnSignIn
 - LoginPanel, 125
- OnThemeChanged
 - SettingsPanel, 146
- OnThumbnailLoaded
 - VideoCard, 176
- OperationProgressDialog, 135
 - ~OperationProgressDialog, 136
 - currentProgress, 137
 - messageLabel, 137
 - OperationProgressDialog, 136

- percentLabel, 138
- progressFill, 138
- progressTrack, 138
- ShowModal, 137
- UpdateProgress, 137
- UpdateProgressFill, 137
- OperationProgressDialog.cpp, 400, 401
- OperationProgressDialog.hpp, 402, 404
- P
 - HttpClient, 117
- Page
 - UIPages.hpp, 417
- Palette, 138
 - Accent, 139
 - AccentHover, 139
 - Background, 139
 - Danger, 139
 - Separator, 139
 - Surface, 140
 - SurfaceRaised, 140
 - TextMuted, 140
 - TextPrimary, 140
 - TextSecondary, 140
- par_format
 - DataExtractor, 83
- parse
 - CsvParser, 77
 - HtmlParser, 104
 - Parser, 141
 - WatchHistoryParser, 185
 - WatchLaterParser, 187
- parse_id
 - CsvParser, 78
 - HtmlParser, 104
- parseBlendID
 - AppController, 32
- parseChatHistory
 - AppController, 32
- parseMultiThreaded
 - WatchHistoryParser, 185
 - WatchLaterParser, 187
- parseParticipantIDs
 - AppController, 33
- Parser, 141
 - getParseId, 141
 - parse, 141
 - setData, 142
- Parser.hpp, 228, 230
- parser_id
 - WatchHistoryParser, 186
 - WatchLaterParser, 188
- parseRange
 - WatchHistoryParser, 185
- parseUserFromAuthResponse
 - AppController, 33
- parseWatchLaterVideos
 - AppController, 33
- participantIDs
 - ChatRoom, 69
- participants
 - Blend, 48
- participantWeights
 - WeightedBlendAlgorithm, 191
- password
 - User, 162
- pendingAccountCreated_
 - AppController, 35
- pendingBlendMessage
 - AppState, 43
- pendingRegistration_
 - AppController, 35
- pendingUploadError
 - AppState, 43
- percentLabel
 - OperationProgressDialog, 138
- post
 - HttpClient, 113
- ProceedAfterLogin
 - LoginPanel, 125
- progressFill
 - OperationProgressDialog, 138
- progressReporter
 - AppController, 35
- progressTrack
 - OperationProgressDialog, 138
- quote
 - ModelJson, 13
- RandomBlendAlgorithm, 142
 - generateBlend, 143
 - RandomBlendAlgorithm, 143
 - videosPerUser, 144
- RandomBlendAlgorithm.cpp, 205, 206
- RandomBlendAlgorithm.hpp, 206, 208
- raw_data
 - CsvParser, 78
 - HtmlParser, 104
- read
 - CsvSource, 80
 - FileSource, 97
 - HtmlSource, 106
- RebuildList
 - ActiveBlendsPanel, 21
- RebuildUserList
 - BlendCreationPanel, 56
- RecolorAll
 - MainFrame, 131
- RecolorWidget
 - MainFrame, 131
- Refresh
 - ActiveBlendsPanel, 21
 - BlendChatPanel, 51
 - BlendCreationPanel, 56
 - BlendFeedPanel, 60
 - IRefreshablePanel, 122
 - LoginPanel, 125

- UserPanel, 166
- refreshBtn
 - MainFrame, 132
- refreshInProgress
 - MainFrame, 132
- RefreshUserInfo
 - SettingsPanel, 146
 - UserPanel, 166
- REG_USER
 - HttpClient, 117
- registerBlendOnServer
 - AppController, 34
- registerEvents
 - AppController, 34
- registerListener
 - EventRouter, 92
- Reload
 - BlendChatPanel, 51
 - BlendCreationPanel, 56
- removeFriend
 - User, 160
- reportProgress
 - AppController, 34
- request
 - HttpClient, 114
- RequestJsonBuilder.cpp, 320, 324
 - buildBlendJson, 321
 - buildDeleteAccountJson, 322
 - buildLoginJson, 322
 - buildRegisterJson, 323
 - buildWatchLaterJson, 323
- RequestJsonBuilder.hpp, 325, 329
 - buildBlendJson, 326
 - buildDeleteAccountJson, 327
 - buildLoginJson, 327
 - buildRegisterJson, 328
 - buildWatchLaterJson, 328
- Reset
 - LoginPanel, 125
- save
 - DataSaver, 90
- saveBlend
 - DataManager, 88
 - SqliteDataManager, 151
- saveWatchLater
 - DataManager, 88
 - SqliteDataManager, 151
- sCurrentTheme
 - UIColors.cpp, 412
- selectWeightedVideos
 - WeightedBlendAlgorithm, 190
- send_message
 - ChatWebSocket, 71
- Separator
 - Palette, 139
 - UIColors, 17
- SERVER_ERR
 - HttpClient, 117
- server_url
 - AppController, 36
- ServerConfig.hpp, 307
 - kAppBackendBaseUrl, 308
 - kAppBackendWsPrefix, 308
 - kTestBackendBaseUrl, 308
 - kTestBackendWsPrefix, 309
- sessionUsers
 - AppState, 43
- set_blendID
 - ChatWebSocket, 72
- set_userID
 - ChatWebSocket, 72
- setActiveBlend
 - AppState, 40
- setBlendID
 - Blend, 46
- setCurrentUser
 - AppState, 41
- setData
 - CsvParser, 77
 - FileFormatter, 96
 - HtmlParser, 104
 - Parser, 142
- SetDeleteReauthVisible
 - UserPanel, 166
- setDisplayNames
 - Friend, 100
- setEmail
 - Friend, 100
 - User, 161
- setFileBin
 - DataSaver, 90
- setFileFormatter
 - DataSaver, 90
- setFileName
 - FileBin, 94
- setIsBlendGenerating
 - AppState, 41
- setLastWatched
 - VideoEntry, 181
- setLikedVideos
 - YouTubeData, 193
- setOnMessage
 - ChatWebSocket, 72
- setOutputFile
 - FileBin, 95
- setParser
 - DataExtractor, 82
- setPassword
 - User, 161
- setPendingBlendMessage
 - AppState, 41
- setPendingUploadError
 - AppState, 41
- setProgressReporter
 - AppController, 34
- setRawData

- FileBin, 95
- setSource
 - CsvSource, 80
 - DataExtractor, 83
 - FileSource, 98
 - HtmlSource, 106
- setSubscribedChannels
 - YouTubeData, 194
- SetTheme
 - UIColors, 15
- SETTINGS
 - UIPages.hpp, 417
- SettingsPanel, 144
 - m_controller, 147
 - m_nav, 147
 - m_themeChoice, 147
 - OnLogout, 146
 - OnShow, 146
 - OnThemeChanged, 146
 - RefreshUserInfo, 146
 - SettingsPanel, 146
- settingsPanel
 - MainFrame, 133
- SettingsPanel.cpp, 404, 405
- SettingsPanel.hpp, 406, 408
- setUsername
 - User, 161
- setUsersWithoutData
 - AppState, 42
- SetVideo
 - VideoCard, 176
- setVideoList
 - Blend, 47
- setWatchCount
 - VideoEntry, 181
- setWatchHistory
 - YouTubeData, 194
- setWatchLaterVideos
 - YouTubeData, 194
- setYouTubeData
 - User, 161
- ShowModal
 - OperationProgressDialog, 137
- ShowTab
 - LoginPanel, 125
- size
 - Blend, 47
- Small
 - ButtonsConfig.hpp, 372
- source_id
 - CsvSource, 80
 - HtmlSource, 107
- SqliteDataManager, 147
 - ~SqliteDataManager, 148
 - createUser, 149
 - db, 152
 - findBlendForUser, 149
 - findUserById, 149
 - initSchema, 149
 - loadBlendParticipants, 150
 - loadWatchLater, 151
 - saveBlend, 151
 - saveWatchLater, 151
 - SqliteDataManager, 148
 - validatePassword, 152
- SqliteDataManager.cpp, 329, 330
- SqliteDataManager.hpp, 334, 336
- src
 - CsvSource, 80
- src_format
 - DataExtractor, 83
- src_path
 - HtmlSource, 107
- subscribedChannels
 - YouTubeData, 195
- SUCC
 - HttpClient, 117
- success
 - ThumbnailEvent, 154
- Surface
 - Palette, 140
 - UIColors, 17
- SurfaceRaised
 - Palette, 140
 - UIColors, 17
- tags
 - Video, 171
- takePendingBlendMessage
 - AppState, 42
- takePendingUploadError
 - AppState, 42
- text
 - Message, 134
- TextMuted
 - Palette, 140
 - UIColors, 17
- TextPrimary
 - Palette, 140
 - UIColors, 18
- TextSecondary
 - Palette, 140
 - UIColors, 18
- ThemeType
 - UIColors.hpp, 415
- ThumbnailEvent, 153
 - Clone, 154
 - generation, 154
 - image, 154
 - success, 154
 - ThumbnailEvent, 154
- thumbnailURL
 - Video, 172
- thumbnailUrl
 - YouTubeMetadataFetcher::VideoMeta, 183
- timestamp
 - Message, 134

- title
 - AppController::BlendSummary, 62
 - Blend, 48
 - Video, 172
 - YouTubeMetadataFetcher::VideoMeta, 183
- toIso8601
 - ModelJson, 14
- TopBar, 155
 - m_backDest, 157
 - m_nav, 157
 - OnBack, 156
 - TopBar, 156
- TopBar.cpp, 408, 409
- TopBar.hpp, 409, 411
- TopBarBack
 - ButtonsConfig.hpp, 372
- toString
 - Blend, 47
 - Channel, 65
 - ChatRoom, 68
 - Friend, 100
 - Message, 134
 - User, 162
 - Video, 171
 - VideoEntry, 181
 - YouTubeData, 194
- TriggerFeedRefresh
 - MainFrame, 131
- UIColors, 14
 - Accent, 15
 - AccentHover, 15
 - Background, 16
 - Current, 16
 - Danger, 16
 - DarkMode, 16
 - GetCurrentTheme, 15
 - LightMode, 16
 - NeonMode, 17
 - Separator, 17
 - SetTheme, 15
 - Surface, 17
 - SurfaceRaised, 17
 - TextMuted, 17
 - TextPrimary, 18
 - TextSecondary, 18
- UIColors.cpp, 411, 412
 - sCurrentTheme, 412
- UIColors.hpp, 413, 415
 - ThemeType, 415
- UIPages.hpp, 416, 418
 - ACTIVE_BLENDS, 417
 - BLEND_CHAT, 417
 - BLEND_CREATION, 417
 - DATA_INSTRUCTIONS, 417
 - HOME, 417
 - LOGIN, 417
 - NavigateFn, 417
 - Page, 417
 - SETTINGS, 417
 - USER, 417
- UNAUTH_RES_REQ_ERR
 - HttpClient, 117
- UpdateCountLabel
 - BlendCreationPanel, 56
- UpdateProgress
 - OperationProgressDialog, 137
- UpdateProgressFill
 - OperationProgressDialog, 137
- url
 - ChatWebSocket, 73
- urlEncode
 - ChatWebSocket.cpp, 310
- USER
 - UIPages.hpp, 417
- User, 157
 - addFriend, 159
 - email, 162
 - friends, 162
 - getEmail, 159
 - getFriends, 159
 - getPassword, 159
 - getUserID, 160
 - getUsername, 160
 - getYouTubeData, 160
 - password, 162
 - removeFriend, 160
 - setEmail, 161
 - setPassword, 161
 - setUsername, 161
 - setYouTubeData, 161
 - toString, 162
 - User, 158
 - userID, 162
 - username, 162
 - youTubeData, 163
- User.cpp, 293, 294
- User.hpp, 294, 296
- userID
 - ChatWebSocket, 73
 - Friend, 101
 - Message, 135
 - User, 162
- username
 - User, 162
- UserPanel, 163
 - m_confirmDeleteBtn, 167
 - m_controller, 167
 - m_deleteErrorLabel, 167
 - m_deletePasswordField, 167
 - m_emailLabel, 167
 - m_nav, 167
 - m_usernameLabel, 167
 - OnConfirmDelete, 165
 - OnDeleteRequest, 165
 - OnLogout, 165
 - OnReuploadCSV, 166

- OnShow, 166
- Refresh, 166
- RefreshUserInfo, 166
- SetDeleteReauthVisible, 166
- UserPanel, 165
- userPanel
 - MainFrame, 133
- UserPanel.cpp, 418, 419
- UserPanel.hpp, 422, 424
- usersWithoutData
 - AppState, 44
- validatePassword
 - DataManager, 88
 - SqliteDataManager, 152
- Video, 168
 - channelID, 171
 - channelLogoURL, 171
 - channelName, 171
 - duration, 171
 - getChannelID, 169
 - getChannelLogoURL, 169
 - getChannelName, 169
 - getDuration, 170
 - getTags, 170
 - getThumbnailURL, 170
 - getTitle, 170
 - getVideoID, 170
 - tags, 171
 - thumbnailURL, 172
 - title, 172
 - toString, 171
 - Video, 169
 - videoID, 172
- video
 - VideoEntry, 182
- Video.cpp, 296, 297
- Video.hpp, 298, 299
- VideoCard, 172
 - Clear, 175
 - DoGetBestSize, 175
 - DrawWrappedText, 175
 - FetchChannelLogoAsync, 175
 - FetchThumbnailAsync, 175
 - m_channelID, 177
 - m_channelLogo, 177
 - m_channelLogoURL, 177
 - m_channelName, 177
 - m_controller, 177
 - m_hovered, 177
 - m_logoFailed, 177
 - m_logoGen, 178
 - m_logoLoading, 178
 - m_thumbFailed, 178
 - m_thumbGen, 178
 - m_thumbLoading, 178
 - m_thumbnail, 178
 - m_thumbnailURL, 178
 - m_title, 179
 - m_videoID, 179
 - OnChannelLogoLoaded, 176
 - OnClick, 176
 - OnMouseEnter, 176
 - OnMouseLeave, 176
 - OnPaint, 176
 - OnThumbnailLoaded, 176
 - SetVideo, 176
 - VideoCard, 174
- VideoCard.cpp, 424, 425
- VideoCard.hpp, 430, 432
- VideoEntry, 179
 - getLastWatched, 180
 - getVideo, 180
 - getWatchCount, 181
 - lastWatched, 182
 - setLastWatched, 181
 - setWatchCount, 181
 - toString, 181
 - video, 182
 - VideoEntry, 180
 - watchCount, 182
- VideoEntry.cpp, 300, 301
- VideoEntry.hpp, 301, 302
- videoID
 - Video, 172
- videoList
 - Blend, 48
- videosPerUser
 - RandomBlendAlgorithm, 144
 - WeightedBlendAlgorithm, 191
- wasLastRequestSuccessful
 - HttpClient, 114
- watchCount
 - VideoEntry, 182
- watchHistory
 - YouTubeData, 195
- WatchHistoryParser, 183
 - extractVideoIDFromHref, 184
 - filePath, 186
 - getParserId, 184
 - parse, 185
 - parseMultiThreaded, 185
 - parser_id, 186
 - parseRange, 185
 - WatchHistoryParser, 184
- WatchHistoryParser.cpp, 230, 231
- WatchHistoryParser.hpp, 233, 235
- WatchLaterParser, 186
 - filePath, 188
 - getParserId, 187
 - parse, 187
 - parseMultiThreaded, 187
 - parser_id, 188
 - WatchLaterParser, 186
- WatchLaterParser.cpp, 235, 236
- WatchLaterParser.hpp, 238, 239

- watchLaterVideos
 - YouTubeData, 195
- WeightedBlendAlgorithm, 188
 - generateBlend, 190
 - normalizeWeights, 190
 - participantWeights, 191
 - selectWeightedVideos, 190
 - videosPerUser, 191
 - WeightedBlendAlgorithm, 189
- WeightedBlendAlgorithm.hpp, 208, 209
- write
 - FileBin, 95
- write_callback
 - HttpClient, 115
- writeCallback
 - YouTubeMetadataFetcher, 199
- ws
 - ChatWebSocket, 73
- YouTubeData, 191
 - getLikedVideos, 192
 - getMostWatchedVideos, 192
 - getSubscribedChannels, 192
 - getTopChannels, 193
 - getWatchHistory, 193
 - getWatchLaterVideos, 193
 - likedVideos, 195
 - setLikedVideos, 193
 - setSubscribedChannels, 194
 - setWatchHistory, 194
 - setWatchLaterVideos, 194
 - subscribedChannels, 195
 - toString, 194
 - watchHistory, 195
 - watchLaterVideos, 195
 - YouTubeData, 192
- youTubeData
 - User, 163
- YouTubeData.cpp, 303
- YouTubeData.hpp, 305, 307
- YouTubeDataImporter, 195
 - import, 196
 - importMultiThreaded, 196
 - lowerExtension, 196
- YouTubeDataImporter.cpp, 239, 240
- YouTubeDataImporter.hpp, 241, 242
- YouTubeMetadataFetcher, 197
 - apiKey, 199
 - enrich, 198
 - fetchChannelLogos, 198
 - fetchVideoBatch, 198
 - httpGet, 198
 - kBatchSize, 199
 - writeCallback, 199
 - YouTubeMetadataFetcher, 198
- YouTubeMetadataFetcher.cpp, 336, 337
 - json, 337
- YouTubeMetadataFetcher.hpp, 340, 341
- YouTubeMetadataFetcher::VideoMeta, 182
 - channelId, 183
 - channelName, 183
 - thumbnailUrl, 183
 - title, 183
- YTBLNDApp, 200
 - controller, 201
 - OnExit, 201
 - OnInit, 201